SQL Handbook — Vol 2 (Ch 12–20)

Queries · joins · window functions · data modeling · normalization · indexes · transactions · scaling · 40+ interview problems — PostgreSQL first

9 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 12 — Indexes, Storage and Query Performance**
   - 12.1 How a database stores data
   - 12.2 Data compression and encoding
   - 12.3 File organization
   - 12.4 What an index is
   - 12.5 Single-level indexing, with numbers
   - 12.6 Tree-based indexes: B-trees and B+ trees
   - 12.7 Hash-based indexing
   - 12.8 Bloom filters
   - 12.9 PostgreSQL's other index types: GIN, GiST, BRIN
   - 12.10 Reading EXPLAIN: the four ways to read a table
   - 12.11 Composite indexes and column order
   - 12.12 Covering, partial and expression indexes
   - 12.13 Anti-patterns that disable indexes
   - 12.14 OFFSET vs keyset pagination
   - 12.15 Join algorithms
   - 12.16 Statistics: how the planner estimates
   - 12.17 A tuning workflow
   - Chapter TL;DR
2. **Chapter 13 — Transactions and Concurrency**
   - 13.1 What a transaction is
   - 13.2 Atomicity
   - 13.3 Consistency
   - 13.4 Isolation — anomalies and isolation levels
   - 13.5 Durability
   - 13.6 Watching isolation anomalies on a real server
   - 13.7 Lost updates and how to prevent them
   - 13.8 Write skew and Serializable isolation
   - 13.9 Concurrency control — the big picture
   - 13.10 Lock-based concurrency control
   - 13.11 Deadlocks
   - 13.12 Row locks for work queues: `FOR UPDATE`, `NOWAIT`, `SKIP LOCKED`
   - 13.13 Timestamp-based concurrency control
   - 13.14 Multiversion concurrency control (MVCC)
   - 13.15 Putting it together
   - Chapter TL;DR
3. **Chapter 14 — Advanced SQL and Using SQL from Code**
   - 14.1 User-defined functions
   - 14.2 Stored procedures
   - 14.3 Triggers
   - 14.4 JSON and JSONB
   - 14.5 Arrays
   - 14.6 Full-text search
   - 14.7 Pivot and unpivot
   - 14.8 Security: roles, privileges and row-level security
   - 14.9 SQL injection and parameterized queries
   - 14.10 Using SQL from Python
   - 14.11 Using SQL from Java (JDBC)
   - 14.12 Connection pooling
   - 14.13 ORMs vs raw SQL
   - Chapter TL;DR
4. **Chapter 15 — Scaling Databases: Partitioning, Sharding, Replication and Recovery**
   - 15.1 Distributed databases: the big picture
   - 15.2 Partitioning
   - 15.3 Sharding
   - 15.4 Distributed query processing
   - 15.5 Replication
   - 15.6 Consistency models
   - 15.7 The CAP theorem (and PACELC)
   - 15.8 Consensus and leader election
   - 15.9 Fault tolerance and recovery
   - 15.10 High availability: replication plus failover
   - 15.11 Data resilience: redundancy, mirroring, backups
   - Chapter 15 TL;DR
5. **Chapter 16 — Beyond Relational: NoSQL, NewSQL and Specialised Databases**
   - 16.1 Why NoSQL?
   - 16.2 Key-value stores
   - 16.3 Document databases
   - 16.4 Column-family and wide-column stores
   - 16.5 Graph databases
   - 16.6 In-memory databases
   - 16.7 NewSQL / distributed SQL
   - 16.8 Time-series and spatial databases
   - 16.9 Big data and distributed processing frameworks
   - 16.10 Choosing a database
   - 16.11 Inside two workhorses: Elasticsearch (and the ELK stack) and Redis
   - Chapter 16 TL;DR
6. **Chapter 17 — Schema Design Case Studies**
   - 17.0 The template
   - 17.1 Hospital management system
   - 17.2 Hotel reservation system
   - 17.3 Banking system
   - 17.4 Instagram-style photo sharing (runnable)
   - 17.5 E-commerce store (runnable)
   - 17.6 Food delivery platform
   - 17.7 Library management system (runnable)
   - 17.8 Ticket booking system (runnable, with seat-hold concurrency)
   - 17.9 Warehouse inventory system
   - 17.10 Quora (questions and answers)
   - 17.11 TikTok (short-video platform)
   - 17.12 Airbnb (short-term rentals)
   - 17.13 Snapchat (ephemeral messaging)
   - 17.14 Recurring modeling patterns

- Chapter 17 TL;DR

7. **Chapter 18 — SQL Interview Problems I: Method and 42 Warm-Up to Medium Problems**
   - 18.1 How to approach a SQL interview question
   - 18.2 Filtering problems (P1–P8)
   - 18.3 Aggregation problems (P9–P16)
   - 18.4 Join problems (P17–P22)
   - 18.5 Anti-join problems (P23–P26)
   - 18.6 Conditional aggregation (P27–P28)
   - 18.7 Date-logic problems (P29–P33)
   - 18.8 Ratios and percentages (P34–P38)
   - 18.9 Window and ranking basics (P39–P42)
   - 18.10 Pattern table
   - Chapter 18 TL;DR
8. **Chapter 19 — SQL Interview Problems 2: Medium and Hard Patterns**
   - 19.1 How to attack a medium or hard problem
   - 19.2 Warm-ups: joins, counts and grids
   - 19.3 Nth highest, top-N per group and "first row per group"
   - 19.4 Running totals and row-to-row comparisons
   - 19.5 Gaps and islands: consecutive rows and streaks
   - 19.6 Pivot and unpivot (static and dynamic)
   - 19.7 Self-joins, "both directions" and graph-like questions
   - 19.8 Date bucketing and retention
   - 19.9 Ratios and cancellation rates
   - 19.10 String building and expression evaluation
   - 19.11 Pattern catalogue
   - Chapter 19 TL;DR
9. **Chapter 20 — Cheat Sheets, Interview Q&A and Glossary**
   - 20.1 The logical query-processing order
   - 20.2 SELECT, filtering, NULL, sorting and paging
   - 20.3 Functions and expressions
   - 20.4 Aggregation
   - 20.5 Joins
   - 20.6 Subqueries and set operations
   - 20.7 CTEs, recursion and views
   - 20.8 Window functions and frames
   - 20.9 DDL, DML, constraints and upsert
   - 20.10 Data modeling and normalization quick rules
   - 20.11 Indexes and reading EXPLAIN
   - 20.12 Transactions, isolation and concurrency
   - 20.13 Advanced PostgreSQL: PL/pgSQL, JSONB, full-text search, security
   - 20.14 Scaling: partitioning, sharding, replication
   - 20.15 Which database?
   - 20.16 Dialect differences
   - 20.17 Verified behaviours
   - 20.18 Interview pattern catalogue
   - 20.19 Interview questions and answers
   - 20.20 Glossary (A–Z)
   - Chapter 20 TL;DR

Chapter 12 — Indexes, Storage and Query Performance

Up to now we have asked *what* a query returns. This chapter asks *how fast*, and why. The answer always comes down to one question: **how many pages of data does the database have to touch?** To answer it you need to know how rows sit on disk, what an index really is, how the query planner (the part of the database that decides *how* to run a query) chooses between reading everything and jumping straight to the rows it needs, and which query habits quietly defeat an index. Every claim below is measured on a real PostgreSQL 14 server with a one-million-row table.

What you'll learn

- How a database stores rows: the storage hierarchy, 8 KB pages, slotted pages, heap files, TOAST, row vs column layout, and compression/encoding tricks.
- File organizations (heap, sorted, hashed, clustered) and why physical order matters.
- What an index is; primary, clustering and secondary indexes; dense vs sparse; single-level and multilevel indexes with block-count arithmetic.
- B-trees and B+ trees, hash indexes (static, extendible, linear hashing), Bloom filters (with a simulation), and PostgreSQL's GIN, GiST, BRIN and bloom indexes.
- Reading `EXPLAIN (ANALYZE, BUFFERS)` (PostgreSQL's command for showing how a query ran): seq scan, index scan, index-only scan, bitmap scan.
- Composite indexes and the leftmost-prefix rule, covering (`INCLUDE`), partial and expression indexes.
- Anti-patterns: functions on columns, leading wildcards (and the `pg_trgm` fix), `OR`, OFFSET paging (and keyset paging), N+1 queries.
- The three join algorithms and the statistics the planner relies on.

How to run the examples. SQL files are in `code/`; the runner (a helper script) builds a fresh database per file, loading `00_schema.sql` first (about 5 seconds, it generates one million orders):

```
bash tools_run_sql.sh sql '12-*'                                    # all files of this chapter
bash tools_pg.sh psql -d ch_12 -f sql/chapters/12-indexes-storage-and-query-performance/code/02_scans.sql
cd sql/chapters/12-indexes-storage-and-query-performance/code
python3 bloom_filter_sim.py; python3 compression_demo.py
../../../../.venv-books/bin/python n_plus_one.py                    # needs psycopg
```

> **About the numbers.** Timings (`actual time`, `Execution Time`, `exec_ms`) depend on the machine and on what is already cached; yours will differ. **Buffer counts** (pages touched) and **plan shapes** are the stable things to compare. Long plan lines are trimmed with `...`.

The data is a small shop. Every value is computed from the row number, so every run builds the same tables:

```
CREATE TABLE customers (id int PRIMARY KEY, name text, email text, country text, signup_on date); -- 100,000 rows
CREATE TABLE orders (id bigint PRIMARY KEY, customer_id int REFERENCES customers(id),
                     status text, amount numeric(10,2), created_at timestamp, note text);
INSERT INTO orders
SELECT g, (g::bigint * 7919) % 100000 + 1,              -- exactly 10 orders per customer, scattered
       CASE WHEN g % 100 < 2 THEN 'pending' WHEN g % 100 < 5 THEN 'refunded'
            ELSE 'shipped' END,                         -- 2 % / 3 % / 95 %
       ((g::bigint * 37) % 50000) / 100.0 + 1,
       TIMESTAMP '2024-01-01' + g * INTERVAL '31 seconds', -- grows with id
       CASE WHEN g % 10 = 0 THEN 'gift wrap please' END   -- 90 % NULL
FROM generate_series(1, 1000000) AS g;
VACUUM ANALYZE orders;
```

`VACUUM ANALYZE` at the end cleans up the table and gathers the statistics the planner relies on (12.16). `psycopg`, needed by the last command, is the Python driver (library) for talking to PostgreSQL. The schema file also defines `plan_stats(query)`, which returns just the execution time and pages touched of a query, for side-by-side tables.

12.1 How a database stores data

What it is

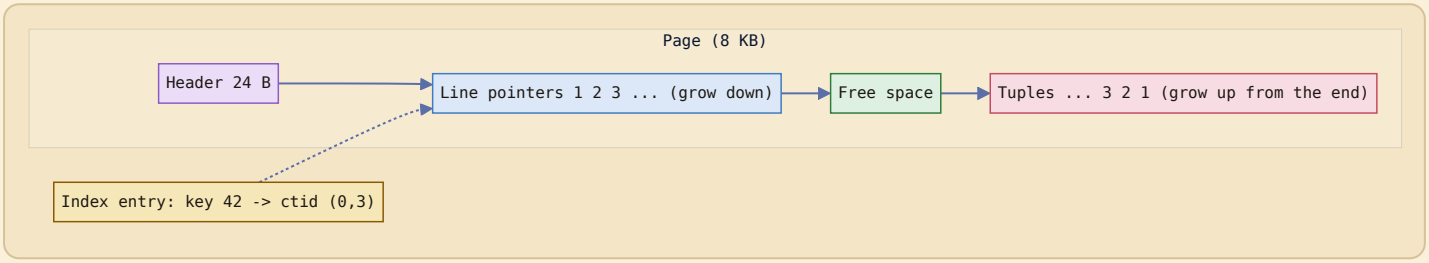
A database keeps data on **persistent storage** and works on it in **memory**. The layers differ by a factor of thousands in speed:

Level	Examples	Survives power loss?	Rough access time	Role in a DBMS
CPU cache / RAM (main memory; primary storage)	L1–L3 cache, DRAM	No (volatile — lost when power goes)	~1–100 ns	Buffer pool: cached pages, sort and hash memory
SSD / NVMe (secondary storage)	flash drives	Yes	~20–100 µs	Data files, indexes, write-ahead log (the change log written before the data files)
Hard disk (secondary)	spinning disks	Yes	~5–10 ms per random seek	Same, on older or cheaper servers
Tertiary / archive	tape, object storage (cloud file storage such as S3 Glacier)	Yes	seconds to hours	Backups, cold archives

Because the slow levels are so slow, a DBMS never fetches single rows from disk; it moves data in fixed-size **blocks** (also called **pages**): 8 KB in PostgreSQL and SQL Server, 16 KB in MySQL InnoDB, 4 KB by default in SQLite. The part of RAM that holds recently used pages is the **buffer pool** (PostgreSQL: `shared_buffers`). A read that finds its page there is a *hit*; one that must go to the operating system is a *read*. In `EXPLAIN (BUFFERS)` output you will see exactly those two words.

How a table is laid out

1. Each table is stored as one or more **files** (PostgreSQL: one file per 1 GB segment under `base/<database-oid>/`, where an *oid* is PostgreSQL's internal object number). A file is a sequence of pages numbered 0, 1, 2, …
2. Each page is a **slotted page**: a small header, an array of **line pointers** (slots) growing from the front, and the row data (**tuples**) growing from the back. Free space sits in the middle.
3. A row's physical address is (page number, slot number). PostgreSQL exposes it as the hidden column `ctid`. Indexes store these addresses. Because an index points at a *slot*, not a byte offset, the page can shuffle tuples around internally without breaking any index.
4. Large values are handled by **TOAST** (The Oversized-Attribute Storage Technique): when a row grows beyond about 2 KB, big values are compressed and, if still too big, cut into chunks stored in a separate TOAST table.



Seeing it on the real server

```
SELECT relname, pg_size_pretty(pg_relation_size(oid)) AS main_fork, ... FROM pg_class WHERE ...;
SELECT ctid, id, status, amount FROM orders WHERE id IN (1, 2, 3, 124, 125) ORDER BY id;
SELECT lower, upper, special, pagesize FROM page_header(get_raw_page('orders', 0));
SELECT lp, lp_off, lp_len, t_ctid FROM heap_page_items(get_raw_page('orders', 0)) LIMIT 4;
```

Output (from `01_storage_pages.sql`, which uses the `pageinspect` extension — an add-on module for looking at raw pages):

```
   relname   | main_fork | pages | est_rows
-------------+-----------+-------+----------
 customers   | 7344 kB   |   918 |   100000
 orders      | 68 MB     |  8662 |  1000000
 orders_pkey | 21 MB     |  2745 |  1000000

 ctid  | id  |  status  | amount
-------+-----+----------+--------
 (0,1) |   1 | pending  |   1.37
 (0,2) |   2 | refunded |   1.74
 (0,3) |   3 | refunded |   2.11
 (1,8) | 124 | shipped  |  46.88
 (1,9) | 125 | shipped  |  47.25

 lower | upper | special | pagesize
-------+-------+---------+----------
   488 |   504 |    8192 |     8192

 lp | lp_off | lp_len | t_ctid
----+--------+--------+--------
  1 |   8128 |     64 | (0,1)
  2 |   8064 |     64 | (0,2)
  3 |   8000 |     64 | (0,3)
  4 |   7936 |     64 | (0,4)
```

What this shows:

- One million orders occupy **8,662 pages**; a full scan must touch all of them. That number is the yardstick for everything that follows.
- Page 0 holds 116 rows (row 124 is the 8th row of page 1). Each tuple is 64 bytes: a 23-byte tuple header (transaction ids used by MVCC — *multi-version concurrency control*, PostgreSQL's way of keeping old row versions so readers and writers don't block each other; see chapter 13) plus the column values, aligned.
- `lower = 488` is where the slot array ends, `upper = 504` where tuple data begins: only 16 bytes free, so the page is full. Tuple 1 sits at the very end (offset 8128 + 64 = 8192) and later tuples are placed in front of it.

TOAST in action — three values of very different shape:

```
 id | logical_bytes | stored_bytes
----+---------------+--------------
  1 |            30 |           31      -- tiny: stored inline, 1-byte length header
  2 |         40000 |          477      -- 'abcdefgh' x 5000: compressed ~84x
  3 |         48000 |        48000      -- hex digests: incompressible, moved out of line as is
```

(Hex digests are hash outputs written in hexadecimal — effectively random text, so they don't compress.)

`pg_column_size` reports the bytes actually stored. PostgreSQL keeps a value uncompressed when compression would not save enough.

From memory to bytes: serialization, endianness, alignment

- **Serialization** turns an in-memory value (a row, a JSON document) into a byte sequence that can be written to a page or sent over the network; **deserialization** rebuilds it. Each engine has its own format: PostgreSQL's tuple layout, MongoDB's BSON (a binary form of JSON), Parquet's (a column-oriented file format) column chunks. Compact formats mean fewer pages.
- **Endianness** is the byte order of multi-byte numbers: *big-endian* writes the most significant byte first, *little-endian* the least significant first (x86 and ARM machines are little-endian). The integer 258 is `01 02` big-endian and `02 01` little-endian. A file format or network protocol must fix one order, or data written on one machine is misread on another — why PostgreSQL's binary files are not portable across architectures while `pg_dump` output is.

- **Alignment**: CPUs read a 4- or 8-byte value fastest at an address that is a multiple of its size, so engines insert padding. In PostgreSQL, column order therefore affects row size: `(bool, bigint, bool, bigint)` pads each `bool` to 8 bytes, while `(bigint, bigint, bool, bool)` does not. Index pages are laid out on aligned boundaries for the same reason.

Row stores vs column stores

Layout	How a page is filled	Good at	Examples
Row-oriented	All columns of a row together	OLTP (online transaction processing — many small reads and writes of single rows): fetch or change whole rows by key	PostgreSQL, MySQL InnoDB, SQL Server (rowstore), SQLite
Column-oriented	All values of *one column* together	Analytics: scan a few columns of billions of rows; compresses very well	ClickHouse, Amazon Redshift, Snowflake, DuckDB, Parquet files, SQL Server columnstore

Summing `amount` in a row store reads every whole row; a column store reads only the `amount` column, which also compresses well. More in DDIA (the book *Designing Data-Intensive Applications*): storage and retrieval.

Self-check

1. Why do databases read whole pages instead of single rows? → Storage transfers blocks anyway, and a random access costs far more than the extra bytes; grouping rows per page amortizes the cost (spreads it over many rows).
2. What is a `ctid`? → The (page, slot) address of a row version in PostgreSQL; indexes store it.

12.2 Data compression and encoding

What it is

Compression stores the same information in fewer bytes. It is **lossless** when the original can be rebuilt exactly (always the case for database data) and **lossy** when some detail is thrown away (JPEG, MP3 — never used for table values). **Encoding** is compression that exploits the *type and shape* of a column rather than treating it as a byte stream.

Why it matters

Fewer bytes means fewer pages to read, more rows fitting in the buffer pool, and cheaper storage and network transfer. The price is CPU time to compress and decompress, and sometimes slower in-place updates.

The common techniques

Technique	Idea	Works best on	Used by
Run-length encoding (RLE)	Store (value, how many times in a row)	Sorted or slowly changing columns	Column stores, Parquet, bitmap indexes
Dictionary encoding	Store each distinct value once; rows hold a small integer code	Few distinct values (country, status)	Parquet, ClickHouse `LowCardinality`, Redshift
Delta encoding	Store the first value, then differences	Timestamps, ids, sorted numbers	Time-series DBs (databases of timestamped measurements: Gorilla/Prometheus), Parquet
Bit-packing	Use only as many bits as the largest value needs	Small integers (codes, deltas)	Combined with the two above
Lempel–Ziv (LZ77/LZ78 family)	Replace a repeated byte sequence with a reference ("copy 8 bytes from 40 bytes back")	Text, repeated strings; no type knowledge	PostgreSQL TOAST (`pglz`, `lz4` since PG 14), zlib/zstd (widely used compression libraries) page compression
Huffman coding	Give frequent symbols short bit codes and rare ones long codes	Skewed symbol frequencies	Inside zlib/DEFLATE and zstd

`compression_demo.py` applies each technique to a column of 100,000 values:

```
status  = ["shipped"] * 60_000 + ["pending"] * 25_000 + ["refunded"] * 15_000   # sorted
country = [["IN", "US", "DE", "BR", "JP"][(i * 7) % 5] for i in range(N)]      # 5 distinct
ts      = [1_700_000_000 + i * 31 + (i % 7) for i in range(N)]                  # increasing
# rle(), dictionary(), delta() are ~6 lines each; see the file
```

Output:

```
status   raw text               :   715,000 bytes
status   RLE (3 runs)           :        34 bytes  -> [['shipped', 60000], ['pending', 25000], ['refunded',
15000]]
country  raw text               :   200,000 bytes
country  dictionary + 3-bit codes:    37,510 bytes  (dictionary ['IN', 'DE', 'JP', 'US', 'BR'], codes [0, 1, 2, 3,
4, 0]...)
ts       raw 8-byte ints        :   800,000 bytes
ts       delta + 6-bit packing  :    75,007 bytes  (first 1700000000, deltas [32, 32, 32, 32, 32, 32]...)
ts       zlib on the text       :   273,868 bytes  (no knowledge of the data type)
```

Type-aware encodings beat the general-purpose compressor by a wide margin and can often be queried without decompressing (counting `'pending'` rows is reading one run length) — why column stores sort data and choose an encoding per column.

Trade-offs and when not to use it

- Compression costs CPU on every write and read; for hot (frequently accessed), small OLTP rows it usually is not worth it (PostgreSQL compresses only when a row exceeds ~2 KB).

12.3 File organization

What it is

File organization is the rule that decides *where* in the data file a new record goes, and therefore how records can be found later.

Organization	Where records go	Fast	Slow	Real systems
Heap (unordered)	Any page with free space (usually the end)	Inserts; full scans	Finding one record without an index (scan everything)	PostgreSQL tables, SQL Server heaps, Oracle default
Sequential (sorted)	Kept in order of a key	Range reads and ordered output on that key; binary search (repeatedly halving the search range)	Inserts/deletes in the middle (shift records or use overflow pages, reorganize periodically)	Old ISAM files, sorted SSTables in LSM engines
Hashed	Page = hash(key) mod number of buckets	Equality lookups on the key: ~1 page read	Ranges and ordering; growing beyond the bucket count	Hash clusters (Oracle), hash-partitioned stores
ISAM (indexed sequential)	Sorted data file plus a static multilevel index built at load time; later inserts go to overflow pages	Reads and ranges on a stable file	Overflow chains grow until the file is reorganized	IBM mainframe ISAM files
B+ tree file organization	Records live in the leaves of a B+ tree on the key (12.6); the tree rebalances itself on insert	Point and range access that stays fast under inserts	Some space kept free in every leaf; splits	Same as the next row
Clustered / index-organized	The table *is* a B+ tree ordered by a key; leaf pages hold the rows	Primary-key lookups and ranges	Inserts in random key order cause page splits; secondary indexes must store the key	MySQL InnoDB, SQL Server clustered index, Oracle IOT (index-organized table), SQLite rowid / `WITHOUT ROWID` tables

Key terms — hashing and LSM engines: a **hash function** turns a key into a seemingly random number; `mod N` (the remainder after dividing by N) then picks one of N **buckets**. **ISAM** (indexed sequential access method) is IBM's classic sorted-file-plus-index design. An **LSM** (log-structured merge) engine buffers writes in memory and flushes them as immutable sorted files called **SSTables** (sorted string tables), merging them in the background — fast writes, at the cost of reads checking several files.

A hashed file with `bucket = key mod 5` puts keys 21, 32 and 43 in buckets 1, 2 and 3; key 36 also lands in bucket 1 — a **collision**, resolved by chaining it next to 21 (12.7). A clustered file with a department key stores all employees of department 7 in the same few blocks, so "list department 7" reads only those blocks.

PostgreSQL has only heap tables; `CLUSTER` sorts a heap **once** by an index, but new rows are not kept in order afterwards.

Why physical order matters — measured

Each customer's 10 orders are scattered through the table (the generator jumps by 7,919 customers per row). Summing the orders of 1,000 customers (10,000 rows) — before and after `CLUSTER` (`09_clustering.sql`):

```
SELECT correlation FROM pg_stats WHERE tablename = 'orders' AND attname = 'customer_id';
EXPLAIN (ANALYZE, BUFFERS, COSTS OFF)
SELECT sum(amount) FROM orders WHERE customer_id BETWEEN 1000 AND 1999;
CLUSTER orders USING orders_customer_idx;  ANALYZE orders;
-- ... same two queries again
```

```
correlation = 0.0034                                          -- before: random order (sampled; varies per run)
Aggregate (actual time=5.814..5.815 rows=1 loops=1)
  Buffers: shared hit=8454 read=14
  ->  Bitmap Heap Scan on orders (... rows=10000 loops=1)
        Heap Blocks: exact=8454                               -- 10,000 rows on 8,454 pages!
...
correlation = 1                                               -- after CLUSTER
Aggregate (actual time=0.895..0.895 rows=1 loops=1)
  Buffers: shared hit=2 read=100
  ->  Index Scan using orders_customer_idx on orders (... rows=10000 loops=1)
Execution Time: 0.901 ms
```

Same rows, same index: **8,468 pages → 102 pages**, because after clustering the 10,000 rows sit next to each other. The statistic **correlation** (−1 … 1) tells the planner how well a column's order matches the physical order.

A related knob is **fillfactor**: how full to pack each page when inserting, leaving room so that updated row versions can stay on the same page (PostgreSQL's heap-only-tuple — HOT — updates, which skip index maintenance, chapter 13). The same 100,000 rows take 443 pages at the default `fillfactor = 100` and 633 pages at `fillfactor = 70`.

Trade-offs

- Only **one** physical order per table — pick the key of your most important range queries (often time or tenant id — the customer organisation a row belongs to in a multi-tenant app).
- In InnoDB and SQL Server the clustered key is copied into every secondary index, so a wide or random key (a random UUID — a 128-bit randomly generated id) bloats every index and causes page splits; prefer a short, increasing key.

12.4 What an index is

What it is

An **index** is a separate, smaller data structure that maps **search-key values to row addresses**, kept sorted (or hashed) so that the database can find matching rows without reading the whole table. Think of the index at the back of a textbook: sorted terms with page numbers. You look up "B+ tree", jump to page 412, and never read pages 1–411.

- **Search key**: the column(s) the index is built on (not necessarily a primary or unique key).
- **Index entry**: (key value, pointer). The pointer is a `ctid` in PostgreSQL, the primary-key value in InnoDB secondary indexes, a row id in Oracle.

The problem it solves

Without an index, `WHERE customer_id = 4242` must check every row. With one, the database walks a handful of index pages and fetches only the matching rows:

```
-- no index
 Seq Scan on orders (actual time=2.944..30.041 rows=10 loops=1)
   Filter: (customer_id = 4242)
   Rows Removed by Filter: 999990
   Buffers: shared hit=8662
 Execution Time: 30.068 ms
-- CREATE INDEX orders_customer_idx ON orders (customer_id);
 Bitmap Heap Scan on orders (actual time=0.015..0.023 rows=10 loops=1)
   Buffers: shared hit=10 read=3
   ->  Bitmap Index Scan on orders_customer_idx (... rows=10 loops=1)
 Execution Time: 0.036 ms
```

8,662 pages → 13 pages; ~30 ms → ~0.04 ms. That is the whole reason indexes exist.

Kinds of index (classic terminology)

Term	Meaning
Primary index	Built on the ordering key of a *sorted* file, usually the primary key; one entry per data block is enough (sparse).
Clustering index	Built on a sorting field that is *not* unique (e.g. file sorted by department); one entry per distinct value, pointing to its first block.
Secondary index	Built on any other column; the file is not sorted by it, so it needs one entry per record (dense). A table can have many.
Dense index	An entry for **every** search-key value (every record).
Sparse index	Entries for only **some** values — typically the first key of each block; works only if the file is sorted on that key.
Clustered vs non-clustered (vendor usage)	*Clustered*: the table rows are stored in index order (at most one per table — SQL Server, InnoDB PK). *Non-clustered*: a separate structure pointing to rows. PostgreSQL indexes are all non-clustered.
Unique index	Rejects duplicate keys; how `PRIMARY KEY` and `UNIQUE` constraints are enforced.

The SQL

```
CREATE INDEX orders_customer_idx ON orders (customer_id);              -- B-tree by default
CREATE UNIQUE INDEX customers_email_uq ON customers (email);          -- enforces uniqueness
CREATE INDEX CONCURRENTLY orders_note_idx ON orders (note);           -- PG: build without blocking writes
DROP INDEX orders_customer_idx;                          -- MySQL / SQL Server: DROP INDEX name ON table
ALTER TABLE t ADD CONSTRAINT t_pk PRIMARY KEY (id);      -- creates a unique index implicitly
```

SQL Server adds `CREATE [CLUSTERED|NONCLUSTERED] INDEX`. `CONCURRENTLY` (PostgreSQL) and `ONLINE = ON` (SQL Server Enterprise) / `ALGORITHM=INPLACE, LOCK=NONE` (MySQL) avoid long write locks (locks that stop other sessions from changing the table) on big tables.

Trade-offs

Benefit	Cost
Fast lookups, ranges, joins, `ORDER BY`, `MIN/MAX`	Every `INSERT`, `DELETE` and many `UPDATE`s must also change every index
Enforces uniqueness	Disk and buffer-pool space (our 1M-row PK index is 21 MB)
Enables index-only scans	Too many indexes slow writes and make the planner's job harder

When not to index

- Small tables (a few pages): a sequential scan is already cheap.
- Columns that most rows share (low selectivity): `status = 'shipped'` matches 95 % of rows — see 12.10.
- Write-heavy tables whose columns are rarely searched.
- Columns only ever used inside functions (index the expression instead — 12.12).

Common mistakes

- "Index every column." Each one costs writes; unused indexes are pure overhead (find them with `pg_stat_user_indexes.idx_scan = 0` — a statistics view that counts how often each index is used).
- Forgetting that **foreign-key columns are not indexed automatically** in PostgreSQL (MySQL InnoDB does create one). Without an index, joins on the FK and `ON DELETE` checks on the parent scan the child table.

In the interview

"An index is a sorted (or hashed) side structure mapping key values to row locations, so a lookup reads a few pages instead of the whole table; it speeds up filters, joins and sorts at the cost of slower writes and extra space. I index selective predicates (`WHERE` conditions that match few rows) and join keys and confirm with EXPLAIN."

- *Why not index everything?* → Write amplification (one logical change causing many physical writes), space, and cache pressure (indexes competing for limited memory); low-selectivity indexes are ignored anyway.
- *Clustered vs non-clustered?* → Clustered stores the rows themselves in key order (one per table); non-clustered is a separate structure with pointers.

12.5 Single-level indexing, with numbers

How it works

An index file is much smaller than the data file because each entry holds only a key and a pointer. Searching it with binary search costs about $\log_2$(index blocks) reads, plus one read for the data block. The arithmetic (from *Database System Concepts* and *Fundamentals of Database Systems*):

- **Blocking factor** `bfr = ⌊block size / record size⌋` — records per block. (`⌊x⌋` rounds down, `⌈x⌉` rounds up.)
- **Blocks** `b = ⌈records / bfr⌉`.
- **Binary search** on `b` sorted blocks: `⌈log₂ b⌉` block reads.

Worked example (own numbers)

A file of **r = 1,000,000** records, each **200 bytes**, block size **4,096 bytes**. Index entries are 8-byte key + 8-byte pointer = **16 bytes**.

Access path	Calculation	Block reads
Data file	bfr = ⌊4096/200⌋ = 20; b = 1,000,000/20 = **50,000 blocks**	—
Linear scan, unordered file	average b/2, worst b	25,000 avg / 50,000 worst
Binary search, file sorted on the key	$\lceil \log_2 50{,}000 \rceil = 16$	**16**
Primary index (sparse): one entry per data block	50,000 entries; index bfr = ⌊4096/16⌋ = 256; 50,000/256 → **196 blocks**; $\lceil \log_2 196 \rceil = 8$, +1 data block	**9**
Secondary index (dense) on an unsorted column: one entry per record	1,000,000/256 → **3,907 blocks**; $\lceil \log_2 3907 \rceil = 12$, +1	**13**
Multilevel (index on the index, fan-out = entries per index block = 256)	level 1: 3,907 blocks → level 2: ⌈3907/256⌉ = 16 → level 3: 1 block; 3 levels + 1 data block	**4**

Two lessons: a sparse index is much smaller than a dense one but requires a sorted file; and building an index **on top of the index** — each level 256× smaller — turns a $\log_2$ search into a $\log_{256}$ search. The top level is small enough to stay in RAM, so in practice only the lower levels and the data block cost disk reads. That idea, made dynamic, is the B+ tree. (Courses often call primary, clustering and secondary indexes *single-level* and B-/B+ trees *multi-level* indexes.)

Trade-offs of single-level indexes

- Sparse: small, but only possible for the file's sort key, and inserts must keep the file sorted.
- Dense: works for any column, but large; still needs binary search over thousands of blocks.

- Both are **static**: inserting into a sorted index means shifting entries. Multilevel trees with free space in each node fix that.

Self-check

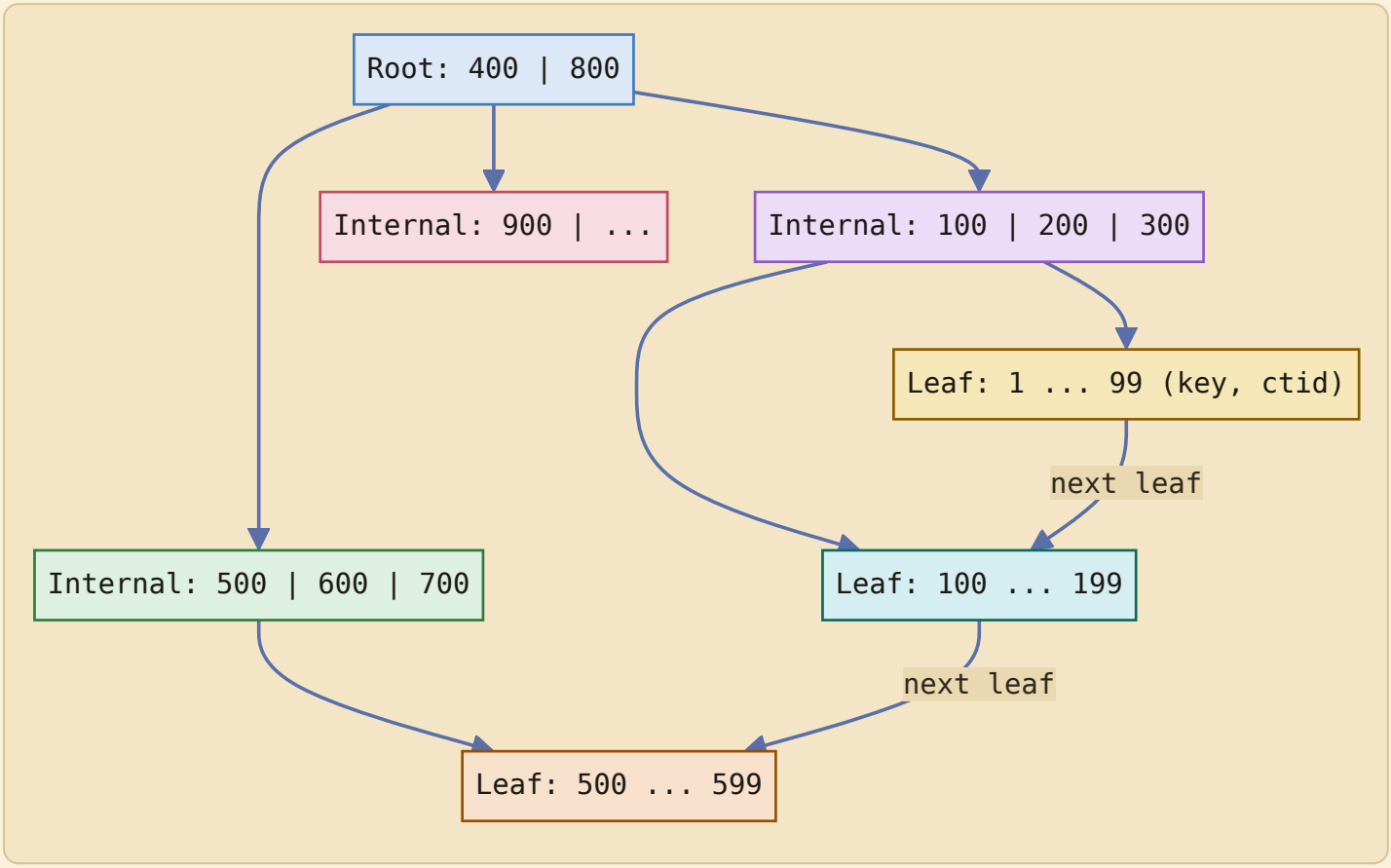
1. Why can a secondary index not be sparse? → The file is not sorted on that column, so the first key of a block says nothing about the others.
2. With fan-out 256, how many index levels for 16 million entries? → $256^3 \approx 16.7$ million, so 3 levels.

12.6 Tree-based indexes: B-trees and B+ trees

What it is

A **B-tree** (Bayer & McCreight, 1972) is a balanced search tree whose nodes are whole disk pages, each holding hundreds of sorted keys. "Balanced" means every leaf is at the same depth, so every lookup costs the same number of page reads. The **B+ tree** — what nearly every database actually uses (PostgreSQL's "btree", InnoDB, SQL Server, SQLite, Oracle) — adds two rules:

1. **All data pointers live in the leaves.** Internal nodes hold only separator keys and child pointers, so more of them fit per page (higher **fan-out**).
2. **Leaves are linked** left-to-right (PostgreSQL: both ways), so a range scan finds the first key and then just walks sideways.



Why a tree at all? In big-O notation (how work grows with the number of entries n), an unsorted array needs O(n) comparisons per search; a sorted array allows O(log n) binary search but each insert must shift up to n entries. A B-tree keeps keys sorted *and* absorbs inserts locally, with O(log n) search, insert and delete.

How it works

- **Search** (e.g. `id = 500000`): read the root, pick the child whose key range contains 500,000, repeat until a leaf, then follow the pointer to the heap (the table's unordered data file). Cost = tree

height + 1. In the figure, looking for 650 goes root (400 < 650 < 800 → middle child) → internal (600 < 650 < 700) → the leaf holding 600–699.

- **Range** (`BETWEEN 1000 AND 1999`): search for 1000, then read leaves left to right until a key > 1999.
- **Insert**: find the leaf; if it has room, insert in sorted position. If it is full, **split** it into two half-full leaves and push the middle key up into the parent; if the parent overflows it splits too. Only a root split makes the tree taller — which is why the tree stays balanced.
- **Delete**: remove the entry; if a node becomes too empty, borrow from a sibling or merge. (PostgreSQL does not merge eagerly; it reclaims empty pages via `VACUUM`.)
- **Order** `n` of a tree: each node holds up to `n − 1` keys and `n` children, and every non-root node stays at least half full.

Seeing it on the server

```sql
SELECT level AS root_level FROM bt_metap('orders_pkey');
SELECT type, live_items, avg_item_size, free_size FROM bt_page_stats('orders_pkey', 1);
```

```
 root_level = 2                 -- levels 2 (root), 1 (internal), 0 (leaves): height 3
 type | live_items | avg_item_size | free_size
------+------------+---------------+-----------
 l    |        367 |            16 |       808
```

A leaf page holds 367 entries of 16 bytes and keeps ~10 % free for future inserts (the B-tree default fillfactor is 90). One million keys need about 2,700 leaves, and just one internal level to point to them. That is why the primary-key lookup in 12.10 touched only **7 buffers** — the three tree levels, the heap page and a few index-metadata accesses — against 8,662 for a full scan. A billion keys would need only one more level.

B-tree vs B+ tree vs others

Structure	Data pointers in	Range scans	Used by
B-tree (classic)	Every node	Needs in-order tree walk	Rare today; some file systems
B+ tree	Leaves only; leaves linked	Walk the leaf chain	PostgreSQL, MySQL, SQL Server, Oracle, SQLite
LSM tree	Sorted immutable files merged in the background	Merge several files	RocksDB, Cassandra, ScyllaDB, LevelDB (write-optimized storage engines and NoSQL databases; see ch. 16)
R-tree	Bounding boxes	Spatial "overlaps/near"	PostGIS (PostgreSQL's maps-and-geometry extension, via GiST), SQLite R*Tree

Trade-offs

- Random-key inserts (random UUIDs) hit random leaves: more page splits and more cache misses (needed pages not already in memory) than an increasing key.
- PostgreSQL 13+ **deduplicates** repeated keys in leaves, so a B-tree on a low-uniqueness column is much smaller (our `customer_id` index: 9 MB vs 21 MB for unique `created_at`).

In the interview

> **30-second answer:** "Databases use B+ trees: balanced trees whose nodes are pages with hundreds of keys, so even a billion rows need only 3–4 levels. Values sit in linked leaves, so range scans walk the leaves. Inserts split full nodes and push keys up, keeping every leaf at the same depth."

- *B-tree vs LSM?* → B+ trees update pages in place (read-optimized); LSM trees append and merge (write-optimized, need Bloom filters for reads).

Self-check

1. Why do B+ tree internal nodes hold no row pointers? → To fit more separator keys per page, raising fan-out and lowering height.
2. When does a B+ tree get taller? → Only when the root splits.

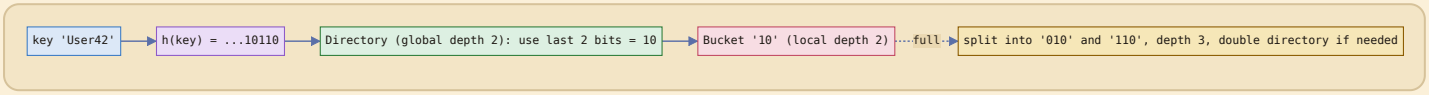
12.7 Hash-based indexing

What it is

A **hash index** computes `hash(key)` and uses it to pick a **bucket** (a page or chain of pages) that holds the entries for that key. A lookup reads one bucket — typically one or two pages — no matter how large the table.

How it works

1. **Hash function**: maps a key to an integer spread evenly across buckets (`bucket = h(key) mod N`). A good one scatters similar keys (`User1`, `User2`) far apart.
2. **Collisions**: different keys can land in the same bucket. Databases put them in the same bucket page; when it fills, an **overflow page** is chained on. (In-memory hash tables also use *open addressing*: probe the next slots.)
3. **Static hashing** fixes `N` up front. Example with `h(k) = k mod 7`: keys 15, 23, 40 go to buckets 1, 2 and 5; inserting 29 (29 mod 7 = 1) collides with 15 and is **chained** in bucket 1, so a search for 29 hashes to bucket 1 and walks its short chain. If the data grows, chains get long; if it shrinks, space is wasted. Rehashing everything into a bigger table is expensive.
4. **Dynamic hashing** grows gradually:
   - **Extendible hashing** keeps a **directory** of `2^d` pointers indexed by the last `d` bits of the hash (**global depth**). Each bucket records how many bits it really uses (**local depth**). When a bucket overflows it splits in two using one more bit; only if its local depth already equals the global depth does the directory double (pointers copied, no data moved).
   - **Linear hashing** has no directory: buckets are split one at a time in a fixed round-robin order (a "next to split" pointer), using `h mod 2N` for already-split buckets. Growth is smooth; overflow chains are temporary. PostgreSQL's hash index (based on Seltzer and Yigit's 1991 hashing package) likewise has no directory: it splits one bucket at a time and allocates bucket pages in power-of-2 groups whose location is computed from the bucket number.



On the server

```
CREATE INDEX o_cust_hash ON orders USING hash (customer_id);
EXPLAIN (COSTS OFF) SELECT * FROM orders WHERE customer_id = 42;   -- Bitmap Index Scan on o_cust_hash
EXPLAIN (COSTS OFF) SELECT * FROM orders WHERE customer_id < 42;   -- Seq Scan: hash can't do ranges
```

```
   index_name    | type  |  size
-----------------+-------+---------
 o_cust_hash     | hash  | 32 MB
 o_created_btree | btree | 21 MB
 o_cust_btree    | btree | 9256 kB
 o_created_brin  | brin  | 24 kB
```

Trade-offs

	Hash index	B-tree
`=` lookups	O(1) buckets	O(log n) pages (3–4 in practice)
Ranges, `ORDER BY`, prefix `LIKE`	No	Yes
Unique constraints	No (PostgreSQL)	Yes
Size	Can be larger (no deduplication — 32 MB vs 9 MB above)	Compact with dedup

In practice the B-tree's extra 2–3 page reads are usually cached, so hash indexes win only for very large, equality-only keys (long strings, URLs). PostgreSQL hash indexes are crash-safe (intact after a server crash) only since version 10 — old advice to avoid them dates from before that. MySQL InnoDB has no user-created hash indexes (its *adaptive hash index* is built automatically in memory); the MEMORY engine does. Hashing is also the heart of **hash joins** (12.15) and **hash partitioning** (ch. 15).

Self-check

1. Why can't a hash index answer `ORDER BY customer_id`? → Hashing destroys order: neighbouring keys land in unrelated buckets.

12.8 Bloom filters

What it is

A **Bloom filter** (Burton Bloom, 1970) is a compact, probabilistic set: it answers "is x in the set?" with either **"definitely not"** or **"probably yes"**. It never gives a false negative; it gives false positives at a rate you choose by spending more bits. Analogy: a bouncer with a smudged guest list who never turns away a real guest but occasionally lets in a stranger.

Why it matters

Checking a filter in RAM costs nanoseconds; checking disk costs micro- to milliseconds. If most lookups are for keys that are *not* there, the filter skips almost all of that disk work.

How it works

1. Start with an array of **m bits**, all 0, and choose **k** independent hash functions.
2. **Add(x)**: compute the k hashes, set those k bits to 1.
3. **Query(x)**: compute the k hashes; if **any** bit is 0, x was never added (definitely absent). If all are 1, x is *probably* present — those bits may have been set by other keys.
4. With n keys inserted, the false-positive probability is approximately **p ≈ (1 − e^(−kn/m))^k**, minimized at **k = (m/n)·ln 2**; then about **1.44·log₂(1/p) bits per key** are needed (≈ 9.6 bits for 1 %).
5. You cannot delete (clearing a bit could erase other keys). **Counting Bloom filters** keep a small counter per position instead of a bit to allow deletes; **scalable Bloom filters** add new, larger filters as n grows.

A tiny example (10 bits, k = 3; hash outputs made up for illustration). Insert `"sql"` → positions 2, 5, 8 set. Insert `"java"` → positions 1, 5, 9 set (5 was already 1). Bits now: `0110010011` (positions 1, 2, 5, 8, 9). Query `"css"` → positions 2, 6, 9: bit 6 is 0, so **definitely absent**. Query `"go"` → positions 1, 2, 8: all 1 though `"go"` was never added — a **false positive**. A *false positive* is "maybe present" for an absent key; a *false negative* ("absent" for a present key) cannot happen. The work per operation depends on k, not on m.

Simulation vs theory

`bloom_filter_sim.py` builds real filters (SHA-256 — a standard cryptographic hash function — with double hashing), inserts 10,000 keys and probes 50,000 keys that were never added:

```
class BloomFilter:
    def __init__(self, m_bits, k_hashes):
        self.m, self.k = m_bits, k_hashes
        self.bits = bytearray((m_bits + 7) // 8)
    def _positions(self, key):                          # h_i = h1 + i*h2 (mod m)
        d = hashlib.sha256(key.encode()).digest()
        h1, h2 = int.from_bytes(d[:8], "big"), int.from_bytes(d[8:16], "big") | 1
        return [(h1 + i * h2) % self.m for i in range(self.k)]
    def add(self, key):
        for p in self._positions(key): self.bits[p // 8] |= 1 << (p % 8)
    def might_contain(self, key):
        return all(self.bits[p // 8] >> (p % 8) & 1 for p in self._positions(key))
```

Output:

```
bits/key   m bits   k  measured FP  theory FP    memory
       4    40000   3     14.6340%   14.6892%     4.9KB
       8    80000   6      2.1700%    2.1577%     9.8KB
      10   100000   7      0.8720%    0.8194%    12.2KB
      16   160000  11      0.0520%    0.0459%    19.5KB

Effect of k at a fixed 8 bits/key:
  k= 1  measured 11.604%   theory 11.750%
  k= 2  measured  4.770%   theory  4.893%
  k= 4  measured  2.324%   theory  2.397%
  k= 6  measured  2.170%   theory  2.158%
  k= 8  measured  2.654%   theory  2.549%
  k=12  measured  4.780%   theory  4.833%
```

The measured rates track the formula closely; ~10 bits per key buys < 1 % false positives regardless of how long the keys are; and too many hash functions hurts (k = 12 fills the array with ones).

Real-world examples

- **LSM-tree databases** (RocksDB, Cassandra, HBase, LevelDB) keep one Bloom filter per sorted file so a read skips files that cannot hold the key.
- **"Is this username taken?"** checks and duplicate-submission checks in web apps: a filter in memory answers most "no" cases without touching the database.
- **Web caches and CDNs** (content delivery networks: servers around the world that keep copies of web content near users; see System Design Fundamentals) use them to avoid caching objects seen only once; routers and distributed caches use them to summarize which keys a peer holds.
- **Join filters**: many engines build a Bloom filter from the smaller join side and push it to the scan of the larger side (Spark — a distributed data-processing engine — SQL Server bitmap filters, Oracle).
- **PostgreSQL `bloom` extension**: an index that stores a small signature per row covering many columns, so one index answers equality on *any* subset of them:

```
CREATE INDEX wide_bloom ON wide USING bloom (a, b, c, d, e) WITH (length = 80);
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF) SELECT * FROM wide WHERE c = 5 AND e = 9;
```

```
 bloom_index_size | table_size
------------------+------------
 4720 kB          | 15 MB
 Bitmap Heap Scan on wide (actual rows=50 loops=1)
   Recheck Cond: ((c = 5) AND (e = 9))
   Rows Removed by Index Recheck: 97          <- the false positives, removed by rechecking
   ->  Bitmap Index Scan on wide_bloom (actual rows=147 loops=1)
```

Self-check

1. Can a Bloom filter say "no" to a key that was added? → Never; added keys set all their bits.
2. What happens to p if you double m at the same n? → Drops sharply (roughly from 2 % to 0.05 % between 8 and 16 bits/key).
3. Why can't you delete from a plain Bloom filter? → Bits are shared among keys; use a counting Bloom filter.

TL;DR

Bit array + k hashes; "no" is certain, "yes" is probable; ~10 bits/key ≈ 1 % false positives; used to skip disk reads in LSM stores and joins.

12.9 PostgreSQL's other index types: GIN, GiST, BRIN

B-trees answer "equal / less / greater". Other questions — *does this array contain X*, *do these ranges overlap*, *which rows fall in this month of a huge log table* — need other structures. All examples are in `10_index_types.sql`.

Type	Structure in one line	Answers	Typical use
B-tree (default)	Balanced sorted tree (12.6)	`= < > BETWEEN`, prefix `LIKE`, `ORDER BY`	Almost everything
Hash	Buckets by hash (12.7)	`=` only	Long keys, equality only
GIN (Generalized Inverted Index)	For each *element* (array item, JSON key/value, word, trigram — a 3-character chunk of text) a list of rows containing it	`@>` contains, `?` key exists, full-text `@@`, `LIKE '%x%'` with `pg_trgm`	Tags, JSONB documents, search
GiST (Generalized Search Tree)	Balanced tree of "bounding" predicates (boxes, ranges) that may overlap	Overlap `&&`, containment, nearest-neighbour `<->`	Geometry (PostGIS), ranges, exclusion constraints
SP-GiST	Space-partitioned trees (quadtrees, tries)	Points, IP ranges, text prefixes	Unevenly distributed data
BRIN (Block Range INdex)	min/max summary per range of 128 pages	Ranges on columns that follow physical order	Append-only (rows only ever added, never changed) logs, time-series
bloom (extension)	Bloom signature per row (12.8)	`=` on any subset of many columns	Wide tables with ad-hoc filters

BRIN — our `created_at` rises with insertion order (correlation 1), so a 24 kB BRIN index replaces a 21 MB B-tree:

```
Aggregate (actual time=0.716..0.717 rows=1 loops=1)
  Buffers: shared hit=130 read=3
  ->  Bitmap Heap Scan on orders (actual time=0.222..0.656 rows=2787 loops=1)
        Rows Removed by Index Recheck: 11992
        Heap Blocks: lossy=128                <- one 128-page block range read
        ->  Bitmap Index Scan on o_created_brin (... rows=1280 loops=1)
```

It is *lossy*: it only knows which 128-page ranges *might* hold matching timestamps, so it reads the whole range and rechecks every row. On a column in random order every range would span everything and BRIN would be useless.

GIN on arrays and JSONB:

```
CREATE INDEX posts_tags_gin ON posts USING gin (tags);
CREATE INDEX posts_meta_gin ON posts USING gin (meta jsonb_path_ops);
SELECT count(*) FROM posts WHERE tags @> ARRAY['go', 'advanced'];     -- 20000, via Bitmap Index Scan on
posts_tags_gin
EXPLAIN (COSTS OFF) SELECT count(*) FROM posts WHERE meta @> '{"lang": "hi", "likes": 7}';
--   ->  Bitmap Index Scan on posts_meta_gin
```

GIN is fast to read but slower to update (one row touches many posting lists — the per-element lists of rows; PostgreSQL buffers new entries in a "pending list").

GiST enforcing "no double booking" — an **exclusion constraint** (no two rows may conflict under the given operators; `tsrange` is a range of timestamps and `&&` means "overlaps"; needs `btree_gist` for the `room WITH =` part):

```
CREATE TABLE bookings (room int, during tsrange, EXCLUDE USING gist (room WITH =, during WITH &&));
INSERT INTO bookings VALUES (101, '[2024-05-01 14:00, 2024-05-03 11:00)');
INSERT INTO bookings VALUES (102, '[2024-05-01 14:00, 2024-05-03 11:00)');  -- other room: fine
INSERT INTO bookings VALUES (101, '[2024-05-02 14:00, 2024-05-04 11:00)');  -- overlaps!
```

```
ERROR:  conflicting key value violates exclusion constraint "bookings_room_during_excl"
DETAIL:  Key (room, during)=(101, ["2024-05-02 14:00:00","2024-05-04 11:00:00")) conflicts with existing key
(room, during)=(101, ["2024-05-01 14:00:00","2024-05-03 11:00:00")).
```

Elsewhere: MySQL offers B-tree, full-text, spatial and multi-valued JSON indexes (8.0.17+); SQL Server rowstore, columnstore, full-text and spatial; SQLite B-tree plus FTS5 and R*Tree.

12.10 Reading EXPLAIN: the four ways to read a table

What it is

`EXPLAIN` shows the **plan** the optimizer (another name for the planner) chose: a tree of operators, read from the innermost (most indented) node outwards. `EXPLAIN ANALYZE` also **runs** the query and adds real row counts and times; `BUFFERS` adds pages hit in cache / read from the OS. Key fields:

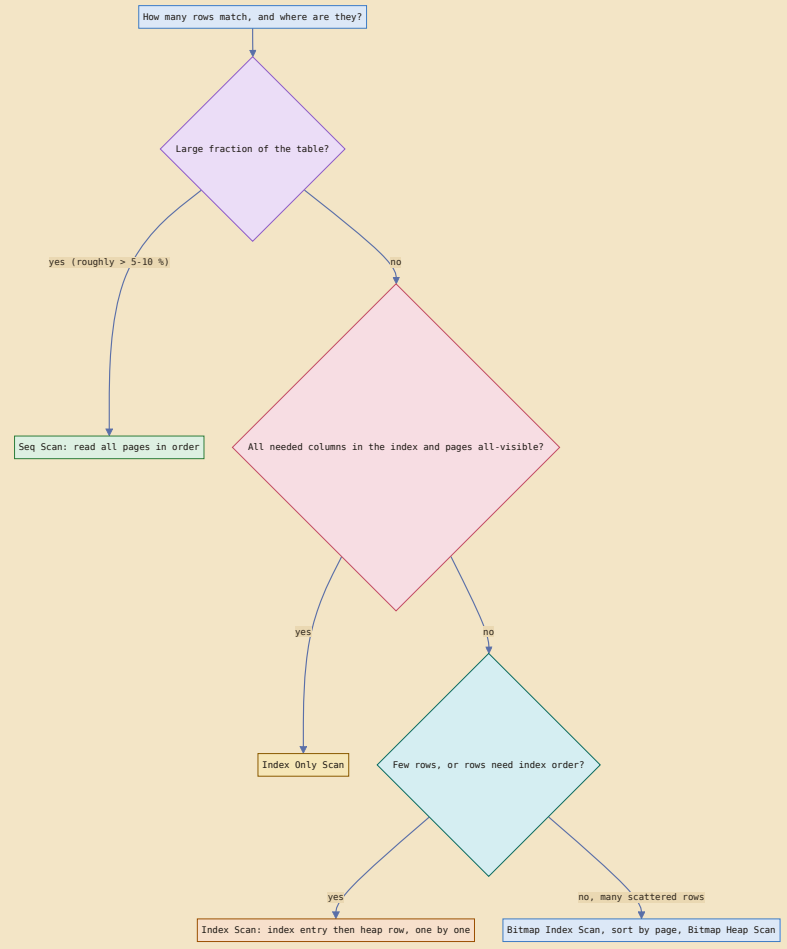
Field	Meaning
`cost=0.42..8.44`	Planner's estimate in abstract units: startup cost .. total cost (a sequential page read = 1.0, random = `random_page_cost` = 4.0)
`rows=` (estimate) vs `actual ... rows=`	Estimated vs real row count — a big gap means bad statistics (12.16)
`loops=`	How many times the node ran (inner side of a nested loop); times and rows are per loop
`Buffers: shared hit=/read=`	Pages found in the buffer pool / fetched from the OS
`Filter` + `Rows Removed by Filter`	Rows read then thrown away — wasted work
`Index Cond`	Condition applied *inside* the index — the useful work

`EXPLAIN ANALYZE` executes the statement — wrap `INSERT/UPDATE/DELETE` in `BEGIN … ROLLBACK`. MySQL has `EXPLAIN` and `EXPLAIN ANALYZE` (8.0.18+); SQL Server shows plans via `SET SHOWPLAN_XML ON` / "actual execution plan"; SQLite uses `EXPLAIN QUERY PLAN`.

The four access methods, measured

From `02_scans.sql` (parallel query — splitting one query across several worker processes — switched off to keep plans small):

Query	Plan	Pages touched	Time
`customer_id = 4242`, no index	**Seq Scan** — read every page, filter	8,662	~30 ms
same, with `orders_customer_idx`	**Bitmap Index + Bitmap Heap Scan**	13	~0.04 ms
`id = 500000`	**Index Scan** on `orders_pkey`	7	~0.01 ms
`count(*) … customer_id BETWEEN 1000 AND 1999`	**Index Only Scan**, `Heap Fetches: 0`	15	~0.5 ms
`sum(amount) … status = 'shipped'` (95 %)	**Seq Scan** despite the index	8,662	~76 ms
`sum(amount) … customer_id BETWEEN 1000 AND 1999`	**Bitmap Heap Scan**, `exact=8454`	8,468	~5 ms
`status = 'refunded' AND customer_id < 2000`	**BitmapAnd** of two indexes	—	—



How to read each:

1. **Seq Scan** reads pages in physical order (storage read-ahead — fetching the next pages before they are requested — makes this fast). For a predicate keeping 95 % of rows it is the right choice.
2. **Index Scan** walks the B-tree, then fetches each heap row; best for a few rows, and rows come out **in index order**, so `ORDER BY … LIMIT` can stop early.
3. **Index Only Scan** answers from the index alone, visiting the heap only for pages not marked all-visible in the **visibility map** (maintained by `VACUUM`; shown as `Heap Fetches`, ch. 13).
4. **Bitmap scan** collects matching `ctid`s into a bitmap (a compact array of bits, one per row or page), sorts them by page and reads each needed page once; it can combine indexes (`BitmapAnd/Or`). Short of memory, the bitmap turns *lossy* (whole pages) and rows are rechecked.

Note the 1 % case: 10,000 rows lived on **8,454 of 8,662 pages**. The index saved almost nothing because the rows were scattered — the reason the planner cares about correlation (12.3) and why "selective" means *few pages*, not just few rows.

Self-check

1. Why did `status = 'shipped'` ignore its index? → 95 % of rows match; reading index + nearly every heap page randomly costs more than one sequential pass.
2. What does `Heap Fetches: 0` mean? → Every page was all-visible, so the index-only scan never touched the table.
3. What is the difference between `Filter` and `Index Cond`? → `Index Cond` narrows the search inside the index; `Filter` discards rows after they were read.

12.11 Composite indexes and column order

What it is

A **composite** (multi-column) index sorts by the first column, then by the second within equal first values, and so on — like a phone book sorted by last name, then first name. From `03_composite_indexes.sql`, with `CREATE INDEX orders_cust_created_idx ON orders (customer_id, created_at)`:

Query	Uses the index?	Why
`WHERE customer_id = 777`	Yes (3 index pages)	Leading column
`WHERE customer_id = 777 ORDER BY created_at DESC LIMIT 3`	Yes, **Index Scan Backward**, no Sort node, 6 pages	Within one customer, entries are already in time order
`WHERE created_at >= '2024-06-01' AND created_at < '2024-06-01 00:10'`	**No** → Seq Scan, 8,662 pages, ~30 ms	Second column alone: one day's rows are spread across all customers' sections
same, after `CREATE INDEX … (created_at)`	Index Only Scan, 4 pages, ~0.04 ms	Now the range is contiguous

```
 Limit (actual time=0.002..0.003 rows=3 loops=1)
   Buffers: shared hit=6
   ->  Index Scan Backward using orders_cust_created_idx on orders (... rows=3 loops=1)
         Index Cond: (customer_id = 777)
```

This is the **leftmost-prefix rule**: an index on `(a, b, c)` serves filters on `a`, `a,b` and `a,b,c` efficiently, but not on `b` or `c` alone. (Oracle, MySQL 8.0.13+ and PostgreSQL 18+ can sometimes "skip scan" a composite index when the leading column has few distinct values; don't design around it.)

Equality first, range last

Same query, `customer_id = 777 AND created_at >= '2024-06-01'`, two column orders:

```
-- (customer_id, created_at)
   ->  Bitmap Index Scan on orders_cust_created_idx (... rows=5 loops=1)
         Buffers: shared hit=3
-- (created_at, customer_id)
 Index Scan using orders_created_cust_idx on orders (actual time=1.541..8.275 rows=5 loops=1)
   Index Cond: ((created_at >= '2024-06-01 ...') AND (customer_id = 777))
   Buffers: shared hit=5 read=2211
```

8 pages (3 of them index) vs 2,216 pages. With the equality column first, all matches sit in one narrow slice. With the range column first, the index must walk every entry after June 1 (≈ 55 % of the index) and test `customer_id` on each. Rule of thumb: **equality columns first, then the range or sort column; among equality columns, the most-used-alone first.**

Common mistakes

- Separate `(a)`, `(a, b)`, `(a, b, c)` indexes — the last covers the others.
- Putting a low-selectivity column first "because it's in every query" when queries also filter on a range — check the range test above.

- Assuming `(a, b)` helps `ORDER BY b` — only when `a` is fixed by equality.

12.12 Covering, partial and expression indexes

From `04_covering_partial_expression.sql`.

Covering index (`INCLUDE`, PostgreSQL 11+, SQL Server). Store extra *payload* columns in the leaf entries so the query never visits the table. They are not part of the key (not sorted, not unique-checked).

```
CREATE INDEX orders_cust_incl_idx ON orders (customer_id) INCLUDE (amount);
SELECT sum(amount) FROM orders WHERE customer_id BETWEEN 5000 AND 5999;
```

Index	Plan	Pages	Time
`(customer_id)`	Bitmap Heap Scan, 8,450 heap pages	8,464	~5.3 ms
`(customer_id) INCLUDE (amount)`	**Index Only Scan**, Heap Fetches: 0	42	~0.75 ms

In MySQL you get the same effect by adding the column to the key (`(customer_id, amount)`); InnoDB secondary indexes also always contain the primary key.

Partial index (PostgreSQL, SQLite, SQL Server "filtered index"; not MySQL). Index only the rows a query cares about:

```
CREATE INDEX orders_pending_idx ON orders (created_at) WHERE status = 'pending';
SELECT id, created_at FROM orders WHERE status = 'pending' ORDER BY created_at LIMIT 5;
```

```
        index_name         |  size
---------------------------+--------
 orders_pending_idx        | 456 kB       -- 2 % of rows
 orders_status_created_idx | 30 MB        -- full (status, created_at) rival
 Limit (actual time=0.009..0.011 rows=5 loops=1)
   ->  Index Scan using orders_pending_idx on orders (... rows=5 loops=1)
         Buffers: shared hit=2 read=2
```

The plan has no `Index Cond` at all: every entry in the index is already a pending order. Perfect for work queues (tables of jobs waiting to be processed: "oldest unprocessed job"), soft deletes (`WHERE deleted_at IS NULL`) and **conditional uniqueness** — `CREATE UNIQUE INDEX ON orders (customer_id) WHERE status = 'pending'` allows at most one pending order per customer. The planner uses a partial index only when the query's `WHERE` provably implies the index predicate; `status = 'refunded'` gets a Seq Scan + Sort.

Expression (functional) index. Index the result of an expression; queries must use the *same* expression:

```
CREATE INDEX customers_lower_email_idx ON customers (lower(email));
SELECT id FROM customers WHERE lower(email) = 'user4242@mail.com';   -- Seq Scan 918 pages -> Index Scan 4 pages
SELECT id FROM customers WHERE upper(email) = 'USER4242@MAIL.COM';   -- Seq Scan: different expression
```

Only `IMMUTABLE` functions are allowed (same input, same output forever). MySQL 8.0.13+ supports functional key parts `((lower(email)))`; SQL Server indexes a computed column; alternatives are a `citext` (case-insensitive text) column or a case-insensitive collation (the rules a database uses to compare and sort text).

12.13 Anti-patterns that disable indexes

From `05_antipatterns.sql`. The index only helps when the **bare column** is compared with something the planner can compute up front — a property called **sargable** (Search ARGument ABLE).

Anti-pattern	Plan	Fix	Plan after fix
`WHERE created_at::date = DATE '2024-06-01'`	Seq Scan, 8,662 pages, ~39 ms	`created_at >= '2024-06-01' AND created_at < '2024-06-02'`	Index Only Scan, 15 pages, ~0.2 ms
`WHERE id + 1 = 500001`	Seq Scan	`WHERE id = 500001 - 1`	Index Scan on `orders_pkey`
`WHERE email LIKE '%4242@corp%'` (leading wildcard)	Seq Scan, 918 pages	trigram GIN index	Bitmap scan, 74 pages
`WHERE email = … OR name = …` (name unindexed)	Seq Scan	index both sides (`BitmapOr`) or rewrite as `UNION`	BitmapOr
`WHERE lower(email) = …` without an expression index	Seq Scan	expression index (12.12)	Index Scan

Other classics: comparing a column to a value of a different type (in MySQL `WHERE varchar_col = 123` casts the column and skips the index), `NOT IN` / `<>` (rarely selective), and `SELECT *` (rules out index-only scans).

Prefix vs leading wildcard. A B-tree is sorted by the *start* of the string, so `LIKE 'User4242%'` becomes a range (`email >= 'User4242' AND email < 'User4243'`, visible in the plan's Index Cond). This database uses the C collation; under other collations create the index with `text_pattern_ops`. For `'%…%'` the sort order is useless; **pg_trgm** splits text into 3-character pieces and a GIN index finds rows sharing the pattern's trigrams:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
SELECT show_trgm('User42');     -- {"  u"," us","42 ",er4,r42,ser,use}
CREATE INDEX customers_email_trgm_idx ON customers USING gin (email gin_trgm_ops);
```

```
 Bitmap Heap Scan on customers (actual time=0.563..0.565 rows=3 loops=1)
   Recheck Cond: (email ~~ '%4242@corp%'::text)
   ->  Bitmap Index Scan on customers_email_trgm_idx (... rows=3 loops=1)
         Buffers: shared hit=71
```

For word search in prose, use full-text search (word-based search that understands word forms and ranks matches) instead (`tsvector` + GIN, ch. 14).

The N+1 query problem

Not an index problem but the most common performance bug in application code: load a list (1 query), then run one more query **per item** (N queries). ORMs do this silently when you touch a lazy relationship inside a loop.

> **Key term — ORM:** an *object-relational mapper* is a library that lets application code work with database rows as ordinary objects (`customer.orders`) and writes the SQL for you. A **lazy** relationship is loaded only when first touched — convenient, but touching it once per item in a loop fires one query per item. **Eager loading** fetches the related rows up front in one or two queries.
> `n_plus_one.py` totals the orders of 500 customers three ways:

```
for cid, _name in customers:                                  # N+1
    conn.execute("SELECT coalesce(sum(amount), 0) FROM orders WHERE customer_id = %s", (cid,))
conn.execute("SELECT customer_id, sum(amount) FROM orders WHERE customer_id = ANY(%s) "
             "GROUP BY customer_id", (ids,))                  # batched: 2 queries
conn.execute("SELECT c.id, coalesce(sum(o.amount), 0) FROM (...) c "
             "LEFT JOIN orders o ON o.customer_id = c.id GROUP BY c.id")   # 1 query
```

```
n_plus_one       queries= 501  local time=   14.5 ms  + network @1ms RTT ~    501 ms
batched_in_list  queries=   2  local time=    1.4 ms  + network @1ms RTT ~      2 ms
single_join      queries=   1  local time=    3.1 ms  + network @1ms RTT ~      1 ms
all three return the same totals: True
```

Over a local Unix socket (a same-machine connection with no network) the loop costs 10×; across a real network each round trip (a request out and its reply back; RTT = round-trip time) adds latency (waiting time), so 501 queries at 1 ms each is half a second. Fixes: a join, an `IN` / `= ANY` batch, or the ORM's eager loading (`select_related` / `prefetch_related` in Django, `JOIN FETCH` / `@EntityGraph` in JPA, `includes` in Rails — popular Python, Java and Ruby frameworks).

12.14 OFFSET vs keyset pagination

`LIMIT 20 OFFSET 999960` does not jump to row 999,960: the database must produce and **discard** all earlier rows. **Keyset** (seek) pagination instead remembers the last row shown and asks for rows *after* it, which an index answers directly. `06_pagination.sql`, with `CREATE INDEX orders_created_id_idx ON orders (created_at, id)`, newest first, 20 per page:

```
-- OFFSET: page N
SELECT id, created_at, amount FROM orders
ORDER BY created_at DESC, id DESC LIMIT 20 OFFSET (N - 1) * 20;
-- Keyset: pass the last (created_at, id) of the previous page
SELECT id, created_at, amount FROM orders
WHERE (created_at, id) < (:'last_ts', :last_id)
ORDER BY created_at DESC, id DESC LIMIT 20;
```

```
       method       | exec_ms | buffers
--------------------+---------+---------
 OFFSET, page 1     |    0.02 |       4
 OFFSET, page 100   |    0.15 |      39
 OFFSET, page 10000 |   15.47 |    3425
 OFFSET, page 49999 |   79.05 |   17114
 keyset, page 49999 |    0.02 |       4
 rows_that_differ = 0
 Limit (actual time=0.004..0.007 rows=20 loops=1)
   ->  Index Scan Backward using orders_created_id_idx on orders (... rows=20 loops=1)
         Index Cond: (ROW(created_at, id) < ROW('2024-01-01 00:21:11'::timestamp ..., 41))
```

OFFSET cost grows linearly with the page number; keyset stays at 4 pages. Rules: make the sort key **unique** by adding the primary key as a tie-breaker (else rows with equal timestamps are skipped or repeated); the row comparison `(a, b) < (x, y)` matches a composite index in PostgreSQL (elsewhere write `a < x OR (a = x AND b < y)`); keyset cannot jump to "page 537" but never skips or duplicates rows when data changes between requests. APIs (the interfaces programs call over the web) hide the last key in an opaque **cursor** — a token the client passes back without reading it (Stripe, Slack, GitHub GraphQL).

12.15 Join algorithms

The logical join (ch. 5) says *what* to match; the planner picks *how*. From `07_join_algorithms.sql`:

Algorithm	How it works	Cost	Best when
Nested loop	For each outer row, look up matching inner rows (ideally via an index)	outer rows × one lookup	Outer side small, inner side indexed on the join key
Hash join	Build a hash table (an in-memory lookup table keyed by hash) on the smaller input's join key, then stream the larger input and probe it	read both once; needs memory (`work_mem`, PostgreSQL's per-operation memory budget)	Large unsorted inputs, equality joins only
Merge join	Both inputs sorted on the key (by index or by a Sort); walk them together like merging two sorted lists	read both once (+ sorts)	Inputs already sorted, or output needs that order; also works for range-ish joins

Three customers (30 orders):

```
Nested Loop (actual time=0.015..0.047 rows=30 loops=1)
  ->  Index Scan using customers_pkey on customers c (... rows=3 loops=1)
  ->  Bitmap Heap Scan on orders o (... rows=10 loops=3)     <- inner side runs 3 times
        ->  Bitmap Index Scan on orders_customer_idx (... loops=3)
```

All customers × all orders, grouped by country:

```
HashAggregate (actual time=362.870..362.906 rows=10 loops=1)
  ->  Hash Join (actual time=11.526..288.224 rows=1000000 loops=1)
        Hash Cond: (o.customer_id = c.id)
        ->  Seq Scan on orders o (... rows=1000000 loops=1)
        ->  Hash (actual time=11.401..11.402 rows=100000 loops=1)
              Buckets: 131072  Batches: 2  Memory Usage: 2976kB
```

`Batches: 2` means the 100,000-row hash table did not fit in `work_mem` (4 MB), so it was split and partly written to temporary files (`temp read/written` in the buffers line). Raising `work_mem` for such a query would make it one batch. With `ORDER BY c.id LIMIT 5` the planner avoids hashing everything: it walks `customers_pkey` in `id` order in a **Nested Loop**, looks up each customer's orders by index, and stops after five rows. A **Merge Join** needs both inputs sorted on the key; here it appears only when forced (below).

Forcing each algorithm with the `enable_*` settings (for experiments only):

```
     join_size        |  forced  |  join_node  | exec_ms
----------------------+----------+-------------+---------
 small: 3 customers   | nestloop | Nested Loop |     0.1
 small: 3 customers   | hash     | Hash Join   |    33.9
 small: 3 customers   | merge    | Merge Join  |     0.2
 large: all 1M orders | nestloop | Nested Loop |   525.8
 large: all 1M orders | hash     | Hash Join   |   236.7
 large: all 1M orders | merge    | Merge Join  |   487.5
```

No algorithm always wins: hash join is 300× slower than a nested loop for the small join (it scans all 1M orders to build/probe) and 2× faster for the large one. The planner chooses using **row estimates** — which is why statistics matter. MySQL used only nested-loop variants until 8.0.18 added hash joins; SQL Server and Oracle have all three.

12.16 Statistics: how the planner estimates

What it is

The planner cannot run every candidate plan, so it **estimates** how many rows each step produces using statistics that `ANALYZE` (the command that gathers table statistics) collects from a random sample (30,000 rows at the default `default_statistics_target = 100`) and stores in `pg_statistic` (readable through the `pg_stats` view). Autovacuum (PostgreSQL's background maintenance process) runs `ANALYZE` automatically after enough rows change. Because sampling is random, your frequencies may differ slightly. From `08_statistics.sql`:

```
   attname   | null_frac | n_distinct | correlation
-------------+-----------+------------+-------------
 amount      |         0 |      50066 |       0.002
 created_at  |         0 |         -1 |       1.000
 customer_id |         0 |  -0.100437 |       0.003
 note        |    0.9009 |          1 |       1.000
 status      |         0 |          3 |       0.907
     most_common_vals      |        most_common_freqs
---------------------------+---------------------------------
 {shipped,refunded,pending} | {0.95103335,0.0293,0.019666666}
```

Statistic	Meaning	Used for
`null_frac`	Fraction of NULLs	`IS NULL` estimates
`n_distinct`	Distinct values; negative = fraction of rows (−1 = unique, −0.1 = 10 rows per value)	`=` on non-common values, `GROUP BY` sizes
`most_common_vals/freqs`	The most frequent values and their shares	`status = 'refunded'` → 0.0293 × 1M ≈ 29,300 rows
`histogram_bounds`	Equal-population buckets (a histogram: how values are spread across ranges) over the other values	Range predicates
`correlation`	Physical order vs column order	Index-scan cost (12.3)

With fresh statistics, estimate and reality agree: `rows=29300` estimated vs `actual rows=30000`.

When statistics go wrong

Stale statistics. A table where `'error'` was 0.1 % of rows when analyzed; then 200,000 error rows arrive and nobody re-analyzes:

```
Index Scan using events_kind_idx on events  (cost=0.42..24.78 rows=440 ...) (actual rows=200200 loops=1)
-- after ANALYZE:
Index Scan using events_kind_idx on events  (cost=0.42..5249.09 rows=200000 ...) (actual rows=200200 loops=1)
```

The estimate was off by roughly 450× (the exact figure varies with ANALYZE's random sample). Here the plan survived (the new rows happen to be stored together), but in a join, such an error is how a planner picks a nested loop that runs 200,000 times instead of one hash join. Run `ANALYZE` after bulk loads; don't wait for autovacuum.

Correlated columns. The planner multiplies selectivities as if columns were independent. `city = 42 AND state = 4` (city determines state): estimated about 100 rows (106 in the run shown; it varies with sampling), actual 1,000. **Extended statistics** teach it the dependency:

```
CREATE STATISTICS addr_city_state (dependencies) ON city, state FROM addresses;
ANALYZE addresses;      -- estimate becomes about 1,000 (1040 in this run), actual 1000
```

Other levers: `ALTER TABLE … ALTER COLUMN … SET STATISTICS 1000` for a finer histogram on a skewed column (one where a few values are far more common than others); `CREATE STATISTICS … (ndistinct, mcv)` for multi-column `GROUP BY` and value combinations.

In the interview

30-second answer: "The optimizer is cost-based (it scores candidate plans by estimated work and picks the lowest): it estimates rows per step from sampled statistics — null fraction, distinct counts, most common values, histograms, correlation — and picks the cheapest plan. When a query is slow, I compare estimated and actual rows in EXPLAIN ANALYZE; a big gap points to stale or missing statistics, fixed with ANALYZE, a higher statistics target or extended statistics."

- *Why would the planner ignore my index?* → Low selectivity, poor correlation, tiny table, stale stats, a function or type mismatch on the column, or the query needs most rows anyway.

12.17 A tuning workflow

1. **Find** the expensive queries: `pg_stat_statements` (an extension that keeps timing statistics for every query), slow-query logs (`log_min_duration_statement`, MySQL slow log), `auto_explain`.
2. **Read** `EXPLAIN (ANALYZE, BUFFERS)`: big `Rows Removed by Filter`, estimate/actual gaps, `Batches > 1` or on-disk sorts, huge `loops=`.

3. **Fix the query first** (sargable predicates, fewer columns, keyset, no N+1), **then add indexes** (equality → range/sort, `INCLUDE`, partial, expression; `CONCURRENTLY` in production).
4. **Verify** with `EXPLAIN`; drop indexes whose `pg_stat_user_indexes.idx_scan` stays 0; keep statistics fresh.
5. **Still too big?** Partitioning (splitting a table into pieces), replicas (read-only copies on other servers), caching, sharding (spreading data across several servers) — see System Design Fundamentals and chapter 15.

Dialect notes

Feature	PostgreSQL	MySQL (InnoDB)	SQL Server	SQLite
Table storage	Heap + separate indexes	Clustered on PK	Heap or clustered index	B-tree by rowid (or `WITHOUT ROWID`)
Page size	8 KB	16 KB	8 KB	4 KB default
Covering columns	`INCLUDE` (11+)	add to key; PK always included	`INCLUDE`	add to key
Partial index	Yes	No	Filtered index	Yes
Expression index	Yes	Functional key parts (8.0.13+)	Index on computed column	Yes
Hash index	Yes	MEMORY engine; adaptive hash (auto)	Memory-optimized tables	No
Plan output	`EXPLAIN (ANALYZE, BUFFERS)`	`EXPLAIN ANALYZE` (8.0.18+)	Actual execution plan	`EXPLAIN QUERY PLAN`

Chapter TL;DR

- Performance = **pages touched**. A table is a file of 8 KB slotted pages; 1M orders = 8,662 pages, and a Seq Scan reads them all (~30 ms here).
- Rows are addressed by `ctid` (page, slot); large values are compressed/moved by TOAST. Column stores + RLE/dictionary/delta encoding make analytics cheap.
- File organizations: heap, sorted, hashed, clustered. Physical order (correlation) decides whether an index read touches 100 pages or 8,400.
- Indexes: primary/clustering/secondary, dense/sparse; multilevel indexes → **B+ trees** (3 levels for 1M keys). Hash indexes: equality only. Bloom filters: "definitely not / probably yes", ~10 bits/key ≈ 1 % false positives.
- GIN for contains/JSON/trigrams, GiST for overlaps and exclusion constraints, BRIN for huge append-only time columns.
- Scans: Seq (many rows) · Index (few rows, ordered) · Index Only (all columns in the index, vacuumed) · Bitmap (many scattered rows, combinable).
- Composite indexes: **leftmost prefix**; **equality first, range last** (8 vs 2,216 pages). `INCLUDE` for index-only scans; partial indexes for hot subsets; expression indexes for `lower(email)`.
- Keep predicates **sargable**; no functions on indexed columns; `pg_trgm` for `'%x%'`; kill N+1 loops; **keyset** instead of deep OFFSET (79 ms → 0.02 ms).
- Joins: nested loop (small + index), hash (big, equality), merge (sorted). The planner chooses from **statistics** — compare estimated vs actual rows and `ANALYZE` when they diverge.
- Next: chapter 13 (MVCC, vacuum — why dead rows and visibility maps affect all of the above) and chapter 15 (partitioning and scaling beyond one server).

Chapter 13 — Transactions and Concurrency

A database is rarely used by one person at a time: a shop's stock table takes hundreds of checkouts per second while reports read it. Two things can go wrong — a failure can stop a change **halfway**, and two users working **at the same time** can trip over each other. This chapter covers the two tools against those problems, **transactions** and **concurrency control**, and shows how PostgreSQL really behaves with live sessions (separate open connections to the database) running side by side.

What you'll learn

- Transactions, their life cycle, and `BEGIN` / `COMMIT` / `ROLLBACK` / `SAVEPOINT`.
- The four ACID guarantees and how databases implement each.
- Every classic anomaly — dirty read/write, non-repeatable read, phantom, lost update, write skew — **reproduced on a real PostgreSQL server** with two interleaved sessions.
- The four isolation levels in PostgreSQL vs MySQL, SQL Server, Oracle and SQLite.
- Concurrency control: locks and two-phase locking, optimistic validation, timestamp ordering, and MVCC (PostgreSQL's `xmin` / `xmax` from real queries).
- Deadlocks, `SELECT … FOR UPDATE`, `NOWAIT`, and the `SKIP LOCKED` job queue.

How to run the examples. All code lives in `code/`. The single-session files run on the book's private PostgreSQL 14 server (port 5544):

```
bash tools_run_sql.sh sql                                   # runs every chapter's .sql files
bash tools_pg.sh psql -d <db> -f sql/chapters/13-transactions-and-concurrency/code/01_commit_and_rollback.sql
```

The two-session demos are in `code/concurrency_demo.py`: connections **A** and **B** send statements in a fixed order; a statement expected to block runs in a background thread (a second line of execution inside the same program) while a third *monitor* connection checks `pg_stat_activity` (a system view listing every connection and what it is doing) to prove it is waiting on a lock:

```
cd sql/chapters/13-transactions-and-concurrency/code
../../../../.venv-books/bin/python concurrency_demo.py            # all scenarios (~2 s)
../../../../.venv-books/bin/python concurrency_demo.py writeskew  # just one group
```

In the output, `A| …` is a statement from session A, `->` the server's answer, `..` a monitor observation. Transaction IDs differ on every run. Each scenario starts from fresh sample data (`00_schema.sql`):

```
CREATE TABLE accounts (id int PRIMARY KEY, owner text NOT NULL,
                       balance numeric(10,2) NOT NULL CHECK (balance >= 0));
INSERT INTO accounts VALUES (1, 'asha', 500), (2, 'ben', 300), (3, 'chen', 50);

CREATE TABLE products (id int PRIMARY KEY, name text NOT NULL,
                       stock int NOT NULL CHECK (stock >= 0), version int NOT NULL DEFAULT 1);
INSERT INTO products VALUES (1, 'concert ticket', 10, 1), (2, 'poster', 3, 1);

CREATE TABLE on_call (doctor text PRIMARY KEY, shift text NOT NULL, on_duty boolean NOT NULL);
INSERT INTO on_call VALUES ('dr_rao', 'night', true), ('dr_kim', 'night', true), ('dr_lee', 'day', true);

CREATE TABLE jobs (id int PRIMARY KEY, payload text NOT NULL,
                   status text NOT NULL DEFAULT 'queued', claimed_by text);
INSERT INTO jobs (id, payload) SELECT g, 'email #' || g FROM generate_series(1, 6) g;
```

13.1 What a transaction is

What it is

A **transaction** is a group of database operations treated as **one logical unit of work**: either every operation takes effect, or none does. It ends with a **commit** (all changes become permanent and visible

to others) or an **abort**/rollback (every change is undone, as if it never ran).

Analogy: a transaction is like a shopping cart at a self-checkout. You can scan ten items, but nothing is paid for until you press "pay". If the card is declined, the whole cart is cancelled — you never end up paying for seven of the ten items.

The problem it solves

Sending ₹100 from Asha to Ben needs two writes: subtract 100 from Asha, add 100 to Ben. If the server crashes or a rule is broken **between** them, Asha has lost 100 and Ben never received it. A transaction makes the pair all-or-nothing, which also makes **retrying safe**: after an abort there are no half-applied leftovers, so the application simply runs it again.

ACID in one table

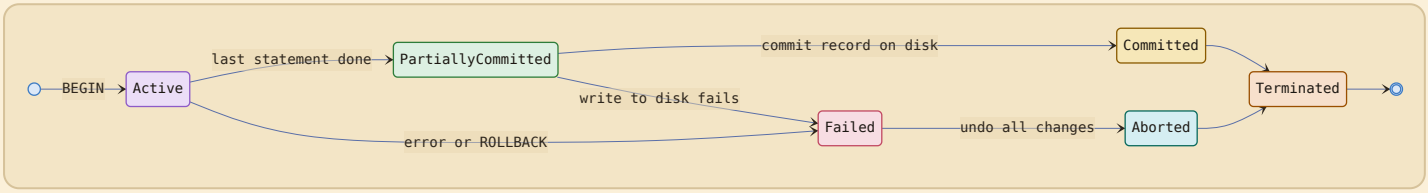
The guarantees a transaction gives are summarised by the acronym **ACID** (coined by Härder and Reuter in 1983). Each letter gets its own section below.

Letter	Guarantee	Main mechanism
Atomicity	All or nothing — no half-done transfers	Undo information / WAL (write-ahead log, §13.5), shadow copies (§13.2)
Consistency	Valid state → valid state (constraints, invariants — rules that must always hold — are true after commit)	Constraints + correct application logic
Isolation	Concurrent transactions don't see each other's in-progress work	Locks, MVCC (multi-version concurrency control, §13.14), SSI (serializable snapshot isolation, §13.8)
Durability	Committed means kept, even after a crash or power loss	WAL flushed to disk, replication (live copies on other servers)

Life cycle of a transaction

A transaction moves through a small set of **states**:

State	What is happening
Active	Statements are running; changes are not yet final.
Partially committed	Last statement done; results not yet safely on permanent storage.
Committed	The commit record is durable; changes are permanent.
Failed	An error (constraint, crash, deadlock — two transactions each waiting for the other, §13.11 — cancel) stops normal progress.
Aborted	All changes rolled back; the database is as before the transaction.
Terminated	Finished (by commit or abort); locks and snapshot (the frozen view of the data it was reading) released.



After an abort the application either **restarts** the transaction (for temporary problems such as a deadlock) or **gives up** and reports the error (for logical problems such as "insufficient funds").

How to write one in SQL

Statement	Effect
`BEGIN` (or `START TRANSACTION`)	Start a transaction block.
`COMMIT`	Make all changes permanent.
`ROLLBACK`	Undo all changes since `BEGIN`.
`SAVEPOINT name`	Mark a point inside the transaction.
`ROLLBACK TO SAVEPOINT name`	Undo only the work done after that mark; the transaction stays open.
`RELEASE SAVEPOINT name`	Forget the mark (keep the work).

Autocommit. If you do *not* write `BEGIN`, PostgreSQL (like psql, JDBC — Java's standard database API — and most drivers, the libraries programs use to talk to a database, by default) wraps **each single statement** in its own transaction and commits it immediately. So a lone `UPDATE` is already atomic; you need `BEGIN … COMMIT` only when several statements must succeed or fail together. (Python's `psycopg`, the usual PostgreSQL driver for Python, is the odd one out: by default it opens a transaction for you and waits for `conn.commit()`.)

`code/01_commit_and_rollback.sql`:

```sql
-- 1) Money transfer that commits: both rows change together.
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;   -- debit asha
UPDATE accounts SET balance = balance + 100 WHERE id = 2;   -- credit ben
COMMIT;
SELECT id, owner, balance, (SELECT sum(balance) FROM accounts) AS bank_total
FROM accounts ORDER BY id;

-- 2) A transfer we change our mind about: ROLLBACK restores the old state.
BEGIN;
UPDATE accounts SET balance = balance - 200 WHERE id = 1;
SELECT 'inside txn' AS moment, balance FROM accounts WHERE id = 1;
ROLLBACK;
SELECT 'after rollback' AS moment, balance FROM accounts WHERE id = 1;

-- 3) Savepoints: undo only part of a transaction.
BEGIN;
UPDATE accounts SET balance = balance + 10 WHERE id = 3;    -- keep this
SAVEPOINT bonus;
UPDATE accounts SET balance = balance + 999 WHERE id = 3;   -- oops
ROLLBACK TO SAVEPOINT bonus;                                -- undo just the oops
COMMIT;
SELECT id, owner, balance FROM accounts WHERE id = 3;
```

Output:

```
BEGIN
UPDATE 1
UPDATE 1
COMMIT
 id | owner | balance | bank_total
----+-------+---------+------------
  1 | asha  |  400.00 |     850.00
  2 | ben   |  400.00 |     850.00
  3 | chen  |   50.00 |     850.00
(3 rows)

BEGIN
UPDATE 1
   moment   | balance
------------+---------
 inside txn |  200.00
(1 row)

ROLLBACK
     moment     | balance
----------------+---------
 after rollback |  400.00
(1 row)

BEGIN
UPDATE 1
SAVEPOINT
UPDATE 1
ROLLBACK
COMMIT
 id | owner | balance
----+-------+---------
  3 | chen  |   60.00
(1 row)
```

The bank total stays 850 (money moved, none created or lost); the session saw its own uncommitted 200, but after `ROLLBACK` the balance is 400 again; and only the +10 survived the savepoint.

Dialect differences

	PostgreSQL	MySQL (InnoDB)	SQL Server	SQLite
Start	`BEGIN` / `START TRANSACTION`	`START TRANSACTION` / `BEGIN`	`BEGIN TRAN`	`BEGIN [DEFERRED|IMMEDIATE|EXCLUSIVE]`
DDL (structure changes like `CREATE TABLE`) inside a transaction	Transactional (`CREATE TABLE` can be rolled back)	Causes an implicit commit (silently commits the open transaction)	Mostly transactional	Transactional
After an error mid-transaction	Whole transaction is "aborted" until `ROLLBACK`	Only the failing statement is undone; transaction continues	Depends on `XACT_ABORT`	Only the failing statement is undone

Weaker guarantees some systems offer

Many NoSQL (non-relational) and caching systems trade multi-statement transactions away for speed or scaling and offer smaller building blocks:

Guarantee	What it means	Example
Single-object atomicity	A write to *one* record/document is all-or-nothing, but nothing spans two records.	A single MongoDB document update; a Cassandra row write.
Atomic increment	The server does read-add-write in one indivisible step, so concurrent `+1`s never lose each other.	Redis (a popular in-memory key-value store) `INCR`; SQL `UPDATE … SET n = n + 1`.
Compare-and-set (CAS)	"Write the new value only if the current value is still what I read." If someone changed it, the write fails and you retry.	Redis `WATCH`/`MULTI`, DynamoDB (Amazon's managed key-value database) conditional writes, a SQL `UPDATE … WHERE version = 7` (§13.7).

They protect **one** object only; keeping several records in step (a transfer, an order plus its stock decrement) needs a real transaction — which is why MongoDB (4.0+) and NewSQL systems (distributed databases that keep SQL and full transactions) added multi-document transactions.

In the interview

30-second answer: "A transaction is a unit of work that fully commits or fully rolls back, with the ACID guarantees. In SQL: `BEGIN … COMMIT/ROLLBACK`, savepoints for partial rollback; without `BEGIN` each statement autocommits."

- *Why not undo step 1 myself on error?* The crash may kill your program too; the database's log handles it even after a power failure.
- *Is a single `UPDATE` atomic?* Yes — in autocommit mode it is its own transaction, and even a multi-row `UPDATE` changes all matching rows or none.

Self-check

1. Name the six transaction states. → Active, partially committed, committed, failed, aborted, terminated.
2. What does autocommit mean? → Each statement is its own transaction, committed as soon as it succeeds.
3. What does `ROLLBACK TO SAVEPOINT s` keep? → Everything done before `SAVEPOINT s`; the transaction stays open.

13.2 Atomicity

What it is

Atomicity means a transaction is **indivisible**: all of its changes are applied (commit), or none are (rollback). There is no visible "half-done" state.

The problem it solves

Take accounts X = 30 and Y = 100 and a transfer of 10 from X to Y. The debit succeeds (X = 20), then the credit fails. Without atomicity the database is left with X = 20, Y = 100: ten units have disappeared. With atomicity the failure rolls back the debit too, leaving X = 30, Y = 100 — exactly as before.

How it works — three implementations

1. Shadow copy (the simplest idea, used by early systems and some editors).

1. When a transaction begins, copy the whole database file; the copy is the **shadow**.
2. Make every change in the shadow only; the original is untouched.
3. To commit, atomically switch a **db-pointer** (a small file naming the current database file) to the shadow, then delete the old file.

4. To abort, delete the shadow; the pointer never moved.

So the only commit point is the pointer switch: if every write succeeds but the crash comes just before the switch, or a failure happens midway, the transaction is aborted and the untouched original remains current.

It is impractical for real databases: copying the **entire** database per transaction is slow and wastes storage, and it allows only **one** transaction at a time, so it can't scale. (Its descendant *shadow paging* — copy only modified pages — was used in System R and lives on in copy-on-write stores — changes always go to fresh copies, never over the original — such as LMDB.)

2. Logging (how PostgreSQL, MySQL, SQL Server and Oracle do it). Changes go into in-memory pages, and every change is also described in the **write-ahead log (WAL)** (§13.5). If the transaction aborts or the system crashes before commit, the engine undoes the changes from its log — or, in PostgreSQL, simply marks the transaction ID *aborted* so its row versions are ignored (§13.14).

3. Across machines. When one transaction touches several databases, **two-phase commit (2PC)** makes them agree: a coordinator asks every participant to *prepare* (make the change durable but not final) and, only if all say yes, tells them to *commit* (chapter 15; PostgreSQL's `PREPARE TRANSACTION`). Optimistic concurrency control (§13.9) also applies a transaction's writes all at once, at commit, only if no conflict was found.

Code: an error aborts the whole transaction

`code/02_atomicity_on_error.sql` — Chen has 50 and tries to send 80. The credit to Asha runs first and succeeds; then the debit breaks the `CHECK (balance >= 0)` rule (`\set ON_ERROR_STOP off` is a psql setting that lets the script continue after an error):

```
\set ON_ERROR_STOP off
BEGIN;
UPDATE accounts SET balance = balance + 80 WHERE id = 1;   -- credit asha: OK
UPDATE accounts SET balance = balance - 80 WHERE id = 3;   -- debit chen: violates CHECK
SELECT balance FROM accounts WHERE id = 1;                  -- refused: txn is aborted
COMMIT;                                                     -- turns into ROLLBACK
SELECT id, owner, balance FROM accounts ORDER BY id;        -- nothing changed

BEGIN;
UPDATE accounts SET balance = balance + 5 WHERE id = 2;
SAVEPOINT before_risky;
INSERT INTO accounts VALUES (1, 'dup', 1);                  -- duplicate primary key
ROLLBACK TO SAVEPOINT before_risky;
COMMIT;
SELECT id, owner, balance FROM accounts WHERE id = 2;
```

Output (psql file path shortened):

```
BEGIN
UPDATE 1
psql:02_atomicity_on_error.sql:10: ERROR:  new row for relation "accounts" violates check constraint
"accounts_balance_check"
psql:02_atomicity_on_error.sql:11: ERROR:  current transaction is aborted, commands ignored until end of
transaction block
ROLLBACK
 id | owner | balance
----+-------+---------
  1 | asha  |  500.00
  2 | ben   |  300.00
  3 | chen  |   50.00
(3 rows)

BEGIN
UPDATE 1
SAVEPOINT
psql:02_atomicity_on_error.sql:19: ERROR:  duplicate key value violates unique constraint "accounts_pkey"
ROLLBACK
COMMIT
 id | owner | balance
----+-------+---------
  2 | ben   |  305.00
(1 row)
```

After an error PostgreSQL marks the transaction **aborted** and refuses further commands; `COMMIT` is answered with `ROLLBACK`, undoing Asha's successful +80 too. Rolling back to a savepoint clears the error and keeps Ben's +5.

Common mistakes

- Running the two halves of a transfer as two autocommitted statements.
- Catching the error in application code and carrying on to `COMMIT` — PostgreSQL silently rolls back; MySQL may commit the half that worked.
- Assuming rollback undoes an email sent or card charged inside the transaction — it undoes only **database** changes; record the action in an *outbox* table (a table of pending messages, written in the same transaction and sent by a separate process) and send after commit.

In the interview

> **30-second answer:** "All-or-nothing. Shadow copy (work on a copy, flip a pointer at commit) explains the idea but copies too much and serialises everything; real engines log changes and roll back from the log; across databases you need two-phase commit."

- *Does atomicity say anything about concurrency?* No — ACID's "atomic" is about failures; other users are isolation's job (unlike "atomic" in multithreaded code).

Self-check

1. In shadow copy, what happens if the crash occurs after all writes but before the pointer update? → The transaction is aborted; the original database is still current.
2. Why doesn't shadow copy scale? → It copies the whole database per transaction and allows one writer at a time.
3. What does PostgreSQL answer to `COMMIT` in an aborted transaction? → `ROLLBACK`.

13.3 Consistency

What it is

Consistency (the C in ACID) means every committed transaction leaves the database **valid if it started valid**. "Valid" is defined by rules: primary and unique keys, foreign keys, `CHECK` and `NOT NULL`

constraints, and business **invariants** such as "money is neither created nor destroyed by a transfer". A transaction that would break a rule must fail and leave the previous valid state untouched; rules may be briefly false only *inside* a transaction.

Worked example — a transfer keeps the total

Asha has 300, Ben has 400: the total is **700**. Transfer 100 from Asha to Ben:

Moment	Step	Asha	Ben	Total	Valid?
t1	initial state	300	400	700	yes
t2	`BEGIN`	300	400	700	yes
t3	debit Asha 100	200	400	**600**	no (intermediate)
t4	credit Ben 100	200	500	700	yes
t5	`COMMIT`	200	500	700	yes
t6	final state	200	500	700	yes

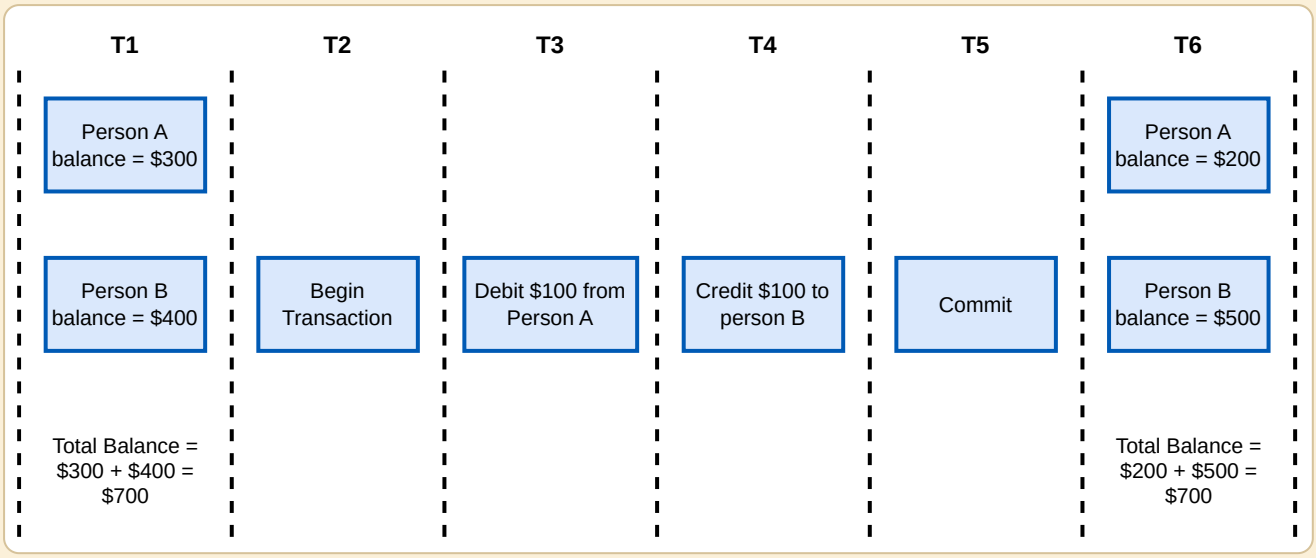


Figure — the total balance is correct before and after the transaction; only the invisible middle is inconsistent (course diagram).

The temporary 600 at t3 is harmless: nobody else can see it (isolation) and it can never be the final state (atomicity).

How it is achieved

1. **Declarative constraints** checked by the database: `PRIMARY KEY`, `UNIQUE`, `FOREIGN KEY`, `CHECK (balance >= 0)`, `NOT NULL` (chapter 9). §13.2's demo showed the `CHECK` rejecting an overdraft.
2. **Pre-transaction checks** in the application or in SQL: e.g., "does Asha have enough money?" before debiting (best done *in the same statement*: `UPDATE … SET balance = balance - 100 WHERE id = 1 AND balance >= 100`).
3. **Post-transaction verification**: deferred constraints (`DEFERRABLE INITIALLY DEFERRED` checks a foreign key only at commit) or audits confirm the invariant, e.g., the ledger still sums to the same total.
4. **Atomicity** and **isolation** stop failures and concurrent transactions from leaving a broken state.

When consistency can be violated

Cause	Example	Who stops it
Constraint violation	Inserting a duplicate primary key	The database rejects the statement
Crash mid-transaction	Crash after debit, before credit	Atomicity + recovery roll it back
Logic bug	Code transfers more than the balance because it forgot the check	Only a constraint (`CHECK`) or correct code
Concurrency without enough isolation	Two withdrawals both see enough money and both proceed	Isolation level / locking (§13.4–13.8)

Trade-offs and a note on the word "consistency"

The database enforces only rules it knows about. Cross-row invariants ("at least one doctor on call") can't be a simple `CHECK`; the application enforces them and needs strong enough isolation (§13.8). Hence Martin Kleppmann (author of DDIA, *Designing Data-Intensive Applications*) calls C a property of the *application*, with A, I and D as the database's tools for it.

Also, **ACID consistency is not CAP consistency.** In CAP (the theorem about trade-offs in distributed databases; see System Design Fundamentals), "consistency" means *linearizability* — every read sees the latest write across replicas (copies of the data on other servers; chapter 15). Same word, different idea.

In the interview

> **30-second answer:** "Every committed state obeys the rules — keys, checks, business invariants. The database enforces declared constraints, the application the rest; atomicity and isolation keep failures and concurrency from breaking them."

- *Give an invariant a CHECK can't express.* "At least one doctor is on call per shift" — it spans rows; needs SERIALIZABLE or explicit locking.

Self-check

1. Is a temporarily wrong total inside a transaction a violation? → No; only committed states must be valid.
2. Name three enforcement mechanisms. → Constraints, pre-checks in the statement/app, atomic rollback (plus isolation).
3. Why can concurrency break consistency? → Two transactions each valid alone can together break an invariant if isolation is too weak.

13.4 Isolation — anomalies and isolation levels

What it is

Isolation controls what one running transaction can see of another's in-progress work. Ideally each behaves **as if it were alone**, but running transactions truly one at a time is too slow, so databases let them overlap and offer **isolation levels**: weaker levels allow more overlap but permit certain **anomalies** (wrong results caused by interleaving); stronger levels forbid more of them at the cost of blocking or aborts.

Analogy: cooks sharing one kitchen. "One cook at a time" is full isolation; real kitchens let everyone work with rules (don't take from someone's half-finished dish). The isolation level is how strict the rules are.

The anomalies

Timelines below: **T1** and **T2** are concurrent; the account starts at 500.

1. Dirty read — reading data another transaction has written but **not committed**.

Time	T1	T2
1	`BEGIN`; balance = 500 − 100 → **400** (uncommitted)	
2		reads balance → **400**
3	`ROLLBACK` → balance is 500 again	
4		has acted on 400, a value that never officially existed

2. Dirty write — overwriting a value another transaction has written but **not committed**. T1 adds 100 (500 → 600, uncommitted); T2 adds 100 on top (→ 700); then T1 aborts. Restoring "500" erases T2's deposit; keeping 700 keeps T1's cancelled deposit; the correct 600 can't be reconstructed. Practically every database, at every level, prevents dirty writes by making the second writer **wait** for the first to finish.

3. Non-repeatable read (also called read skew) — the same row read twice in one transaction gives **different committed values**, because another transaction committed a change in between.

Time	T1	T2
1	reads balance → 500	
2		balance = 400; `COMMIT`
3	reads balance again → **400**	

"Read skew" is the broader name for seeing *different moments in time* in one transaction — e.g., reading account 1 before a transfer and account 2 after it, so the total looks wrong. §13.6 shows exactly this on PostgreSQL.

4. Phantom read — the same **query** run twice returns a **different set of rows**, because another transaction inserted or deleted rows that match the condition.

Time	T1	T2
1	`SELECT count(*) … WHERE balance >= 100` → 2	
2		`INSERT` a matching row; `COMMIT`
3	same query → **3** (a "phantom" row appeared)	

5. Lost update — two transactions each **read** a value, **compute** a new value, and **write** it; the second write silently overwrites the first.

Time	T1	T2
1	reads stock → 10	
2		reads stock → 10
3	writes 10 − 1 = 9	
4		writes 10 − 1 = 9
	Two items sold, stock only fell by one.	

6. Write skew — two transactions read the **same rows**, each decides based on what it read, and each writes **a different row**. Neither overwrote the other, yet together they break a rule each checked: two on-call doctors each see "2 on call", both go off duty, nobody is left (§13.8). A lost update is the special case where both write the *same* row.

The isolation levels

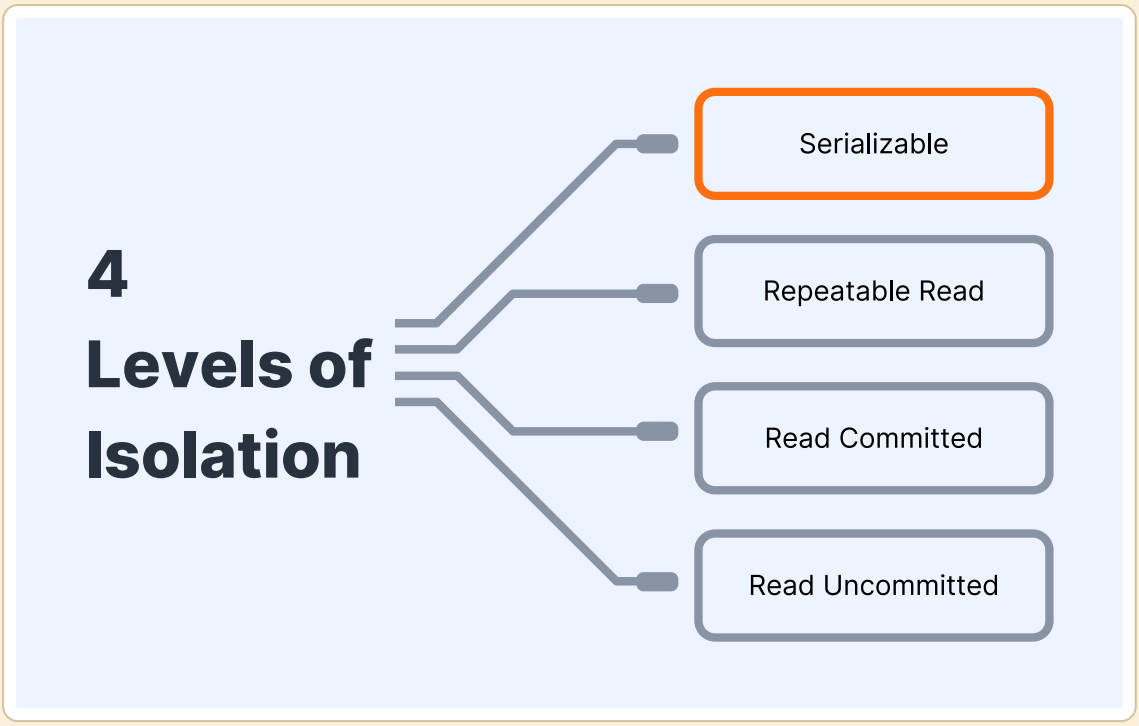


Figure — the four SQL-standard isolation levels (course diagram).

The SQL standard defines four levels by which of the first three read anomalies they must prevent:

Level	Dirty read	Non-repeatable read	Phantom
Read Uncommitted	possible	possible	possible
Read Committed	prevented	possible	possible
Repeatable Read	prevented	prevented	possible
Serializable	prevented	prevented	prevented

That is the *minimum*; real engines often give more, and the standard ignores lost updates and write skew. What **PostgreSQL** actually provides (verified in §13.6–13.8):

Level in PostgreSQL	Dirty read	Dirty write	Non-repeatable / read skew	Phantom	Lost update	Write skew	How it's done
Read Uncommitted	**no** (treated as Read Committed)	no	yes	yes	yes	yes	new snapshot per statement
Read Committed (default)	no	no	yes	yes	yes (read-then-write in app)	yes	new snapshot per statement + row locks for writes
Repeatable Read	no	no	no	**no** (stronger than the standard)	no — aborts with a serialization error	**yes**	one snapshot per transaction (snapshot isolation)
Serializable	no	no	no	no	no	no — aborts one	snapshot isolation + SSI conflict tracking

Two mechanisms matter. At **Read Committed**, MVCC engines hand readers the last committed version so they never wait, while lock-based engines (SQL Server unless `READ_COMMITTED_SNAPSHOT` is on) make readers wait for the writer. **Serializable** (outcome equals *some* one-at-a-time order) is built with **two-phase locking** (take locks as you go and release none until the end, §13.10; MySQL, SQL Server) or **Serializable Snapshot Isolation (SSI)**, which tracks read/write dependencies and aborts a transaction when needed (PostgreSQL 9.1+) — so always be ready to retry. Read Uncommitted (SQL Server's `NOLOCK` hint) is almost never used.

Other engines at a glance

Engine	Default level	Notes
PostgreSQL	Read Committed	RR = snapshot isolation (no phantoms); Serializable = SSI.
MySQL InnoDB	Repeatable Read	Plain `SELECT` reads a snapshot; locking reads (`FOR UPDATE`, `UPDATE`) read the *latest* row with next-key locks (locks on index entries plus the gaps between them, §13.10) against phantoms. Does **not** abort lost updates at RR — use `FOR UPDATE` or atomic updates.
SQL Server	Read Committed (locking)	Optional `READ_COMMITTED_SNAPSHOT` and `SNAPSHOT` isolation use row versions in tempdb (SQL Server's shared scratch database).
Oracle	Read Committed	Its "SERIALIZABLE" is actually snapshot isolation (write skew possible).
SQLite	Serializable	Only one writer at a time for the whole database file; WAL mode lets readers run during a write.

How to choose and set a level

```
BEGIN ISOLATION LEVEL REPEATABLE READ;                  -- for one transaction
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;           -- first statement inside BEGIN
SET default_transaction_isolation = 'serializable';     -- for the session
SHOW transaction_isolation;
```

If you need…	Use
Normal OLTP (online transaction processing: many short reads and writes), single-statement updates (`SET x = x + 1`)	Read Committed
A consistent report or export across many queries	Repeatable Read (often `READ ONLY`)
Read-then-write logic on the same row	`SELECT … FOR UPDATE` at Read Committed, or Repeatable Read + retry
Invariants that span rows (no double booking, at least one on call)	Serializable + retry, or explicit locks

In the interview

30-second answer: "Isolation levels trade correctness for concurrency; the standard defines four by which of dirty reads, non-repeatable reads and phantoms they allow. Postgres never allows dirty reads, its Repeatable Read is snapshot isolation (no phantoms or lost updates, but write skew), and Serializable uses SSI to abort one transaction of a write skew."

- *What's the difference between a non-repeatable read and a phantom?* Non-repeatable: the *same row* changed value. Phantom: the *set of rows* matching a condition changed (insert/delete).
- *Why isn't everyone on Serializable?* More aborts (or blocking with 2PL) and the application must retry; most workloads are fine at Read Committed with a few targeted locks.

Self-check

1. Which anomaly involves an insert? → Phantom read.
2. What is PostgreSQL's default level? MySQL's? → Read Committed; Repeatable Read.
3. Two transactions update *different* rows and break a rule together — which anomaly? → Write skew.

13.5 Durability

What it is

Durability promises that once the database says "committed", the change **stays** — even if the power fails or the process is killed a millisecond later. People act on "committed" (ship the order, release the ticket), so recovery must restore every committed transaction and nothing uncommitted.

Analogy: a shop that writes every sale in an ink ledger *before* handing over the receipt; if the till loses power, the ledger still says what was sold.

How it works

1. **Write-ahead logging (WAL).** A record describing each change must reach the log before the changed data page may be written. At `COMMIT` the database appends a commit record and **flushes** (`fsync` — the operating-system call that forces data out of memory caches onto the physical disk) the log to disk; only then does it answer "COMMIT". Data pages are written later, lazily; after a crash, recovery **replays** the log, redoing committed changes that never reached the data files. (One sequential append is much faster than updating scattered pages.)
2. **Checkpoints.** Periodically all dirty pages (pages changed in memory but not yet written to the data files) are written to the data files and "everything up to log position X is on disk" is recorded, so recovery replays only the log *after* the latest checkpoint (PostgreSQL: `checkpoint_timeout`, default 5 min, and `max_wal_size`).
3. **Synchronous disk writes.** A commit is acknowledged only after the log is on stable storage, not just in RAM or the OS cache. PostgreSQL's `synchronous_commit = on` and `fsync = on` (both defaults) guarantee this; `full_page_writes` protects against *torn pages* (a page only partly written when power fails).
4. **Replication.** Disks die, so the WAL is shipped to other machines (PostgreSQL streaming replication — sending WAL records to replicas continuously as they are written); with *synchronous* replication a commit also waits for a replica (chapter 15).

Code: watching the WAL grow

`code/05_durability_wal.sql` inserts 1,000 rows and measures how far the WAL position (**LSN**, log sequence number) moved:

```
SELECT name, setting FROM pg_settings
WHERE name IN ('fsync', 'synchronous_commit', 'wal_level', 'full_page_writes', 'checkpoint_timeout')
ORDER BY name;

CREATE TEMP TABLE lsn_marks (label text, lsn pg_lsn);
INSERT INTO lsn_marks VALUES ('before', pg_current_wal_lsn());
INSERT INTO accounts SELECT g, 'user' || g, 100 FROM generate_series(10, 1009) g;   -- 1,000 rows
INSERT INTO lsn_marks VALUES ('after', pg_current_wal_lsn());

SELECT pg_size_pretty(pg_wal_lsn_diff(
         (SELECT lsn FROM lsn_marks WHERE label = 'after'),
         (SELECT lsn FROM lsn_marks WHERE label = 'before'))) AS wal_written_for_1000_rows;

BEGIN;
SET LOCAL synchronous_commit = off;     -- faster commit; a crash may lose this txn, never corrupts
UPDATE accounts SET balance = balance + 1 WHERE id = 10;
COMMIT;
```

Output:

```
        name        | setting
--------------------+---------
 checkpoint_timeout | 300
 fsync              | on
 full_page_writes   | on
 synchronous_commit | on
 wal_level          | replica
(5 rows)
...
 wal_written_for_1000_rows
---------------------------
 135 kB
(1 row)
...
```

The 1,000 rows produced about 135 kB of WAL (the exact size can vary slightly by version and settings). The `synchronous_commit = off` transaction returns before its WAL is flushed: a crash within about three times `wal_writer_delay` could lose that one commit, but never corrupt the database — fine for click logging, never for payments. Conversely, a normal committed transfer or order survives a power cut a moment later, because WAL replay restores it.

Common mistakes

- Turning off `fsync` for speed in production — a crash can then **corrupt** the database, not just lose recent commits.
- Assuming a replica makes a commit durable when replication is asynchronous (the primary — the main server that accepts writes — doesn't wait for replicas) — the primary can die before shipping it.
- Disks or cloud volumes that lie about flushes (a write cache — fast memory on the drive — without battery backup, so its contents vanish on power loss).

In the interview

30-second answer: "Committed data survives crashes: the WAL record is fsynced before the commit is acknowledged, data pages are written later, and restart replays the log from the last checkpoint. Replication protects against losing the machine."

- *Why not write the data pages at commit?* Scattered page writes cost far more than a single sequential append to the log, and the log record alone is enough to redo the change.

Self-check

1. What must be on disk before `COMMIT` returns? → The transaction's WAL records including the commit record.
2. What does a checkpoint shorten? → Crash recovery (replay starts at the last checkpoint).
3. Does asynchronous replication strengthen durability of the latest commit? → No; only synchronous replication does.

13.6 Watching isolation anomalies on a real server

This section runs the anomalies from §13.4 on PostgreSQL 14 with two live sessions (A and B) using `code/concurrency_demo.py`. The core of the driver:

```python
class Session:
    def __init__(self, name):
        self.name = name
        self.conn = psycopg.connect(f"{BASE} dbname={DB}", autocommit=True)  # we send BEGIN/COMMIT ourselves
        self.pid = self.conn.info.backend_pid

    def run(self, sql):                          # send one statement, print the answer
        print(f"  {self.name}| {sql}  -> {self._exec(sql)}")

    def start(self, sql):                        # a statement we expect to BLOCK: run it in a thread
        print(f"  {self.name}| {sql}")
        box = {}
        t = threading.Thread(target=lambda: box.setdefault("r", self._exec(sql)))
        t.start()
        return self, t, box

class Monitor:                                   # a third connection that watches the other two
    def wait_until_blocked(self, s, timeout=5.0):
        # poll pg_stat_activity until session s shows wait_event_type = 'Lock'
        ...
        # SELECT wait_event_type, wait_event, pg_blocking_pids(pid)
        # FROM pg_stat_activity WHERE pid = %s
```

`_exec` returns the rows, the command tag (`UPDATE 1`), or `ERROR <sqlstate>: <message>` (SQLSTATE is the standard five-character error code, e.g. `40001`). In deadlock messages, it replaces the process IDs with `<A>`/`<B>` so the output is readable.

Dirty read and dirty write — impossible in PostgreSQL

Session A changes Asha's balance to 0 but doesn't commit. Session B even asks for `READ UNCOMMITTED`:

```
=== Dirty read and dirty write: impossible in PostgreSQL ===
  A| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = 0 WHERE id = 1  -> UPDATE 1
  B| BEGIN ISOLATION LEVEL READ UNCOMMITTED  -> BEGIN
  B| SHOW transaction_isolation  -> read uncommitted
  B| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  B| UPDATE accounts SET balance = balance + 1 WHERE id = 1
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| ROLLBACK  -> ROLLBACK
     -> (B resumes) UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT balance FROM accounts WHERE id = 1  -> 501.00
```

- **No dirty read:** B really is at `read uncommitted`, yet reads the committed **500**, not A's 0 — PostgreSQL implements Read Uncommitted as Read Committed. The read also didn't block: MVCC hands B the older committed version.
- **No dirty write:** B's `UPDATE` of the same row **waits** (wait event `transactionid`: B waits for A's transaction to end). After A's rollback B updates the committed 500 → 501.

Non-repeatable read / read skew: Read Committed vs Repeatable Read

A reads Asha's balance. Meanwhile B transfers 100 from Asha (account 1) to Ben (account 2) and commits. A then reads both accounts.

```
=== Non-repeatable read / read skew: READ COMMITTED ===
  A| BEGIN ISOLATION LEVEL READ COMMITTED  -> BEGIN
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  B| BEGIN  -> BEGIN
  B| UPDATE accounts SET balance = balance - 100 WHERE id = 1  -> UPDATE 1
  B| UPDATE accounts SET balance = balance + 100 WHERE id = 2  -> UPDATE 1
  B| COMMIT  -> COMMIT
  A| SELECT balance FROM accounts WHERE id = 1  -> 400.00
  A| SELECT balance FROM accounts WHERE id = 2  -> 400.00
  A| COMMIT  -> COMMIT
```

At Read Committed the same row gave **500 then 400** — a non-repeatable read. An auditor adding its first read of account 1 (500) to its read of account 2 (400) would report **900**, though the true total was 800 at every instant: **read skew**.

```
=== Non-repeatable read / read skew: REPEATABLE READ ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  ... (B runs the same transfer as above and commits)
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  A| SELECT balance FROM accounts WHERE id = 2  -> 300.00
  A| COMMIT  -> COMMIT
```

At Repeatable Read A keeps seeing the world as of its **first statement** (not `BEGIN`): 500 and 300, total 800 — consistent, just slightly old.

Phantom read: Read Committed vs Repeatable Read

```
=== Phantom read: READ COMMITTED ===
  A| BEGIN ISOLATION LEVEL READ COMMITTED  -> BEGIN
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  B| INSERT INTO accounts VALUES (4, 'dina', 900)  -> INSERT 0 1
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 3
  A| COMMIT  -> COMMIT

=== Phantom read: REPEATABLE READ (PostgreSQL prevents it) ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  B| INSERT INTO accounts VALUES (4, 'dina', 900)  -> INSERT 0 1
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  A| COMMIT  -> COMMIT
```

At Read Committed a phantom appears (2 → 3). At Repeatable Read the count stays 2 — the standard would *allow* a phantom, but PostgreSQL's snapshot simply doesn't contain Dina's row.

Common mistakes

- Believing Repeatable Read shows "the latest data" — a long Repeatable Read transaction sees an increasingly stale world (and holds back cleanup, §13.14).
- Running a multi-query report at Read Committed and getting totals that never existed. Use `BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY`.

13.7 Lost updates and how to prevent them

The problem

A **lost update** comes from the *read–modify–write* pattern: the application reads a value, computes a new one in its own code, and writes it back; if two copies overlap, the later write wipes out the earlier one. It is the most common concurrency bug in web apps (stock counters, likes, wallet balances, wiki edits).

Seeing it happen

Two customers buy a concert ticket at the same moment. Each app instance reads the stock, subtracts one in Python/Java, and writes the result:

```
=== Lost update 1: read-modify-write in the application (READ COMMITTED) ===
  A| BEGIN  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1  -> 10
  B| BEGIN  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1  -> 10
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 9
```

Two tickets sold, stock fell only to **9**, and nothing errored. The fix must make read and write one protected step; there are four standard ways.

Fix 1 — let the database do the arithmetic (atomic update)

```
=== Lost update fix 1: atomic UPDATE ... SET stock = stock - 1 ===
  A| BEGIN  -> BEGIN
  A| UPDATE products SET stock = stock - 1 WHERE id = 1 RETURNING stock  -> 9
  B| BEGIN  -> BEGIN
  B| UPDATE products SET stock = stock - 1 WHERE id = 1 RETURNING stock
     .. B is WAITING (wait_event=transactionid, blocked by A)
     .. pg_locks for B: (transactionid, ExclusiveLock, True), (tuple, ExclusiveLock, True), (transactionid,
ShareLock, False)
  A| COMMIT  -> COMMIT
     -> (B resumes) 8
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's `UPDATE` waits for A's row lock; when A commits, PostgreSQL (at Read Committed) **re-reads the newest row version** and applies `stock - 1` to it: 9 → 8. The `pg_locks` line (a system view of locks held and awaited) shows the mechanism: B holds its own transaction-ID lock and a *tuple* lock (first in line for the row) and waits (`granted = False`) for a `ShareLock` on **A's transaction ID** — "wake me when A ends". The row lock itself is recorded in the row (its `xmax`), not in a lock table, so locking a million rows needs no million lock entries.

Use when the new value can be computed in SQL (`n = n + 1`, `balance = balance - 100 WHERE balance >= 100`) — the simplest, fastest fix.

Fix 2 — pessimistic lock: `SELECT … FOR UPDATE`

When the new value needs application logic (call a pricing service, validate a complex rule), lock the row while reading it:

```
=== Lost update fix 2: pessimistic SELECT ... FOR UPDATE ===
  A| BEGIN  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1 FOR UPDATE  -> 10
  B| BEGIN  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1 FOR UPDATE
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
     -> (B resumes) 9
  B| UPDATE products SET stock = 8 WHERE id = 1  -> UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's read waits until A commits and then returns **9**, not 10, so B computes 8. The lock lasts until the end of the transaction — keep such transactions short and never wait for a human while holding one.

Fix 3 — Repeatable Read detects it (optimistic, by the database)

```
=== Lost update fix 3: REPEATABLE READ detects the conflict ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1  -> 10
  B| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1  -> 10
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| UPDATE products SET stock = 9 WHERE id = 1  -> ERROR 40001: could not serialize access due to concurrent
update
  B| ROLLBACK  -> ROLLBACK
  (B retries the whole transaction)
  ... (B re-reads 9, writes 8, commits)
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's snapshot says 10, but the row was changed by a transaction that committed after that snapshot, so PostgreSQL refuses with SQLSTATE **40001** (`serialization_failure`). The application **retries the whole transaction** and writes 8. (MySQL/InnoDB at Repeatable Read raises no such error — its `UPDATE` reads the latest row version — so there use Fix 1 or 2.)

Fix 4 — optimistic locking with a version column (compare-and-set)

Compare-and-set (CAS) means "change the value only if it is still what I read". It holds no lock, so it suits long gaps between read and write (a user editing a form for minutes, several HTTP requests — separate web requests, each in its own transaction). Add a `version` column and include it in the `WHERE`:

```
=== Lost update fix 4: optimistic locking with a version column (compare-and-set) ===
  A| SELECT stock, version FROM products WHERE id = 1  -> (10, 1)
  B| SELECT stock, version FROM products WHERE id = 1  -> (10, 1)
  A| UPDATE products SET stock = 9, version = version + 1 WHERE id = 1 AND version = 1  -> UPDATE 1
  B| UPDATE products SET stock = 9, version = version + 1 WHERE id = 1 AND version = 1  -> UPDATE 0
  (B sees UPDATE 0 -> someone changed the row; re-read and retry)
  B| SELECT stock, version FROM products WHERE id = 1  -> (9, 2)
  B| UPDATE products SET stock = 8, version = version + 1 WHERE id = 1 AND version = 2  -> UPDATE 1
  M| SELECT stock, version FROM products WHERE id = 1  -> (8, 3)
```

B's first update matched **0 rows** — the conflict signal (the version is no longer the *expected* 1). B re-reads (9, version 2) and succeeds. The database checks and writes in one atomic step; `UPDATE 0` means "someone else won — reload and retry, or tell the user the record changed". JPA/Hibernate's (Java) `@Version`, Rails' (Ruby) `lock_version`, DynamoDB conditional writes and Redis `WATCH`/`MULTI`/`EXEC` are the same idea: no locks (so no deadlocks), great when conflicts are rare, wasteful retries when they are frequent.

Comparison

Fix	Blocking?	Retry needed?	Best for
Atomic `UPDATE … SET x = x - 1`	Briefly (row lock)	No	Counters, balances — anything expressible in SQL
`SELECT … FOR UPDATE`	Yes, until commit	No	Complex logic between read and write, short transactions
Repeatable Read / Serializable	No (aborts instead)	Yes (40001)	Code that already has a retry loop
Version column (CAS)	No	Yes (0 rows)	Long "think time", web forms, APIs, distributed caches

In the interview

> **30-second answer:** "Two read-modify-write cycles interleave and one write overwrites the other. Fixes: atomic SQL arithmetic; SELECT FOR UPDATE; Repeatable Read, where Postgres aborts the second writer (40001); or a version column where zero updated rows means retry."

- *Which would you use for a "like" counter?* Atomic `UPDATE posts SET likes = likes + 1` (or a separate likes table and count).
- *Which for a user editing a product page for minutes?* Version column — never hold a DB lock across user think time.
- *Does Read Committed prevent lost updates?* Only if the write is expressed as one atomic statement; not for app-side read-modify-write.

Self-check

1. Why did the naive version end at 9? → Both read 10 and both wrote 10 − 1.
2. What does `UPDATE 0` mean in optimistic locking? → The version changed; someone else updated first.
3. What SQLSTATE does PostgreSQL use for a serialization failure? → 40001.

13.8 Write skew and Serializable isolation

The problem

Hospital rule: **at least one doctor on call for the night shift**. Dr Rao and Dr Kim, both on call, each click "go off duty"; the app checks "is anyone else on call?" and, if so, updates *that doctor's own row*. Different rows, so no row-lock conflict — yet the rule can break.

Repeatable Read (snapshot isolation) allows it

```
=== Write skew: REPEATABLE READ lets both doctors go off duty ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  B| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  B| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  B| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_kim'  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| COMMIT  -> COMMIT
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 0
```

Nobody is on call. Each transaction was correct against its snapshot, but run one after the other the second would have seen 1 and refused. The same shape appears in double-booking a room, claiming a username checked by `SELECT` first, or overspending a shared budget.

Serializable (SSI) aborts one

```
=== Write skew: SERIALIZABLE aborts one of them ===
  A| BEGIN ISOLATION LEVEL SERIALIZABLE  -> BEGIN
  B| BEGIN ISOLATION LEVEL SERIALIZABLE  -> BEGIN
  A| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  B| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  B| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_kim'  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| COMMIT  -> ERROR 40001: could not serialize access due to read/write dependencies among transactions
     DETAIL: Reason code: Canceled on identification as a pivot, during commit attempt.
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 1
```

How **Serializable Snapshot Isolation (SSI)** caught it, in steps:

1. Each Serializable transaction still reads from a snapshot (readers and writers don't block).

2. It records what it **read** as *SIREAD* "locks" — non-blocking notes such as "B read the night-shift rows".
3. When a transaction writes something a concurrent one read, a **read-write dependency** (rw-conflict) is recorded: here "A wrote what B read" and "B wrote what A read".
4. Such a cycle means no serial order could produce the result. On the dangerous pattern (a "pivot" transaction with a conflict in and a conflict out) PostgreSQL aborts one transaction with **40001**; the retried B sees only 1 doctor on call and refuses.

The error can appear at any statement, including `COMMIT`, so **every Serializable transaction needs a retry loop**:

```
for attempt in range(5):
    try:
        with conn.transaction():                        # BEGIN ... COMMIT
            conn.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")
            go_off_duty(conn, doctor)
        break
    except psycopg.errors.SerializationFailure:         # SQLSTATE 40001
        continue                                        # rerun the whole transaction
```

SSI can also abort harmless transactions (false positives) — its bookkeeping is approximate and coarsens from rows to pages or whole tables when a transaction reads a lot. Keep Serializable transactions short and mark read-only ones `READ ONLY`.

Fix without Serializable: lock what you based the decision on

If you know exactly which rows the decision depends on, lock them. Here both transactions run at the default Read Committed:

```
=== Write skew fix without SERIALIZABLE: lock the rows you base the decision on ===
  A| BEGIN  -> BEGIN
  B| BEGIN  -> BEGIN
  A| SELECT doctor FROM on_call WHERE shift = 'night' AND on_duty FOR UPDATE  -> (dr_rao), (dr_kim)
  B| SELECT doctor FROM on_call WHERE shift = 'night' AND on_duty FOR UPDATE
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  A| COMMIT  -> COMMIT
     -> (B resumes) dr_kim
  (B's app sees only one doctor left, so it refuses to go off duty)
  B| ROLLBACK  -> ROLLBACK
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 1
```

After waiting, B's `FOR UPDATE` re-checks the rows and returns only Dr Kim. Limitation: `FOR UPDATE` locks only rows that **exist**. If the rule is about the *absence* of rows ("no booking overlaps 10:00–11:00"), there is nothing to lock. Then use Serializable; a constraint (`UNIQUE`, or PostgreSQL's exclusion constraint — which forbids any two rows that conflict, here overlapping times for the same room — `EXCLUDE USING gist (room WITH =, during WITH &&)`, which needs the `btree_gist` extension for the `=` part); or *materialize the conflict* — pre-create lockable rows, e.g., one per room per time slot.

In the interview

30-second answer: "Two transactions read the same data, decide on it, and write different rows — no write conflict, but an invariant breaks. Snapshot isolation allows it. Fixes: Serializable (SSI aborts one with 40001 — retry), FOR UPDATE on the rows the decision depends on, or a unique/exclusion constraint."

- *Why doesn't FOR UPDATE fix double-booking of an empty slot?* The conflicting row doesn't exist yet (a phantom); use a constraint, Serializable, or materialized slot rows.

Self-check

1. At Repeatable Read, why was no conflict detected? → The two transactions updated different rows.

2. What are SIREAD locks? → Non-blocking records of what a Serializable transaction read, used to find rw-dependencies.
3. Name two non-Serializable fixes. → `FOR UPDATE` on the rows read; a unique/exclusion constraint.

13.9 Concurrency control — the big picture

What it is

Concurrency control is the part of a database that decides how overlapping transactions may interleave so the result stays correct: no anomalies from §13.4 (data integrity), each transaction isolated to the degree its level promises, and as much parallelism as possible (performance).

The formal correctness goal is **serializability**: a concurrent execution (a *schedule*) is correct if its effect equals that of *some* serial order of the same transactions. Two operations **conflict** if they belong to different transactions, touch the same item, and at least one is a write; a schedule is *conflict-serializable* if its conflict graph (an arrow T1 → T2 whenever an operation of T1 conflicts with, and comes before, one of T2) has no cycle.

The four families

Technique	Core idea	Waits or aborts?	Used by
Pessimistic / lock-based (e.g., 2PL)	Lock before you touch; others wait	Waits (deadlocks possible)	SQL Server, MySQL locking reads, DB2; `FOR UPDATE` everywhere
Optimistic (OCC)	Work freely, validate at commit, abort on conflict	Aborts	App-level version columns, some in-memory and distributed DBs
Timestamp ordering	Each txn gets a timestamp; operations must respect timestamp order	Aborts	Mostly textbook; ideas used in distributed DBs
MVCC	Keep several versions; readers read an old version, never block writers	Readers never wait; writers wait for writers	PostgreSQL, Oracle, MySQL InnoDB, SQL Server snapshot modes

These are combined in practice: PostgreSQL = MVCC for reads + row locks for writes + SSI (optimistic validation) for Serializable.

Optimistic vs pessimistic

Optimistic concurrency control (OCC) assumes conflicts are **rare**. A transaction runs in three phases: (1) **read/execute** in a private workspace, taking no locks; (2) **validate** at commit — did a transaction that committed meanwhile change anything it read? (3) **write** if not, otherwise **abort and retry**.

Example: users A and B open the same product; A saves first; B's save fails validation and retries — the version-column pattern of §13.7.

Pessimistic concurrency control (PCC) assumes conflicts are **likely**: **lock** data when first touching it, make conflicting transactions **wait**, and **release** at commit or rollback.

Example: A locks the product row; B waits until A finishes (§13.7 Fix 2).

Aspect	Optimistic (OCC)	Pessimistic (PCC)
Assumes	Conflicts are rare	Conflicts are common
Locks	None during execution	Taken on access, held to the end
Cost of a conflict	Abort and redo the work	Waiting
Overhead with no conflicts	Very low	Lock management on every access
Deadlocks	None (no waits)	Possible — need detection
Fits	Read-heavy, low-contention (few transactions competing for the same rows), distributed systems, long user think time	Write-heavy, hot rows (a popular product's stock), critical updates where retrying is expensive

Real-world examples. **Git** (the version-control tool) is optimistic: nobody locks a file, and conflicts surface only at push/merge time. **Google Docs** lets everyone type and reconciles edits as they sync. **Banking** and **ticketing** lean pessimistic on hot rows so a seat is never sold twice; **e-commerce** locks (or atomically decrements) the stock row at purchase.

In the interview

30-second answer: "Keep interleaved transactions serializable. Pessimistic: lock and wait (high contention; deadlocks possible). Optimistic: validate at commit and abort on conflict (rare conflicts). Timestamp ordering: follow start order. MVCC: versions so readers never block writers. Postgres combines MVCC, row locks and SSI."

Self-check

1. Three phases of OCC? → Read/execute, validate, write (or abort).
2. Why does OCC have no deadlocks? → Transactions never wait for each other's locks.
3. Which is better for a flash sale on one product? → Pessimistic (or an atomic update): contention is very high, OCC would abort constantly.

13.10 Lock-based concurrency control

What it is

With **lock-based concurrency control**, a transaction must obtain a **lock** on data before using it; a conflicting request waits until the lock is released — so others can't read or overwrite your in-progress changes.

Types of locks

Lock	Purpose	Others may…
Shared (S)	Reading	also take S (read together); may **not** write
Exclusive (X)	Writing	do nothing with the item (no S, no X) until released
Intention Shared (IS)	On a table: "I will take S locks on some rows inside"	—
Intention Exclusive (IX)	On a table: "I will take X locks on some rows inside"	—

Lock granularity

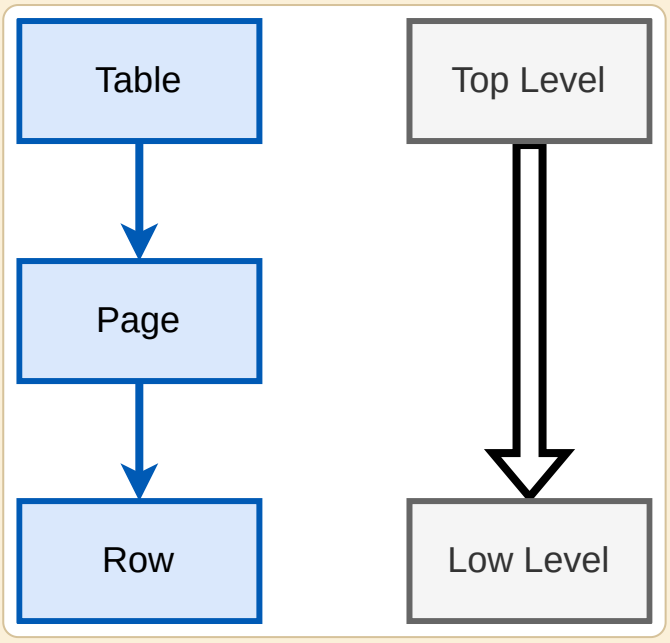


Figure — locks can be taken on a whole table, a page or a single row (course diagram).

Granularity	Advantage	Disadvantage	Example
Row	Maximum concurrency — other rows stay free	Many locks to manage (memory, CPU)	Update one customer's row
Page	Middle ground	Locks unrelated rows that share the page	Some storage engines (older SQL Server, DB2)
Table	Simple, very low overhead	Blocks everyone else	Batch update of the whole `orders` table

Some engines **escalate**: when a transaction holds thousands of row locks, they are swapped for one table lock (SQL Server does this). PostgreSQL never escalates row locks because it stores them in the rows themselves.

Intention locks — why they exist

T1 holds an X lock on row 5 of `employees`; T2 asks for an S lock on the **whole table**. Without extra information the lock manager would have to scan every row lock for conflicts. **Intention locks** make locking *hierarchical*:

1. T1 first places **IX** on the `employees` table, then **X** on row 5.
2. T2 asks for **S** on the table. The manager only checks table-level locks: S conflicts with IX, so T2 waits — no row scan needed.
3. A T3 wanting X on row 9 places IX on the table (IX is compatible with IX) and proceeds in parallel with T1.

Compatibility (can the requested lock be granted while another transaction holds…?):

Held → / Requested ↓	IS	IX	S	X
IS	yes	yes	yes	no
IX	yes	yes	no	no
S	yes	no	yes	no
X	no	no	no	no

Two-phase locking (2PL)

2PL is the classic protocol that guarantees conflict-serializability, in two phases:

1. **Growing (expanding) phase** — it may acquire locks but not release any.
2. **Shrinking phase** — once it releases its first lock, it may not acquire any new one.

The moment it holds the most locks is the **lock point**; transactions are serialized in the order of their lock points.

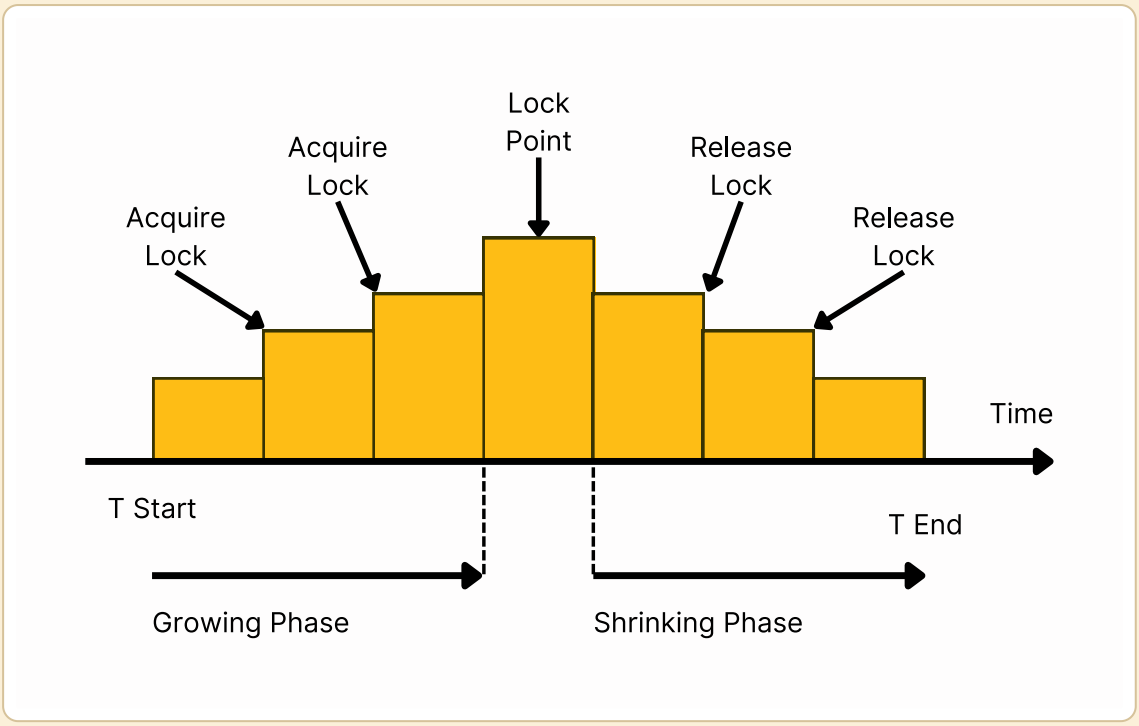


Figure — the number of locks held rises, peaks at the lock point, then falls (course diagram).

Variants you should know:

Variant	Rule	Why
Basic 2PL	grow, then shrink	Serializable, but a transaction could read data from another that later aborts (*cascading aborts*)
Strict 2PL	hold all **X** locks until commit/abort	No one reads uncommitted data; what most lock-based databases do
Rigorous 2PL	hold **all** locks until commit/abort	Simplest to reason about; serial order = commit order
Conservative 2PL	take every lock before starting	No deadlocks, but you must know the data set in advance

Pros: guaranteed serializability. *Cons:* **deadlocks** (§13.11) and lower throughput (work completed per second) under contention — a long reader blocks writers and vice versa. MySQL and SQL Server implement Serializable with (strict) 2PL; it suits strong-isolation domains such as finance.

Predicate locking and index-range locking

2PL on existing rows can't stop **phantoms**: a new row that *would* match your query didn't exist when you locked. Two answers:

Predicate locking extends 2PL by locking the **condition** itself — every row, existing or future, that satisfies `WHERE …`. Example: table `seats` holds seat numbers 10 and 30; transaction T runs `UPDATE seats SET held = true WHERE seat_no BETWEEN 10 AND 30`.

Seat	Exists now?	Locked by predicate lock?
10	yes	yes
20	no	yes — an insert of seat 20 must wait
30	yes	yes

It prevents phantoms and write skew, but checking every lock request against every active predicate is expensive, and it over-locks.

Index-range locking (*next-key* or *gap* locking) approximates the predicate with ranges of an **index**: it locks the matching index entries *and the gaps between them*, so an insert of seat 20 must wait. (Locking only the existing keys 10 and 30 would let seat 20 in — phantoms would remain; covering the gap is the whole point.) It is cheaper than predicate locking because it reuses the index, at the price of sometimes locking a slightly wider range; without a suitable index the engine locks far more (up to the whole table). MySQL InnoDB's next-key locks at Repeatable Read work this way.

Aspect	2PL on rows	Predicate locking	Index-range (next-key) locking
What is locked	Rows actually touched	Every row matching the condition, even future ones	Matching index entries plus gaps between them
Phantoms prevented?	No	Yes	Yes (needs an index on the condition)
Seen in	All lock-based engines	Theory; PostgreSQL SSI's SIREAD locks are a non-blocking cousin	MySQL InnoDB, SQL Server key-range locks

PostgreSQL's locks in practice

PostgreSQL has eight **table-level lock modes** (taken automatically by statements) and four **row-level** modes. `code/04_lock_modes.sql` shows what a transaction holds:

```
BEGIN;
SELECT * FROM accounts WHERE id = 1;                          -- plain read
UPDATE products SET stock = stock - 1 WHERE id = 1;           -- write
SELECT * FROM on_call WHERE doctor = 'dr_rao' FOR SHARE;      -- shared row lock
LOCK TABLE jobs IN SHARE MODE;                                -- explicit table lock
SELECT c.relname, l.mode, l.granted
FROM pg_locks l JOIN pg_class c ON c.oid = l.relation
WHERE l.pid = pg_backend_pid() AND c.relnamespace = 'public'::regnamespace
ORDER BY c.relname, l.mode;
COMMIT;

SELECT pg_try_advisory_lock(42) AS got_it_first_time;
SELECT count(*) AS advisory_locks_held FROM pg_locks WHERE locktype = 'advisory' AND pid = pg_backend_pid();
SELECT pg_advisory_unlock(42) AS released;
```

Output (row echoes trimmed; the three advisory-lock results condensed onto one line):

```
    relname    |       mode       | granted
---------------+------------------+---------
 accounts      | AccessShareLock  | t
 accounts_pkey | AccessShareLock  | t
 jobs          | ShareLock        | t
 on_call       | RowShareLock     | t
 on_call_pkey  | RowShareLock     | t
 products      | RowExclusiveLock | t
 products_pkey | RowExclusiveLock | t
(7 rows)
...
 got_it_first_time → t     advisory_locks_held → 1     released → t
```

These table modes play the role of intention locks: `RowExclusiveLock` (taken by `UPDATE/INSERT/DELETE`) means "I'm changing rows in here"; it doesn't conflict with other writers but does conflict with `SHARE`

and stronger modes:

Common statement	Table lock taken	Blocks
`SELECT`	ACCESS SHARE	only ACCESS EXCLUSIVE (`DROP`, most `ALTER TABLE`)
`SELECT … FOR UPDATE/SHARE`	ROW SHARE	EXCLUSIVE, ACCESS EXCLUSIVE
`INSERT/UPDATE/DELETE`	ROW EXCLUSIVE	SHARE and stronger
`CREATE INDEX`	SHARE	writers
`CREATE INDEX CONCURRENTLY`, `VACUUM`	SHARE UPDATE EXCLUSIVE	other schema changes / vacuums
most `ALTER TABLE`, `DROP`, `TRUNCATE`, `VACUUM FULL`	ACCESS EXCLUSIVE	everything, even `SELECT`

Row-level lock modes, strongest first: `FOR UPDATE` (taken by `DELETE` or an update of key columns), `FOR NO KEY UPDATE` (a normal `UPDATE`), `FOR SHARE` (read, block changes), `FOR KEY SHARE` (foreign-key checks; blocks only key changes/deletes). **Advisory locks** (`pg_advisory_lock(n)`) lock a number that only your application gives meaning to — e.g., "only one nightly job may run".

Common mistakes

- An `ALTER TABLE` on a busy table waits for ACCESS EXCLUSIVE — and **every** later query queues behind it. Use `SET lock_timeout = '2s'` for migrations (scripted schema changes).
- Holding locks during slow work (HTTP calls, user input).
- Relying on the absence of a row without a unique constraint or Serializable (phantoms).

In the interview

> **30-second answer:** "S locks for reads, X for writes; only S is compatible with S. Finer granularity means more concurrency and more overhead; intention locks let the manager check table-versus-row conflicts without scanning rows. 2PL (grow then shrink) guarantees serializability, strict 2PL keeps write locks to commit, and next-key locks stop phantoms."

- *Why does strict 2PL hold X locks until commit?* So nobody reads uncommitted data and cascading aborts can't happen.
- *How does Postgres avoid lock-table explosion for row locks?* Row locks are stored in the tuple header (`xmax` + infomask, a field of status bits); waiters wait on the holder's transaction ID.

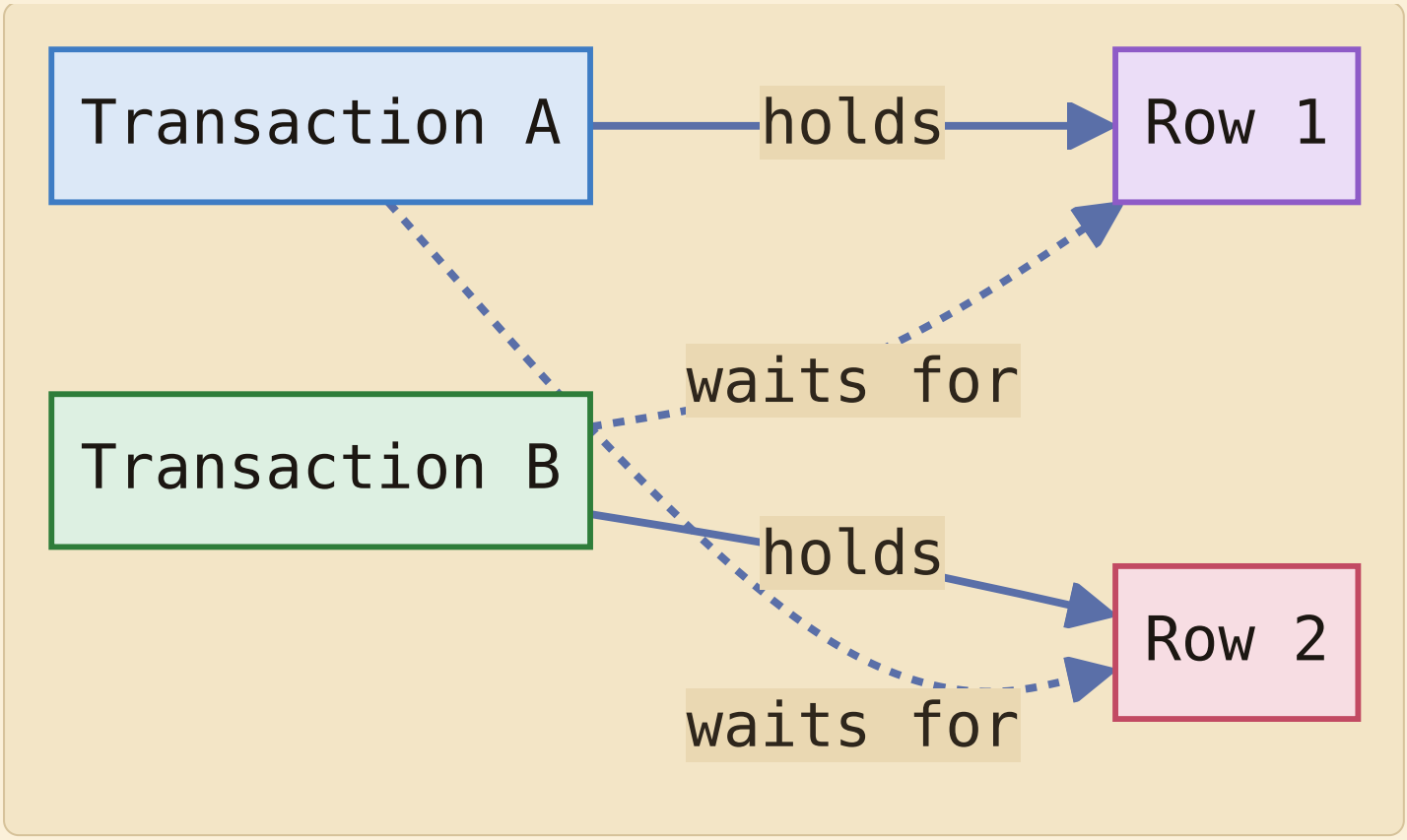
Self-check

1. Can two transactions hold S on the same row? X? → S yes; X no.
2. What does IX on a table announce? → The transaction will take X locks on some rows in it.
3. How does index-range locking stop phantoms? → It locks the gaps between index keys, so matching inserts wait.

13.11 Deadlocks

What it is

A **deadlock** is a cycle of waiting: T1 holds resource 1 and waits for resource 2, while T2 holds 2 and waits for 1. Neither can ever proceed unless something intervenes.



How databases handle it

Strategy	How	Trade-off
Detection	Build a *wait-for graph* (edge T1 → T2 when T1 waits for T2); a cycle is a deadlock; abort one victim	Some work lost; used by PostgreSQL, MySQL, SQL Server, Oracle
Timeout	Abort any transaction that waits too long	Simple, but may abort non-deadlocked waiters or react slowly (`lock_timeout`, InnoDB `innodb_lock_wait_timeout`)
Prevention by ordering	Always lock resources in the same global order (e.g., lower `id` first)	Needs discipline in application code — the most practical fix
Wait-die / wound-wait	By start timestamp: in *wait-die* an older requester waits, a younger one aborts ("dies"); in *wound-wait* an older requester aborts ("wounds") the younger holder, a younger one waits	No cycles; more aborts; some distributed databases
Conservative 2PL	Take all locks up front	Must know everything in advance

Seeing PostgreSQL detect one

Two transfers in opposite directions: A moves 10 from account 1 to 2; B moves 10 from 2 to 1. Each locks its "from" row first.

```
=== Deadlock: PostgreSQL detects it and aborts one transaction ===
  A| SET deadlock_timeout = '10s'  -> SET
  B| SET deadlock_timeout = '200ms'  -> SET
  A| BEGIN  -> BEGIN
  B| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = balance - 10 WHERE id = 1  -> UPDATE 1
  B| UPDATE accounts SET balance = balance - 10 WHERE id = 2  -> UPDATE 1
  A| UPDATE accounts SET balance = balance + 10 WHERE id = 2
     .. A is WAITING (wait_event=transactionid, blocked by B)
  B| UPDATE accounts SET balance = balance + 10 WHERE id = 1  -> ERROR 40P01: deadlock detected
     DETAIL: Process <B> waits for ShareLock on transaction <xid>; blocked by process <A>.
             Process <A> waits for ShareLock on transaction <xid>; blocked by process <B>.
  B| ROLLBACK  -> ROLLBACK
     -> (A resumes) UPDATE 1
  A| COMMIT  -> COMMIT
  M| SELECT id, balance FROM accounts WHERE id IN (1, 2) ORDER BY id  -> (1, 490.00), (2, 310.00)
```

How PostgreSQL does it:

1. A waiting backend (the server process serving one connection) first sleeps for `deadlock_timeout` (default **1 s**) — most waits end by themselves, and the check is costly.
2. Then it inspects the wait-for graph. B's timer fired first (the demo set 200 ms for B, 10 s for A to make the victim predictable); it found the cycle A → B → A and aborted **B** with SQLSTATE **40P01**.
3. B's locks are released, A's update completes and commits; the application retries B.

The fix for this pattern is ordering: always lock the lower account id first (`SELECT … FROM accounts WHERE id IN (1, 2) ORDER BY id FOR UPDATE`), so both transfers queue for row 1 and no cycle can form.

Common mistakes

- Not retrying on 40P01 (or 40001) — both mean "try the whole transaction again".
- Locking rows in input order (an unsorted `for item in cart` loop) — sort by primary key first.

In the interview

30-second answer: "A cycle in the wait-for graph. Databases detect it (Postgres after `deadlock_timeout`, 1 s by default) and abort a victim, which the app retries. Prevent by locking in a consistent order and keeping transactions short; textbook alternatives are wait-die and wound-wait."

Self-check

1. What graph is used for detection? → The wait-for graph between transactions.
2. PostgreSQL's deadlock SQLSTATE? → 40P01.
3. Simplest prevention in application code? → Acquire locks in a fixed global order.

13.12 Row locks for work queues: `FOR UPDATE`, `NOWAIT`, `SKIP LOCKED`

The problem it solves

Many apps keep a **job table** (emails to send, images to resize) served by several workers (processes that pick up and run jobs), each of which must grab jobs **no other worker** holds. With plain `FOR UPDATE` all workers lock the *same* first rows and queue behind each other.

The options

Clause	If the row is locked by someone else…
`FOR UPDATE`	wait until they finish
`FOR UPDATE NOWAIT`	fail immediately with error 55P03
`FOR UPDATE SKIP LOCKED`	silently skip that row and take the next unlocked one

`SKIP LOCKED` exists in PostgreSQL (9.5+), MySQL (8.0+) and Oracle; SQL Server's `READPAST` hint does the same.

Seeing it work

```
=== Job queue: FOR UPDATE SKIP LOCKED (and NOWAIT) ===
  A| BEGIN  -> BEGIN
  A| SELECT id, payload FROM jobs WHERE status = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED  -> (1, email
#1), (2, email #2)
  B| BEGIN  -> BEGIN
  B| SELECT id, payload FROM jobs WHERE status = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED  -> (3, email
#3), (4, email #4)
  B| SELECT id FROM jobs WHERE id = 1 FOR UPDATE NOWAIT  -> ERROR 55P03: could not obtain lock on row in relation
"jobs"
  B| ROLLBACK  -> ROLLBACK
  B| UPDATE jobs SET status = 'running', claimed_by = 'B' WHERE id IN (SELECT id FROM jobs WHERE status = 'queued'
ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED) RETURNING id  -> (3), (4)
  A| UPDATE jobs SET status = 'done', claimed_by = 'A' WHERE id IN (1, 2)  -> UPDATE 2
  A| COMMIT  -> COMMIT
  M| SELECT id, status, claimed_by FROM jobs ORDER BY id  -> (1, done, A), (2, done, A), (3, running, B), (4,
running, B), (5, queued, None), (6, queued, None)
```

Worker A locked jobs 1–2; worker B, running the identical query, got 3–4 **without waiting**, and `NOWAIT` on job 1 failed at once. The usual form **claims and marks in one statement**:

```
UPDATE jobs SET status = 'running', claimed_by = $1
WHERE id IN (SELECT id FROM jobs
             WHERE status = 'queued'
             ORDER BY id
             LIMIT 10
             FOR UPDATE SKIP LOCKED)
RETURNING id, payload;
```

Two designs: (a) keep the transaction open while processing — a crashed worker's rollback releases its jobs automatically; (b) mark `running` and commit at once, plus a `claimed_at` column and a sweeper (a periodic clean-up job) that re-queues stuck jobs. A partial index `CREATE INDEX ON jobs (id) WHERE status = 'queued'` keeps the claim fast (chapter 12).

When not to use it

`SKIP LOCKED` deliberately hides locked rows, so never use it for reports or business-rule checks. For very high throughput, fan-out (delivering one message to many consumers) or replay (re-reading old messages), use a message broker — a dedicated queueing service such as Kafka, RabbitMQ or SQS (System Design Fundamentals); a Postgres queue shines when jobs must be created in the same transaction as the business data.

Real-world examples

Job-queue libraries such as GoodJob (Ruby), Oban (Elixir), graphile-worker (Node) and Procrastinate (Python) use this pattern; ticketing systems use it to hand concurrent buyers different free seats.

13.13 Timestamp-based concurrency control

What it is

Timestamp ordering (TO) gives every transaction a unique, increasing **timestamp** TS(T) at start (clock or counter). The result must equal running the transactions **in timestamp order**, older first; instead of waiting for locks, a transaction that would break the order is **aborted and restarted** with a new timestamp. The result is always serializable.

Key bookkeeping

Symbol	Kept for	Meaning
TS(T)	each transaction	its start timestamp — its place in the serial order
RTS(X)	each data item X	the largest timestamp of any transaction that has **read** X
WTS(X)	each data item X	the largest timestamp of any transaction that has **written** X

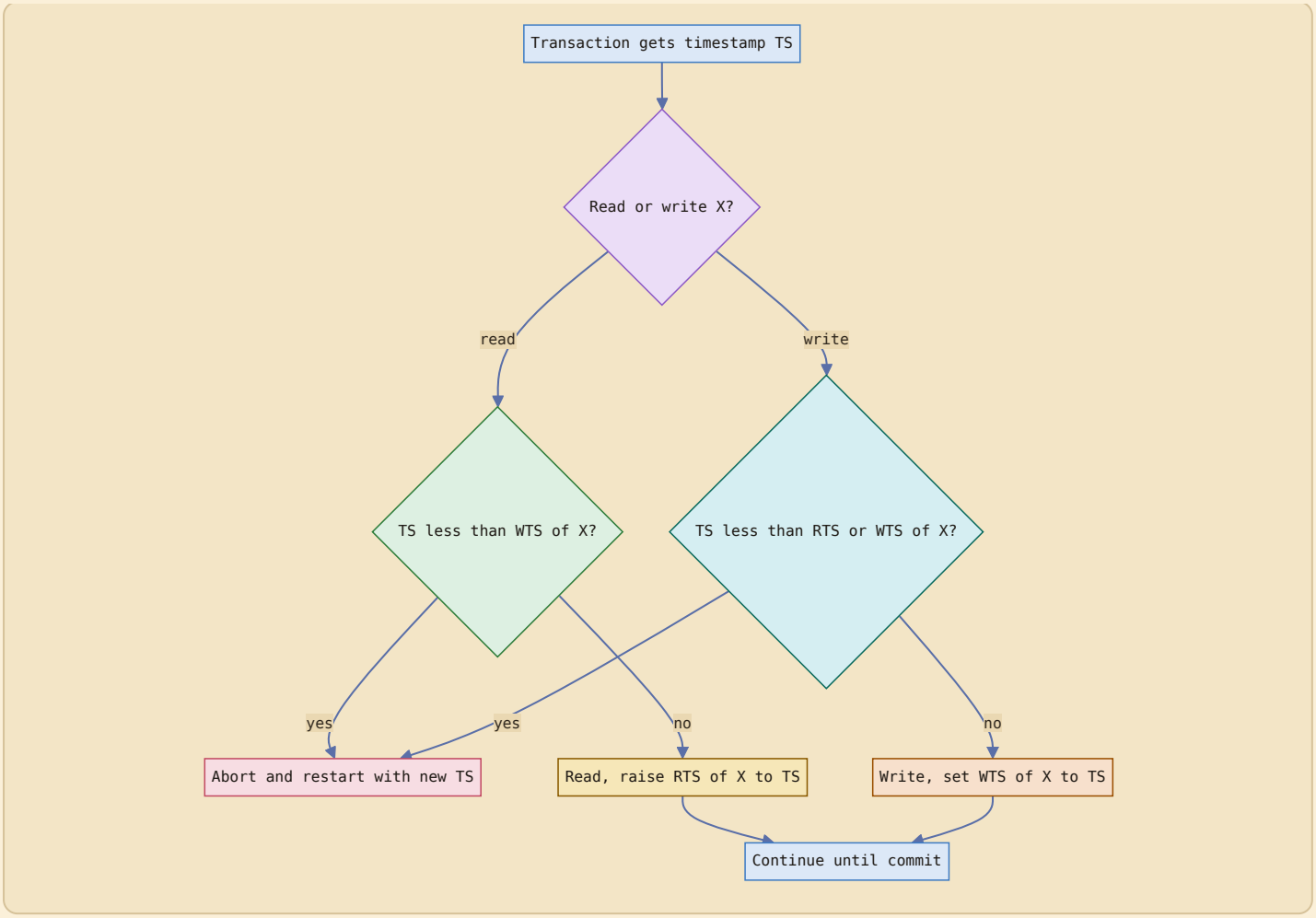
How it works — the rules

When T wants to read X:

1. If TS(T) < WTS(X): a *younger* transaction already overwrote X — T would read a value "from its future". **Abort T** (restart with a new timestamp).
2. Otherwise read X and set RTS(X) = max(RTS(X), TS(T)).

When T wants to write X:

1. If TS(T) < RTS(X): a younger transaction already read the old value. **Abort T.**
2. If TS(T) < WTS(X): a younger transaction already wrote X, so T's write is outdated. **Abort T** — or, under the **Thomas write rule**, **skip** the obsolete write (timestamp order would have overwritten it anyway).
3. Otherwise write X and set WTS(X) = TS(T).



Worked example — booking the last seats

Item **S** = seats left for a show (initially RTS = 0, WTS = 0). Priya's booking transaction starts first and gets TS = 5; Omar's starts a moment later with TS = 7.

Step	Operation	Check	Result	RTS(S)	WTS(S)
1	T5 reads S	5 ≥ WTS 0	allowed	5	0
2	T7 reads S	7 ≥ WTS 0	allowed	7	0
3	T7 writes S (books a seat)	7 ≥ RTS 7 and ≥ WTS 0	allowed	7	7
4	T5 writes S (books a seat)	5 < RTS 7	**T5 aborted**	7	7
5	T5 restarts as T9, reads S	9 ≥ WTS 7	allowed, sees Omar's booking	9	7

Priya's first attempt relied on a count that a "later" transaction had already read and changed, so it was rolled back instead of double-selling; the retry sees the current count.

Trade-offs

Advantages	Disadvantages
No deadlocks — nobody waits for a lock	**Many rollbacks** under contention; work is thrown away
Guarantees **serializability** by construction	**Starvation** — a long transaction may be restarted again and again because younger ones keep touching its data
Simple conflict rule: compare two numbers	**Overhead** — every read updates RTS (reads become metadata writes — writes of bookkeeping data); extra care is needed to avoid reading uncommitted values

Where it is used

Pure TO is mostly a textbook protocol. Its descendant **multiversion timestamp ordering** (versions tagged with timestamps; each read picks the right one) underlies many MVCC systems, and distributed databases rely on timestamps: Google Spanner uses TrueTime (GPS- and atomic-clock-backed time with a known error bound), CockroachDB hybrid logical clocks (physical time combined with a counter) plus MVCC.

In the interview

30-second answer: "Transactions must behave as if run in start-timestamp order. Each item keeps its largest read and write timestamps; a read is rejected if a younger transaction already wrote the item, a write if a younger one already read or wrote it, and the loser restarts with a new timestamp. No deadlocks, but many aborts and possible starvation; the Thomas write rule skips obsolete writes."

Self-check

1. T with TS 4 wants to write X where RTS(X) = 6. → Abort T (a younger transaction read X).
2. What does the Thomas write rule change? → An outdated write (TS < WTS, but ≥ RTS) is ignored instead of causing an abort.
3. Why can't TO deadlock? → Transactions never wait; conflicts cause aborts.

13.14 Multiversion concurrency control (MVCC)

What it is

MVCC keeps **several versions** of each row. A write creates a **new version** and marks the old one as replaced; each transaction reads from a **snapshot** — the versions committed as of a certain moment. So **readers and writers never block each other** (writers still block writers on the same row), and every transaction sees a consistent picture.

Analogy: a shared document with full version history. Editing creates a new revision; someone reading "as of 10:00" keeps reading that revision undisturbed.

How it works

1. **Versions carry metadata:** the transaction that **created** it and the one that **deleted/replaced** it (empty if current).
2. **Snapshots:** when a transaction (Repeatable Read) or statement (Read Committed) starts, the database notes which transactions had committed.
3. **Visibility:** a version is visible if its creator committed before the snapshot and its deleter is empty, aborted, or not committed as of the snapshot.
4. **Writes** create new versions; concurrent writers of one row are serialized by a row lock (§13.7), and at Repeatable Read/Serializable a concurrent committed change raises a serialization error.
5. **Cleanup:** versions no snapshot can see are garbage (PostgreSQL's `VACUUM` cleanup command, InnoDB's purge thread — its background cleaner).

Worked example — versions of an account

Account A starts at 500 (created by transaction 1); account B at 300. Transaction 2 adds 200 to A; later, transaction 3 subtracts 100 from A.

After	Versions of account A (created_by, deleted_by, balance)	Account B
txn 1	(1, –, **500**)	(1, –, 300)
txn 2	(1, **2**, 500) · (2, –, **700**)	unchanged
txn 3	(1, 2, 500) · (2, **3**, 700) · (3, –, **600**)	unchanged

A reader whose snapshot predates txn 2's commit still sees 500; one taken between txn 2 and txn 3 sees 700; new readers see 600 — simultaneously, without locks.

MVCC in PostgreSQL: `xmin` and `xmax` for real

PostgreSQL stores the "created by" and "deleted by" fields in every row version as hidden system columns: `xmin` (inserting transaction ID) and `xmax` (deleting/locking transaction ID, 0 if none). `ctid` is the version's physical address (page, slot).

One session — `code/03_mvcc_xmin_xmax.sql` repeats the worked example (500 → +200 → −100) and then uses the `pageinspect` extension to look *inside* the table's first 8 KB page:

```
CREATE EXTENSION IF NOT EXISTS pageinspect;
CREATE TABLE wallet (id int PRIMARY KEY, balance int);
INSERT INTO wallet VALUES (1, 500), (2, 300);
SELECT ctid, xmin, xmax, id, balance FROM wallet ORDER BY id;

UPDATE wallet SET balance = balance + 200 WHERE id = 1;
UPDATE wallet SET balance = balance - 100 WHERE id = 1;
SELECT ctid, xmin, xmax, id, balance FROM wallet ORDER BY id;

SELECT lp AS slot, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('wallet', 0)) ORDER BY lp;

BEGIN;
UPDATE wallet SET balance = 0 WHERE id = 2;
SELECT txid_current() AS my_txid;
ROLLBACK;
SELECT ctid, xmin, xmax, id, balance FROM wallet WHERE id = 2;

VACUUM wallet;
SELECT lp AS slot, lp_flags, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('wallet', 0)) ORDER BY lp;
```

Output (transaction IDs differ on every run; command tags trimmed):

```
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,1) |  802 |    0 |  1 |     500
 (0,2) |  802 |    0 |  2 |     300
(2 rows)

UPDATE 1
UPDATE 1
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,4) |  804 |    0 |  1 |     600
 (0,2) |  802 |    0 |  2 |     300
(2 rows)

 slot | t_xmin | t_xmax | t_ctid
------+--------+--------+--------
    1 |    802 |    803 | (0,3)
    2 |    802 |      0 | (0,2)
    3 |    803 |    804 | (0,4)
    4 |    804 |      0 | (0,4)
(4 rows)

BEGIN
UPDATE 1
 my_txid
---------
     805
(1 row)

ROLLBACK
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,2) |  802 |  805 |  2 |     300
(1 row)

VACUUM
 slot | lp_flags | t_xmin | t_xmax | t_ctid
------+----------+--------+--------+--------
    1 |        2 |        |        |
    2 |        1 |    802 |    805 | (0,5)
    3 |        0 |        |        |
    4 |        1 |    804 |      0 | (0,4)
    5 |        0 |        |        |
(5 rows)
```

Reading it:

1. Transaction 802 inserted both rows (`xmax = 0`: not replaced).
2. After the updates, `SELECT` shows only the current version of row 1, at a **new address** `(0,4)` with `xmin = 804`: an update is "insert a new version + mark the old one".
3. The raw page still holds **all three versions** of row 1 — the worked example exactly: slot 1 (500, created by 802, replaced by 803), slot 3 (700, 803 → 804), slot 4 (600, current). Each old version's `t_ctid` points to its successor.
4. The rolled-back update left `xmax = 805` on row 2, yet the row is visible with 300: transaction 805 is marked **aborted** in the commit log (`pg_xact`), so its `xmax` is ignored. PostgreSQL rolls back instantly by marking, not undoing.
5. `VACUUM` removed the dead versions: slots 3 and 5 (the aborted version) are unused (`lp_flags = 0`) and slot 1 became a **redirect** (`lp_flags = 2`) to slot 4 — a *HOT* (heap-only tuple) chain, so the index can keep pointing at slot 1.

Two sessions — the `mvcc` scenario of the driver shows what *another* transaction sees while an update is uncommitted:

```
=== MVCC: xmin / xmax seen from two sessions ===
  A| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1697, 0, 500.00)
  A| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = 450 WHERE id = 1  -> UPDATE 1
  A| SELECT txid_current()  -> 1698
  A| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1698, 0, 450.00)
  B| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1697, 1698, 500.00)
  A| COMMIT  -> COMMIT
  B| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1698, 0, 450.00)
```

While A's transaction 1698 is open, **two versions** exist: A sees its new one (450); B sees the **old** one (500) with `xmax = 1698` — "being replaced by an uncommitted transaction, so still visible to me". After A commits, B's next statement sees 450. No one waited.

Trade-offs

Advantages	Costs
Readers and writers don't block each other	Extra storage for old versions; tables and indexes can **bloat** (grow with wasted space from dead versions)
Consistent snapshots for reports and backups (`pg_dump` uses one)	Needs garbage collection (autovacuum — the background process that runs `VACUUM` automatically — must keep up)
Cheap rollback in PostgreSQL (mark aborted)	Long-running transactions keep old versions alive for everyone
Basis for snapshot isolation and SSI	Snapshot isolation alone still allows write skew (§13.8)

Two designs. PostgreSQL keeps every version **in the table** (vacuum later). MySQL InnoDB and Oracle update **in place** and rebuild older versions from an **undo log** — less bloat, more work for long readers ("snapshot too old" in Oracle).

Common mistakes

- Leaving a session "idle in transaction" for hours: vacuum can't remove versions newer than its snapshot, so tables bloat. Set `idle_in_transaction_session_timeout`.
- Disabling autovacuum: besides bloat, PostgreSQL's 32-bit transaction IDs must be "frozen" (marked as old enough to be visible to everyone) by vacuum before they wrap around — run out and restart from zero (~2 billion transactions) — or the server stops accepting writes.
- Thinking MVCC means "no locks at all" — writes to the same row still lock and wait.

Real-world examples

Shoppers browse while prices change, customers view balances during batch posting, and `pg_dump` takes a coherent backup while the database keeps running — each of these works because readers get snapshots.

In the interview

30-second answer: "Keep multiple row versions tagged with the writing transaction; readers pick the version visible to their snapshot, so readers and writers don't block each other. In Postgres an update inserts a tuple and sets xmax on the old one; VACUUM removes dead tuples. Costs: bloat, garbage collection, and long transactions hurt."

- *Non-zero `xmax` on a visible row?* Its deleter/updater hasn't committed or aborted — or it's only a row lock.

Self-check

1. What do `xmin` and `xmax` record? → The creating transaction and the deleting/replacing (or locking) transaction.
2. Why did the updated row get a new `ctid`? → An UPDATE writes a new tuple version elsewhere in the page.

3. What makes a rolled-back change invisible in PostgreSQL? → The transaction is marked aborted in the commit log; its versions are ignored.

13.15 Putting it together

A practical checklist

1. Keep transactions **short** — never open across user think time or slow network calls.
2. Prefer **atomic statements** (`SET n = n - 1 WHERE n > 0`) over read-then-write in code; otherwise `FOR UPDATE`, or a **version column** for long gaps.
3. Declare every invariant you can as a **constraint** (unique, check, foreign key, exclusion); for cross-row rules use **Serializable + retry** or lock the rows the decision depends on.
4. **Retry** the whole transaction on SQLSTATE 40001 and 40P01, with backoff (waiting a little longer before each new attempt) and a limit.
5. Lock rows in a **consistent order**; set `lock_timeout` for migrations and `idle_in_transaction_session_timeout` to protect vacuum.
6. For consistent multi-query reports use `REPEATABLE READ READ ONLY`.

Deeper theory: DDIA ch. 7 notes. Thread-level locking in application code: LLD (low-level design) handbook concurrency concepts and concurrency problems. Distributed transactions and replication: chapter 15; transactions from Python/Java: chapter 14. A one-page summary of isolation levels, anomalies and lock choices is in chapter 20, §20.12.

Chapter TL;DR

- A **transaction** is all-or-nothing; write it with `BEGIN … COMMIT/ROLLBACK`, using savepoints for partial undo.
- **ACID:** atomicity (logs/shadow copies), consistency (constraints + code), isolation (locks/MVCC/SSI), durability (WAL + fsync + checkpoints + replication).
- **Anomalies:** dirty read, dirty write, non-repeatable read/read skew, phantom, lost update, write skew. PostgreSQL never allows dirty reads/writes; Read Committed allows the next four; Repeatable Read (snapshot isolation) stops all but write skew; Serializable (SSI) stops everything by aborting with 40001.
- **Lost updates:** prefer atomic SQL updates; otherwise `FOR UPDATE`, Repeatable Read + retry, or a version column (CAS) for long think times.
- **Concurrency control:** pessimistic (S/X/intention locks, 2PL, predicate/next-key locks) vs optimistic (validate at commit) vs timestamp ordering vs **MVCC** (PostgreSQL's `xmin`/`xmax`, cleaned by VACUUM).
- **Deadlocks** are detected (40P01) and prevented by consistent lock ordering; **retry** 40001/40P01.
- `FOR UPDATE SKIP LOCKED` turns a table into a safe multi-worker job queue.

Next: Chapter 14 — Advanced SQL and using SQL from code, which runs these transactions from Python and Java. For quick revision, see the transactions sheet in Chapter 20.

Chapter 14 — Advanced SQL and Using SQL from Code

So far every query in this book has been typed into `psql` (PostgreSQL's command-line client) by a person. Real systems are different: the database runs **code of its own** (functions, procedures, triggers), stores **semi-structured data** — data whose shape varies from row to row (JSON, a text format for nested data; arrays; searchable text) — protects data **per user and per row**, and — most of all — is driven by **application programs** written in Python, Java and other languages. This chapter covers those "last mile" skills.

What you'll learn

- Writing functions and stored procedures in SQL and PL/pgSQL, and when logic belongs in the database.
- Triggers: automatic `updated_at` columns, validation rules, a generic audit log, statement-level triggers.
- JSON/JSONB and arrays: storing, querying, updating and indexing them.
- Full-text search with `tsvector` / `tsquery`, ranking and highlighting.
- Pivoting rows into columns (and back) with `FILTER`, `CASE`, `crosstab` and `LATERAL`.
- Security: roles, `GRANT` / `REVOKE`, column privileges and row-level security (RLS).
- SQL injection — seeing it succeed, then fixing it with parameters.
- Calling the database from Python (`psycopg`, `sqlite3`) and Java (JDBC): parameters, transactions, batches, streaming.
- Connection pooling, and the trade-offs between ORMs and hand-written SQL.

How to run the examples. Everything is in `code/` and runs on the book's private PostgreSQL 14 server (port 5544). The shared sample database from Chapter 1 — a small office-gear company with `departments`, `employees`, `customers`, `products`, `orders` and `order_items` — is loaded first from `code/00_schema.sql`; each numbered file then runs in a fresh database.

```
bash tools_run_sql.sh sql '14-*'                                    # all SQL files of this chapter
.venv-books/bin/python sql/chapters/14-advanced-sql-and-using-sql-from-code/code/psycopg_demo.py
python3 sql/chapters/14-advanced-sql-and-using-sql-from-code/code/sqlite3_demo.py
java --enable-native-access=ALL-UNNAMED -cp '.java-books/lib/*' \
     sql/chapters/14-advanced-sql-and-using-sql-from-code/code/JdbcDemo.java
```

14.1 User-defined functions

What it is

A **user-defined function** (UDF) is a named piece of logic stored *inside* the database that takes arguments and returns a value (or a set of rows). You call it from SQL exactly like a built-in function such as `upper()` or `round()`. Think of it as a spreadsheet formula you give a name, so everyone uses the same one.

PostgreSQL lets you write the body in several **procedural languages**:

Language	What the body looks like	Use it for
`sql`	One or more plain SQL statements; the last one's result is returned	Short calculations and reusable queries; the planner (the part of the database that decides how to run a query) can often *inline* them — paste their body into the calling query
`plpgsql`	PL/pgSQL: SQL plus variables, `IF`, loops, error handling	Logic with branches, loops, or several steps
`plpython3u`, `plperl`, …	Other languages (extensions, often "untrusted" = superuser — an all-powerful admin role — only)	Rare; needs the extension installed

Why it matters

- **One definition of a rule.** "Line total = quantity × price × (1 − discount)" written once, used by every report and every application.
- **Fewer round trips** (a request to the server and its reply). A multi-step operation runs next to the data instead of sending many statements over the network.
- **Reusable query building blocks.** A function that returns a table can be joined like a table.

How it works

1. `CREATE FUNCTION name(args) RETURNS type LANGUAGE … AS $$ body $$` stores the function in the catalog (PostgreSQL's internal tables describing every object; here `pg_proc`).
2. `$$ … $$` is **dollar quoting**: a way to write a string literal without escaping (marking as "literal, not special") the single quotes inside the body.
3. Each argument can have a `DEFAULT`; callers may pass arguments by position or by name (`p_limit => 1`).
4. A **volatility** label tells the planner how the function behaves — it affects optimisation and whether the function may be used in an index:

Label	Promise	Example
`IMMUTABLE`	Same inputs → same result forever; reads no tables	`line_total(2, 10, 0)`
`STABLE`	Same result within one statement; may read tables	"tier of customer 3"
`VOLATILE` (default)	Anything may change; may modify data	`random()`, a function that inserts rows

Only `IMMUTABLE` functions can be used in an index expression. Lying (marking a table-reading function `IMMUTABLE`) gives wrong, cached-looking results.

Example — SQL and PL/pgSQL functions

From `code/01_functions.sql`:

```sql
-- (a) A plain SQL function, SQL-standard body (PostgreSQL 14+)
CREATE FUNCTION line_total(qty integer, price numeric, disc numeric DEFAULT 0)
RETURNS numeric
LANGUAGE sql IMMUTABLE
RETURN round(qty * price * (1 - disc), 2);

-- (b) PL/pgSQL: variables, IF, SELECT ... INTO, RAISE
CREATE FUNCTION customer_tier(p_customer_id integer)
RETURNS text
LANGUAGE plpgsql STABLE
AS $$
DECLARE
    v_spent numeric;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM customers WHERE customer_id = p_customer_id) THEN
        RAISE EXCEPTION 'customer % does not exist', p_customer_id
              USING ERRCODE = 'no_data_found';
    END IF;

    SELECT coalesce(sum(line_total(oi.quantity, oi.unit_price, oi.discount)), 0)
    INTO   v_spent
    FROM   orders o JOIN order_items oi USING (order_id)
    WHERE  o.customer_id = p_customer_id AND o.status <> 'cancelled';

    RETURN CASE WHEN v_spent >= 1500 THEN 'gold'
                WHEN v_spent >= 300  THEN 'silver'
                ELSE 'bronze' END;
END;
$$;

SELECT customer_id, first_name, customer_tier(customer_id) AS tier
FROM   customers ORDER BY customer_id LIMIT 5;
```

Output:

```
 customer_id | first_name |  tier
-------------+------------+--------
           1 | Anita      | gold
           2 | Ben        | gold
           3 | Chen       | bronze
           4 | Divya      | bronze
           5 | Ethan      | silver
```

What each part does: `DECLARE` lists local variables; `SELECT … INTO v_spent` stores a query result in a variable; `RAISE EXCEPTION … USING ERRCODE` aborts with a chosen SQLSTATE code (the standard five-character error code); `%` in the message is replaced by the next argument. `RAISE NOTICE` (used below) only prints a message.

Catching errors. A `BEGIN … EXCEPTION WHEN no_data_found THEN … END` block catches errors raised inside it (`SQLERRM` holds the message): the file's `safe_tier(999)` returns `unknown (customer 999 does not exist)` instead of failing. An `EXCEPTION` block silently creates a **savepoint** (a marker inside a transaction that you can roll back to) on entry, so it costs a little — don't wrap every statement in one.

Set-returning functions. `RETURNS TABLE (...)` makes a function usable in `FROM`, like a parameterised view:

```sql
CREATE FUNCTION top_products(p_limit integer DEFAULT 3)
RETURNS TABLE (product text, units bigint, revenue numeric)
LANGUAGE sql STABLE AS $$
    SELECT p.name, sum(oi.quantity), sum(line_total(oi.quantity, oi.unit_price, oi.discount))
    FROM   order_items oi JOIN orders o USING (order_id) JOIN products p USING (product_id)
    WHERE  o.status <> 'cancelled'
    GROUP  BY p.name ORDER BY 3 DESC LIMIT p_limit
$$;
SELECT * FROM top_products(p_limit => 1);  -- named argument; top_products() uses the default 3
```

```
    product    | units | revenue
---------------+-------+---------
 Laptop Pro 14 |     3 | 3782.05
```

In PL/pgSQL, a set-returning function can instead loop (`FOR r IN SELECT … LOOP`) and emit one row at a time with `RETURN NEXT` — the file's `salary_bands(50000)` does this to count employees per 50,000 salary band.

One-off code without a function: `DO $$ … $$;` runs an anonymous PL/pgSQL block once. The file ends with one (it prints `NOTICE: pending orders: 2`) and with a look at the catalog, where `prokind` is `f` for function and `provolatile` is `i`/`s`/`v`.

Dialect differences

	PostgreSQL	MySQL	SQL Server	SQLite
Stored functions	`CREATE FUNCTION`, many languages	`CREATE FUNCTION` (SQL/PSM style)	`CREATE FUNCTION` (T-SQL, SQL Server's dialect), scalar and table-valued	None in SQL; register functions from the host language (the language of the program using SQLite) (e.g. Python's `conn.create_function`)
Replace	`CREATE OR REPLACE FUNCTION`	`DROP` then `CREATE`	`CREATE OR ALTER`	—
Error raising	`RAISE EXCEPTION`	`SIGNAL SQLSTATE`	`THROW`	`RAISE()` only inside triggers

Common mistakes

- **Wrong volatility**: a function that reads tables marked `IMMUTABLE` can return stale results; a pure calculation left `VOLATILE` blocks optimisations.
- **Hiding row-by-row loops in functions.** Calling a query-running function once per row of a million-row table runs a million queries; a set-based (one statement over all rows at once) `JOIN` is usually far faster.
- **Name clashes** between PL/pgSQL variables and column names (`WHERE customer_id = customer_id`). Prefix parameters (`p_`) and variables (`v_`).

In the interview

30-second answer: "A UDF packages logic in the database so it's reused and runs next to the data. In Postgres I'd use a `LANGUAGE sql` function for simple expressions — it can be inlined — and PL/pgSQL for branching or loops. I'd set the right volatility, keep heavy business rules in the application unless they guard data integrity, and prefer set-based SQL over per-row function calls."

- *Function vs view?* → A view has no parameters; a set-returning function does. For a fixed query, a view is simpler and fully optimisable.
- *Why can't I index `lower(email)` with my own function?* → You can, but only if the function is `IMMUTABLE`.

Self-check

1. What does `$$` do? → It quotes the function body without needing to double every `'` inside it.
2. Can a `STABLE` function update a table? → It shouldn't; it promises not to change the database. Use `VOLATILE`.
3. How do you return several rows? → `RETURNS TABLE(...)` or `RETURNS SETOF type`, with a query body or `RETURN NEXT` / `RETURN QUERY`.

TL;DR

Functions are named, reusable logic stored in the database: `sql` for simple, `plpgsql` for procedural. Label volatility honestly, prefix variable names, and don't replace set-based SQL with row-by-row calls.

14.2 Stored procedures

What it is

A **stored procedure** is like a function that *does* something rather than *computes* something. You run it with `CALL`, it returns no value (except through `INOUT`/`OUT` parameters), and — the key difference in PostgreSQL (procedures arrived in version 11) — it may **commit or roll back transactions inside its body**.

	Function	Procedure
Invoked by	`SELECT f(...)`, inside any expression	`CALL p(...)` only
Returns	A value or a set of rows	Nothing; results via `INOUT` / `OUT` parameters
`COMMIT` / `ROLLBACK` inside	Not allowed — always runs inside the caller's transaction	Allowed when called outside an explicit transaction block
Typical use	Calculations, reusable queries, trigger bodies	Batch jobs, maintenance, multi-step operations

Why it matters

Long batch jobs (archive a year of orders, back-fill a column) should commit in **chunks**: a single giant transaction holds locks for a long time, keeps old row versions alive (blocking vacuum, PostgreSQL's clean-up of dead row versions — see Chapter 13), and loses all its work if it fails at the end.

Example — archiving in committed batches

From `code/02_procedures.sql`:

```
CREATE TABLE orders_archive (LIKE orders INCLUDING ALL);

CREATE PROCEDURE archive_delivered(p_before date, p_batch int, INOUT moved int DEFAULT 0)
LANGUAGE plpgsql AS $$
DECLARE n int;
BEGIN
    LOOP
        WITH picked AS (
            SELECT order_id FROM orders
            WHERE  status = 'delivered' AND placed_at < p_before
              AND  order_id NOT IN (SELECT order_id FROM orders_archive)
            ORDER  BY order_id LIMIT p_batch
        )
        INSERT INTO orders_archive
        SELECT o.* FROM orders o JOIN picked USING (order_id);
        GET DIAGNOSTICS n = ROW_COUNT;     -- rows affected by the last statement
        EXIT WHEN n = 0;
        moved := moved + n;
        RAISE NOTICE 'batch of % committed', n;
        COMMIT;
    END LOOP;
END;
$$;

CALL archive_delivered('2025-04-01', 2);
```

Output:

```
NOTICE:  batch of 2 committed
NOTICE:  batch of 2 committed
NOTICE:  batch of 1 committed
 moved
-------
     5
```

The five delivered orders placed before April 2025 (101, 102, 103, 104, 106) were copied in three separately committed batches, and the `INOUT` parameter came back as a one-row result.

The catch. If the caller already opened a transaction (`BEGIN; CALL …`), the procedure is inside it and cannot end it:

```
BEGIN;
DO $$ DECLARE m int := 0; BEGIN
    CALL archive_delivered('2025-12-31', 2, m);
EXCEPTION WHEN invalid_transaction_termination THEN
    RAISE NOTICE 'caught: %', SQLERRM;
END $$;
ROLLBACK;
```

```
NOTICE:  batch of 2 committed
NOTICE:  caught: invalid transaction termination
```

The first `NOTICE` prints before the `COMMIT` fails — the message is misleading, which is why the file rolls everything back. Many drivers (the libraries programs use to talk to a database) open a transaction automatically (see 14.10), so a procedure that commits must be called in **auto-commit** mode (each statement committed on its own).

Dialect differences

	Syntax	Transaction control inside
PostgreSQL 11+	`CREATE PROCEDURE … ; CALL p()`	Yes, if not called inside a transaction block
MySQL	`CREATE PROCEDURE` (the `mysql` client needs `DELIMITER //` to change the statement terminator — the character that ends a statement, normally `;`); `CALL p()`	Yes
SQL Server	`CREATE PROCEDURE`; `EXEC p`	Yes; can also return result sets directly
SQLite	No stored procedures	—

When not to use it

Stored procedures tie logic to one database product, are harder to unit-test (check with small automated tests), version and deploy (release to servers) than application code, and scale only as far as the database server. A common, balanced rule: **integrity rules and data-heavy batch work in the database; business workflows in the application.**

Self-check

1. How do you get a value back from a procedure? → An `INOUT` (or, from PG 14, `OUT`) parameter.
2. Why does `COMMIT` in a procedure sometimes fail? → The caller is already inside a transaction block.
3. Why commit in batches? → Shorter locks, less bloat (wasted space from dead row versions), and completed work survives a later failure.

TL;DR

Procedures are `CALL`ed, return through `INOUT` parameters, and can `COMMIT` inside — ideal for batch jobs, but only when called in auto-commit mode.

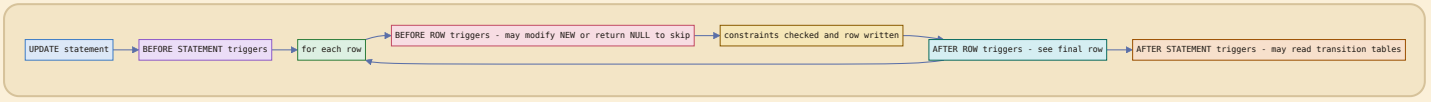
14.3 Triggers

What it is

A **trigger** tells the database: "whenever *this event* happens on *this table*, run *this function* automatically". Events are `INSERT`, `UPDATE`, `DELETE` and `TRUNCATE`. It's like a motion-sensor light: nobody presses a switch, the event itself does.

Choice	Options	Meaning
Timing	`BEFORE` / `AFTER` / `INSTEAD OF` (views only)	Before the row is written (can change or skip it), after it's written (can react), or replacing the write on a view
Level	`FOR EACH ROW` / `FOR EACH STATEMENT`	Once per affected row, or once per SQL statement (even if 0 rows)
Condition	`WHEN (…)`	Fire only if the condition on `OLD` / `NEW` is true
Column filter	`UPDATE OF col`	Fire only when those columns are named in the `UPDATE`

Inside a row trigger, `NEW` is the new version of the row (INSERT/UPDATE), `OLD` the previous one (UPDATE/DELETE); `TG_OP`, `TG_TABLE_NAME` and friends describe the event. A trigger function is declared `RETURNS trigger`.



Why it matters

Triggers enforce behaviour **no matter which application, script or person** changes the data: an `updated_at` stamp, an audit trail (a record of who changed what, when), a denormalised counter (a stored copy of a count, kept for fast reads) kept in sync, a rule a `CHECK` constraint cannot express because it needs another table.

Example 1 — `updated_at`

From `code/03_triggers.sql`:

```sql
ALTER TABLE products ADD COLUMN updated_at timestamptz NOT NULL DEFAULT '2025-01-01 00:00+00';

CREATE FUNCTION set_updated_at() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at := now();      -- change the row before it is written
    RETURN NEW;                   -- BEFORE row trigger: return the row to store (NULL = skip it)
END;
$$;

CREATE TRIGGER products_updated_at
BEFORE UPDATE ON products
FOR EACH ROW
WHEN (OLD.* IS DISTINCT FROM NEW.*)          -- skip no-op updates
EXECUTE FUNCTION set_updated_at();

UPDATE products SET unit_price = 22.99 WHERE product_id = 2;
UPDATE products SET unit_price = 35.00 WHERE product_id = 8;   -- same value as before
SELECT product_id, unit_price, updated_at > '2025-01-02' AS stamped_now
FROM   products WHERE product_id IN (2, 8) ORDER BY product_id;
```

```
 product_id | unit_price | stamped_now
------------+------------+-------------
          2 |      22.99 | t
          8 |      35.00 | f
```

`EXECUTE FUNCTION` is the PostgreSQL 11+ spelling (older versions write `EXECUTE PROCEDURE`, still accepted).

Example 2 — a validation rule that needs another table

```sql
CREATE FUNCTION forbid_discontinued() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF (SELECT discontinued FROM products WHERE product_id = NEW.product_id) THEN
        RAISE EXCEPTION 'product % is discontinued', NEW.product_id;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER order_items_check_product
BEFORE INSERT OR UPDATE OF product_id ON order_items
FOR EACH ROW EXECUTE FUNCTION forbid_discontinued();
-- inserting the discontinued Desk Lamp (id 8):
-- NOTICE:  rejected: product 8 is discontinued
```

Example 3 — a generic audit log

One function works for any table because `to_jsonb(OLD)` / `to_jsonb(NEW)` turn a whole row into JSON:

```sql
CREATE TABLE audit_log (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    table_name  text        NOT NULL,
    op          text        NOT NULL,
    changed_by  text        NOT NULL DEFAULT current_user,
    changed_at  timestamptz NOT NULL DEFAULT now(),
    old_row     jsonb,
    new_row     jsonb
);

CREATE FUNCTION audit_row() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO audit_log (table_name, op, old_row, new_row)
    VALUES (TG_TABLE_NAME, TG_OP,
            CASE WHEN TG_OP IN ('UPDATE','DELETE') THEN to_jsonb(OLD) END,
            CASE WHEN TG_OP IN ('INSERT','UPDATE') THEN to_jsonb(NEW) END);
    RETURN NULL;                 -- the return value of an AFTER trigger is ignored
END;
$$;

CREATE TRIGGER customers_audit
AFTER INSERT OR UPDATE OR DELETE ON customers
FOR EACH ROW EXECUTE FUNCTION audit_row();

INSERT INTO customers VALUES (11, 'Jia', 'Wong', 'jia.wong@mail.example', 'Singapore', 'Singapore', '2025-06-20');
UPDATE customers SET city = 'Mumbai' WHERE customer_id = 8;
DELETE FROM customers WHERE customer_id = 11;

SELECT id, table_name, op, changed_by, old_row ->> 'city' AS old_city, new_row ->> 'city' AS new_city
FROM   audit_log ORDER BY id;
```

```
 id | table_name |   op   | changed_by | old_city  | new_city
----+------------+--------+------------+-----------+-----------
  1 | customers  | INSERT | postgres   |           | Singapore
  2 | customers  | UPDATE | postgres   | mumbai    | Mumbai
  3 | customers  | DELETE | postgres   | Singapore |
```

Which columns changed? Compare the two JSON objects key by key:

```sql
SELECT n.key, o.value AS before, n.value AS after
FROM   audit_log a, jsonb_each(a.new_row) n
JOIN   jsonb_each(a.old_row) o USING (key)
WHERE  a.op = 'UPDATE' AND n.value IS DISTINCT FROM o.value;
```

```
 key  |  before  |  after
------+----------+----------
 city | "mumbai" | "Mumbai"
```

Because the audit insert runs **in the same transaction** as the change, a rolled-back change leaves no audit row — the log never lies.

Example 4 — statement-level trigger with transition tables

A row trigger on a 1-million-row `UPDATE` fires a million times. A **statement-level** trigger fires once, and (PostgreSQL 10+) can see all changed rows at once through **transition tables**:

```sql
CREATE TRIGGER products_bulk_price
AFTER UPDATE ON products
REFERENCING OLD TABLE AS old_rows NEW TABLE AS new_rows
FOR EACH STATEMENT EXECUTE FUNCTION log_bulk_price_change();
-- the function reads new_rows / old_rows like tables (count, avg of price differences)

UPDATE products SET unit_price = unit_price * 1.10 WHERE category = 'Furniture';
-- NOTICE:  3 rows changed price, average change 29.47
```

Trade-offs and dialect differences

	PostgreSQL	MySQL	SQL Server	SQLite
Row-level	Yes	Yes (only kind)	No — statement-level only, with `inserted` / `deleted` pseudo-tables	Yes (only kind)
Statement-level	Yes (+ transition tables)	No	Yes (the only kind)	No
`INSTEAD OF`	On views	No	On tables and views	On views
Body	A separate trigger function	Inline SQL body	Inline T-SQL	Inline SQL body

When not to use it

- For **business workflows** (send an email, call a payment API — the interface another service exposes to programs) — triggers run inside the transaction and must be fast; put a row in an *outbox* table (a table of pending messages) instead and let a worker (a background process) act on it.
- When a **constraint** can do the job (`CHECK`, `FOREIGN KEY`, `UNIQUE`, generated column) — constraints are declarative, faster and visible in the schema.
- When the logic surprises people: "invisible" side effects are the most common complaint about triggers.

Common mistakes

- Forgetting `RETURN NEW` in a `BEFORE` row trigger — returning `NULL` silently **skips** the row.
- A trigger that updates its own table, re-firing itself in a loop (guard with `WHEN` or `pg_trigger_depth()`).
- Expecting triggers on `TRUNCATE` from a row-level trigger — `TRUNCATE` only fires statement-level `ON TRUNCATE` triggers.

Real-world examples

Compliance audit trails (who changed a salary, when), `updated_at` columns used by sync jobs, and the **transactional outbox** pattern for reliable event publishing (telling other systems that something changed).

In the interview

> **30-second answer:** "A trigger runs a function automatically on INSERT/UPDATE/DELETE, before or after, per row or per statement. I use them for cross-cutting (applying everywhere, regardless of which code writes) data rules — audit logs, timestamps, integrity checks a constraint can't express — and avoid them for business workflows because they're invisible and slow down writes."

- *BEFORE vs AFTER?* → BEFORE can modify or reject the row; AFTER sees the final row (after constraints) and is for reactions like auditing.
- *Does a trigger's work roll back with the statement?* → Yes, it runs in the same transaction.
- *How to audit a bulk update efficiently?* → A statement-level trigger with transition tables.

Self-check

1. Which record is available in a DELETE trigger? → `OLD` only.
2. What happens if a BEFORE row trigger returns NULL? → That row's operation is skipped.
3. Why is `WHEN (OLD.* IS DISTINCT FROM NEW.*)` useful? → It skips no-op updates, so `updated_at` changes only when data really changed.

TL;DR

Triggers = automatic functions on data events. Great for audit logs, `updated_at` and cross-table checks; keep them small, prefer constraints when possible, and never put slow external work in them.

14.4 JSON and JSONB

What it is

JSON (JavaScript Object Notation) is the text format of web APIs: objects `{"key": value}`, arrays `[…]`, strings, numbers, booleans and `null`. PostgreSQL has two column types for it:

	`json`	`jsonb` ("JSON binary")
Storage	The exact text you sent	Parsed into a binary tree
Duplicate keys, whitespace, key order	Kept as written	Last duplicate wins; whitespace removed; keys reordered
Speed	Fast to store, re-parsed on every read	Slightly slower to store, fast to query
Operators & indexing	Few	Containment `@>`, key-exists `?`, GIN (inverted) indexes, SQL/JSON path

Use `jsonb` unless you must keep the original text byte-for-byte.

```
SELECT '{"b":1, "a":2, "a":3}'::json AS as_json, '{"b":1, "a":2, "a":3}'::jsonb AS as_jsonb;
```

```
       as_json       |     as_jsonb
---------------------+------------------
 {"b":1, "a":2, "a":3} | {"a": 3, "b": 1}
```

Why it matters

Some data really is **semi-structured**: product specifications differ by category (a laptop has RAM, a desk has a height range), webhook payloads (data another service pushes to your app over HTTP) change shape, user preferences grow new keys. A `jsonb` column stores that without a new column (or a new table) for every attribute — while the stable, important fields stay in normal, typed, constrained columns. For whole-document databases see Chapter 16.

How it works — operators

Operator / function	Returns	Example → result
`->` key or index	`jsonb`	`specs -> 'brand'` → `"Nimbus"`
`->>` key or index	`text`	`specs ->> 'brand'` → `Nimbus`
`#>` / `#>>` path	`jsonb` / `text`	`specs #>> '{screen,type}'` → `OLED`
`@>` contains	boolean	`specs @> '{"brand":"Nimbus"}'`
`?`, `?`	`, ?&`	boolean
`@?`, `@@`	boolean	SQL/JSON path exists / path predicate (PG 12+)
`		`, -, #-`
`jsonb_set(doc, path, value)`	`jsonb`	replace or add a value at a path

Example — product specs

From `code/04_jsonb.sql` (the file first adds `specs jsonb` and fills it for five products):

```
SELECT name,
       specs -> 'brand'                AS brand_json,
       specs ->> 'brand'               AS brand_text,
       specs #>> '{screen,type}'       AS screen_type,
       (specs ->> 'ram_gb')::int       AS ram_gb,
       specs -> 'ports' -> 0           AS first_port,
       jsonb_array_length(specs -> 'ports') AS n_ports
FROM   products WHERE specs ? 'ports' ORDER BY product_id;
```

```
      name      | brand_json | brand_text | screen_type | ram_gb | first_port | n_ports
----------------+------------+------------+-------------+--------+------------+---------
 Laptop Pro 14  | "Nimbus"   | Nimbus     | OLED        |     16 | "usb-c"    |       2
 Wireless Mouse | "Clicko"   | Clicko     |             |        | "usb-a"    |       1
 USB-C Hub      | "Nimbus"   | Nimbus     |             |        | "usb-c"    |       4
```

Note the quotes: `->` gives JSON (`"Nimbus"`), `->>` gives plain text. Missing keys give `NULL`, not an error. To compare numbers, cast the text: `(specs ->> 'ram_gb')::int`.

Filtering:

```
SELECT name FROM products WHERE specs @> '{"brand":"Nimbus"}';          -- Laptop Pro 14, USB-C Hub
SELECT name FROM products WHERE specs -> 'ports' ? 'hdmi';              -- Laptop Pro 14, USB-C Hub
SELECT name, jsonb_path_query_first(specs, '$.height_cm.max') AS max_height
FROM   products WHERE specs @@ '$.height_cm.max > 100';                 -- Standing Desk | 120
```

(`?` on an array checks whether a string **element** is present.)

Updating — `jsonb` values are immutable, so each operation builds a new document:

```
UPDATE products SET specs = jsonb_set(specs, '{ram_gb}', '32')  WHERE product_id = 1;
UPDATE products SET specs = specs || '{"color":"black"}'         WHERE product_id = 2;
UPDATE products SET specs = specs - 'wireless'                   WHERE product_id = 7;
UPDATE products SET specs = specs #- '{screen,type}'             WHERE product_id = 1;
```

```
 product_id |                                          specs
------------+-------------------------------------------------------------------------------------------
          1 | {"brand": "Nimbus", "ports": ["usb-c", "hdmi"], "ram_gb": 32, "screen": {"inches": 14}}
          2 | {"brand": "Clicko", "color": "black", "ports": ["usb-a"], "wireless": true}
          7 | {"brand": "Hushh", "battery_h": 30}
```

Rows → JSON and back. `jsonb_build_object` + `jsonb_agg` build a nested document — e.g. an API response for order 108 in one query: `{"items": [{"qty": 1, "product": "Laptop Pro 14"}, …], "status": "delivered", "order_id": 108}`. In the other direction, `jsonb_to_recordset('[{"sku":"A1","qty":2},{"sku":"B7","qty":5}]') AS t(sku text, qty int)` turns an incoming payload into rows `(A1, 2)`, `(B7, 5)`. `jsonb_each(doc)` similarly turns an object into `(key, value)` rows.

Indexing

A **GIN** (Generalized Inverted Index — an index from each key/value to the rows containing it) serves `@>`, `?`, `?|`, `?&` and the path operators; a normal B-tree **expression index** serves equality and ranges on one extracted key. (The file disables sequential scans only because the demo table is tiny, and runs `VACUUM products` first: an index built over rows that still carry old versions from recent `UPDATE`s may be ignored by transactions whose snapshots are older than the index. See Chapter 12 for reading plans.)

```
CREATE INDEX products_specs_gin ON products USING gin (specs);
EXPLAIN (COSTS OFF) SELECT name FROM products WHERE specs @> '{"brand":"Nimbus"}';
CREATE INDEX products_brand_idx ON products ((specs ->> 'brand'));
EXPLAIN (COSTS OFF) SELECT name FROM products WHERE specs ->> 'brand' = 'Nimbus';
```

```
Bitmap Heap Scan on products
  Recheck Cond: (specs @> '{"brand": "Nimbus"}'::jsonb)
  ->  Bitmap Index Scan on products_specs_gin
        Index Cond: (specs @> '{"brand": "Nimbus"}'::jsonb)

Index Scan using products_brand_idx on products
  Index Cond: ((specs ->> 'brand'::text) = 'Nimbus'::text)
```

The GIN index is *not* used for `specs ->> 'brand' = 'Nimbus'` — write the filter as `@>` or add the expression index.

Dialect differences

	Type	Extract text	Notes
PostgreSQL	`json`, `jsonb`	`->>`, `#>>`, `jsonb_path_query`	GIN indexes; `JSON_TABLE` and SQL/JSON query functions added in PG 17
MySQL 5.7+	`JSON` (binary)	`->>`, `JSON_EXTRACT`, `JSON_UNQUOTE`	Index via generated columns or multi-valued indexes (8.0.17+)
SQL Server	`nvarchar` + `JSON_VALUE` / `OPENJSON`; native `json` type in SQL Server 2025 and Azure SQL	`JSON_VALUE(doc, '$.brand')`	Index a computed column
SQLite	`TEXT` with built-in JSON functions; binary JSONB since 3.45	`->>` (3.38+), `json_extract`	Expression indexes

When not to use it

When fields are known, always present, filtered or joined on, or need constraints (`NOT NULL`, foreign keys) — make them **real columns**. Stuffing everything into one JSON blob is the "EAV / schemaless" antipattern (EAV = entity–attribute–value, storing every attribute as a generic key/value pair): no types, no constraints, poor statistics, awkward queries.

Common mistakes

- Comparing `->` output with text (`specs -> 'brand' = 'Nimbus'` fails: jsonb vs text). Use `->>` or `@>`.
- Sorting or comparing numbers as text (`'9' > '10'`). Cast first.
- Updating one key of a large document still rewrites the whole row.

In the interview

30-second answer: "I use `jsonb` for genuinely variable attributes alongside normal columns. I query with `->>` / `@>`, index with GIN for containment or a B-tree expression index for one hot key, and promote any field that becomes important to a real column."

- *json vs jsonb?* → jsonb is parsed, indexable and fast to query; json keeps exact text.

Self-check

1. `'{"a":1}'::jsonb -> 'a'` vs `->> 'a'`? → jsonb `1` vs text `'1'`.
2. Which index helps `WHERE specs @> '{"wireless":true}'`? → A GIN index on `specs`.
3. How to append a key? → `specs || '{"k":"v"}'` or `jsonb_set`.

TL;DR

`jsonb` + `->>`/`@>` + GIN = flexible attributes inside a relational table. Keep core fields as columns.

14.5 Arrays

What it is

PostgreSQL columns can hold an **array** — an ordered list of values of one type (`text[]`, `int[]`). Arrays are **1-based**, and reading past the end returns `NULL` instead of failing.

Example — product tags

From `code/05_arrays.sql`:

```
ALTER TABLE products ADD COLUMN tags text[] NOT NULL DEFAULT '{}';
UPDATE products SET tags = CASE product_id
    WHEN 1 THEN ARRAY['laptop','work','premium']
    WHEN 2 THEN '{mouse,wireless,budget}'          -- text literal form
    WHEN 4 THEN ARRAY['keyboard','work']
    WHEN 5 THEN ARRAY['desk','work','premium']
    WHEN 7 THEN ARRAY['audio','wireless','premium']
    ELSE '{}' END;

SELECT name, tags, tags[1] AS first_tag, tags[2:3] AS slice, cardinality(tags) AS n, tags[9] AS no_such
FROM   products WHERE product_id IN (1, 2, 9) ORDER BY product_id;
```

```
      name      |          tags           | first_tag |       slice       | n | no_such
----------------+-------------------------+-----------+-------------------+---+---------
 Laptop Pro 14  | {laptop,work,premium}   | laptop    | {work,premium}    | 3 |
 Wireless Mouse | {mouse,wireless,budget} | mouse     | {wireless,budget} | 3 |
 Gift Card      | {}                      |           | {}                | 0 |
```

Operation	Syntax	Result in the demo
Element in array	`'premium' = ANY (tags)`	Laptop, Headphones, Desk
Contains all	`tags @> ARRAY['work','premium']`	Laptop, Desk
Overlaps (any in common)	`tags && ARRAY['wireless','keyboard']`	Keyboard, Headphones, Mouse
Append / remove	`array_append(a, x)`, `array_remove(a, x)`	`{a,b,c}`, `{b}`
Concatenate / find	`a	
Array ↔ rows	`unnest(a) WITH ORDINALITY`, `array_agg(x ORDER BY …)`	see below
Array ↔ string	`string_to_array(s, ',')`, `array_to_string(a, '	')`

`unnest` and `array_agg` are opposites; together they answer "which tags are popular?":

```
SELECT tag, count(*) AS products, array_agg(name ORDER BY name) AS which
FROM   products, unnest(tags) AS tag
GROUP  BY tag HAVING count(*) > 1 ORDER BY products DESC, tag;
```

```
   tag    | products |                            which
----------+----------+-------------------------------------------------------------
 premium  |        3 | {"Laptop Pro 14","Noise-Cancelling Headphones","Standing Desk"}
 work     |        3 | {"Laptop Pro 14","Mechanical Keyboard","Standing Desk"}
 wireless |        2 | {"Noise-Cancelling Headphones","Wireless Mouse"}
```

Arrays as a parameter. Application code often needs "rows whose id is in this list". Instead of building `IN (3, 5, 7)` as a string (injection risk — see 14.9 — a new statement per list length), pass **one array parameter**: `WHERE product_id = ANY ($1)`. The file shows it with `PREPARE by_ids(int[])` / `EXECUTE by_ids('{3,5,7}')` (a server-side prepared statement: parsed once, run with different values), which returns USB-C Hub, Standing Desk and Noise-Cancelling Headphones. A GIN index on `tags` serves `@>`, `&&` and `<@` (the plan in the file shows `Bitmap Index Scan on products_tags_gin`).

When not to use it

If you need to **join** on the elements, enforce **foreign keys** on them, or update single elements often, use a child table (`product_tags(product_id, tag)`) — that's what first normal form (Chapter 11) recommends. Arrays suit small, read-mostly lists like tags. MySQL, SQL Server and SQLite have no array column type (use JSON arrays or a child table).

Self-check

1. What is `ARRAY[10,20][0]`? → `NULL` — arrays start at 1.
2. `= ANY(arr)` vs `@>`? → `x = ANY(arr)` tests one value; `arr @> other` tests that every element of `other` is present.
3. Why pass `= ANY($1)` from code? → One statement for any list length, and values are bound safely.

TL;DR

Arrays: 1-based lists in one column; query with `ANY`, `@>`, `&&`, explode with `unnest`, build with `array_agg`, index with GIN; use a child table when elements need relationships.

14.6 Full-text search

What it is

Full-text search (FTS) finds documents that match *words* — ignoring case, word endings and filler words — and ranks them by relevance, like a small search engine inside the database.

Key terms:

Term	Meaning
Lexeme	A normalised word: lower-cased and **stemmed** ("shipping", "shipped" → `ship`)
Stop word	A very common word ("the", "and", "was") that is dropped
`tsvector`	A document turned into sorted lexemes with positions: `'chair':2 'ship':4,8`
`tsquery`	A search: lexemes joined by `&` (and), `
`@@`	"Does this vector match this query?"
Text search configuration	Language rules, e.g. `'english'` (stemmer + stop-word list)

Why it matters

`LIKE '%word%'` has three problems: no stemming (searching "returning" misses "returned"), no ranking, and a leading `%` cannot use a B-tree index, so every row is scanned. FTS solves all three and is often enough to postpone running a separate search engine such as Elasticsearch/OpenSearch (dedicated search servers).

How it works

1. `to_tsvector('english', text)` parses the text, drops stop words and stems the rest.
2. A query function builds a `tsquery` from user input:
   - `to_tsquery` — expects operator syntax (`'ship & chair'`); errors on bad syntax.
   - `plainto_tsquery` — all words ANDed.
   - `websearch_to_tsquery` (PG 11+) — Google-like: quotes for phrases, `or`, `-` to exclude; never errors on user input.
3. `vector @@ query` tests a match; `ts_rank` scores it; `ts_headline` highlights matches.
4. A GIN index on the `tsvector` makes matching fast.

From `code/06_full_text_search.sql`:

```
SELECT to_tsvector('english', 'The chairs were shipped quickly and the shipping was free') AS doc;
SELECT to_tsquery('english', 'shipping & chair')                          AS q1,
       plainto_tsquery('english', 'fast shipping chairs')                 AS q2,
       websearch_to_tsquery('english', '"standing desk" -cheap or lamp')  AS q3;
```

```
                   doc
-----------------------------------------
 'chair':2 'free':10 'quick':5 'ship':4,8

       q1       |           q2            |                  q3
----------------+-------------------------+---------------------------------------
 'ship' & 'chair' | 'fast' & 'ship' & 'chair' | 'stand' <-> 'desk' & !'cheap' | 'lamp'
```

Example — a help-centre search

The search vector is a **generated column** (computed automatically from other columns on every write; PostgreSQL 12+) so it can never go stale; `setweight` marks title words `A` (most important) and body words `B`, which `ts_rank` rewards:

```
CREATE TABLE help_articles (
    id     int PRIMARY KEY,
    title  text NOT NULL,
    body   text NOT NULL,
    search tsvector GENERATED ALWAYS AS (
             setweight(to_tsvector('english', title), 'A') ||
             setweight(to_tsvector('english', body),  'B')) STORED
);
-- 5 articles: 'Returning a chair', 'Standing desk assembly', 'Shipping times',
--             'Keyboard not connecting', 'Cheap desk lamps discontinued'

SELECT id, title, round(ts_rank(search, q)::numeric, 3) AS rank
FROM   help_articles, websearch_to_tsquery('english', 'chair shipping') AS q
WHERE  search @@ q
ORDER  BY rank DESC, id;
```

```
 id |       title       | rank
----+-------------------+-------
  1 | Returning a chair | 0.588
  3 | Shipping times    | 0.529
```

More query shapes from the same file:

Query	Matches	Why
`to_tsquery('english', 'assem:*')`	2 Standing desk assembly	`:*` = prefix match (autocomplete)
`websearch_to_tsquery('english', '"standing desk"')`	2	Phrase: `stand <-> desk`
`websearch_to_tsquery('english', 'desk -lamp')`	2, 3	Article 5 excluded by `-lamp`

Highlighting shows the user *why* a result matched:

```
SELECT id, ts_headline('english', body, websearch_to_tsquery('english', 'desk chair'),
                       'StartSel=[, StopSel=], MaxWords=12, MinWords=5') AS snippet
FROM   help_articles WHERE search @@ websearch_to_tsquery('english', 'desk chair');
```

```
 id |             snippet
----+---------------------------------
  3 | [desk] or [chair] ships in a week
```

And stemming in action — no body contains the word "returning", but article 1's body has "returned" and "returns":

```
 like_hits | fts_hits          -- body ILIKE '%returning%'  vs  to_tsvector(body) @@ 'returning'
-----------+----------
         0 |        1
```

With `CREATE INDEX … USING gin (search)` the plan becomes `Bitmap Index Scan on help_articles_search_gin`.

Trade-offs

	PostgreSQL FTS	Dedicated engine (Elasticsearch / OpenSearch / Solr)
Setup	None — same database, same transaction	Separate cluster (group of servers), sync pipeline (a job copying data across)
Freshness	Immediately consistent	Near-real-time, eventually consistent (copies catch up after a short delay)
Features	Stemming, ranking, phrases, prefixes, weights	Fuzzy/typo tolerance, facets (match counts per category, shown as filters), synonyms at scale, relevance tuning, distributed scoring
Best for	Up to millions of documents, app-level search	Very large corpora, search-centric products

Typo tolerance in Postgres needs the `pg_trgm` extension (trigram similarity — comparing 3-character chunks of text), which is not installed on the book's server. Other databases: MySQL `FULLTEXT` indexes with `MATCH … AGAINST`, SQL Server full-text catalogs with `CONTAINS` / `FREETEXT`, SQLite's `FTS5` virtual tables.

Common mistakes

- Mixing configurations: indexing with `to_tsvector('english', …)` but querying with `to_tsvector(body)` (default config) — the index won't be used. Always name the configuration.
- Passing raw user input to `to_tsquery` — a stray `&` or `(` raises a syntax error. Use `websearch_to_tsquery`.
- Computing `to_tsvector` in every query instead of storing it (generated column) and indexing it.

In the interview

30-second answer: "Postgres FTS turns text into a `tsvector` of stemmed lexemes and matches a `tsquery` with `@@`. I store the vector as a weighted generated column, index it with GIN, parse user input with `websearch_to_tsquery`, and rank with `ts_rank`. I'd move to Elasticsearch only when I need fuzzy matching, facets or scale beyond one database."

Self-check

1. Why does "shipping" match "shipped"? → Both stem to the lexeme `ship`.
2. Which index type? → GIN on the `tsvector`.
3. How to make titles count more? → `setweight(…, 'A')` for title lexemes; `ts_rank` weights A highest.

TL;DR

FTS = `tsvector @@ tsquery` + GIN + `ts_rank` + `ts_headline`; beats `LIKE` on stemming, ranking and speed.

14.7 Pivot and unpivot

What it is

Pivoting turns *values in rows* into *columns* — like an Excel pivot table: one row per category, one column per month. **Unpivoting** is the reverse: a wide table becomes tall again. Databases prefer tall (long) data; reports and spreadsheets prefer wide data. Standard SQL has no `PIVOT` keyword, so we build pivots from conditional aggregation (introduced in Chapter 4).

Example — revenue by category and month

From `code/07_pivot.sql`. A view first produces the tall data (`category, month, revenue`):

```
SELECT category,
       round(sum(revenue) FILTER (WHERE month = '2025-01'), 2) AS jan,
       round(sum(revenue) FILTER (WHERE month = '2025-02'), 2) AS feb,
       round(sum(revenue) FILTER (WHERE month = '2025-03'), 2) AS mar,
       round(sum(revenue) FILTER (WHERE month = '2025-04'), 2) AS apr,
       round(sum(revenue), 2)                                  AS all_months
FROM   monthly_category_revenue
GROUP  BY category ORDER BY category;
```

```
  category   |   jan   |  feb   |  mar   |   apr   | all_months
-------------+---------+--------+--------+---------+------------
 (none)      |         |        | 100.00 |         |     100.00
 Accessories |  188.00 |        |        |  200.80 |     671.35
 Electronics | 1683.90 |        |  74.97 | 1448.95 |    4691.80
 Furniture   |         | 811.49 | 629.98 |  499.00 |    1940.47
```

(`all_months` also includes May and June, which have no column here.) The portable form, which works in every database, puts `CASE` inside the aggregate: `sum(CASE WHEN month = '2025-01' THEN revenue ELSE 0 END) AS jan` — with `ELSE 0`, empty cells show `0.00` instead of `NULL`. The same technique counts order statuses per customer (`count(*) FILTER (WHERE o.status = 'delivered') AS delivered`, …).

`crosstab` (tablefunc extension)

PostgreSQL's `tablefunc` extension provides `crosstab(source_sql, category_sql)`, which pivots a three-column query (row id, category, value). It is **not installed on the book's server** (Homebrew's — a macOS package manager — build here ships without it), so this is not run:

```
-- (PostgreSQL with contrib — not run)
CREATE EXTENSION IF NOT EXISTS tablefunc;
SELECT *
FROM crosstab(
  $$ SELECT category, month, round(revenue, 2) FROM monthly_category_revenue ORDER BY 1, 2 $$,
  $$ VALUES ('2025-01'), ('2025-02'), ('2025-03'), ('2025-04') $$
) AS ct(category text, jan numeric, feb numeric, mar numeric, apr numeric);
```

You must still list the output columns and their types in `AS ct(...)`; the two-argument form places each value in the right column even when some categories are missing. `FILTER` / `CASE` does the same job without an extension, so most teams use them.

Unpivot with `LATERAL (VALUES …)`

```
CREATE TABLE quarterly_targets (region text PRIMARY KEY, q1 int, q2 int, q3 int, q4 int);
INSERT INTO quarterly_targets VALUES ('India', 120, 150, 170, 200), ('UK', 60, 65, NULL, 80);

SELECT t.region, v.quarter, v.target
FROM   quarterly_targets t
CROSS  JOIN LATERAL (VALUES ('Q1', t.q1), ('Q2', t.q2), ('Q3', t.q3), ('Q4', t.q4)) AS v(quarter, target)
WHERE  v.target IS NOT NULL
ORDER  BY t.region, v.quarter;
```

This returns 7 rows — four for India and three for the UK (its `NULL` Q3 target is filtered out).

`LATERAL` (Chapter 6) lets the `VALUES` list refer to the current row's columns. The portable alternative is a `UNION ALL` of one `SELECT` per column.

Dynamic pivots

SQL needs the column list **when the query is planned**, so a pivot whose columns depend on the data (every month that exists) requires either building the SQL text in code (with identifiers quoted — see 14.9) or returning a JSON object per row and letting the application lay it out:

```
SELECT category, jsonb_object_agg(month, round(revenue, 2) ORDER BY month) AS by_month
FROM   monthly_category_revenue GROUP BY category ORDER BY category;
```

```
  category   |                          by_month
-------------+-------------------------------------------------------------
 (none)      | {"2025-03": 100.00}
 Accessories | {"2025-01": 188.00, "2025-04": 200.80, "2025-06": 282.55}
 ...           (Electronics and Furniture rows similar)
```

Dialect differences

	Pivot	Unpivot
PostgreSQL	`FILTER`, `CASE`, `crosstab` (tablefunc)	`LATERAL (VALUES …)`, `unnest`
SQL Server / Oracle	`PIVOT (SUM(x) FOR col IN (…))` operator	`UNPIVOT` operator (skips NULLs)
MySQL	`CASE` (or `SUM(cond)` since booleans are 0/1)	`UNION ALL`; `LATERAL` from 8.0.14
SQLite	`FILTER` (3.30+) or `CASE`	`UNION ALL`

In the interview

30-second answer: "Pivot = group by the row key and write one conditional aggregate per output column — `SUM(x) FILTER (WHERE col = 'v')` or `SUM(CASE WHEN …)`. Unpivot = `CROSS JOIN LATERAL (VALUES …)` or `UNION ALL`. If the columns aren't known in advance, generate the SQL in code or return JSON."

- *`count(CASE WHEN … THEN 1 END)` vs `sum(CASE WHEN … THEN 1 ELSE 0 END)`?* → Same counts; `count` ignores the NULLs from the missing `ELSE`.

TL;DR

Pivot with conditional aggregates, unpivot with `LATERAL (VALUES …)`; dynamic column lists need generated SQL or JSON.

14.8 Security: roles, privileges and row-level security

What it is

- A **role** is PostgreSQL's single concept for both *users* and *groups*. A role with `LOGIN` can connect (a user); a role without it is a group that other roles are members of. Roles are **cluster-wide** (shared by all databases on one server).
- A **privilege** is permission for one action on one object: `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`, `REFERENCES`, `TRIGGER` on tables; `USAGE`/`CREATE` on schemas; `EXECUTE` on functions; `CONNECT` on databases.
- `GRANT` gives privileges; `REVOKE` takes them away. These are the **DCL** (data control language) statements from Chapter 1.
- **Row-level security (RLS)** adds *per-row* filters: two users may read the same table but see different rows.

Why it matters

The **principle of least privilege**: every account gets only what it needs. If the reporting tool's password leaks, the attacker can read reports — not delete orders. If the web app has an injection bug (14.9), a read-only role limits the damage. RLS pushes tenant isolation (a *tenant* is one customer organisation sharing the app: "customer A never sees customer B's data") into the database, so one forgotten `WHERE tenant_id = …` in the app can't leak data.

Example — groups, grants, column privileges

From `code/08_security.sql`. (Roles use a `ch14_` prefix and are dropped at the end, because they are shared server-wide.)

```
CREATE ROLE ch14_reporting NOLOGIN;                          -- a group
CREATE ROLE ch14_analyst  LOGIN IN ROLE ch14_reporting;      -- a user in that group
CREATE ROLE ch14_app      LOGIN;                             -- the application's account

GRANT SELECT ON orders, order_items TO ch14_reporting;
GRANT SELECT, INSERT, UPDATE ON orders TO ch14_app;
GRANT SELECT (customer_id, first_name, city, country) ON customers TO ch14_reporting;  -- column-level
```

Querying `information_schema.role_table_grants` (`information_schema` is the SQL-standard set of catalog views) confirms: `ch14_app` has INSERT, SELECT, UPDATE on `orders`; `ch14_reporting` has SELECT on `orders` and `order_items`. (Column-level grants appear in `information_schema.column_privileges`, not in this table-level view.)

`SET ROLE` lets a superuser act as another role for testing — the same checks apply as if that role had logged in:

```
SET ROLE ch14_analyst;
SELECT count(*) FROM orders;                                      -- 14
SELECT first_name, city FROM customers WHERE customer_id = 1;     -- Anita | Mumbai
PERFORM email FROM customers;            -- NOTICE: analyst: permission denied for table customers
DELETE FROM orders WHERE order_id = 114;  -- NOTICE: analyst: permission denied for table orders
RESET ROLE;
REVOKE SELECT ON order_items FROM ch14_reporting;   -- analyst now denied on order_items too
```

(The failing statements are wrapped in `DO` blocks — one-off anonymous PL/pgSQL code — in the file so the script continues; `PERFORM` is PL/pgSQL's way to run a query and discard its result.) The analyst had no direct grants: privileges were **inherited** from the group, so revoking from the group affected every member at once — grant to groups, not to individuals.

Example — row-level security: each sales rep sees only their orders

```
CREATE ROLE ch14_priya LOGIN;      -- employee 7
CREATE ROLE ch14_arjun LOGIN;      -- employee 8
CREATE TABLE rep_logins (role_name text PRIMARY KEY, emp_id int NOT NULL REFERENCES employees);
INSERT INTO rep_logins VALUES ('ch14_priya', 7), ('ch14_arjun', 8);
GRANT SELECT ON rep_logins TO ch14_priya, ch14_arjun;
GRANT SELECT, UPDATE ON orders TO ch14_priya, ch14_arjun;

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;          -- from now on: no matching policy = no rows

CREATE POLICY rep_sees_own_orders ON orders
    FOR ALL TO ch14_priya, ch14_arjun
    USING      (sales_rep_id = (SELECT emp_id FROM rep_logins WHERE role_name = current_user))
    WITH CHECK (sales_rep_id = (SELECT emp_id FROM rep_logins WHERE role_name = current_user));

CREATE POLICY reporting_reads_all ON orders FOR SELECT TO ch14_reporting USING (true);

SET ROLE ch14_priya;
SELECT current_user, order_id, sales_rep_id, status FROM orders ORDER BY order_id;
UPDATE orders SET status = 'shipped' WHERE order_id = 103;     -- Arjun's order
UPDATE orders SET sales_rep_id = 8 WHERE order_id = 101;       -- hand her order to Arjun (in a DO block)
```

```
 current_user | order_id | sales_rep_id |  status
--------------+----------+--------------+-----------
 ch14_priya   |      101 |            7 | delivered
 ch14_priya   |      105 |            7 | cancelled
 ch14_priya   |      108 |            7 | delivered
 ch14_priya   |      113 |            7 | pending

UPDATE 0
NOTICE:  priya: new row violates row-level security policy for table "orders"
```

Role	Policy that applies	`count(*) from orders`
`ch14_priya`	`rep_sees_own_orders`	4
`ch14_arjun`	`rep_sees_own_orders`	2
`ch14_analyst` (member of reporting)	`reporting_reads_all`	14
`ch14_app`	none — has `SELECT` but no policy	**0**
`postgres` (superuser / owner)	bypasses RLS	14

- `USING` filters which *existing* rows are visible (to `SELECT`, `UPDATE`, `DELETE`) — so the `UPDATE` of order 103 found 0 rows rather than failing.
- `WITH CHECK` validates rows being *written* (`INSERT`, `UPDATE`) — so Priya can't move a row out of her own view.
- Policies are **permissive** by default (combined with `OR`); `AS RESTRICTIVE` policies are combined with `AND`.

Multi-tenant isolation with a session setting

SaaS (software as a service: one hosted app shared by many customer organisations) apps usually connect as **one** application role and tell the database which tenant the current request belongs to:

```
ALTER TABLE notes ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON notes
    USING (tenant = current_setting('app.tenant', true));    -- true: unset -> NULL, not an error

SET ROLE ch14_app;
SET app.tenant = 'acme';    SELECT * FROM notes;          -- only acme's 2 notes
SET app.tenant = 'globex';  SELECT * FROM notes;          -- only globex's note
RESET app.tenant;           SELECT count(*) FROM notes;   -- 0: no tenant, no rows
```

The app runs `SET LOCAL app.tenant = …` at the start of each transaction (`SET LOCAL` resets at commit — important with connection pools (sets of reusable open connections), 14.12, where the next request reuses the connection).

Common mistakes

- **Testing RLS as a superuser or the table owner** — both bypass it, so "it works" proves nothing. Test with `SET ROLE`, and use `ALTER TABLE … FORCE ROW LEVEL SECURITY` if the owner must be filtered too.
- Letting the application connect as a superuser or the table owner "because it's easier".
- Forgetting that new tables aren't covered by old grants — use `ALTER DEFAULT PRIVILEGES … GRANT …` for future tables.
- Leaving the `PUBLIC` pseudo-role (an implicit role every user belongs to) with rights you didn't intend (in PG ≤ 14, everyone can `CREATE` in the `public` schema; PG 15 removed that default).

Dialect differences

	Roles / users	Row-level security
PostgreSQL	Roles (users = roles with `LOGIN`)	`CREATE POLICY` (9.5+)
MySQL 8.0	Users + roles	None built in — use views or app logic
SQL Server 2016+	Logins, users, roles	Security policies with inline predicate functions (functions returning a true/false filter)
Oracle	Users, roles	Virtual Private Database (VPD)
SQLite	No users — file-system permissions only	None

In the interview

> **30-second answer:** "I apply least privilege: group roles that hold grants, login roles as members, the app gets only the DML it needs, reporting gets read-only, and nobody uses the superuser from code. For multi-tenant data I add row-level security keyed on a session variable, test it with `SET ROLE` — never as owner — and keep a tenant_id index so the policy filter is cheap."

- *USING vs WITH CHECK?* → Visibility of existing rows vs validity of new/changed rows.

Self-check

1. Why did `ch14_app` see 0 orders? → RLS is enabled and no policy applies to it — default deny.
2. What does `REVOKE` on a group do to its members? → They lose the inherited privilege.
3. Why `SET LOCAL` with pooled connections? → The setting ends with the transaction, so it can't leak to the next request.

TL;DR

Roles + `GRANT`/`REVOKE` for what each account may do; RLS policies for which rows it may see or write; test as the real role, never as owner.

14.9 SQL injection and parameterized queries

What it is

SQL injection happens when user input is pasted into the *text* of a SQL statement, so specially crafted input changes the statement's meaning. It has been in the OWASP (Open Worldwide Application Security Project) Top 10 list of web risks since the list began, and has caused many real breaches (for example the 2015 TalkTalk breach in the UK and the 2023 MOVEit Transfer attacks).

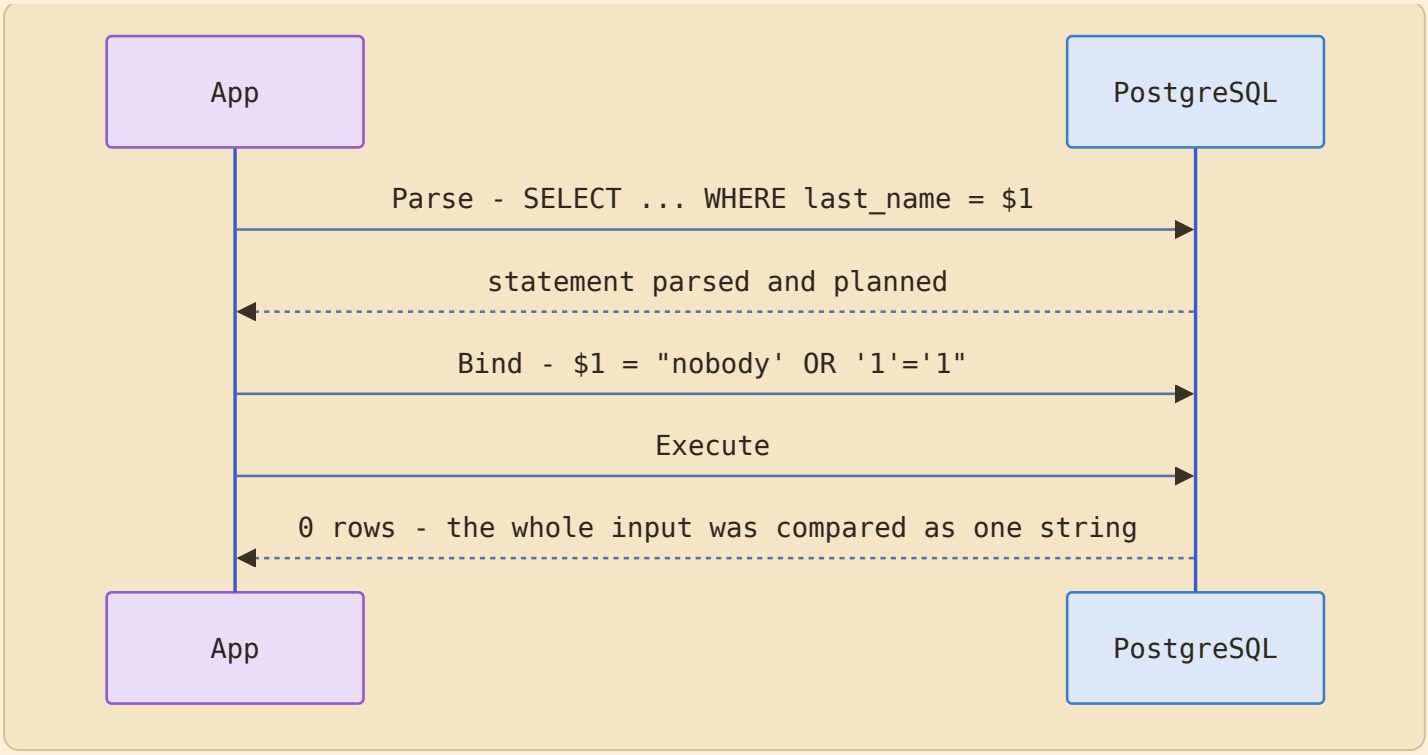
Suppose the code builds:

```
SELECT … FROM customers WHERE last_name = '<input>'
```

With the input `nobody' OR '1'='1`, the text becomes `… WHERE last_name = 'nobody' OR '1'='1'` — a condition that is always true.

How it's fixed — parameters

A **parameterized query** (also *prepared statement* or *bind variables*) sends the SQL text with placeholders (`$1`, `?`, `%s`) and the values **separately**. The server parses the SQL first, then plugs the values in as data. Input can never become SQL syntax, however many quotes it contains.



Example — injection inside the database (dynamic SQL)

The same bug appears in PL/pgSQL when `EXECUTE` runs a string. From `code/09_sql_injection.sql`:

```
-- VULNERABLE
CREATE FUNCTION find_customers_bad(p_last_name text)
RETURNS TABLE (customer_id int, first_name text, email text)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY EXECUTE
        'SELECT customer_id, first_name, email FROM customers WHERE last_name = '''
        || p_last_name || '''';
END;
$$;
SELECT * FROM find_customers_bad('Carter');                -- 1 row: Ben
SELECT * FROM find_customers_bad($$nobody' OR '1'='1$$);     -- 10 rows: every customer leaks

-- SAFE: the value is passed with USING, referenced as $1
CREATE FUNCTION find_customers_good(p_last_name text) ... AS $$
BEGIN
    RETURN QUERY EXECUTE
        'SELECT customer_id, first_name, email FROM customers WHERE last_name = $1'
        USING p_last_name;
END;
$$;
SELECT count(*) AS rows_leaked FROM find_customers_good($$nobody' OR '1'='1$$);   -- 0
```

Identifiers can't be parameters. Placeholders stand for *values*, never for table or column names. When a name must be dynamic, check it against an **allow-list** (a fixed list of permitted names) and quote it with `format('%I')` (identifier) — use `%L` for a literal:

```
SELECT format('SELECT * FROM %I WHERE name = %L', 'Order Items', $$O'Brien$$);
--  SELECT * FROM "Order Items" WHERE name = 'O''Brien'
```

`count_rows(p_table)` in the file raises an error for any table outside `('orders','customers','products')` before building its query. SQL-level prepared statements work the same way: `PREPARE orders_of(int) AS … WHERE customer_id = $1; EXECUTE orders_of(2);`.

The client-side versions of this demo (Python f-string vs `%s`, Java string concatenation vs `PreparedStatement`) are in 14.10 and 14.11 — all show **10 rows vs 0 rows**.

Defence in depth

Layer	Defence
Query	Parameters for every value; allow-list + `%I` / `quote_ident` for identifiers
Account	Least-privilege database role (14.8) — injection can't drop tables the role can't touch
Data	RLS for tenant isolation
Code	Linters/static analysis (tools that inspect code without running it) that flag string-built SQL; code review
Errors	Don't show raw database errors to users (they help attackers map the schema)

Escaping by hand (doubling quotes yourself) is fragile — encodings (the byte representation of characters) and edge cases have broken it repeatedly. Use parameters.

In the interview

30-second answer: "Injection is user input interpreted as SQL because it was concatenated into the query. The fix is parameterized queries — values travel separately and can't change the parse tree (the server's structured reading of the statement). For dynamic identifiers use an allow-list and identifier quoting. Add least privilege so a bug has limited blast radius (the amount of damage it can do). ORMs (object-relational mappers — libraries that generate SQL from code objects, 14.13) parameterize by default, but raw-SQL escape hatches (helpers for writing your own SQL) in ORMs are still injectable."

- *Do prepared statements help performance too?* → Yes — the server can reuse the parsed/planned statement across executions.
- *Is `LIKE ?` safe?* → From injection yes, but `%` and `_` in the input still act as wildcards; escape them if they should be literal.

TL;DR

Never build SQL from input strings; bind values as parameters, allow-list identifiers, and limit what the database account can do.

14.10 Using SQL from Python

Drivers and the DB-API

Python talks to databases through **drivers** that follow the **DB-API 2.0** standard (PEP 249 — a PEP is a Python Enhancement Proposal, an official design document): `connect()` → a **connection** (a session with the server) → **cursors** (objects that run statements and hold results) → `execute()`, `fetchone()`, `fetchall()`, `commit()`, `rollback()`. Two drivers are shown here:

	psycopg (version 3)	sqlite3
Database	PostgreSQL server	SQLite file or in-memory DB, in-process (runs inside your program, no separate server)
Install	`pip install "psycopg[binary]"` (pip is Python's package installer; already in `.venv-books`, the book's virtual environment)	Built into Python
Placeholder	`%s` or `%(name)s` (*never* Python `%` formatting)	`?` or `:name`
Transactions	Implicit `BEGIN` unless `autocommit=True`; `with conn.transaction():` blocks	Implicit transaction before DML (data-changing statements: `INSERT`, `UPDATE`, `DELETE`); `with conn:` commits/rolls back

See also the Python book's standard-library tour for `sqlite3` basics (Python ch. 14).

Example — psycopg against the private server

From `code/psycopg_demo.py` (it creates a fresh database `ch_14_code` and loads the shared schema). Key parts:

```
SERVER = "host=/tmp port=5544 user=postgres"       # Unix socket directory + port
DSN = SERVER + " dbname=ch_14_code"

# autocommit=True: each statement commits on its own, and conn.transaction() blocks are
# real BEGIN ... COMMIT/ROLLBACK units
with psycopg.connect(DSN, autocommit=True) as conn:
    # 1. parameters: values are sent separately from the SQL text
    cur = conn.execute(
        "SELECT first_name, city FROM customers WHERE country = %s AND signup_date >= %s ORDER BY customer_id",
        ("India", "2025-01-01"))
    for row in cur.fetchall():
        print("  ", row)
    with conn.cursor(row_factory=dict_row) as cur:              # rows as dicts
        cur.execute("SELECT name, unit_price FROM products WHERE unit_price > %(min)s ORDER BY unit_price DESC",
                    {"min": 300})

    # 2. injection: f-string vs parameter; identifiers via psycopg.sql
    evil = "nobody' OR '1'='1"
    conn.execute(f"SELECT count(*) FROM customers WHERE last_name = '{evil}'")      # NEVER
    conn.execute("SELECT count(*) FROM customers WHERE last_name = %s", (evil,))
    q = sql.SQL("SELECT {} FROM customers WHERE customer_id = %s").format(sql.Identifier("city"))

    # 3. a transaction: both updates or neither
    try:
        with conn.transaction():
            conn.execute("UPDATE products SET unit_price = unit_price * 0.9 WHERE category = 'Furniture'")
            conn.execute("UPDATE products SET unit_price = -1 WHERE product_id = 2")   # CHECK fails
    except psycopg.errors.CheckViolation as e:
        print("rolled back:", e.diag.message_primary)

    # 4. batch (executemany), bulk load (cur.copy("COPY ... FROM STDIN") + write_row),
    #    and a named = server-side cursor that streams rows in chunks
    with conn.transaction(), conn.cursor(name="big_scan") as cur:
        cur.itersize = 4                                        # rows per round trip
        cur.execute("SELECT g FROM generate_series(1, 10) g")
        print("streamed:", [r[0] for r in cur])
```

Output:

```
connected to ch_14_code on port 5544 | server 14

== 1. query, fetch, parameters ==
customers: 10
   ('Chen', 'Bengaluru')
   ('Divya', 'Delhi')
   ('Fatima', None)
   ('Anita', 'mumbai')
   ('Ivan', 'Delhi')
dict rows: [{'name': 'Laptop Pro 14', 'unit_price': Decimal('1299.00')}, ...]   (3 dicts, shortened here)

== 2. SQL injection: string building vs parameters ==
built SQL : SELECT count(*) FROM customers WHERE last_name = 'nobody' OR '1'='1'
f-string  : 10 rows
parameter : 0 rows
composed  : SELECT "city" FROM customers WHERE customer_id = %s -> Bengaluru

== 3. transactions ==
rolled back: new row for relation "products" violates check constraint "products_unit_price_check"
desk price after rollback: 499.00
desk price after commit: 479.00

== 4. batch insert, COPY, streaming ==
executemany rowcount: 3
customers now: 16
streamed: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

== 5. connection pooling ==
no pool : 20 requests used 20 server processes
pool(2) : 20 requests used 2 server processes
```

Notes on what the output shows:

- **Type mapping**: SQL `NULL` → `None`, `numeric` → `Decimal` (exact, never `float` for money), `date` → `datetime.date`.
- **Transactions**: the 10 % furniture discount was rolled back together with the failing update — the desk still costs 499.00. Exceptions are typed by SQLSTATE (`psycopg.errors.CheckViolation`, `UniqueViolation`, …).
- **Bulk loading speed**, fastest last: one `execute` per row → `executemany` (psycopg 3 pipelines it — sends many statements without waiting for each reply) → `COPY` (PostgreSQL's bulk-load command).
- **Server-side cursor** (`conn.cursor(name=…)`): the result stays on the server and is fetched in chunks of `itersize`, so a 10-million-row export doesn't need 10 million rows in memory.
- **Default transaction trap**: without `autocommit=True`, psycopg sends `BEGIN` before your first statement and nothing is saved until `conn.commit()` (or leaving `with psycopg.connect(...)` normally). A forgotten commit, or a connection left "idle in transaction" (connected, with a transaction open, doing nothing), is a classic bug.

Example — sqlite3

From `code/sqlite3_demo.py` — same ideas, `?` placeholders, no server:

```
conn = sqlite3.connect(":memory:")
conn.row_factory = sqlite3.Row
conn.execute("PRAGMA foreign_keys = ON")    # SQLite leaves FK checks off unless asked
...
with conn:                                  # commit on success, rollback on exception
    conn.executemany("INSERT INTO products (name, price, specs) VALUES (?, ?, ?)", rows)
for r in conn.execute("SELECT product_id, name, price FROM products WHERE price < :max ORDER BY price", {"max":
40}):
    print(dict(r))
```

```
{'product_id': 1, 'name': 'Wireless Mouse', 'price': 24.99}
{'product_id': 3, 'name': 'Desk Lamp', 'price': 35.0}
rolled back: UNIQUE constraint failed: products.name
hub price: 49.5
f-string  rows: 3
parameter rows: 0
sqlite 3.53.4 | brands: ['Clicko', 'Nimbus']
```

(The version line shows the SQLite library bundled with this Python; yours may differ.) Note `with conn:` in `sqlite3` manages the **transaction**, not closing — call `conn.close()` yourself.

Self-check

1. Why is `cur.execute("… = %s" % value)` wrong? → That's Python string formatting — injection again. Pass values as the second argument.
2. How do you stream a huge result in psycopg? → A named (server-side) cursor with `itersize`.
3. What type does `numeric` become? → `decimal.Decimal`.

14.11 Using SQL from Java (JDBC)

What it is

JDBC (Java Database Connectivity) is Java's standard database API in `java.sql`. A vendor **driver** jar (a packaged Java library file) implements it; for PostgreSQL that's `org.postgresql:postgresql` (version 42.7.13 here, from Maven Central, the main public repository of Java libraries). The core types:

Type	Role
`DriverManager` / `DataSource`	Hands out connections (a `DataSource` is how pools plug in)
`Connection`	A session; controls auto-commit, commit/rollback, isolation level
`PreparedStatement`	SQL with `?` placeholders, typed setters (`setString`, `setObject`, `setBigDecimal`)
`ResultSet`	A cursor over rows; `next()` + typed getters
`SQLException`	Errors, with `getSQLState()` (e.g. `23514` = check violation)

Resources are closed with **try-with-resources** (Java syntax that closes objects automatically at the end of a block) so connections are never leaked. For the Java language side see the Java book (ch. 8 exceptions, ch. 18 tooling & JDBC).

Running it. The jar lives in `.java-books/lib/`. Because the book's private server listens only on a Unix socket (`/tmp/.s.PGSQL.5544`), the URL adds the `junixsocket` socket factory (a plug-in that lets Java connect through a Unix socket — a same-machine connection without networking; two more jars in the same folder); against a normal server the URL is just `jdbc:postgresql://localhost:5544/ch_14_jdbc`. The `java` launcher can compile and run a single source file in one step (source-file mode, Java 11+; the book targets Java 25):

```
java --enable-native-access=ALL-UNNAMED -cp '.java-books/lib/*' \
     sql/chapters/14-advanced-sql-and-using-sql-from-code/code/JdbcDemo.java
```

(The file starts with `// tools_run: skip` so the Java book's generic runner doesn't try to run it without the driver.)

Example — key parts of `code/JdbcDemo.java`

```
// PreparedStatement: '?' placeholders, typed setters and getters
String sql = """
        SELECT first_name, city, signup_date
        FROM customers WHERE country = ? AND signup_date >= ? ORDER BY customer_id""";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, "India");
    ps.setObject(2, LocalDate.of(2025, 1, 1));              // java.time maps to DATE
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            String city = rs.getString("city");              // SQL NULL -> Java null
            LocalDate d = rs.getObject("signup_date", LocalDate.class);
            ...
        }
    }
}

// Transaction: turn auto-commit off, commit or roll back as one unit
conn.setAutoCommit(false);
conn.setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ);
try (PreparedStatement up = conn.prepareStatement(
        "UPDATE products SET unit_price = unit_price + ? WHERE product_id = ?")) {
    up.setBigDecimal(1, new BigDecimal("-10.00")); up.setInt(2, 5); up.executeUpdate();
    up.setBigDecimal(1, new BigDecimal("-99.00")); up.setInt(2, 2); up.executeUpdate(); // CHECK fails
    conn.commit();
} catch (SQLException e) {
    conn.rollback();                                         // undo BOTH updates
} finally {
    conn.setAutoCommit(true);
}

// Batch: setXxx(...) + ps.addBatch() per row, then int[] counts = ps.executeBatch(); conn.commit();
```

Output:

```
connected: PostgreSQL 14.15 (Homebrew) via PostgreSQL JDBC Driver 42.7.13

== PreparedStatement ==
  Chen   Bengaluru  2025-01-08
  Divya  Delhi      2025-01-20
  Fatima (null)     2025-02-14
  Anita  mumbai     2025-03-09
  Ivan   Delhi      2025-05-30

== SQL injection ==
concatenated : 10 rows
parameterized: 0 rows

== transaction ==
rolled back, SQLState 23514: ERROR: new row for relation "products" violates check constraint
"products_unit_price_check"
desk price unchanged: 499.00

== batch ==
batch update counts: [1, 1, 1]
  Monitor Arm -> 61.95
  Webcam HD -> 83.90
  Cable Tray -> 20.48
```

The last block used `UPDATE … RETURNING name, unit_price` with `executeQuery()` — PostgreSQL's `RETURNING` gives back computed values without a second query (JDBC's `getGeneratedKeys()` is the portable alternative for new ids).

Common mistakes (Python and Java alike)

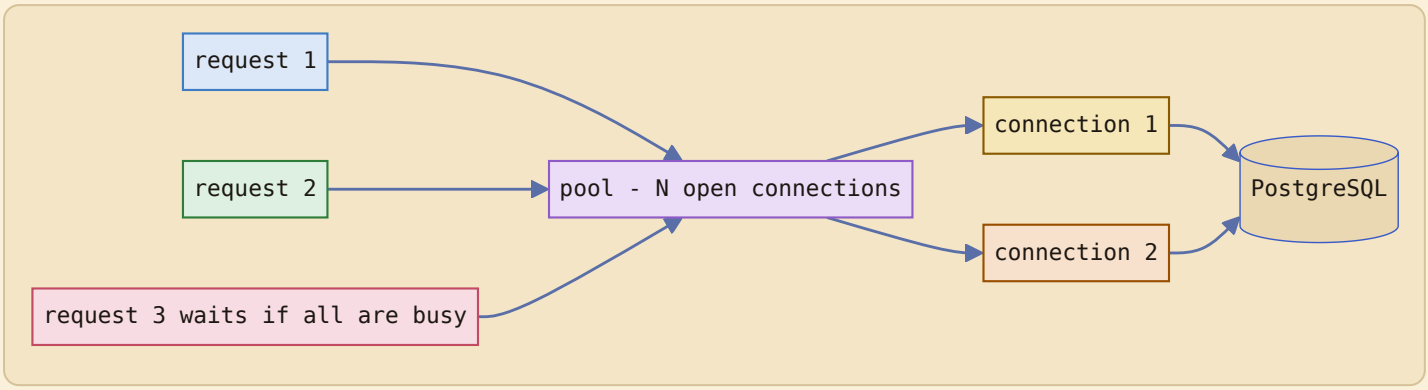
- **Leaking connections** — not closing them (use `with` / try-with-resources); the server runs out of connection slots.

- **Using `float`/`double` for money** — use `Decimal` / `BigDecimal`.
- **Forgetting `rs.wasNull()`**: `getInt` returns `0` for SQL `NULL`; use `getObject(col, Integer.class)` or check `wasNull()`.
- **Auto-commit surprises** — JDBC starts in auto-commit *on* (every statement its own transaction); psycopg starts with it *off*. Know which one your code is in.
- **One query per row** in a loop (the N+1 pattern, 14.13) instead of a join or `= ANY(?)` with an array (`conn.createArrayOf("int", ids)`).

14.12 Connection pooling

What it is

Opening a PostgreSQL connection is expensive: a network handshake (the opening back-and-forth that sets up a connection), TLS (the encryption layer for network traffic; see System Design Fundamentals), authentication (checking who you are), and — PostgreSQL uses a **process per connection** — forking (the operating system copying a process to create) a new server process that allocates its own memory. That costs milliseconds, while a simple indexed query takes well under one. A **connection pool** opens a few connections once and **lends** them to requests, like a library lending books instead of printing a new copy for each reader.



The last section of `psycopg_demo.py` shows the difference with a 15-line `TinyPool` (a `queue.Queue` of open connections): 20 requests without a pool created **20 server processes**; with a pool of 2 they used **2**.

How it works

1. At start-up the pool opens *min* connections.
2. A request **borrows** a connection (`getConnection()` / `pool.connection()`), waiting up to a timeout if all are busy.
3. When done, it **returns** it; the pool resets session state (rolls back an open transaction, may run a validation query) and keeps it open.
4. The pool closes connections that are idle too long or reach a maximum lifetime.

Kinds of pools

Where	Examples	Notes
In the application	HikariCP (Java; Spring Boot's — a popular Java web framework — default), `psycopg_pool`, SQLAlchemy's pool, node-postgres `Pool`	Per process; total = processes × pool size
External proxy (a separate program sitting between the apps and the database)	PgBouncer, pgcat, Amazon RDS Proxy	Shares a small set of server connections among many app processes

PgBouncer's **transaction mode** gives a server connection to a client only for the length of one transaction — very efficient, but session features (plain `SET`, session advisory locks — application-

defined locks on a number — and `LISTEN`, PostgreSQL's notification subscription) don't carry across transactions; use `SET LOCAL` (as in 14.8).

Sizing

More connections is **not** faster: past a point, processes compete for CPU cores and disk, and each costs memory (`max_connections` defaults to 100). A widely quoted starting point from the HikariCP project is roughly *(CPU cores × 2) + effective disk spindles* (the number of physical disks working in parallel), then measure. Watch the pool's **wait time** — if requests queue, look for slow queries before raising the size.

Common mistakes

- Holding a connection during slow non-database work (calling an external API mid-transaction).
- Session state leaking between borrowers (`SET` without `LOCAL`, temp tables, open transactions).
- Every microservice replica (one running copy of a small, separately deployed service) with a pool of 50 → thousands of connections; use an external pooler.

Self-check

1. Why are PostgreSQL connections costly? → Handshake + auth + a forked server process per connection.
2. What breaks in PgBouncer transaction mode? → Session-level state such as plain `SET` and session advisory locks.

14.13 ORMs vs raw SQL

What it is

An **ORM** (Object-Relational Mapper) maps tables to classes and rows to objects, and generates SQL for you: Hibernate/JPA (Java), SQLAlchemy and the Django ORM (Python), Entity Framework (.NET), ActiveRecord (Ruby), Prisma (TypeScript). In between sit **query builders** and **mappers** — jOOQ, MyBatis, Knex, SQLAlchemy Core — which help write SQL without hiding it. The mismatch between object graphs and tables is called the **object-relational impedance mismatch**.

Trade-offs

	ORM	Raw SQL / query builder
CRUD (create, read, update, delete) code	Very little	More boilerplate (repetitive setup code)
Safety	Parameterized by default	Parameterized if you do it (always do)
Complex queries (windows, CTEs, pivots, JSONB, FTS)	Awkward or impossible → raw-SQL escape hatch	Full power of the database
Performance visibility	SQL hidden; easy to generate N+1 or huge joins	You see and tune every query
Portability	Easier to switch databases	Tied to a dialect (usually fine)
Learning curve	ORM concepts (sessions, lazy loading, dirty checking — the ORM tracking which objects changed) *plus* SQL	SQL
Schema changes	Often paired with migrations (versioned schema-change scripts: Alembic, Flyway, Liquibase — these work without an ORM too)	Same migration tools

The N+1 query problem

The classic ORM trap: load 100 orders (1 query), then touch `order.customer` for each one, and **lazy loading** fires one more query per order (100 queries). In SQL terms:

```
-- N+1: what lazy loading does
SELECT * FROM orders WHERE status = 'shipped';           -- 1 query
SELECT * FROM customers WHERE customer_id = 6;           -- + 1 per order ...
SELECT * FROM customers WHERE customer_id = 7;
-- fix: one join (ORM: eager loading, e.g. JOIN FETCH / joinedload / select_related)
SELECT o.*, c.first_name FROM orders o JOIN customers c USING (customer_id) WHERE o.status = 'shipped';
-- or one batched lookup: WHERE customer_id = ANY($1)   (ORM: batch fetching / selectinload / prefetch_related)
```

Spot it by logging SQL in development and counting queries per request. (Indexing side: Chapter 12.)

When to use which

- **ORM**: CRUD-heavy business apps, many simple entities, teams that value productivity — while still reading the SQL it generates.
- **Raw SQL / builder**: reporting, analytics, bulk jobs, anything using window functions, CTEs, JSONB or FTS, and hot paths (the most frequently run code) that need tuning.
- **Most real systems mix both**: ORM for everyday writes, hand-written SQL for the hard 10 %.

In the interview

30-second answer: "ORMs speed up CRUD and parameterize by default, but hide the SQL, which invites N+1 queries and inefficient joins. I use an ORM for simple entity work, drop to SQL or a query builder for reporting and performance-critical paths, log generated SQL, and fix N+1 with eager or batched loading."

- *How do you detect N+1?* → Count queries per request in logs/APM (application performance monitoring tools); a request running the same query shape 100 times is the tell.
- *Are ORMs immune to injection?* → Their generated queries are, but raw-SQL helpers with string concatenation are not.

Self-check

1. What is lazy loading? → Related objects are fetched only when first accessed — a hidden query each time.
2. Two fixes for N+1? → A join (eager loading) or one `IN` / `= ANY` batch query.

Chapter TL;DR

Topic	Remember
Functions	`sql` for simple, `plpgsql` for procedural; honest volatility; set-based beats per-row calls
Procedures	`CALL`, `INOUT`, can `COMMIT` in batches — only from auto-commit mode
Triggers	BEFORE modifies/rejects, AFTER reacts; audit with `to_jsonb(OLD/NEW)`; statement-level + transition tables for bulk; no slow external work
JSONB	`->` json, `->>` text, `@>` contains, GIN index; core fields stay columns
Arrays	1-based; `ANY`, `@>`, `&&`, `unnest`, `array_agg`; `= ANY($1)` for id lists from code
Full-text search	`tsvector @@ tsquery`, stemming, `websearch_to_tsquery`, `ts_rank`, weighted generated column + GIN
Pivot	`FILTER` / `CASE` aggregates; `crosstab` needs tablefunc; unpivot with `LATERAL (VALUES …)`
Security	Group roles, least privilege, `GRANT` / `REVOKE`; RLS `USING` (read) + `WITH CHECK` (write); owners/superusers bypass
Injection	Never concatenate input; bind parameters; allow-list + `%I` for identifiers
From code	psycopg `%s`, sqlite3 `?`, JDBC `?`; explicit transactions; `Decimal` / `BigDecimal`; batch/COPY; stream big results
Pooling	Reuse connections; small pools; `SET LOCAL`; PgBouncer for many clients
ORMs	Productive for CRUD, watch for N+1; raw SQL for complex/hot queries

Next: Chapter 15 — Scaling databases, which takes one server to many: partitioning, replication, sharding and failover.

Chapter 15 — Scaling Databases: Partitioning, Sharding, Replication and Recovery

A single PostgreSQL server goes a long way: one machine with plenty of **RAM** (the fast working memory a computer loses when powered off), fast **SSDs** (solid-state drives — disks with no moving parts, much faster than spinning hard drives) and good indexes (Chapter 12) can serve thousands of queries per second. Sooner or later, though, the data outgrows one disk, the traffic outgrows one **CPU** (the processor that executes instructions), or the business can't accept that one broken machine takes the whole product offline. This chapter is about what databases do when **one box is no longer enough**.

Two ideas carry most of the weight:

- **Partitioning / sharding**: split the data so each piece is smaller (more capacity, faster scans).
- **Replication**: copy the data so there is more than one of each piece (availability, read scaling, disaster recovery).

Then come the problems they create: copies disagree (consistency models, CAP), machines must agree (consensus, leader election), and failures must not lose committed data (logging, checkpoints, two-phase commit, failover, backups).

What you'll learn

Section	Topic	Runnable part
15.1	What a distributed database is, scale up vs scale out	—
15.2	Partitioning: horizontal/vertical, range/list/hash/composite/round-robin	Postgres declarative partitioning + pruning with `EXPLAIN`
15.3	Sharding, shard keys, consistent hashing	Python: keys moved when a shard is added
15.4	Distributed query processing	`postgres_fdw`: predicate/aggregate push-down
15.5	Replication: types, sync vs async, single/multi/leaderless	Python: quorum reads/writes
15.6	Consistency models	—
15.7	CAP and PACELC	—
15.8	Consensus (Paxos, Raft, PBFT, PoW) and leader election	Python: Bully vs Ring message counts
15.9	Fault tolerance: WAL, redo/undo, LSNs, checkpoints, 2PC/3PC	Postgres WAL/LSN demo, Python recovery + 2PC
15.10	High availability and failover	—
15.11	Data resilience: redundancy, mirroring, backups	—

> **Going deeper.** This chapter explains each idea once, at interview depth. For book-level depth see your DDIA notes — Replication, Partitioning, Trouble with distributed systems, Consistency and consensus — and for the design-interview pattern view the system-design-patterns notes: Storing data (primary-replica, sharding, consistent hashing, WAL), Consistency (2PC, saga, quorum, vector clocks), Scaling, Reliability.

All SQL files are in `code/` and run on the book's private PostgreSQL 14 server (`bash tools_run_sql.sh sql '15-*'`). `00_schema.sql` creates 12 customers in three regions (`NA`, `EU`, `APAC`) and 3,000 orders spread over 2022–2024. Python simulations run with the books' venv (`.venv-books/bin/python code/<file>.py`).

15.1 Distributed databases: the big picture

What it is

A **distributed database** stores its data on several machines (**nodes**), possibly in different **data centres** (buildings full of servers, power and network gear), but presents itself to applications as *one* database. Think of a library with branches across a city: you search one catalogue and borrow from whichever branch has the book, without knowing which building holds which shelf.

Term / characteristic	Meaning
Node	One server (physical or virtual) running a database process. **Cluster** = the group of nodes forming the database.
Transparency	The client doesn't need to know *where* a row is stored (**location transparency**) or *how many copies* exist (**replication transparency**).
Fault tolerance	The system keeps working when individual nodes fail, because data is copied elsewhere.
Concurrency	Many transactions on many nodes run at once without corrupting shared data (the concurrency control of Chapter 13, now across machines).
Scalability	Capacity grows by adding nodes.
Network partition	Some nodes can't talk to others because the network between them failed. Don't confuse it with *data* partitioning (15.2).

Scale up vs scale out

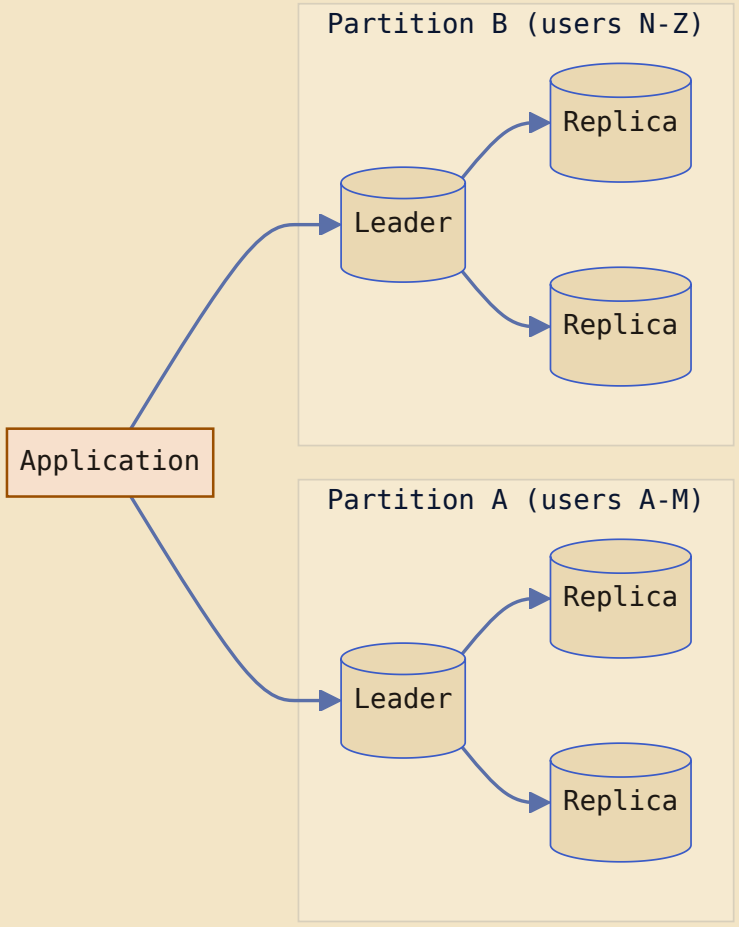
	Vertical scaling (scale up)	Horizontal scaling (scale out)
How	Bigger machine: more CPU, RAM, faster disks	More machines, each holding part of the data/load
Example	Move Postgres from 8 to 64 cores for Black Friday	Add nodes to a Cassandra or CockroachDB cluster as users grow
Limit	The biggest machine you can buy (and its price)	Coordination cost between nodes
Failure	Still one machine = one point of failure	Survives node loss if data is replicated
App changes	None	Often needed (shard keys, cross-node queries)

Distributed databases are built for **scale-out**. Scale-up is simpler and usually comes first — many companies run a large Postgres or MySQL **primary** (the one server that accepts writes) plus **read replicas** (copies that serve only reads) for years before they shard. For a gentle introduction to these system-design building blocks see System Design Fundamentals.

Benefits and challenges

Benefits	Challenges
High availability (the service stays usable almost all the time) — other copies serve if a node or region dies	**Consistency** of copies written in different places (15.5–15.7)
Lower latency (latency = the time one request takes, start to finish) — data near its users (Singapore reads a Singapore node)	**Network latency** — milliseconds per cross-region hop
Load spreading — no node takes every request	**Failure detection** — silent may mean dead or slow (15.8–15.10)
Capacity grows with node count	**Operational complexity** — harder debugging and upgrades

How scale-out works: partition *and* replicate



Partitioning gives each node a *subset* of the data (capacity, parallel work); **replication** stores each subset on *several* nodes (fault tolerance, read capacity). Most real systems do both. *Example:* a food-delivery app in 20 countries partitions menus and orders by country (queries are local, and data-residency laws such as **GDPR** (the EU's General Data Protection Regulation, which restricts where and how personal data is stored) are easier to meet), replicates each partition to three **availability zones** (separate data centres in one cloud region, with independent power and networking), and adds nodes to the busiest cities on New Year's Eve.

Self-check

1. *Which two techniques does almost every distributed database combine?* Partitioning (split) and replication (copy).

2. *Why try scale-up first?* No application changes and no distributed-systems problems; its limits are cost and a **single point of failure** (one component whose failure takes everything down).

15.2 Partitioning

What it is

Partitioning splits one large table into smaller pieces (**partitions**) that are stored and managed separately while queries still see one logical table — like a filing cabinet with one drawer per year. Its goals: **faster queries** (a query for March 2023 reads only the 2023 partition — **partition pruning**), **easier maintenance** (archive a year by detaching one partition), **scalability** (partitions can live on different nodes — then it's *sharding*, 15.3), and **fault tolerance** (each partition can be replicated independently).

Horizontal vs vertical

Aspect	Horizontal (split rows)	Vertical (split columns)
Each piece has	All columns, some rows (2023 orders, 2024 orders)	Some columns, all rows; pieces share the primary key (1:1)
Speeds up	Queries that filter on the partition key	Queries that need only a few columns
Cost	Routing; queries without the key hit every piece	Joins to reassemble a full row
Typical use	Huge event/order/log tables, distributed DBs	Wide tables with big text/JSON/blob columns

A vertical split (`code/05_vertical_partitioning.sql`) keeps listing-page columns (`product_id`, `name`, `price`) in `product_core` and the long `description` / `spec_sheet` text in `product_details`, whose primary key also references `product_core`. With 1,000 products, `pg_total_relation_size` shows:

```
 core_size | details_size
-----------+--------------
 128 kB    | 1672 kB
```

The listing page now reads 128 kB instead of about 1.8 MB; a view (`product_full`) joins the halves back when needed. Postgres already moves very large values out of line by itself (**TOAST**, Chapter 12), so a manual split pays off most for medium-sized, rarely read columns.

Partitioning strategies

Strategy	Rule	Good for	Watch out for
Range	Key in an interval, e.g. `2023-01-01 ≤ date < 2024-01-01`	Time data, "last N days", archiving	**Hot partition**: all new writes hit the newest range
List	Key in a set of values, e.g. `region IN ('EU')`	Regions, tenants, data-residency rules	Uneven sizes; new values need a partition (or DEFAULT)
Hash	`hash(key) mod N`	Even spread with no natural range (`order_id`, `user_id`)	Range queries can't be pruned; changing N moves data
Composite	Two levels, e.g. range by month, then hash by server	High-volume time series	More partitions to manage
Round-robin	Row 1 → P1, row 2 → P2, row 3 → P3, row 4 → P1…	Perfectly even bulk loads and parallel scans	Can't find a row by key — every lookup scans all partitions

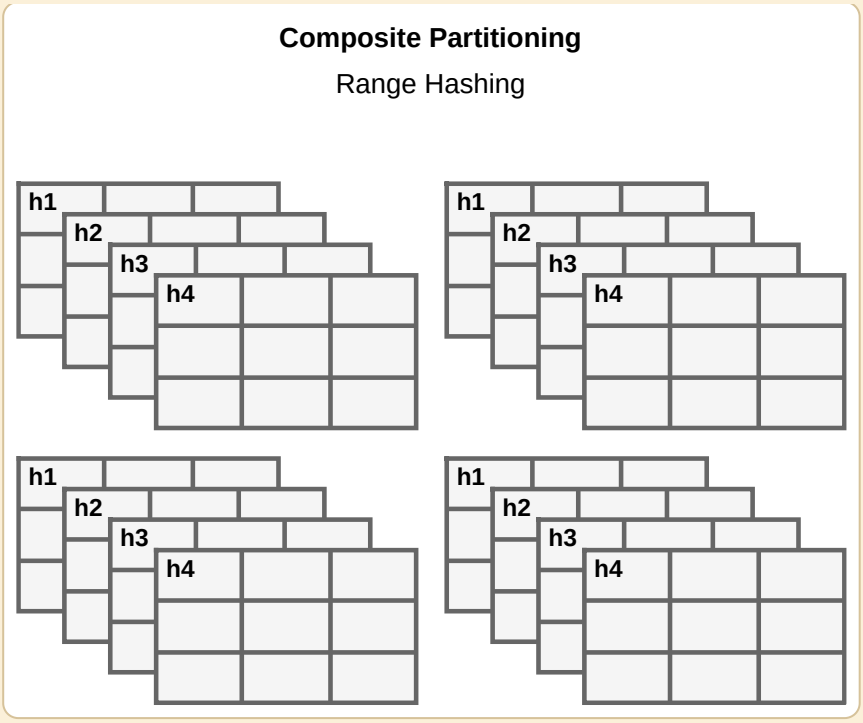


Figure — Composite (range + hash) partitioning: each range partition is itself split into hash buckets h1–h4 (course diagram).

Postgres supports range, list and hash natively, and composite by nesting them; it has no round-robin (that appears in warehouse systems, e.g. Azure Synapse's `ROUND_ROBIN` distribution). **To choose**, look at your most common and most expensive queries: the partition key should be in their `WHERE` clause so pruning works, it should spread data reasonably evenly, and adding/dropping partitions should stay simple.

How it works in PostgreSQL (declarative partitioning)

Declarative partitioning arrived in PostgreSQL 10 (hash partitioning, default partitions and primary keys on partitioned tables in 11). You create a **parent** with `PARTITION BY RANGE | LIST | HASH (columns)` — it stores no rows — and each **partition** with `CREATE TABLE … PARTITION OF parent FOR VALUES …`. Inserts into the parent are routed to the right partition (a row that fits none is rejected unless a `DEFAULT` partition exists), and a `WHERE` on the partition key lets the **planner** (the part of Postgres that decides how to execute a query) **prune** partitions that can't match.

Range partitioning, with pruning

`code/01_range_partitioning.sql`:

```
CREATE TABLE orders_by_year (
  order_id    int  NOT NULL,
  customer_id int  NOT NULL,
  order_date  date NOT NULL,
  amount      numeric(10,2) NOT NULL,
  PRIMARY KEY (order_id, order_date)          -- a PK must include the partition key
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2022 PARTITION OF orders_by_year FOR VALUES FROM ('2022-01-01') TO ('2023-01-01');
CREATE TABLE orders_2023 PARTITION OF orders_by_year FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
CREATE TABLE orders_2024 PARTITION OF orders_by_year FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
CREATE TABLE orders_other PARTITION OF orders_by_year DEFAULT;   -- anything outside the ranges

INSERT INTO orders_by_year SELECT * FROM orders;
INSERT INTO orders_by_year VALUES (9001, 1, '2025-03-15', 99);   -- no 2025 partition -> DEFAULT

SELECT tableoid::regclass AS partition, count(*) AS rows,
       min(order_date) AS first_day, max(order_date) AS last_day
FROM orders_by_year GROUP BY 1 ORDER BY 1;
```

```
  partition   | rows | first_day  |  last_day
--------------+------+------------+------------
 orders_2022  | 1016 | 2022-01-01 | 2022-12-31
 orders_2023  |  990 | 2023-01-01 | 2023-12-31
 orders_2024  |  994 | 2024-01-01 | 2024-12-31
 orders_other |    1 | 2025-03-15 | 2025-03-15
```

`FROM … TO …` has an **inclusive** lower and **exclusive** upper bound, so `2023-01-01` belongs to `orders_2023`; `tableoid::regclass` is a hidden column naming the physical table holding each row; and the PK includes `order_date` because each partition enforces uniqueness only on its own rows.

Pruning at plan time, versus a query that doesn't filter on the key:

```
EXPLAIN (COSTS OFF) SELECT sum(amount) FROM orders_by_year
WHERE order_date >= '2023-03-01' AND order_date < '2023-04-01';
EXPLAIN (COSTS OFF) SELECT sum(amount) FROM orders_by_year WHERE customer_id = 7;
```

```
 Aggregate
   ->  Seq Scan on orders_2023 orders_by_year
         Filter: ((order_date >= '2023-03-01'::date) AND (order_date < '2023-04-01'::date))

 Aggregate
   ->  Append
         ->  Seq Scan on orders_2022 orders_by_year_1
               Filter: (customer_id = 7)
         ...   (same for orders_2023, orders_2024 and orders_other)
```

`Append` means "scan each child and concatenate" — all four partitions are read. **Partitioning only helps queries that filter on the partition key.** When the value is known only at run time (a **prepared-statement** parameter — a prepared statement is a query parsed and planned once, then run many times with different values plugged into `$1`, `$2`…), Postgres 11+ prunes during execution:

```
SET plan_cache_mode = force_generic_plan;
PREPARE by_day(date) AS SELECT count(*) FROM orders_by_year WHERE order_date = $1;
EXPLAIN (COSTS OFF) EXECUTE by_day('2023-05-05');
```

```
 Aggregate
   ->  Append
         Subplans Removed: 3
         ->  Seq Scan on orders_2023 orders_by_year_1
               Filter: (order_date = $1)
```

Maintenance is where range partitioning shines — archiving a year is a **metadata** change (an edit to the database's own bookkeeping about which tables exist, not to the rows), not a giant `DELETE`:

```
ALTER TABLE orders_by_year DETACH PARTITION orders_2022;        -- instant; now a plain table
DELETE FROM orders_other WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01';
CREATE TABLE orders_2025 PARTITION OF orders_by_year FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
INSERT INTO orders_by_year VALUES (9001, 1, '2025-03-15', 99);
```

Afterwards the parent holds `orders_2023` (990), `orders_2024` (994) and `orders_2025` (1), and the detached `orders_2022` still has its 1,016 rows as an ordinary table. The `DELETE` is needed because Postgres refuses to create `orders_2025` while the DEFAULT partition still holds 2025 rows — better to create partitions ahead of time (a scheduled job or the `pg_partman` **extension** — an add-on package installed into Postgres with `CREATE EXTENSION`). `DETACH PARTITION … CONCURRENTLY` (Postgres 14+) avoids blocking queries on the parent.

List, hash and composite

`code/02_list_partitioning.sql` splits customers by region (`FOR VALUES IN ('NA')`, `('EU')`, `('APAC')`):

```
   partition    |         customers
----------------+---------------------------
 customers_na   | Ben, Elena, Grace, Jack
 customers_eu   | Chloe, Dmitri, Ines, Liam
 customers_apac | Asha, Farid, Hiro, Kavya

 Seq Scan on customers_eu customers_by_region
   Filter: (region = 'EU'::text)

NOTICE:  rejected: no partition of relation "customers_by_region" found for row
```

The last line comes from inserting a `'LATAM'` customer with no matching partition and no `DEFAULT` — a useful safety net that stops data landing in the wrong jurisdiction.

`code/03_hash_partitioning.sql` spreads orders over four partitions (`FOR VALUES WITH (MODULUS 4, REMAINDER 0..3)`): 730 / 752 / 774 / 744 rows — close to even. `WHERE order_id = 2024` prunes to one partition (`Seq Scan on orders_h2`), but `order_id BETWEEN 100 AND 110` produces an `Append` over all four, because hashing scatters neighbouring values.

`code/04_composite_partitioning.sql` partitions server logs by month, and each month is itself declared `PARTITION BY HASH (server_id)` with two leaves (`MODULUS 2, REMAINDER 0/1`):

```
 leaf_partition  | rows | servers
-----------------+------+---------
 logs_2024_01_h0 |  403 | 1,2
 logs_2024_01_h1 |  804 | 3,4,5,6
 logs_2024_02_h0 |  265 | 1,2
 logs_2024_02_h1 |  529 | 3,4,5,6

 Aggregate
   ->  Seq Scan on logs_2024_02_h1 server_logs
         Filter: ((logged_at >= '2024-02-10 00:00:00+00'::timestamp with time zone) AND
                  (logged_at < '2024-02-11 00:00:00+00'::timestamp with time zone) AND (server_id = 3))
```

A filter on both keys prunes to **one leaf of four**. The 2-vs-4 server split also teaches that **hashing evens things out only with many distinct key values** (six servers into two buckets is lumpy). `pg_partition_tree('server_logs')` lists the tree: parent (level 0), months (level 1), leaves (level 2).

Trade-offs and dialects

Postgres limits to know: unique/PK constraints must include every partition-key column; planning time grows with partition count (hundreds are fine, many thousands are heavy); an index on the parent becomes a *local* index on each partition — there is no global index; an `UPDATE` that changes the key moves the row (PG 11+); and `enable_partitionwise_join` / `_aggregate` (off by default) process matching partitions pairwise.

Other dialects *(not run)*: **MySQL** declares `PARTITION BY RANGE/LIST/HASH/KEY (...) (PARTITION p0 VALUES LESS THAN (...), ...)` inside `CREATE TABLE`, and every unique key must include the partitioning columns; **SQL Server** uses a partition function plus a partition scheme (range only); **Oracle** adds interval partitioning (auto-created ranges); **SQLite** has none.

Common mistakes

- A key the hot queries don't filter on (every query scans every partition); partitioning tables under a few million rows at all — a good index is simpler.
- Forgetting next month's partition, so inserts fail or pile into DEFAULT; thousands of tiny partitions; expecting hash partitioning to speed up range scans.

In the interview

30-second answer: "Partitioning splits a big table into physical pieces by a key: range for time, list for categories, hash for even spread, composite to combine. The win is pruning — the planner skips partitions the `WHERE` clause excludes — plus cheap maintenance like dropping a month

instantly. It only helps queries that filter on the key, and in Postgres unique keys must include it."

- *Range vs hash for an `events` table?* Range by time if queries are "recent events" and old data is archived; hash by `user_id` if queries are per user and write load must be spread.
- *Hot-partition problem?* Range-by-time sends all current writes to the newest partition; mitigate with time + hash composite.

Self-check

1. *Why is `2023-01-01` in `orders_2023`?* Range upper bounds are exclusive.
2. *What does `Subplans Removed` mean?* Partitions pruned at execution time because the value was a parameter.
3. *Which strategy can't find a row by key without scanning everything?* Round-robin.

TL;DR

Partition by the column your big queries filter on: range for time, list for categories, hash for even spread, composite to combine. Check pruning with `EXPLAIN`; unique keys must include the partition key.

15.3 Sharding

What it is

Sharding is horizontal partitioning where each partition — a **shard** — lives in a **separate database server**, a complete, independent database holding a subset of the rows. A **shard key** decides which shard owns a row, and a **router** sends each query to the right shard(s). Example: `order_id` 1–1,000,000 on shard A and 1,000,001–2,000,000 on shard B; `WHERE order_id = 42` goes only to A.

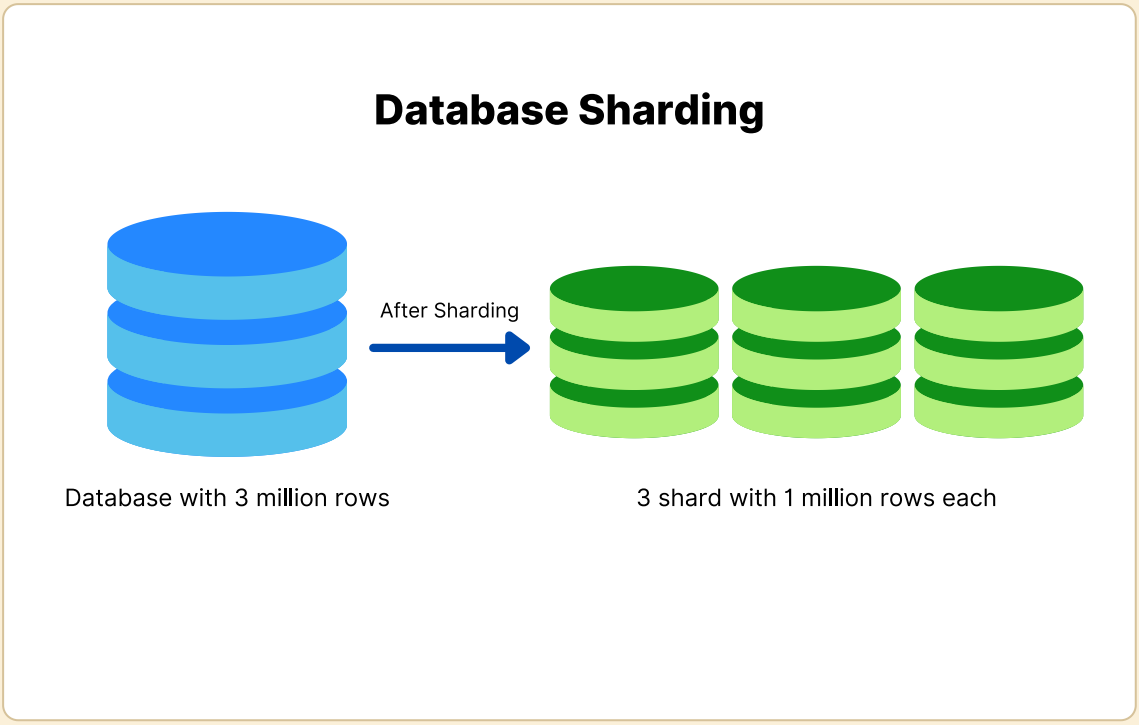


Figure — One database with millions of rows split into three shards, each an independent database (course diagram).

How it works

1. Pick a **shard key** (`user_id`, `tenant_id`).
2. Choose a **mapping** from key to shard: a range table, a hash, or a lookup directory.

3. A **router** applies it: the application, a **proxy** (a middle server that receives client connections and forwards them to the right database — Vitess for MySQL; pgcat/pgdog for Postgres), or the database itself (Citus, MongoDB `mongos`, CockroachDB).
4. Single-shard queries go straight to one shard; queries without the key are **scattered** to all shards and **gathered** — slower.
5. Each shard is **replicated** (15.5) — sharding alone doesn't remove single points of failure; a shard without replicas loses its slice of data when it dies.

Choosing the shard key (the most important decision)

Property	Why	Example
Spreads data and load evenly	Avoids a **hotspot** — one overloaded shard sets the speed of the whole system	`user_id` (millions of values), not `country`
Matches the main query pattern	Most queries hit exactly one shard	B2B SaaS sharded by `tenant_id`
Keeps related data together	Joins and transactions stay on one shard (**co-location**)	A user's orders on the user's shard
Rarely changes / minimises rebalancing	Changing the key or shard count means moving data	Immutable ids, not email addresses

Typical choices: `user_id` for social apps, `tenant_id` for SaaS, time ranges for time-series (but then all current writes hit the newest shard).

Compared with partitioning inside one server, sharding needs a router instead of the planner, turns cross-piece queries into network scatter-gather and cross-piece transactions into distributed commits (2PC, 15.9), and lets one shard fail while the others keep serving (**fault isolation**). Its benefits: **write scaling**, **smaller working sets** (the **working set** is the data a system touches often; here each shard's hot data fits in RAM), fault isolation and more total capacity.

Rebalancing: why `hash % N` hurts and consistent hashing helps

With naive `hash(key) % N`, changing N from 4 to 5 changes the answer for most keys. (A **hash** function turns any key into a fixed-size, random-looking number; the same key always gives the same number.) **Consistent hashing** puts shards and keys on a ring (0 to 2^{32}); each key belongs to the next shard clockwise, so a new shard only takes keys from its neighbour. **Virtual nodes** give each shard many small points on the ring to even out the load (Cassandra, Riak and Dynamo-style systems). `code/consistent_hashing.py` places 100,000 keys, then adds a fifth shard:

The ring is a sorted list of `(hash, shard)` points; `lookup` uses `bisect` (Python's binary search over a sorted list) to find the first point clockwise of `h(key)`:

```
def lookup(self, key):
    i = bisect.bisect(self.hashes, h(key)) % len(self.points)  # next point clockwise
    return self.points[i][1]
```

```
hash % N                    moved  79.7%   max/min shard load 1.01
ring, 1 point per shard     moved   2.8%   max/min shard load 10.44
ring, 200 vnodes per shard  moved  17.1%   max/min shard load 1.28
ideal fraction to move when going 4 -> 5 shards: 1/5 = 20.0%
```

- `hash % N` balances perfectly but moves **80%** of keys — a huge migration.
- One ring point per shard moves little only because the new shard landed on a tiny arc; the biggest shard holds 10× the smallest.
- **200 vnodes** move about 17% (near the ideal 20%) and stay balanced — why real systems use them.

Another approach: a **fixed number of logical partitions** (say 1,024) mapped to servers by a directory; adding a server moves whole partitions and nothing is re-hashed (Citus shards, Vitess keyspace ranges and Kafka partitions work roughly this way). More in DDIA ch.6 — rebalancing.

Sharding Postgres in practice. Core Postgres has no built-in sharding: use application-level routing, the **Citus** extension (`create_distributed_table('orders','customer_id')` co-locates tables sharing the key), partitions whose leaves are `postgres_fdw` foreign tables (15.4), or a Postgres-compatible distributed SQL database (CockroachDB, YugabyteDB — Chapter 16).

Common mistakes

- **Monotonic key + range sharding** (auto-increment id, timestamp): every new row hits the last shard.
- **Low-cardinality keys** (cardinality = the number of distinct values; country, status): too few shards, some huge.
- **Sharding too early**: read replicas, caching (Caching patterns) and a bigger machine are cheaper first steps.
- **Forgetting cross-shard needs**: global unique constraints, cross-shard joins and all-user reports become hard.

In the interview

> **30-second answer:** "Sharding puts horizontal partitions on separate servers so writes and storage scale out. The shard key should spread load evenly, match the main access pattern so most queries hit one shard, and keep related rows together. Queries without the key become scatter-gather and cross-shard transactions need 2PC. For rebalancing I'd use consistent hashing with virtual nodes, or fixed logical partitions mapped to nodes, never `hash mod N`."

- *A celebrity user overloads one shard?* Split that key (`user123#0..9`) to spread writes and merge on read, or cache their reads.
- *Unique ids across shards?* UUIDv7 (a **UUID** is a 128-bit random-looking id that can be generated anywhere without coordination; v7 starts with a timestamp so ids sort by time), Snowflake-style ids (timestamp + machine id + sequence), or per-shard sequences with disjoint ranges.

Self-check

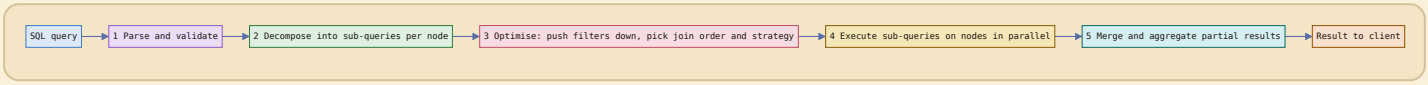
1. *Why is `hash % N` bad for resharding?* Changing N remaps most keys (≈80% going from 4 to 5).
2. *What do virtual nodes fix?* Uneven arcs on the ring, hence uneven load.

15.4 Distributed query processing

What it is

When a query's rows are spread over several nodes, the database must decide **which nodes to ask, what to ask each one, and how to combine the answers** — moving as little data as possible, using nodes in parallel, and still returning a correct answer.

How it works



Step 1 includes **data localisation** (finding which nodes hold each table); step 3 minimises network traffic and time given data location, link speeds and node load; step 5 merges partial results (union, final sort, final sum).

Technique	Idea
Query fragmentation	Independent fragments run simultaneously on different nodes.
Predicate push-down	Apply `WHERE` on the node storing the data, so only matching rows travel.
Aggregate push-down	Each node computes partial `count` / `sum`; only small results travel.
Local (co-located) joins	Both tables sharded on the join key → each node joins its own pieces.
Distributed joins	Otherwise **broadcast** the small table, **repartition (shuffle)** both sides by the join key, or **semi-join**: send only join keys first, then fetch matching rows.
Smart sharding, caching	Keep common queries on one node; reuse frequent intermediate results.

Challenges: data localisation, network latency, a far larger plan space, nodes failing mid-query, and consistency when pieces come from replicas at slightly different moments.

Seeing it in Postgres with `postgres_fdw`

`postgres_fdw` (a **foreign data wrapper**) lets one Postgres query tables on another; the remote tables appear locally as **foreign tables**. `code/06_distributed_query_fdw.sql` points it back at the same private server: schema `shard_b` plays the "remote node", and `remote.orders` is the foreign table:

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;
CREATE SERVER node_b FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host '/tmp', port '5544', dbname 'ch_15');          -- in real life: another host
-- ... user mapping; shard_b.orders created as a copy of orders ...
IMPORT FOREIGN SCHEMA shard_b LIMIT TO (orders) FROM SERVER node_b INTO remote;

EXPLAIN (VERBOSE, COSTS OFF) SELECT order_id, amount FROM remote.orders WHERE amount > 500;
EXPLAIN (VERBOSE, COSTS OFF) SELECT customer_id, count(*), sum(amount) FROM remote.orders GROUP BY 1;
```

```
 Foreign Scan on remote.orders
   Output: order_id, amount
   Remote SQL: SELECT order_id, amount FROM shard_b.orders WHERE ((amount > 500::numeric))

 Foreign Scan
   Output: customer_id, (count(*)), (sum(amount))
   Relations: Aggregate on (remote.orders)
   Remote SQL: SELECT customer_id, count(*), sum(amount) FROM shard_b.orders GROUP BY 1
```

The `Remote SQL` line shows what was pushed down: the filter (only two columns and matching rows travel) and the whole `GROUP BY` (only 12 rows come back). A **cross-node join** — local `customers` in the USA joined to remote `orders` — returns the right answer (Ben 250 orders / 64,750.00, Grace 250 / 65,000.00, Jack 250 / 64,750.00), but its plan reveals a weakness (a **Hash Join** builds an in-memory lookup table from one side, then probes it with each row of the other):

```
Hash Join
  Hash Cond: (o.customer_id = c.customer_id)
  ->  Foreign Scan on remote.orders o
        Remote SQL: SELECT order_id, customer_id FROM shard_b.orders
  ->  Hash
        ->  Seq Scan on public.customers c
              Filter: (c.country = 'USA'::text)
```

The remote query has **no WHERE**: all 3,000 orders cross the network and only 750 survive. The **semi-join** fix finds the USA customer ids locally first and ships only those:

```
EXPLAIN (VERBOSE, COSTS OFF)
SELECT o.order_id, o.customer_id FROM remote.orders o
WHERE o.customer_id = ANY (ARRAY(SELECT customer_id FROM customers WHERE country = 'USA'));
```

```
Foreign Scan on remote.orders o
  Output: o.order_id, o.customer_id
  Remote SQL: SELECT order_id, customer_id FROM shard_b.orders WHERE ((customer_id = ANY ($1::integer[])))
  InitPlan 1 (returns $0)
    ->  Seq Scan on public.customers
          Filter: (customers.country = 'USA'::text)
```

The `InitPlan` (a sub-query Postgres runs once before the main query) runs locally; its three ids travel as `$1`, and only the 750 matching orders come back. Distributed **optimisers** (the components that choose the cheapest execution plan — Citus, CockroachDB, Spark, Trino) make such choices automatically from table sizes and data location.

Common mistakes

- Wrapping remote columns in functions the remote side can't evaluate, which blocks push-down — check `Remote SQL` in `EXPLAIN VERBOSE`.
- Joining two large tables sharded on *different* keys in the hot path — co-locate or denormalise.
- `ORDER BY … LIMIT` across shards without per-shard limits — each shard should return its own top N first.

In the interview

30-second answer: "The **coordinator** (the node that receives the query and orchestrates the others) localises the data, splits the query into per-node fragments, pushes filters and partial aggregates to the data, runs fragments in parallel and merges results. Network transfer dominates, so the optimiser prefers co-located joins, broadcasts small tables and uses semi-joins to ship keys instead of rows."

- *Why can't `AVG` be pushed down as `AVG`?* An average of averages is wrong when groups differ in size; push down `SUM` and `COUNT`, divide at the end.

Self-check

1. *Where do you see what `postgres_fdw` pushed down?* The `Remote SQL:` line of `EXPLAIN VERBOSE`.
2. *What does a semi-join ship first?* Only the join keys from the smaller side.

15.5 Replication

What it is

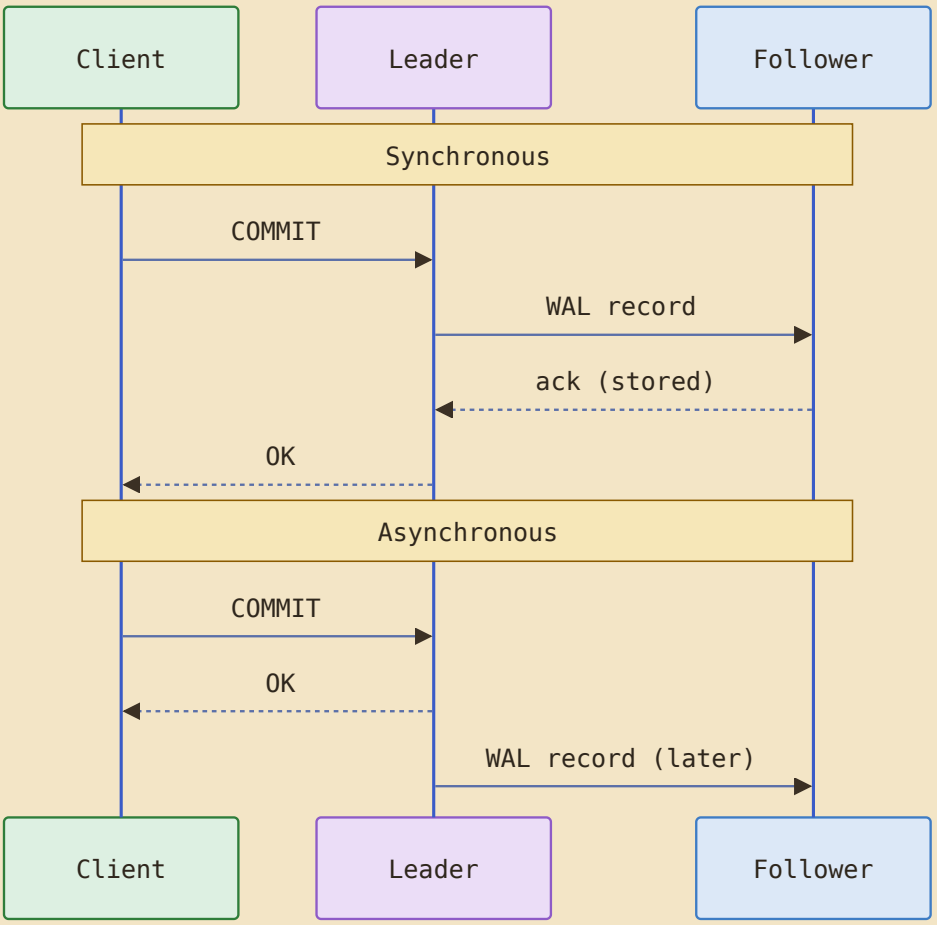
Replication keeps **copies of the same data on several nodes** (**replicas**) — like keeping photos on your laptop, an external drive and the cloud. Benefits: **high availability** (a replica takes over when the primary dies), **read scaling** (reports read from replicas), **lower latency** (a Frankfurt replica serves Europe), **disaster recovery** (**DR** — getting back to work after losing a whole site; here, a copy in another region). Costs: keeping replicas in sync, extra storage and network traffic, and **conflicts** when several copies accept writes.

Kinds of replication (what gets copied, and when)

Type	How it works	Trade-off	Typical use
Full	Every replica holds the whole database	Any replica answers anything / storage × replicas	DR copies
Transactional	Each committed transaction streams to replicas in commit order, near real time	Close and ordered / continuous overhead	OLTP (day-to-day transactional workload) read replicas, Postgres logical replication
Snapshot	Periodically copy a full snapshot	Very simple / only as fresh as the last copy	Nightly reporting copies
Merge	Copies change independently (even offline), then merge by rules	Works disconnected / conflicts	Field and mobile apps
Key-based incremental	Copy rows whose `updated_at`/version is newer than the last sync	Moves little data / **misses hard deletes** (rows physically removed) unless tombstoned (marked deleted instead)	ETL (extract-transform-load pipelines) into warehouses, IoT (internet-connected device) sync

A retailer might mix them: orders and stock transactionally, the catalogue as nightly snapshots, a full replica in another region.

Synchronous vs asynchronous



Synchronous replication returns a commit only when the sync replica(s) also have it: no acknowledged commit is lost, but every write pays a network **round trip** (a message there plus the reply back) and commits **block** if that replica is down (use it for money and orders). **Asynchronous** returns once the

leader has it: fast, but the newest commits may be lost if the leader dies, and replicas lag (**replication lag**) — fine for read replicas, cross-region copies and analytics. Most systems are **semi-synchronous**: wait for *one* (or a quorum of) replicas, the rest async. **In PostgreSQL**, physical **streaming replication** ships the **WAL** (write-ahead log — the database's append-only record of every change, 15.9) to **standbys** (replica servers ready to take over), async by default; `synchronous_standby_names` (e.g. `'ANY 1 (s1, s2)'`) makes commits wait, and `synchronous_commit` sets how far: `remote_write` (received), `on` (flushed), `remote_apply` (applied, so standby reads see it). **Logical replication** (`CREATE PUBLICATION` / `SUBSCRIPTION`, PG 10+) streams row changes per table.

Replication topologies

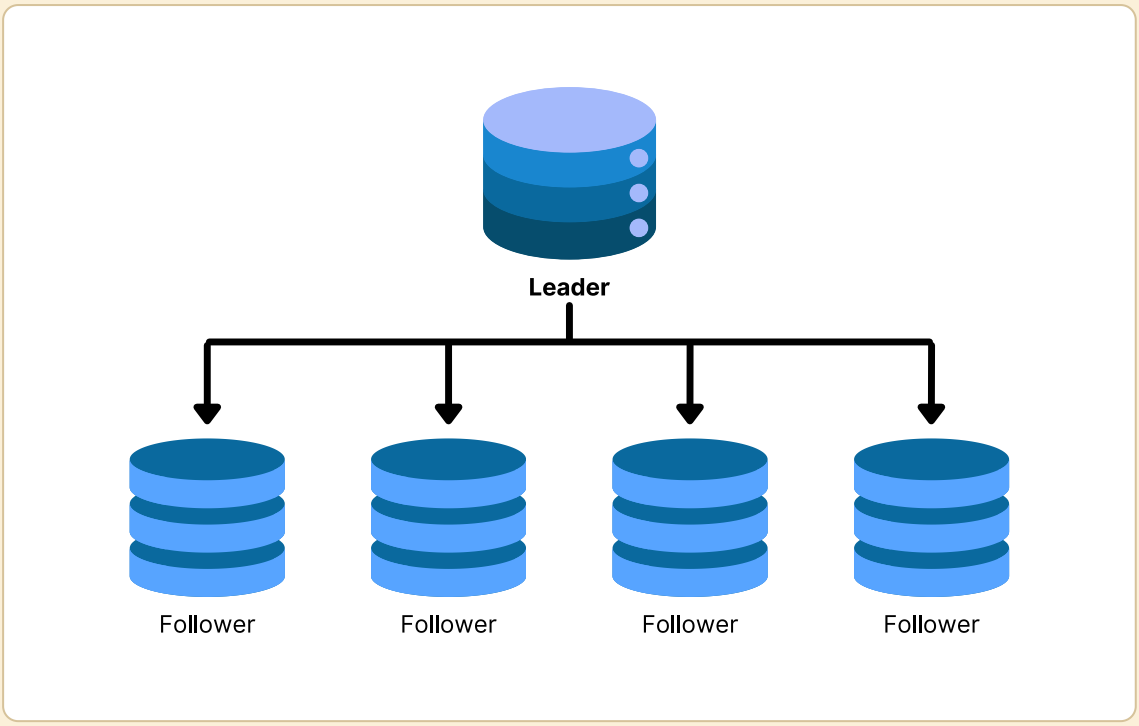


Figure — Single-leader: one leader takes writes and streams them to followers, which serve reads (course diagram).

1. Single-leader (primary/replica; formerly master/slave). All writes go to the leader, which records them in its **replication log / WAL** and ships the log to **followers** (sync or async). Reads may use any node. If the leader fails, **failover** promotes a follower (15.10). Simple, no write conflicts, strong consistency possible; but write throughput is one machine's and async followers can be stale. Examples: PostgreSQL, MySQL, SQL Server Always On, MongoDB replica sets.

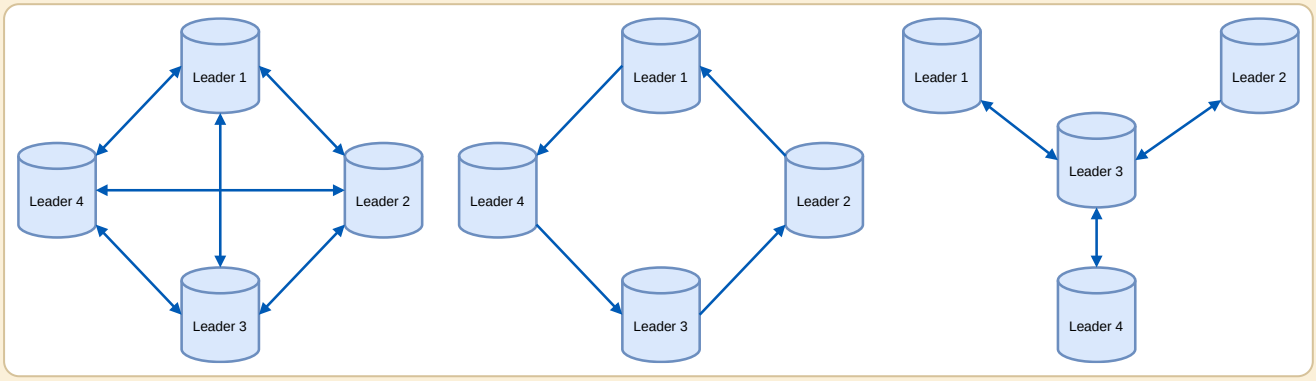


Figure — Multi-leader: several leaders accept writes and replicate to each other (course diagram).

2. Multi-leader (active-active). Several nodes (often one per region) accept writes and replicate to each other. The same row changed at two leaders at once is a **conflict**, resolved by **last-write-wins** (by timestamp — silently drops one write), custom merge code, or **CRDTs** (conflict-free replicated data types, which merge automatically). If one leader fails, clients switch to another. Used for multi-region writes, offline apps (each device is a leader) and collaborative editing. Examples: MySQL Group Replication (multi-primary), Postgres with BDR/pgEdge, CouchDB.

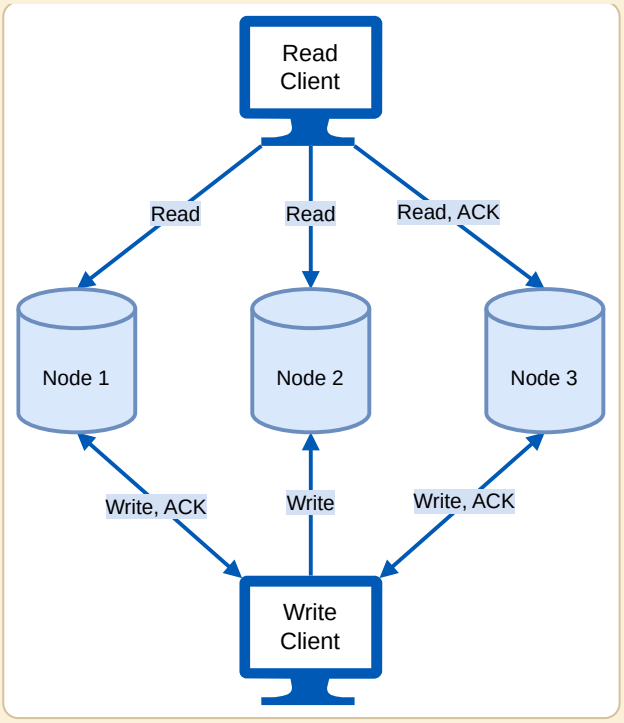


Figure — Leaderless: the client writes to and reads from several nodes directly (course diagram).

3. Leaderless (Dynamo-style). The client (or a coordinator for it) sends each write to all **N** replicas and succeeds once **W** acknowledge; reads ask **R** replicas and keep the newest version. If **R + W > N**, every read set overlaps every write set (the **quorum condition**). Stale replicas are fixed by **read repair** (the reader writes the newest value back), background **anti-entropy** (Merkle-tree comparison — a **Merkle tree** is a tree of hashes where each parent hashes its children, so two replicas can find differing data by comparing a few hashes) and **hinted handoff** (writes for a down node are held and delivered later). Concurrent writes are detected with **version vectors / vector clocks** (a per-node list of counters attached to each value; comparing two lists tells whether one write happened after the other or they were concurrent). Examples: Cassandra, ScyllaDB, Riak, Amazon's original Dynamo (the DynamoDB *service* actually uses a leader per partition). Leaderless systems use **quorums, not consensus** — no Paxos/Raft per write — so they stay available but guarantee less.

`code/quorum.py` simulates N = 3: in each trial a new version reaches only W random replicas, then a read asks R random replicas, keeps the highest version, and counts as stale if that is still the old value:

```
N = 3 replicas, 10,000 trials per setting
 W  R  R+W>N  stale reads
 1  1  False    66.4%
 1  2  False    34.0%
 1  3  True      0.0%
 2  1  False    33.8%
 2  2  True      0.0%
 ...  (every remaining row has R+W>N and 0.0%)

N=3, W=2, R=2: writes and reads still succeed with 1 replica down (2 of 3 answer); with 2 down they fail -> the
system chooses consistency.
```

Every setting with R + W > N had **zero** stale reads. N=3, W=2, R=2 survives one failed replica — Cassandra's `QUORUM` level; `ONE` is fast but possibly stale. Quorums still aren't full **linearizability** (the strongest guarantee, defined in 15.6): sloppy quorums (writes accepted by stand-in nodes when the usual ones are down), concurrent writes and clock-based last-write-wins can produce anomalies (DDIA ch.5).

Choosing a topology: correctness first → single-leader with a sync replica; writes in several regions → multi-leader with a conflict plan; must stay writable through failures and eventual is acceptable → leaderless with quorums.

Replication lag in practice. A user updates their profile, the next page is served by a lagging replica, and the old version appears. Fixes: read your own recent writes from the leader, pin a session

to one replica, or wait until the replica's `pg_last_wal_replay_lsn()` reaches the write's LSN — the session guarantees of 15.6.

In the interview

> **30-second answer:** "Replication keeps copies for availability, read scaling and DR. Sync loses no acknowledged commits but slows writes; async is fast but can lose recent commits and serve stale reads, so most systems are semi-sync. Topologies: single-leader (simple, no conflicts), multi-leader (multi-region writes, conflicts), leaderless (quorums with R + W > N, read repair)."

- *A user doesn't see their own comment?* A lagging async replica served the read; use read-your-writes routing.
- *Last-write-wins downside?* Concurrent updates are silently lost, and **clock skew** (machines' clocks disagreeing slightly) can discard the newer one.

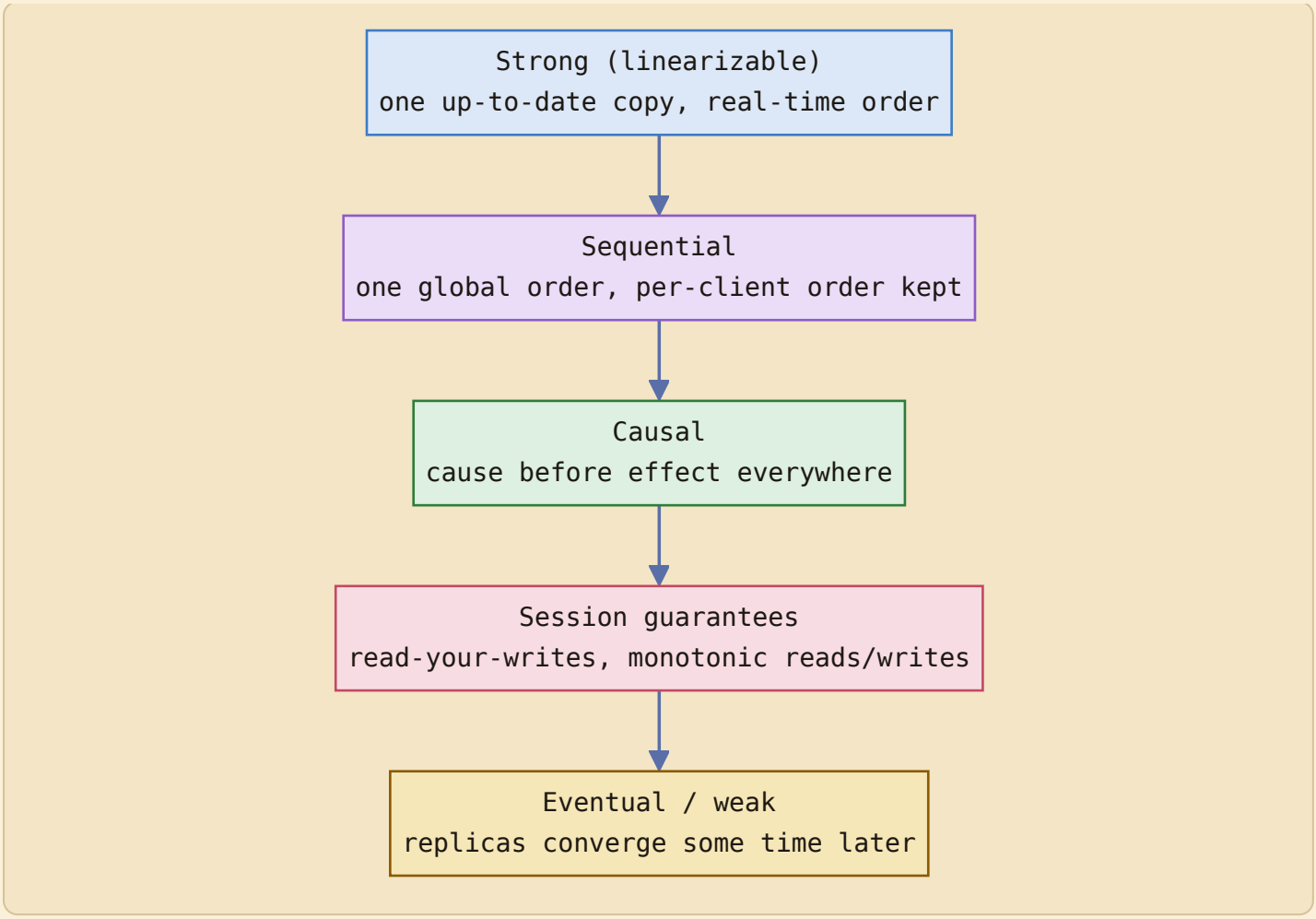
Self-check

1. *Which replication type can miss hard deletes?* Key-based incremental.
2. *N=5: which W and R guarantee overlap?* Any pair with R + W > 5, e.g. W=3, R=3.

15.6 Consistency models

What it is

With several copies, "what will a read return?" has more than one answer. A **consistency model** is the contract: which writes a read is guaranteed to see, and in what order clients observe operations. Stronger models are easier to program against but need more coordination (slower, less available); weaker ones are faster but push work onto the application.


Strong (linearizable)
one up-to-date copy, real-time order
Sequential
one global order, per-client order kept
Causal
cause before effect everywhere
Session guarantees
read-your-writes, monotonic reads/writes
Eventual / weak
replicas converge some time later

The models

Model	Guarantee	How it's built	Pros / cons	Use
Strong (linearizable)	Every read sees the **latest completed write**; the system acts like one copy with operations in **real-time** order (if A finishes before B starts, everyone sees A first)	Synchronous replication or consensus (15.8), atomic operations, one global order	Simplest to reason about, no stale reads / coordination latency, scales less, may be **unavailable** during partitions	Balances, stock at checkout, "is this username taken?", locks. etcd, ZooKeeper writes, Spanner, CockroachDB, single-node Postgres
Sequential	All clients see **the same order**, and each client's operations in its own program order — but not necessarily real-time order	A global sequencer	One agreed history / still needs coordination, no recency promise	CPU memory models, distributed shared memory; rare by name in databases
Causal	If A could have influenced B (same client did A then B, or B read A), **everyone sees A before B**; unrelated (**concurrent**) operations may appear in different orders	Track dependencies with **vector clocks**; delay an update until its dependencies are visible	Little coordination, works across regions, matches intuition / complex tracking, brief divergence	Replies after questions, chat, collaborative editing; MongoDB causally consistent sessions
Weak / eventual	Few promises; **eventual consistency**: if writes stop, replicas converge	Async propagation, anti-entropy, conflict resolution	Lowest latency, highest availability / stale and out-of-order reads, confusing UX (a like count that jumps)	Caches/CDNs (content delivery networks — servers worldwide holding copies of content), counters, analytics, DNS (the internet's name-to-address lookup)

Sequential example: Alice posts P1 then P2, Bob posts Q1; everyone sees one interleaving (say P1, Q1, P2), never P2 before P1 — but a reader just after Q1 finished may not see it yet.

Session (client-centric) guarantees

Weaker than the global models, but they fix the anomalies users notice, **within one session**:

Guarantee	Promise	Anomaly prevented	Example
Read-your-writes	You see your own earlier writes	"My new profile photo reverted"	Profile edits, cart
Monotonic reads	Never an older version after a newer one	A comment *disappears* on refresh	Feeds, notifications, config values
Monotonic writes	Your writes apply in the order issued	"price = 12" applied before the earlier "price = 10"	Version-control commits, debit then credit
Writes-follow-reads	A write after reading X is ordered after X	Reply stored before the post it answers	Comment threads

Session consistency bundles them (Azure Cosmos DB's default level): track the last version/LSN each session saw and route its reads to replicas at least that fresh — coherent for the user and cheap, though sessions may see different states and routing loses some **load-balancing** (spreading requests across servers) freedom.

How to choose: strong for money and stock; session guarantees where users mostly need their own actions to stick; causal for cause-and-effect content; eventual for high-volume, tolerant data. Many systems

let you **choose per operation** — Cassandra consistency levels, DynamoDB `ConsistentRead`, MongoDB read/write concerns, Cosmos DB's five levels.

> **Don't confuse** this "C" with the **C in ACID** (Chapter 13), which means "transactions keep constraints and invariants true". Consistency *models* are about **replicas and ordering**.

In the interview

> **30-second answer:** "A consistency model says what reads can return under replication. Linearizable: one up-to-date copy in real-time order. Sequential: one global order without the real-time promise. Causal: cause before effect. Eventual: convergence only. Session guarantees — read-your-writes, monotonic reads — fix what users see. Stronger costs latency and availability, so I pick per feature."

- *Model for a comment thread?* Causal — replies must never precede their parent.
- *Linearizable vs serializable?* Linearizability is about single objects and real-time recency; serializability is about multi-object transactions behaving as if run one at a time. Both together = "strict serializability" (Spanner).

Self-check

1. *A refresh shows an older comment list. Which guarantee broke?* Monotonic reads.
2. *Sequential vs strong?* Strong also respects real-time order.

15.7 The CAP theorem (and PACELC)

What it is

The **CAP theorem** (Eric Brewer's conjecture, 2000; proved by Gilbert and Lynch, 2002) concerns three properties of a replicated store:

- **C — Consistency** (here *linearizability*; see also System Design Fundamentals for CAP in context): every read sees the most recent successful write. If Priya's balance drops from ₹500 to ₹300, no node may still report ₹500.
- **A — Availability**: every request to a **non-failed** node gets a non-error response, without waiting indefinitely.
- **P — Partition tolerance**: the system keeps working when the network drops or delays messages between groups of nodes.

The theorem: *during a network partition, a system must choose between consistency and availability.*

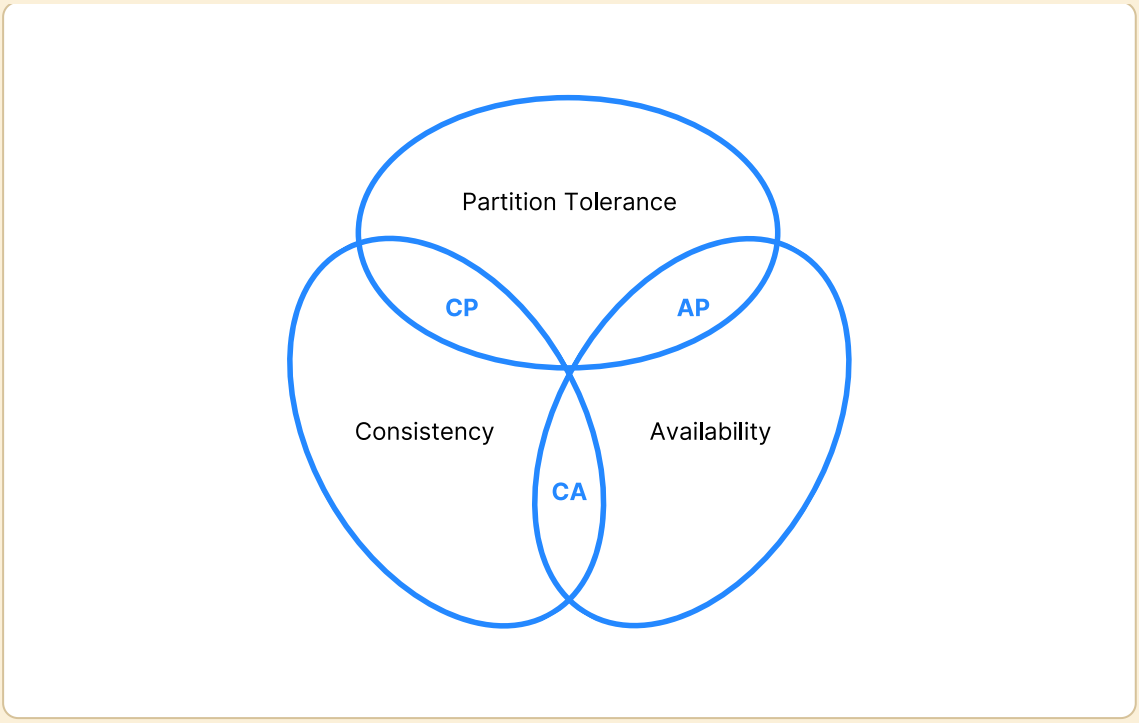


Figure — The three CAP properties and the CP, AP and CA combinations (course diagram).

Why "pick two of three" is misleading

In anything spanning more than one machine, **partitions will happen** (switches fail, cables get cut), so P can't be given up. The real choice is what to do *during* a partition:

Choice	During a partition	Pros	Cons	Examples
CP	Refuse or delay some requests (usually on the minority side) rather than return stale data	Correct, no divergence	Errors/timeouts for some users, coordination latency	etcd, ZooKeeper (small, strongly consistent coordination stores), HBase, Spanner, CockroachDB
AP	Keep answering on both sides, reconcile later	Always responsive	Temporarily inconsistent, needs conflict resolution	Cassandra/ScyllaDB (at `ONE`), Riak, CouchDB, DNS
CA	Only if partitions never happen — effectively one node	Simple and strong	No tolerance of network failure	A single Postgres server (not distributed, so CAP doesn't really apply)

Example: a subscriber counter is kept in two data centres and the link breaks. **AP** lets both sides keep accepting clicks and merges the counts later; **CP** makes the minority side refuse clicks until the link returns.

Nuances: the choice is often **per operation** (CP for "place order and decrement stock", AP for "show reviews"); with **no** partition a system can be both C and A; and CAP's A is strict — many "CP" systems are highly available in practice.

PACELC

Daniel Abadi's **PACELC** (2010): **if Partition, choose A or C; Else, choose Latency or Consistency** — even on a good day, synchronous coordination costs latency.

System	Partition	Else
Cassandra, Riak, DynamoDB (default reads)	PA	EL (fast, possibly stale)
Spanner, CockroachDB, etcd	PC	EC (pay latency for consistency)

In the interview

> **30-second answer:** "CAP says that during a partition a replicated store gives up either linearizable consistency or availability. Partitions are unavoidable, so the real choice is CP or AP — banking and inventory lean CP, feeds and carts lean AP with reconciliation. PACELC adds that without partitions you still trade latency for consistency."

- *Is a single Postgres server CA?* It isn't distributed, so CAP isn't about it; with async replicas plus failover it's neither fully C (can lose recent commits) nor fully A.

Self-check

1. *What does C mean in CAP, and what does PACELC add?* Linearizability; the latency-vs-consistency trade-off in normal operation.

15.8 Consensus and leader election

What consensus is

Consensus gets several nodes to **agree on one value** (or one ordered log) even when some crash or messages are delayed. Everything that needs "exactly one" rests on it — one leader, one commit decision, one log order, one lock owner — so it underpins consistency, fault tolerance, event ordering and coordination. A correct protocol guarantees **agreement** (all non-faulty nodes decide the same value), **validity** (the decision was proposed by someone; if all propose v, it's v), **termination** (non-faulty nodes eventually decide) and **fault tolerance** (it survives a minority of **crash faults**, or fewer than a third of **Byzantine faults** — nodes that lie). It's hard because nodes crash, networks partition and reorder messages, some nodes may be malicious, and every round trip adds latency. The **FLP result** (Fischer, Lynch, Paterson, 1985) proves no **deterministic** (same inputs, always same steps) protocol can *guarantee* termination in a fully asynchronous network if even one node may crash — so real protocols are always *safe* and use timeouts to *usually* make progress.

The algorithms in brief

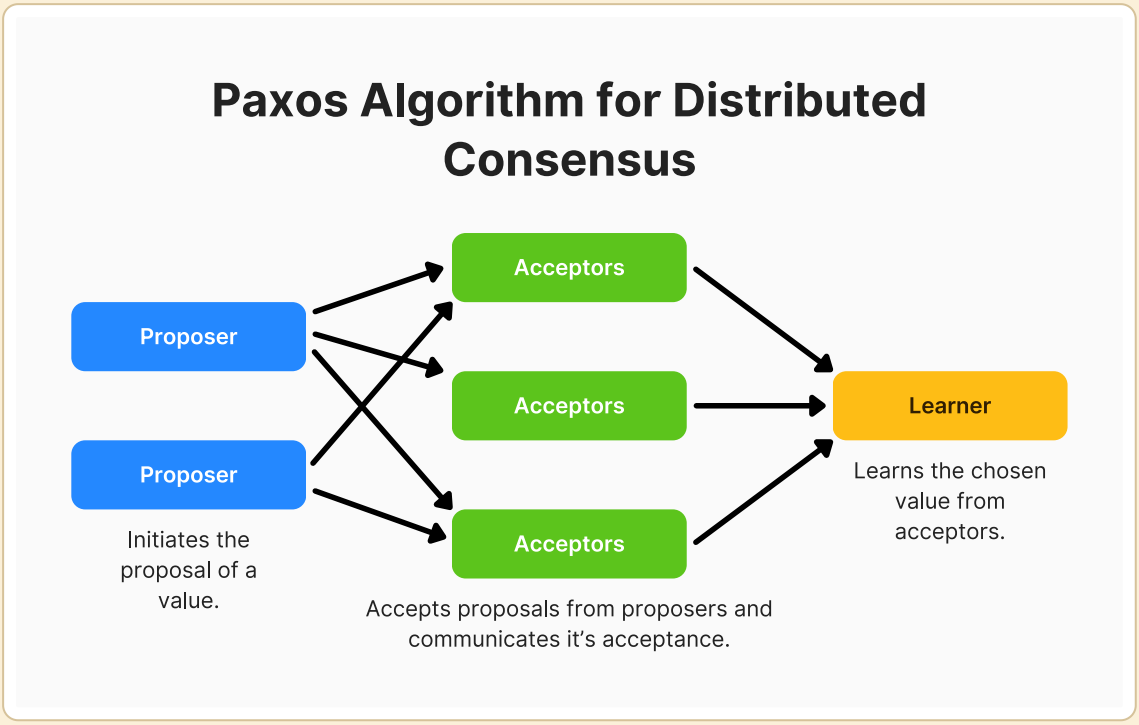


Figure — Paxos roles: proposers send numbered proposals to a set of acceptors, which accept and communicate the chosen value (course diagram).

Paxos (Leslie Lamport; written 1989, published 1998) has **proposers**, **acceptors** and **learners**. (1) **Prepare/Promise:** a proposer sends *prepare(n)* with a unique, higher number to a majority; each acceptor promises to ignore anything below *n* and reports the highest-numbered proposal it already accepted. (2) **Accept:** with a majority of promises the proposer sends *accept(n, v)*, where *v* must be the highest-numbered value reported (its own only if none). (3) Once a **majority accepts**, *v* is chosen and learners are told. It tolerates ⌊(N−1)/2⌋ crashes, needs no synchronised clocks for safety and is formally proven, but it's hard to implement, costs two round trips per decision (**Multi-Paxos** keeps a stable leader to skip phase 1), and duelling proposers can stall it. Used in Google's Chubby and Spanner.

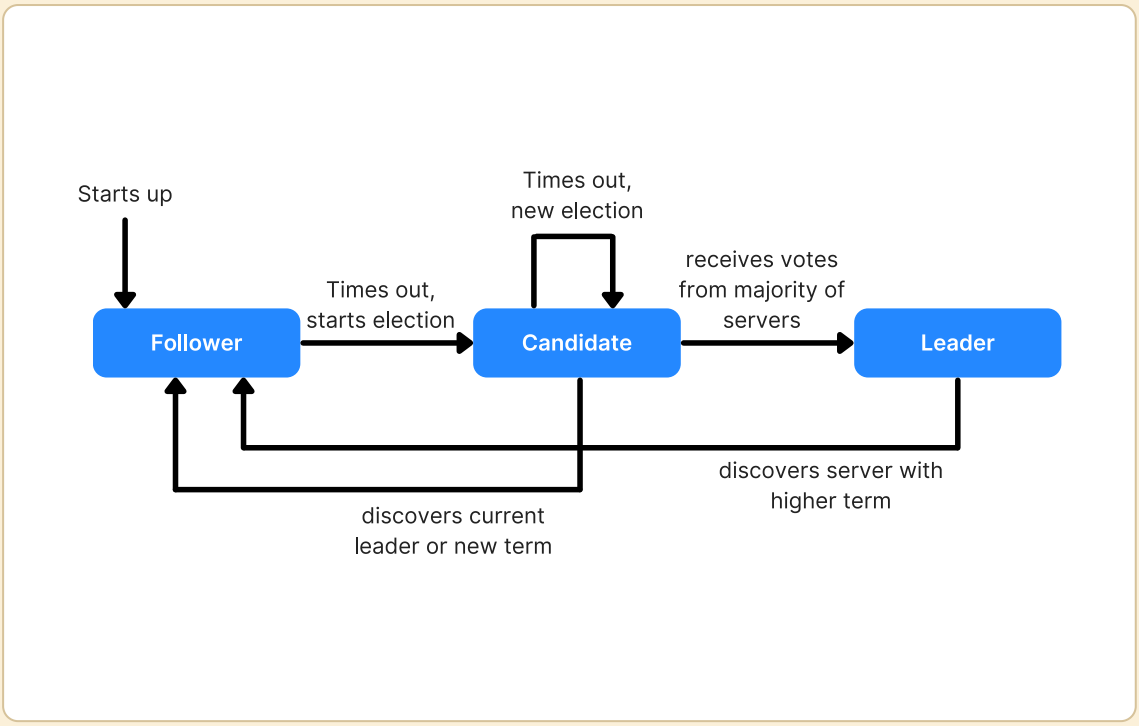


Figure — Raft node states: follower → candidate (on timeout) → leader (on majority vote), and back to follower on discovering a newer term (course diagram).

Raft (Ongaro and Ousterhout, 2014) gives the same guarantees but was designed to be **understandable**. (1) **Election:** time is split into numbered **terms**; a follower that hears no **heartbeat** (a small periodic "I'm alive" message) within its **randomised election timeout** (150–300 ms in the paper) becomes a **candidate**, increments the term and requests votes; each node votes once per term, only for a candidate whose log is at least as up to date as its own; a majority makes a **leader**, and the random timeouts make split votes rare. (2) **Log replication:** the leader sends **AppendEntries** (also the heartbeat); an entry is **committed** once a majority stores it. (3) **Safety:** only a node holding every committed entry can win. Its weaknesses: the leader is a throughput ceiling, unstable networks cause repeated elections, and each commit waits for a majority. Used in etcd (Kubernetes' store), Consul, CockroachDB, TiKV, YugabyteDB and Kafka's KRaft; MongoDB's replica-set protocol is Raft-inspired.

PBFT (Practical Byzantine Fault Tolerance; Castro and Liskov, 1999) tolerates **f lying nodes out of N ≥ 3f + 1**. In each **view** a **primary** runs *pre-prepare* (assign a sequence number) → *prepare* (replicas broadcast; prepared after 2f matching messages from others) → *commit* (after 2f + 1 matching commits) → *reply* (the client waits for f + 1 identical replies, so one is honest). It costs **O(N²) messages**, so clusters stay at tens of nodes; used in **permissioned blockchains** (shared ledgers where only approved participants may join; Hyperledger family) and critical control systems.

Proof of Work (Bitcoin) serves **open** networks, where fake identities make vote-counting useless: miners race to find a **nonce** (a throwaway number tried over and over) that puts the block hash under a target, others verify cheaply, and the chain with the most accumulated work wins — no central authority, but heavy energy use, slow finality (a block every ~10 minutes) and exposure to a **51% attack** (whoever controls most of the computing power can rewrite recent history). **Proof of Stake** (Ethereum since the 2022 "Merge") weights votes by locked-up currency instead.

Algorithm	Fault model, tolerance	Cost	Fits
Paxos / Multi-Paxos	Crash, ⌊(N−1)/2⌋	Majority round trips; hard to implement	Databases, lock services
Raft	Crash, ⌊(N−1)/2⌋	Majority round trips, leader-bound	Config stores, distributed SQL
PBFT	Byzantine, ⌊(N−1)/3⌋	$O(N^2)$ messages	Permissioned ledgers
PoW / PoS	Byzantine, open membership (honest majority of hash power / stake)	Slow, energy-hungry / better throughput	Public blockchains

Known members that may crash → Raft or Paxos (what you'll meet inside databases); known members that may lie → PBFT family; unknown members → PoW/PoS. For the full reasoning see DDIA ch.9.

Leader election

A **leader** is the single decider for writes, task assignment or a lock; election picks one and re-picks when it fails. Hard parts: a slow leader looks dead, partitions can create **two leaders** (**split brain**), and clocks differ between nodes.

Strategy	How it works	Strengths	Weaknesses
Bully	The detecting node sends ELECTION to all **higher** ids; if none answers it wins, else the higher node takes over; the winner broadcasts COORDINATOR	Simple, deterministic (highest live id)	Up to **$O(n^2)$** messages; the top node always leads, even if weak
Ring	ELECTION travels round a logical ring, each node substituting its id if higher; whoever gets its **own** id back wins and circulates COORDINATOR	About **2n** messages, equal participation	Slow on big rings; a dead node breaks the ring unless skipped
Randomised	Candidates wait a **random delay**, the first to announce wins, collisions retry	Simple, collisions rare	Non-deterministic (Raft adds majority votes)
ZooKeeper	Each candidate creates an **ephemeral sequential** znode (gone when its session dies); the **lowest** leads; each watches only the znode just **before** its own	Reliable, simple API; ZooKeeper runs the consensus protocol **ZAB**	Extra system to run (same pattern: etcd, Consul, Kubernetes **Lease**)

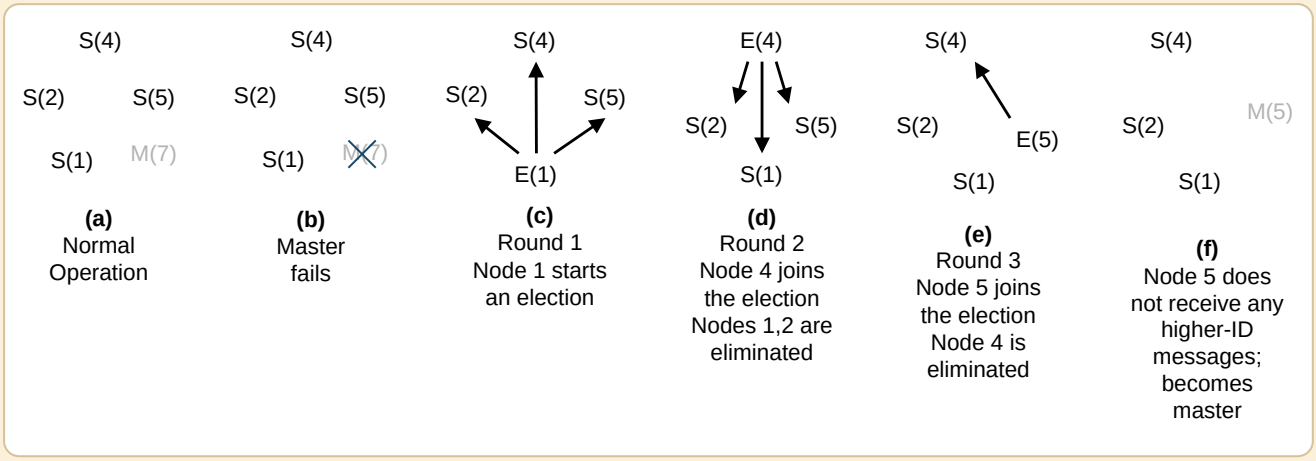


Figure — Bully election: after the leader fails, election messages flow to higher ids until the highest live node announces itself (course diagram).

`code/leader_election.py` counts messages when the highest-id leader has crashed and the *lowest* id notices first (Bully's worst case):

```
nodes bully leader bully msgs ring leader ring msgs
    5             4          19             4         8
   10             9          89             9        18
   50            49        2449            49        98
```

Same leader either way; Bully grows roughly with n^2, Ring with $2n$.

Fencing. A correctly elected leader can be *paused* (long **GC** — garbage-collection pause, when a runtime stops the program to free unused memory — or a **VM** (virtual machine) freeze), replaced, then wake up still believing it leads. Give each new leader a higher term/epoch number (a **fencing token**) and have storage reject writes carrying an older one — Raft terms do exactly this.

In the interview

30-second answer: "Consensus lets nodes agree on a value or log despite failures. Raft elects a leader with randomised timeouts and majority votes and commits an entry once a majority stores it, tolerating $\lfloor (N-1)/2 \rfloor$ crashes; Paxos is the older equivalent. PBFT tolerates liars with $3f+1$ nodes at $O(N^2)$ messages. For leader election I'd use etcd or ZooKeeper and fence stale leaders with epoch numbers."

- *Why 3 or 5 nodes, not 4?* A majority of 4 is 3, so 4 nodes tolerate one failure — no better than 3.
- *What is split brain?* Two nodes both acting as leader after a partition; prevent it with majority quorums and fencing.

Self-check

1. *A 5-node Raft cluster tolerates how many failures?* 2.
2. *PBFT with $f = 2$ needs at least?* $3f + 1 = 7$ nodes.
3. *Which znode does a ZooKeeper candidate watch?* The one just before its own.
4. *What can't be guaranteed in a fully asynchronous system (FLP)?* Termination.

TL;DR

Consensus = majority agreement that survives crashes (Raft/Paxos) or lies (PBFT, 3f+1). Leader election needs consensus underneath and fencing on top — use etcd, ZooKeeper or Consul rather than inventing your own.

15.9 Fault tolerance and recovery

What it is

Fault tolerance is the ability to **keep working correctly while parts are failing**. A **fault** is one component going wrong (dead disk, crashed process, dropped packet); a **failure** is the whole system stopping or answering wrongly — fault tolerance stops the first becoming the second, like a twin-engine plane landing on one engine. Its objectives are **continuity of service**, **data integrity**, **graceful degradation** (recommendations vanish but checkout works) and **recovery**, in support of **availability**, **reliability**, **safety** (failures stay contained — aviation, cars) and **maintainability** (easy repair and upgrade).

Handling a fault has five phases: **detection** (heartbeats, health checks, monitoring) → **diagnosis** (logs, error codes, tools) → **evidence generation** (detailed logs, snapshots, audit trails) → **assessment** (impact analysis, dependency maps) → **recovery** (fail over, restart, reconfigure, restore). Techniques exist at every level: **hardware** (RAID — several disks combined so one can fail without data loss, see 15.11 — hot-swappable drives, dual power supplies), **software** (exception handling, retries with **backoff** (waiting longer between each retry), timeouts, graceful degradation, **circuit breakers** (stop calling a failing service for a while so it can recover) — see Reliability patterns) and **system** (checkpointing, mirroring, load balancing, replication and failover). The rest of this section covers the database machinery: WAL, checkpoints and distributed commit.

Write-ahead logging (WAL)

The rule: *a change's log record must be safely on disk before the changed data page is written.* The log is an append-only sequential file (fast to write); data pages are written back lazily. After a crash the data files may be stale or half-written, but **the log has the truth**. A transaction changes **pages** (fixed-size blocks, 8 kB in Postgres, in which tables are stored) in memory (the **buffer pool** — the database's in-RAM cache of pages) and creates a log record per change; at **COMMIT** the log up to the commit record is **flushed (fsync'd** — `fsync` is the system call that forces data out of OS caches onto the physical disk), and only then is the commit acknowledged — now **durable** (the D of ACID, Chapter 13). *Example:* Ravi moves ₹5,000 from A to B; "A −5,000", "B +5,000" and a commit record are flushed and he sees "Transfer complete". If the server crashes a millisecond later, before the data pages were written, recovery replays those records.

Redo, undo and LSNs

- **Redo** makes **committed** work survive: recovery re-applies logged changes (**after-images** — the new values; **before-images** are the old values) that may not have reached the data files.
- **Undo** makes **uncommitted** work vanish: before-images restore any in-flight change that reached disk.
- Most databases do both. The classic design, **ARIES** (analysis → redo, "repeat history" → undo the losers), is used by DB2 and SQL Server; InnoDB uses the same redo + undo idea.
- **PostgreSQL's WAL is redo-only.** Thanks to **MVCC** (multi-version concurrency control, Chapter 13 — readers see an older version of a row while a writer creates a new one) an update writes a *new* row version; an aborted transaction is just marked not-committed in `pg_xact`, so its versions are invisible and **VACUUM** (Postgres's background clean-up of dead row versions) removes them later. InnoDB keeps separate **undo logs**, which also serve its MVCC snapshots.

Every WAL record has a **log sequence number (LSN)** — its byte position in the log, always increasing. LSNs **order** redo/undo, tell recovery **what's already applied** (each data page stores the LSN of its last change, so older records are skipped), **mark checkpoints** (replay starts from the checkpoint's redo LSN) and **measure replication lag** (primary LSN vs replica replay LSN).

`code/07_wal_lsn_checkpoint.sql` first prints the server's defaults (`wal_level` = replica, `synchronous_commit` = on — commit waits for the WAL flush, `checkpoint_timeout` = 5min, `max_wal_size` = 1GB, `full_page_writes` = on), then measures WAL growth:

```
INSERT INTO lsn_marks VALUES ('before', pg_current_wal_insert_lsn());
INSERT INTO ledger SELECT i, 'acct-' || (i % 5), (i % 7) - 3 FROM generate_series(1, 5000) i;
INSERT INTO lsn_marks VALUES ('after 5,000 inserts', pg_current_wal_insert_lsn());

SELECT a.label AS from_mark, b.label AS to_mark,
       pg_wal_lsn_diff(b.lsn, a.lsn) > 0         AS wal_grew,
       pg_wal_lsn_diff(b.lsn, a.lsn) > 5000 * 50 AS more_than_50_bytes_per_row
FROM lsn_marks a, lsn_marks b WHERE a.label = 'before' AND b.label = 'after 5,000 inserts';

CHECKPOINT;
SELECT redo_lsn >= (SELECT lsn FROM lsn_marks WHERE label = 'after 5,000 inserts')
       AS redo_point_moved_past_our_inserts
FROM pg_control_checkpoint();
```

```
 from_mark |       to_mark       | wal_grew | more_than_50_bytes_per_row
-----------+---------------------+----------+----------------------------
 before    | after 5,000 inserts | t        | t

 redo_point_moved_past_our_inserts
-----------------------------------
 t
```

`pg_current_wal_insert_lsn()` returns the current end of WAL (like `0/1A2B3C4`; booleans are printed because exact LSNs differ per run). Each row cost over 50 bytes of WAL — row data, index entries and headers. With `full_page_writes = on`, the first change to a page after a checkpoint logs the **whole page**, guarding against **torn pages** (pages only partly written when power was lost, mixing old and new bytes). After `CHECKPOINT` the **redo LSN** is past our inserts, so a crash now would start replay there.

Checkpointing

A **checkpoint** flushes all **dirty pages** (pages changed in memory but not yet written to disk) and logs *"everything before LSN X is on disk"*, so recovery replays only from there and older WAL (not needed by replicas or archiving) can be recycled.

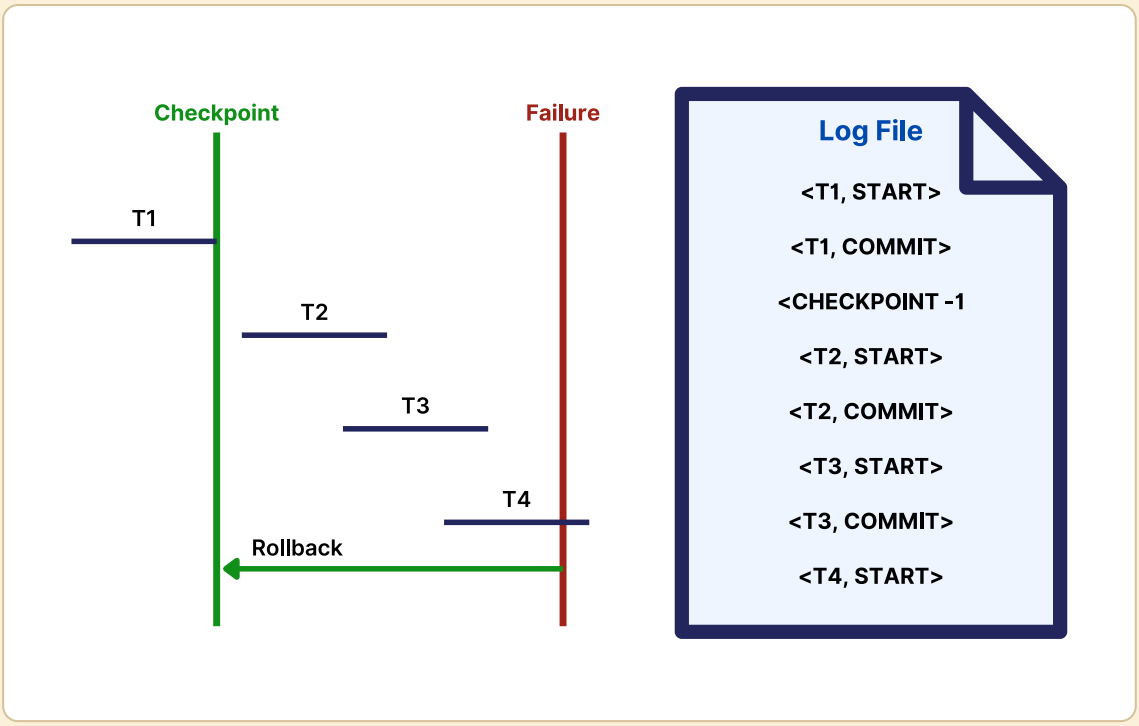


Figure — T1 finishes before the checkpoint, T2 and T3 commit after it, T4 is still running at the failure and is rolled back (course diagram).

At the failure: **T1** is already in the data files (ignored); **T2 and T3** committed after the checkpoint and may exist only in the log (**redone**); **T4** never committed (**undone** — in Postgres, simply invisible). `code/wal_recovery.py` runs this on a toy log — T1 (A→B 300) commits before the checkpoint, T2 (C→D 50) after it, and T3 (A→C 20) is interrupted after one of its pages was flushed — with a forward **redo** pass over updates after the checkpoint and a backward **undo** pass using before-images of transactions that never committed:

```
on disk after crash : {'A': 580, 'B': 400, 'C': 500, 'D': 0}
redo  LSN 206 T2: C = 450
redo  LSN 207 T2: D = 50
redo  LSN 210 T3: A = 580
undo  LSN 210 T3: A = 600
recovered state     : {'A': 600, 'B': 400, 'C': 450, 'D': 50}
money conserved     : True
```

T1 needed nothing, T2 was replayed, T3's early-flushed change was rolled back, and the 1,500 total is intact.

Checkpoints come in two types. A **sharp (consistent)** checkpoint pauses new work, flushes every dirty page and resumes — simple and fully consistent, but a visible stall. A **fuzzy** checkpoint logs a start point, flushes pages *while work continues*, then logs the end; recovery starts from the start point — no downtime, more bookkeeping. PostgreSQL uses the fuzzy style: every `checkpoint_timeout` (5 min default) or when `max_wal_size` of WAL accumulates, spreading writes over `checkpoint_completion_target` (0.9 of the interval by default since PG 14).

Distributed checkpointing. The nodes' saved states must form a **consistent global state**: if node 1's checkpoint records *sending* a message, node 2's must record *receiving* it. **Coordinated** checkpointing (a coordinator, or markers along channels as in the Chandy–Lamport snapshot) is always consistent at the cost of a short pause; **uncoordinated** checkpointing is cheaper but recovery must search for a consistent combination and can cascade rollbacks (the **domino effect**). *Example:* payment, stock and email nodes restore from one global checkpoint, so you never get "stock decremented but payment never recorded" (Apache Flink — a stream-processing engine — uses barrier checkpoints that work this way).

Coordinated commit: two-phase commit (2PC)

Distributed recovery is hard because networks partition, nodes fail and recover at different times, many transactions run concurrently, and every node must agree to commit or abort. When one transaction changes data on **several nodes** (a transfer from an account on node 1 to one on node 2), atomicity must span machines. **2PC** uses a **coordinator**:

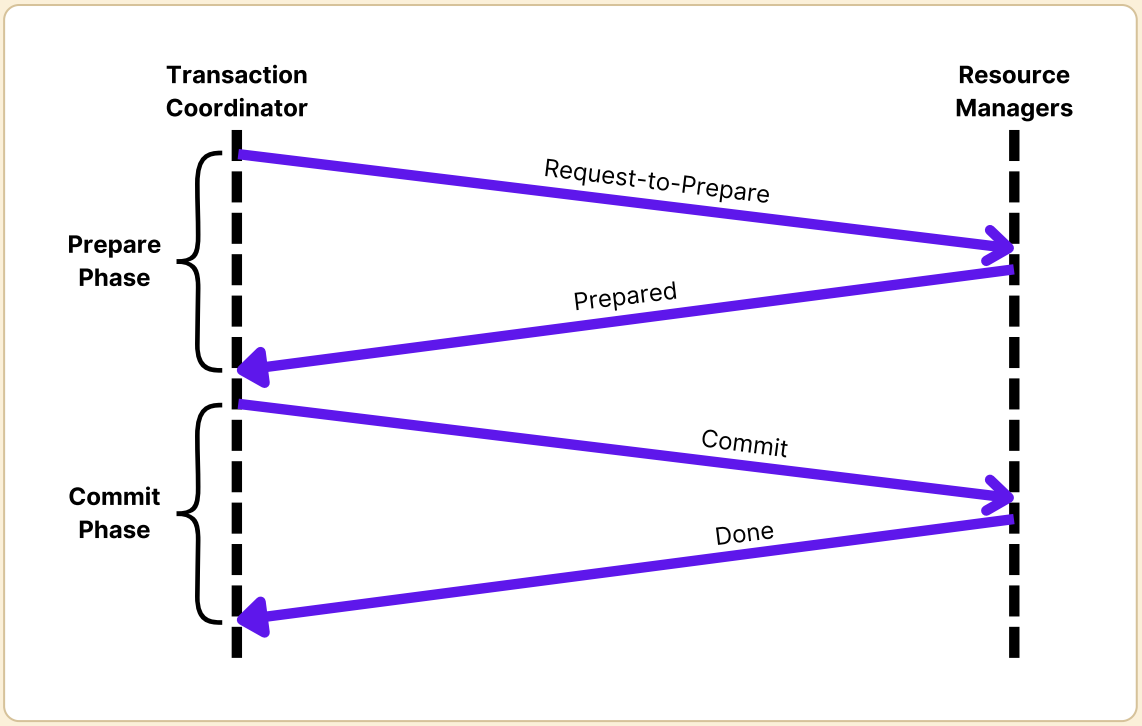


Figure — 2PC message flow: phase 1 asks every participant to prepare and collects their votes; phase 2 sends the commit and waits for acknowledgements (course diagram).

1. **Phase 1 — prepare (vote).** "Can you commit?" Each participant makes its changes durable, logs **PREPARED** and votes **YES** — from then on it may not decide alone — or votes **NO** (constraint failure, disk full).
2. **Phase 2 — decide.** All YES → the coordinator logs **COMMIT** and tells everyone to commit; any NO or timeout → it logs **ABORT** and everyone rolls back.

Recovery: a participant restarting without a PREPARED record aborts alone; one *with* PREPARED must **ask the coordinator**. If the coordinator crashes, participants that haven't voted can abort, but those that **voted YES must wait** — guessing could break atomicity. That wait, with **locks** held (locks stop other transactions touching the same rows), is **2PC's blocking problem**; network delays and partitions also stall decisions. `code/two_phase_commit.py` shows all three cases:

```
--- happy path: debit on node 1, credit on node 2
  ...
  phase 2 final states: {'node1': 'committed', 'node2': 'committed'}
--- one participant votes no (e.g. constraint check failed)
  ...
  phase 2 final states: {'node1': 'aborted', 'node2': 'aborted'}
--- coordinator failure after the votes
  phase 1 votes: {'node1': True, 'node2': True}
  coordinator crashes before sending a decision
  node1: prepared, holding locks, cannot commit or abort alone -> BLOCKED
  node2: prepared, holding locks, cannot commit or abort alone -> BLOCKED
```

2PC in PostgreSQL — the participant side, driven by an external transaction manager *(not run: it needs `max_prepared_transactions > 0`, and the book server keeps the default 0)*:

```
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE id = 'A';
PREPARE TRANSACTION 'transfer-42';   -- phase 1: durable, survives a crash, locks kept
COMMIT PREPARED 'transfer-42';       -- later, on the coordinator's decision (or ROLLBACK PREPARED)
```

Java XA/JTA transaction managers (XA is the standard interface for distributed transactions; JTA is Java's API for it) and MySQL's `XA START … XA PREPARE … XA COMMIT` work the same way. Forgotten prepared transactions hold locks and block VACUUM — monitor `pg_prepared_xacts`.

Three-phase commit (3PC)

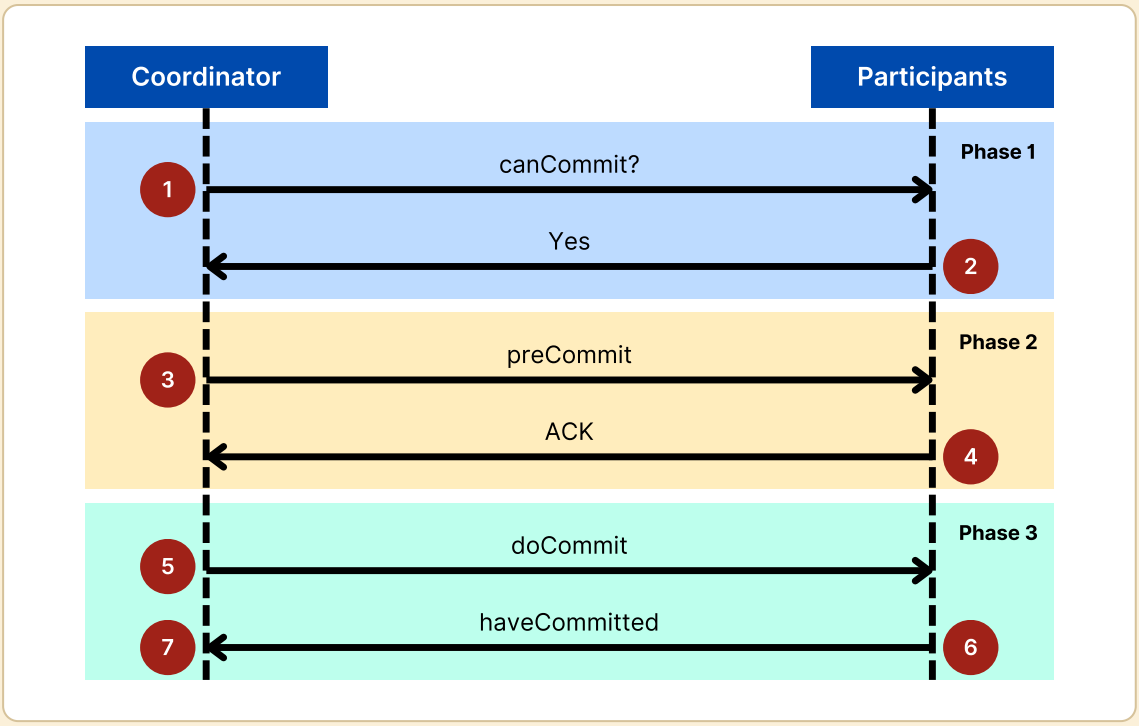


Figure — 3PC: canCommit?/Yes, then preCommit/ACK, then doCommit/haveCommitted (course diagram).

3PC inserts a **PreCommit** step between voting and committing: (1) **CanCommit** — vote as in 2PC; (2) **PreCommit** — if all voted YES, participants acknowledge and log that the outcome will be commit; (3) **DoCommit** — the final commit. If the coordinator dies, participants that logged pre-commit know everyone voted YES and can commit after a timeout; if no live participant has seen pre-commit, nobody can have committed, so they abort. This is **non-blocking** — *provided* message delays are bounded and nodes only crash. Under a real **partition** both sides can time out and decide differently, so 3PC is rarely used. Modern systems make the coordinator fault-tolerant by **running the commit decision through consensus** (Spanner: 2PC over Paxos groups; CockroachDB: a transaction record replicated by Raft), and cross-service workflows use **sagas** (compensating steps) — see 2PC and Saga patterns.

In the interview

30-second answer: "Databases survive crashes with write-ahead logging: the log is flushed before commit is acknowledged and data pages are written later. Recovery starts at the last checkpoint, redoes committed changes and undoes uncommitted ones — Postgres needs no undo thanks to MVCC. Across nodes, 2PC gives atomic commit, but prepared participants block if the coordinator dies, so real systems replicate the decision with Paxos/Raft or use sagas."

- *What does `synchronous_commit = off` trade?* Commits return before the WAL flush: a crash can lose the last fraction of a second of commits, never corrupt data.

Self-check

1. *T2 committed after the last checkpoint; T4 hadn't committed. At a crash?* T2 is redone, T4 undone (invisible in Postgres).
2. *A 2PC participant voted YES and lost the coordinator. Can it abort?* No — it must wait.
3. *Sharp vs fuzzy checkpoint?* Sharp pauses work to flush; fuzzy flushes while work continues.

TL;DR

Log first, write data later; checkpoints bound recovery; recovery = redo committed + undo uncommitted. 2PC blocks on coordinator failure and 3PC isn't partition-safe, so modern systems put the commit decision on a consensus group.

15.10 High availability: replication plus failover

What it is

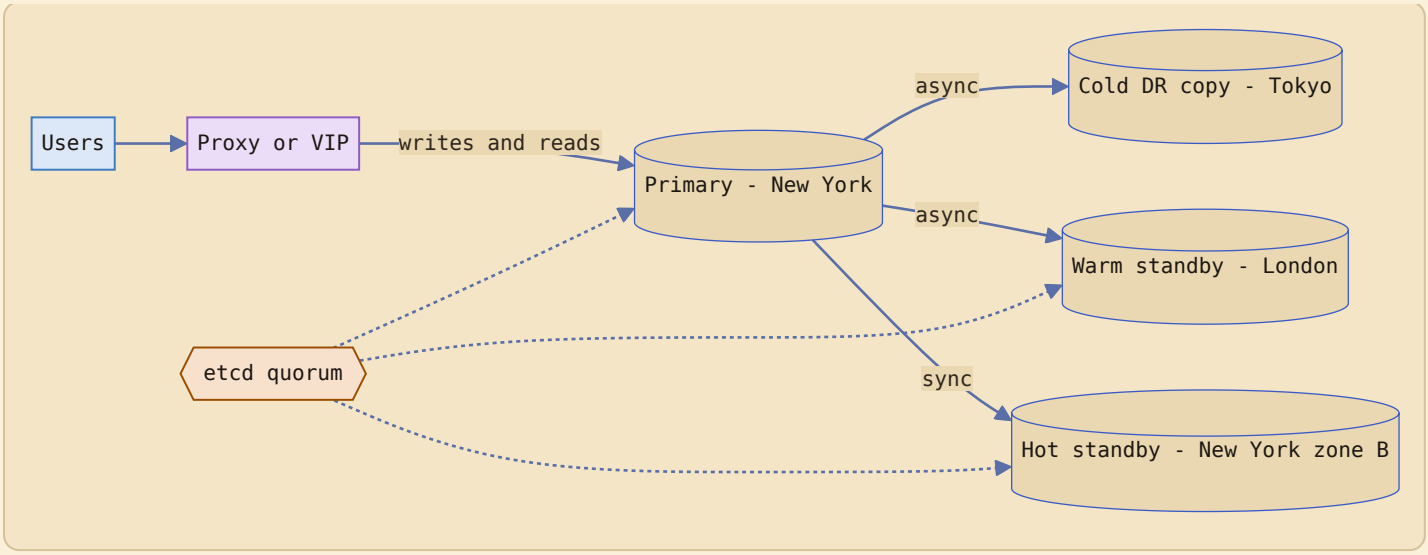
High availability (HA) keeps the service up despite failures, measured in "nines" (the percentage of time the service is up): 99.9% ≈ 8.8 hours of downtime a year, 99.99% ≈ 53 minutes. Two targets make it concrete: **RPO** (recovery point objective — how much *data* you may lose; synchronous replication gives RPO = 0) and **RTO** (recovery time objective — how long you may be *down*; hot standbys and automatic failover make it short). HA = **replication** (15.5; a copy exists) + **failover** (switch to it). A sync standby has every acknowledged commit at failover (RPO 0, slower commits — typical for a local standby in financial systems); an async one may miss the last commits (RPO > 0, fast commits — remote regions, read replicas).

Failover

Type	Standby state	Recovery time	Cost
Cold	Not running; must start and restore/catch up	Minutes to hours	Cheapest
Warm	Running and receiving updates, but needs some catch-up or config	Seconds to minutes	Medium
Hot	Running, in sync, ready to serve immediately	Seconds	Highest

1. **Heartbeats** — several missed "I'm alive" signals raise suspicion.
2. **Decision** — a **quorum** (majority of nodes or witnesses) must agree the primary is dead, preventing **split brain**; **priority rules** pick the standby (highest replayed LSN, preferred zone).
3. **Promotion** — Postgres: `pg_ctl promote` or `SELECT pg_promote()` (PG 12+).
4. **Fencing** the old primary — STONITH ("shoot the other node in the head"), lease revocation, or blocking at the proxy.
5. **Redirecting clients** — DNS, a **virtual IP** (an address that can be moved from one machine to another), a proxy (HAProxy, PgBouncer) or **service discovery** (a registry clients ask "where is the primary right now?").

Core PostgreSQL has **no automatic failover** (switching to a standby without a human — see the steps above); use **Patroni** (leader state in etcd/Consul/ZooKeeper — consensus-backed), **repmgr** or **pg_auto_failover**, or a managed service (Amazon RDS/Aurora, Cloud SQL, Azure Database). **Best practices:** redundancy across failure domains (racks, zones, regions), load-balanced reads with writes routed to the current primary, monitoring (replication lag, heartbeats, disk), **backups too** (a bad `DELETE` replicates in milliseconds — HA is not a backup), and **regular failover tests** (game days, **chaos experiments** — deliberately breaking things in a controlled way to prove recovery works).



A zone failure in New York fails over to the sync hot standby in seconds with no data loss. Losing all of New York promotes London (RPO seconds, RTO about a minute; it also serves European reads). Tokyo is a cold copy for when both are gone. The etcd quorum spans three sites, so a partitioned minority can never promote itself.

In the interview

30-second answer: "HA = replicas plus automatic failover: heartbeats detect failure, a quorum confirms it to avoid split brain, the most up-to-date standby is promoted, the old primary is fenced, clients are redirected. Sync standbys give zero data loss, async lower latency with some loss. I'd state RPO/RTO, choose hot/warm/cold to match, and test failover regularly."

Self-check

1. *RPO with async replication?* Greater than zero — the lag at the moment of failure.

15.11 Data resilience: redundancy, mirroring, backups

HA keeps the service **running**; **data resilience** makes sure the **data itself** survives disk crashes, corruption, ransomware, operator mistakes and site disasters.

Redundancy

Technique	How	Overhead	Survives
Replication	Full copies on other nodes (15.5)	N×	Node or site loss
RAID 1 / 10	Mirror each disk (10 = mirrors, striped; popular for databases)	2×	1 disk per mirror pair
RAID 5 / 6	Stripe data (spread it across disks) with one / two parity blocks (extra blocks computed from the data, used to rebuild a lost disk)	1 / 2 disks	Any 1 / any 2 disks
Erasure coding	k data + m parity fragments; any k rebuild the data	(k+m)/k: 6+3 → 1.5×	Any m fragments

Erasure coding (e.g. Reed–Solomon) gives replication-level safety at 1.5× instead of 3× storage, paid for with CPU and slower rebuilds — **object stores** (services like Amazon S3 that store whole files by key) and HDFS (Hadoop's distributed file system) use it for large, cooler (rarely read) data.

Mirroring

Mirroring keeps an **exact, real-time copy**: every write goes to primary and mirror together, and the mirror takes over immediately. Near-instant failover and always current, but double storage and extra write latency. *Example:* a bank mirrors its transaction database synchronously between two nearby data centres. (SQL Server literally called it "Database Mirroring"; Postgres's equivalent is synchronous streaming replication.)

Media failure recovery

A **media failure** is lost or damaged storage — **physical** (dead drive, bad sectors) or **logical** (corruption from software/firmware bugs or malware). Defences: **hot swapping** failed RAID drives while running; **health monitoring** (**SMART** drive telemetry, cloud volume metrics); **data scrubbing** (periodically read every block, verify **checksums** (small fingerprints computed from data; a mismatch means corruption), repair from redundancy — ZFS, Ceph; Postgres detects corrupted pages with `initdb --data-checksums`); and **replication** to other nodes and regions.

Backup and restoration

A **backup** is an independent copy that can take you **back in time** — replicas can't, they follow the primary instantly. **Restoration** gets data back from it.

Type	Contains	Backup cost	Restore needs
Full	Everything	Slowest, largest	Just that backup (simplest restore)
Incremental	Changes since the **last backup of any kind**	Fastest, smallest	Last full + **every** incremental since, in order
Differential	Changes since the **last full**	Grows through the week	Last full + the **latest** differential

PostgreSQL (commands in Chapter 9 — backup and restore): `pg_dump` / `pg_restore` make **logical** backups of one database, portable across versions. `pg_basebackup` plus **continuous WAL archiving** (`archive_mode`, `archive_command`) gives a **physical** backup you can replay to *any moment* — **point-in-time recovery (PITR)** with `recovery_target_time`, the cure for "someone ran `DELETE` without `WHERE` at 14:02". PostgreSQL 17 added incremental base backups (`pg_basebackup --incremental`); pgBackRest and Barman manage all of this.

Best practices: backup frequency to match RPO (daily plus continuous WAL for critical data); **off-site** copies following the **3-2-1 rule** (3 copies, 2 media, 1 off-site); **test restores** regularly; **versioning and immutability** (object lock — stored copies can't be changed or deleted for a set period) so **ransomware** (malware that encrypts or deletes data and demands payment) can't destroy every copy. *Example:* a hospital takes a full backup every Sunday, incrementals Monday–Saturday and continuous WAL, stores copies locally and in another cloud region, and each month restores "last Wednesday 15:00" into a test environment and checks row counts.

In the interview

> **30-second answer:** "Replication and RAID protect against hardware loss but not bad writes, which replicate too. So I add backups — full plus incremental or differential, off-site, versioned, test-restored. In Postgres that's base backups plus WAL archiving for PITR. Erasure coding gives durable storage at about 1.5× instead of 3× for large, cold data."

- *Accidental `DROP TABLE` at 14:02?* Restore the base backup into a separate instance, replay WAL to 14:01:59, copy the table back.

Self-check

1. *Which RAID level survives any two disk failures?* RAID 6.
2. *Why isn't a synchronous replica a backup?* It copies mistakes instantly; there's no going back in time.

TL;DR

Redundancy (replicas, RAID, erasure coding) survives hardware loss; backups (full/incremental/differential + WAL archiving) survive mistakes and disasters. Keep copies off-site and versioned, and test restores.

Chapter 15 TL;DR

Topic	Remember
Scale	Scale up first, then scale out with **partitioning + replication** together.
Partitioning	Range (time), list (category), hash (even), composite. Verify pruning with `EXPLAIN`. Unique keys include the partition key.
Sharding	Partitions on separate servers; a high-cardinality shard key matching the main query; consistent hashing with vnodes.
Distributed queries	Push filters and partial aggregates to the data, co-locate joins, semi-join to ship keys not rows.
Replication	Sync = no loss but slower; async = fast, lag, possible loss. Single-leader by default. Leaderless quorums need R + W > N.
Consistency	Strong > sequential > causal > session > eventual. Pick per feature.
CAP / PACELC	Under partition choose C or A; otherwise latency or consistency.
Consensus	Raft/Paxos: 2f+1 nodes survive f crashes. PBFT: 3f+1 for f liars. Elect leaders via etcd/ZooKeeper and fence stale ones.
Recovery	WAL first, checkpoints bound replay, redo committed + undo uncommitted (Postgres: redo-only + MVCC). 2PC blocks, 3PC isn't partition-safe; consensus-backed commit is the modern answer.
HA	Heartbeat → quorum decision → promote → fence → redirect. State RPO/RTO.
Resilience	Replicas are not backups. Full/incremental/differential + PITR, off-site, tested.

Next: Chapter 16 — Beyond relational looks at the NoSQL, NewSQL and specialised databases that make these trade-offs in different ways.

Chapter 16 — Beyond Relational: NoSQL, NewSQL and Specialised Databases

Relational databases are the right default for most applications. They offer a flexible query language, joins, constraints and **ACID** transactions (Atomic, Consistent, Isolated, Durable — a transaction either fully happens or not at all, and once committed it survives crashes; Chapter 13). But some workloads fit tables awkwardly. Examples include a product catalogue where every item has different attributes, a **session cache** (fast storage for logged-in users' temporary state) that needs sub-millisecond reads, billions of sensor readings per day, "friends of friends who like jazz", or "every restaurant within 2 km". A whole family of databases grew up around those shapes of data. This chapter tours them, shows how much of each you can already do **inside PostgreSQL**, and ends with a decision table for choosing.

What you'll learn

Section	Topic	Runnable part
16.1	Why NoSQL appeared, CAP and NoSQL, BASE vs ACID, the four families	—
16.2	Key-value stores (Redis, Riak)	KV + TTL + leaderboard in Postgres
16.3	Document databases (MongoDB, CouchDB)	**JSONB as a document store** with GIN indexes
16.4	Column-family and wide-column stores (Cassandra, HBase)	Wide-row time-series table
16.5	Graph databases (Neo4j, Neptune)	Friends-of-friends and shortest path with recursive CTEs
16.6	In-memory databases (SAP HANA, SingleStore, Redis)	—
16.7	NewSQL / distributed SQL (Spanner, CockroachDB)	—
16.8	Time-series and spatial databases	Partitions, BRIN, `date_bin`, geometric KNN with GiST
16.9	Big data frameworks: Hadoop, Spark, SQL-on-Hadoop, Kafka, Flink	Python MapReduce in miniature
16.10	Choosing a database	Decision table
16.11	Inside Elasticsearch/ELK (inverted index, analyzers, shards, refresh) and Redis (event loop, encodings, RDB/AOF, Cluster slots, eviction)	**Python inverted index + BM25 over two shards**

Prerequisites: Chapter 15 (partitioning, replication, CAP, consensus) and Chapter 12 (indexes). For the data-model theory behind this chapter, see your DDIA notes: ch.2 — data models, ch.3 — storage engines (LSM trees, column storage), ch.10 — batch and ch.11 — stream processing. For the pattern view, see Caching, Batch and streaming and Storing data.

All SQL runs on the book's private PostgreSQL 14 (`bash tools_run_sql.sh sql`); the two Python toys (MapReduce, inverted index) run with `bash tools_run_python.sh sql`. Snippets for other products (MongoDB, Redis, Cassandra, Neo4j, PostGIS) are marked **(not run)**.

16.1 Why NoSQL?

What it is

NoSQL ("Not only SQL") is an umbrella term for databases that **don't use the relational table model** as their primary model. The term caught on around 2009. They trade some relational features, usually joins, a fixed **schema** (the declared list of tables, columns and types), and sometimes multi-row transactions, for **flexible data models, easy horizontal scaling, and speed on specific access patterns**.

The problem it solves

In the 2000s, web companies hit limits with the relational setups of the time:

Pain with relational (as typically deployed then)	NoSQL answer
Rigid schema: every row has the same columns, and changing them means **migrations** (scripted schema changes applied to a live database)	**Flexible/dynamic schema**: each record carries its own fields (JSON — a text format of nested key/value objects and lists — or BSON documents, dynamic columns)
Scale-up only: one bigger server, which gets expensive and eventually hits a ceiling	**Scale-out built in**: automatic sharding and replication across cheap nodes
Joins and transactions get slow at huge volume	Data **stored the way it's read** (denormalised — deliberately duplicated — or pre-aggregated), so there are no joins in the **hot path** (the code run on every request)

Example. An online marketplace sells laptops (RAM, CPU, warranty), T-shirts (size, material) and books (authors, pages). In a relational schema that means **sparse columns** (many columns that are empty for most rows), an **entity-attribute-value** table (one row per `(item, attribute name, value)` triple), or one table per category. In a document store, each product is simply a document with whatever fields it has (16.3).

Modern relational systems have closed much of this gap: Postgres has JSONB (a binary, indexable JSON column type, 16.3), partitioning, logical replication, and **Citus** (an extension that spreads Postgres tables across many servers) for scale-out. That's why this chapter shows the Postgres equivalent of each model.

CAP and NoSQL

Recall CAP (15.7; also summarised in System Design Fundamentals). During a **network partition** (some nodes can't reach others) a distributed store must choose **consistency** or **availability**:

- **CP** stores refuse or delay requests on the minority side to stay consistent. Examples: **HBase**, MongoDB (with majority write/read concern — the setting for how many replicas must confirm a write or agree on a read), etcd.
- **AP** stores keep answering and reconcile later. Examples: **Cassandra**, Riak, CouchDB, DynamoDB reads (eventually consistent unless you request a strongly consistent read).

Many NoSQL systems make this **tunable per request**, such as Cassandra's `ONE` vs `QUORUM`.

BASE: the other end of the spectrum from ACID

Many AP-leaning NoSQL systems describe their guarantees as **BASE**:

- **Basically Available**: the system answers almost every request, even during partial failures, though the answer may not be the newest. *Example:* a Cassandra read at consistency `ONE` succeeds as long as one replica is up.
- **Soft state**: replicas may be temporarily out of sync, and the state can change without new input as updates propagate. *Example:* asynchronous replication in a MongoDB **replica set** (a group of servers holding the same data: one **primary** takes writes, **secondaries** copy it), where a secondary can lag the primary.
- **Eventual consistency**: if writes stop, all replicas converge (15.6). *Example:* a Cassandra write that reached only some replicas is spread by hinted handoff (another node holds the write until the missing replica returns), read repair (a reader fixes stale copies it notices) and anti-entropy (background comparison and syncing of replicas).

BASE is essentially the **AP** choice from CAP, packaged as a design philosophy. It suits social feeds, activity streams and metrics, where being up matters more than every reader seeing the very latest value.

Property	ACID (typical relational)	BASE (typical AP NoSQL)
Atomicity	All-or-nothing, including multi-row transactions	Often per single record/document only
Consistency	Constraints hold after every commit, and reads see committed data	Replicas may briefly disagree and converge later
Isolation	Concurrent transactions don't interfere (per isolation level)	Weaker, and concurrent writes may conflict and need resolution
Durability	Committed = on disk (WAL)	Configurable (write concern, replication factor = number of copies, fsync policy = how often data is forced to disk)
Availability focus	Consistency first during failures	Availability first
Scaling	Traditionally vertical plus read replicas	Horizontal by design

Correction to a common myth: the line has blurred. MongoDB has supported multi-document ACID transactions since 4.0 (2018). DynamoDB has had `TransactWriteItems` since 2018. Cassandra has lightweight transactions (Paxos-based **compare-and-set** — "set the value to X only if it is still Y", done atomically; Paxos is a consensus protocol, 15.8). Meanwhile Postgres stores schemaless JSON. Treat ACID vs BASE as a **spectrum of defaults**, not a hard divide.

The four NoSQL families

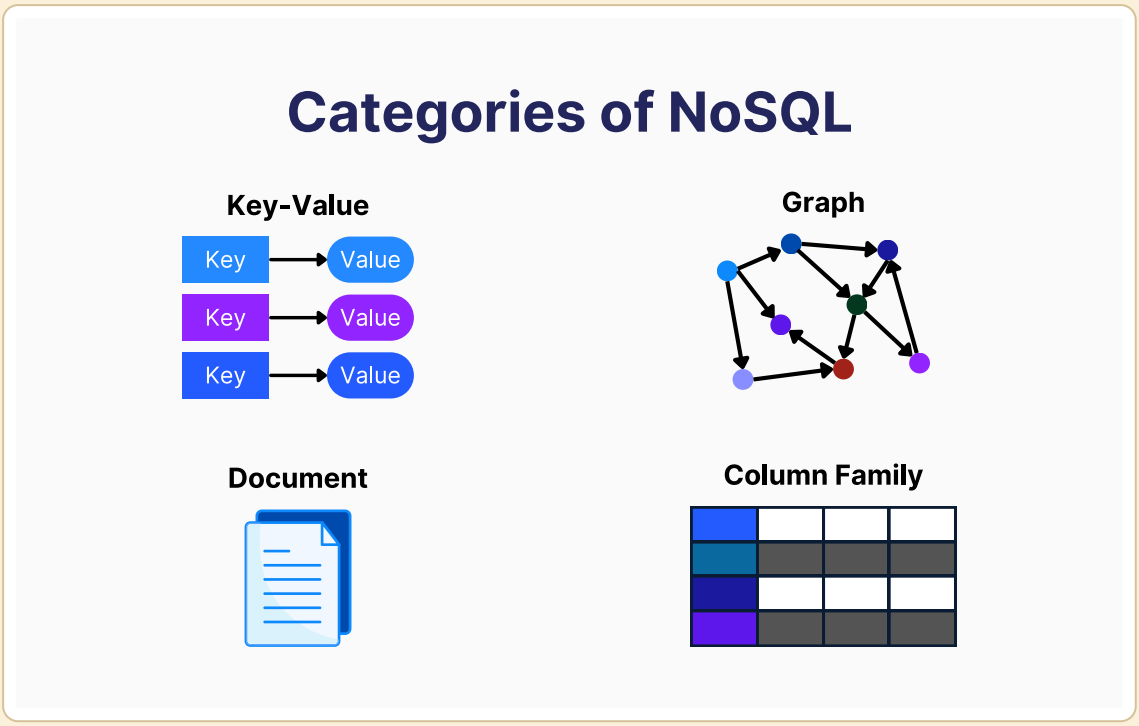


Figure — The main NoSQL families: key-value, document, column-family and graph (course diagram).

Family	Data model	Great at	Examples
Key-value	Opaque (not interpreted by the database) value looked up by a unique key	Caching, sessions, counters, leaderboards	Redis, Riak KV, Memcached, DynamoDB (also a document store)
Document	JSON-like documents grouped into collections	Catalogues, CMS (content management system) content, user profiles	MongoDB, CouchDB, Firestore
Column-family / wide-column	Rows with a partition key, columns grouped into families, sorted within a partition	Huge write volumes, time-series, logs	Cassandra, HBase, ScyllaDB, Bigtable
Graph	Nodes and edges, both with properties	Relationships: social, fraud rings, recommendations	Neo4j, Amazon Neptune, ArangoDB

Features they share: **dynamic schema** (no migration to add a field), **horizontal scaling** (add nodes), **performance tuned to one access pattern**, and **fault tolerance** through built-in replication.

In the interview

30-second answer: "NoSQL databases drop the relational model to get flexible schemas, easy scale-out and speed on specific access patterns. There are four families: key-value, document, wide-column and graph. Many lean AP in CAP terms and follow BASE (basically available, soft state, eventually consistent) instead of strict ACID, though modern ones offer transactions and tunable consistency. You model data around queries, not around normalisation."

- *Is MongoDB ACID?* Single-document operations always were. Multi-document transactions exist since 4.0 (replica sets) and 4.2 (sharded clusters).
- *When is Postgres JSONB enough?* When you need flexible attributes *plus* joins, constraints and transactions, at a scale one Postgres (or Citus) can handle.

Self-check

1. *What does the S in BASE mean?* Soft state: replica state may change as updates propagate, even with no new writes.
2. *Name a CP and an AP NoSQL store.* HBase (CP) and Cassandra (AP by default).
3. *Which family fits "who are my friends' friends"?* Graph.

TL;DR

NoSQL = flexible models + scale-out + access-pattern-specific speed, often with BASE semantics. The ACID/BASE divide is now a spectrum.

16.2 Key-value stores

What it is

The simplest database: a giant, distributed **dictionary**. Each **key** is unique (a string or number), and the **value** is whatever you store: text, a number, JSON, or a binary **blob** (raw bytes such as an image). The database doesn't interpret the value. You `GET key`, `SET key value` and `DELETE key`, and there are no joins and usually no queries by value.

```
session:9f2c      -> {"user_id": 42, "cart": ["LAP-13"], "expires": "..."}
rate:10.0.0.7     -> 17
weather:paris     -> {"temp_c": 18}
```

Designing keys is the whole data model. Use prefixes like `user:42:profile`, because the key is the only way in.

Key-value stores fit when you need very fast access by a known identifier, when relationships between items are minimal or handled in the application, and when each pair can be treated independently.

Popular systems

Redis ("Remote Dictionary Server") keeps everything **in memory** and answers in well under a millisecond. Values can be rich **data structures**: strings, hashes (small field→value maps), lists, sets, **sorted sets** (members kept ordered by a numeric score), streams, bitmaps and HyperLogLogs (a tiny structure that estimates how many distinct items it has seen). It also offers **pub/sub** messaging (publishers send to a channel; every subscriber to it receives the message), **Lua** scripting (small scripts run inside the server), key expiry (**TTL**, time to live — the key deletes itself after a set time) and **eviction policies** (rules for which keys to throw out) such as **LRU** (least recently used) / **LFU** (least frequently used) when memory is full. **Persistence** (saving data to disk so it survives a restart) is optional, via RDB snapshots and/or an append-only file (AOF). Redis Cluster shards keys over 16,384 hash slots (fixed buckets each key is hashed into; 16.11).

```
# Redis (not run)
SET session:42 '{"user":42}' EX 1800     # expires in 30 min
GET session:42
INCR page:home:views                      # atomic counter
ZADD steps 12050 ana 8300 bo 15010 cy     # sorted set = leaderboard
ZREVRANGE steps 0 2 WITHSCORES            # top 3
```

Riak KV is a Dynamo-style (modelled on Amazon's Dynamo paper) **distributed** key-value store: leaderless replication, consistent hashing, tunable N/R/W quorums (N copies, W must confirm a write, R are asked on a read — 15.5), and **eventual consistency**, with sibling values for conflicts. It was built for high availability and survives node failures without failover. Riak's original company (Basho) closed in 2017, and the code lives on as open source.

Use cases

Use	Example	Why KV fits
Caching	A weather app caches the latest forecast per city. A shop caches product pages in Redis to spare the database.	Read by key, expiry built in
Session storage	Logged-in user state for a web app	Tiny, per-user, high-churn data
Real-time data	Live stock prices or ride locations updated many times a second	In-memory speed
Counters and rate limits (caps on how often a client may call)	"100 requests per minute per API key"	Atomic `INCR` with a TTL
Leaderboards	A fitness app ranks users by daily steps	Sorted sets keep order as scores change

Performance: strengths and limits

Strengths	Limits
O(1) lookups by key (constant time, however big the data), very high **throughput** (operations per second)	**No ad-hoc queries** (one-off questions not planned in advance): filtering, joins and secondary lookups must be built in the app or with extra keys
Easy to shard (hash the key) and replicate	**Poor fit for relationships** between items
Values can be anything	In-memory stores are **bounded by RAM** (and RAM is expensive)

The same pattern in Postgres

`code/02_key_value_style.sql` builds a KV table with TTL, **UPSERT** (insert, or update if the key already exists) "put", and a leaderboard:

```
CREATE UNLOGGED TABLE kv (            -- UNLOGGED: no WAL, fast, emptied after a crash (cache-like)
  k          text PRIMARY KEY,
  v          jsonb NOT NULL,
  expires_at timestamptz
);
-- PUT = insert or overwrite
INSERT INTO kv VALUES ('session:42', '{"user":42,"cart":["LAP-13","TSH-01"]}', now() + interval '30 minutes')
ON CONFLICT (k) DO UPDATE SET v = EXCLUDED.v, expires_at = EXCLUDED.expires_at;
INSERT INTO kv VALUES ('weather:paris', '{"temp_c":18}', now() - interval '1 minute');  -- already expired
-- GET only unexpired keys
SELECT k, v FROM kv WHERE k = 'session:42' AND (expires_at IS NULL OR expires_at > now());
```

Output:

```
     k      |                    v
------------+-------------------------------------------
 session:42 | {"cart": ["LAP-13", "TSH-01"], "user": 42}

 weather_hits
--------------
            0
```

The expired `weather:paris` key isn't returned. The leaderboard is a window function (Chapter 8), and ties share a rank:

```
 rank | user_name | steps
------+-----------+-------
    1 | cy        | 15010
    2 | ana       | 12050
    3 | bo        |  8300
    3 | di        |  8300
```

Cleanup is `DELETE FROM kv WHERE expires_at <= now() RETURNING k`, which evicts `weather:paris`. Redis does this expiry automatically. And `pg_class.relpersistence = 'u'` confirms the table is unlogged. This is fine for moderate loads. Redis still wins on latency, on built-in expiry and on data structures.

Common mistakes

- Using a cache as the **source of truth** (the authoritative copy every other copy derives from) without persistence, then losing data on restart.
- Huge values (MBs) or "hot keys" (single keys getting a large share of all traffic) that one shard must serve alone.
- Forgetting TTLs, so memory fills and eviction starts dropping useful keys.

In the interview

30-second answer: "A key-value store is a distributed hash map: O(1) get/put by key, easy to shard and replicate, no queries by value. Redis adds in-memory speed and data structures like sorted sets. I use it for caching, sessions, counters, rate limiting and leaderboards, with a TTL on keys, and keep the durable truth in a primary database."

- *How would you build a rate limiter?* `INCR key` with `EXPIRE` for a fixed window, or a sorted set of timestamps for a sliding window.
- *Cache invalidation strategy?* **Cache-aside** (the app checks the cache, and on a miss reads the database and fills the cache) with TTLs, and delete or refresh the key on writes. See the caching patterns.

Self-check

1. *Why can't a pure KV store answer "all sessions of user 42"?* It only looks up by key. You'd need a secondary key such as `user:42:sessions` maintained by the app.
2. *What does Redis's sorted set give a leaderboard?* Members kept ordered by score, with O(log n) updates and fast range queries.
3. *What does `UNLOGGED` trade in Postgres?* Speed (no WAL) against durability. The table is truncated after a crash and is not replicated.

TL;DR

Key-value = dictionary at scale. It's the fastest way to get by key, with nothing beyond that. Perfect for caches, sessions, counters and leaderboards.

16.3 Document databases

What it is

A **document database** stores data as self-contained **documents**, usually JSON or a binary form of it (MongoDB's **BSON**). Documents are grouped into **collections** (roughly, tables), and collections into a **database**. Every document has a unique id (`_id` in MongoDB). Two documents in one collection may carry **different fields** and can **nest** objects and arrays.

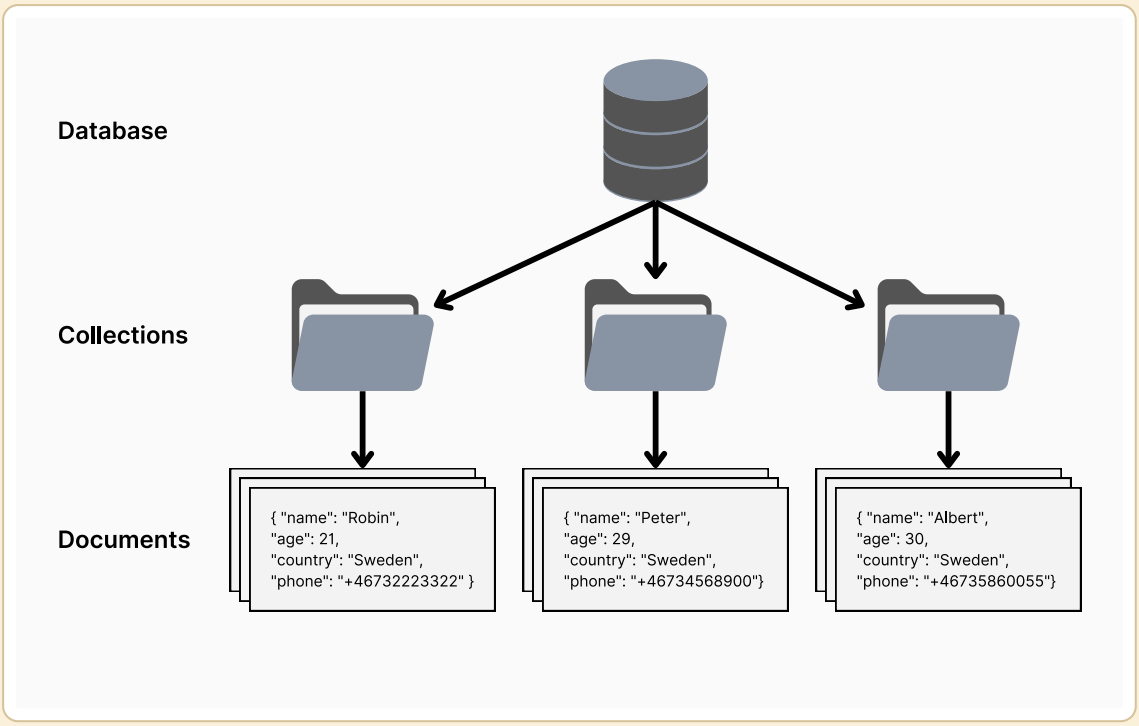


Figure — A document database: a database holds collections, and each collection holds JSON documents (course diagram).

```
{ "_id": 1, "name": "Meera", "city": "Pune",  "phone": "+91 98xxxxxx01" }
{ "_id": 2, "name": "Tomás", "city": "Porto", "languages": ["pt", "en"] }
```

The second document has a field the first lacks, and that's allowed.

Key features

1. **Flexible schema**: add fields without migrations. A product catalogue can hold electronics with `warranty` and `brand` next to clothing with `size` and `material`.
2. **Nested structures**: a customer document can **embed** their addresses and recent orders, so one read returns everything a page needs.

3. **Performance without joins**: data that is read together is stored together.
4. **Horizontal scaling**: sharding by a shard key is built in (MongoDB).

Popular systems

- **MongoDB**: BSON documents, a rich query language and **secondary indexes** (indexes on fields other than the primary id), **aggregation pipelines** (`$match` → `$group` → `$sort`, similar to SQL's `WHERE`/`GROUP BY`), replica sets (single-leader, 15.5), sharding, and multi-document transactions since 4.0.
- **CouchDB**: JSON over a **RESTful HTTP API** (resources addressed by URLs and manipulated with standard web verbs such as GET, PUT and DELETE), multi-leader **replication and sync** designed for occasionally connected devices (PouchDB in the browser syncs with it), and **MVCC** (multi-version concurrency control — keeping several versions of a record) document revisions for conflict detection.

```
// MongoDB shell (not run)
db.products.insertOne({ sku: "LAP-15", type: "laptop", price: 1399, specs: { ram_gb: 32 } })
db.products.find({ type: "laptop", "specs.ram_gb": { $gte: 16 } }).sort({ price: 1 })
db.products.aggregate([{ $group: { _id: "$type", avgPrice: { $avg: "$price" } } }])
db.products.createIndex({ type: 1, price: 1 })
```

Use cases

Use	Example
Content management	Blog posts with title, body, tags, author and a varying set of metadata
Product catalogues	Each category has different attributes, all in one collection
User profiles and settings	Optional, evolving fields per user
Event and activity logging	A game records player actions as documents for later analysis

Modelling: embed or reference?

Embed (nest inside the parent)	Reference (store the id, look it up separately)
Data is read together and owned by the parent (a customer's addresses)	Data is shared by many parents (a product referenced by many orders)
Bounded size (a few addresses)	Unbounded growth (all of a user's comments ever)
One read, atomic single-document update	Needs a second query or `$lookup` (the document world's join)

Strengths and limits

Strengths	Limits
Schema flexibility for evolving data	**Many-to-many relationships** and cross-document constraints are awkward. Relational handles them better.
Fast reads when documents match the page	**Duplicated data** (denormalised) must be kept in sync by the app
Built-in scale-out	Some configurations favour availability over strong consistency (e.g. reading from secondaries)

A document store inside Postgres: JSONB

PostgreSQL's `jsonb` type stores parsed, binary JSON. It supports operators, indexing and **SQL/JSON path** queries (a mini-language like `$.authors[*]` for navigating inside a JSON value). `code/01_jsonb_document_store.sql` builds a mini product catalogue:

```
CREATE TABLE products (
  id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  doc jsonb NOT NULL
);
INSERT INTO products (doc) VALUES
 ('{"sku":"LAP-13","type":"laptop","name":"Aero 13","price":999,
    "brand":"Nimbus","warranty_years":2,"specs":{"ram_gb":16,"cpu":"8-core"}}'),
 ('{"sku":"TSH-01","type":"tshirt","name":"Plain Tee","price":15,
    "sizes":["S","M","L"],"material":"cotton"}'),
 ('{"sku":"LAP-15","type":"laptop","name":"Aero 15","price":1399,
    "brand":"Nimbus","warranty_years":3,"specs":{"ram_gb":32,"cpu":"12-core"}}'),
 ('{"sku":"BK-042","type":"book","name":"Data at Scale","price":45,
    "authors":["R. Rao","M. Chen"],"pages":412}');

SELECT doc->>'sku' AS sku, (SELECT string_agg(k, ',' ORDER BY k) FROM jsonb_object_keys(doc) k) AS keys
FROM products ORDER BY id;
```

```
  sku   |                      keys
--------+-------------------------------------------------
 LAP-13 | brand,name,price,sku,specs,type,warranty_years
 TSH-01 | material,name,price,sizes,sku,type
 LAP-15 | brand,name,price,sku,specs,type,warranty_years
 BK-042 | authors,name,pages,price,sku,type
```

Every row has its own shape, just like documents in a collection.

Querying. `->` returns JSON, `->>` returns text, and `#>>` follows a path:

```
SELECT doc->>'name' AS name, (doc->>'price')::numeric AS price, doc#>>'{specs,ram_gb}' AS ram_gb
FROM products WHERE doc->>'type' = 'laptop' ORDER BY price;
```

```
  name   | price | ram_gb
---------+-------+--------
 Aero 13 |   999 | 16
 Aero 15 |  1399 | 32
```

Containment `@>` means "does the document contain this sub-document?". Arrays match if they contain the listed elements:

```
SELECT doc->>'name' AS name FROM products WHERE doc @> '{"sizes":["M"]}';                       -- Plain Tee
SELECT doc->>'name' AS name FROM products WHERE doc @> '{"brand":"Nimbus","specs":{"ram_gb":32}}'; -- Aero 15
SELECT doc->>'name' AS name FROM products WHERE doc ? 'authors';                                -- Data at Scale
SELECT jsonb_path_query(doc, '$.authors[*]') AS author FROM products WHERE doc->>'sku' = 'BK-042';
```

```
  author
-----------
 "R. Rao"
 "M. Chen"
```

Partial updates use `jsonb_set` (set a path), `||` (merge keys) and `-` (remove a key):

```
UPDATE products SET doc = jsonb_set(doc, '{price}', '1299') WHERE doc->>'sku' = 'LAP-15';
UPDATE products SET doc = doc || '{"on_sale":true}'         WHERE doc->>'sku' = 'TSH-01';
UPDATE products SET doc = doc - 'warranty_years'            WHERE doc->>'sku' = 'LAP-13';
```

```
  sku   | price | on_sale | warranty
--------+-------+---------+----------
 LAP-13 | 999   |         |
 TSH-01 | 15    | true    |
 LAP-15 | 1299  |         | 3
 BK-042 | 45    |         |
```

(Under the hood Postgres rewrites the whole row version. JSONB is not updated in place, which is why huge, frequently updated documents are a poor fit.)

Indexing. After adding 20,000 generated products:

```
CREATE INDEX products_doc_gin ON products USING gin (doc jsonb_path_ops);   -- serves @> on ANY key
EXPLAIN (COSTS OFF) SELECT count(*) FROM products WHERE doc @> '{"sku":"GEN-777"}';

CREATE INDEX products_price_idx ON products (((doc->>'price')::numeric));  -- B-tree on ONE field
EXPLAIN (COSTS OFF) SELECT doc->>'sku' FROM products WHERE (doc->>'price')::numeric > 503;
```

```
 Aggregate
   ->  Bitmap Heap Scan on products
         Recheck Cond: (doc @> '{"sku": "GEN-777"}'::jsonb)
         ->  Bitmap Index Scan on products_doc_gin
               Index Cond: (doc @> '{"sku": "GEN-777"}'::jsonb)

 Bitmap Heap Scan on products
   Recheck Cond: (((doc ->> 'price'::text))::numeric > '503'::numeric)
   ->  Bitmap Index Scan on products_price_idx
         Index Cond: (((doc ->> 'price'::text))::numeric > '503'::numeric)
```

The **GIN** index (generalised inverted index, Chapter 12 — it maps each key/value inside the documents to the rows containing it) covers containment queries on any key. (A **Bitmap Index Scan** collects matching row locations from the index; the **Bitmap Heap Scan** then fetches those rows from the table.) `jsonb_path_ops` is smaller and faster for `@>` but doesn't support `?`, while the default `jsonb_ops` supports both. An **expression B-tree** (a normal sorted index built on the result of an expression rather than a plain column) serves range and sort queries on one field.

Guard rails that document databases often lack. A `CHECK` constraint can enforce required keys and types:

```
ALTER TABLE products ADD CONSTRAINT doc_has_sku_and_price
  CHECK (doc ? 'sku' AND jsonb_typeof(doc->'price') = 'number');
-- inserting '{"name":"No SKU","price":"cheap"}' now fails:
-- NOTICE:  rejected: new row for relation "products" violates check constraint "doc_has_sku_and_price"
```

A common hybrid design is **regular columns for the stable, queried fields** (sku, price, type) plus a **JSONB column for the variable attributes**. That gives relational integrity where it matters and flexibility where it helps. More JSON functions are in Chapter 14.

In the interview

30-second answer: "A document store keeps self-contained JSON documents in collections, with flexible schemas and nesting, so a page can be served by one read. I embed data that's owned and read together, and reference data that's shared or unbounded. It's weaker at many-to-many relationships and cross-document constraints. If I also need joins and transactions, Postgres JSONB with a GIN index gives most of the benefit."

- *`json` vs `jsonb` in Postgres?* `json` stores the exact text (keeps whitespace, duplicate keys and key order). `jsonb` stores a parsed binary form that is indexable and faster to query. Use `jsonb`.
- *How do you index a JSONB field for range queries?* A B-tree expression index on `((doc->>'field')::type)`.

Self-check

1. *Which operator asks "does this JSON contain that sub-JSON"?* `@>`.
2. *When should you reference instead of embed?* When the child data is shared or grows without bound.
3. *Which index type serves `@>` on any key?* GIN.

TL;DR

Documents = flexible, nested records read in one go. Model around your queries. In Postgres, `jsonb` + GIN + CHECK constraints gives you a document store with SQL around it.

16.4 Column-family and wide-column stores

What it is

A **column-family** database stores rows identified by a **row key**. Each row's columns are grouped into **column families**, which are named groups of related columns stored together on disk. Rows don't need the same columns, and a row can have thousands of them, hence **"wide"**.

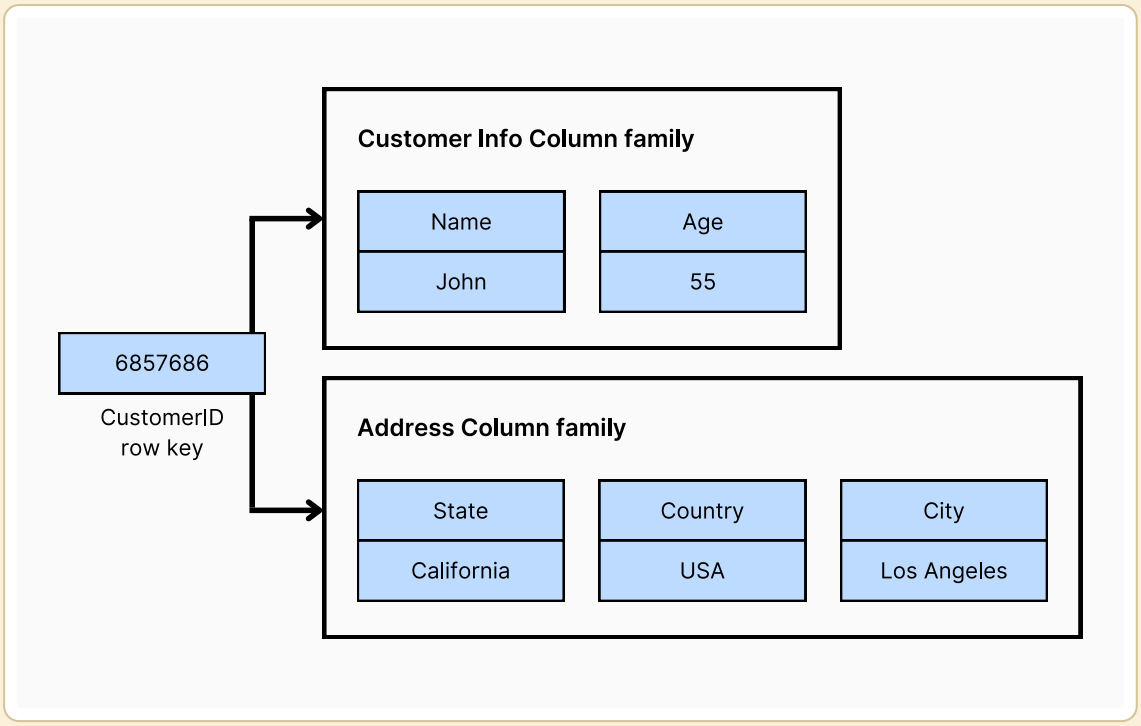


Figure — Row key `CustomerID` with a "Customer Info" family (Name, Age) and an "Address" family (State, Country, City) (course diagram).

For example, row key `cust#6857686` might have:

- family **info**: `name = "John"`, `age = 55`
- family **address**: `state = "California"`, `country = "US"`, `city = "..."`

A query for the address reads only the `address` family's storage. This is the **column-family model** of Google's Bigtable (2006), which HBase copied.

The **wide-column model**, as used by Cassandra and ScyllaDB, builds on it for distribution:

- A **partition key** decides which node stores the row (consistent hashing, 15.3). All rows with the same partition key live together.
- **Clustering columns** sort the rows *within* a partition, which makes range scans within a partition cheap.
- Replication is typically **eventually consistent** with **tunable** consistency.

```
-- Cassandra CQL (not run): one partition per server per day, rows sorted by time
CREATE TABLE metrics (
  server_id text, day date, ts timestamp, cpu_pct float, mem_mb int,
  PRIMARY KEY ((server_id, day), ts)          -- (partition key), clustering column
) WITH CLUSTERING ORDER BY (ts DESC);
SELECT ts, cpu_pct FROM metrics WHERE server_id = 'S1' AND day = '2024-12-06' LIMIT 60;
```

> **Don't confuse** *wide-column* stores (Cassandra, HBase) with **columnar** analytics databases (ClickHouse, Redshift, BigQuery, Parquet files). Columnar stores keep each *column* of a table contiguous on disk so that `SUM(amount)` over billions of rows reads only one column. Wide-column stores keep each *row partition* together, organised by key. The names are similar but the purposes are different (DDIA ch.3 — column-oriented storage).

Why they fit time-series and write-heavy data

- **Sequential writes**: most use **LSM trees** (log-structured merge trees). Writes go to an in-memory table plus a commit log, then are flushed as **immutable** (never modified after writing) sorted files (**SSTables**, sorted string tables) that are merged in the background (**compaction**). Writes are appends, so throughput is very high.
- **Partitioning by time range**: a partition key such as `(sensor, day)` spreads load and keeps each day's data together.
- **Flexible columns**: a new metric is just a new column, with no schema migration for the existing rows.

server_id	ts	cpu_pct	mem_mb
web-1	2024-12-06 10:00	41	1980
web-1	2024-12-06 10:01	44	2012
db-1	2024-12-06 10:00	63	1705

A server-monitoring feed: partitioned by server, clustered by time, so "last hour of web-1" is one sequential read.

Cassandra vs HBase

	Cassandra (wide-column)	**HBase** (column-family)
Origin	Facebook (2008), combining Dynamo's distribution with Bigtable's data model	Apache clone of Google Bigtable, on Hadoop
Architecture	Leaderless ring, every node equal	Master + RegionServers, data in **HDFS** (Hadoop's distributed file system, 16.9)
Scaling unit	Token ranges on the ring	**Regions** (key ranges) that split automatically as they grow
Consistency	**Tunable** per query (`ONE`, `QUORUM`, `ALL`), AP by default	**Strong** per-row consistency (CP)
Query	CQL (Cassandra Query Language: SQL-like, no joins)	Java API, get/put/scan by row key, or SQL through Apache Phoenix
Typical use	User activity feeds, messaging, IoT at global scale	Big-data random read/write next to Hadoop and Spark jobs, e.g. transaction histories for fraud analysis

Use cases

- **Time-series and IoT**: temperature and humidity from millions of smart-home sensors.
- **Logs**: application and server logs for fast troubleshooting.
- **Activity and recommendation data**: every view and purchase per user, fed into recommenders.
- **Real-time analytics**: counters and recent windows for dashboards and alerts, such as price movements.

Limits: you must design tables **per query** (the same data is often written to several tables), there are no joins, secondary indexes are limited and ad-hoc queries are hard, and deletes create **tombstones** (markers saying "this was deleted", since immutable files can't be edited) that slow reads until compaction.

The same idea in Postgres

`code/04_time_series_and_wide_rows.sql` mirrors the wide-column layout. A composite primary key `(server_id, ts)` acts like partition key + clustering column, and range partitions by day provide retention:

```
CREATE TABLE metrics (
  server_id text        NOT NULL,       -- like a wide-column "partition key"
  ts        timestamptz NOT NULL,       -- like a "clustering key": rows sorted by time
  cpu_pct   real,
  mem_mb    int,
  PRIMARY KEY (server_id, ts)
) PARTITION BY RANGE (ts);
-- one partition per day; 3 servers x 2 days x 1,440 minutes = 8,640 readings
EXPLAIN (COSTS OFF)
SELECT ts, cpu_pct FROM metrics
WHERE server_id = 'web-1' AND ts >= '2024-12-06 23:30' AND ts < '2024-12-07 00:00';
```

```
 Bitmap Heap Scan on metrics_2024_12_06 metrics
   Recheck Cond: ((server_id = 'web-1'::text) AND (ts >= '2024-12-06 23:30:00+00'::timestamp with time zone) AND
...)
   ->  Bitmap Index Scan on metrics_2024_12_06_pkey
         Index Cond: ((server_id = 'web-1'::text) AND (ts >= '2024-12-06 23:30:00+00'::timestamp with time zone)
AND ...)
```

One partition is scanned, and inside it the primary key's `(server_id, ts)` order gives a narrow range scan, which is exactly the wide-column access pattern. (More in 16.8.)

In the interview

30-second answer: "Wide-column stores like Cassandra distribute rows by a partition key and sort them within the partition by clustering columns. Writes go to LSM trees, so they're append-fast. You design one table per query and live with no joins. Consistency is tunable per request. They're great for time-series, logs and activity feeds at huge write rates. They aren't the same as columnar analytics stores."

- *How would you model "latest 50 messages per chat" in Cassandra?* Partition key `chat_id` (plus a time bucket if chats are huge), clustering column `sent_at DESC`, then `LIMIT 50`.
- *Why bucket the partition key by day?* To keep partitions bounded in size (Cassandra's guidance is roughly 100 MB or less per partition) and to spread hot writes.

Self-check

1. *What decides which node stores a Cassandra row?* The hash of its partition key.
2. *What orders rows inside a partition?* The clustering columns.
3. *HBase consistency vs Cassandra default?* HBase is strongly consistent per row. Cassandra is tunable, eventual by default.

TL;DR

Wide-column = partition key for placement + clustering key for order + LSM for fast writes. One table per query. Built for massive write-heavy, time-ordered data.

16.5 Graph databases

What it is

A **graph database** stores **nodes** (entities such as people, products or places), **edges** (relationships such as `KNOWS`, `BOUGHT`, `MEMBER_OF`) and **properties** on both (`name`, `since`, `weight`). Edges are first-class, stored data, not something computed at query time from foreign keys.

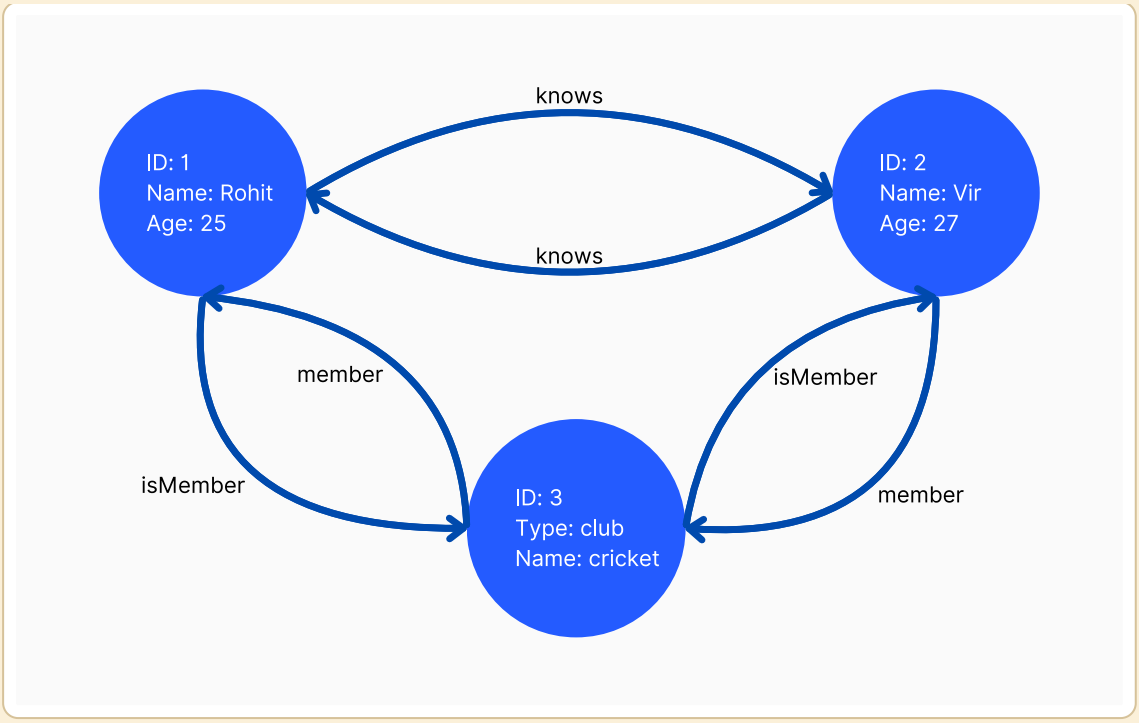


Figure — Two people connected by `knows` edges, each a member of a cricket club (`isMember`) (course diagram).

Choose a graph database when the **relationships matter as much as, or more than, the records**: social connections, recommendations, fraud rings, network topology, knowledge graphs, access-control hierarchies.

Relational vs graph for the same data

Relational: a `person` table plus a `knows(src, dst)` link table. For friendships, store both directions or query both columns:

src	dst
1 (Ava)	2 (Bilal)
1 (Ava)	3 (Chen)
2 (Bilal)	4 (Dana)
…	…

Finding one person's friends means looking up the person's id, scanning or index-seeking the link table for matching `src`, and joining back to `person` for names. Each extra hop ("friends of friends of friends") adds **another join**. With B-tree lookups, each hop costs O(log n) per edge, and intermediate results can explode.

Graph: each node stores direct pointers to its edges (Neo4j calls this **index-free adjacency**). Finding friends means going to the node and following its `KNOWS` edges. The cost depends on the number of edges traversed, **not the total size of the database**, so deep, variable-length traversals stay fast.

```
// Neo4j Cypher (not run)
CREATE (a:Person {name:'Ava'})-[:KNOWS]->(b:Person {name:'Bilal'});
MATCH (me:Person {name:'Ava'})-[:KNOWS]->(f)-[:KNOWS]->(fof)
WHERE NOT (me)-[:KNOWS]->(fof) AND fof <> me
RETURN DISTINCT fof.name;                                  // friend suggestions
MATCH p = shortestPath((:Person {name:'Ava'})-[:KNOWS*]-(:Person {name:'Goran'}))
RETURN [n IN nodes(p) | n.name];
```

Advantages: relationship queries are **simpler to write** (patterns instead of join chains), **fast at depth** ("who are Chen's friends?" or six hops away), and the model is **flexible** (add new node or edge types freely).

Query languages: **Cypher/openCypher** (Neo4j, Neptune, Memgraph), **Gremlin** (Apache TinkerPop), **SPARQL** (RDF triple stores), and the new ISO standard **GQL** (ISO/IEC 39075, 2024). SQL:2023 also added **SQL/PGQ** for property-graph queries over relational tables.

Use cases

1. **Social networks**: mutual friends, degrees of separation, "people you may know".
2. **Recommendations**: nodes for users, products and categories, and edges for bought, liked and similar-to. "Customers who bought this also bought…" becomes a two-hop traversal.
3. **Fraud detection**: accounts sharing devices, addresses or cards form rings that are easy to spot as **connected components** (groups of nodes all reachable from one another).
4. **Network/IT dependency maps and knowledge graphs.**

Scaling limits

- **Horizontal sharding is hard**: cutting a highly connected graph across machines means traversals cross the network on almost every hop. Many graph databases scale reads with replicas but keep the graph on one machine (or require careful partitioning).
- **Bulk updates** of every node matching a property are slower than in column or document stores.
- **Aggregations over everything** (totals, reports) are not their strength. Export to a warehouse for that.

Graph queries in Postgres

`code/03_graph_in_sql.sql` models seven people and symmetric `knows` edges, then walks the graph with recursive **CTEs** (common table expressions — named sub-queries in a `WITH` clause; a recursive one repeatedly feeds its output back in) (Chapter 7):

```
-- 2 hops: friend-of-friend suggestions for Ava
SELECT DISTINCT p.name AS suggestion
FROM knows k1 JOIN knows k2 ON k2.src = k1.dst JOIN person p ON p.id = k2.dst
WHERE k1.src = 1 AND k2.dst <> 1
  AND k2.dst NOT IN (SELECT dst FROM knows WHERE src = 1);

-- N hops: degrees of separation (path array prevents cycles)
WITH RECURSIVE walk(id, depth, path) AS (
  SELECT 1, 0, ARRAY[1]
  UNION ALL
  SELECT k.dst, w.depth + 1, w.path || k.dst
  FROM walk w JOIN knows k ON k.src = w.id
  WHERE k.dst <> ALL (w.path) AND w.depth < 6
)
SELECT p.name, min(depth) AS degrees
FROM walk JOIN person p USING (id) GROUP BY p.name ORDER BY degrees, p.name;
```

Output (Ava's friends are Bilal and Chen, the suggestion, degrees, and the shortest path to Goran):

```
 suggestion
------------
 Dana

  name  | degrees
--------+---------
 Ava    |       0
 Bilal  |       1
 Chen   |       1
 Dana   |       2
 Eli    |       3
 Fatima |       4
 Goran  |       5

                 shortest_path
-----------------------------------------------
 Ava -> Bilal -> Dana -> Eli -> Fatima -> Goran
```

This works well for small graphs and shallow depths. Recursive CTEs enumerate **every path**, so on large, dense graphs the work grows exponentially with depth. That's the point where a graph engine (or the Apache AGE extension, which adds Cypher to Postgres) pays off.

In the interview

30-second answer: "A graph database stores nodes, edges and properties, and follows edges directly, so multi-hop queries cost in proportion to the edges touched rather than the table sizes. It's ideal for social graphs, recommendations and fraud rings. It's harder to shard and weaker at bulk aggregates. For a few hops on modest data, SQL with a link table and recursive CTEs is fine."

- *Why are deep joins slow in SQL?* Each hop is another join over the whole link table (through an index), and intermediate results multiply.
- *How would you detect a fraud ring?* Link accounts to shared devices, cards and addresses, then find connected components or short cycles.

Self-check

1. *What's index-free adjacency?* Each node points directly to its neighbours, so traversal needs no index lookup.
2. *Why store friendships in both directions in the SQL model?* So a single `src = ?` lookup finds all friends of a person.
3. *Name two graph query languages.* Cypher and Gremlin (also SPARQL and GQL).

TL;DR

Graphs make relationships first-class. Use one when the questions are "how is X connected to Y". Recursive CTEs cover the small cases.

16.6 In-memory databases

What it is

An **in-memory database (IMDB)** keeps its primary copy of the data in **RAM** rather than on disk. RAM access takes about 100 **nanoseconds** (billionths of a second), while an SSD read takes tens to hundreds of **microseconds** (millionths of a second). The engine can use data structures designed for memory (**hash tables** — arrays indexed by a key's hash for near-instant lookup; **skip lists** — sorted linked lists with "express lanes" for fast search; compact column vectors) instead of disk pages and buffer management.

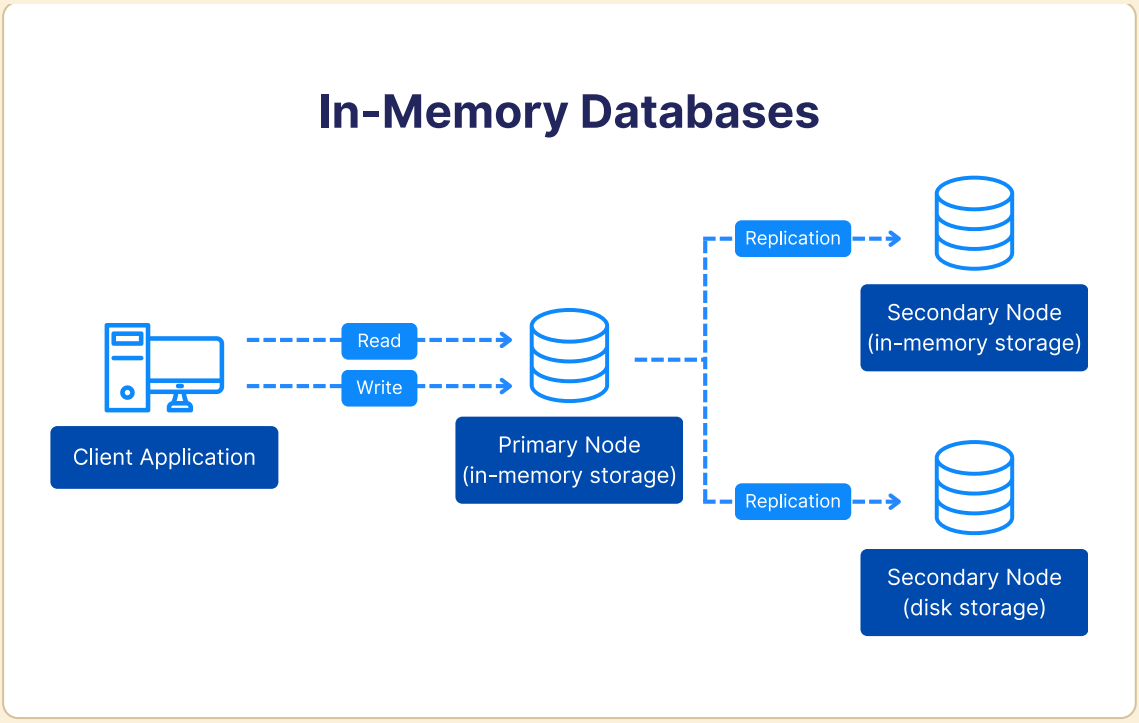


Figure — The client reads and writes the primary node's in-memory store, and data is replicated to secondaries, some of which persist to disk (course diagram).

Architecture

1. **Primary node**: holds the working data in memory and serves reads and writes.
2. **Replication**: changes stream to **secondary nodes** (in memory, or disk-backed) for availability.
3. **Persistence**: because RAM is **volatile** (lost on power-off or crash), IMDBs write **snapshots** and/or an **append-only / write-ahead log** to disk. On restart they reload the snapshot and replay the log. This is the same WAL idea as in 15.9.

Benefits

Benefit	Example
Speed	A user's recent purchase history in well under a millisecond
Real-time processing	Live game scores and leaderboards, fraud checks during card authorisation, IoT alerts
Simpler, memory-optimised structures	Redis's native hashes and sorted sets
High concurrency	Hundreds of thousands of operations per second per node

Limitations

- **Volatility**: without persistence you lose data on restart, and with it you add **I/O** (input/output — reading and writing disk or network) and recovery time.
- **Cost**: RAM costs far more per GB than disk, so large datasets are expensive.
- **Scaling**: adding capacity means adding RAM on more nodes, and data size is bounded by total cluster memory.

Real systems

- **SAP HANA**: an in-memory, **column-oriented** database that runs **OLTP and OLAP on the same data** (OLTP = online transaction processing, many small reads/writes like placing orders; OLAP = online analytical processing, big scans and aggregates for reports) (sometimes called HTAP, hybrid transactional/analytical processing). *Example:* a retailer analyses today's sales in real time and adjusts stock orders the same afternoon.

- **SingleStore** (called **MemSQL** until 2020): distributed SQL with in-memory row storage plus disk-based columnstore tables, and MySQL-wire-compatible (speaks MySQL's network protocol, so MySQL drivers work) **ANSI SQL** (the standard SQL language). *Example:* a bank ingests millions of card transactions a day and scores each one for fraud in real time.
- **Redis** (16.2), **Memcached** (a cache only, no persistence), and **VoltDB** (in-memory NewSQL).

Postgres note: Postgres is not an IMDB, but with enough `shared_buffers` (Postgres's own in-memory page cache) and OS cache, its hot data is served from memory anyway. `UNLOGGED` tables (16.2) skip the WAL for cache-like data. The on-disk WAL is what lets it survive a crash.

In the interview

> **30-second answer:** "In-memory databases keep data in RAM for microsecond access and use snapshots plus an append-only log, plus replicas, for durability. They're ideal for caches, sessions, leaderboards and real-time analytics. The trade-offs are cost per GB and dataset size limited by memory."

- *How does Redis survive a restart?* RDB snapshots and/or AOF (append-only file) replay, and replicas.

Self-check

1. *What makes RAM data durable?* Snapshots and logs on disk, plus replication.
2. *What's HTAP?* Running transactional and analytical workloads on the same live data (SAP HANA, SingleStore).

TL;DR

RAM-first storage for speed. Durability comes from logs, snapshots and replicas. The price is cost and a size ceiling.

16.7 NewSQL / distributed SQL

What it is

NewSQL (today usually called **distributed SQL**) databases keep the **relational model, SQL and ACID transactions** but **scale out horizontally** like NoSQL, with automatic sharding, replication and failover.

The gap they fill

- **Traditional SQL** (MySQL, PostgreSQL): strong consistency and ACID, but designed around one primary node. Scaling writes means scaling up or sharding by hand, and cross-shard transactions are your problem.
- **NoSQL** (MongoDB, Cassandra): scale-out and flexibility, but historically weaker guarantees (eventual consistency, limited transactions). That's risky for balances, inventory and bookings.

NewSQL aims to offer **both**: SQL and ACID *at* horizontal scale.

How it works

1. **Automatic sharding.** Tables are split into ranges (CockroachDB calls them *ranges*, TiDB *regions*, Spanner *splits*) that move between nodes automatically as they grow or get hot. *Example:* a ride-hailing app's trips spread across nodes, so one busy city doesn't slow the others.
2. **Consensus-based replication.** Each shard is a **Raft** (CockroachDB, TiDB, YugabyteDB) or **Paxos** (Spanner) group of usually 3 or 5 replicas. A write commits once a majority stores it, so the loss of a minority loses nothing and needs no manual failover (15.8).
3. **Distributed transactions.** Transactions spanning shards use **2PC whose coordinator state is itself replicated by consensus**, which avoids 2PC's blocking problem (15.9). They provide **serializable** isolation (concurrent transactions behave as if run one after another), the default in CockroachDB and Spanner.

4. **Clock handling for global consistency.** **Google Spanner** uses **TrueTime**, GPS- and atomic-clock-backed time with a known uncertainty bound. It waits out that uncertainty before acknowledging a commit ("commit wait"), which gives externally consistent (**linearizable** — every reader sees commits in real-time order) transactions worldwide. CockroachDB uses **hybrid logical clocks** (timestamps combining the machine's physical clock with a counter that keeps causally related events ordered) on ordinary servers, with an uncertainty interval and retries.
5. **SQL front end**: familiar SQL, and often a wire-compatible protocol (CockroachDB and YugabyteDB speak PostgreSQL's protocol, TiDB speaks MySQL's).

Examples

System	Notes
Google Spanner (paper 2012, Cloud Spanner service 2017)	Global, Paxos + TrueTime, strict serializability (serializable + linearizable). *Use:* a global shop keeping stock counts correct across regions during a flash sale.
CockroachDB	Postgres-compatible, Raft per range, serializable by default, survives zone or region loss. *Use:* a fintech processing payments across several data centres.
TiDB	MySQL-compatible, TiKV storage (Raft), with an optional columnar replica (TiFlash) for HTAP.
YugabyteDB	Reuses PostgreSQL's query layer on a Raft-replicated distributed store.
VoltDB	In-memory, partitioned, serial execution per partition.

Where it fits: global apps that need correct transactions (payments, inventory, bookings), multi-region active-active with strong consistency, and real-time queries over distributed data. The costs are higher write latency (consensus round trips, and cross-region round trips for global writes), more complex operations, and a SQL dialect that is compatible but not identical.

SQL vs NoSQL vs NewSQL

Feature	Traditional SQL	NoSQL	NewSQL
Data model	Relational tables	Key-value, document, wide-column, graph	Relational tables
Schema	Defined up front (JSON columns add flexibility)	Dynamic	Defined up front, with JSON support
Scaling	Vertical + read replicas, manual sharding	Horizontal, built in	Horizontal, built in
Consistency	Strong, ACID	Often eventual/tunable (BASE)	Strong, ACID (serializable)
Typical use	ERP (enterprise resource planning: finance, stock, HR systems), banking, most business apps	Caches, feeds, IoT, catalogues	Global transactional apps, high-scale OLTP
Examples	PostgreSQL, MySQL, SQL Server	Redis, MongoDB, Cassandra, Neo4j	Spanner, CockroachDB, TiDB, YugabyteDB

In the interview

30-second answer: "NewSQL keeps SQL and ACID but scales out. Data is auto-sharded into ranges, each replicated with Raft or Paxos, and cross-shard transactions use 2PC on top of those consensus groups, so there's no single blocking coordinator. Spanner uses TrueTime for global external consistency, and CockroachDB uses hybrid logical clocks. The cost is write latency from consensus."

- *Why is a global write slower in Spanner/CockroachDB?* A majority of replicas, possibly in other regions, must acknowledge it, and Spanner adds the commit-wait.

Self-check

1. *Which consensus protocol does CockroachDB use?* Raft.
2. *What problem does TrueTime solve?* Ordering transactions globally without a central clock, by waiting out a bounded clock uncertainty.

TL;DR

NewSQL = relational + ACID + automatic scale-out through consensus-replicated shards. You pay for it with latency.

16.8 Time-series and spatial databases

Time-series databases

What it is

A **time-series database (TSDB)** is optimised for **sequences of timestamped measurements**: metrics, sensor readings, prices. Data arrives in time order, is mostly appended and rarely updated, and is queried by **time range**, often **downsampled** (averages per minute or hour).

How they index and store temporal data

- **Time-partitioned storage**: data lives in chunks per hour, day or week, so "last 7 days" touches only 7 chunks, and dropping old data drops whole chunks. *Example:* smart-home sensor readings stored one partition per day.
- **B-trees and LSM trees**: B-trees serve range scans. **LSM trees** make high-rate appends cheap (InfluxDB's engine, Cassandra and others; see 16.4).
- **Compression**: consecutive timestamps and values are similar, so encodings like **delta** (store the difference from the previous value), delta-of-delta and XOR floats (Facebook's **Gorilla** paper, 2015) shrink data by around 10×. Columnar layouts help too.

Features

Feature	Meaning
Write optimisation	Millions of points per second
Retention policies	Delete (or downsample) data older than N days automatically
Continuous aggregates / downsampling	Keep per-minute data for a week and per-hour data for a year
Time functions	Buckets, gap filling, interpolation, "last value"

Applications

- **IoT**: temperature, humidity and power usage in homes and factories.
- **Server monitoring**: CPU, memory and network metrics, as in **Prometheus** (its own TSDB) and the Datadog platform.
- **Finance**: tick-by-tick prices and volumes for trend analysis.

Products: **InfluxDB**, **TimescaleDB** (a PostgreSQL extension with automatic time partitions called *hypertables*, compression and continuous aggregates), **Prometheus**, **QuestDB**, kdb+.

Time-series in plain Postgres

The rest of `code/04_time_series_and_wide_rows.sql` shows the TSDB toolkit using built-in features:

```sql
-- Downsample: 6-hour buckets per server (date_bin: PostgreSQL 14+)
SELECT server_id, date_bin('6 hours', ts, timestamptz '2024-12-06') AS bucket,
       round(avg(cpu_pct)::numeric, 1) AS avg_cpu, max(mem_mb) AS peak_mem
FROM metrics WHERE ts >= '2024-12-06'
GROUP BY 1, 2 ORDER BY 1, 2 LIMIT 6;
```

```
 server_id |         bucket         | avg_cpu | peak_mem
-----------+------------------------+---------+----------
 db-1      | 2024-12-06 00:00:00+00 |    42.0 |     1923
 db-1      | 2024-12-06 06:00:00+00 |    54.0 |     1915
 db-1      | 2024-12-06 12:00:00+00 |    66.0 |     1922
 db-1      | 2024-12-06 18:00:00+00 |    78.0 |     1923
 web-1     | 2024-12-06 00:00:00+00 |    22.0 |     1923
 web-1     | 2024-12-06 06:00:00+00 |    34.0 |     1917
```

```sql
CREATE INDEX metrics_ts_brin ON metrics USING brin (ts);
-- index sizes on one day's partition (4,320 rows)
```

```
           index           |  size
---------------------------+--------
 metrics_2024_12_06_pkey   | 152 kB
 metrics_2024_12_06_ts_idx | 24 kB
```

A **BRIN** index (block range index, Chapter 12) stores only min/max per block range. That's tiny, and it's ideal when rows arrive in time order. (24 kB is BRIN's minimum footprint. On millions of rows it stays small while the B-tree grows in proportion to the rows.)

```sql
-- Retention policy: dropping a day is instant (no giant DELETE, no vacuum debt).
DROP TABLE metrics_2024_12_05;
SELECT min(ts)::date AS oldest_day_kept, count(*) AS readings FROM metrics;
```

```
 oldest_day_kept | readings
-----------------+----------
 2024-12-06      |     4320
```

Spatial databases

What it is

A **spatial database** stores and queries **geometry and geography**: where things are and how shapes relate.

Type	Represents	Example
Point	A location	A shop's GPS coordinates
Line (LineString)	A path	A road, a delivery route
Polygon	An area	A city boundary, a delivery zone

Spatial indexing

Ordinary B-trees order values along one dimension and can't answer "near this point" in 2-D. Spatial indexes can:

- **R-tree**: groups nearby objects into nested **bounding rectangles**. A query descends only into the rectangles that overlap the search area. *Example:* "stores within 10 km" checks a few boxes instead of every store. PostgreSQL's **GiST** index (generalised search tree — a framework for tree indexes over non-sortable data) implements R-tree-style indexing for geometric types and PostGIS.
- **Quadtree**: recursively splits 2-D space into **four quadrants** until each cell holds few points. It is good for points and uneven density (Postgres's SP-GiST supports quadtrees).
- **Geohash**: encodes latitude/longitude as a short **string** whose shared prefix means nearby. That lets spatial lookups run on a plain key-value store or B-tree (e.g. `u09tv` is a cell covering central

Paris). Edge cases at cell borders mean you also check neighbouring cells. Related grid systems are Uber's **H3** (hexagons) and Google's **S2**.

Spatial queries

- **Proximity**: "restaurants within 5 km", "the 3 nearest drivers" (k-nearest-neighbour, KNN).
- **Intersection / containment**: "which delivery zones overlap?", "is this address inside the service area?"
- **Routing**: "shortest route from A to B" (pgRouting, graph algorithms over road networks).

Applications

- **Location-based services**: nearby restaurants and fuel stations, as in map apps.
- **Smart cities and logistics**: traffic, bus routes, rubbish-collection routes.
- **Geo-analytics**: where customers cluster, and where to open the next store.

Products: **PostGIS** (the standard geospatial extension for PostgreSQL), **BigQuery GIS**, Oracle Spatial, SQL Server spatial types, MongoDB 2dsphere indexes, Redis GEO commands.

Spatial queries with Postgres built-ins

PostGIS isn't installed on the book server, so `code/05_spatial_builtin.sql` uses Postgres's **built-in geometric types** (flat x/y plane, units here are km) with a **GiST** index:

```
CREATE TABLE store (id int PRIMARY KEY, name text, loc point NOT NULL);
-- 6 named stores + 5,000 kiosks scattered on the grid
CREATE INDEX store_loc_gist ON store USING gist (loc);    -- R-tree-like index

SELECT name, round((loc <-> point(2, 2))::numeric, 2) AS km          -- <-> = distance
FROM store WHERE id < 100 ORDER BY loc <-> point(2, 2) LIMIT 3;
SELECT name FROM store WHERE id < 100 AND loc <@ circle(point(0, 0), 5) ORDER BY name;   -- radius
SELECT name FROM store WHERE id < 100 AND loc <@ box(point(2, 3), point(7, 9)) ORDER BY name; -- zone
EXPLAIN (COSTS OFF) SELECT id FROM store ORDER BY loc <-> point(2, 2) LIMIT 3;
```

```
   name    |  km
-----------+------
 Riverside | 2.24
 Central   | 2.83
 Mall      | 3.16

   name       (within 5 km of (0,0))
-----------
 Central
 Mall
 Old Town
 Riverside

   name       (inside the delivery-zone box)
-----------
 Harbour
 Riverside

 Limit
   ->  Index Scan using store_loc_gist on store
         Order By: (loc <-> '(2,2)'::point)
```

The last plan is **KNN search through the GiST index**: the index returns points in distance order, so `LIMIT 3` stops after three, with no full sort.

For real latitude/longitude on the curved Earth, use PostGIS (`4326` below is the id of **WGS 84**, the coordinate system GPS uses):

```sql
-- PostGIS (not run)
CREATE EXTENSION postgis;
CREATE TABLE shop (id int PRIMARY KEY, name text, geom geography(Point, 4326));  -- WGS 84 lon/lat
CREATE INDEX ON shop USING gist (geom);
SELECT name FROM shop
WHERE ST_DWithin(geom, ST_MakePoint(2.3522, 48.8566)::geography, 2000);        -- within 2 km of Paris centre
SELECT z.name FROM zone z JOIN shop s ON ST_Intersects(z.area, s.geom);
```

Time-series vs spatial

Feature	Time-series	Spatial
Data focus	Timestamped measurements	Coordinates, lines, areas
Indexing	Time partitions/chunks, B-tree/LSM, BRIN, compression	R-tree (GiST), quadtree, geohash/H3/S2
Typical query	"Average CPU per hour, last 24 h"	"Shops within 10 km", "which zone contains this point"
Applications	IoT, monitoring, finance	Maps, logistics, smart cities

In the interview

30-second answer: "Time-series stores partition by time, append with LSM or B-trees, compress heavily and apply retention and downsampling. In Postgres that's time partitions, BRIN, `date_bin`, or TimescaleDB. Spatial stores use R-trees, quadtrees or geohashes for near/within/intersects queries. In Postgres that's GiST, with PostGIS for real geography."

- *Design "drivers near me" at scale?* Put driver locations in a geohash/H3 cell index (Redis GEO or an in-memory grid) updated every few seconds, query the neighbouring cells, then rank by exact distance.

Self-check

1. *Why is BRIN a good fit for time-series?* Rows arrive in time order, so per-block min/max summaries are selective and tiny.
2. *What does an R-tree store in internal nodes?* Bounding rectangles of its children.
3. *Why check neighbouring geohash cells?* Two nearby points can sit on opposite sides of a cell border.

TL;DR

Time-series: partition by time, compress, retain, downsample. Spatial: R-tree/GiST, quadtrees and geohashes for near/within/intersects. PostGIS and TimescaleDB bring both to Postgres.

16.9 Big data and distributed processing frameworks

What "big data" means

Big data is data too large, too fast or too varied for one traditional database server to store and process in reasonable time. The classic **3 Vs**:

- **Volume**: terabytes to petabytes (1 TB = 1,000 GB; 1 PB = 1,000 TB) and beyond.
- **Velocity**: data arrives continuously and fast (**clickstreams** — logs of every user click — and sensors) and must be processed quickly.
- **Variety**: structured tables, semi-structured JSON and logs, unstructured text, images and video.

(Veracity and value are often added as extra Vs.) The challenges are **storage** at that size, **processing** in acceptable time, and **scaling** as it grows. The answer is the same as in Chapter 15: **spread storage and computation over many machines and run in parallel**, moving computation to where the data is.

Hadoop: HDFS + MapReduce

- **HDFS (Hadoop Distributed File System)** splits files into large **blocks** (128 MB by default) spread across nodes, with each block **replicated** (3 copies by default) for fault tolerance.
- **MapReduce** is a programming model with a **map** phase (each node transforms its local blocks into key/value pairs), a **shuffle** (pairs are grouped by key across the network), and a **reduce** phase (each group is combined into a result). Intermediate results are written to disk between stages, which is robust but slow for multi-step jobs.

Example: a retailer stores years of sales files in HDFS and runs MapReduce jobs to total sales per product per month.

`code/mapreduce_word_count.py` runs the phases in miniature over three "nodes" of order-event logs, including a **combiner** that pre-aggregates on each node before the shuffle:

```
def map_phase(lines):                # runs locally on each node
    for line in lines:
        action, item = line.split()
        yield (f"{action}:{item}", 1)
# ... combine per node, then send each key to reducer  hash(key) % 2 ...
```

```
map+combine on node-1: [('buy:laptop', 1), ('view:laptop', 2), ('view:tshirt', 1)]
map+combine on node-2: [('buy:book', 1), ('view:book', 1), ('view:laptop', 1)]
map+combine on node-3: [('buy:laptop', 1), ('view:laptop', 1), ('view:tshirt', 1)]
reducer 0: {'buy:laptop': 2, 'view:book': 1}
reducer 1: {'buy:book': 1, 'view:laptop': 4, 'view:tshirt': 2}
```

Each key ends up at exactly one reducer, which is why the totals are complete. It's the same as `SELECT action, item, count(*) … GROUP BY action, item`, but executed across machines.

Apache Spark

Spark is a general-purpose distributed engine built to fix MapReduce's slowness:

- **In-memory processing**: intermediate results stay in RAM from one step to the next (originally as **RDDs**, resilient distributed datasets that can be recomputed from their **lineage** (the recorded chain of steps that produced them) if a node fails, and today mostly as **DataFrames** — distributed tables with named columns). That makes iterative jobs such as machine learning and multi-stage queries much faster.
- **Many workloads in one engine**: batch processing, **Structured Streaming**, machine learning (MLlib) and graph processing (GraphX).
- **APIs** in Python (PySpark), Scala, Java, R, and SQL.

Example: a social platform uses Spark to crunch behaviour logs into features for personalised recommendations.

SQL-on-Hadoop: SQL over big data

Tool	What it is
Apache Hive	A **data warehouse** (a database built for analytics over large historical data) over HDFS. HiveQL queries are compiled into distributed jobs (originally MapReduce, later Tez or Spark). Good for large batch queries on structured files.
Presto / Trino	Distributed SQL query engines (Presto from Facebook, 2012, and Trino is its fork) for **fast, interactive** queries that can **federate** (query several separate systems as if one) many sources: HDFS/S3 (Amazon's object storage) files, MySQL, Postgres, Kafka…
Spark SQL	SQL and DataFrames on Spark, sharing one optimiser with the other Spark APIs

Example: a retailer uses Trino to join in-store sales (in Postgres) with online clickstream data (**Parquet** — a compressed, column-oriented file format — on S3) in one query.

Real-time: Kafka and Flink

- **Apache Kafka** is a distributed, replicated, append-only **log**, used as a **message broker** (middleware that passes messages from senders to receivers). **Producers** append events to **topics** (named streams, split into partitions), and **consumers** read at their own pace and can replay. *Example:* a logistics company streams GPS pings from delivery vans through Kafka to live dashboards.
- **Apache Flink** is a **stream processor**: low-latency, **event-driven**, **stateful** computations (windows, joins, pattern detection) with **event-time** handling (using when an event actually happened, not when it arrived) and **exactly-once** state (each event affects results once, even after failures and retries) through distributed checkpoints (15.9). *Example:* a trading platform flags unusual trade patterns within milliseconds.

A common architecture streams events into Kafka, processes them in Flink or Spark Streaming for real-time views, and lands them in a **data lake** (cheap storage of raw files in any format, usually on object storage) or warehouse for **batch** analysis (processing large accumulated chunks at once) with Spark, Hive or Trino. **Change data capture (CDC)** — reading the database's change log and publishing every insert/update/delete as an event — from Postgres into Kafka (for example with Debezium, an open-source CDC tool) feeds the same pipeline from your OLTP database (Batch and streaming patterns).

Applications

Area	Example
IoT / sensors	Smart factories analyse machine telemetry with Spark to predict failures
Recommendations	Streaming services update suggestions from live viewing events with Flink
Fraud detection	Banks score transactions from Kafka streams with Spark/Flink in real time
Data integration	Retailers combine store and web sales with Presto/Trino

In the interview

30-second answer: "Big data is volume, velocity and variety beyond one server. Hadoop stores blocks replicated in HDFS and processes them with MapReduce (map, shuffle, reduce, via disk). Spark keeps data in memory and covers batch, streaming, ML and SQL. Hive, Trino and Spark SQL put SQL on top. For real time, Kafka is the durable event log and Flink does stateful, exactly-once stream processing."

- *Why is Spark faster than MapReduce for iterative jobs?* It keeps intermediate data in memory instead of writing it to HDFS after every step.
- *What is a combiner?* A local reduce before the shuffle that shrinks network traffic. It only works for associative operations like sum and count.

Self-check

1. *What are the 3 Vs?* Volume, velocity, variety.
2. *What does the shuffle guarantee?* All values for a key reach the same reducer.
3. *Kafka vs Flink?* Kafka stores and transports event streams, and Flink computes on them.

TL;DR

Split data into replicated blocks, move code to data, map → shuffle → reduce. Spark does it in memory. SQL engines sit on top. Kafka + Flink handle real time.

16.10 Choosing a database

Start from the **access patterns and guarantees you need**, not from fashion. For most new applications the right first answer is still **PostgreSQL**, adding JSONB, partitioning, PostGIS or TimescaleDB as needed. Add a specialised store when a requirement clearly exceeds what it does well.

If you need…	Consider	Why	Postgres can go far with
General business data, joins, constraints, transactions	**PostgreSQL / MySQL**	ACID, SQL, mature tooling	— (this is its home)
Sub-millisecond cache, sessions, counters, rate limits, leaderboards	**Redis** (Memcached for a pure cache)	In memory, TTL, data structures	`UNLOGGED` tables for moderate load
Flexible, nested, per-item attributes, read as whole objects	**MongoDB**, Firestore, CouchDB (offline sync)	Document model, easy scale-out	`jsonb` + GIN + CHECK
Massive write throughput, time-ordered per key, multi-region always-on	**Cassandra / ScyllaDB**, HBase/Bigtable	LSM writes, partitioned wide rows, tunable consistency	Partitioning + BRIN up to a point
Deep relationship queries (paths, communities, fraud rings)	**Neo4j**, Neptune	Traversals cost per edge touched	Recursive CTEs for shallow/small graphs
Metrics and IoT with retention and downsampling	**TimescaleDB**, InfluxDB, Prometheus	Time chunks, compression, continuous aggregates	Range partitions + `date_bin` + BRIN
Maps, "near me", zones	**PostGIS**, BigQuery GIS, Redis GEO	Spatial types + R-tree indexes	Built-in `point`/`box` + GiST (planar only)
SQL + ACID beyond one node, multi-region strong consistency	**CockroachDB, Spanner, YugabyteDB, TiDB**	Consensus-replicated shards	Citus for scale-out Postgres
Ad-hoc analytics over billions of rows	**Columnar warehouse**: BigQuery, Snowflake, Redshift, ClickHouse	Column storage, vectorised scans (processing batches of values per CPU instruction)	Read replicas and summary tables
Petabyte batch and stream processing	**Spark, Trino, Kafka + Flink**	Distributed compute over a data lake	CDC out of Postgres into these
Full-text search with relevance ranking and facets	**Elasticsearch / OpenSearch**	Inverted indexes, scoring	`tsvector` + GIN full-text search (Chapter 14)

Questions to ask:

1. What are the top queries, and do they need joins, ranges, aggregations or traversals?
2. What consistency does each piece of data need (15.6)? Money needs strong consistency, and like counts can be eventual.
3. What's the read/write ratio and the peak throughput? How big will the data be in 3 years?
4. Single region or multi-region? What RPO/RTO (15.10)?
5. What can the team operate? Each extra database adds backups, monitoring, upgrades and **on-call** load (engineers on rota to respond to outages).

Polyglot persistence. Large systems commonly use several stores, each for what it does best. For example: Postgres for orders and payments, Redis for sessions and caching, Elasticsearch for product search, Kafka + a warehouse for analytics, and all of it fed by CDC from the primary database so there's **one source of truth**. See the database-design case studies in Chapter 17.

In the interview

30-second answer: "I start from access patterns and consistency needs. Relational Postgres is the default for transactional data. I add Redis for caching and sessions, a document store if data is truly schemaless and read as whole objects, Cassandra for huge write-heavy time-ordered data, a graph DB for deep relationship queries, a TSDB for metrics, NewSQL if I need SQL + ACID across

> regions, and a columnar warehouse for analytics. Every extra store is operational cost, so each must earn its place."

Self-check

1. *Where should like counts vs account balances live, consistency-wise?* Like counts can be eventually consistent. Balances need strong consistency and ACID.
2. *What keeps several stores in sync in polyglot persistence?* One source of truth plus CDC/event streams feeding the others.

16.11 Inside two workhorses: Elasticsearch (and the ELK stack) and Redis

Two non-relational systems sit next to almost every production relational database: a **search engine** for text search and log analytics, and **Redis** for caching and fast data structures. §16.2 and §16.10 said *when* to use them; this section opens them up so you can explain *how* they work and where they fail.

Search engines: what it is

Elasticsearch (and its open-source **fork** — an independent copy of a project's code that is developed separately — **OpenSearch**, created by AWS in 2021 after Elastic changed its licence) is a distributed search and analytics engine built on the **Apache Lucene** library. You send it JSON documents; it makes every field searchable within about a second and answers relevance-ranked full-text queries, filters and aggregations over billions of documents. Solr is an older Lucene-based alternative.

The problem it solves

`WHERE description LIKE '%running shoe%'` cannot use a B-tree index (the pattern starts with `%`), so it scans every row, does not match "shoes" or "Running", and returns rows in no useful order. Users expect typo tolerance, **stemming** (reducing words to a root, so "running" finds "run"), **relevance ranking** (best matches first) and **facets** (per-category match counts such as "Brand: Nike (42)"). PostgreSQL's `tsvector` + GIN (14.6) covers the basics inside the database; a dedicated engine takes over when search is a core feature or the data is huge.

How it works — the inverted index

A B-tree maps *row → values*; an **inverted index** maps each **term → the list of documents containing it** (the **posting list**), like the index at the back of a book.

1. **Analysis.** At index time an **analyzer** turns text into terms: *character filters* (strip HTML), a *tokenizer* (split on word boundaries into **tokens**, i.e. individual words), then *token filters* (lowercase, remove **stop words** — very common words that carry little meaning — such as "the", **stem** "running" → "run", add synonyms). The same analyzer runs on the query, so "Running SHOES" and "running shoe" meet on the same terms. A `text` field is analysed; a `keyword` field is stored whole for exact filters, sorting and aggregations.
2. **Matching.** A query for `run AND trail` intersects two sorted posting lists — fast even for millions of entries.
3. **Scoring.** Matches are ranked with **BM25** (the default since Elasticsearch 5): a term counts more if it is frequent in *this* document (term frequency, with diminishing returns), rare across *all* documents (inverse document frequency), and the document is short (length normalisation).

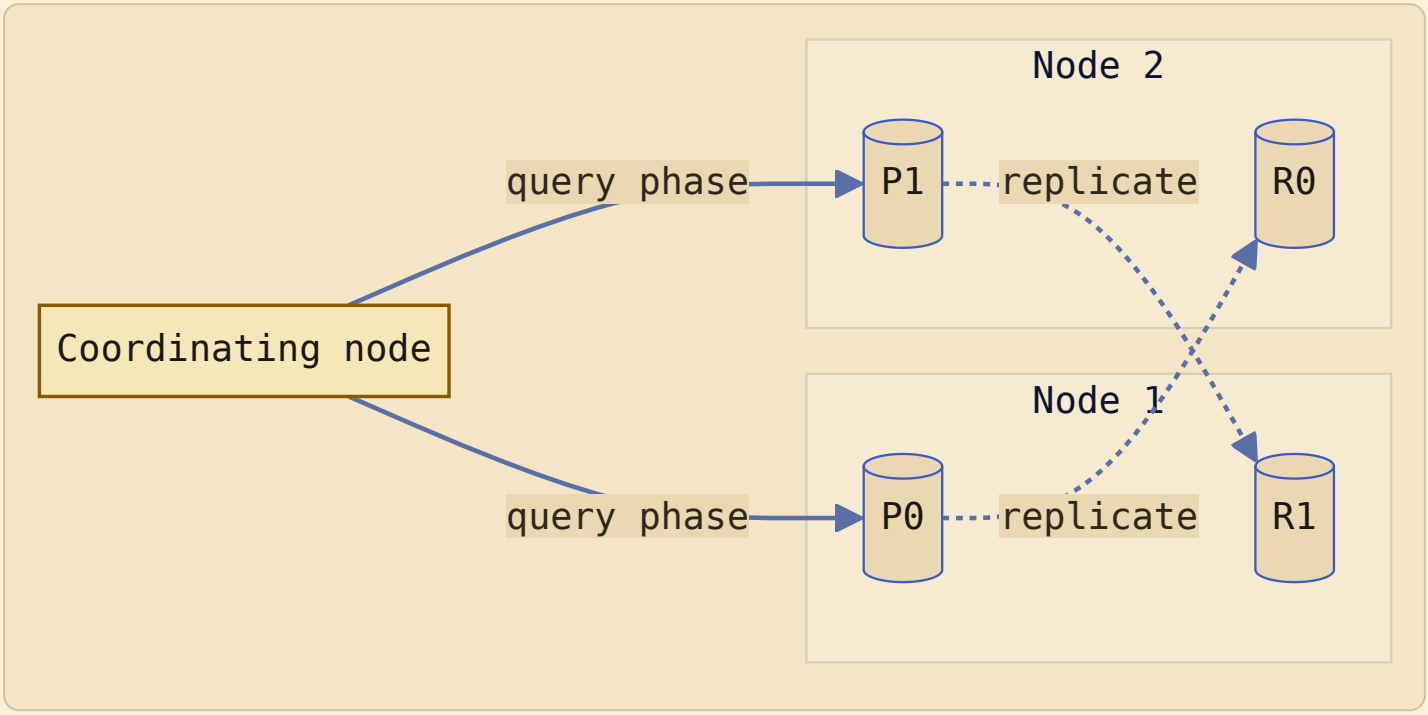
`code/inverted_index.py` builds all three in miniature, splits five products over two shards and merges the per-shard results:

```
analyze('Running shoes for the TRAIL') -> ['run', 'shoe', 'trail']
shard 0: docs [2, 4]
shard 1: docs [1, 3, 5]
postings['run'         ] -> {1: 2, 3: 1, 5: 1}
postings['trail'       ] -> {1: 1, 2: 1, 4: 1}
postings['shoe'        ] -> {1: 1, 3: 1}
postings['hik'         ] -> {2: 1, 5: 1}
postings['lightweight' ] -> {1: 1, 5: 1}
docs matching run AND trail: [1]
query terms: ['trail', 'run', 'shoe']
shard 0 top hits: [(2, 0.578), (4, 0.53)]
shard 1 top hits: [(1, 2.123), (3, 1.391), (5, 0.53)]
merged top 3: [(1, 2.123), (3, 1.391), (2, 0.578)]
  doc 1: Running shoes for trail running, lightweight
  doc 3: Road running shoes with extra cushioning
  doc 2: Waterproof hiking boots for the trail
```

The toy stemmer turns "hiking" into `hik`; real stemmers (Porter, Snowball) and dictionary-based lemmatisers do better, but the idea is the same. Notice doc 4 ("Trail mix snack") matches `trail` yet loses to documents that match more, rarer terms.

How it works — shards, replicas and near-real-time search

- An **index** (like a table) is split into **primary shards**, each a complete Lucene index. A document goes to `shard = hash(_routing or _id) % number_of_primary_shards` (`%` is the remainder after division), so the primary shard count is fixed at creation (changing it means `_split`, `_shrink` or reindexing).
- Each primary has **replica shards** on other nodes for failover and extra read capacity. Writes go to the primary, which forwards them to the in-sync replicas.
- A search is **scatter-gather**: the coordinating node sends the query to one copy of every shard (*query phase*: each returns its top-N ids and scores), merges them, then fetches the winning documents (*fetch phase*). Deep **pagination** (fetching results page by page, here starting at hit 10,000) is expensive because every shard must return 10,000+ hits; use `search_after` instead.
- Lucene writes new documents into small, **immutable segments**. A **refresh** (every 1 second by default) opens the latest segment for search — hence **near-real-time**: a document is searchable about a second after indexing, not instantly. A **translog** (write-ahead log) makes acknowledged writes durable before the next Lucene commit (**flush**), and background **merges** combine small segments and purge deleted documents.

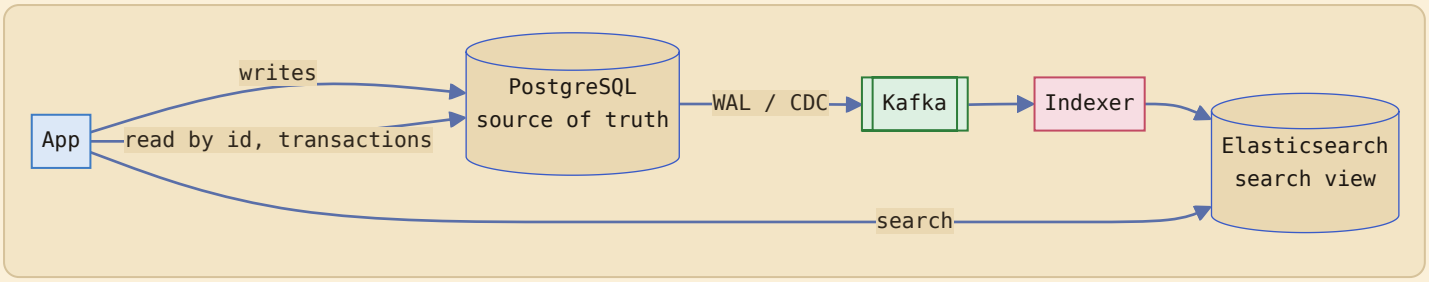


The ELK stack

ELK = **Elasticsearch** (store and search) + **Logstash** (ingest pipelines: parse, enrich, route) + **Kibana** (dashboards and search UI). Lightweight shippers (**Beats**, now largely **Elastic Agent**) were added, and Elastic calls the whole set the **Elastic Stack**. The typical log pipeline: applications write JSON logs → an agent ships them (often via Kafka as a buffer) → ingest parsing → time-based indices (`logs-2026.09.28`) with **index lifecycle management** moving old indices to cheaper nodes and deleting them after the retention period. OpenSearch offers the same shape (OpenSearch Dashboards instead of Kibana). Loki and ClickHouse are popular cheaper alternatives for logs.

When to pair it with a database — and when not

Keep the **relational database as the source of truth** and treat the search index as a derived, rebuildable view. Feed it by **change data capture** (e.g. Debezium reading the Postgres WAL — its write-ahead log of changes — → Kafka → an indexer) or an **outbox table** (the app writes the event into a table in the same transaction as the data, and a separate process forwards it), not by **dual writes** (the app writing to both systems itself) from the application, which drift apart when one write fails.



Do **not** use Elasticsearch as the primary store for money or orders: no multi-document transactions, no joins, eventual visibility (refresh), and mapping changes need reindexing. Stay with Postgres full-text search while the data fits one database and relevance needs are simple.

Redis internals: what it is

§16.2 introduced **Redis** as an in-memory key-value store with rich data structures. (Redis moved to **source-available** licences (code you can read but with restrictions on use) in 2024 and added the AGPLv3 open-source option in 2025; the Linux Foundation's **Valkey** fork keeps the BSD licence. The internals below apply to both.)

How it works — one thread, an event loop

Redis executes all commands on **one main thread** (a thread is one independent sequence of execution inside a program). It uses **I/O multiplexing** (`epoll` on Linux, `kqueue` on macOS/BSD — operating-system calls that report which of many connections have data ready) to watch thousands of client **sockets** (open network connections) at once and runs each ready command to completion, one at a time.

- **Why it is fast anyway:** data is in RAM, commands are O(1) or O(log n) on compact structures, and there are no **locks** (mechanisms that stop two threads touching the same data at once) or **context switches** (the CPU pausing one thread to run another). A single instance handles on the order of 100,000+ simple operations per second.
- **Why every command is atomic:** nothing else runs in the middle of `INCR` or a Lua script / `MULTI`–`EXEC` block.
- **The catch:** one slow command blocks *everyone*. `KEYS *`, `HGETALL` on a hash with millions of fields, `DEL` of a huge key or a long Lua script stall all clients. Use `SCAN` (iterates keys in small batches), `UNLINK` (frees memory in a background thread) and keep keys small. Since Redis 6, optional **I/O threads** parallelise reading and writing sockets, but command execution stays single-threaded.
- **Expiry** is lazy (checked when a key is touched) plus an active background sampler that deletes a portion of expired keys each cycle.

How it works — data structures and encodings

Each type switches internal encoding by size to save memory: small hashes, lists and sorted sets use a compact **listpack** (one contiguous block), then convert to a **hash table**, **quicklist** (linked list — items

chained by pointers — of listpacks) or **skip list + hash table** as they grow. The sorted set's skip list gives O(log n) insert, rank and range — why it is the classic leaderboard structure (§16.2). Sets of small integers use an **intset**. `OBJECT ENCODING key` shows which one a key uses.

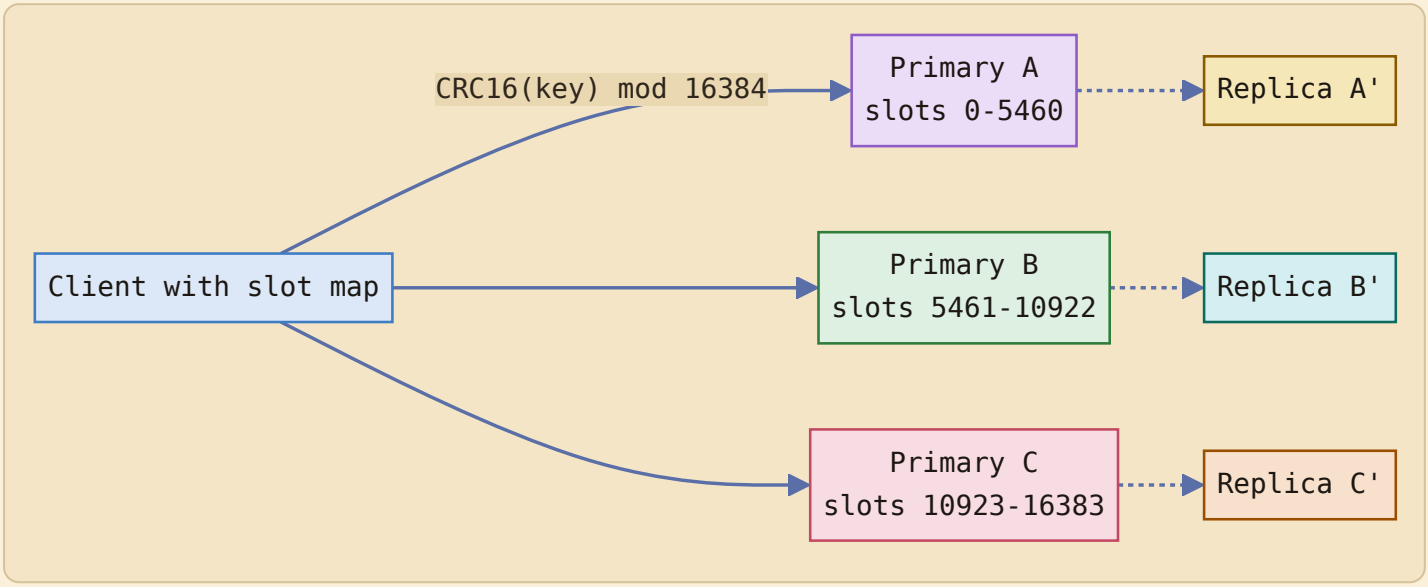
How it works — persistence: RDB and AOF

	RDB snapshot	AOF (append-only file)
What	point-in-time binary dump of the whole dataset	log of every write command
How	`fork()` (the OS call that clones a running process); the child writes the snapshot while the parent keeps serving; the OS copies memory pages only when they change (**copy-on-write**)	appended on each write; `fsync` policy `always`, `everysec` (default) or `no`; rewritten in the background to stay compact
Data loss on crash	everything since the last snapshot (minutes)	about 1 second with `everysec`
Restart speed	fast load	slower replay (hybrid AOF with an RDB preamble helps)

Most deployments enable both, or run Redis as a pure cache with persistence off. Watch memory during `fork()`: a write-heavy instance can briefly need much more RAM as pages are copied.

How it works — replication, Sentinel and Cluster

- **Replication** is asynchronous primary → replicas. A replica does a full sync (RDB transfer) once, then streams commands; after a short disconnect it resumes from the primary's **replication backlog** (`PSYNC`). Asynchronous means an acknowledged write can be lost if the primary dies before replicating it (`WAIT` reduces, not removes, that window).
- **Sentinel** monitors primaries and promotes a replica on failure (high availability without sharding).
- **Redis Cluster** shards data over **16,384 hash slots**: `slot = CRC16(key) mod 16384` (CRC16 is a fast 16-bit checksum function used here as a hash), and each primary owns a range of slots. Clients cache the slot map; a request to the wrong node gets a `MOVED` redirect (or `ASK` during slot migration). Multi-key commands work only when all keys are in one slot — force that with a **hash tag**: `{user:42}:cart` and `{user:42}:profile` hash only the part in braces. Nodes **gossip** (each periodically swaps status with a few random peers, so news spreads through the cluster) to detect failures and promote replicas.



How it works — eviction when memory is full

With `maxmemory` set, `maxmemory-policy` decides what to drop: `noeviction` (the default: writes fail with an error), `allkeys-lru` / `allkeys-lfu` (the usual cache choice), `volatile-lru` / `volatile-lfu` / `volatile-ttl` (only keys with a TTL), or `allkeys-random` / `volatile-random`. LRU and LFU are **approximated** by sampling a

few keys (`maxmemory-samples`, default 5) and evicting the best candidate, which avoids the memory cost of a full LRU list. Memcached, by contrast, is multi-threaded, strings-only, has no persistence or replication, and uses a **slab allocator** (memory pre-split into fixed-size chunks grouped by size class) with per-slab LRU — simpler and good for a pure cache, but without Redis's data structures.

Trade-offs

	Elasticsearch / OpenSearch	Redis
Best at	relevance search, filters + facets, log analytics	sub-millisecond reads/writes, counters, sessions, leaderboards, rate limits
Consistency	near-real-time (≈1 s refresh), no multi-doc transactions	atomic single commands/scripts; async replication can lose recent writes
Scaling	shards (fixed count per index) + replicas	Cluster hash slots + replicas; data bounded by RAM
Role	derived view fed from the source of truth	cache or fast secondary store; primary only if losing ~1 s of writes is acceptable

Common mistakes

- **Dual writes** to Postgres and Elasticsearch from the app → use CDC or an outbox so the index can always be rebuilt.
- **Too many small shards** (thousands of tiny indices) → each shard costs **heap** (the memory Elasticsearch's Java runtime keeps objects in) and **file handles** (OS slots for open files); aim for shards of roughly 10–50 GB.
- **Expecting read-your-writes from search** → a new document appears after the next refresh; read it back by id from the database, or use `refresh=wait_for` sparingly.
- **`KEYS *` or giant keys in production Redis** → blocks the single thread; use `SCAN`, split big keys, `UNLINK`.
- **No `maxmemory` / `noeviction` on a cache** → writes start failing when memory fills; pick an eviction policy.
- **Multi-key operations across Cluster slots** → `CROSSSLOT` error; design keys with hash tags.

In the interview

30-second answer: "Elasticsearch stores an inverted index per shard: analysers turn text into terms, posting lists map terms to documents, and BM25 ranks matches. An index is split into a fixed number of primary shards with replicas; queries scatter to every shard and the results are merged. New documents become searchable after a one-second refresh, so it is near-real-time. I keep Postgres as the source of truth and feed Elasticsearch via CDC. Redis runs commands on a single thread with an epoll event loop, so each command is atomic but a slow one blocks everyone; it persists with RDB snapshots and/or an AOF, replicates asynchronously, shards over 16,384 hash slots in Cluster mode, and evicts with approximated LRU or LFU when it hits `maxmemory`."

- *Why is Redis single-threaded yet fast?* → In-memory data, no locks, cheap commands, and I/O multiplexing; the bottleneck is usually the network, not the CPU.
- *RDB vs AOF?* → Snapshots are compact and fast to load but lose minutes; AOF with `everysec` loses about one second but replays slower. Many use both.
- *Why can't you change Elasticsearch's primary shard count in place?* → Routing is `hash(id) % shards`; changing the divisor moves most documents, so you split, shrink or reindex.

Self-check

1. What does an analyzer do at query time? → Applies the same tokenising and filtering as at index time so query and document terms match.
2. Why is Elasticsearch "near" real-time? → New segments become searchable only at the next refresh (1 s default).

3. Which Redis structure backs a leaderboard and why? → A sorted set (skip list + hash): O(log n) score updates and rank queries.
4. Two keys must be updated in one `MULTI` on Redis Cluster — how? → Give them the same hash tag so they land in the same slot.

TL;DR

Search engines = analysers + inverted index + BM25, split into fixed primary shards with replicas, near-real-time via 1 s refresh; ELK = Elasticsearch + Logstash + Kibana for logs; pair with a relational source of truth through CDC. Redis = single-threaded event loop (atomic, but beware slow commands), size-adaptive encodings, RDB + AOF persistence, async replication, 16,384-slot Cluster with hash tags, approximated LRU/LFU eviction.

Sources: Elasticsearch Guide (near real-time search, shards and replicas, text analysis, similarity/BM25, index lifecycle management), Apache Lucene documentation, Robertson & Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond" (2009), Redis documentation (persistence, replication, cluster specification, eviction, `OBJECT ENCODING`, I/O threads), Memcached wiki, Debezium documentation.

Chapter 16 TL;DR

Model	One-line summary	Examples	In Postgres
Key-value	Get/put by key, fastest possible	Redis, Riak	PK table, `UNLOGGED`, UPSERT
Document	Flexible nested JSON per record	MongoDB, CouchDB	`jsonb` + GIN
Wide-column	Partition key + clustering order + LSM writes	Cassandra, HBase	Composite PK + partitions
Graph	Nodes and edges, fast multi-hop traversal	Neo4j, Neptune	Link table + recursive CTE
In-memory	RAM-first, persisted by snapshot + log	SAP HANA, SingleStore, Redis	Buffer cache, unlogged tables
NewSQL	SQL + ACID + auto-sharding via Raft/Paxos	Spanner, CockroachDB, TiDB	Citus (scale-out)
Time-series	Time chunks, compression, retention	TimescaleDB, InfluxDB	Range partitions + BRIN + `date_bin`
Spatial	Geometry types + R-tree style indexes	PostGIS	`point` / `box` + GiST, PostGIS
Big data	Distributed batch and stream processing	Hadoop, Spark, Trino, Kafka, Flink	CDC out to them
Search engine	Analyzer + inverted index + BM25; shards + replicas; ~1 s refresh	Elasticsearch, OpenSearch (ELK for logs)	`tsvector` + GIN

Redis internals (16.11): one command thread + epoll event loop, RDB/AOF persistence, async replication, 16,384 Cluster slots, approximated LRU/LFU eviction.

BASE vs ACID is a spectrum. Choose by access pattern and consistency need, default to Postgres, and add specialised stores only when they clearly earn their operational cost.

Next: Chapter 17 — Schema design case studies.

Chapter 17 — Schema Design Case Studies

Chapters 10–16 gave you the tools one at a time: modeling, normalization, indexes, transactions, scaling, and the non-relational options. Interviews and real projects never ask for one tool at a time. They say "design the database for a hotel booking site" and expect you to go from a vague brief to tables, keys, the queries that matter, and a view on what breaks at scale. This chapter does exactly that thirteen times, always with the same template, so the method becomes a habit rather than a memory exercise.

What you'll learn

1. A repeatable eight-step template for any schema-design question, plus the mapping rules and judgement calls every case reuses.
2. Nine classic business systems — hospital, hotel, bank, photo-sharing (Instagram), e-commerce, food delivery, library, ticket booking, warehouse inventory.
3. Four large-scale product designs — Quora, TikTok, Airbnb, Snapchat — where the relational core meets caches (fast in-memory copies of hot data), **object storage** (services such as Amazon S3 that store whole files by key) and NoSQL (non-relational databases, Chapter 16).
4. Four of the designs built for real on PostgreSQL (e-commerce, ticket booking with seat-hold concurrency, library, Instagram feed) with seed data and real output.
5. The modeling patterns that keep coming back — junction tables, status enums, soft deletes, audit columns, polymorphic associations, denormalized counters (each is defined where it first appears below) — compared in one table at the end.

How to run the examples. All code is in `code/`. `00_schema.sql` creates four PostgreSQL schemas — `shop`, `tix`, `lib`, `insta` — with small invented data; files `01`–`04` each run on a fresh copy:

```
bash tools_run_sql.sh sql '17-*'                          # run this chapter's files
bash tools_pg.sh psql -d ch_17 -f sql/chapters/17-schema-design-case-studies/code/02_ticket_booking.sql
```

Every "Output:" block below is real psql output (command tags like `UPDATE 2` trimmed). This chapter **applies** earlier material rather than re-teaching it: modeling and ER notation are in Chapter 10, normal forms in Chapter 11, index types and `EXPLAIN` in Chapter 12, locking and isolation in Chapter 13, partitioning/sharding/replication in Chapter 15, and NoSQL families in Chapter 16.

17.0 The template

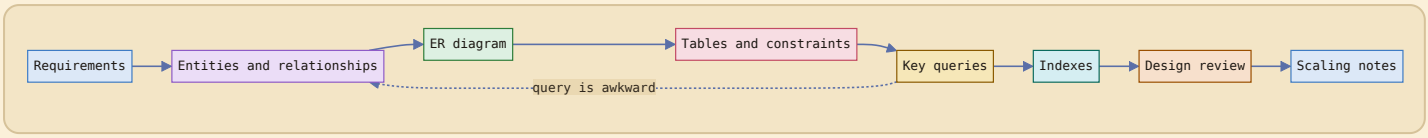
What it is

A **schema-design case study** is a short, structured answer to "what tables would you build for X?". The template below is the modeling process of Chapter 10 §10.3 compressed into interview-sized steps.

How it works

1. **Requirements.** List what users do (functional) and the numbers and guarantees that matter (non-functional: reads vs writes, consistency, **retention** — how long data must be kept). Then **write down the 5–6 questions the system must answer** ("which rooms are free between two dates?") — they drive the tables *and* the indexes. Ask; don't assume.
2. **Entities and relationships.** Nouns that you store several facts about, with several instances, become entities; verbs become relationships. For each relationship decide **cardinality** (how many rows on each side can be linked: 1:1 one-to-one, 1:N one-to-many, M:N many-to-many) **and participation** (total = every row must take part; partial = optional). Say each one aloud as a sentence.
3. **ER diagram.** Draw it — an **entity-relationship diagram** shows tables as boxes and relationships as lines (here with `erDiagram` **crow's-foot notation**, where a three-pronged "foot" on a line end means "many" and a bar means "one"). The diagram is where a reviewer spots a missing entity fastest.

4. **Key tables (DDL).** **DDL** (data definition language) is the `CREATE`/`ALTER` part of SQL. Map entities to tables: **primary keys** (the column(s) that uniquely identify a row), **foreign keys** (columns that must match a primary key in another table), `NOT NULL`, `UNIQUE`, `CHECK`. An M:N relationship becomes a **junction table** (a table whose primary key is the pair of foreign keys). Aim for **3NF** (third normal form: every non-key column depends on the key, the whole key and nothing but the key — Chapter 11), then **denormalize** (deliberately store a duplicate or derived value for speed) on purpose.
5. **Key queries.** Write the 3–5 queries the product runs most. They validate the schema: if a core question needs six joins and a `DISTINCT`, the model is probably wrong.
6. **Indexes.** One index per hot access path, usually a **composite index** (an index on several columns, in a chosen order) that matches the query's `WHERE` + `ORDER BY` (see Chapter 12).
7. **Design review.** Check the tables against 1NF → BCNF (the ladder of **normal forms**, rules that remove redundant storage of facts — Chapter 11), name every deliberate redundancy, and list what the design *cannot* do yet (no history, an unenforced rule). Volunteering the limits reads as judgement, not as a mistake.
8. **Scaling notes.** Which table grows fastest, what to **partition** (split into pieces inside one database) or **shard** (split across several database servers) it by, what to cache, what to move to another store, and where strict consistency is required.



Rules every case reuses

These mapping rules (Chapter 10 §10.15) and judgement calls come up in all thirteen designs, so the cases below only point at them.

Situation	Rule
Total participation (every order has a customer)	The foreign key is `NOT NULL`; partial participation stays nullable
1:1 relationship	Foreign key **plus** `UNIQUE` (or a shared primary key) — without it the relationship is silently 1:N
M:N relationship	Junction table; an attribute of the *pairing* (rate agreed, price of a seat at a show, dosage per medicine) lives there
Composite attribute (name = first + last)	Split into columns
Multivalued attribute (several phone numbers)	Its own table keyed by (owner, value)
Money	`numeric` / `DECIMAL`, never `float` (0.10 has no exact binary form) or `int` (no cents)
Times that order events	`timestamptz`, not `date` — two postings on one day still need an order
Codes with letters or leading zeros (policy, passport, room "204B")	`text` / `varchar`, never `int`
Table named `user`, `order`, `group`, `like`, `transaction`, `show`	Reserved (or keyword) in some dialects — rename (`app_user`, `customer_order`, `screening`) rather than quote forever
Passwords	Store a slow salted hash (a one-way scramble of the password plus a random per-user value, `password_hash`), never the password
One entity doing two jobs	Split **type** from **instance**: `titles` / `copies`, `room_types` / `rooms`, `seats` / `show_seats`
Only current state stored	Keep history rows (loans, bookings, status events, ledger) — "when did…?" needs them
A value that must not change later	Snapshot it (price paid, address shipped to, fine charged)
Designing for Instagram scale on day one	State the single-node design first, then say what changes at 100×

In the interview

30-second answer: "I'll clarify the main use cases and scale, pick the entities and relationships, sketch the ER diagram, turn it into tables with keys and constraints, check it against the top queries, add indexes for those queries, and finish with what I'd partition, cache or move to another store as it grows."

- *How do you know the design is right?* — Every core query is simple and index-friendly, every business rule that must never be broken is a **constraint** (a rule the database itself enforces) or runs inside one **transaction** (a group of statements that succeed or fail together), and no fact is stored twice without a reason.
- *When do you denormalize?* — When a measured hot read needs it (a feed, a counter), and only with a plan to keep the copy correct (same transaction, **trigger** — code the database runs automatically on insert/update/delete — or a periodic **reconciliation** job that recomputes the value and flags differences).

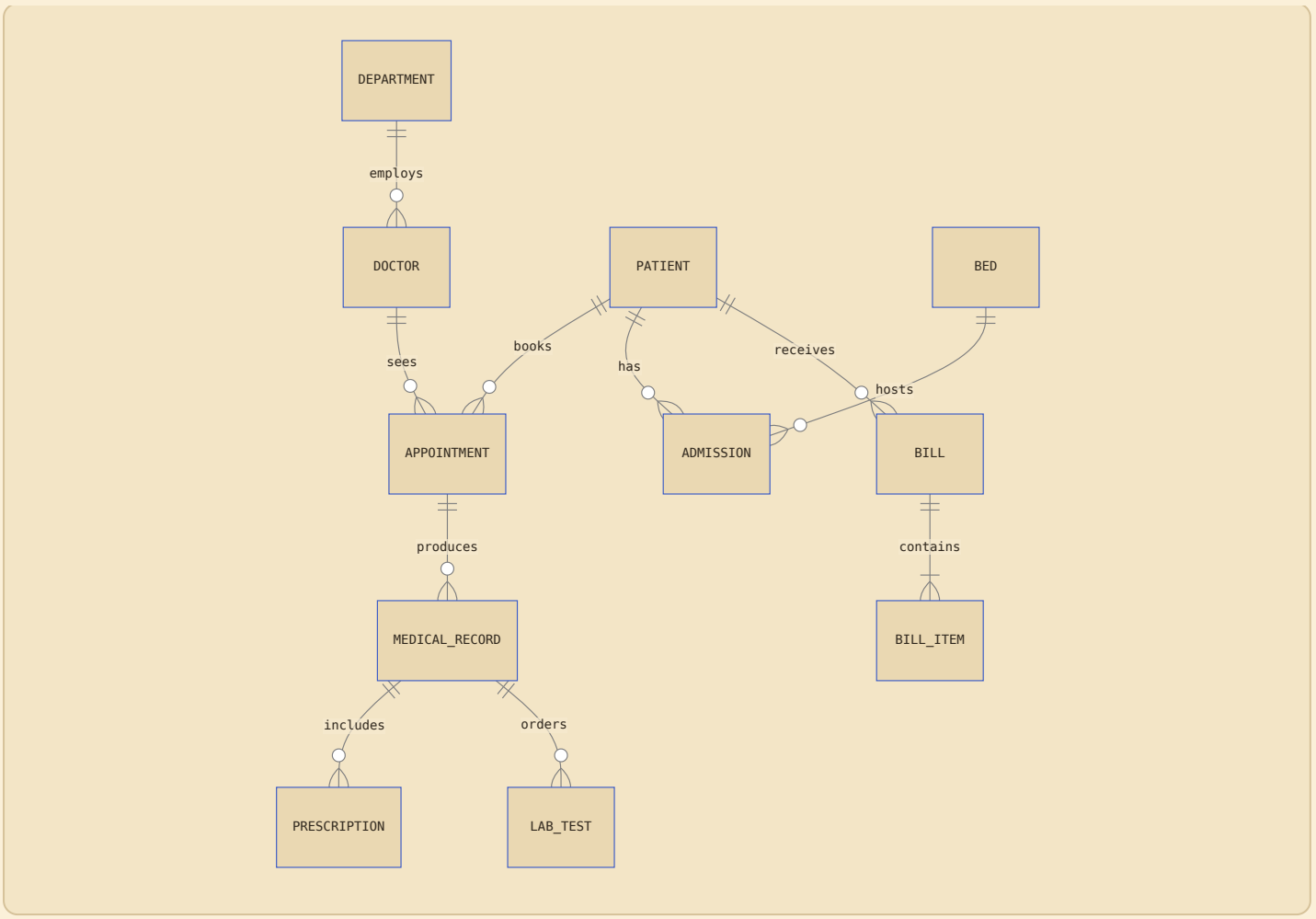
17.1 Hospital management system

Requirements

- Register **patients** (name, birth date, contact, one or more phone numbers, insurance policies) and **doctors** (specialty) grouped into **departments**.
- Schedule **appointments** (each pairs one patient with one doctor at a time slot); a doctor is never in two overlapping slots.
- Record **admissions** to a **room/bed** (in-patients), with admit and discharge dates.
- Keep **medical records** (diagnosis, treatment, written by a doctor), **prescriptions** listing one or more **medicines** from a catalogue, and **lab tests** with results.
- Produce **bills** per visit/admission with line items and payments.
- Non-functional: records are legally retained for years and must be **auditable** (every change traceable to who and when); access is strictly **role-based** (permissions depend on the user's role, e.g. doctor or clerk).
- Questions to answer: a doctor's appointments tomorrow, a patient's full history, free rooms now, what a patient still owes, the medicines on a prescription.

Entities and relationships

Relationship	Cardinality	Note
Department — Doctor	1:N	a doctor belongs to one department
Patient — Appointment — Doctor	M:N via `appointments`	the appointment is the junction row with its own attributes (time, status)
Patient — Admission — Bed	1:N and N:1	a bed has many admissions over time, one at a time
Appointment / Admission — Medical record	1:N	a record belongs to one encounter
Doctor — Medical record	1:N	the author; total participation
Record — Prescription, Record — Lab test	1:N	
Prescription — Medicine	M:N	junction `prescription_items`; dose and frequency per medicine live there
Patient — Insurance policy	1:N	partial: a patient may have none; `(provider, policy_no)` is `UNIQUE`
Patient — Phone	1:N	multivalued attribute → `patient_phones (patient_id, phone)`
Patient — Bill — Bill items	1:N, 1:N	



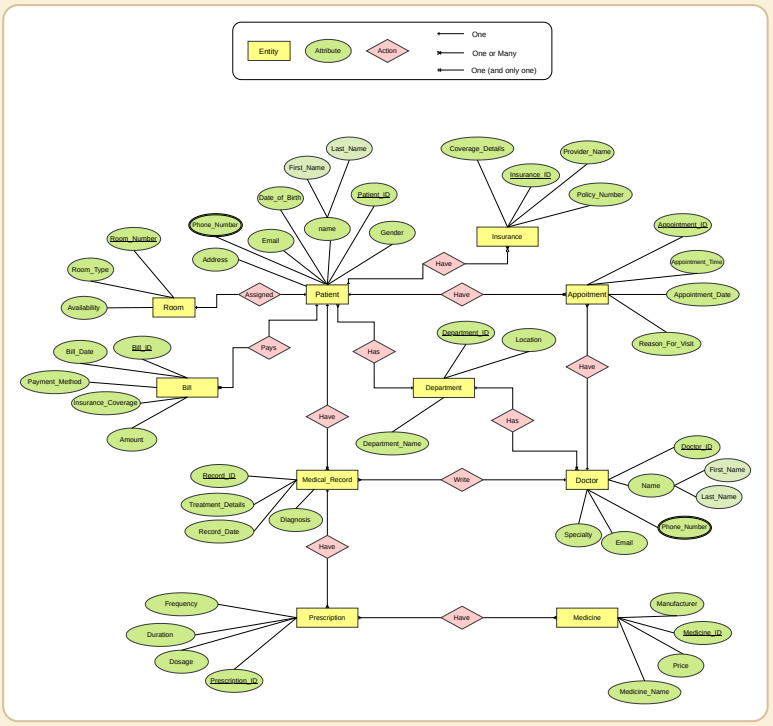


Figure — the course's hospital ER diagram: ten entities with cardinality and participation marked (course diagram)

Key tables

```
CREATE TABLE appointments (
  appointment_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  patient_id bigint NOT NULL REFERENCES patients,
  doctor_id  bigint NOT NULL REFERENCES doctors,
  slot       tstzrange NOT NULL,                    -- [start, end)
  status     text NOT NULL CHECK (status IN ('booked','checked_in','completed','cancelled','no_show')),
  -- no two active appointments of the same doctor may overlap (needs: CREATE EXTENSION btree_gist)
  EXCLUDE USING gist (doctor_id WITH =, slot WITH &&) WHERE (status <> 'cancelled')
);
CREATE TABLE admissions (
  admission_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  patient_id bigint NOT NULL REFERENCES patients,
  bed_id     int    NOT NULL REFERENCES beds,
  admitted_at   timestamptz NOT NULL,
  discharged_at timestamptz CHECK (discharged_at > admitted_at)
);
CREATE UNIQUE INDEX one_patient_per_bed ON admissions (bed_id) WHERE discharged_at IS NULL;
```

(`tstzrange` is a time-range type — a start and end timestamp in one value; `&&` means "overlaps".) The **exclusion constraint** (`EXCLUDE USING gist … WITH &&`) is PostgreSQL's way of saying "no two rows may have overlapping ranges for the same doctor" — a rule a plain `UNIQUE` cannot express. MySQL and SQL Server have no equivalent; there you **lock** the doctor's row (`SELECT … FOR UPDATE` — read rows and lock them so other transactions must wait before changing them until you finish) and check for overlaps inside the booking transaction (Chapter 13).

Key queries

1. A doctor's schedule for a day: `WHERE doctor_id = $1 AND slot && tstzrange($day, $day + interval '1 day')`.
2. Free beds per ward: beds with no open admission (`NOT EXISTS … discharged_at IS NULL`).
3. A patient's full history: records, prescriptions and lab results across all encounters, newest first.
4. Unpaid bills older than 30 days.

Indexes

The exclusion constraint's **GiST** index (a Postgres index type that handles ranges and geometry, Chapter 12) serves schedule lookups; `admissions (patient_id)`, `medical_records (patient_id, created_at DESC)`, and the partial unique index above for bed occupancy.

Design review

- **Normal forms.** The phone and junction tables have no non-key columns, so 2NF holds; `department_name` stays in `departments` (copying it onto patients would be a **transitive dependency** — a column depending on another non-key column rather than on the key — breaking 3NF); the extra **candidate keys** (other columns that could also uniquely identify a row) (`email`, `(provider, policy_no)`) keep BCNF.
- **Room occupancy.** The simplest model is `rooms.patient_id UNIQUE` (a 1:1 "occupies"), but it only knows who is in the room *now*. The `admissions` table with a partial unique index keeps the same rule *and* the history.
- **Dose per medicine.** If dose sat on `prescriptions`, every medicine on one prescription would share it; it belongs to the pairing.
- **Still missing:** who cancelled an appointment and when (status history), and a patient's department over time.

Scaling notes

A hospital's volume is modest — one well-tuned PostgreSQL **primary** (the one server that accepts writes) with a **replica** (a read-only copy kept in sync) for reports is plenty. The hard requirements are **integrity and audit**: append-only record tables (corrections are new rows that reference the old one), `created_by` / `created_at` on every clinical row, **row-level security** (database rules that filter which rows

each user may see) for role-based access (Chapter 14), and encrypted backups with **point-in-time recovery** (restoring the database as it was at any chosen moment) (Chapter 15 §15.9). Large files (scans, X-rays) go to object storage; the table keeps only the key.

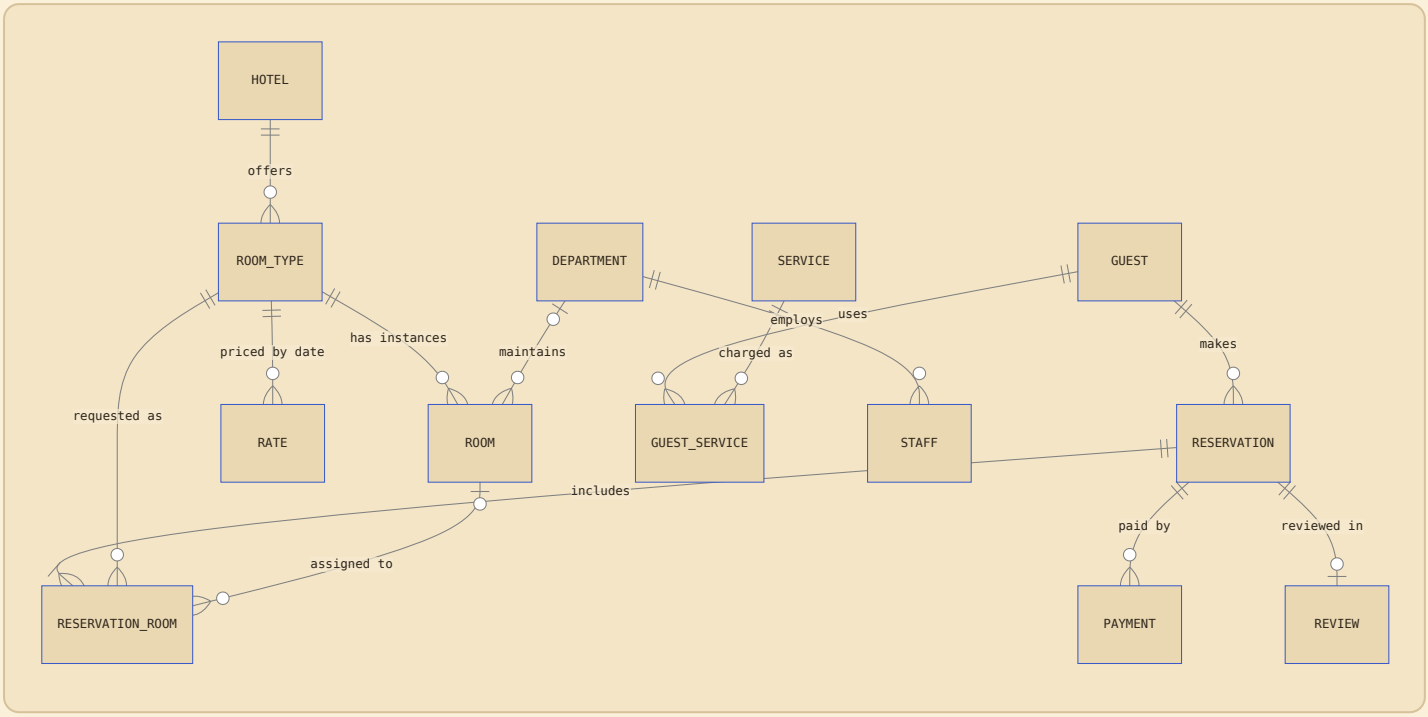
17.2 Hotel reservation system

Requirements

- **Hotels** have **room types** (single, double, suite) with a base price, and physical **rooms** of each type.
- **Guests** search availability by city and dates, then create a **reservation** for one or more rooms; no room may be double-booked.
- Reservation life cycle: pending → confirmed → checked-in → checked-out, or cancelled. **Payments** and **reviews** attach to reservations.
- Prices vary by date (seasonal rates).
- Guests are identified by a document (passport, national ID) and may have several phone numbers; they also use **extra services** (dining, spa, laundry) charged to the stay.
- **Staff** belong to **departments** (housekeeping, front desk); a department looks after rooms.
- Questions: free rooms between two dates, what a guest paid and owes, today's check-ins, services a guest used, revenue per room type last month, who works in housekeeping.

Entities and relationships

The central decision is **type versus instance**: guests book a *room type* for a date range; a specific *room* is assigned at or before check-in. Modeling only rooms makes search slow; modeling only types makes check-in impossible.



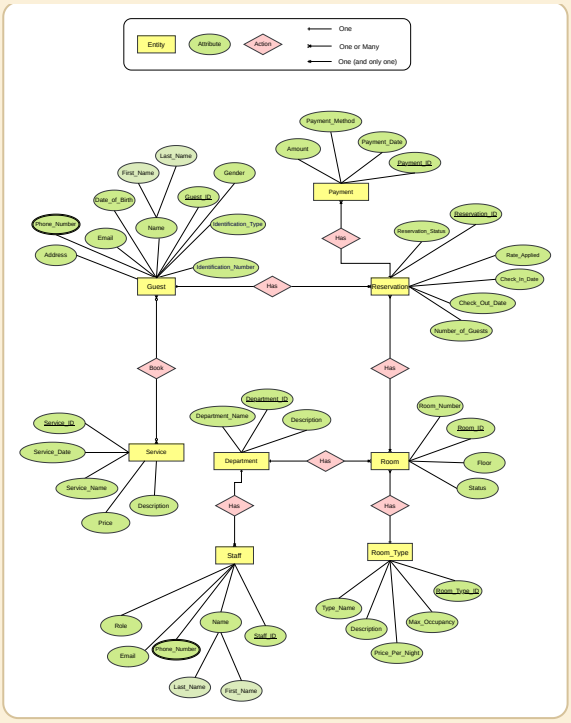


Figure — the course's hotel ER diagram (eight entities); reservation-to-room and guest-to-service are M:N (course diagram)

Two details from the full model: the price and maximum occupancy belong to the **room type**, not to each room (ten identical doubles share one price); and the usage of a service is a junction row `guest_services (guest_services_id, guest_id, service_id, reservation_id, used_on, quantity, price_charged)` with **its own surrogate key**, because the same guest orders dinner on several nights — a `(guest_id, service_id)` key would allow only one use ever. `guests` carries `UNIQUE (id_type, id_number)` so the same identity document can never appear on two guest rows.

Key tables

```
-- reservations (guest_id NOT NULL, hotel_id, check_in, check_out CHECK (check_out > check_in), status CHECK
(...))
CREATE TABLE reservation_rooms (
  reservation_id bigint REFERENCES reservations ON DELETE CASCADE,
  line_no        smallint,
  room_type_id   int NOT NULL REFERENCES room_types,
  room_id        int REFERENCES rooms,            -- NULL until assigned
  stay           daterange NOT NULL,              -- copied from the reservation for the constraint
  nightly_price  numeric(10,2) NOT NULL,          -- snapshot
  PRIMARY KEY (reservation_id, line_no),
  EXCLUDE USING gist (room_id WITH =, stay WITH &&)   -- a room is never in two overlapping stays
);
-- Inventory per type per night: the row that search reads and booking decrements.
CREATE TABLE room_inventory (
  room_type_id int REFERENCES room_types,
  night        date,
  total        smallint NOT NULL,
  reserved     smallint NOT NULL DEFAULT 0 CHECK (reserved <= total),
  PRIMARY KEY (room_type_id, night)
);
```

The exclusion constraint needs `CREATE EXTENSION btree_gist` (for the `=` on an integer inside a GiST index); rows with `room_id` still `NULL` never conflict, and cancelling a reservation must clear or delete its lines so the room is released.

`room_inventory` is a deliberate **denormalization**: counting overlapping reservations for every search is too slow, so each night keeps a counter. Booking increments `reserved` for every night of the stay in one transaction; the `CHECK` makes overbooking fail. (Hotels that *allow* a small overbooking set the check to `reserved <= total * 1.1`.)

Key queries

```sql
-- Room types in a hotel with at least one room free on every night of 10–13 June
SELECT rt.room_type_id, rt.name
FROM room_types rt JOIN room_inventory ri USING (room_type_id)
WHERE rt.hotel_id = $1 AND ri.night >= '2026-06-10' AND ri.night < '2026-06-13'
GROUP BY rt.room_type_id
HAVING count(*) = 3 AND min(ri.total - ri.reserved) > 0;
```

Other hot queries: today's arrivals and departures per hotel, a guest's upcoming stays, average rating per hotel.

Indexes

`reservations (hotel_id, check_in)` for arrival lists, `reservations (guest_id, check_in DESC)`, the inventory primary key for search, and the GiST index behind the exclusion constraint.

Design review

- **Normal forms.** Price lives on `room_types`, not `rooms` (3NF); `nightly_price` on the booking line is a *snapshot*, a different fact from today's list price.
- **Double booking.** `UNIQUE` only detects identical values, never two date ranges that overlap — hence the exclusion constraint (PostgreSQL) or a check inside the booking transaction elsewhere.
- **A room status column** (free/occupied) repeats what reservations already say and can disagree with them; keep it only for things bookings cannot express, such as "under maintenance".
- **Still missing:** mid-stay room changes (the line would need its own date range, which `stay` provides here), and who cancelled a booking and when.

Scaling notes

Search traffic is 100–1,000× booking traffic. Serve search from replicas or a search index (Elasticsearch/OpenSearch, search engines covered in Chapter 16) and a cache of inventory, but make the **booking** transaction hit the primary and re-check the inventory row under lock — search results may be a few seconds stale, bookings may not. Partition `reservations` by `check_in` month; shard by `hotel_id` if one cluster is not enough, since nearly every transaction touches a single hotel.

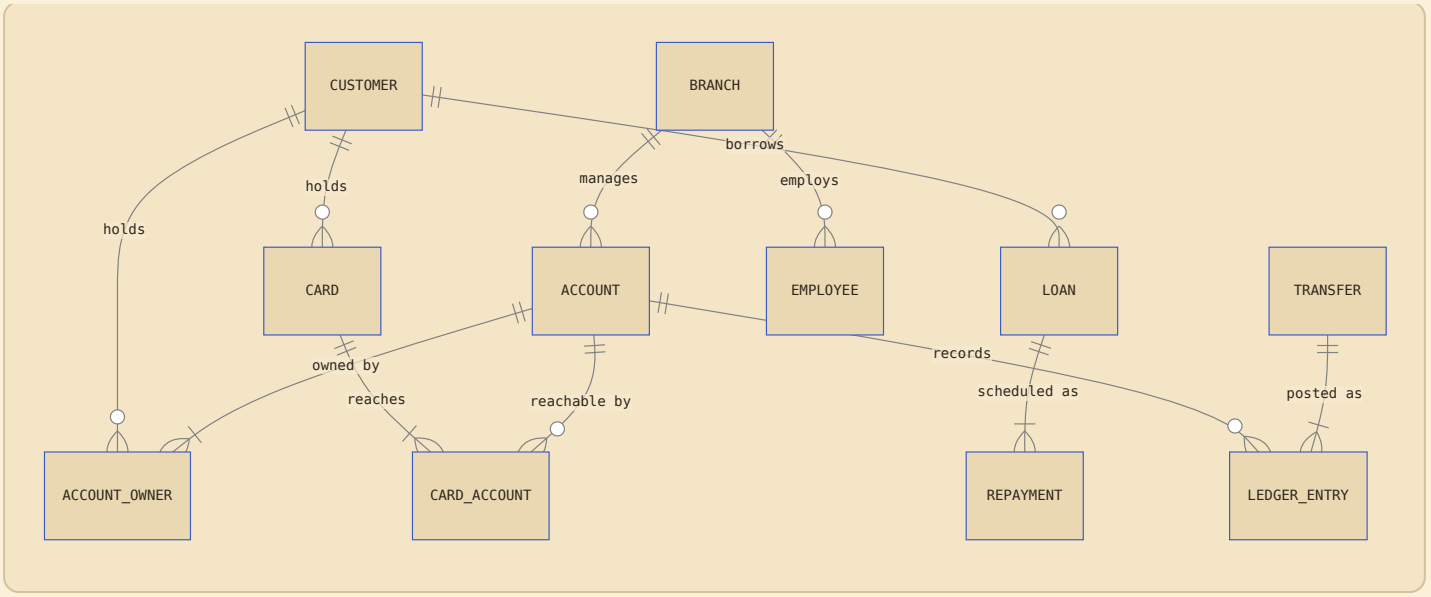
17.3 Banking system

Requirements

- **Customers** own one or more **accounts** (checking, savings, loan); an account may have joint owners.
- Deposits, withdrawals and **transfers** between accounts; balances may never go below an allowed limit.
- A full, immutable history of every money movement; statements per month.
- **Branches** (each publishing several phone numbers) with their **employees**; **cards**, where one card can reach several accounts; **loans** with repayments.
- Questions: the balance now, last month's activity, a customer's accounts, what is still owed on a loan, which accounts a card reaches, who works at a branch.
- Non-functional: strict consistency (**ACID** — atomic, consistent, isolated, durable transactions, Chapter 13), auditability, regulatory retention.

Entities and relationships

The key modeling idea is **double-entry bookkeeping**: money is never "updated", it is *moved*. A **transaction** (the business event) has two or more **entries** (ledger lines), each a signed amount on one account, and the entries of one transaction sum to zero.



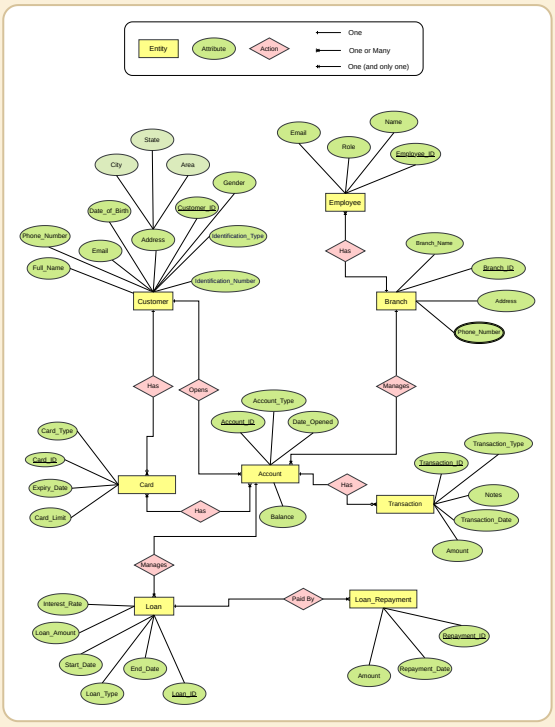


Figure — the course's bank ER diagram (eight entities); the card-to-account link is M:N (course diagram)

Key tables

```
CREATE TABLE accounts (
  account_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  iban        text NOT NULL UNIQUE,
  kind        text NOT NULL CHECK (kind IN ('checking','savings','loan')),
  currency    char(3) NOT NULL,
  balance     numeric(18,2) NOT NULL DEFAULT 0,   -- cached; the ledger is the truth
  overdraft_limit numeric(18,2) NOT NULL DEFAULT 0,
  CHECK (balance >= -overdraft_limit),
  status      text NOT NULL DEFAULT 'open' CHECK (status IN ('open','frozen','closed'))
);
CREATE TABLE transfers (
  transfer_id     bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  idempotency_key uuid NOT NULL UNIQUE,           -- a retried request cannot post twice
  kind            text NOT NULL,
  created_at      timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE ledger_entries (
  entry_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  transfer_id bigint NOT NULL REFERENCES transfers,
  account_id  bigint NOT NULL REFERENCES accounts,
  amount      numeric(18,2) NOT NULL CHECK (amount <> 0),   -- + credit, - debit
  balance_after numeric(18,2) NOT NULL
);
```

Money is `numeric`, never `float` (binary floating point cannot represent 0.10 exactly). `idempotency_key` makes the transfer **idempotent** — repeating the same request (e.g. after a network timeout) has the same effect as sending it once; a `uuid` is a 128-bit random id the client generates. The ledger is **append-only** (rows are only ever added, never changed or deleted): revoke `UPDATE` and `DELETE` on it; a mistake is fixed with a reversing transfer.

Key queries

A transfer is the textbook transaction (Chapter 13 §13.1):

```
BEGIN;
INSERT INTO transfers (idempotency_key, kind) VALUES ($key, 'transfer') RETURNING transfer_id;
-- lock both rows in a fixed order (lower id first) so two opposite transfers cannot deadlock
SELECT account_id FROM accounts WHERE account_id IN ($from, $to) ORDER BY account_id FOR UPDATE;
UPDATE accounts SET balance = balance - $amt WHERE account_id = $from;   -- CHECK stops overdraft
UPDATE accounts SET balance = balance + $amt WHERE account_id = $to;
INSERT INTO ledger_entries (transfer_id, account_id, amount, balance_after) VALUES (...), (...);
COMMIT;
```

Others: a monthly statement (`WHERE account_id = $1 AND created_at` in the month, ordered), a reconciliation check that every transfer's entries sum to zero (`GROUP BY transfer_id HAVING sum(amount) <> 0` must return nothing), and "balance = sum of entries" per account.

Indexes

(A **deadlock**, mentioned in the code comment above, is two transactions each waiting for a lock the other holds, so neither can proceed; the database kills one.) `ledger_entries (account_id, entry_id DESC)` for statements, `account_owners (customer_id)`, and the unique `idempotency_key`. The junction `card_accounts` has PK `(card_id, account_id)`, which already answers "which accounts can this card reach"; the reverse question needs an extra index on `account_id`, because an index is usable only from its leading column.

Design review

- **Transfers need a pairing key.** A simpler model with one `transactions` row per account (type = deposit/withdrawal/transfer) cannot name both sides of a transfer; the `transfer_id` shared by the debit and credit entries fixes that. Name the table `ledger_entries` or `account_transactions` — `TRANSACTION` is reserved in SQL Server and a keyword elsewhere.
- **Cached balance.** `accounts.balance` is a stored total of the entries. Normal forms don't forbid it (it isn't a dependency *inside* one row), but it can drift, so a nightly job recomputes and compares.

- **Rules a foreign key cannot express:** that a card only reaches accounts its owner holds; that each transfer's entries sum to zero. Check them in the transaction or with a periodic query.
- **Still missing:** which employee opened an account or approved a loan (audit columns), card number and status (blocked ≠ expired), and the width of `numeric` — pick it for the largest expected value, because widening later rewrites the table. Customer `city` and `state` together are a small 3NF compromise most teams accept knowingly.

Scaling notes

Correctness beats **throughput** (work done per second): **synchronous replication** (a commit waits until a copy on another server has it) to a **standby** (a replica ready to take over) so a **failover** (switching to the standby when the primary dies) loses no committed money (Chapter 15 §15.5), `SERIALIZABLE` (the strictest isolation level: transactions behave as if run one at a time) or explicit row locks for balance checks, and range partitioning of the ledger by month for archiving. When one node is not enough, shard by account (or customer); a transfer between shards then needs a **saga** (a chain of local steps, each with a compensating "undo" step) or **two-phase commit** (a coordinator asks every server to prepare, then tells all to commit — Chapter 15 §15.9) — which is why many banks keep the core ledger on a single, very large, **vertically scaled** (made bigger rather than multiplied) database or a distributed SQL system (Spanner, CockroachDB).

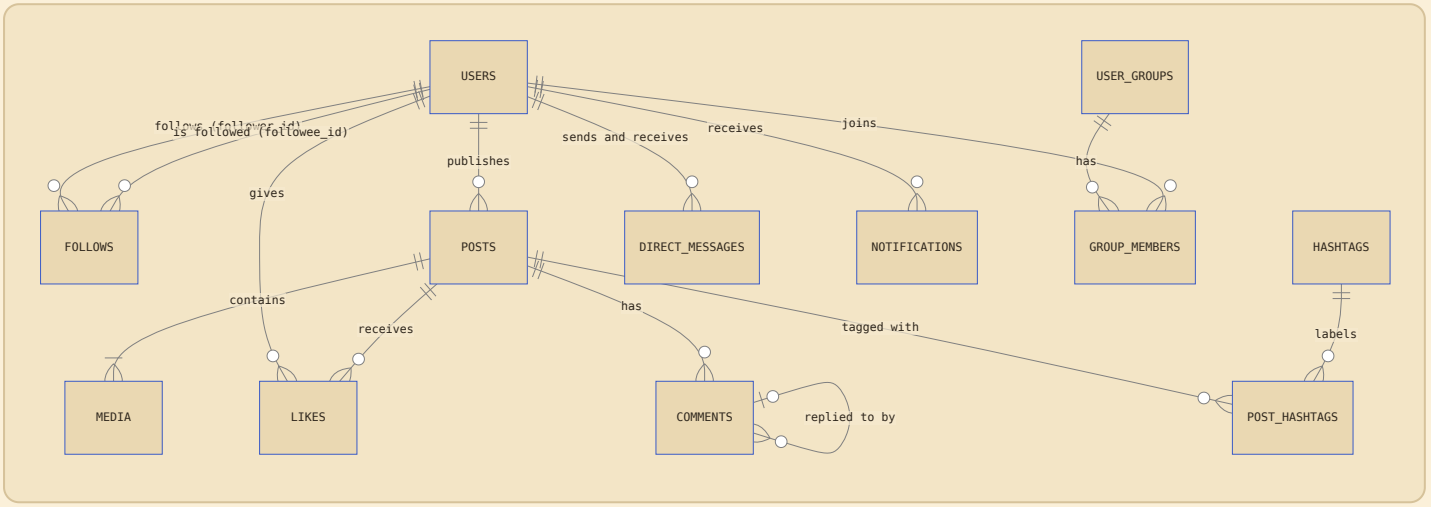
17.4 Instagram-style photo sharing (runnable)

Requirements

- **Users** have a profile, can be public or private, and **follow** other users (one-directional; "friends" = mutual follows).
- Users publish **posts** with a caption and one or more **media** items (photos/videos, a carousel keeps their order), tagged with **hashtags**.
- Other users **like** posts (at most once each) and **comment**, including replies to comments.
- Users send **direct messages** to one other user, receive **notifications** (likes, comments, follows), and create or join **groups**.
- The **home feed** shows recent posts from followed accounts, newest first, paged; profiles show like and follower counts instantly.
- Non-functional: extremely read-heavy (feeds and profiles), hundreds of millions of posts, media bytes far larger than the rows.

Entities and relationships

`follows` is a **self-referencing M:N** relationship on `users` (both columns point to `users`). `likes` and `post_hashtags` are junction tables. `comments` references itself through `parent_id` to form reply threads.



Key tables

From `00_schema.sql` (schema `insta`):

```
CREATE TABLE follows (
  follower_id bigint REFERENCES users,
  followee_id bigint REFERENCES users,
  created_at  timestamptz NOT NULL,
  PRIMARY KEY (follower_id, followee_id),       -- "whom do I follow?" + no duplicate follows
  CHECK (follower_id <> followee_id)
);
CREATE INDEX ON follows (followee_id);          -- "who follows me?"
CREATE TABLE posts (
  post_id bigint PRIMARY KEY, user_id bigint NOT NULL REFERENCES users,
  caption text, created_at timestamptz NOT NULL,
  like_count int NOT NULL DEFAULT 0, comment_count int NOT NULL DEFAULT 0
);
CREATE INDEX ON posts (user_id, created_at DESC, post_id DESC);
-- media (post_id, position, url = object-store key, kind): carousel order kept by position
CREATE TABLE likes (
  user_id bigint REFERENCES users, post_id bigint REFERENCES posts ON DELETE CASCADE,
  created_at timestamptz NOT NULL,
  PRIMARY KEY (post_id, user_id)                  -- one like per user per post
);
-- comments (comment_id, post_id, user_id, parent_id REFERENCES comments, body, created_at)
```

The same file also creates `direct_messages` (two FKs to `users`, sender and recipient), `notifications` (with `actor_id` and `post_id`, so tapping one can open the post that caused it), and `user_groups` + `group_members` — a group's multivalued "members" attribute becomes a junction table that can also record `joined_at`. `user`, `group` and `like` are reserved words in PostgreSQL or MySQL, hence the plural or prefixed names.

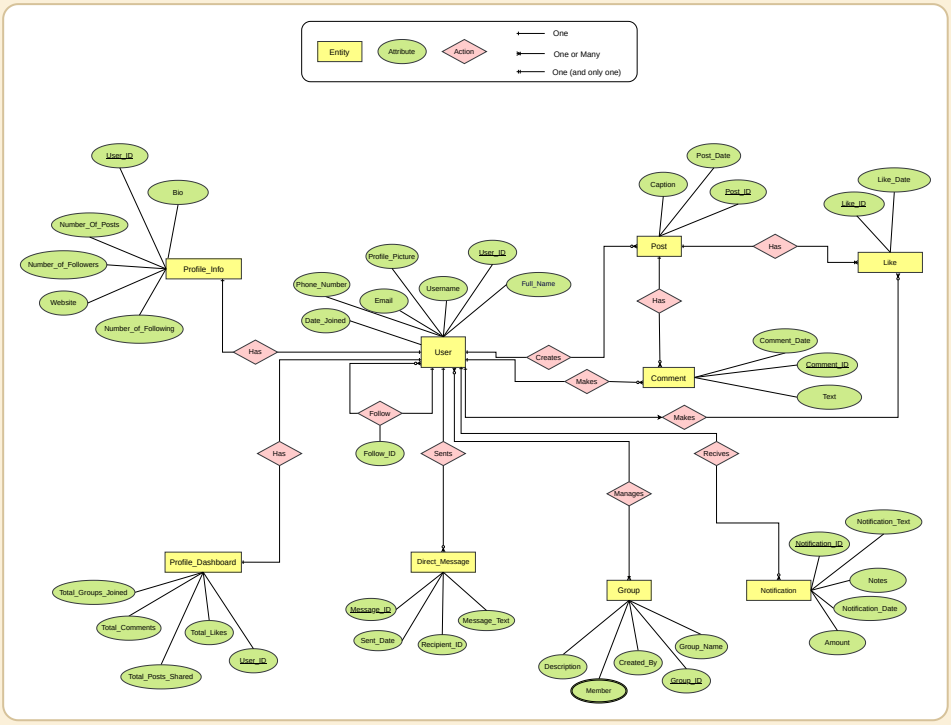


Figure — the course's Instagram ER diagram, with Follow drawn as a relationship from User back to User and profile data as 1:1 tables (course diagram)

The course version keeps profile numbers (posts, followers, following) in a separate **1:1 table** whose primary key is also the FK to the user — a *shared primary key*, the simplest way to guarantee one row per user. Here those counters sit on `users` directly; either way they are a speed decision the normal forms cannot see, because the redundancy spans two tables.

Key queries

The home feed is a join from "whom I follow" to their posts. **Fan-out on read** means computing it at request time, as here:

```
SELECT p.post_id, u.username, p.caption, p.created_at, p.like_count
FROM follows f
JOIN posts p ON p.user_id = f.followee_id
JOIN users u ON u.user_id = p.user_id
WHERE f.follower_id = 1
ORDER BY p.created_at DESC, p.post_id DESC
LIMIT 2;
-- next page: add  AND (p.created_at, p.post_id) < ('2026-05-02 07:30+00', 1003)
```

Output:

```
 post_id | username |           caption           |       created_at       | like_count
---------+----------+-----------------------------+------------------------+------------
    1005 | cy       | Sourdough attempt 2 #baking | 2026-05-03 10:15:00+00 |          1
    1003 | bo       | Trail views #running #hills | 2026-05-02 07:30:00+00 |          2

 post_id | username |       caption        |       created_at        (page 2)
---------+----------+----------------------+------------------------
    1002 | cy       | Fresh bread #baking  | 2026-05-01 08:00:00+00
    1001 | bo       | Sunrise run #running | 2026-05-01 06:10:00+00
```

Page 2 uses **keyset pagination** (`OFFSET n` would skip the first n rows by reading and discarding them): "continue after the last `(created_at, post_id)` I showed" instead of `OFFSET`, so it stays fast on page 500 and never repeats or skips a post when new ones arrive (see Chapter 12). `post_id` breaks ties between posts with the same timestamp.

A like must insert the row **and** bump the counter together, and a double tap must not count twice:

```
BEGIN;
WITH ins AS (
  INSERT INTO likes (user_id, post_id, created_at) VALUES (4, 1005, '2026-05-03 13:00+00')
  ON CONFLICT DO NOTHING RETURNING post_id
)
UPDATE posts SET like_count = like_count + 1 WHERE post_id IN (SELECT post_id FROM ins)
RETURNING post_id, like_count;
-- ...the same statement again a minute later...
COMMIT;
```

Output:

```
 post_id | like_count
---------+------------
    1005 |          2
(1 row)

 post_id | like_count          <- the repeat like changed nothing
---------+------------
(0 rows)
```

(`ON CONFLICT DO NOTHING` silently skips an insert that would violate a unique key; the **CTE** — `WITH ins AS (…)`, a named sub-query — returns a row only if the insert happened.) `04_instagram.sql` also finds mutual follows (a **self-join** — a table joined to itself — of `follows` with the columns swapped), prints the comment thread under a post indented by depth with a recursive CTE over `parent_id` (the technique of Chapter 7), and ranks hashtags by posts in the last three days. The file's last query is a **reconciliation check** that recounts likes and lists posts whose cached `like_count` disagrees (it returns 0 rows). Every denormalized counter deserves one.

Indexes

Access path	Index
Whom does X follow?	`follows` PK `(follower_id, followee_id)`
Who follows X?	`follows (followee_id)`
X's posts, newest first (profile grid, feed merge)	`posts (user_id, created_at DESC, post_id DESC)`
Has X liked this post? / who liked it	`likes` PK `(post_id, user_id)`
Posts with a tag	`post_hashtags` PK `(tag_id, post_id)`

Design review

The classic defect in a social schema is writing follow and like as 1:N: both are M:N and need a uniqueness rule on the pair (here the composite PKs), plus `CHECK (follower_id <> followee_id)`. The PK order decides which direction each index serves, so `follows (followee_id)` and `group_members (user_id)` exist for the reverse questions. Follow rows carry `created_at` ("since when?"), comments carry `parent_id` for replies, and media is a table because a post can hold several.

Scaling notes

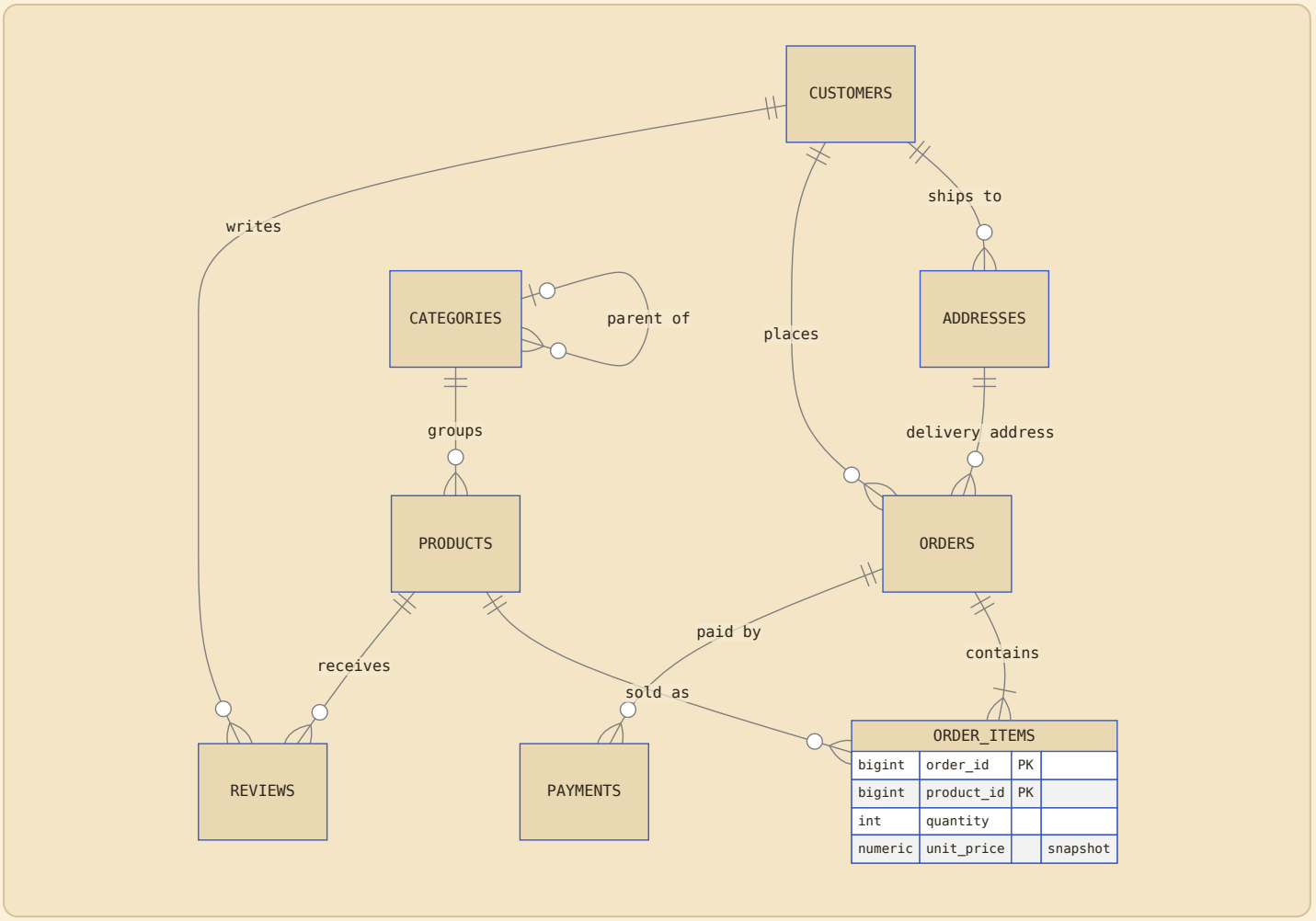
- **Media** lives in object storage (S3-like) behind a **CDN** (content delivery network — servers around the world that cache files close to users; see System Design Fundamentals); rows keep keys only.
- **Feed:** fan-out on read is fine for small follow counts. At scale, **fan-out on write** pushes each new post id into every follower's precomputed feed list (often Redis or a wide-column store); celebrities with millions of followers are handled by fan-out on read, merged at request time — the classic hybrid (Chapter 16).
- **Counters** are the hottest rows: a viral post turns `UPDATE posts SET like_count = …` into a lock queue (**lock contention**: many transactions waiting for the same row). Batch increments in a cache and flush periodically, or keep sharded counter rows and sum them.
- **Sharding** by `user_id` keeps a user's posts, likes-given and follows together; ids come from a generator that embeds time (Snowflake-style — Twitter's scheme of timestamp + machine id + counter) so they sort by creation without a central sequence (Chapter 15 §15.3).

17.5 E-commerce store (runnable)

Requirements

- **Customers** register, keep several **addresses**, browse **products** (each identified by a **SKU**, stock-keeping unit — the shop's product code) organized in a **category tree**, and read/write **reviews** (one per customer per product).
- A customer places an **order** of several products; each **order item** records quantity and the price *at purchase time*.
- Stock is decremented on purchase and may never go negative. Orders move through `pending → paid → shipped → delivered` (or `cancelled`); **payments** attach to orders.
- Discontinued products disappear from the shop but old orders still show them.
- Each order has one **delivery** record (status, date, partner) and may use one **coupon** (code, discount, expiry, usage limit).
- Non-functional: browsing is read-heavy; checkout must be strictly consistent.
- Questions: a customer's orders newest first, the lines of an order and their prices, products in a category with stock, a product's average rating, undelivered orders, whether a coupon code is valid.

Entities and relationships



`order_items` is the junction between `orders` and `products` (M:N) carrying its own attributes. `categories.parent_id` is a self-reference that forms a tree (Electronics → Audio → Headphones).

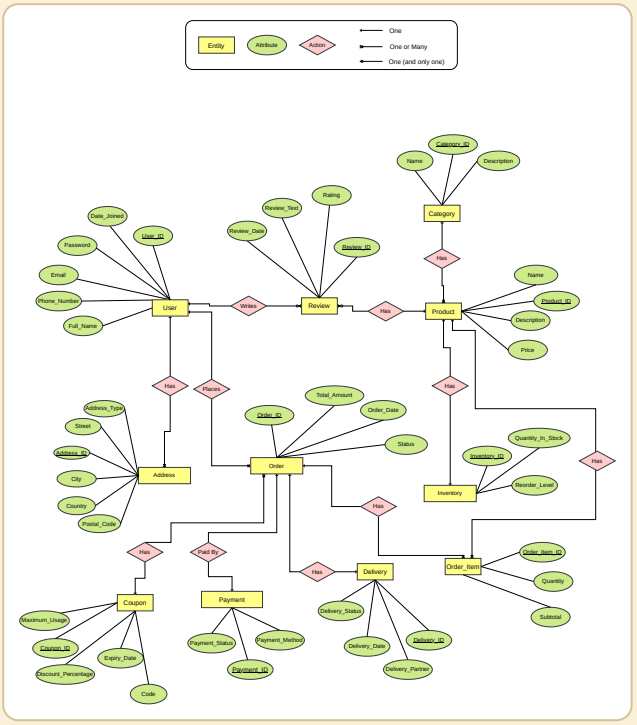


Figure — the course's e-commerce ER diagram with eleven entities, including inventory, delivery and coupon (course diagram)

Not in the runnable schema but part of the full design: `deliveries (order_id UNIQUE REFERENCES orders, status, delivered_on, partner)` and `payments` with `order_id UNIQUE` if exactly one payment per order is the rule — the `UNIQUE` is what makes each 1:1; `coupons (code UNIQUE, discount_pct, expires_on, max_uses)` with a **nullable** `orders.coupon_id` (partial participation); stock either on `products` (as here) or in a 1:1 `inventory` table keyed by `product_id` with a reorder level; and `password_hash` on customers.

Key tables

```
CREATE TYPE order_status AS ENUM ('pending', 'paid', 'shipped', 'delivered', 'cancelled');

CREATE TABLE products (
  product_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sku         text NOT NULL UNIQUE,
  name        text NOT NULL,
  category_id int NOT NULL REFERENCES categories,
  price       numeric(10,2) NOT NULL CHECK (price >= 0),
  stock       int NOT NULL CHECK (stock >= 0),     -- overselling becomes impossible
  deleted_at  timestamptz                          -- soft delete
);
CREATE TABLE orders (
  order_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES customers,
  ship_to     bigint NOT NULL REFERENCES addresses,
  status      order_status NOT NULL DEFAULT 'pending',
  placed_at   timestamptz NOT NULL,
  total       numeric(12,2) NOT NULL DEFAULT 0      -- denormalized sum of the items
);
CREATE TABLE order_items (
  order_id   bigint REFERENCES orders ON DELETE CASCADE,
  product_id bigint REFERENCES products,
  quantity   int NOT NULL CHECK (quantity > 0),
  unit_price numeric(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id)
);
CREATE INDEX ON orders (customer_id, placed_at DESC);
CREATE INDEX ON products (category_id) WHERE deleted_at IS NULL;   -- partial: live products only
```

Three decisions worth saying out loud in an interview:

- **Price snapshot.** `order_items.unit_price` copies the price when the order is placed. It is not a normalization violation: the price *paid* is a different fact from the price *now*.
- **Soft delete.** Deleting a product row would break the foreign keys of old orders (or **cascade** — automatically delete — the history with it). `deleted_at` hides it from the shop instead; the **partial index** (an index over only the rows matching a `WHERE`) keeps the live catalogue lookups small.
- **Status enum.** A PostgreSQL `ENUM` (a custom type allowing only a fixed list of values; or a `CHECK … IN (…)`, or a **lookup table** — a small table of allowed values referenced by a foreign key) limits `status` to known values. Enums are compact but adding a value needs `ALTER TYPE`; a lookup table is the most flexible (MySQL has inline `ENUM`; SQL Server and SQLite use `CHECK`).

Key queries

Placing an order is one transaction: create the order, decrement stock **only where enough stock exists**, copy the prices, then set the total (`01_ecommerce.sql`):

```
BEGIN;
INSERT INTO orders (customer_id, ship_to, placed_at)
VALUES (2, 2, '2026-05-05 12:00+00') RETURNING order_id \gset
WITH cart(product_id, qty) AS (VALUES (1::bigint, 2), (3::bigint, 1)),
reserved AS (
  UPDATE products p SET stock = p.stock - c.qty
  FROM cart c
  WHERE p.product_id = c.product_id AND p.stock >= c.qty AND p.deleted_at IS NULL
  RETURNING p.product_id, c.qty, p.price
)
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT :order_id, product_id, qty, price FROM reserved
RETURNING order_id, product_id, quantity, unit_price;
-- the application compares the row count with the cart size and issues ROLLBACK if short
UPDATE orders SET total = (SELECT sum(quantity * unit_price) FROM order_items WHERE order_id = :order_id)
WHERE order_id = :order_id RETURNING order_id, total;
COMMIT;
```

Output:

```
 order_id | product_id | quantity | unit_price
----------+------------+----------+------------
        5 |          1 |        2 |      89.00
        5 |          3 |        1 |      29.99
(2 rows)

 order_id | total
----------+--------
        5 | 207.99
```

(`\gset` is a psql command that stores the returned `order_id` in a variable; application code would read it from the `RETURNING` result.) Because the `UPDATE` both checks and decrements stock in one statement, two concurrent checkouts for the last unit cannot both succeed: the second one waits for the first's **row lock** (released when the first transaction commits or rolls back — `ROLLBACK` undoes everything since `BEGIN`), then re-checks `stock >= qty` on the new value (Chapter 13 §13.7). The `CHECK (stock >= 0)` is the last line of defence:

```
psql:01_ecommerce.sql:35: ERROR:  new row for relation "products" violates check constraint "products_stock_check"
DETAIL:  Failing row contains (2, SP-200, Bluetooth speaker, 2, 45.50, -1, null).
```

A best-seller report still counts the soft-deleted earbuds (2 units, 19.98), because history must not change when the catalogue does. The file also walks the category tree with a recursive CTE (all live products under Electronics at any depth) and runs a **payment reconciliation**: orders marked paid whose payments do not add up to the total. It finds order 3 (total 45.50, paid 30.00) — the kind of mismatch a nightly job should flag.

Indexes

`orders (customer_id, placed_at DESC)` serves "my orders"; `order_items (product_id)` serves sales per product (the PK already covers lookups by `order_id`); the partial index on live products serves category pages. Product search by text belongs in full-text search (Chapter 14) or a search engine.

Design review

- **Normal forms.** `order_items` and `reviews` have two-column candidate keys and every other column depends on the whole pair, so 2NF holds. A stored line `subtotal = quantity × unit_price` would break 3NF (and BCNF) on purpose; `orders.total` is the same redundancy across two tables — kept for fast order lists and checked by a reconciliation query.
- **Freeze what the order promised.** The unit price is snapshotted; the shipping address should be too (copy it onto the order), or editing an address rewrites where old orders "went".
- **Carts** look like orders (`carts` + `cart_items`) but read the *live* price every time.
- **Still missing:** counting coupon uses against `max_uses` (per customer or overall — concurrent checkouts must lock or count atomically) and an order status history.

Scaling notes

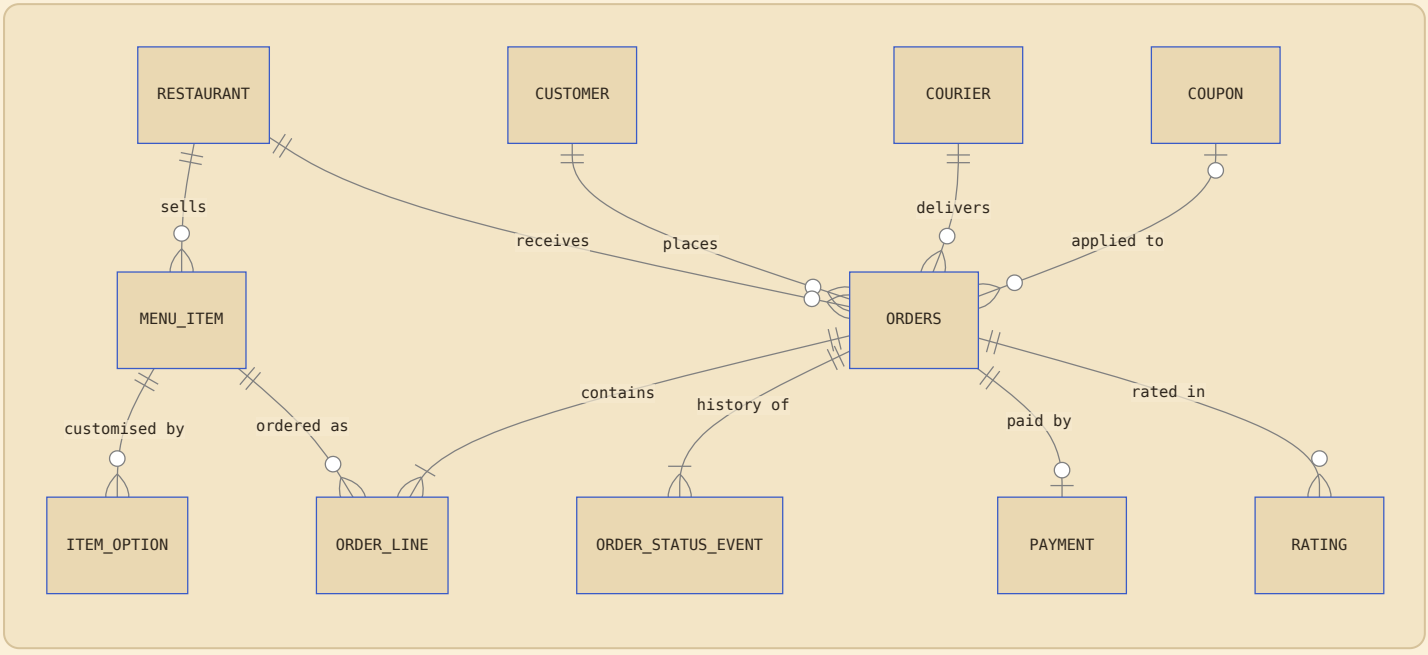
The catalogue is read-mostly: cache it and serve it from replicas or a search index. Carts are short-lived and **high-churn** (created and discarded constantly) — a **key-value store** (a database that just maps a key to a value, like Redis) with expiry fits them well. Orders and payments stay relational and ACID; partition `orders` by `placed_at` month for archiving and shard by `customer_id` when a single primary is saturated. For flash sales, a single **hot** (very frequently updated) `stock` row becomes the **bottleneck** (the one part that limits overall speed) — pre-split inventory into several rows (buckets) or reserve stock in a fast counter and settle in the database.

17.6 Food delivery platform

Requirements

- **Customers** order from **restaurants**; each restaurant has a **menu** of **items** (with options such as size or toppings) and opening hours.
- An **order** contains items from one restaurant; it moves `placed → accepted → preparing → picked_up → delivered` (or `cancelled`), and every change is time-stamped.
- A **delivery partner** (courier) is assigned to each order; their location updates every few seconds while active.
- **Payments**, **ratings** for restaurant and courier, and **promotions/coupons**. Customers keep several delivery **addresses**; menu items belong to **categories**.
- Non-functional: lunch and dinner peaks, live tracking, search by distance.
- Questions: today's menu of a restaurant, a customer's orders this month, the lines of an order, where an order is and when it was picked up, a courier's deliveries today, a restaurant's reviews.

Entities and relationships



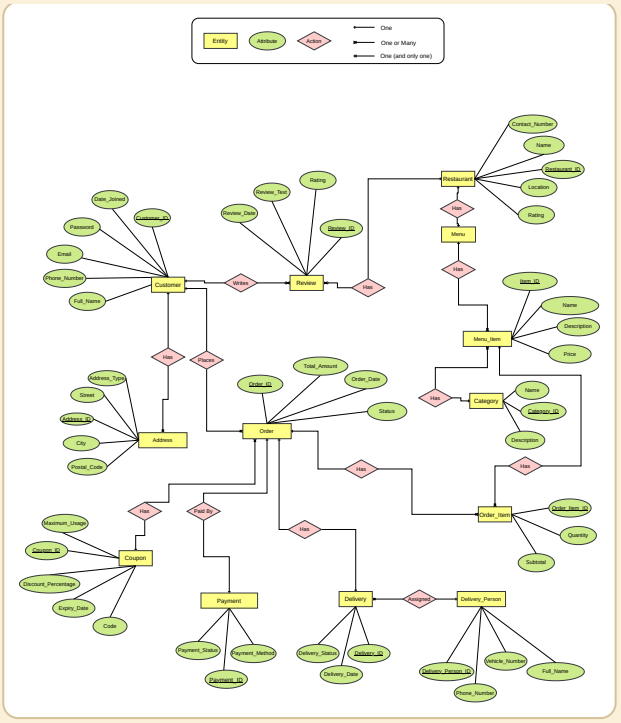


Figure — the course's food-delivery ER diagram: thirteen entities, with restaurant–menu, order–payment and order–delivery as 1:1 relationships (course diagram)

The course model reaches the restaurant only through order line → menu item → menu, which cannot stop one order mixing two kitchens; the `restaurant_id` on `orders` above enforces "one restaurant per order" directly. It also has a separate 1:1 `menus` table that stores nothing but a key — menu items can point at the restaurant instead. Its **delivery** is a 1:1 table (`deliveries.order_id UNIQUE`) with the current status and the courier, plus a status-history table — the same pairing as `orders.status` + `order_status_events` here.

Key tables

```
CREATE TABLE orders (
  order_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id   bigint NOT NULL REFERENCES customers,
  restaurant_id bigint NOT NULL REFERENCES restaurants,
  courier_id    bigint REFERENCES couriers,          -- NULL until assigned
  status        text NOT NULL,                        -- current state (denormalized)
  delivery_addr jsonb NOT NULL,                       -- snapshot of the address + coordinates
  subtotal numeric(10,2) NOT NULL, delivery_fee numeric(10,2) NOT NULL,
  placed_at     timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE order_status_events (                    -- append-only history
  order_id  bigint REFERENCES orders,
  status    text NOT NULL,
  at        timestamptz NOT NULL DEFAULT now(),
  actor     text NOT NULL,                            -- 'customer', 'restaurant', 'courier', 'system'
  PRIMARY KEY (order_id, at)
);
-- order_lines (order_id, line_no, item_id, quantity, unit_price snapshot, options jsonb snapshot)
```

The pairing of a **current-status column** with an **append-only event table** is a pattern worth remembering: the column answers "what state is it in?" with no join, the events answer "when did each step happen?" (for **SLAs** — service-level agreements, promised targets such as "delivered within 40 minutes" — disputes, and delivery-time analytics).

Key queries

1. Open restaurants within 5 km that serve "pizza" — needs a **geospatial** index (one that understands map coordinates; PostGIS, Postgres's geography extension, `geography` + GiST; see Chapter 16 §16.8).

2. A restaurant's live queue: `WHERE restaurant_id = $1 AND status IN ('placed','accepted','preparing') ORDER BY placed_at`.
3. Nearest available courier for an order (geo query over the couriers' latest positions).
4. Average prep time per restaurant: time between the `accepted` and `picked_up` events.

Indexes

`orders (restaurant_id, status)` partial `WHERE status NOT IN ('delivered','cancelled')` keeps the live-queue index tiny; `orders (customer_id, placed_at DESC)`; a GiST index on restaurant location.

Design review

A stored line subtotal and a stored `restaurants.rating` (instead of `avg()` over reviews) are the deliberate redundancies — name both. Payments should carry their own amount (partial payments, refunds, tips); a review should reference an order, or anyone can review a restaurant they never ordered from; a coupon's usage limit needs an atomic count, because two simultaneous checks in application code both pass; and the delivery address is snapshotted (`delivery_addr jsonb`) so editing an address never moves last month's order.

Scaling notes

Courier locations are high-frequency, short-lived writes — keep them in an **in-memory store** (a database that keeps data in RAM for speed) with geo support (Redis `GEOADD` / `GEOSEARCH`) and write only trip summaries to PostgreSQL. Shard orders by city/region: almost no order crosses a city. Menus are read-heavy and cacheable.

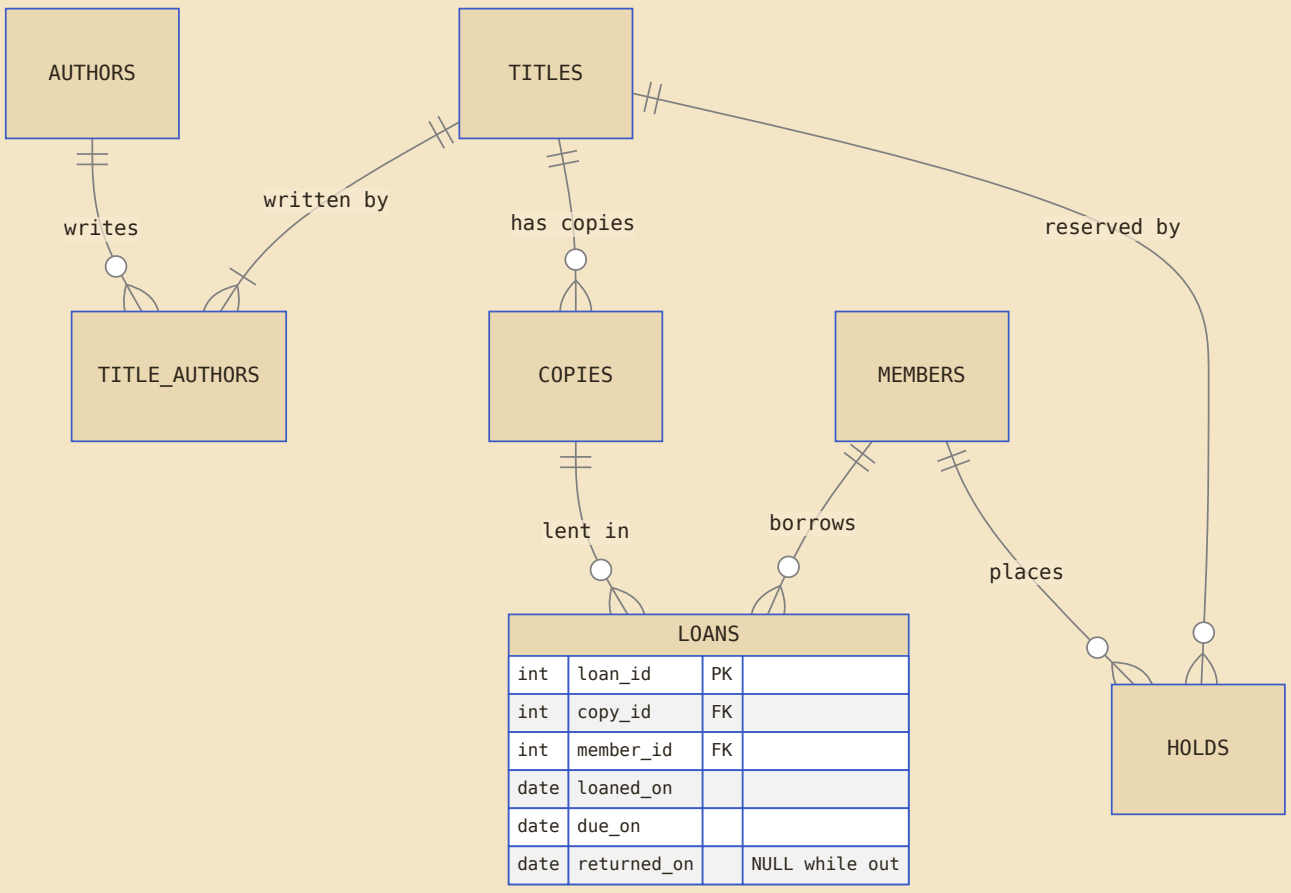
17.7 Library management system (runnable)

Requirements

- The catalogue lists **titles** (identified by **ISBN**, the international book number) with one or more **authors** in a fixed order; the library owns several physical **copies** of a title across **branches**.
- **Members** borrow copies (**loans**) for a fixed period; each member has a maximum number of open loans.
- A copy can be on at most one open loan. Late returns incur a **fine** per day.
- Members place **holds** (reservations) on a title when no copy is free; holds are served first-come, first-served.
- Lost or discarded copies leave the catalogue but their loan history stays.
- Questions: can a member get this title today; which of a member's loans are late; the lending history of one physical copy; the order of the hold queue; a member's unpaid total.

Entities and relationships

The library is the cleanest example of **type versus instance**: a *title* is the abstract book, a *copy* is the object on the shelf. Loans point at copies; holds point at titles (you want *any* copy).



Key tables

```
CREATE TABLE title_authors (                        -- M:N junction with an ordering column
  isbn char(13) REFERENCES titles, author_id int REFERENCES authors,
  position smallint NOT NULL,
  PRIMARY KEY (isbn, author_id)
);
CREATE TABLE copies (
  copy_id int PRIMARY KEY, isbn char(13) NOT NULL REFERENCES titles,
  branch text NOT NULL, condition text NOT NULL DEFAULT 'good',
  withdrawn_at date                                 -- soft delete
);
CREATE TABLE loans (
  loan_id int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  copy_id int NOT NULL REFERENCES copies, member_id int NOT NULL REFERENCES members,
  loaned_on date NOT NULL, due_on date NOT NULL, returned_on date,
  CHECK (due_on > loaned_on),
  CHECK (returned_on IS NULL OR returned_on >= loaned_on)
);
-- At most ONE open loan per copy:
CREATE UNIQUE INDEX one_open_loan_per_copy ON loans (copy_id) WHERE returned_on IS NULL;
CREATE TABLE holds (
  isbn char(13) REFERENCES titles, member_id int REFERENCES members,
  placed_at timestamptz NOT NULL,
  PRIMARY KEY (isbn, member_id)                     -- one hold per member per title
);
```

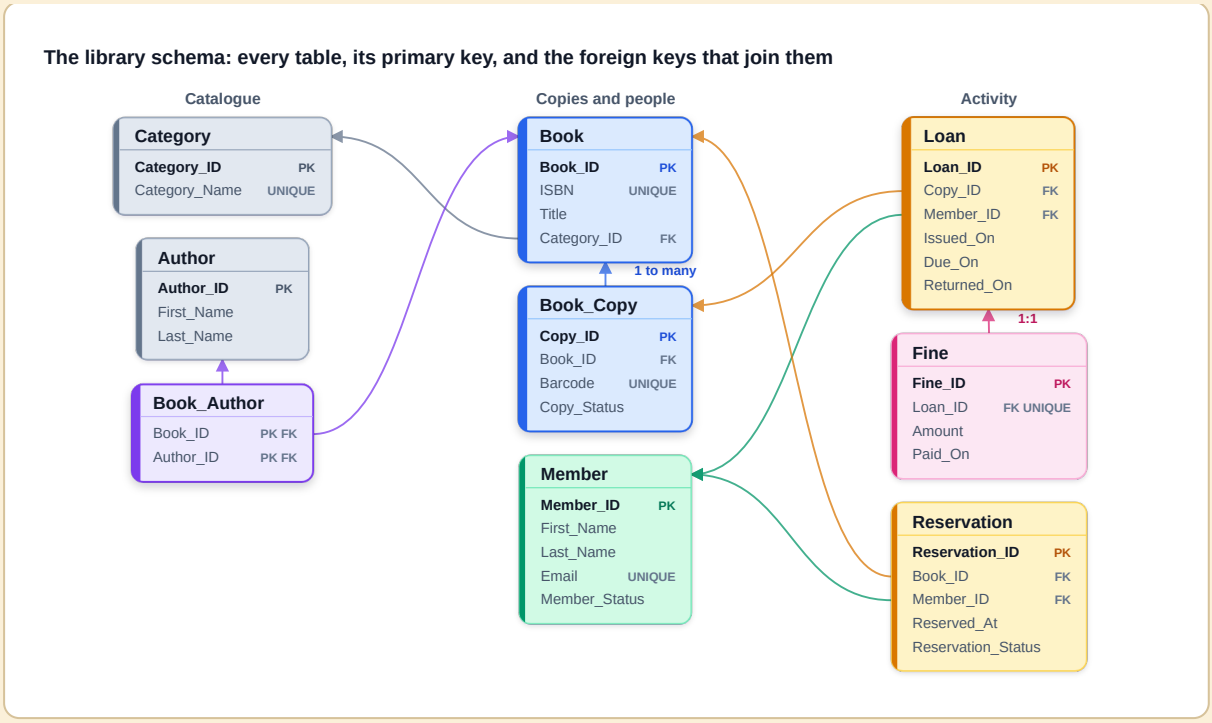


Figure — the course's library model: Book and Book_Copy at the centre, with loans pointing at copies and reservations at books (course diagram)

The tell-tale pair of foreign keys: `loans.copy_id` **but** `holds.isbn` — members borrow one physical object but reserve a title. A copy's `copy_id` plays the role of its barcode. The schema also has `fines (loan_id PRIMARY KEY REFERENCES loans, amount, paid_on)`: using the loan id as the key makes it 1:1 (at most one fine per loan), and the amount is **stored** although it looks derived, because a later change of the daily rate must not rewrite old fines. The course version gives books a **surrogate** `book_id` (an artificial id with no business meaning, as opposed to a **natural key** such as the ISBN) with `isbn UNIQUE` (shorter FKs, and some items have no ISBN) and a `category` table; authors live in a junction table rather than a comma-separated `authors` column, which would break 1NF and turn "books by this author" into a text search.

Note what is **not** stored: a copy's "available" flag. Availability is derived from "no open loan and not withdrawn", so it can never disagree with the loans table. The **partial unique index** is the star: it enforces a rule about *some* rows (open loans) that a plain `UNIQUE` cannot, and it doubles as the index for "is this copy out?". (SQLite and PostgreSQL support partial indexes; SQL Server calls them *filtered* indexes; MySQL has neither — there you would use a **generated column** (a column computed automatically from others) that is non-NULL only for open loans and put a unique index on it.)

Key queries

The catalogue view — authors in order, and live availability (`03_library.sql`, "today" fixed at 2026-05-10):

```
SELECT t.title,
       string_agg(a.name, ', ' ORDER BY ta.position) AS authors,
       (SELECT count(*) FROM copies c
         WHERE c.isbn = t.isbn AND c.withdrawn_at IS NULL
           AND NOT EXISTS (SELECT 1 FROM loans l WHERE l.copy_id = c.copy_id AND l.returned_on IS NULL)
       ) AS available
FROM titles t JOIN title_authors ta USING (isbn) JOIN authors a USING (author_id)
GROUP BY t.isbn
ORDER BY t.title;
```

Output:

```
         title         |      authors       | available
-----------------------+--------------------+-----------
 Joins at Scale        | J. Ortiz, S. Novak |         0
 Query Plans Explained | R. Kumar           |         1
 Tables and Keys       | R. Kumar           |         1
```

(`Query Plans Explained` has two copies but one is withdrawn; `Tables and Keys` has two, one of them out.) Overdue loans and fines come straight from dates — `date - date` is a whole number of days in PostgreSQL:

```
 name |      title      |   due_on   | days_late | fine
------+-----------------+------------+-----------+------
 Omar | Tables and Keys | 2026-05-04 |         6 | 1.50
```

Trying to lend copy 101 while it is still out hits the partial unique index:

```
psql:03_library.sql:32: ERROR:  duplicate key value violates unique constraint "one_open_loan_per_copy"
DETAIL:  Key (copy_id)=(101) already exists.
```

Check-out with the member's limit is a conditional insert — it inserts only if the member has fewer open loans than `max_loans`:

```
INSERT INTO loans (copy_id, member_id, loaned_on, due_on)
SELECT 102, 2, :today, date :today + 14
WHERE (SELECT count(*) FROM loans WHERE member_id = 2 AND returned_on IS NULL)
      < (SELECT max_loans FROM members WHERE member_id = 2)
RETURNING loan_id, copy_id, member_id, due_on;
```

Output: `7 | 102 | 2 | 2026-05-24`. Under concurrency two check-outs by the same member could both see "1 open loan"; lock the member row first (`SELECT … FROM members WHERE member_id = 2 FOR UPDATE`) to serialize them (make them run one after the other; Chapter 13 §13.8 explains this **write-skew** shape — two transactions each read the same state, then both write based on it). The hold queue is a **window function** (a function that computes over a set of related rows without collapsing them, Chapter 8) — `row_number() OVER (PARTITION BY isbn ORDER BY placed_at)` — giving Eva position 1 and Omar position 2 for *Joins at Scale*.

Indexes

The partial unique index (open loan per copy), `loans (member_id) WHERE returned_on IS NULL` (a member's current loans), `copies (isbn)`, and a full-text index on titles/authors for search.

Design review

Loans are a *history* table — a `current_member_id` column on copies would answer only "who has it now". `title_authors` has no non-key columns beyond `position`, which depends on the whole key, so 2NF holds; ISBN and copy id are candidate keys, so BCNF holds. Still missing: holds that expire (an `expires_at` plus a sweep job when a copy is set aside) and a `CHECK` on copy condition so a typo cannot become a new status.

Scaling notes

A library is small — a single PostgreSQL instance serves a city's whole network. Historical loans can be partitioned by year. The only real concurrency hot spot is the popular new release with a long hold queue; handling "copy returned → offer to the next hold → expire after 3 days" as a small **state machine** (a fixed set of states with allowed transitions between them) in one transaction keeps it correct.

17.8 Ticket booking system (runnable, with seat-hold concurrency)

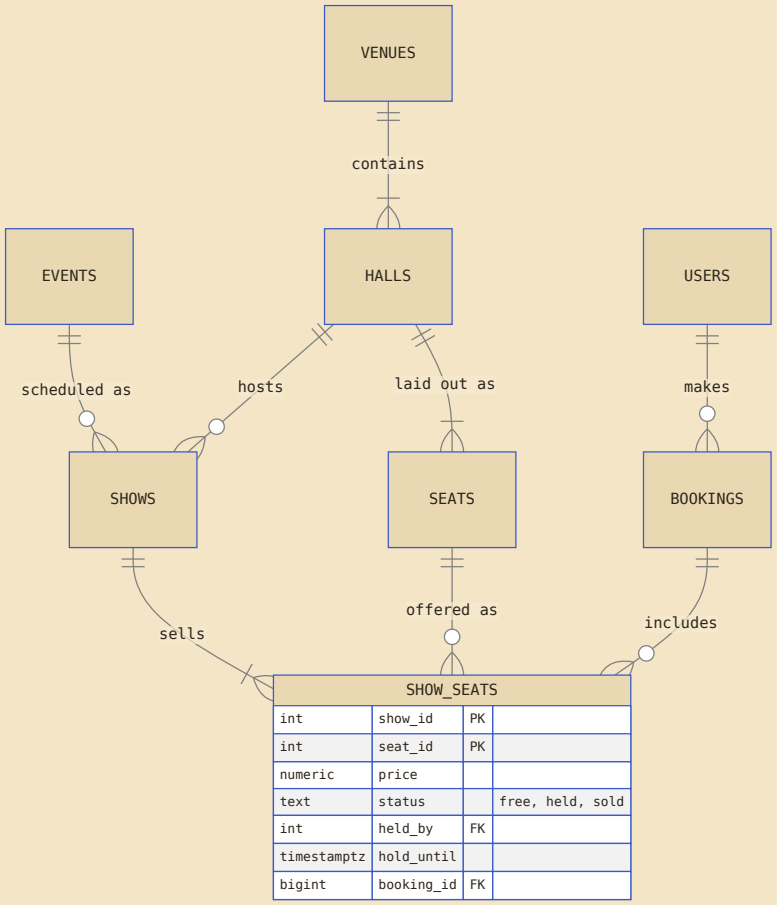
Requirements

- **Venues** have **halls** (screens/auditoriums) with a fixed **seat** layout (row, number, tier).
- **Events** (films, concerts) run as **shows** at a hall and time. Each seat of each show has a price.
- A user selects specific seats; they are **held** for a few minutes while the user pays. If payment succeeds the seats become **sold** under a **booking**; if the hold expires they return to sale.
- **No seat may ever be sold twice**, even when thousands of people click at the same moment when sales open.
- A booking has at most one **payment**; cancelling a booking releases its seats.

- Questions: free seats for a show, a customer's bookings, what is on in a city today, how full a show was, whether a booking is paid.

Entities and relationships

The key table is `show_seats` — one row per seat *per show*. The physical `seats` table never changes; `show_seats` is the inventory users compete for, and its row lock is the unit of concurrency.



Key tables

```sql
CREATE TABLE show_seats (
  show_id    int REFERENCES shows,
  seat_id    int REFERENCES seats,
  price      numeric(8,2) NOT NULL,
  status     text NOT NULL DEFAULT 'free' CHECK (status IN ('free', 'held', 'sold')),
  held_by    int REFERENCES users,
  hold_until timestamptz,
  booking_id bigint REFERENCES bookings,
  PRIMARY KEY (show_id, seat_id),               -- a seat appears once per show
  CONSTRAINT held_needs_owner_and_expiry
    CHECK ((status = 'held') = (hold_until IS NOT NULL AND held_by IS NOT NULL)),
  CONSTRAINT sold_needs_booking CHECK ((status = 'sold') = (booking_id IS NOT NULL))
);
CREATE INDEX ON show_seats (show_id) WHERE status = 'free';
```

The two named `CHECK`s tie the status to the columns that must accompany it: a held seat always has an owner and an expiry, a sold seat always has a booking, and a free seat has neither. Impossible states become impossible to store.

Three things all sound like "seat": the physical seat (row H, number 14), **the seat at one screening** (sold at 20:00, free at 18:00), and a claim on it. A single `is_booked` flag on `seats` fails as soon as a hall has a second show. The **price** also belongs to the seat-at-a-show pairing — neither the seat alone (weekends cost more) nor the show alone (recliners cost more) determines it.

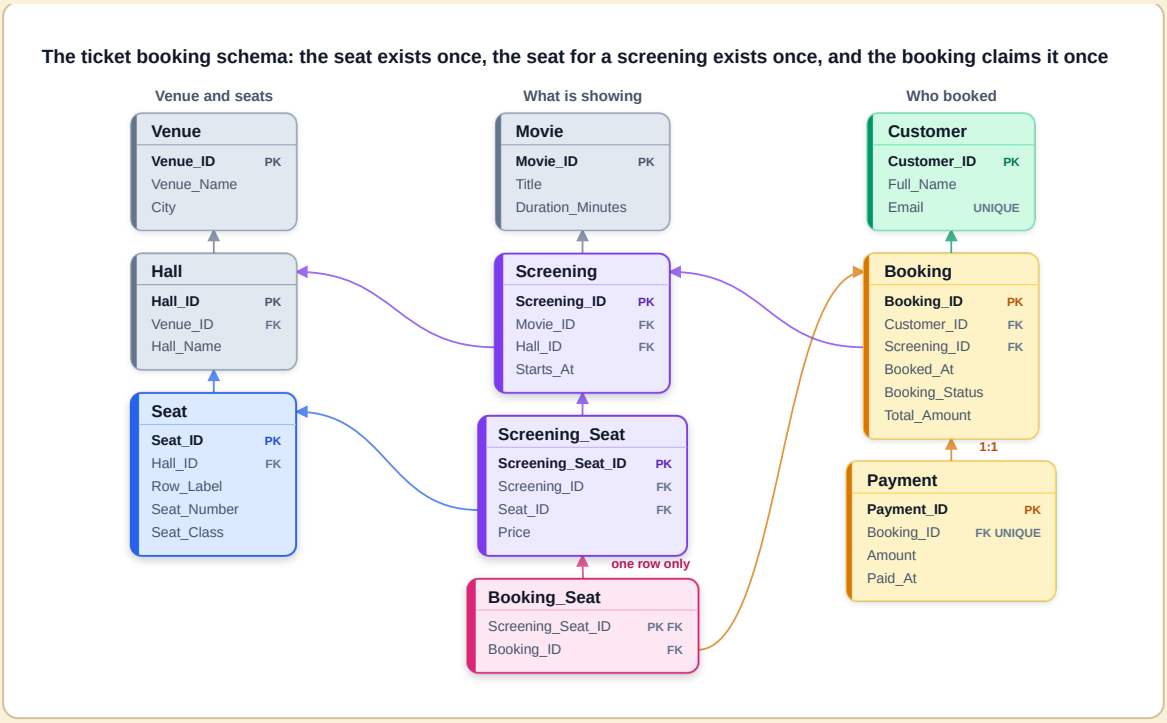


Figure — the course's ticket model: Seat → Screening_Seat → Booking_Seat is where the correctness lives (course diagram)

The course models the claim as a separate junction `booking_seats (screening_seat_id PRIMARY KEY, booking_id)`. The trap it highlights: a composite PK `(booking_id, screening_seat_id)` only stops *one booking* claiming a seat twice — two different bookings could still both claim H14. Making `screening_seat_id` the **whole** primary key means a seat can be claimed once, ever, and the second insert fails in the database. That design has no holds and must *delete* the claim rows when a booking is cancelled (marking the booking cancelled would block the seat forever); the `status` column used here gives the same once-only guarantee through the guarded `UPDATE` below and adds holds. The course also names the table `screening` because `SHOW` is reserved in MySQL; `shows` (plural) is safe.

Key queries

Hold seats with a *guarded* update: the `WHERE` clause only matches seats that are free (or whose hold has expired), and the rows it returns tell you exactly which seats you got (`02_ticket_booking.sql`, "now" fixed at 10:00):

```
UPDATE show_seats ss
SET status = 'held', held_by = 1, hold_until = timestamptz :now + interval '10 minutes'
FROM seats se
WHERE ss.seat_id = se.seat_id AND ss.show_id = 10
  AND se.row_label = 'B' AND se.seat_no IN (2, 3)
  AND (ss.status = 'free' OR (ss.status = 'held' AND ss.hold_until < :now))
RETURNING se.row_label || se.seat_no AS seat, ss.status, ss.hold_until;
```

Output — Asha gets B2 and B3; Ben's request a moment later for B3 and B4 returns only B4, so the app releases it and asks him to choose again:

```
 seat | status |       hold_until
------+--------+------------------------
 B2   | held   | 2026-05-20 10:10:00+00
 B3   | held   | 2026-05-20 10:10:00+00

 seat | held_by          <- Ben asked for B3 and B4
------+---------
 B4   |       2
```

Confirm after payment in one transaction — create the booking, then flip only this user's *unexpired* holds to sold:

```
BEGIN;
INSERT INTO bookings (user_id, show_id, status, created_at)
VALUES (1, 10, 'confirmed', :now) RETURNING booking_id \gset
UPDATE show_seats
SET status = 'sold', booking_id = :booking_id, held_by = NULL, hold_until = NULL
WHERE show_id = 10 AND held_by = 1 AND status = 'held' AND hold_until >= :now
RETURNING seat_id, status, booking_id;
COMMIT;
```

The file continues with "any two free premium seats" using `FOR UPDATE SKIP LOCKED` (lock matching rows but skip any another transaction already has locked) (many buyers each get *different* seats without waiting on each other — the pattern of Chapter 13 §13.12), an expiry sweep that frees stale holds, a failed attempt to mark a seat sold without a booking (`violates check constraint "sold_needs_booking"`), and occupancy per show with `count(*) FILTER (…)` (show 10: 2 sold of 12, revenue 400.00).

Proof on two live sessions

Does the guarded update really stop two *simultaneous* buyers? `05_seat_race.sql` opens two real connections, A and B, with PostgreSQL's `dblink` extension (which lets one session open and run queries on another connection), and runs the same hold on seat 9 in both, at the default `READ COMMITTED` **isolation level** (each statement sees only data committed before it started):

```
--- A: holds seat 9 (C1) but has not committed yet
 seat_id | held_by
---------+---------
       9 |       1
--- B: sends the same UPDATE for user 2; it must wait for A's row lock
 b_still_running | b_waits_on
-----------------+------------
               1 | Lock
--- A commits. B wakes up, re-checks status = 'free' on the new row version, and updates nothing
 rows_b_changed
----------------
              0
--- final state of the seat: held by user 1 only
 seat_id | status | held_by
---------+--------+---------
       9 | held   |       1
```

How it works:

1. A's `UPDATE` locks the row (seat 9) until A commits.
2. B's `UPDATE` finds the same row, sees it is locked, and waits (`wait_event_type = Lock`).
3. When A commits, PostgreSQL re-evaluates B's `WHERE` clause against the **newest** version of the row. `status = 'free'` is now false, so B updates zero rows.
4. The application sees 0 rows returned and tells user 2 "seat taken".

No `SELECT` then `UPDATE` (which *would* race), no application lock, no `SERIALIZABLE` needed. The alternatives: **optimistic locking** with a `version` column (don't lock; at write time check the version is unchanged and retry if it isn't — good when contention is low), or a unique constraint on a separate `seat_holds (show_id, seat_id)` table (insert-to-claim).

Indexes

The PK `(show_id, seat_id)` serves the seat map and every hold; the partial index on free seats serves "best available" and "how many left"; `show_seats (held_by) WHERE status = 'held'` would speed the confirm step; `shows (event_id, starts_at)` for listings.

Two rules remain outside these tables: someone must generate the `show_seats` rows for every new show (300 seats × 6 shows = 1,800 rows a day per hall — a real job), and two shows must not overlap in one hall, which needs an exclusion constraint on `(hall_id, tstzrange(starts_at, ends_at))` or a check in the scheduling transaction.

Scaling notes

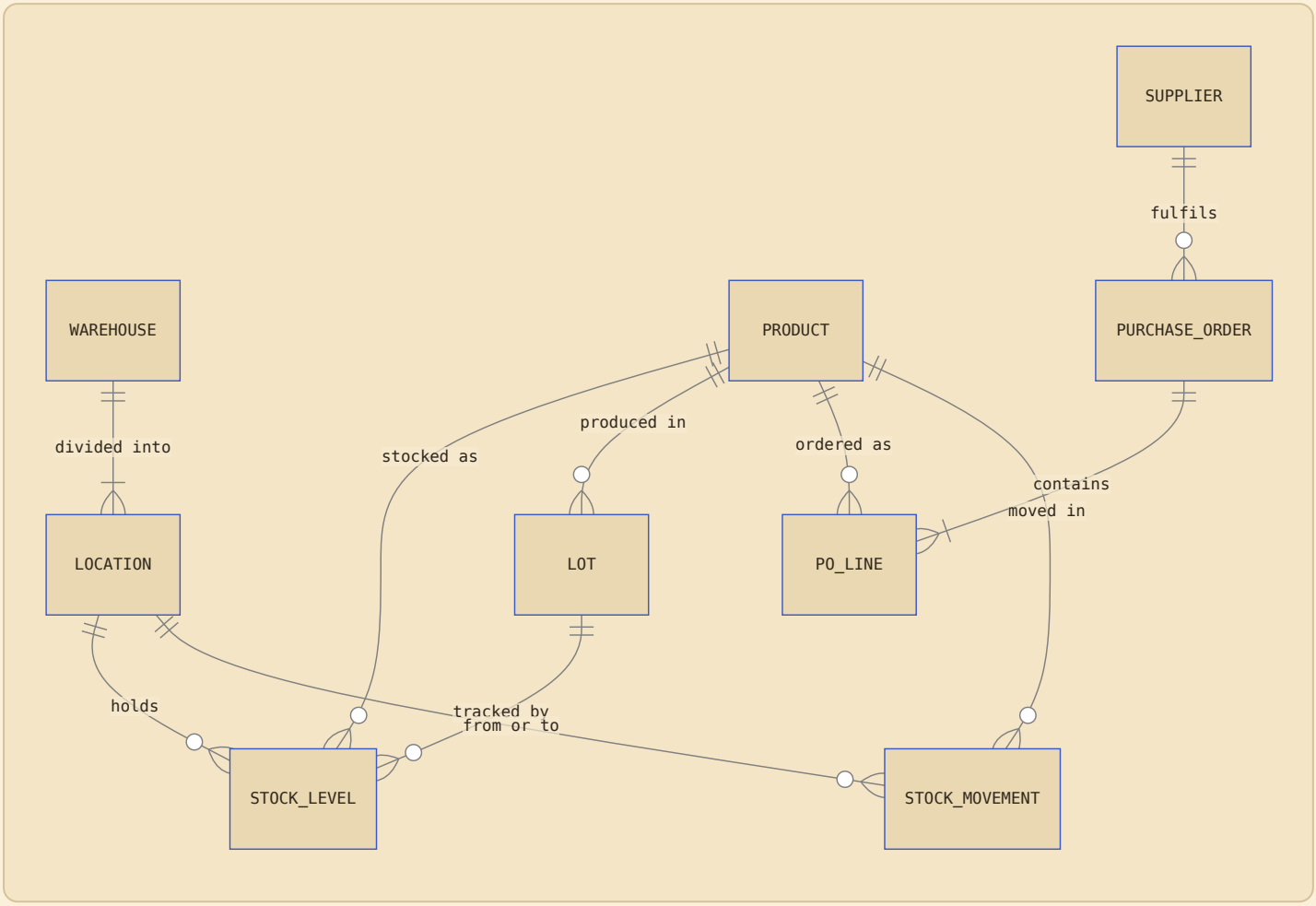
The whole problem is a **burst of contention on a small set of rows** (a big concert's on-sale minute). Rows per show are few, so the database can hold them, but connections cannot: put a **virtual waiting room / queue** in front (a holding page that admits users in batches), let only N users into seat selection at a time, and keep holds short. Shows are independent, so shard by `show_id` (or event); a hot show can even get a dedicated shard. Cache the seat map (it may be a second stale) but never the hold itself.

17.9 Warehouse inventory system

Requirements

- **Products** (SKUs) are stored in several **warehouses**, each divided into **bin locations** (aisle-rack-shelf).
- Track **stock on hand** per SKU per location, plus **reserved** quantity for open orders.
- Goods arrive via **purchase orders** from **suppliers** (receipts), leave via **shipments** for sales orders, and move between locations (**transfers**); every movement is recorded.
- Support **batches/lots** (groups of units made together, sharing an expiry date) with expiry dates (first-expired, first-out) and reorder alerts when stock falls below a **reorder point**.
- Each supplier has its own SKU and **lead time** (days from ordering to delivery) for a product; purchase orders have numbered lines.
- Questions: total stock of a SKU and its bin locations; a bin's movements over the past week; the cause of a count mismatch; the quickest supplier for a SKU; open purchase-order quantities.

Entities and relationships



Key tables

```sql
CREATE TABLE stock_levels (                     -- current state, one row per product-lot-location
  product_id  bigint REFERENCES products,
  location_id bigint REFERENCES locations,
  lot_id      bigint REFERENCES lots,
  on_hand     int NOT NULL CHECK (on_hand >= 0),
  reserved    int NOT NULL DEFAULT 0 CHECK (reserved BETWEEN 0 AND on_hand),
  PRIMARY KEY (product_id, location_id, lot_id)
);
CREATE TABLE stock_movements (                  -- append-only ledger of every change
  movement_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  product_id  bigint NOT NULL REFERENCES products,
  lot_id      bigint REFERENCES lots,
  from_location bigint REFERENCES locations,    -- NULL for receipts
  to_location   bigint REFERENCES locations,    -- NULL for shipments / write-offs
  quantity    int NOT NULL CHECK (quantity > 0),
  reason      text NOT NULL CHECK (reason IN ('receipt','shipment','transfer','adjustment','return')),
  reference   text,                             -- PO number, order number ...
  moved_at    timestamptz NOT NULL DEFAULT now(),
  CHECK (from_location IS NOT NULL OR to_location IS NOT NULL)
);
```

This is the same **ledger + cached balance** design as the bank (§17.3): movements are the truth, `stock_levels` is the fast answer, and both change in the same transaction. A periodic query that rebuilds levels from movements and compares them catches bugs — and explains a shelf count that disagrees with the screen by replaying one product's movements in order. Two opposite reasons to store a derivable value: the library fine is kept so it **never** changes; this balance is kept because summing every movement on each read would be too slow.

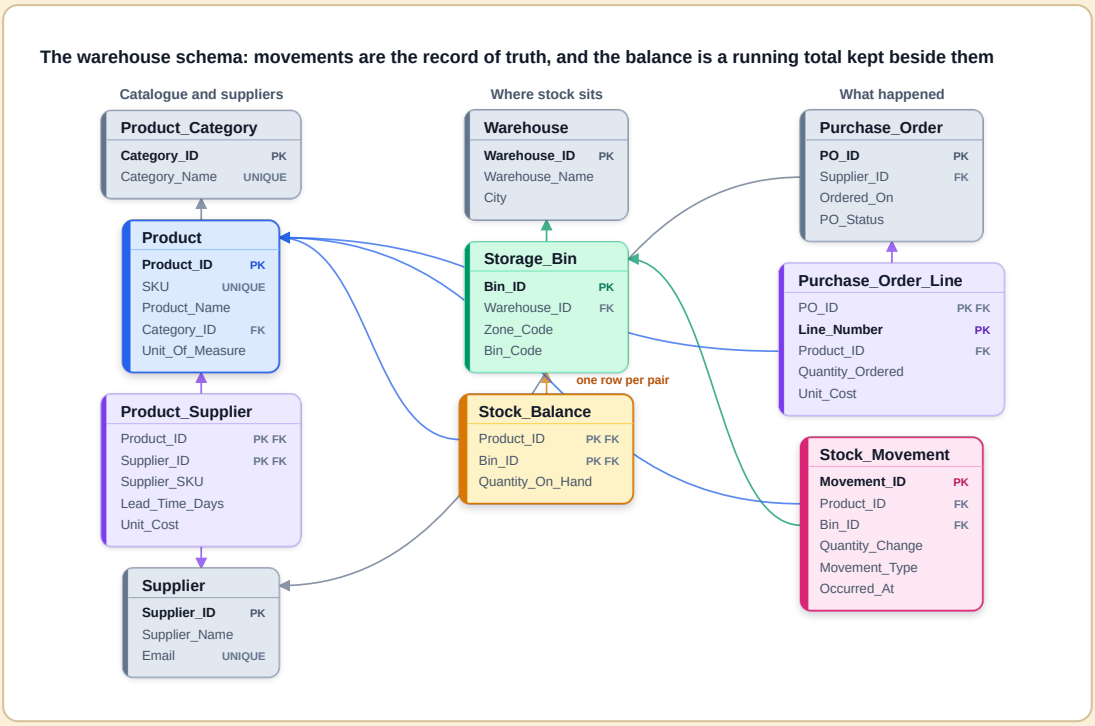


Figure — the course's warehouse model: Stock_Movement records what happened, Stock_Balance is the running total beside it (course diagram)

The course variant records one row per bin with a **signed** `quantity_change` (+50 received, −3 picked) instead of from/to columns — one table for every movement type, and a transfer becomes two rows. Its other tables: `product_suppliers (product_id, supplier_id, supplier_sku, lead_time_days, unit_cost)` — attributes of the *pairing*, so putting `supplier_name` there would be a 2NF **partial dependency** (a column depending on only part of a composite key); and `purchase_order_lines (po_id, line_no, …)`, a **weak entity** whose key includes its parent's, with `ON DELETE CASCADE`. `unit_cost` appears on both: the agreed price now versus the price paid on that order. A movement's cause is stored as `reference_type` + `reference_id` — a polymorphic reference no FK can check (see §17.14). Whether `on_hand` may drop below zero is a business

choice — a few operations like negative stock as an alarm for an unrecorded delivery — so choose deliberately and document it.

Key queries

1. Available to promise for a SKU across warehouses: `sum(on_hand - reserved)` grouped by warehouse.
2. Pick list for an order, first-expired-first-out: stock rows for the SKU joined to `lots`, ordered by `expires_on`, with a running total (window `sum() OVER (ORDER BY expires_on)`) until the ordered quantity is covered.
3. Reorder alerts: products whose total available is below `reorder_point`.
4. Movement history of a lot (for recalls).

Indexes

`stock_levels` PK for picks; `stock_levels (location_id)` for bin counts; `stock_movements (product_id, moved_at DESC)` and `(reference)`; `lots (product_id, expires_on)`.

Scaling notes

Each warehouse is mostly independent: shard or even run one database per site, with a central read model for company-wide stock. Movement tables grow forever — partition by month. Reservations for online orders are the contention point; a reservation row per order line (with expiry, like the ticket holds) is safer than decrementing a shared counter.

The last four cases are large consumer products. The relational core looks like the earlier designs; what changes is that **no single database fits every part**, so each design ends with a short "which store for which data" table (Chapter 16 §16.10 has the general decision guide).

At this scale the template gains three steps before the tables:

1. **Non-functional targets** — **high availability** (the service is almost always up), low **latency** (the time one request takes; here a few hundred ms), **horizontal scalability** (growing by adding machines), and *where **eventual consistency** (copies may briefly disagree but converge) is acceptable* (vote and view counts) versus where it is not (money, bookings, identity).
2. **A back-of-the-envelope storage estimate** — users × bytes per row, items per day × size. Multiply carefully (1 KB × 10 million = 10 GB, not 10 TB); the usual result is that rows are small and **media dominates** by orders of magnitude, which is why media goes to object storage.
3. **A high-level architecture** — clients → **load balancer** (spreads incoming requests over many servers) → **API gateway** (the single entry point that handles routing and authentication) → a few **microservices** (small, separately deployed services: user, content, search …), each owning its data store, with a **message queue** (Kafka, RabbitMQ — a buffer where one service leaves tasks for another to process later) for **asynchronous** work (done in the background, not while the user waits) such as search indexing, notifications and video processing.

After the schema come **sharding** (pick the key that keeps a request on one shard; use **consistent hashing** (a key-to-shard mapping where adding a shard moves only a small share of keys, §15.3) so adding a shard moves little data) and **replication** (primary–replica with automatic failover for the relational part, **leaderless** multi-node replication (any replica accepts writes) for Cassandra-style stores, replicas in several regions) — the mechanics are in Chapter 15.

17.10 Quora (questions and answers)

Requirements

- Users ask **questions**, write **answers**, and **comment** on both; questions carry **topics**.
- Users **upvote/downvote** answers; answers are ranked by votes and quality.
- Users **follow** people, topics and questions; a personalized feed shows new questions and answers from what they follow.
- Full-text **search** over questions; suggestions of similar questions while typing.

- Scale: hundreds of millions of monthly users, far more reads than writes. Vote counts may lag (eventual consistency is fine there).
- Estimate (100 M users at 2 KB; per year 10 M questions at 1 KB, 50 M answers at 2 KB, 500 M votes at 0.5 KB, 10 M media files at 500 KB): about 200 GB of user rows, then roughly 10 GB + 100 GB + 250 GB of text and **5 TB of media** per year — the relational data fits comfortably; media and replicas drive the storage budget.

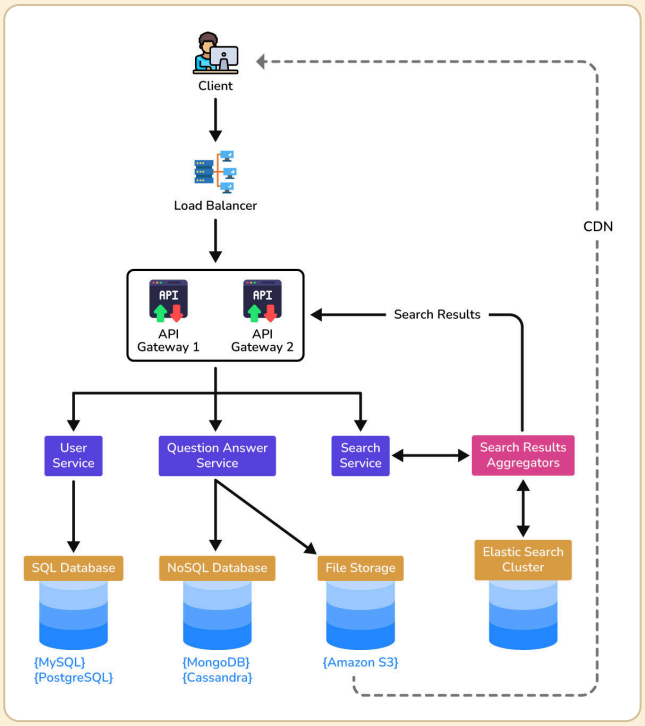
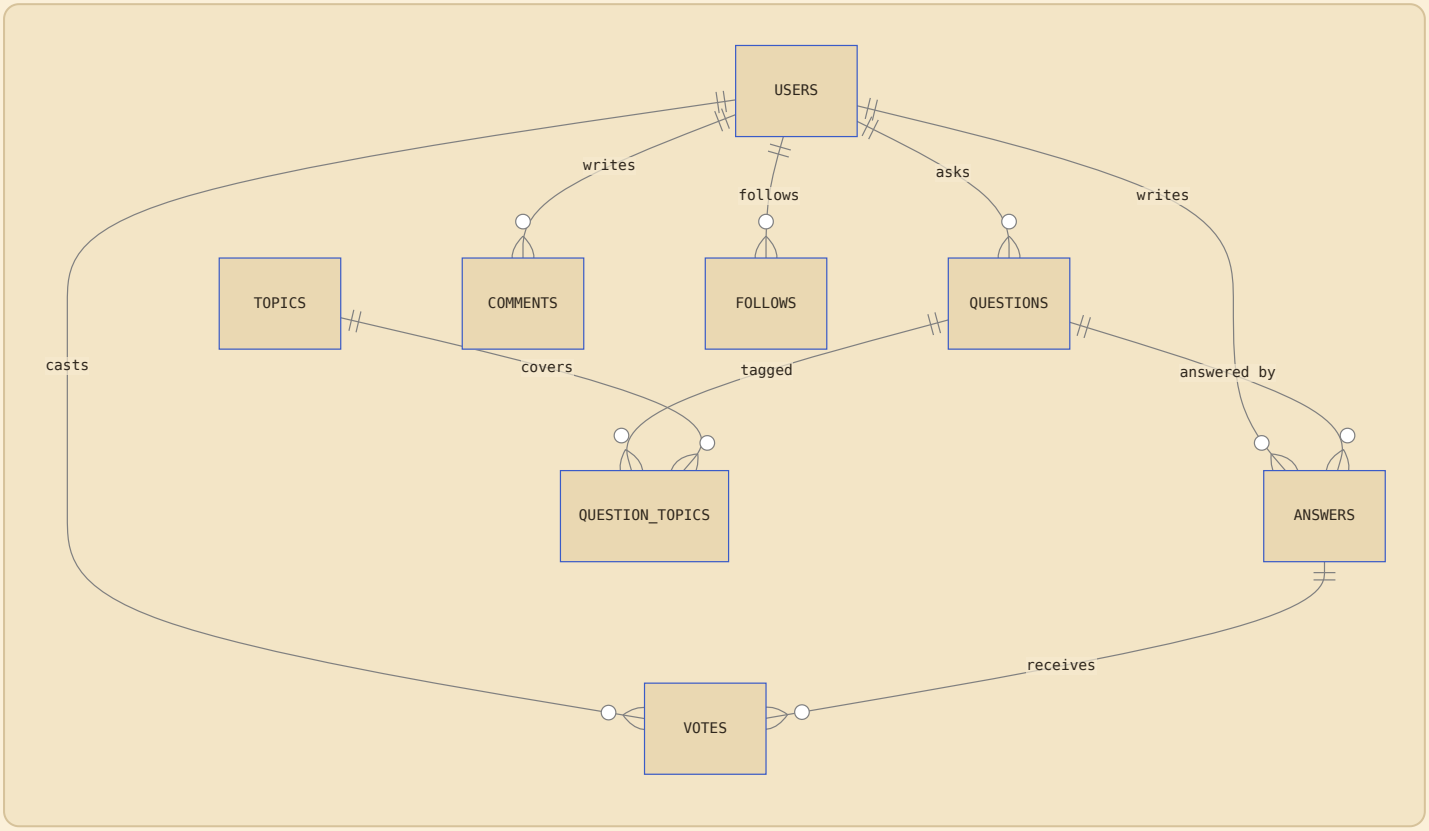


Figure — the course's Quora high-level design: clients, load balancer, API gateway, user / question-answer / search services, a relational DB, a NoSQL store, Elasticsearch, a message queue and file storage (course diagram)

Entities and relationships



Key tables

```sql
CREATE TABLE answers (
  answer_id   bigint PRIMARY KEY,
  question_id bigint NOT NULL REFERENCES questions,
  author_id   bigint NOT NULL REFERENCES users,
  body        text NOT NULL,
  upvotes     int NOT NULL DEFAULT 0, downvotes int NOT NULL DEFAULT 0,   -- denormalized
  created_at  timestamptz NOT NULL, updated_at timestamptz,
  UNIQUE (question_id, author_id)                -- one answer per user per question
);
-- votes (answer_id, user_id, value CHECK (value IN (-1, 1)), PRIMARY KEY (answer_id, user_id))
CREATE TABLE comments (                          -- polymorphic target: a question OR an answer
  comment_id  bigint PRIMARY KEY,
  question_id bigint REFERENCES questions,
  answer_id   bigint REFERENCES answers,
  author_id   bigint NOT NULL REFERENCES users,
  body text NOT NULL, created_at timestamptz NOT NULL,
  CHECK (num_nonnulls(question_id, answer_id) = 1)
);
CREATE TABLE follows (                           -- polymorphic: follow a user, topic or question
  follower_id bigint REFERENCES users,
  target_type text NOT NULL CHECK (target_type IN ('user','topic','question')),
  target_id   bigint NOT NULL,                   -- no FK possible: it points at 3 tables
  PRIMARY KEY (follower_id, target_type, target_id)
);
```

These two tables show the two ways to model a **polymorphic association** (one row that may point to different parent tables). `comments` uses **one nullable FK per possible parent** plus a `CHECK` that exactly one is set — foreign keys still work. `follows` uses a **type + id pair**, which is compact and extensible but gives up referential integrity (the database cannot check that topic 42 exists). Prefer the first when the parent types are few and fixed.

Key queries

1. Answers to a question ranked by score: `ORDER BY upvotes - downvotes DESC, created_at` (or a stored `score` that also weighs author credibility).
2. Feed: new answers in followed topics/questions/people since the last visit.
3. Casting a vote: `INSERT … ON CONFLICT (answer_id, user_id) DO UPDATE SET value = EXCLUDED.value`, adjusting the counters by the difference.

Indexes

`answers (question_id, upvotes DESC)`, `question_topics (topic_id, question_id)`, `follows (target_type, target_id)` for "who follows this topic", full-text (`tsvector` GIN) or an external search engine for questions.

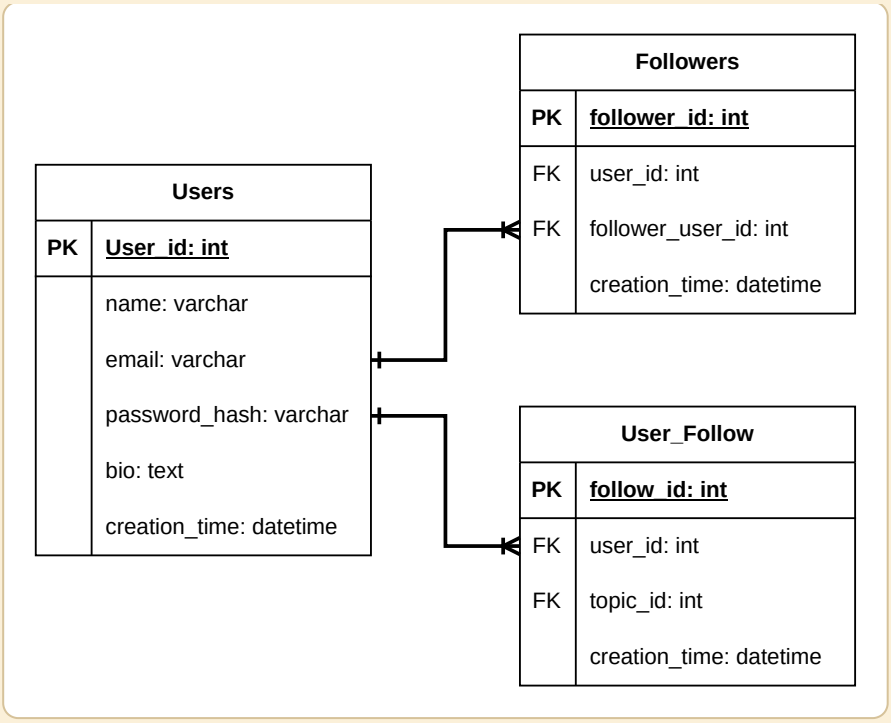


Figure — the course's split: users, follower and topic-follow tables in the relational DB; questions with embedded answers in a document store (course diagram)

The document alternative. The course stores each question as one MongoDB-style document with its answers (and their comments) **embedded**, so a question page is a single read; a mirrored user document carries follower counts and followed topics. Embedding is denormalization: great for "show this question", awkward for "all answers by this user" or for a question with 10,000 answers (MongoDB caps a document at 16 MB) — the usual compromise embeds the top answers and keeps the rest in their own collection.

Sharding. By `user_id`, a user's content is local but celebrity users become **hot spots** (shards overloaded by one popular key); by `question_id` (hashed), load spreads evenly but a user's timeline needs a **scatter-gather** (send the query to every shard and merge the answers) across shards. The recommended mix: question data sharded by a hash of `question_id`, user data (profiles, follows) by a hash of `user_id`, consistent hashing, and a shard-manager service that maps keys to shards.

Which store for which data

Data	Store
Users, questions, answers, votes	Sharded relational DB (Quora itself runs sharded MySQL — tens of terabytes and hundreds of thousands of queries per second, per its 2020 engineering blog) — or questions/answers in a document or wide-column store, as in the course design
Answer rankings, feed lists, counters	In-memory cache (Redis/Memcached)
Search and "similar question" suggestions	Search engine (Elasticsearch/OpenSearch)
Images in answers	Object storage + CDN

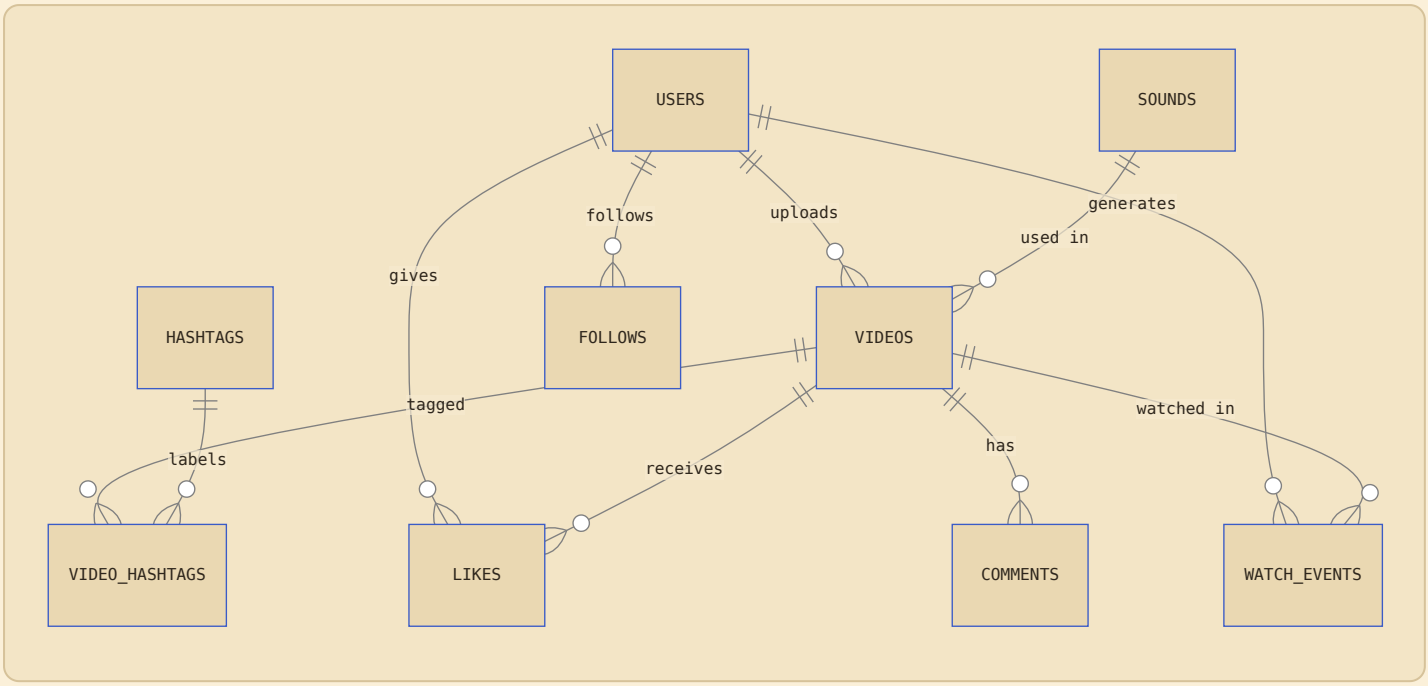
17.11 TikTok (short-video platform)

Requirements

- Users upload short **videos** with captions, **sounds** (music tracks reusable across videos) and **hashtags**.
- Viewers **like**, **comment**, **share**, and **follow** creators; view counts are shown on every video.

- The main surface is a recommendation-driven **"For You" feed**, fed by **watch events** (what was watched, for how long, skipped, replayed).
- Real-time **notifications** (new follower, like, comment); **trending** videos and hashtags; videos grouped into **categories** (dance, comedy).
- Scale: billions of video views per day; view and watch events vastly outnumber everything else.
- Estimate: with generous assumptions (1 B daily users each uploading 2 videos of 50 MB) raw video is 1 B × 2 × 50 MB = **100 PB a day**, against about 4 TB of metadata (2 B videos × 2 KB) and about 2 TB of comments (2 B × 1 KB). Video bytes outweigh the metadata and comments by more than 10,000 to 1, so they live in object storage with **lifecycle rules** (automatic policies that move old files to cheaper storage or delete them), never in a database.

Entities and relationships



Key tables

```
CREATE TABLE videos (
  video_id    bigint PRIMARY KEY,                -- time-ordered id (Snowflake-style)
  creator_id  bigint NOT NULL REFERENCES users,
  sound_id    bigint REFERENCES sounds,
  caption     text,
  storage_key text NOT NULL,                     -- original upload in object storage
  duration_ms int NOT NULL,
  status      text NOT NULL CHECK (status IN ('processing','public','private','removed')),
  created_at  timestamptz NOT NULL
);
-- video_renditions (video_id, resolution, bitrate_kbps, storage_key): transcoded copies
CREATE TABLE video_stats (                       -- hot counters kept apart from the video row
  video_id bigint PRIMARY KEY REFERENCES videos,
  views bigint NOT NULL DEFAULT 0, likes bigint NOT NULL DEFAULT 0,
  comments bigint NOT NULL DEFAULT 0, shares bigint NOT NULL DEFAULT 0
);
```

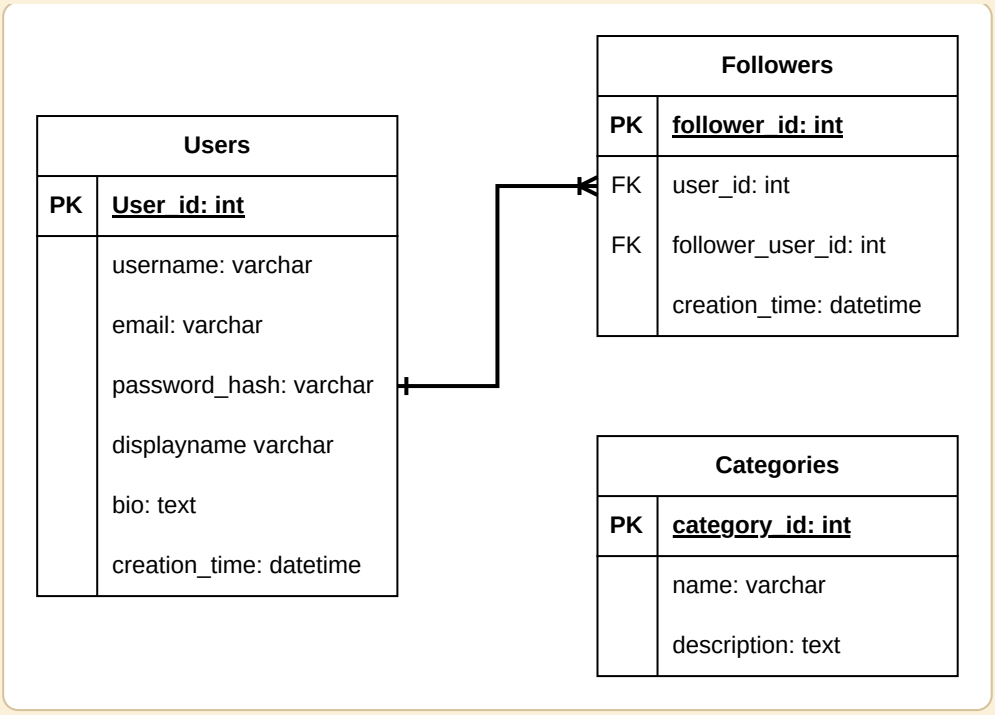


Figure — the course's TikTok schema: users, followers and categories relational; videos, interactions and user activity in Cassandra (course diagram)

In the course design the high-volume tables live in **Cassandra**, modeled around the query rather than the entity. A correct **CQL** (Cassandra Query Language) sketch (Cassandra has no `ENUM` type, and **counter** columns — special columns that can only be incremented or decremented — must sit in a table of their own; a `timeuuid` is a unique id that embeds a timestamp):

```
-- CQL (Cassandra) — not run
CREATE TABLE interactions_by_video (
  video_id uuid, created_at timeuuid, user_id bigint,
  kind text,                      -- 'like' | 'comment' | 'share'
  comment_text text, parent_id timeuuid,
  PRIMARY KEY ((video_id), created_at)          -- partition key, then clustering column
) WITH CLUSTERING ORDER BY (created_at DESC);
CREATE TABLE video_counters (
  video_id uuid PRIMARY KEY, views counter, likes counter, comments counter, shares counter
);
CREATE TABLE user_activity (
  user_id bigint, activity_time timestamp, activity_type text, details text,
  PRIMARY KEY ((user_id), activity_time)
) WITH CLUSTERING ORDER BY (activity_time DESC);
```

Relational tables keep what needs strong consistency (accounts, follows, categories); Cassandra keeps what needs write throughput and can be eventually consistent. The same idea appears relationally below.

(`video_renditions` holds **transcoded** copies — the same video re-encoded at several resolutions and **bitrates**, i.e. data per second, for different devices and connections.) Splitting `video_stats` from `videos` is a **vertical partition** (splitting a table's columns into two tables sharing a key): the counters change constantly, the video metadata almost never, so separating them keeps metadata rows and their cache entries stable. Watch events are **not** a table in the primary database at all — they are an **event stream** (an append-only, ordered flow of records, e.g. in Kafka, consumed by other systems).

Key queries

1. A creator's profile grid: `videos WHERE creator_id = $1 AND status = 'public' ORDER BY video_id DESC` (time-ordered ids sort by upload time).
2. Videos using a sound (the "use this sound" page).
3. Trending hashtags in the last hour — computed by the **stream processor** (a system that computes results continuously as events arrive), not by SQL over billions of rows.

4. For You feed — candidate generation and ranking by a recommendation service; the database only fetches metadata for the chosen ids.

Indexes

`videos (creator_id, video_id DESC)`, `videos (sound_id)`, `video_hashtags (hashtag_id, video_id)`, `likes (video_id, user_id)` primary key plus `likes (user_id, created_at)` for "videos I liked".

Which store for which data

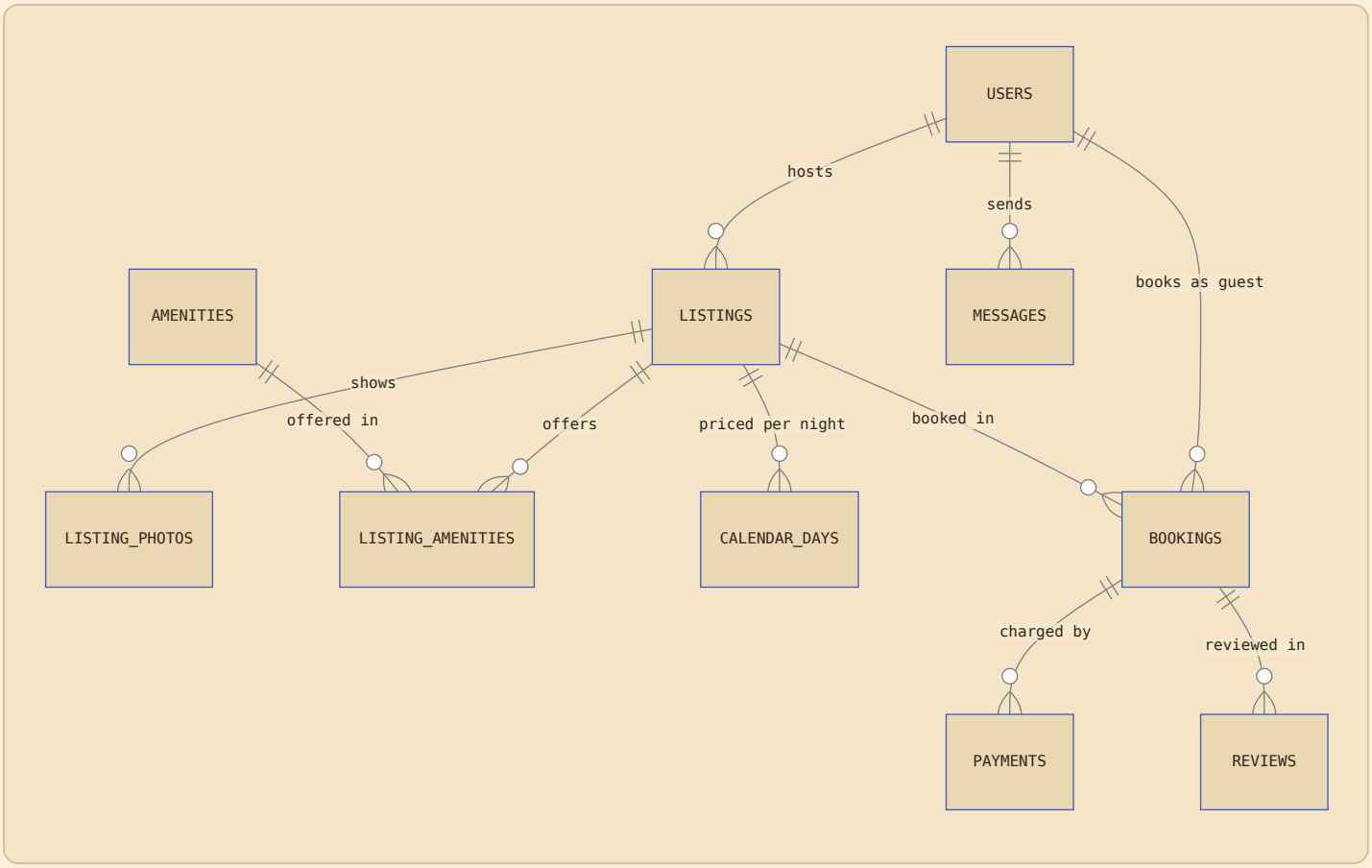
Data	Store
Users, video metadata, follows	Sharded relational or distributed SQL
Video files and renditions	Object storage + CDN (edge caches near viewers)
Views, watch time, skips	Event log (Kafka) → stream processing → analytics warehouse / wide-column store
Counters	In-memory counters, flushed periodically; displayed counts may lag a little
Recommendation features, embeddings (lists of numbers that represent an item so similar items are close)	Feature store (a database of precomputed model inputs) + vector index (finds nearest embeddings; FAISS library, Milvus) fed by training pipelines
Notifications, processing jobs	Message queue → notification and transcoding **workers** (background processes that take jobs from the queue)

17.12 Airbnb (short-term rentals)

Requirements

- **Hosts** publish **listings** (address, geo-location, capacity, amenities, photos, house rules) with a **calendar** of availability and nightly prices.
- **Guests** search by location, dates and guests; filter by price and amenities; then **book** (instant or on host approval).
- No listing may be booked twice for overlapping nights.
- **Payments** (charge the guest, pay out the host later, refunds and deposits through an external **payment gateway** — a third-party service that processes card payments), **reviews** in both directions after a stay, and **messaging** between guest and host. Hosts accept or decline booking requests.
- Estimate (per day: 1 M listings created or updated with ~5 KB of data and 10 photos of ~1 MB; 10 M reservations and 10 M payments at 1 KB; 5 M reviews at 0.5 KB): photos ≈ **10 TB/day**, while reservations, payments and reviews together are only ≈ 22 GB/day — the transactional core is small enough for a relational cluster.

Entities and relationships



A user is both a potential **host** and a **guest**: one `users` table with two roles, not two tables.

Key tables

```
-- needs the PostGIS extension for geography (not run here)
CREATE TABLE listings (
  listing_id bigint PRIMARY KEY,
  host_id    bigint NOT NULL REFERENCES users,
  title      text NOT NULL,
  location   geography(Point, 4326) NOT NULL,     -- PostGIS; lat/long for radius search
  max_guests smallint NOT NULL,
  base_price numeric(10,2) NOT NULL,
  status     text NOT NULL CHECK (status IN ('draft','active','snoozed','deleted'))
);
CREATE TABLE calendar_days (                      -- per-night availability and price overrides
  listing_id bigint REFERENCES listings,
  night      date,
  price      numeric(10,2),                       -- NULL = base price
  blocked    boolean NOT NULL DEFAULT false,      -- host blocked it
  PRIMARY KEY (listing_id, night)
);
CREATE TABLE bookings (
  booking_id bigint PRIMARY KEY,
  listing_id bigint NOT NULL REFERENCES listings,
  guest_id   bigint NOT NULL REFERENCES users,
  stay       daterange NOT NULL,
  guests     smallint NOT NULL,
  total      numeric(12,2) NOT NULL,
  status     text NOT NULL CHECK (status IN ('requested','confirmed','cancelled','completed')),
  EXCLUDE USING gist (listing_id WITH =, stay WITH &&) WHERE (status IN ('requested','confirmed'))
);
-- reviews (booking_id, author_role 'guest' | 'host', rating, body,
--          PRIMARY KEY (booking_id, author_role))  -> each side reviews once per stay
```

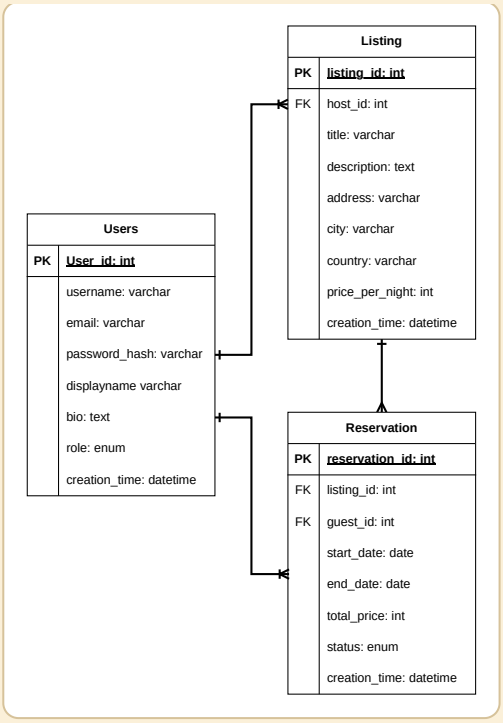


Figure — the course's Airbnb schema: users (with a host/guest role), listings and reservations relational; reviews, user activity and property images in Cassandra (course diagram)

The course splits the work into **user, listing, booking and payment services**. Users, listings, reservations and payments stay relational (atomic booking and money); reviews, activity logs and `property_images (listing_id, image_id, url)` — partitioned by `listing_id` so one read returns a listing's photos — go to Cassandra; photo bytes go to object storage. Its `users.role` column (host or guest) works only if a person is never both; a second listing-owner check (`listings.host_id`) is the more flexible way to express "host".

The **exclusion constraint** on `(listing_id, stay)` is the same tool as the hospital appointments: two active bookings of one listing cannot overlap. Unlike a hotel, each listing is a single unit, so no inventory counter is needed.

Key queries

1. Search: active listings within 10 km (`ST_DWithin(location, $point, 10000)`) with enough capacity and **no** overlapping active booking and no blocked night in the range — plus price filtering.
2. Book: insert the booking; the exclusion constraint rejects a conflicting stay, even under concurrency.
3. Host dashboard: upcoming bookings per listing; payouts due.
4. Listing rating: `avg(rating)` of guest reviews, usually cached on the listing.

Indexes

GiST on `listings.location`; the exclusion constraint's GiST index on `(listing_id, stay)`; `bookings (guest_id, lower(stay))` for "my trips"; `calendar_days` PK.

Which store for which data

Data	Store
Users, listings, bookings, payments	Relational (strong consistency for bookings and money)
Search with geo + filters + ranking	Search engine fed by change-data-capture from the DB
Photos	Object storage + CDN
Messages	Relational or wide-column, partitioned by conversation
Pricing suggestions, analytics	Data warehouse (replicated nodes plus regular backups)

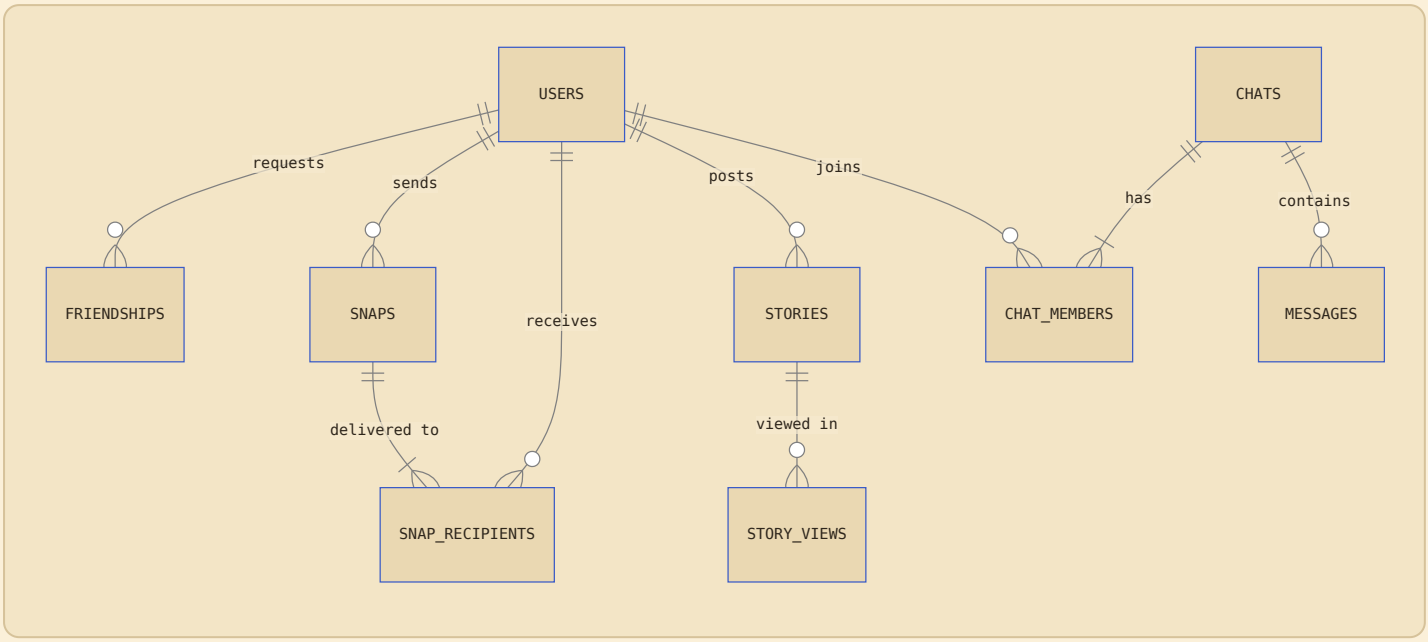
Replication follows the pattern above: a primary with read replicas in several regions and automatic failover for the relational core, multi-node replication with eventual consistency for Cassandra.

17.13 Snapchat (ephemeral messaging)

Requirements

- Users add **friends** (mutual, request/accept), send **snaps** (photo/video) to one or more friends, and **chat**.
- A snap **disappears** after every recipient has opened it, or after a maximum time (e.g. 30 days unopened); a **story** is visible to friends for 24 hours.
- Senders see delivery and "opened" status per recipient; screenshots are reported.
- Snaps and stories can carry **filters/lenses**; users can be **searched** by name.
- Scale: billions of snaps a day; almost all data is short-lived.
- Estimate (500 M daily users, 10 snaps each — 80 % photos of 1 MB, 20 % videos of 5 MB — plus one 10 MB story each): 4 PB + 5 PB of snaps and 5 PB of stories, about **14 PB of new media a day**. Because snaps vanish on viewing and stories after 24 hours, what is *held* stays bounded — deletion is as important a workload as writing.

Entities and relationships



Key tables

```
CREATE TABLE friendships (                        -- one row per pair, stored in canonical order
  user_low   bigint REFERENCES users,
  user_high  bigint REFERENCES users,
  status     text NOT NULL CHECK (status IN ('pending','accepted','blocked')),
  requested_by bigint NOT NULL,
  PRIMARY KEY (user_low, user_high),
  CHECK (user_low < user_high)
);
CREATE TABLE snaps (
  snap_id    bigint PRIMARY KEY,
  sender_id  bigint NOT NULL REFERENCES users,
  media_key  text NOT NULL,                       -- encrypted blob in object storage
  created_at timestamptz NOT NULL,
  expires_at timestamptz NOT NULL                 -- hard deadline even if never opened
);
CREATE TABLE snap_recipients (
  snap_id      bigint REFERENCES snaps ON DELETE CASCADE,
  recipient_id bigint REFERENCES users,
  delivered_at timestamptz, opened_at timestamptz, screenshot_at timestamptz,
  PRIMARY KEY (recipient_id, snap_id)             -- "my inbox" is a PK range scan
);
-- stories (story_id, user_id, media_key, created_at, expires_at = created_at + 24 h, set on insert)
```

(`expires_at` cannot be a `GENERATED … STORED` column: `timestamptz + interval` depends on the session time zone, so PostgreSQL rejects it as not **immutable** — a generated column's expression must always give the same output for the same input.)

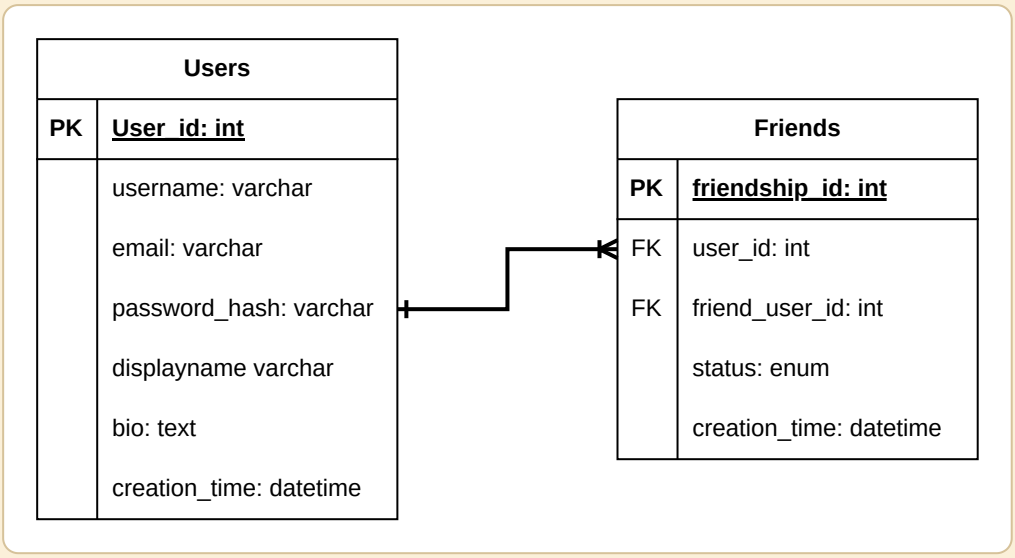


Figure — the course's Snapchat schema: users and friends relational; snaps, stories and messages in Cassandra; users, snaps and stories indexed in Elasticsearch for search (course diagram)

The course's Cassandra `snaps` table is keyed by `snap_id` with a secondary index on the receiver. Cassandra works best when the **partition key is what you query by**, so an inbox table partitioned by recipient (as `snap_recipients` above does relationally) avoids the secondary index. A search engine (Elasticsearch) holds a user index for "find friends", sharded automatically by the number of primary shards you configure.

The **canonical pair** trick (`user_low < user_high`) stores a symmetric relationship once, so "are A and B friends?" is one lookup with `(least(a,b), greatest(a,b))`. Compare Instagram's `follows`, which is directional and therefore needs one row per direction.

Key queries

1. Inbox: unopened snaps for a recipient, newest first.
2. Open a snap: set `opened_at`; when every recipient row has `opened_at`, delete the media and the snap.
3. Friends' stories: stories of accepted friends where `expires_at > now()`.
4. Expiry sweep: delete snaps and stories past `expires_at` (in batches).

Indexes

`snap_recipients` PK `(recipient_id, snap_id)`, `snaps (expires_at)` for the sweeper, `stories (user_id, created_at DESC)`, `friendships (user_high)` for the reverse lookup.

Scaling notes and stores

Ephemeral (short-lived) data is a poor fit for row-by-row `DELETE` at this volume (**dead tuples** — deleted row versions still occupying space — **vacuum**, Postgres's clean-up process for them, and **index bloat**, indexes swollen with entries for dead rows). Two better tools: **time partitions** (partition `snaps` by day and `DROP` whole partitions — instant and bloat-free, Chapter 15 §15.2) or a store with native **TTL** (time-to-live), such as Cassandra/ScyllaDB or DynamoDB, where rows expire by themselves. Snap's 2017 **IPO** (initial public offering — first sale of shares on the stock market) filing disclosed five-year cloud commitments of $2 billion with Google Cloud and $1 billion with AWS — at this scale, managed cloud databases are the usual choice rather than self-run servers.

Data	Store
Accounts, friend graph	Relational or key-value, sharded by user
Snaps, recipients, chat messages	Wide-column / key-value with TTL, partitioned by recipient or chat
Media	Encrypted object storage + CDN, lifecycle rules delete it
Presence ("online now" status), typing indicators	In-memory store
User search	Search engine index

Partitioning happens at three levels: **horizontal** (rows split by `user_id` or `snap_id` — users by range or consistent hashing, trying to keep friend lookups on one shard; Cassandra hashes its partition keys itself), **vertical** (user data and media data in separate stores), and **functional** (each microservice — user, media, messaging, search — owns its data).

17.14 Recurring modeling patterns

Across the thirteen cases the same few patterns did most of the work. This table is the chapter in one page.

Pattern	What it is	Seen in	Use it when	Watch out for
Junction table	A table whose PK is two FKs, resolving an M:N relationship, often with its own attributes	`order_items`, `likes`, `title_authors`, `post_hashtags`, `reservation_rooms`	Any M:N relationship; the PK doubles as the uniqueness rule ("like once")	Add an index on the *second* column for the reverse lookup
Type vs instance	Separate the kind of thing from the physical thing	`titles` / `copies`, `room_types` / `rooms`, `seats` / `show_seats`	Users choose a kind but you track individual items	Forgetting it forces one table to do two jobs
Status enum + state history	A constrained `status` column for the current state; optionally an append-only table of transitions	orders, bookings, reservations, food-delivery events	Every entity with a life cycle	Enforce allowed values (`ENUM`, `CHECK`, or lookup table); `CHECK`s can tie a status to required columns (ticket seats)
Soft delete	A `deleted_at` / `withdrawn_at` column instead of `DELETE`	products, library copies, listings	History or FKs must survive removal	Every query must filter it — use partial indexes and views; unique constraints need `WHERE deleted_at IS NULL`; privacy laws may still require a hard delete
Audit columns / ledger	`created_at`, `updated_at`, `created_by`; or a full append-only ledger of changes	hospital records, bank ledger, stock movements	Legal, financial or medical data; debugging "who changed this?"	Maintain with defaults/triggers, not by hand; ledgers need a cached balance for speed and a reconciliation query
Polymorphic association	One row points to one of several parent types	Quora comments (nullable FKs + `CHECK`), follows (`target_type` + `target_id`)	A comment/like/follow can target several entity kinds	The type+id form loses foreign keys; prefer one nullable FK per parent, or a shared supertype table
Denormalized counter / copy	Store a derived value (count, total, current status) to avoid recomputing	`like_count`, `orders.total`, `room_inventory`, `answers.upvotes`, `video_stats`	A measured hot read	Update it in the same transaction (or trigger), add a reconciliation check, and beware lock contention on hot rows
Snapshot of volatile data	Copy a value that may later change	`unit_price`, delivery address, `nightly_price`	The fact is "what it was at the time"	It is not a normalization bug — document it as intentional
Self-reference	A FK to the same table	category tree, comment replies, `follows`	Hierarchies and graphs	Query with recursive CTEs; guard against cycles
Overlap prevention	Exclusion constraint on a range, or partial unique index on "open" rows	appointments, Airbnb stays, hotel rooms, open library loans	"No two X at the same time"	PostgreSQL-specific; elsewhere lock and check in a transaction
Guarded update / claim	`UPDATE … WHERE state = 'free' RETURNING …`	seat holds, stock decrement	Many users compete for the same item	Check the returned row count; hold short; `SKIP LOCKED` for "any available"

Pattern	What it is	Seen in	Use it when	Watch out for
Time partitioning / TTL	Split big tables by time; drop old partitions or let rows expire	ledgers, movements, snaps, stories	Append-heavy data with a retention rule	Queries should include the partition key in `WHERE`

Judgement calls — cover the answer, say it aloud

1. *`order_items.unit_price` duplicates `products.price` — a 3NF violation?* → No: price paid on that order and today's list price are different facts.
2. *Why no stored `is_available` flag on copies or seats?* → Availability is derived from loans / seat status; a second copy of the truth drifts.
3. *A foreign key that should be 1:1 — what else does it need?* → `UNIQUE` (or make it the primary key); otherwise it is 1:N.
4. *Where does the agreed rate, a seat's price, or a medicine's dose go?* → On the junction row: it describes the pairing, not either side.
5. *Hotel search counts per night; Airbnb uses an exclusion constraint. Why?* → A room type has many interchangeable units; a listing is one unit.
6. *When is `target_type + target_id` acceptable?* → When target kinds change often and you accept checking integrity outside the database.
7. *The one-sentence recipe for "no double booking"?* → Make the conflicting state impossible to store (unique / exclusion / check constraint) and claim the item with one guarded statement in a short transaction.

Chapter 17 TL;DR

- Use one template every time: **requirements & questions → entities, cardinality & participation → ER diagram → tables & constraints → key queries → indexes → design review → scaling** (the review checks normal forms, names deliberate redundancy, and lists what the design cannot do).
- Put business rules **in the schema** where possible: `CHECK` for ranges and state/column consistency, partial unique indexes for "one open X", exclusion constraints for "no overlapping X", unique keys for "once per user".
- Separate **type from instance** (titles/copies, room types/rooms, seats/show seats) and **current state from history** (status column + events or ledger).
- **Snapshot** values that must not change later (prices paid, addresses shipped to); **soft-delete** rows that history still references.
- Denormalize **deliberately** (counters, totals, inventory per night) — update in the same transaction and keep a reconciliation query.
- Concurrency for scarce items: a **guarded** `UPDATE … WHERE status = 'free'` in a short transaction is enough — demonstrated on two live sessions; `SKIP LOCKED` for "any available".
- At scale, the relational core stays for money, bookings and identity; media goes to object storage + CDN, feeds and counters to caches, events to logs, search to a search engine, and short-lived data to TTL stores or dropped time partitions.

Next: Chapter 18 — SQL interview problems, part 1, which practises the query skills from Chapters 2–8 on interview-style problems.

Chapter 18 — SQL Interview Problems I: Method and 42 Warm-Up to Medium Problems

Chapters 2–8 taught the tools one at a time. Interviews mix them. You get a short business question and a schema, and you have to decide what to filter, what to group, what to join and what could go wrong, then say it all out loud while you type. This chapter covers the method first and then works through 42 practice problems (easy up to lower-medium). They are grouped by the **pattern** they train, so you learn to spot the pattern rather than memorise answers.

What you'll learn

#	Section	Patterns
18.1	How to approach a SQL interview question	clarify → sample → build → edge cases → verify
18.2	Filtering (P1–P8)	`WHERE`, `AND` / `OR`, `%`, `IS DISTINCT FROM`, `char_length`, regex, row-level `CASE`
18.3	Aggregation (P9–P16)	`GROUP BY`, `HAVING`, `MIN` / `MAX` / `COUNT`, `string_agg`
18.4	Joins (P17–P22)	inner, self, cross + left, "top 1 with ties"
18.5	Anti-joins (P23–P26)	`NOT EXISTS`, `LEFT JOIN … IS NULL`, the `NOT IN` + NULL trap
18.6	Conditional aggregation (P27–P28)	`FILTER`, `SUM(CASE …)`, `UNION ALL` then pivot
18.7	Date logic (P29–P33)	half-open month ranges, date arithmetic, gaps and islands (finding runs of consecutive values), "value as of a date"
18.8	Ratios and percentages (P34–P38)	integer division, `NULLIF`, weighted averages, retention (share of users who come back)
18.9	Window and ranking basics (P39–P42)	second highest, first row per group, running total, compare to group average
18.10	Pattern table	every problem → pattern → key SQL feature

The harder problems (Nth highest, top-N per group, streaks, pivots, trips and users) are in Chapter 19.

How to run the examples. Every problem has its own file in `code/`. Each file creates its own small tables, fills them with invented data, and runs the solution plus an edge case or alternative. The runner gives each file a fresh PostgreSQL 14 database. The output under each query is real `psql` (PostgreSQL's command-line client) output, and `(null)` is how we display `NULL` (SQL's marker for a missing or unknown value).

```
bash tools_run_sql.sh sql '18-*'                    # all 42 files, fresh DB each
bash tools_pg.sh psql -d postgres -c 'CREATE DATABASE scratch'
bash tools_pg.sh psql -d scratch -f sql/chapters/18-sql-interview-problems-1/code/26_reps_without_red.sql
```

Every problem statement, dataset and explanation below is written for this book. The problems are classic interview-style exercises; many also appear on public practice sites under similar names.

18.1 How to approach a SQL interview question

What it is

A repeatable routine for turning a vague question ("which sellers were idle last year?") into a correct query while the interviewer watches. Think of it as a pilot's checklist: the steps are dull, but they catch the mistakes that nerves cause.

Why it matters

Most wrong answers in SQL interviews come from **misreading the question** or **missing an edge case** (an unusual input at the boundary of what the query must handle, such as an empty table or a NULL), not from syntax: counting rows instead of distinct users, losing people with zero activity, integer division turning 0.67 into 0, `NOT IN` silently returning nothing. Interviewers also score your *process*: do you ask questions, test your idea on a few rows, and explain your trade-offs?

How it works — the five steps



1. **Clarify.** Repeat the question in your own words, then ask:
   - What is one **row** of each table (the *grain*)? What is the **primary key** (the column or columns that uniquely identify a row)? Can there be **duplicate rows**?
   - Which columns can be **NULL**? What should happen to rows with NULLs?
   - Should entities with **no activity** appear (with 0) or be left out?
   - How should **ties** be handled: return all of them, or any one?
   - Output: which columns, what names, what **order**, how much **rounding**?
   - Are dates **inclusive**? Which time zone? Is "consecutive" by calendar day or by record?
2. **Sketch sample data.** Write 3–5 rows, including one awkward row (a NULL, a duplicate, a tie), plus the output you expect. This settles misunderstandings before you write code.
3. **Build in the logical order.** Write the query in the order the database evaluates it (see Chapter 4, logical query order): `FROM` / `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`. Use **CTEs** (common table expressions: named sub-queries written as `WITH step1 AS (…)`) so each step can be checked on its own (Chapter 7).
4. **Attack it with edge cases**, using the checklist below.
5. **Verify and discuss.** Run the query against your sample in your head (or for real). Then mention what **index** (a sorted lookup structure that lets the database find rows without reading the whole table) would help and give one alternative formulation.

The edge-case checklist

Risk	Typical symptom	Usual fix
NULL in a comparison	rows vanish from `<>`, `NOT IN` returns nothing	`IS DISTINCT FROM`, `NOT EXISTS`, `COALESCE` (returns its first non-NULL argument)
Entities with no rows	people with zero orders missing	`LEFT JOIN` from the entity table, `count(col)` not `count(*)`
Duplicates	counts too high, repeated ids	`DISTINCT`, `count(DISTINCT …)`, check the join's grain
Join fan-out (one row matching many, so rows multiply)	sums doubled after joining two child tables	aggregate before joining, or `count(DISTINCT)`
Ties	"the winner" is really two people	`RANK` / `DENSE_RANK` (ranking functions; ties share a rank), `FETCH FIRST … WITH TIES`
Integer division	`1/3 = 0`	multiply by `100.0` or cast to `numeric` first
Division by zero	runtime error	`NULLIF(denominator, 0)`
Empty result vs NULL	"return NULL if none" gives zero rows	wrap in an aggregate or a scalar subquery (a sub-query used as a single value)
Date boundaries	last day of the month (or 29 Feb) lost	half-open ranges: `>= start AND < next_start`

Common mistakes

- Starting to type before knowing the grain of each table.
- `SELECT`ing a column that is neither grouped nor aggregated (PostgreSQL rejects it; MySQL with the `ONLY_FULL_GROUP_BY` setting off returns an arbitrary value).
- Filtering the right-hand table of a `LEFT JOIN` in `WHERE`, which turns it back into an inner join (Chapter 5).
- Leaving `ORDER BY` out when the question asks for an order. Without it, SQL guarantees **no** order.

In the interview

30-second answer to "how do you approach a SQL problem?" "I first pin down the grain and keys of each table, what NULLs and duplicates mean, and whether zero-activity rows and ties should appear. I sketch a few rows with the expected output, then build the query in evaluation order (joins, filters, grouping, having, select, sort) using CTEs so I can check each step. Finally I test NULLs, duplicates, ties, empty groups and date boundaries, and mention an index or an alternative."

- *"Your query returns nothing for a new customer — is that right?"* It depends on the spec. If they should appear with 0, start from the customer table with a `LEFT JOIN` and `COALESCE` the aggregate.
- *"Would this be fast on 100 M rows?"* Talk about the filter/join columns and a matching index, **sargable** predicates (conditions written so an index can be used — "Search ARGument ABLE"; e.g. no function wrapped around the indexed column), and **pre-aggregation** (summarising rows before a join or storing summaries ahead of time). See Chapter 12.

Self-check

1. Why ask about the grain first? → It tells you what one row means, so you know whether to count rows or distinct ids, and whether a join can multiply rows.
2. What's wrong with `WHERE year(d) = 2020`? → It works, but wrapping the column in a function prevents an ordinary index on `d` from being used. Use `d >= '2020-01-01' AND d < '2021-01-01'`.
3. `count(*)` vs `count(col)` after a `LEFT JOIN`? → `count(*)` counts the NULL-padded row as 1; `count(col)` counts it as 0.

TL;DR

Clarify (grain, NULLs, zeros, ties, output), sketch sample rows, build in evaluation order with CTEs, attack with the edge-case checklist, then verify and discuss performance.

18.2 Filtering problems (P1–P8)

The simplest pattern: keep or drop **individual rows** with `WHERE`. The traps are `NULL` (**three-valued logic**: a condition can be `TRUE`, `FALSE` or `UNKNOWN`, and comparisons with NULL give `UNKNOWN`), boundaries (`>` vs `>=`) and string rules. Background: Chapter 2 and Chapter 3.

P1 · Big cities — `OR` filter

Problem. A `city` table stores name, country, area in km² and population. List the cities that are *either* larger than 3,000 km² *or* home to more than 5 million people.

Data (`01_big_cities.sql`): six invented towns. Coralbay sits exactly on both limits (3000 km², 5,000,000). Eastvale and Fernhill have an unknown (NULL) area.

```sql
SELECT name FROM city
WHERE area_km2 > 3000 OR population > 5000000
ORDER BY name;
```

Output:

```
   name
-----------
 Aldport
 Brimstead
 Eastvale
(3 rows)
```

Why it works. Each row is tested on its own. Coralbay fails because "more than" is strict. Eastvale's area test gives `UNKNOWN`, but `UNKNOWN OR TRUE` is `TRUE`, so its population is enough. Fernhill gives `UNKNOWN OR FALSE` = `UNKNOWN`, and `WHERE` keeps only `TRUE`.

Alternative. `SELECT … WHERE area_km2 > 3000 UNION SELECT … WHERE population > 5000000` returns the same set. On some engines (notably older MySQL) this lets each branch use its own index. PostgreSQL can instead combine two indexes with a *BitmapOr* (a plan step that merges the row lists found by two index scans) for the plain `OR`.

P2 · Odd-numbered, not-boring movies — modulo and `<>`

Problem. From `movie(id, title, description, rating)`, return movies whose id is odd and whose description isn't `'boring'`, highest rating first.

Data (`02_odd_movies.sql`): ids 1–5 and 7, one of them described as boring, one with a NULL description, plus id −3.

```sql
SELECT * FROM movie
WHERE id % 2 = 1 AND description <> 'boring'
ORDER BY rating DESC;
```

Output:

```
 id |    title    | description | rating
----+-------------+-------------+--------
  5 | Deep Blue   | documentary |   8.90
  1 | Night Train | thriller    |   8.10
(2 rows)
```

Why it works. `%` is the remainder. Odd ids leave remainder 1. The `<>` drops the boring one.

Edge cases. Two rows went missing silently. In PostgreSQL, `-3 % 2` is `-1` (the result takes the sign of the **dividend**, the number being divided), so `= 1` misses negative odd ids. And `NULL <> 'boring'` is `UNKNOWN`. If an unknown description should count as "not boring", say so explicitly:

```sql
SELECT * FROM movie
WHERE id % 2 <> 0 AND description IS DISTINCT FROM 'boring'
ORDER BY rating DESC;
```

```
 id |    title    | description | rating
----+-------------+-------------+--------
  5 | Deep Blue   | documentary |   8.90
  1 | Night Train | thriller    |   8.10
 -3 | Rewind      | comedy      |   6.00
  7 | Untitled    | (null)      |   5.00
(4 rows)
```

MySQL has `MOD(id, 2)` as well as `%`. Its **NULL-safe** equality (one that treats two NULLs as equal and never returns `UNKNOWN`) is `<=>`, so "is distinct from" is written `NOT (a <=> b)`.

P3 · Green products — `AND` on flag columns

Problem. `inventory(product_id, organic, compostable)` holds `'Y'`/`'N'` flags. Return the products that have both flags set.

```sql
SELECT product_id FROM inventory
WHERE organic = 'Y' AND compostable = 'Y'
ORDER BY product_id;
```

```
 product_id
------------
         10
         13
(2 rows)
```

Why it works. Both conditions must be `TRUE` for the same row.

Edge case / design note. Flags stored as text invite dirty values such as `'y'` or `' Y'`. The file puts a `CHECK (organic IN ('Y','N'))` on each column, and inserting `'y'` fails with *violates check constraint "inventory_organic_check"*. In PostgreSQL a `boolean NOT NULL` column is cleaner still: `WHERE organic AND compostable`. MySQL's source table often uses `ENUM('Y','N')`.

P4 · Everyone outside department 3 — NULL-safe "not equal"

Problem. From `staff(id, name, dept_id)`, list people who are not in department 3. One person has no department yet.

```sql
SELECT name FROM staff WHERE dept_id <> 3 ORDER BY name;              -- naive
```

```
 name
------
 Ana
 Chen
(2 rows)
```

```sql
SELECT name FROM staff WHERE dept_id IS DISTINCT FROM 3 ORDER BY name; -- NULL-safe
```

```
 name
------
 Ana
 Chen
 Dara
(3 rows)
```

Why it matters. `NULL <> 3` is `UNKNOWN`, so Dara disappears from the naive version. Whether Dara *should* appear is a clarifying question. Most people would say "not in department 3" includes "in no department". `IS DISTINCT FROM` treats NULL as an ordinary comparable value. The portable spelling is `dept_id <> 3 OR dept_id IS NULL`.

P5 · Self-assigned tasks — comparing two columns, removing duplicates

Problem. `task_log(task_id, assigner_id, assignee_id, done_on)` may contain duplicate rows. Return, in ascending order, each person who completed at least one task they had given to themselves.

```sql
SELECT DISTINCT assigner_id AS employee_id
FROM task_log
WHERE assigner_id = assignee_id
ORDER BY employee_id;
```

```
 employee_id
-------------
           7
           9
(2 rows)
```

Why it works. The `WHERE` compares two columns of the same row. `DISTINCT` collapses repeats: without it the output is `7, 7, 9, 9` (one duplicate row for person 7, and two self-assigned tasks for person 9). `GROUP BY assigner_id` gives the same result. "At least one" nearly always means you need `DISTINCT`, `GROUP BY` or `EXISTS`.

P6 · Long comments — length in characters

Problem. `comment(id, body)`: return ids of comments longer than 100 characters (spaces and punctuation count).

```
SELECT id FROM comment WHERE char_length(body) > 100 ORDER BY id;
```

```
 id
----
  2
(1 row)
```

Why it works / edge cases. Comment 3 is exactly 100 characters, so it is not "more than" 100. Comment 4 is 60 copies of `é`: 60 **characters** but 120 **bytes** in **UTF-8** (the standard text encoding, which stores accented and non-Latin characters in 2–4 bytes each).

```
SELECT id, char_length(body) AS chars, octet_length(body) AS bytes FROM comment ORDER BY id;
```

```
 id | chars | bytes
----+-------+-------
  1 |    10 |    10
  2 |   102 |   102
  3 |   100 |   100
  4 |    60 |   120
(4 rows)
```

Engine	Characters	Bytes
PostgreSQL	`char_length(s)`, `length(s)`	`octet_length(s)`
MySQL	`CHAR_LENGTH(s)`	`LENGTH(s)` (a classic trap)
SQL Server	`LEN(s)` (ignores trailing spaces)	`DATALENGTH(s)`
SQLite	`length(s)`	`length(CAST(s AS BLOB))`

P7 · Valid company e-mails — regular expressions

Problem. `employee(id, name, email)`. An address is valid when the part before `@` starts with a letter and continues with letters, digits, `_`, `.` or `-`, and the domain is exactly `northwind.dev`. Return the valid rows.

Data (`07_valid_emails.sql`): eight addresses. They include one starting with `_`, one starting with a digit, `northwindXdev`, `northwind.dev.evil.com`, an upper-case domain and one containing `#`.

```
SELECT id, name, email FROM employee
WHERE email ~ '^[A-Za-z][A-Za-z0-9_.-]*@northwind\.dev$'
ORDER BY id;
```

```
 id | name |        email
----+------+---------------------
  1 | Ana  | ana.k@northwind.dev
  5 | Eve  | Eve-2@northwind.dev
(2 rows)
```

Why it works. A **regular expression** (regex) is a mini-language for describing text patterns, and `~` tests whether a string matches one. `^` and `$` **anchor** the pattern to the start and end, so it must match the whole string. `[A-Za-z]` forces a letter first, and the class after it allows the other characters (inside `[...]` a `.` is literal, and a `-` placed last is literal). `\.` is a literal dot.

Edge cases. Remove the anchors and the backslash, and six rows match. That includes `dee@northwindXdev`, because an unescaped `.` matches any character, and `fay@northwind.dev.evil.com`, because the end is not anchored. `~` is case-sensitive, so `gus@NORTHWIND.DEV` fails. Use `~*` if the domain should be case-insensitive (it then matches ids 1, 5, 7). Dialects: MySQL `REGEXP` / `REGEXP_LIKE()`, Oracle

`REGEXP_LIKE()`. SQL Server gained `REGEXP_LIKE()` only in SQL Server 2025 (database compatibility level 170) and Azure SQL; older versions fall back to `LIKE` patterns. In MySQL string literals the backslash itself must be doubled (`'\\.'`).

P8 · Triangle test — a `CASE` per row

Problem. `segment(a, b, c)` holds three lengths per row. Add a column saying `'Yes'` if they can form a triangle and `'No'` otherwise.

```
SELECT a, b, c,
       CASE WHEN a + b > c AND a + c > b AND b + c > a THEN 'Yes' ELSE 'No' END AS triangle
FROM segment;
```

```
 a  |   b    | c | triangle
----+--------+---+----------
  3 |      4 | 5 | Yes
 10 |      2 | 3 | No
  5 |      5 | 5 | Yes
  1 |      2 | 3 | No
  7 | (null) | 4 | No
(5 rows)
```

Why it works. The **triangle inequality** says every pair of sides must be longer than the third side. `(1,2,3)` is *degenerate*: it is flat, because 1 + 2 is not *greater* than 3.

Alternative / edge case. It is enough to check the longest side: `2 * greatest(a,b,c) < a + b + c`. A missing side makes the condition `UNKNOWN`, and `CASE` falls through to `ELSE 'No'`, which quietly claims something we don't know. Adding `WHEN a IS NULL OR b IS NULL OR c IS NULL THEN 'Unknown'` makes that row say `Unknown` (see the file). PostgreSQL's `greatest()` ignores NULLs, whereas MySQL's returns NULL if any argument is NULL.

18.3 Aggregation problems (P9–P16)

Collapse many rows into one per group. The traps: which rows exist before grouping, how aggregates treat NULLs, and what an **empty** input returns. Background: Chapter 4.

P9 · Largest number that appears once — `HAVING` then `MAX`

Problem. `nums(n)` has no key and contains repeats. Return the biggest value that occurs exactly once, or NULL if no value is unique.

```
SELECT max(n) AS num
FROM (SELECT n FROM nums GROUP BY n HAVING count(*) = 1) AS singles;
```

```
 num
-----
   5
(1 row)
```

Why it works. The inner query groups equal values and keeps groups of size 1: {1, 4, 5}. The outer `max` picks 5.

Edge case — empty input. After inserting 1, 4, 5 again, no value is unique. An aggregate over zero rows still returns **one row** (`max` → NULL), which is exactly what "report NULL" asks for:

```
  num
--------
 (null)
(1 row)
```

The tempting `… HAVING count(*) = 1 ORDER BY n DESC LIMIT 1` returns **0 rows** in the same situation. That is a different answer, and graders that expect a NULL row mark it wrong.

P10 · Units sold per product — `SUM` with `GROUP BY`

Problem. `sale(sale_id, product_id, yr, qty, unit_price)` references `product`. Report total quantity per product.

```
SELECT product_id, sum(qty) AS total_quantity
FROM sale
GROUP BY product_id
ORDER BY product_id;
```

```
 product_id | total_quantity
------------+----------------
        100 |             25
        200 |             15
(2 rows)
```

Edge case — zero sellers. Product 300 never sold, so it has no group. If the business wants it listed, start from `product`:

```
SELECT p.product_id, COALESCE(sum(s.qty), 0) AS total_quantity
FROM product p LEFT JOIN sale s USING (product_id)
GROUP BY p.product_id ORDER BY p.product_id;
```

```
 product_id | total_quantity
------------+----------------
        100 |             25
        200 |             15
        300 |              0
(3 rows)
```

`sum` over only NULLs is NULL, not 0, which is why the `COALESCE` is needed.

P11 · First login per player — `MIN` on dates

Problem. `activity(player_id, device_id, played_on, games)` has one row per player per day. Give each player's first login date.

```
SELECT player_id, min(played_on) AS first_login
FROM activity GROUP BY player_id ORDER BY player_id;
```

```
 player_id | first_login
-----------+-------------
         1 | 2024-02-27
         2 | 2024-06-25
         3 | 2024-03-02
(3 rows)
```

Why it works. `min` works on anything sortable. Dates sort chronologically, and player 1's earliest row was inserted last, which makes no difference. **Follow-up trap:** "and the device used that day?" cannot be answered by adding `device_id` to this query, because it is neither grouped nor aggregated. That is P40.

P12 · Follower count — `COUNT(*)` vs `COUNT(col)`

Problem. `follows(user_id, follower_id)` has one row per "follower follows user". Count followers per user, ordered by user id.

```
SELECT user_id, count(*) AS followers_count
FROM follows GROUP BY user_id ORDER BY user_id;
```

```
 user_id | followers_count
---------+-----------------
       1 |               2
       2 |               1
       3 |               3
(3 rows)
```

Edge case — users with zero followers. User 4 exists in `app_user` but nobody follows them. Joining from the user table fixes the missing row, but only if you count a column from the right-hand side:

```sql
SELECT u.user_id, count(f.follower_id) AS good, count(*) AS wrong
FROM app_user u LEFT JOIN follows f USING (user_id)
GROUP BY u.user_id ORDER BY u.user_id;
```

```
 user_id | good | wrong
---------+------+-------
       1 |    2 |     2
       2 |    1 |     1
       3 |    3 |     3
       4 |    0 |     1
(4 rows)
```

The LEFT JOIN produces one NULL-padded row for user 4. `count(*)` counts it, while `count(f.follower_id)` skips the NULL.

P13 · When was each bike last used — `MAX` and sorting by an aggregate

Problem. `ride(ride_id, bike_no, started_at, ended_at)`. Report the latest end time per bike, with the most recently used bike first.

```sql
SELECT bike_no, max(ended_at) AS last_used
FROM ride GROUP BY bike_no ORDER BY last_used DESC;
```

```
 bike_no |      last_used
---------+---------------------
 B-10    | 2024-05-03 17:45:00
 B-20    | 2024-05-01 09:50:00
 B-30    | 2024-04-28 12:20:00
(3 rows)
```

Edge case. Bike B-20 is out on a ride right now (`ended_at` is NULL). `max` skips NULLs, so B-20 reports its previous ride. Ask whether "in use now" should count as most recent. The file adds `bool_or(ended_at IS NULL) AS out_now` to flag it (`t` for B-20). Also note that in PostgreSQL `ORDER BY … DESC` puts NULLs **first**. Add `NULLS LAST` if some bike could have only unfinished rides.

P14 · Missed appointments — filter, then group

Problem. `appointment(id, patient_id, doctor_id, appt_date, status)` with status Completed / Cancelled / No-Show. Count no-shows per patient and leave out patients who never missed one.

```sql
SELECT patient_id, count(*) AS no_shows
FROM appointment
WHERE status = 'No-Show'
GROUP BY patient_id
ORDER BY patient_id;
```

```
 patient_id | no_shows
------------+----------
         11 |        2
         12 |        2
(2 rows)
```

Why it works. `WHERE` removes the other statuses *before* grouping, so patient 13 (only a cancellation) never forms a group. This matches the spec that such patients are left out.

Alternative. When you need several counts at once, keep all rows and count conditionally: `count(*) FILTER (WHERE status = 'No-Show')`, with `HAVING` that count `> 0`. In the same pass it can also report `count(*) AS booked` (patient 11: 2 of 3). That idea is developed in 18.6.

P15 · Products sold per day — `COUNT(DISTINCT)` and `string_agg`

Problem. `sales_log(sold_on, product)` has no key and contains duplicates. For each date, return how many *different* products were sold and a comma-separated, alphabetical list of them, ordered by date.

```
SELECT sold_on,
       count(DISTINCT product)                           AS num_sold,
       string_agg(DISTINCT product, ',' ORDER BY product) AS products
FROM sales_log
GROUP BY sold_on
ORDER BY sold_on;
```

```
  sold_on   | num_sold |   products
------------+----------+-------------
 2024-06-01 |        2 | Ink,Pen
 2024-06-02 |        1 | Stapler
 2024-06-03 |        3 | Clip,Ink,Pen
(3 rows)
```

Why it works / edge cases. `DISTINCT` inside both aggregates removes the second "Pen" on 1 June. A NULL product on 3 June is ignored by both aggregates. With `DISTINCT`, PostgreSQL requires the `ORDER BY` inside `string_agg` to use the aggregated expression itself.

Engine	Ordered, de-duplicated list
PostgreSQL	`string_agg(DISTINCT p, ',' ORDER BY p)`
MySQL	`GROUP_CONCAT(DISTINCT p ORDER BY p SEPARATOR ',')` (output capped by `group_concat_max_len`, default 1024 bytes)
SQL Server 2017+	`STRING_AGG(p, ',') WITHIN GROUP (ORDER BY p)`: no `DISTINCT`, so de-duplicate in a subquery first
SQLite	`group_concat(DISTINCT p)`: order not guaranteed before 3.44

P16 · Customer basket summary — the same pattern, per customer

Problem. `purchase(bought_on, customer_id, product)` may repeat. For each customer, show the number of distinct products and their alphabetical list.

```
SELECT customer_id,
       count(DISTINCT product) AS num_products,
       string_agg(DISTINCT product, ',' ORDER BY product) AS products
FROM purchase GROUP BY customer_id ORDER BY customer_id;
```

```
 customer_id | num_products |    products
-------------+--------------+----------------
           1 |            2 | Mug,Tea
           2 |            1 | Coffee
           3 |            3 | Coffee,Mug,Tea
(3 rows)
```

Alternative. If the result goes to application code, return a real array rather than a string you'll have to split again: `array_agg(DISTINCT product ORDER BY product)` → `{Mug,Tea}`. The lesson of P15 and P16 is that "one row per X with a list of Y" is a `GROUP BY X` plus a string or array aggregate.

18.4 Join problems (P17–P22)

Combine rows from two tables, or from one table with itself. The traps: rows that have no partner, rows that get multiplied, and ties. Background: Chapter 5.

P17 · Product name for every sale — basic inner join

Problem. `sale(sale_id, product_id, yr, qty, unit_price)` and `product(product_id, name)`. For every sale, show the product name, year and unit price, ordered by year.

```
SELECT p.name AS product_name, s.yr AS year, s.unit_price AS price
FROM sale s
JOIN product p ON p.product_id = s.product_id
ORDER BY s.yr, s.sale_id;
```

```
 product_name | year | price
--------------+------+-------
 Kettle       | 2022 |    27
 Kettle       | 2023 |    25
 Toaster      | 2023 |    40
(3 rows)
```

Why it works. Each sale finds its single matching product (`product_id` is the product table's primary key), so the join neither loses nor duplicates sales. **Edge case:** with a foreign key, every sale has a product. Without one, an orphan sale would vanish from an inner join, so switch to `LEFT JOIN` and show a NULL name. Break ties in the sort (`, s.sale_id`) so the order is **deterministic** (the same every run). An **orphan** here is a child row whose parent row doesn't exist.

P18 · Average process time per machine — self-join start to end

Problem. `machine_event(machine_id, process_id, kind, ts)` logs a `'start'` row and an `'end'` row per process. For each machine, report the average of (end − start), rounded to 3 decimals.

Data (`18_process_time.sql`): three machines with two finished processes each; machine 2 also has a third process that never ended.

```
SELECT s.machine_id, round(avg(e.ts - s.ts), 3) AS processing_time
FROM machine_event s
JOIN machine_event e
  ON e.machine_id = s.machine_id AND e.process_id = s.process_id
 AND s.kind = 'start' AND e.kind = 'end'
GROUP BY s.machine_id
ORDER BY s.machine_id;
```

```
 machine_id | processing_time
------------+-----------------
          0 |           0.894
          1 |           0.995
          2 |           1.456
(3 rows)
```

Why it works. This is a **self-join** (a table joined to itself). **Aliases** (short alternative names) `s` and `e` give two views of the same table. The join puts each process's start and end on one row, and `avg` then averages the differences. The unfinished process has no `end` row, so the inner join drops it.

Alternative / edge case. You can pivot each process to one row with `max(ts) FILTER (WHERE kind = 'start')` and `… 'end'`, and then average the differences. That gives the same numbers, because the unfinished process's difference is NULL and `avg` skips it. A tempting shortcut is `(sum of end ts − sum of start ts) / count(DISTINCT process_id)`, and it would be wrong here, because it adds the unfinished start into the sum. **PostgreSQL note:** `round(x, 3)` exists only for `numeric`. Here `ts` is `numeric`. If it were `double precision` (a `float` — an approximate binary number type), write `round((…)::numeric, 3)` (`::numeric` is PostgreSQL's shorthand for a **cast**, i.e. converting a value to another type).

P19 · Fiction books per author — join, filter, group, two-key sort

Problem. `book(book_id, title, genre)` and `book_author(author_id, book_id, author_name)`. Count each author's Fiction titles, leave out authors with none, and sort by count (highest first) and then by name.

```
SELECT a.author_id, a.author_name, count(*) AS fiction_books_count
FROM book_author a
JOIN book b ON b.book_id = a.book_id
WHERE b.genre = 'Fiction'
GROUP BY a.author_id, a.author_name
ORDER BY fiction_books_count DESC, a.author_name, a.author_id;
```

```
 author_id | author_name | fiction_books_count
-----------+-------------+--------------------
        10 | Iris Vale   |                   2
        13 | Iris Vale   |                   1
        11 | Omar Reyes  |                   1
(3 rows)
```

Why it works. The inner join plus `WHERE` keeps only fiction rows, so authors with zero fiction form no group, which is what the spec wants. **Edge case — duplicate names:** two different people are called Iris Vale. Grouping by `author_name` alone merges them into one row showing `3`, which is wrong. Always group by the **key** and add the name for display.

P20 · Exam sittings per student and subject — CROSS JOIN + LEFT JOIN

Problem. `student`, `subject`, and `exam_sitting(student_id, subject_name)` (duplicates allowed, one row per sitting). For every student × subject pair, report how many times the student sat that exam (0 included), ordered by student id and subject.

```
SELECT st.student_id, st.student_name, su.subject_name,
       count(e.subject_name) AS attended_exams
FROM student st
CROSS JOIN subject su
LEFT JOIN exam_sitting e
       ON e.student_id = st.student_id AND e.subject_name = su.subject_name
GROUP BY st.student_id, st.student_name, su.subject_name
ORDER BY st.student_id, su.subject_name;
```

```
 student_id | student_name | subject_name | attended_exams
------------+--------------+--------------+----------------
          1 | Noor         | Art          |              0
          1 | Noor         | Math         |              3
          1 | Noor         | Physics      |              1
          2 | Kofi         | Art          |              1
          2 | Kofi         | Math         |              1
          2 | Kofi         | Physics      |              0
          6 | Lin          | Art          |              0
          6 | Lin          | Math         |              0
          6 | Lin          | Physics      |              0
(9 rows)
```

Why it works. "Every pair, including zeros" means you must *create* the pairs, because the **fact table** (the table recording events — here, sittings) only has pairs that happened. `CROSS JOIN` (every row of one table combined with every row of the other) builds the full 3 × 3 grid. The `LEFT JOIN` attaches matching sittings (both columns in `ON`), and `count(e.subject_name)` turns missing matches into 0. This **grid + left join** pattern also fills in missing days in time series (with `generate_series`, which produces a sequence of numbers or dates).

P21 · Election winner — ORDER BY an aggregate, and ties

Problem. `candidate(id, name)` and `vote(id, candidate_id)` (one row per ballot). Return the name of the candidate with the most votes (the data guarantees a single winner).

```sql
SELECT c.name
FROM candidate c
JOIN vote v ON v.candidate_id = c.id
GROUP BY c.id, c.name
ORDER BY count(*) DESC
LIMIT 1;
```

```
 name
-------
 Tariq
(1 row)
```

Why it works. Group ballots per candidate, sort by the count, and keep the top row. Grouping by `c.id` (not just name) keeps namesakes apart.

Edge case — ties. The guarantee rarely holds in real life. After one more vote for Victor, both have 3 and `LIMIT 1` would pick one arbitrarily. The standard `FETCH FIRST 1 ROWS WITH TIES` (PostgreSQL 13+, Oracle 12c+; SQL Server uses `TOP 1 WITH TIES`) returns every row tied with the last one kept:

```
  name  | votes
--------+-------
 Tariq  |     3
 Victor |     3
(2 rows)
```

MySQL has no `WITH TIES`. Use `RANK() OVER (ORDER BY count(*) DESC)` in a subquery and keep rank 1 (Chapter 8).

P22 · "Middle" users in a follow graph — self-join or EXISTS

Problem. `follow(followee, follower)`: the follower follows the followee, and nobody follows themselves. Find users who follow at least one person *and* have at least one follower. Report each one's follower count, alphabetically.

Data: amy←bo, bo←cy, bo←dee, cy←eve, cy←bo, dee←amy (read "amy is followed by bo").

```sql
SELECT f.followee AS follower, count(*) AS num
FROM follow f
WHERE EXISTS (SELECT 1 FROM follow g WHERE g.follower = f.followee)
GROUP BY f.followee
ORDER BY f.followee;
```

```
 follower | num
----------+-----
 amy      |   1
 bo       |   2
 cy       |   2
 dee      |   1
(4 rows)
```

Why it works. Each row of `f` is one follower of `f.followee`. `EXISTS` keeps the row only if that same user also appears as somebody's *follower*. Eve is followed by no one, so she is excluded.

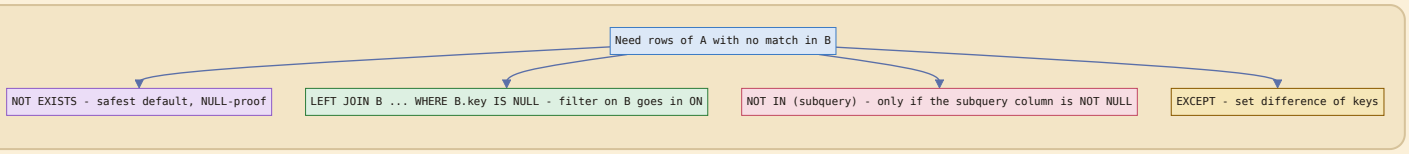
Alternative — and the fan-out trap. The self-join `follow a JOIN follow b ON b.followee = a.follower` also works. However, each of bo's followers pairs with each person bo follows, so the rows multiply:

```
 follower | num | inflated
----------+-----+----------
 amy      |   1 |        1
 bo       |   2 |        4
 cy       |   2 |        2
 dee      |   1 |        1
(4 rows)
```

Only `count(DISTINCT b.follower)` is correct there. When a join can match many rows on **both** sides, either de-duplicate or use `EXISTS` (a *semi-join* — it keeps a row if at least one match exists — which never multiplies).

18.5 Anti-join problems (P23–P26)

"Find X that has **no** Y" is an **anti-join**. There are three common spellings. They behave identically, except that `NOT IN` breaks when the subquery contains a NULL. Background: Chapter 5 §5.8 and Chapter 6 §6.4.



P23 · Sellers with no 2020 sales — NOT EXISTS with a date range

Problem. `seller(seller_id, seller_name)` and `orders(order_id, sold_on, cost, customer_id, seller_id)`. List sellers who made no sale at all in 2020, by name.

```
SELECT s.seller_name
FROM seller s
WHERE NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.seller_id = s.seller_id
    AND o.sold_on >= DATE '2020-01-01' AND o.sold_on < DATE '2021-01-01')
ORDER BY s.seller_name;
```

```
 seller_name
-------------
 Reed Ltd
 Tern LLC
(2 rows)
```

Why it works. For each seller, the subquery asks "is there any 2020 order?". `NOT EXISTS` keeps sellers where the answer is no: Reed sold only in 2019, and Tern never sold. Quill's 31 December order counts, and Sable's 1 January 2021 order doesn't. The **half-open** range `>= 1 Jan AND < next 1 Jan` is safe for `date` and `timestamp` alike. `BETWEEN '2020-01-01' AND '2020-12-31'` would miss 31 December after midnight if the column were a timestamp. PostgreSQL executes `NOT EXISTS` as a hash or merge **anti-join** (two internal join algorithms: one builds an in-memory lookup table, the other walks two sorted inputs side by side), and an index on `orders(seller_id, sold_on)` makes each probe cheap.

P24 · Dormant accounts — LEFT JOIN … IS NULL

Problem. `account(account_id, account_name)` and `txn(txn_id, account_id, txn_date)`. Return accounts that made no transaction in 2020, by name.

```
SELECT a.account_name
FROM account a
LEFT JOIN txn t
       ON t.account_id = a.account_id
      AND t.txn_date >= DATE '2020-01-01' AND t.txn_date < DATE '2021-01-01'
WHERE t.txn_id IS NULL
ORDER BY a.account_name;
```

```
 account_name
--------------
 Juniper
 Kestrel
(2 rows)
```

Why it works. The LEFT JOIN attaches only *2020* transactions. Accounts with none get a NULL-padded row, and `WHERE t.txn_id IS NULL` keeps exactly those. Test a column that can't be NULL in a real match, such as the primary key.

The classic bug. Move the date test into `WHERE` and you ask for rows where `t` is NULL **and** `t.txn_date` is in 2020. No row can be both, so the result is `(0 rows)`. **Alternative:** a **set difference** of ids (rows in the first result that are not in the second), `SELECT account_id FROM account EXCEPT SELECT account_id FROM txn WHERE …2020…`, which returns the same Juniper and Kestrel.

P25 · Sessions with no ads — anti-join on a range condition

Problem. `playback(session_id, customer_id, start_t, end_t)` and `ad(ad_id, customer_id, shown_at)`. A session is ad-free if that customer was shown no ad between `start_t` and `end_t` (inclusive). List ad-free sessions.

Data: customer 1 has sessions [1,5] and [15,23] and ads at 5 and 9. Customer 2 has [10,12] and [17,28] and ads at 17 and 20. Customer 3 has [2,8] and an ad at 30.

```
SELECT p.session_id
FROM playback p
WHERE NOT EXISTS (
  SELECT 1 FROM ad a
  WHERE a.customer_id = p.customer_id
    AND a.shown_at BETWEEN p.start_t AND p.end_t)
ORDER BY p.session_id;
```

```
 session_id
------------
          2
          3
          5
(3 rows)
```

Why it works / edge cases. The match condition is not plain equality. It is "same customer **and** timestamp inside the interval". Session 1 is excluded because the ad at 5 sits exactly on its (inclusive) end, and session 4 because of the ad at 17. The ad at 9 falls between customer 1's sessions, so it affects neither. Forgetting `a.customer_id = p.customer_id` would let another viewer's ads disqualify a session.

P26 · Reps who never sold to RED — NOT EXISTS vs NOT IN

Problem. `sales_rep(rep_id, name)`, `company(com_id, name, city)`, `orders(order_id, order_date, com_id, rep_id, amount)`. Return the reps with no order from the company named `'RED'`.

```
SELECT r.name
FROM sales_rep r
WHERE NOT EXISTS (
  SELECT 1 FROM orders o JOIN company c ON c.com_id = o.com_id
  WHERE o.rep_id = r.rep_id AND c.name = 'RED')
ORDER BY r.name;
```

```
 name
------
 Kai
 Lu
 Ned
(3 rows)
```

Why it works. The subquery joins orders to companies so it can filter by company *name*, and it is **correlated** on the rep (it refers to the outer row `r`, so it is evaluated per rep).

Edge case — the NOT IN trap. `WHERE r.rep_id NOT IN (SELECT o.rep_id FROM … WHERE c.name = 'RED')` returns the same three names. Then we insert one RED order with no rep (`rep_id` NULL), and the `NOT IN` version returns:

```
 name
------
(0 rows)
```

`x NOT IN (1, 4, NULL)` means `x <> 1 AND x <> 4 AND x <> NULL`. The last test is `UNKNOWN` for every `x`, so no row qualifies. `NOT EXISTS` still returns Kai, Lu, Ned. Default to `NOT EXISTS`. If you must use `NOT IN`, add `WHERE o.rep_id IS NOT NULL` inside.

18.6 Conditional aggregation (P27–P28)

Conditional aggregation computes several filtered aggregates over the same groups in one pass. It is SQL's way to turn row values (statuses, types) into columns, a simple **pivot**. PostgreSQL and standard SQL spell it `agg(...) FILTER (WHERE cond)`. Every engine accepts `SUM(CASE WHEN cond THEN x ELSE 0 END)`. See Chapter 4 §4.4.

P27 · Attendance summary — one column per status

Problem. `employee(employee_id, employee_name)` and `attendance(employee_id, att_date, status)` with statuses Present / Absent / Late (duplicates possible). Show, for every employee, the number of rows of each status, ordered by id.

```sql
SELECT e.employee_id, e.employee_name,
       count(*) FILTER (WHERE a.status = 'Present') AS days_present,
       count(*) FILTER (WHERE a.status = 'Absent')  AS days_absent,
       count(*) FILTER (WHERE a.status = 'Late')    AS days_late
FROM employee e
LEFT JOIN attendance a ON a.employee_id = e.employee_id
GROUP BY e.employee_id, e.employee_name
ORDER BY e.employee_id;
```

```
 employee_id | employee_name | days_present | days_absent | days_late
-------------+---------------+--------------+-------------+-----------
           1 | Aiko          |            2 |           0 |         1
           2 | Bram          |            2 |           1 |         0
           3 | Cleo          |            0 |           0 |         0
(3 rows)
```

Why it works. The LEFT JOIN keeps Cleo, who has no records. Each `count(*) FILTER` counts only the rows passing its condition, and `count` never returns NULL. Bram's duplicated "Present" row counts twice, as the spec says to count *rows*. If it meant *days*, use `count(DISTINCT a.att_date) FILTER (…)`.

Portable form and a trap. The `sum(CASE …)` spelling with `THEN 1 ELSE 0` also gives Cleo 0 (column `present_else0`). Drop the `ELSE 0` and the `CASE` yields NULL for her one padded row, so `sum` returns **NULL**:

```
 employee_id | present_else0 | present_no_else
-------------+---------------+-----------------
           1 |             2 |               2
           2 |             2 |               2
           3 |             0 |          (null)
(3 rows)
```

P28 · Monthly approvals and chargebacks — UNION ALL, then pivot

Problem. `txn(id, country, state, amount, trans_date)` with state approved/declined, and `chargeback(trans_id, cb_date)`. For each month and country, report the approved count and amount, and the chargeback count and amount. A chargeback belongs to the month it **arrived** and carries the original transaction's amount. Leave out month/country pairs where everything is zero.

```sql
WITH events AS (
  SELECT to_char(trans_date, 'YYYY-MM') AS month, country, 'approved' AS kind, amount
  FROM txn WHERE state = 'approved'
  UNION ALL
  SELECT to_char(c.cb_date, 'YYYY-MM'), t.country, 'chargeback', t.amount
  FROM chargeback c JOIN txn t ON t.id = c.trans_id
)
SELECT month, country,
       count(*) FILTER (WHERE kind = 'approved')                   AS approved_count,
       COALESCE(sum(amount) FILTER (WHERE kind = 'approved'), 0)   AS approved_amount,
       count(*) FILTER (WHERE kind = 'chargeback')                 AS chargeback_count,
       COALESCE(sum(amount) FILTER (WHERE kind = 'chargeback'), 0) AS chargeback_amount
FROM events
GROUP BY month, country
ORDER BY month, country;
```

```
  month  | country | approved_count | approved_amount | chargeback_count | chargeback_amount
---------+---------+----------------+-----------------+------------------+-------------------
 2024-05 | US      |              1 |            1000 |                1 |              2000
 2024-06 | US      |              2 |            8000 |                1 |              1000
 2024-09 | US      |              0 |               0 |                1 |              5000
(3 rows)
```

Why it works. The two event kinds have **different dates**, so a single join can't bucket them. The trick is to stack them into one long list of `(month, country, kind, amount)` with `UNION ALL`, which keeps duplicates, unlike `UNION`. After that, one `GROUP BY` plus four `FILTER`s pivots the list. Notice what this data exercises. The declined May transaction still produces a May chargeback. Chargebacks land in later months (June, September). The DE row, which is only a declined transaction, produces no event, so the all-zero rows disappear with no `HAVING` needed. `sum … FILTER` over no rows is NULL, hence the `COALESCE`. The `to_char(d, 'YYYY-MM')` month label is `DATE_FORMAT(d, '%Y-%m')` in MySQL and `FORMAT(d, 'yyyy-MM')` in SQL Server.

18.7 Date-logic problems (P29–P33)

Date questions test ranges, arithmetic and "what was true on day X". Useful PostgreSQL facts: `date - date` is an **integer** number of days; `date + 1` is the next day; `date_trunc('month', ts)` gives the first instant of the month. A **half-open** range includes its start but excludes its end (`>= start AND < next_start`). Background: Chapter 3.

P29 · Weather class per country in November 2019 — month range + CASE on an average

Problem. `country(country_id, country_name)` and `weather(country_id, weather_state, day)`. Using only November 2019 readings, label each country Cold if its average is ≤ 15, Hot if ≥ 25, and Warm otherwise.

```sql
SELECT c.country_name,
       CASE WHEN avg(w.weather_state) <= 15 THEN 'Cold'
            WHEN avg(w.weather_state) >= 25 THEN 'Hot'
            ELSE 'Warm' END AS weather_type
FROM weather w
JOIN country c ON c.country_id = w.country_id
WHERE w.day >= DATE '2019-11-01' AND w.day < DATE '2019-12-01'
GROUP BY c.country_id, c.country_name
ORDER BY c.country_name;
```

```
 country_name | weather_type
--------------+--------------
 Arland       | Cold
 Borea        | Cold
 Calidra      | Warm
 Dune         | Hot
(4 rows)
```

Why it works. `WHERE` keeps November rows. Arland's October readings and Dune's December readings are ignored, so Dune's single November reading of 25 makes it Hot. `CASE` can test an aggregate because it runs after grouping. `CASE` stops at the first true branch, so order the thresholds carefully. PostgreSQL's `avg(int)` returns `numeric`, so there is no truncation. **Edge case:** a country with no November data simply doesn't appear. If it must appear, `LEFT JOIN` from `country` with the date filter in `ON`, and decide what label an unknown average gets.

P30 · Engagement level for February 2020 — leap-year boundary

Problem. `app_user(user_id, user_name)` and `engagement(user_id, score, day)`. For February 2020, classify each user's average score: below 20 Low, 20–60 inclusive Medium, above 60 High.

```sql
SELECT u.user_name,
       CASE WHEN avg(e.score) < 20  THEN 'Low'
            WHEN avg(e.score) <= 60 THEN 'Medium'
            ELSE 'High' END AS engagement_level
FROM app_user u
JOIN engagement e ON e.user_id = u.user_id
WHERE e.day >= DATE '2020-02-01' AND e.day < DATE '2020-03-01'
GROUP BY u.user_id, u.user_name
ORDER BY u.user_name;
```

```
 user_name | engagement_level
-----------+-----------------
 Ravi      | Medium
 Sofia     | Medium
 Theo      | High
(3 rows)
```

Why it works / edge case. Same shape as P29, but this data hits a boundary. 2020 is a leap year, and Ravi's 90 was recorded on **29 February**. A hand-written `BETWEEN '2020-02-01' AND '2020-02-28'` drops it, and Ravi's average falls from 50 (Medium) to 10 (Low). The half-open `< DATE '2020-03-01'` never needs to know how long the month is. Sofia's 1 March reading is correctly excluded. (A MySQL-style `YEAR(d) = 2020 AND MONTH(d) = 2` is also correct but can't use a plain index on `d`.)

P31 · Absences longer than three days — date subtraction, inclusive counting

Problem. `absence(employee_id, employee_name, start_date, end_date)` with start ≤ end. Return absences that last more than three days, where both the first and the last day count as days off.

```sql
SELECT employee_name, start_date, end_date, end_date - start_date + 1 AS days_off
FROM absence
WHERE end_date - start_date + 1 > 3
ORDER BY employee_name;
```

```
 employee_name | start_date |  end_date  | days_off
---------------+------------+------------+----------
 Hana          | 2024-03-01 | 2024-03-05 |        5
 Jade          | 2024-02-27 | 2024-03-01 |        4
(2 rows)
```

Why it works / edge cases. `end - start` counts the *gaps* between days, and `+ 1` counts the *days*. Ivo (10–12 March) is exactly 3 days, so it is not "more than three". Jade's absence crosses 29 February 2024 and is still counted correctly as 4 days. **Clarify** whether "more than three days" means calendar or working days. For working days, count `generate_series(start, end, '1 day')` rows whose `extract(isodow …) < 6` (ISO day of week: Monday = 1 … Sunday = 7). Dialects: MySQL `DATEDIFF(end, start)` (days), SQL Server `DATEDIFF(day, start, end)`, SQLite `julianday(end) - julianday(start)`.

P32 · Absence streaks of 3+ days — gaps and islands (first taste)

Problem. `attendance(student_id, att_date, status)` with one row per student per day, status Present/Absent. Find every run of **three or more consecutive calendar days** of absence, and report the student and the run's first day.

Data: student 1 is absent 1–3 Jan, present 4 Jan, then absent 5–8 Jan. Student 2 is absent 1–2 and 4–5 Jan (no run of 3). Student 3 is absent 30 Jan – 1 Feb, across a month boundary.

The trick: within a run of consecutive dates, `date − row_number()` is constant. (`row_number()` is a **window function** numbering rows 1, 2, 3… within each `PARTITION BY` group in the given order; `PARTITION BY` splits rows into groups without collapsing them.)

```
SELECT student_id, att_date,
       row_number() OVER w AS rn,
       att_date - (row_number() OVER w)::int AS grp
FROM attendance
WHERE status = 'Absent' AND student_id = 1
WINDOW w AS (PARTITION BY student_id ORDER BY att_date)
ORDER BY att_date;
```

```
 student_id |  att_date  | rn |    grp
------------+------------+----+------------
          1 | 2024-01-01 |  1 | 2023-12-31
          1 | 2024-01-02 |  2 | 2023-12-31
          1 | 2024-01-03 |  3 | 2023-12-31
          1 | 2024-01-05 |  4 | 2024-01-01
          1 | 2024-01-06 |  5 | 2024-01-01
          1 | 2024-01-07 |  6 | 2024-01-01
          1 | 2024-01-08 |  7 | 2024-01-01
(7 rows)
```

```
WITH absent AS (
  SELECT student_id, att_date,
         att_date - (row_number() OVER (PARTITION BY student_id ORDER BY att_date))::int AS grp
  FROM attendance
  WHERE status = 'Absent'
)
SELECT student_id, min(att_date) AS start_date, count(*) AS days
FROM absent
GROUP BY student_id, grp
HAVING count(*) >= 3
ORDER BY student_id, start_date;
```

```
 student_id | start_date | days
------------+------------+------
          1 | 2024-01-01 |    3
          1 | 2024-01-05 |    4
          3 | 2024-01-30 |    3
(3 rows)
```

Why it works. When the date goes up by 1 and the row number also goes up by 1, their difference stays the same. A gap in the dates bumps the difference, which starts a new group (an *island*). Grouping by `(student, grp)` then yields one row per run. `min` gives its start and `count` its length.

Common mistake. Subtracting two `row_number()`s that use the **same** partition and order always gives 0, so every absence lands in one group. The "two row numbers" variant only works when the two numberings differ: one over *all* rows and one over *absent* rows. That variant is what you want when "consecutive" means consecutive **records** (for example school days, skipping weekends) rather than calendar days. Full treatment: Chapter 8 §8.9 and Chapter 19.

P33 · Price on a given date — latest row on or before a date

Problem. `price_change(product_id, new_price, effective_on)`. Each product costs 10 until its first change. Report every product's price on 15 March 2024.

```sql
WITH latest AS (
  SELECT DISTINCT ON (product_id) product_id, new_price
  FROM price_change
  WHERE effective_on <= DATE '2024-03-15'
  ORDER BY product_id, effective_on DESC
)
SELECT p.product_id, COALESCE(l.new_price, 10) AS price
FROM (SELECT DISTINCT product_id FROM price_change) AS p
LEFT JOIN latest l USING (product_id)
ORDER BY p.product_id;
```

```
 product_id | price
------------+-------
          1 |    30
          2 |    50
          3 |    10
(3 rows)
```

Why it works. The CTE throws away future changes and then keeps, per product, the row with the newest `effective_on` (`DISTINCT ON` keeps the first row of each group in the `ORDER BY`). Product 1 changed on the 14th and again **on** the 15th, and `<=` picks the 15th (30). Product 3's first change is on the 18th, so it has no row in `latest`, and the LEFT JOIN plus `COALESCE` gives the default 10.

Alternatives. A correlated scalar subquery, `COALESCE((SELECT new_price … ORDER BY effective_on DESC LIMIT 1), 10)`, gives the same result and is fast with an index on `(product_id, effective_on)`. `ROW_NUMBER() … = 1` works in every engine. In a real schema, the list of products should come from a `product` table, because products that never changed price would be missing from `price_change`.

18.8 Ratios and percentages (P34–P38)

Ratio questions are mostly about **arithmetic types**. In PostgreSQL, SQL Server and SQLite, `int / int` is integer division (`2/3 = 0`), whereas MySQL's `/` returns a decimal. Guard every denominator that can be 0 with `NULLIF(den, 0)`, and round at the end, not in the middle.

P34 · Search query quality — averaging a ratio, share of "poor" rows

Problem. `search_log(query_name, result, position, rating)` (duplicates allowed). Per query name: *quality* is the average of `rating / position`, and *poor %* is the percentage of rows with rating below 3. Round both to 2 decimals.

```sql
SELECT query_name,
       round(avg(rating::numeric / position), 2)                      AS quality,
       round(100.0 * count(*) FILTER (WHERE rating < 3) / count(*), 2) AS poor_query_percentage
FROM search_log
GROUP BY query_name
ORDER BY query_name;
```

```
 query_name | quality | poor_query_percentage
------------+---------+-----------------------
 Cat        |    0.66 |                 33.33
 Dog        |    2.50 |                 33.33
(2 rows)
```

Why it works. The cast makes each ratio exact (Dog: 5, 2.5, 0.005 → average 2.50). `100.0 * …` forces decimal arithmetic before the division.

The bug to mention. Without the casts, every `rating / position` truncates (5/2 → 2, 1/200 → 0) and the poor share becomes `1 / 3 * 100 = 0`:

```
 query_name |      int_quality       | int_poor_pct
------------+------------------------+--------------
 Cat        | 0.3333333333333333333 |            0
 Dog        |     2.3333333333333333 |            0
(2 rows)
```

MySQL gets these right without casts (its `/` is decimal), which is why solutions ported from MySQL often break on PostgreSQL. Also note that `round(double precision, int)` doesn't exist in PostgreSQL. Keep the arithmetic in `numeric`.

P35 · Ad click-through rate — divide by a possibly-zero count

Problem. `ad_action(ad_id, user_id, action)` with actions Clicked / Viewed / Ignored. **CTR** (click-through rate, the share of impressions that led to a click) = clicks × 100 / (clicks + views), or 0 when an ad has neither. Round to 2 decimals and sort by CTR (highest first), then by ad id.

```sql
SELECT ad_id,
       round(COALESCE(100.0 * count(*) FILTER (WHERE action = 'Clicked')
                      / NULLIF(count(*) FILTER (WHERE action IN ('Clicked','Viewed')), 0), 0), 2) AS ctr
FROM ad_action
GROUP BY ad_id
ORDER BY ctr DESC, ad_id;
```

```
 ad_id |  ctr
-------+-------
     1 | 66.67
     3 | 50.00
     2 | 33.33
     5 |  0.00
(4 rows)
```

Why it works. Two conditional counts give clicks and clicks + views. Ad 5 was only ignored, so its denominator is 0. `NULLIF(0, 0)` turns it into NULL, the division yields NULL instead of an error, and `COALESCE(…, 0)` applies the spec's rule. "Ignored" rows are in the group but excluded from both counts. The secondary sort key `ad_id` makes ties deterministic.

P36 · Each book's share of all loans — ratio against a grand total

Problem. `book(book_id, title)` and `loan(loan_id, book_id, user_id, loan_date)`. A book's loan share is (its loans) / (all loans). Show every book's share rounded to 2 decimals, highest first, ties by id.

```sql
SELECT b.book_id,
       round(count(l.loan_id)::numeric / NULLIF((SELECT count(*) FROM loan), 0), 2) AS lf
FROM book b
LEFT JOIN loan l ON l.book_id = b.book_id
GROUP BY b.book_id
ORDER BY lf DESC, b.book_id;
```

```
 book_id |  lf
---------+------
       1 | 0.50
       2 | 0.33
       3 | 0.17
       4 | 0.00
(4 rows)
```

Why it works. The numerator is per group. The **denominator** (the bottom of the fraction; the **numerator** is the top) is the whole-table count from an uncorrelated scalar subquery, which runs once. Book 4 was never lent, so the LEFT JOIN keeps it and `count(l.loan_id)` is 0. Without `::numeric`, 3/6 would be 0 in PostgreSQL.

Alternative. A window over the grouped result avoids the second scan: `count(l.loan_id)::numeric / NULLIF(sum(count(l.loan_id)) OVER (), 0)`. Window functions run after `GROUP BY`, so they can aggregate the aggregates. This gives the same output here. Keep the `NULLIF`: with an empty `loan` table the window sum

is 0 and the division would fail. The two denominators differ only if `loan` could hold book ids missing from `book`, which the foreign key rules out.

P37 · Average selling price — weighted average with a range join

Problem. `price(product_id, start_d, end_d, price)` holds non-overlapping price periods. `units_sold(product_id, bought_on, units)` holds sales. For each product, the average selling price is total revenue / total units, using the price valid on each sale's date. Products that never sold show 0. Round to 2 decimals.

```
SELECT p.product_id,
       round(COALESCE(sum(u.units * p.price)::numeric / sum(u.units), 0), 2) AS average_price
FROM price p
LEFT JOIN units_sold u
       ON u.product_id = p.product_id
      AND u.bought_on BETWEEN p.start_d AND p.end_d
GROUP BY p.product_id
ORDER BY p.product_id;
```

```
 product_id | average_price
------------+---------------
          1 |          6.96
          2 |         16.96
          3 |          0.00
(3 rows)
```

Why it works. This is a **range join** (the join condition is "falls inside an interval", not equality). The join matches each sale to exactly one price period (same product, date inside the period). Product 1 sold 100 units at 5 and 15 at 20: (500 + 300) / 115 = 6.96. Product 3 has a price but no sales. Its sums are NULL, NULL/NULL is NULL (not an error), and `COALESCE` makes it 0.

The bug. `avg(p.price)` averages the *price rows* (5 and 20 → 12.50) and ignores that most units sold at 5. Whenever rows carry a quantity, use a **weighted** average: `sum(value × weight) / sum(weight)`. If a sale could fall outside every period, it would silently vanish, so check with an anti-join.

P38 · Came back the next day — retention fraction

Problem. `activity(player_id, device_id, played_on, games)`. What share of players showed up again exactly one day after their first-ever login? Round to 2 decimals.

```
WITH first_day AS (
  SELECT player_id, min(played_on) AS first_on FROM activity GROUP BY player_id
)
SELECT round(count(a.player_id)::numeric / count(*), 2) AS fraction
FROM first_day f
LEFT JOIN activity a ON a.player_id = f.player_id AND a.played_on = f.first_on + 1;
```

```
 fraction
----------
     0.50
(1 row)
```

Why it works. The CTE gives one row per player (the denominator). The LEFT JOIN looks for a row exactly one day later (`date + 1`). `count(a.player_id)` counts only players who matched (the numerator). Players 1 and 4 returned (player 4 on 29 February 2024, which `+ 1` handles). Player 3 came back two days later, which doesn't count, and player 2 never came back. The result is 2/4. Without the cast, `count / count` is `0`. This "day-1 retention" metric is a staple of **product-analytics** interviews (questions about measuring how users behave in an app). Variants ask for the fraction *per install date* (group by `first_on`) or for day-7 retention (`+ 7`).

18.9 Window and ranking basics (P39–P42)

A first look at problems that are easiest with **window functions**: functions that compute over related rows without collapsing them. Full coverage is in Chapter 8, and the harder variants are in Chapter 19.

P39 · Second-highest salary — and "return NULL if none"

Problem. `emp(id, salary)`. Return the second-highest **distinct** salary, or NULL when there isn't one.

Data: salaries 300, 300, 200, 100 (a tie at the top).

```
SELECT (SELECT DISTINCT salary FROM emp ORDER BY salary DESC OFFSET 1 LIMIT 1)
       AS second_highest_salary;
```

```
 second_highest_salary
----------------------
                  200
(1 row)
```

Why it works. `DISTINCT` collapses the two 300s, so `OFFSET 1` skips exactly one value. Wrapping the query in an outer `SELECT (…)` makes it a **scalar subquery**, which yields NULL when the inner query finds nothing. After deleting all but the 300s, the wrapped query returns `(null)` in one row, while an unwrapped query (the file shows the bare `dense_rank` version) returns 0 rows.

Alternative. `dense_rank() OVER (ORDER BY salary DESC)` numbers distinct values 1, 1, 2, 3. Keep `dr = 2`, and wrap the result in `max(salary)` so an empty result still becomes one NULL row. Without `DISTINCT`, `OFFSET 1` would return the duplicate 300, the most common wrong answer. Another classic is `SELECT max(salary) FROM emp WHERE salary < (SELECT max(salary) FROM emp)`. The general N-th version is in Chapter 19.

P40 · Device used at first login — first row per group

Problem. Using `activity(player_id, device_id, played_on, games)`, report the device each player used on their earliest day.

```
SELECT DISTINCT ON (player_id) player_id, device_id
FROM activity
ORDER BY player_id, played_on;
```

```
 player_id | device_id
-----------+-----------
         1 |         2
         2 |         3
         3 |         1
(3 rows)
```

Why it works. `DISTINCT ON (player_id)` (PostgreSQL only) keeps the first row per player in `ORDER BY` order, so the earliest day wins, and `device_id` comes along with it. That's exactly what `min(played_on)` alone (P11) could not give.

Portable alternatives (same output in the file):

- `row_number() OVER (PARTITION BY player_id ORDER BY played_on) = 1` in a subquery. Use `rank()` if two rows could share the first date and you want both.
- `WHERE (player_id, played_on) IN (SELECT player_id, min(played_on) … GROUP BY player_id)`, a **row-value** `IN` (comparing several columns at once as a tuple). SQL Server doesn't support row-value `IN`, so there you'd join to the grouped subquery instead.

P41 · Games played so far — running total

Problem. For every `(player, day)` in `activity`, show the total games that player has played up to and including that day.

```sql
SELECT player_id, played_on,
       sum(games) OVER (PARTITION BY player_id ORDER BY played_on) AS games_played_so_far
FROM activity
ORDER BY player_id, played_on;
```

```
 player_id | played_on  | games_played_so_far
-----------+------------+---------------------
         1 | 2024-03-01 |                   5
         1 | 2024-05-02 |                  11
         1 | 2024-06-25 |                  12
         3 | 2024-03-02 |                   0
         3 | 2024-07-03 |                   5
(5 rows)
```

Why it works. `PARTITION BY` restarts the sum per player. `ORDER BY` inside `OVER` makes the default **frame** (the set of rows the window function looks at for the current row) "from the first row up to the current row, including rows that tie on the sort key" (`RANGE UNBOUNDED PRECEDING`). Here `(player_id, played_on)` is unique, so there are no ties. If two rows could share a date and you want a strict row-by-row total, write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

Alternative. Before window functions (MySQL < 8.0), you'd self-join each row to all earlier-or-same rows of the same player and `sum`. It gives the same output but does O(n²) work per player (work grows with the square of the number of rows: 10× the rows, 100× the work).

P42 · Active businesses — compare each row to its group's average

Problem. `event(business_id, event_type, occurrences)`. The average for an event type is the mean of `occurrences` over the businesses that have it. A business is **active** if, on *more than one* event type, its occurrences are strictly above that type's average. List active businesses.

First attach each type's average to every row:

```
 business_id | event_type | occurrences | type_avg
-------------+------------+-------------+----------
           1 | ads        |          11 |     8.00
           2 | ads        |           7 |     8.00
           3 | ads        |           6 |     8.00
           4 | coupons    |           9 |     9.00
           1 | page views |           3 |     7.50
           2 | page views |          12 |     7.50
           1 | reviews    |           7 |     5.00
           3 | reviews    |           3 |     5.00
(8 rows)
```

```sql
SELECT business_id
FROM (SELECT business_id, occurrences,
             avg(occurrences) OVER (PARTITION BY event_type) AS type_avg
      FROM event) AS e
WHERE occurrences > type_avg
GROUP BY business_id
HAVING count(*) > 1
ORDER BY business_id;
```

```
 business_id
-------------
           1
(1 row)
```

Why it works. The window `avg` keeps every row and adds the type average next to it. You can't write `WHERE occurrences > avg(...)`, because windows are evaluated after `WHERE`, hence the subquery. Business 1 beats the average on ads and reviews (2 types), and business 2 only on page views. Business 4 is alone in "coupons", so it equals the average and never beats it. **Alternative without windows:** join `event` to `SELECT event_type, avg(occurrences) … GROUP BY event_type` on `event_type`. **Edge case:** "more than one" means `> 1`, not `>= 1`, and "strictly greater" means `>`. Read comparison words carefully.

18.10 Pattern table

#	Problem	Pattern	Key SQL feature
P1	Big cities	Filtering	`OR`, three-valued logic with NULL
P2	Odd non-boring movies	Filtering	`%`, `<>`, `IS DISTINCT FROM`, `ORDER BY DESC`
P3	Green products	Filtering	`AND`, `CHECK` constraint
P4	Outside department 3	Filtering	NULL-safe `IS DISTINCT FROM`
P5	Self-assigned tasks	Filtering	column-to-column compare, `DISTINCT`
P6	Long comments	Filtering	`char_length` vs `octet_length`
P7	Valid e-mails	Filtering	regex `~`, anchors, escaping
P8	Triangle test	Row-level logic	`CASE`, `greatest`
P9	Largest unique number	Aggregation	`HAVING count(*) = 1`, aggregate over empty set → NULL
P10	Units per product	Aggregation	`SUM` + `GROUP BY`, `LEFT JOIN` + `COALESCE`
P11	First login	Aggregation	`MIN` on dates
P12	Follower count	Aggregation	`count(*)` vs `count(col)`
P13	Bike last used	Aggregation	`MAX`, sort by aggregate, NULL handling
P14	Missed appointments	Aggregation	`WHERE` before `GROUP BY`; `FILTER` + `HAVING`
P15	Products per day	Aggregation	`count(DISTINCT)`, `string_agg(DISTINCT … ORDER BY)`
P16	Customer basket	Aggregation	`string_agg` / `array_agg`
P17	Sale product names	Joins	`INNER JOIN` on a key
P18	Process time	Joins	self-join; `FILTER` pivot alternative
P19	Fiction per author	Joins	join + filter + group by key, two-key sort
P20	Exam sittings	Joins	`CROSS JOIN` grid + `LEFT JOIN` + `count(col)`
P21	Election winner	Joins / top-1	`ORDER BY count(*) DESC LIMIT 1`, `WITH TIES`
P22	Middle followers	Joins / semi-join	`EXISTS`; self-join needs `count(DISTINCT)`
P23	Idle sellers	Anti-join	`NOT EXISTS`, half-open date range
P24	Dormant accounts	Anti-join	`LEFT JOIN … IS NULL`, filter in `ON`; `EXCEPT`
P25	Ad-free sessions	Anti-join	`NOT EXISTS` with `BETWEEN` range condition
P26	Reps without RED	Anti-join	`NOT EXISTS` vs `NOT IN` + NULL
P27	Attendance summary	Conditional aggregation	`count(*) FILTER`, `SUM(CASE … ELSE 0)`
P28	Monthly chargebacks	Conditional aggregation	`UNION ALL` then `FILTER`, `to_char` month
P29	Weather class	Date logic	month range + `CASE` on `avg`
P30	Engagement level	Date logic	half-open range (leap day), `CASE` buckets
P31	Long absences	Date logic	`date - date`, inclusive counting
P32	Absence streaks	Date logic / gaps & islands	`date - row_number()` grouping
P33	Price on a date	Date logic / as-of	`DISTINCT ON` or `LIMIT 1` subquery + `COALESCE` default

#	Problem	Pattern	Key SQL feature
P34	Query quality	Ratios	cast before dividing, `FILTER` share
P35	Ad CTR	Ratios	`NULLIF` + `COALESCE` for zero denominators
P36	Loan share	Ratios	ratio to grand total (scalar subquery or `sum() OVER ()`)
P37	Average selling price	Ratios	range join, weighted average
P38	Next-day return	Ratios / retention	`min` CTE + `LEFT JOIN` on `date + 1`
P39	Second-highest salary	Ranking	`DISTINCT … OFFSET 1` in scalar subquery, `dense_rank`
P40	First-login device	Ranking / first per group	`DISTINCT ON`, `row_number() = 1`
P41	Games so far	Window	running `sum() OVER (PARTITION BY … ORDER BY …)`
P42	Active businesses	Window	`avg() OVER (PARTITION BY)` then `HAVING count(*) > 1`

Chapter 18 TL;DR

- **Method:** clarify grain, keys, NULLs, zero rows, ties and output. Sketch rows, build in evaluation order with CTEs, attack edge cases, then verify and discuss indexes.
- **Filtering:** `WHERE` keeps only `TRUE`. `NULL` comparisons give `UNKNOWN`, so use `IS DISTINCT FROM` when NULLs must count. Anchor and escape regexes. Count characters, not bytes.
- **Aggregation:** filter before grouping. `count(col)` skips NULLs, which is what makes zero-counts work after a LEFT JOIN. An aggregate over nothing returns one NULL row, while `LIMIT 1` over nothing returns no row.
- **Joins:** group by keys, not names. Build "every pair" grids with `CROSS JOIN` + `LEFT JOIN`. Watch for fan-out and prefer `EXISTS` for "has at least one". Use `WITH TIES` or `RANK` for fair top-1 answers.
- **Anti-joins:** `NOT EXISTS` is the safe default. With `LEFT JOIN … IS NULL`, the right-table filters belong in `ON`. `NOT IN` returns nothing if the subquery holds a NULL.
- **Conditional aggregation:** `agg(...) FILTER (WHERE …)` or `SUM(CASE … ELSE 0)` pivots values into columns. Stack differently-dated events with `UNION ALL` first.
- **Dates:** half-open ranges (`>= start AND < next_start`); `date - date` is days; `+ 1` for "inclusive" counts; `date - row_number()` finds consecutive runs; "as of a date" = latest row `<=` the date + a default.
- **Ratios:** cast to `numeric` (or `* 100.0`) before dividing, `NULLIF` the denominator, round last, and use weighted averages when rows carry quantities.
- **Windows:** `dense_rank`/`DISTINCT … OFFSET` for N-th highest (wrap for NULL), `DISTINCT ON`/`row_number` for the first row per group, `sum() OVER (… ORDER BY …)` for running totals, `avg() OVER (PARTITION BY …)` to compare a row with its group.

Next: Chapter 19 — SQL interview problems, part 2, with harder problems: top-N per group, streaks, pivots and relational division.

Chapter 19 — SQL Interview Problems 2: Medium and Hard Patterns

The previous chapter practised the basics (filters, grouping, joins, anti-joins, simple ratios). This chapter covers the medium and hard problems: ranking inside groups, running sums, streaks in ordered data, rows into columns (even when the columns are unknown), tables as small graphs, date buckets, and strings built from rows.

Each problem has a statement in our own words, a self-contained file in `code/` (tables, a tiny invented sample, the solution and often an alternative), the real `psql` (PostgreSQL's command-line client) output (`(null)` marks NULL, SQL's "missing/unknown" value), why it works, and the trap or alternative worth mentioning. All 41 files run on PostgreSQL 14 with `bash tools_run_sql.sh sql '19-*'`.

What you'll learn

- The patterns behind most medium/hard questions, with a template for each (19.11).
- `ROW_NUMBER` vs `RANK` vs `DENSE_RANK` (**window functions** — functions that compute over a set of related rows without collapsing them — that number rows), and why ties change the answer.
- The *gaps-and-islands* trick (`value - ROW_NUMBER()`) for streaks and "N in a row".
- Static pivots (rows turned into columns) with `FILTER`, dynamic pivots with **PL/pgSQL** (PostgreSQL's built-in procedural language, with variables and loops) + `format()` (builds a string from a template), and unpivot (columns turned back into rows) via **JSON** (a text format of nested key/value objects).
- PostgreSQL traps: **integer division** (`int / int` drops the fraction, so `1/3 = 0`), division by zero, `NOT IN` with NULLs, `COUNT(*)` after a `LEFT JOIN`.

Background you may want open: window functions (ranking, frames, gaps and islands), subqueries and set operations, CTEs, and the interview method in Chapter 18.

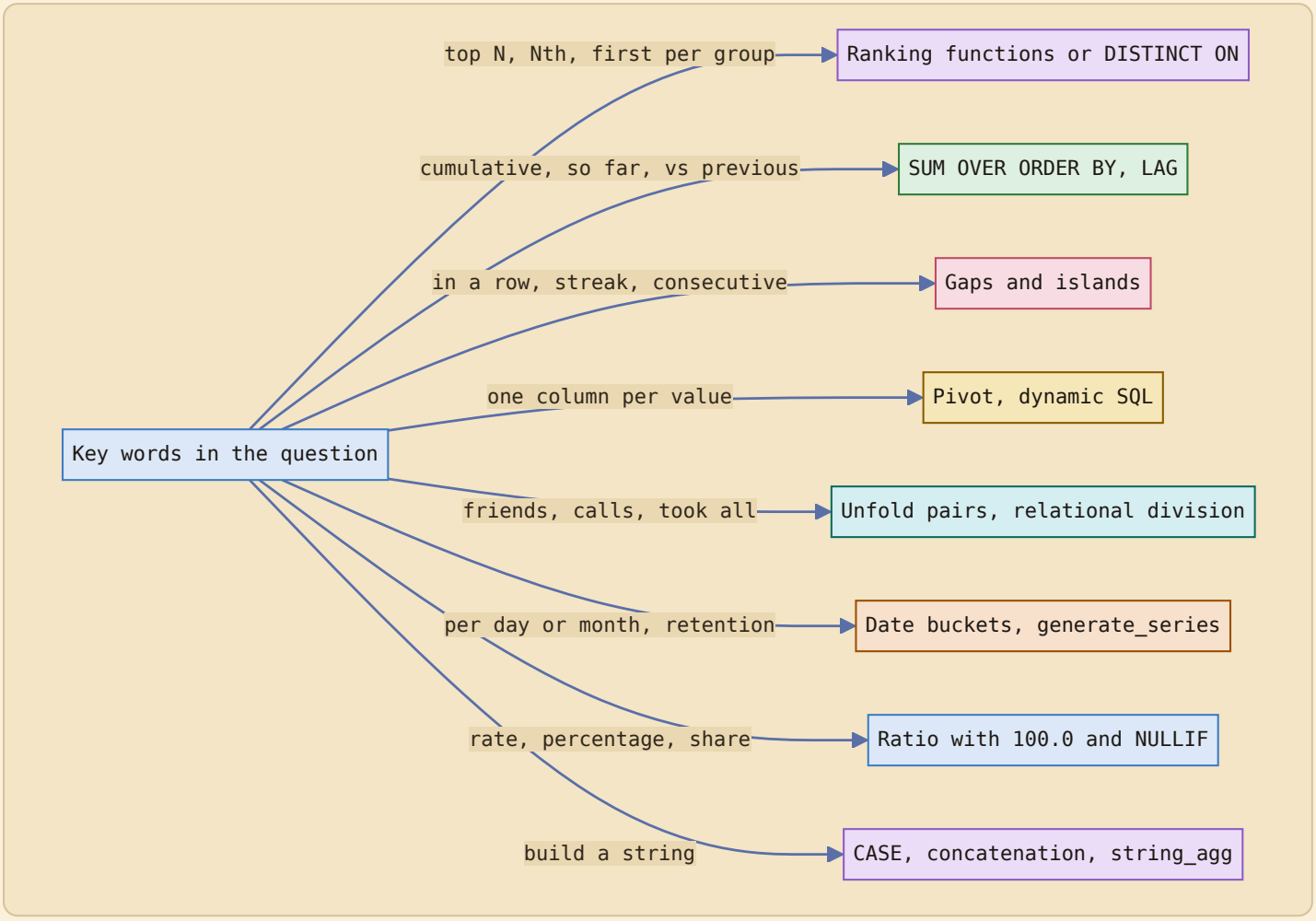


Figure — typical wording in a question and the pattern it usually points to.

19.1 How to attack a medium or hard problem

The Chapter 18 method still applies; three habits matter most here:

Habit	What it means in practice
Build in layers	One **CTE** (common table expression — a named sub-query in a `WITH` clause) per idea ("one row per user-day", then "rank within the day", then "keep rank 1"); run each layer before adding the next.
Say the tie rule out loud	"If two invoices have the same total, which one wins?" decides between `ROW_NUMBER`, `RANK` and `DENSE_RANK`.
Hunt the empty case	Players with no wins, orders with no deliveries, days with no visits: decide whether they appear with 0, with NULL, or not at all.

Run any file on its own with `bash tools_pg.sh psql -d <empty_db> -f code/NN_name.sql`.

19.2 Warm-ups: joins, counts and grids

Three short problems that hide a classic trap each: NULL in a comparison, `COUNT(*)` after an outer join, and "every combination" questions.

19.2.1 Staff rated above 8

Problem. Two tables: `staff` (id, name, team) and `reviews` (one score per staff member). List the names of people whose score is strictly greater than 8.

`code/01_high_rated_staff.sql`

```
CREATE TABLE staff   (staff_id int PRIMARY KEY, full_name text, team text);
CREATE TABLE reviews (staff_id int PRIMARY KEY REFERENCES staff, score int);
INSERT INTO staff VALUES (1,'Asha','Web'),(2,'Bruno','Web'),(3,'Chen','Data'),(4,'Dana','Data');
INSERT INTO reviews VALUES (1,9),(2,8),(3,10),(4,NULL);

SELECT s.full_name
FROM   staff s
JOIN   reviews r ON r.staff_id = s.staff_id
WHERE  r.score > 8
ORDER  BY s.full_name;

-- Edge case: NULL score is neither > 8 nor <= 8
SELECT count(*) FILTER (WHERE score > 8)      AS above,
       count(*) FILTER (WHERE NOT score > 8)  AS not_above,
       count(*)                               AS all_rows
FROM   reviews;
```

Output:

```
 full_name
-----------
 Asha
 Chen
(2 rows)

 above | not_above | all_rows
-------+-----------+----------
     2 |         1 |        4
(1 row)
```

Why it works. The **inner join** (keeps only rows that have a match on both sides) pairs each person with their review; `WHERE r.score > 8` keeps the high ones. Anyone without a review row disappears in the join, which is what "has a rating above 8" means.

Edge case. Dana's score is NULL: `NULL > 8` and `NOT (NULL > 8)` are both *unknown*, so she lands in neither bucket (2 + 1 ≠ 4). "Everyone not above 8" needs `OR score IS NULL`.

19.2.2 Likes per post

Problem. `posts` lists posts; `post_likes` has one row per (post, user who liked it). Show every post with its number of likes, including posts nobody liked, in post order.

`code/02_likes_per_post.sql`

```
CREATE TABLE posts      (post_id int PRIMARY KEY, author text);
CREATE TABLE post_likes (post_id int REFERENCES posts, liker_id int, PRIMARY KEY (post_id, liker_id));
INSERT INTO posts VALUES (10,'ana'),(11,'ben'),(12,'cy');
INSERT INTO post_likes VALUES (10,1),(10,2),(10,3),(11,1);

SELECT p.post_id, count(l.liker_id) AS likes
FROM   posts p
LEFT   JOIN post_likes l ON l.post_id = p.post_id
GROUP  BY p.post_id
ORDER  BY p.post_id;
```

Output:

```
 post_id | likes
---------+-------
      10 |     3
      11 |     1
      12 |     0
(3 rows)
```

Why it works. The `LEFT JOIN` (an **outer join**: keeps every row of the left table, matched or not) keeps post 12 even though it has no likes; its join columns come back as NULL (a **NULL-padded** row). `count(l.liker_id)` counts only non-NULL values, so the padded row contributes 0.

Common mistake. `count(*)` counts the NULL-padded row too, so post 12 would show 1 (the file runs it; the same trap as Chapter 18, P12). After an outer join, count a column from the *optional* side.

19.2.3 Average mark for every learner × module

Problem. For every learner and every module in the catalogue (even ones the learner has no marks in), show the learner's average mark in that module, rounded to 2 decimals; NULL when there are no marks. Sort by learner, then module.

`code/03_course_grid_averages.sql`

```
CREATE TABLE learners (learner_id int PRIMARY KEY, name text);
CREATE TABLE modules  (module text PRIMARY KEY);
CREATE TABLE marks    (learner_id int, module text, mark int);
INSERT INTO learners VALUES (1,'Ivy'),(2,'Omar');
INSERT INTO modules  VALUES ('SQL'),('Stats');
INSERT INTO marks    VALUES (1,'SQL',70),(1,'SQL',85),(1,'Stats',60),(2,'SQL',90);

SELECT l.learner_id, l.name, m.module,
       round(avg(k.mark), 2) AS avg_mark
FROM   learners l
CROSS  JOIN modules m
LEFT   JOIN marks k ON k.learner_id = l.learner_id AND k.module = m.module
GROUP  BY l.learner_id, l.name, m.module
ORDER  BY l.learner_id, m.module;
```

Output:

```
 learner_id | name | module | avg_mark
------------+------+--------+----------
          1 | Ivy  | SQL    |    77.50
          1 | Ivy  | Stats  |    60.00
          2 | Omar | SQL    |    90.00
          2 | Omar | Stats  |   (null)
(4 rows)
```

Why it works. "Every A with every B" is a `CROSS JOIN` (every row of one table paired with every row of the other) — it builds the full grid of 2 × 2 pairs first. The `LEFT JOIN` then attaches marks where they exist, and `avg()` of only NULLs is NULL, which is exactly the "no marks" answer (Omar/Stats).

Alternative. If the question only wants modules the learner actually took, drop the cross join and group the `marks` table directly. Always ask which one is meant.

19.3 Nth highest, top-N per group and "first row per group"

The ranking family. The three ranking functions differ only on ties:

Scores	ROW_NUMBER	RANK	DENSE_RANK
95, 95, 90	1, 2, 3	1, 1, 3	1, 1, 2

Use `ROW_NUMBER` when exactly one row must win, `RANK` when "all tied for first" should be kept, and `DENSE_RANK` when the question says "top N *distinct* values". PostgreSQL's `DISTINCT ON (cols)` is a compact shortcut for "first row per group": it keeps the first row of each group of equal `cols` values, in `ORDER BY` order.

19.3.1 Second-fastest lap time

Problem. Each runner has one lap time, in seconds. Return the second-smallest *distinct* time as one row, or a single NULL row when there is no second value.

`code/04_second_fastest.sql`

```sql
CREATE TABLE laps (runner_id int PRIMARY KEY, seconds int NOT NULL);
INSERT INTO laps VALUES (1,62),(2,58),(3,58),(4,65),(5,61);

-- Solution: scalar subquery -> NULL when OFFSET skips past the end
SELECT (SELECT DISTINCT seconds FROM laps
        ORDER  BY seconds
        OFFSET 1 LIMIT 1) AS second_fastest;

-- Alternative: DENSE_RANK generalises to the N-th
SELECT max(seconds) FILTER (WHERE dr = 2) AS second_fastest
FROM  (SELECT seconds, dense_rank() OVER (ORDER BY seconds) AS dr FROM laps) t;

-- Edge case: only one distinct time left
DELETE FROM laps WHERE seconds <> 58;
SELECT (SELECT DISTINCT seconds FROM laps ORDER BY seconds OFFSET 1 LIMIT 1) AS second_fastest;
```

Output (both formulations give 61; after the `DELETE` the answer is a NULL row):

```
 second_fastest
----------------
             61
(1 row)
...
 second_fastest
----------------
         (null)
(1 row)
```

Why it works. Same idea as Chapter 18, P39: `DISTINCT` collapses the two 58s, `OFFSET 1 LIMIT 1` takes the next value, and the **scalar-subquery** wrapper (a sub-query used as a single value) turns "no row" into

one NULL row.

Alternatives. `min(seconds) WHERE seconds > (SELECT min(seconds) …)` (also in the file) works only for N = 2. `DENSE_RANK` handles any N (`dr = N`), and wrapping it in `max(...) FILTER (...)` again turns "no row" into NULL.

19.3.2 Top three distinct scores per subject

Problem. Pupils' scores per subject. For each subject, list every pupil whose score is one of the three highest *different* scores in that subject (ties included).

`code/05_top3_per_subject.sql`

```
CREATE TABLE subjects (subject_id int PRIMARY KEY, title text);
CREATE TABLE results  (result_id int PRIMARY KEY, pupil text, score int,
                       subject_id int REFERENCES subjects);
INSERT INTO subjects VALUES (1,'Maths'),(2,'History');
INSERT INTO results VALUES
 (1,'Ann',95,1),(2,'Bob',95,1),(3,'Cal',90,1),(4,'Dee',82,1),(5,'Eve',70,1),
 (6,'Fay',88,2),(7,'Gus',75,2);

WITH ranked AS (
  SELECT s.title, r.pupil, r.score,
         dense_rank() OVER (PARTITION BY r.subject_id ORDER BY r.score DESC) AS dr
  FROM   results r
  JOIN   subjects s USING (subject_id)
)
SELECT title AS subject, pupil, score
FROM   ranked
WHERE  dr <= 3
ORDER  BY subject, score DESC, pupil;

-- Alternative: correlated subquery "fewer than 3 distinct higher scores"
SELECT s.title AS subject, r.pupil, r.score
FROM   results r
JOIN   subjects s USING (subject_id)
WHERE  (SELECT count(DISTINCT r2.score) FROM results r2
        WHERE r2.subject_id = r.subject_id AND r2.score > r.score) < 3
ORDER  BY subject, score DESC, pupil;
```

Output (all queries return these rows):

```
 subject | pupil | score
---------+-------+-------
 History | Fay   |    88
 History | Gus   |    75
 Maths   | Ann   |    95
 Maths   | Bob   |    95
 Maths   | Cal   |    90
 Maths   | Dee   |    82
(6 rows)
```

Why it works. `DENSE_RANK` gives Ann and Bob rank 1, Cal rank 2, Dee rank 3, so four pupils appear for Maths — ties don't use up places. History has only two pupils, so both appear.

Alternative. The **correlated** version (its sub-query refers to the outer row, so it re-runs for each row) keeps a row if fewer than three distinct scores in its subject beat it: **portable** (works on any SQL database), but O(n²) per group (work grows with the square of the row count). With `RANK`, Dee would get rank 4 and vanish — a common wrong answer.

19.3.3 One home office per employee

Problem. Employees can be assigned to several offices; exactly one of them is flagged `'Y'` as the home office. Someone with only one office has that office as home, whatever the flag says. Return each employee with their home office.

`code/06_home_office.sql`

```
CREATE TABLE desk_assignments (emp_id int, office_id int, is_home char(1) NOT NULL CHECK (is_home IN ('Y','N')),
                               PRIMARY KEY (emp_id, office_id));
INSERT INTO desk_assignments VALUES
 (1,100,'N'),(1,200,'Y'),(1,300,'N'),   -- several offices, one flagged
 (2,100,'N'),                            -- single office, flag ignored
 (3,300,'Y');

-- Solution: DISTINCT ON keeps the first row of each emp_id in ORDER BY order
SELECT DISTINCT ON (emp_id) emp_id, office_id
FROM   desk_assignments
ORDER  BY emp_id, is_home DESC;      -- 'Y' sorts after 'N', so DESC puts it first

-- Alternative: the rule written out literally
SELECT emp_id, office_id FROM desk_assignments WHERE is_home = 'Y'
UNION
SELECT emp_id, min(office_id) FROM desk_assignments
GROUP  BY emp_id HAVING count(*) = 1
ORDER  BY emp_id;
```

Output (all queries return these rows):

```
 emp_id | office_id
--------+-----------
      1 |       200
      2 |       100
      3 |       300
(3 rows)
```

Why it works. `DISTINCT ON (emp_id)` keeps each employee's first row under `ORDER BY emp_id, is_home DESC`; `'Y' > 'N'`, so a flagged office sorts first, and a single office is kept as is. The flag is `NOT NULL` on purpose: PostgreSQL sorts NULLs *first* under `DESC`, so a NULL flag would beat `'Y'` (add `NULLS LAST` if NULLs are possible). Portable form: `ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY is_home DESC) = 1`.

Alternative / edge case. The `UNION` version states the rule literally. If the data breaks the rule (several offices, none flagged), the ranking version silently picks *some* office and the `UNION` version returns nothing for that person — choose your failure.

19.3.4 The most expensive invoice's lines

Problem. `items` holds unit prices; `line_items` holds (invoice, item, quantity). Find the invoice with the largest total — smallest invoice id wins a tie — and list its lines with quantity and line price.

`code/07_largest_invoice.sql`

```
CREATE TABLE items      (item_id int PRIMARY KEY, unit_price int);
CREATE TABLE line_items (invoice_id int, item_id int REFERENCES items, qty int,
                         PRIMARY KEY (invoice_id, item_id));
INSERT INTO items VALUES (1,10),(2,25),(3,4);
INSERT INTO line_items VALUES
 (501,1,3),(501,3,5),        -- 30 + 20 = 50
 (502,2,2),                  -- 50  (tie with 501)
 (503,1,1),(503,3,2);        -- 18

WITH priced AS (
  SELECT l.invoice_id, l.item_id, l.qty, l.qty * i.unit_price AS line_total,
         sum(l.qty * i.unit_price) OVER (PARTITION BY l.invoice_id) AS invoice_total
  FROM   line_items l JOIN items i USING (item_id)
), winner AS (
  SELECT invoice_id FROM priced
  ORDER  BY invoice_total DESC, invoice_id
  LIMIT  1
)
SELECT item_id, qty, line_total
FROM   priced
WHERE  invoice_id = (SELECT invoice_id FROM winner)
ORDER  BY item_id;
```

Output:

```
 item_id | qty | line_total
---------+-----+------------
       1 |   3 |         30
       3 |   5 |         20
(2 rows)
```

Why it works. A window `sum(...) OVER (PARTITION BY invoice_id)` puts each invoice's total next to each of its lines without collapsing them. The `winner` CTE sorts by total descending, then id ascending, and keeps one row. Invoices 501 and 502 both total 50, so 501 wins on id.

Alternative. Group `line_items` by invoice to get totals, pick the winner with `ORDER BY ... LIMIT 1`, then join back — the same logic with one more pass. `FETCH FIRST 1 ROW WITH TIES` is the tool if the question instead wanted *all* tied invoices.

19.3.5 Confirmed or waitlisted

Problem. Flights have a seat count; bookings have a passenger, a flight and a unique booking time. A booking is confirmed if, in booking-time order, it is within the flight's seat count; otherwise it is waitlisted.

`code/08_seat_status.sql`

```sql
CREATE TABLE flights  (flight_id int PRIMARY KEY, seats int);
CREATE TABLE bookings (passenger_id int PRIMARY KEY, flight_id int, booked_at timestamp UNIQUE);
INSERT INTO flights VALUES (7,2),(8,1),(9,0);
INSERT INTO bookings VALUES
 (1,7,'2026-03-01 09:00'),(2,7,'2026-03-01 08:00'),(3,7,'2026-03-02 10:00'),
 (4,8,'2026-03-03 12:00'),(5,8,'2026-03-01 12:00'),
 (6,9,'2026-03-01 07:00');

SELECT b.passenger_id,
       CASE WHEN row_number() OVER (PARTITION BY b.flight_id ORDER BY b.booked_at) <= f.seats
            THEN 'Confirmed' ELSE 'Waitlist' END AS status
FROM   bookings b
JOIN   flights f USING (flight_id)
ORDER  BY b.passenger_id;
```

Output:

```
 passenger_id |  status
--------------+-----------
            1 | Confirmed
            2 | Confirmed
            3 | Waitlist
            4 | Waitlist
            5 | Confirmed
            6 | Waitlist
(6 rows)
```

Why it works. `row_number()` numbers the bookings of each flight in time order; comparing that number with `seats` gives the status in one pass. On flight 7, passenger 2 booked first, then 1, then 3 (waitlisted). Flight 9 has zero seats, so even the first booking waits.

Edge case. Booking times are unique here, so `RANK` and `ROW_NUMBER` agree. If two bookings could share a timestamp, `RANK` would confirm both (possibly overbooking); `ROW_NUMBER` with a tie-breaker such as `passenger_id` keeps the count exact.

19.3.6 Pupils who are never top or bottom

Problem. A "middle-of-the-pack" pupil sat at least one test and never had the highest or the lowest score in any test they sat (ties for top or bottom count as top or bottom). List them by id.

`code/09_middle_of_the_pack.sql`

```
CREATE TABLE pupils (pupil_id int PRIMARY KEY, name text);
CREATE TABLE tests  (test_id int, pupil_id int, score int, PRIMARY KEY (test_id, pupil_id));
INSERT INTO pupils VALUES (1,'Ada'),(2,'Ben'),(3,'Cleo'),(4,'Dev'),(5,'Eli'),(6,'Fin');
INSERT INTO tests VALUES
 (10,1,60),(10,2,75),(10,3,90),(10,5,70),
 (20,1,55),(20,2,80),(20,4,80),(20,5,62);   -- Ben and Dev tie for top of test 20; Fin sat nothing

WITH marked AS (
  SELECT pupil_id,
         score = max(score) OVER (PARTITION BY test_id)
      OR score = min(score) OVER (PARTITION BY test_id) AS at_extreme
  FROM   tests
)
SELECT p.pupil_id, p.name
FROM   pupils p
JOIN   marked m USING (pupil_id)
GROUP  BY p.pupil_id, p.name
HAVING NOT bool_or(m.at_extreme)
ORDER  BY p.pupil_id;
```

Output:

```
 pupil_id | name
----------+------
        5 | Eli
(1 row)
```

Why it works. Two window aggregates put each test's max and min on every row, so `at_extreme` is true when a pupil matches either. Grouping per pupil and keeping `NOT bool_or(at_extreme)` (`bool_or` is true if any value in the group is true) means "no test where they were extreme". The inner join to `marked` drops Fin, who sat nothing.

Alternative. Mark extremes with `RANK() ... DESC = 1 OR RANK() ... ASC = 1` and exclude those pupils — with `NOT EXISTS`, not `NOT IN`, which returns nothing if the subquery yields a NULL.

19.3.7 Viewers who became streamers

Problem. A sessions table records each user's sessions as `viewer` or `streamer`. For users whose earliest session was as a viewer and who streamed at least once, return how many streaming sessions they have. Sort by that count, then user id, both descending.

`code/10_viewers_then_streamers.sql`

```
CREATE TABLE sessions (session_id int PRIMARY KEY, user_id int, started_at timestamp,
                       kind text CHECK (kind IN ('viewer','streamer')));
INSERT INTO sessions VALUES
 (1,101,'2026-01-01 10:00','viewer'),(2,101,'2026-01-02 10:00','streamer'),(3,101,'2026-01-05 10:00','streamer'),
 (4,102,'2026-01-01 09:00','streamer'),(5,102,'2026-01-03 09:00','viewer'),
 (6,103,'2026-01-02 08:00','viewer'),(7,103,'2026-01-04 08:00','streamer'),
 (8,104,'2026-01-02 08:00','viewer');                                  -- never streamed

WITH firsts AS (
  SELECT DISTINCT ON (user_id) user_id, kind AS first_kind
  FROM   sessions
  ORDER  BY user_id, started_at
)
SELECT s.user_id, count(*) FILTER (WHERE s.kind = 'streamer') AS streams
FROM   sessions s
JOIN   firsts f USING (user_id)
WHERE  f.first_kind = 'viewer'
GROUP  BY s.user_id
HAVING count(*) FILTER (WHERE s.kind = 'streamer') > 0
ORDER  BY streams DESC, s.user_id DESC;
```

Output:

```
 user_id | streams
---------+---------
     101 |       2
     103 |       1
(2 rows)
```

Why it works. `DISTINCT ON` finds each user's first session kind (user 102 started as a streamer, so is out). Joining back and counting with `FILTER (WHERE kind = 'streamer')` gives the streams; `HAVING ... > 0` drops user 104, who never streamed.

Alternative. `first_value(kind) OVER (PARTITION BY user_id ORDER BY started_at)` (the value from the first row of the window) puts the first kind on every row (the file prints it: viewer, streamer, viewer, viewer).

19.3.8 First year on loan (keep the ties)

Problem. A library keeps yearly loan statistics per branch and book: number of loans and fee per loan. For each book, report its first year on loan with the loans and fee from that year — if the first year has several rows, report all of them.

`code/11_first_year_loans.sql`

```
CREATE TABLE loan_stats (branch_id int, book_id int, yr int, loans int, fee int,
                         PRIMARY KEY (branch_id, book_id, yr));
INSERT INTO loan_stats VALUES
 (1,100,2023,12,2),(1,100,2024,20,2),
 (2,100,2023, 5,3),                    -- second row for book 100 in its first year
 (1,200,2025, 7,1);

-- Solution: RANK keeps every row tied for the earliest year
SELECT book_id, yr AS first_year, loans, fee
FROM  (SELECT *, rank() OVER (PARTITION BY book_id ORDER BY yr) AS rk FROM loan_stats) t
WHERE  rk = 1
ORDER  BY book_id, loans DESC;

-- Pitfall: ROW_NUMBER (or DISTINCT ON) silently drops one of the 2023 rows of book 100
SELECT DISTINCT ON (book_id) book_id, yr AS first_year, loans, fee
FROM   loan_stats
ORDER  BY book_id, yr;
```

Output:

```
 book_id | first_year | loans | fee
---------+------------+-------+-----
     100 |       2023 |    12 |   2
     100 |       2023 |     5 |   3
     200 |       2025 |     7 |   1
(3 rows)

 book_id | first_year | loans | fee
---------+------------+-------+-----
     100 |       2023 |    12 |   2
     200 |       2025 |     7 |   1
(2 rows)
```

Why it works. `rank() OVER (PARTITION BY book_id ORDER BY yr)` gives rank 1 to *every* row in the earliest year, so both 2023 rows of book 100 survive. The classic equivalent is "find `min(yr)` per book, then join back on (book, year)".

Edge case. The second query is the tempting shortcut and it is wrong for this data: `DISTINCT ON` (like `ROW_NUMBER`) keeps exactly one row per book, losing branch 2's 2023 figures. Whenever "first" can tie, prefer `RANK` or the join-back.

19.3.9 Busiest collaborators (all ties)

Problem. Each row says one person invited another to a project and it was accepted. Count, for every person, the distinct projects they were part of (as inviter *or* invitee) and return everyone tied for the highest count.

code/12_busiest_collaborators.sql

```
CREATE TABLE collabs (inviter int, invitee int, project_id int, accepted_on date,
                      PRIMARY KEY (inviter, invitee, project_id));
INSERT INTO collabs VALUES
 (1,2,900,'2026-02-01'),(1,3,900,'2026-02-02'),(2,3,901,'2026-02-03'),
 (4,1,902,'2026-02-04'),(3,4,903,'2026-02-05'),(1,4,903,'2026-02-06');

WITH involvement AS (
  SELECT inviter AS person, project_id FROM collabs
  UNION                                     -- UNION also removes duplicate (person, project)
  SELECT invitee, project_id FROM collabs
), counts AS (
  SELECT person, count(*) AS projects,
         rank() OVER (ORDER BY count(*) DESC) AS rk
  FROM   involvement
  GROUP  BY person
)
SELECT person, projects FROM counts WHERE rk = 1 ORDER BY person;
```

Output:

```
 person | projects
--------+----------
      1 |        3
      3 |        3
(2 rows)
```

Why it works. Two `SELECT`s turn each row into two "person took part in project" facts; `UNION` removes repeats (person 1 twice in project 900). A window `rank()` over the grouped counts marks every top scorer with 1 — persons 1 and 3, three projects each.

Alternative. `WHERE projects = (SELECT max(projects) FROM counts)` is equally correct and reads well; `ORDER BY ... LIMIT 1` would be wrong because it drops ties.

19.4 Running totals and row-to-row comparisons

A running total is `SUM(...) OVER (PARTITION BY group ORDER BY time)`; "compared with last time" is `LAG` (a window function returning a value from the previous row; `LEAD` returns the next row's). Chapter 8 explains **frames** (the range of rows a window function looks at for each row) in depth (§8.7).

19.4.1 Running total per team

Problem. Each row is one player's points for a team on a day ((team, day) is unique). Show, for each team and day, the team's cumulative points up to and including that day.

code/13_running_total_by_team.sql

```
CREATE TABLE team_points (player text, team char(1), day date, points int, PRIMARY KEY (team, day));
INSERT INTO team_points VALUES
 ('Lia','A','2026-05-01',10),('Max','B','2026-05-01',7),
 ('Noa','A','2026-05-03',4), ('Oli','B','2026-05-02',9),
 ('Pia','A','2026-05-06',6);

SELECT team, day,
       sum(points) OVER (PARTITION BY team ORDER BY day) AS running_total
FROM   team_points
ORDER  BY team, day;
```

Output (the file's alternative returns the same rows):

```
 team |    day     | running_total
------+------------+---------------
 A    | 2026-05-01 |            10
 A    | 2026-05-03 |            14
 A    | 2026-05-06 |            20
 B    | 2026-05-01 |             7
 B    | 2026-05-02 |            16
(5 rows)
```

Why it works. `PARTITION BY team` restarts the sum per team; `ORDER BY day` makes it cumulative. With an `ORDER BY` and no explicit frame the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

Alternative / edge case. A self-join summing rows of the same team with `day <=` this day (in the file) works on old databases but is O(n²). If days could repeat, the default `RANGE` frame adds all same-day peers at once; use `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` for row-by-row totals.

19.4.2 Last person who fits in the lift

Problem. People queue for a lift with a 500 kg limit; each has a weight and a queue position (`turn`). People board strictly in order. Return the last person who can board without the total exceeding the limit (assume the first person always fits).

`code/14_lift_capacity.sql`

```sql
CREATE TABLE lift_queue (person_id int PRIMARY KEY, name text, kg int, turn int UNIQUE);
INSERT INTO lift_queue VALUES
 (1,'Rae',90,3),(2,'Sid',120,1),(3,'Tom',200,2),(4,'Uma',60,5),(5,'Vik',85,4);

WITH boarding AS (
  SELECT name, turn, sum(kg) OVER (ORDER BY turn) AS load_kg
  FROM   lift_queue
)
SELECT name, load_kg
FROM   boarding
WHERE  load_kg <= 500
ORDER  BY turn DESC
LIMIT  1;
```

Output:

```
 name | load_kg
------+---------
 Vik  |     495
(1 row)
```

Why it works. The running sum in queue order gives the load after each person boards: 120, 320, 410, 495, 555. Keep loads ≤ 500 and take the latest in the queue — Vik. Weights are positive, so the sum only grows and "latest turn" equals "largest load".

Alternative. A correlated subquery (in the file) recomputes each prefix sum — correct, O(n²), and the answer to "how would you do this without window functions?".

19.4.3 Performance and improvement rate

Problem. Review rounds per person (round number, score 1–100). Report, per person, (a) *performance* = average of score ÷ round, and (b) *improvement rate* = percentage of that person's reviews where the score beat the previous round's score **and** is above 50. Round both to 2 decimals.

`code/15_review_trends.sql`

```sql
CREATE TABLE review_rounds (person text, rnd int, score int);
INSERT INTO review_rounds VALUES
 ('Kim',1,40),('Kim',2,55),('Kim',3,70),('Kim',4,65),
 ('Lou',1,80),('Lou',2,45);

WITH prev AS (
  SELECT person, rnd, score,
         lag(score) OVER (PARTITION BY person ORDER BY rnd) AS prev_score
  FROM   review_rounds
)
SELECT person,
       round(avg(score::numeric / rnd), 2)                              AS performance,
       round(100.0 * count(*) FILTER (WHERE score > 50 AND score > prev_score)
             / count(*), 2)                                             AS improvement_pct
FROM   prev
GROUP  BY person
ORDER  BY person;
```

Output:

```
 person | performance | improvement_pct
--------+-------------+-----------------
 Kim    |       26.77 |           50.00
 Lou    |       51.25 |            0.00
(2 rows)
```

Why it works. `lag(score)` puts the previous round's score on each row (NULL for round 1, and `score > NULL` is not true, so round 1 never counts). `count(*) FILTER (...)` counts qualifying rows; multiplying by `100.0` first keeps the maths in `numeric`.

Common mistake. In PostgreSQL `55 / 2` is `27`: integer division truncates, and `avg(score / rnd)` gives Kim 26.50 instead of 26.77 (shown in the file). Cast one side first, as in Chapter 18, P34.

19.5 Gaps and islands: consecutive rows and streaks

An *island* is a maximal run of rows that belong together in order (consecutive ids, days or wins); the *gaps* are the breaks. If values rise by exactly 1 inside a run, `value - ROW_NUMBER()` is **constant** inside it and jumps at every gap, so grouping by it gives one group per island.

id	visitors ≥ 100?	row_number among busy rows	id − row_number
1	yes	1	0
3	yes	2	1
4	yes	3	1
5	yes	4	1
7	yes	5	2

19.5.1 Runs of busy days at the gate

Problem. Daily visitor counts with an increasing id. Return every row that belongs to a run of three or more *consecutive ids* where each row has at least 100 visitors, ordered by date.

`code/16_busy_gate_runs.sql`

```sql
CREATE TABLE gate_counts (id int PRIMARY KEY, visit_day date UNIQUE, visitors int);
INSERT INTO gate_counts VALUES
 (1,'2026-07-01',150),(2,'2026-07-02', 90),(3,'2026-07-03',120),(4,'2026-07-04',130),
 (5,'2026-07-05',101),(6,'2026-07-06', 20),(7,'2026-07-07',300),(8,'2026-07-08',310);

WITH busy AS (
  SELECT id, visit_day, visitors,
         id - row_number() OVER (ORDER BY id) AS island
  FROM   gate_counts
  WHERE  visitors >= 100
), sized AS (
  SELECT *, count(*) OVER (PARTITION BY island) AS run_len FROM busy
)
SELECT id, visit_day, visitors, island, run_len
FROM   sized
WHERE  run_len >= 3
ORDER  BY visit_day;
```

Output:

```
 id | visit_day  | visitors | island | run_len
----+------------+----------+--------+---------
  3 | 2026-07-03 |      120 |      1 |       3
  4 | 2026-07-04 |      130 |      1 |       3
  5 | 2026-07-05 |      101 |      1 |       3
(3 rows)
```

Why it works. Filtering first leaves only busy rows; `id - row_number()` labels ids 3, 4, 5 with island 1, while id 1 (island 0) and ids 7–8 (island 2) are too short. `count(*) OVER (PARTITION BY island)` attaches each island's length to its rows, so no join-back is needed.

Alternative. Group by `island` with `HAVING count(*) >= 3` and join those islands back to the rows — the same idea in two steps. A `LAG`/`LEAD` solution needs three cases (row is first, middle or last of a run) and gets messy.

19.5.2 Values repeated three times in a row

Problem. A sequence of readings ordered by id (ids may have gaps). Return every value that occurs at least three times consecutively.

`code/17_triple_repeats.sql`

```sql
CREATE TABLE readings (id int PRIMARY KEY, val int);
INSERT INTO readings VALUES (1,5),(2,5),(3,5),(4,2),(5,7),(7,7),(9,7),(10,5),(11,5);

-- Solution: compare each row with its neighbours (ids have gaps, order still defines "in a row")
SELECT DISTINCT val
FROM  (SELECT val,
              lag(val)  OVER (ORDER BY id) AS before,
              lead(val) OVER (ORDER BY id) AS after
       FROM   readings) t
WHERE  val = before AND val = after
ORDER  BY val;

-- Alternative: islands via the difference of two ROW_NUMBERs, works for any run length k
SELECT DISTINCT val
FROM  (SELECT val,
              row_number() OVER (ORDER BY id)
            - row_number() OVER (PARTITION BY val ORDER BY id) AS island
       FROM   readings) t
GROUP  BY val, island
HAVING count(*) >= 3
ORDER  BY val;
```

Output (all queries return these rows):

```
 val
-----
   5
   7
(2 rows)
```

Why it works. `lag` and `lead` expose each row's neighbours in id order; a row equal to both is the middle of a run of three or more. The 7s sit at ids 5, 7, 9 — id gaps don't matter, because "consecutive" means consecutive *in the ordering*.

Alternative. Overall position minus position among equal values is constant within each run; `GROUP BY` it with `HAVING count(*) >= k` works for any k. The two versions disagree on NULLs: three NULLs in a row fail `val = before` but form one island (`PARTITION BY val` groups NULLs), so decide whether a missing reading can repeat.

19.5.3 Members active five days in a row

Problem. A login log (duplicates possible, several logins per day) and a members table. Return members who logged in on five or more consecutive calendar days, ordered by id.

`code/18_five_day_streaks.sql`

```sql
CREATE TABLE members (member_id int PRIMARY KEY, name text);
CREATE TABLE logins  (member_id int, login_day date);          -- duplicates allowed
INSERT INTO members VALUES (1,'Ana'),(2,'Bo'),(3,'Cy');
INSERT INTO logins VALUES
 (1,'2026-06-01'),(1,'2026-06-02'),(1,'2026-06-02'),(1,'2026-06-03'),(1,'2026-06-04'),(1,'2026-06-05'),
 (2,'2026-06-01'),(2,'2026-06-02'),(2,'2026-06-04'),(2,'2026-06-05'),(2,'2026-06-06'),
 (3,'2026-06-28'),(3,'2026-06-29'),(3,'2026-06-30'),(3,'2026-07-01'),(3,'2026-07-02');

WITH days AS (SELECT DISTINCT member_id, login_day FROM logins),
islands AS (
  SELECT member_id, login_day,
         login_day - (row_number() OVER (PARTITION BY member_id ORDER BY login_day))::int AS anchor
  FROM   days
)
SELECT m.member_id, m.name
FROM   islands i JOIN members m USING (member_id)
GROUP  BY m.member_id, m.name, i.anchor
HAVING count(*) >= 5
ORDER  BY m.member_id;

-- Alternative: the 5th distinct day after this one is exactly 4 days later
SELECT DISTINCT member_id
FROM  (SELECT member_id, login_day,
              lead(login_day, 4) OVER (PARTITION BY member_id ORDER BY login_day) AS day5
       FROM  (SELECT DISTINCT member_id, login_day FROM logins) d) t
WHERE  day5 = login_day + 4
ORDER  BY member_id;
```

Output:

```
 member_id | name
-----------+------
         1 | Ana
         3 | Cy
(2 rows)

 member_id
-----------
         1
         3
(2 rows)
```

Why it works. First reduce to *distinct* days — duplicates would break the numbering. Then `login_day - row_number` (a date minus an integer is a date in PostgreSQL) gives the same "anchor" date for every day

in a streak. Cy's streak crosses June → July and still works, because date arithmetic knows month lengths.

Alternative. `lead(login_day, 4)` looks four distinct days ahead; if that day is exactly `login_day + 4`, the five days in between must all be present. It only handles a fixed length; islands also give each streak's length.

19.5.4 Interview candidates from contest medals

Problem. Each contest row stores the gold, silver and bronze winners; contest ids are consecutive. A coder qualifies if they reached the podium (any colour) in at least three back-to-back contests, **or** took gold at least three times overall, in any contests. Return candidates' names and emails.

`code/19_medal_candidates.sql`

```
CREATE TABLE contests (contest_id int PRIMARY KEY, gold int, silver int, bronze int);
CREATE TABLE coders   (coder_id int PRIMARY KEY, name text, email text);
INSERT INTO coders VALUES (1,'Mia','mia@example.com'),(2,'Ned','ned@example.com'),
                          (3,'Oz','oz@example.com'),(4,'Pat','pat@example.com');
INSERT INTO contests VALUES
 (51,1,2,3),(52,2,1,4),(53,3,4,1),   -- Mia medals in 51,52,53
 (54,4,3,2),(55,4,2,3),(56,4,3,1);   -- Pat wins gold 3 times

WITH medals AS (
  SELECT contest_id, m.coder_id, m.kind
  FROM   contests c
  CROSS  JOIN LATERAL (VALUES (c.gold,'gold'),(c.silver,'silver'),(c.bronze,'bronze')) AS m(coder_id, kind)
), runs AS (
  SELECT coder_id
  FROM  (SELECT coder_id, contest_id - row_number() OVER (PARTITION BY coder_id ORDER BY contest_id) AS island
         FROM medals) t
  GROUP  BY coder_id, island
  HAVING count(*) >= 3
), golds AS (
  SELECT coder_id FROM medals WHERE kind = 'gold'
  GROUP  BY coder_id HAVING count(*) >= 3
)
SELECT name, email
FROM   coders
WHERE  coder_id IN (SELECT coder_id FROM runs UNION SELECT coder_id FROM golds)
ORDER  BY name;
```

Output:

```
 name |      email
------+-----------------
 Mia  | mia@example.com
 Oz   | oz@example.com
 Pat  | pat@example.com
(3 rows)
```

Why it works. `CROSS JOIN LATERAL (VALUES ...)` (`VALUES` builds a small inline table; `LATERAL` lets it refer to columns of the current row `c`) unpivots the medal columns into one row per (contest, coder, kind). Rule 1 is an island on `contest_id - row_number()` per coder; rule 2 is `GROUP BY ... HAVING count(*) >= 3` on golds; `IN (... UNION ...)` combines them. Mia medals in 51–53, Oz in 53–56, Pat has three golds; Ned's medals (51, 52, 54, 55) break at 53.

Edge case. A coder cannot win two medals in the same contest here; if they could, deduplicate `(coder_id, contest_id)` before numbering, or the island arithmetic breaks.

19.5.5 Longest winning streak

Problem. Match results per player (`win`, `draw`, `loss`) by date. For every player, return the length of their longest run of consecutive wins — 0 for a player who never won.

`code/20_longest_win_streak.sql`

```sql
CREATE TABLE games (player_id int, played_on date, outcome text CHECK (outcome IN ('win','draw','loss')),
                    PRIMARY KEY (player_id, played_on));
INSERT INTO games VALUES
 (1,'2026-01-01','win'),(1,'2026-01-02','win'),(1,'2026-01-03','loss'),
 (1,'2026-01-04','win'),(1,'2026-01-05','win'),(1,'2026-01-06','win'),
 (2,'2026-01-01','draw'),(2,'2026-01-02','win'),
 (3,'2026-01-01','loss');

WITH tagged AS (
  SELECT player_id, outcome,
         row_number() OVER (PARTITION BY player_id ORDER BY played_on)
       - row_number() OVER (PARTITION BY player_id, outcome ORDER BY played_on) AS island
  FROM   games
), win_runs AS (
  SELECT player_id, count(*) AS run_len
  FROM   tagged
  WHERE  outcome = 'win'
  GROUP  BY player_id, island
)
SELECT p.player_id, coalesce(max(w.run_len), 0) AS longest_streak
FROM  (SELECT DISTINCT player_id FROM games) p
LEFT  JOIN win_runs w USING (player_id)
GROUP  BY p.player_id
ORDER  BY p.player_id;

-- Alternative: a running count of non-wins gives every streak its own label
SELECT player_id, max(len) AS longest_streak
FROM  (SELECT player_id, breaks, count(*) FILTER (WHERE outcome = 'win') AS len
       FROM  (SELECT player_id, outcome,
                     count(*) FILTER (WHERE outcome <> 'win')
                       OVER (PARTITION BY player_id ORDER BY played_on) AS breaks
              FROM games) g
       GROUP  BY player_id, breaks) s
GROUP  BY player_id
ORDER  BY player_id;
```

Output (all queries return these rows):

```
 player_id | longest_streak
-----------+----------------
         1 |              3
         2 |              1
         3 |              0
(3 rows)
```

Why it works. Row number over all games minus row number within the same outcome is constant inside each run of equal outcomes. Counting `win` rows per (player, island) gives each streak's length and `max` the longest; the `LEFT JOIN` from all players plus `coalesce(..., 0)` keeps player 3, who never won.

Alternative. A running count of non-wins (`count(*) FILTER (WHERE outcome <> 'win') OVER (...)`) stays the same across a winning streak and bumps at each draw or loss, so it also labels streaks. Grouping by it and counting wins returns 0 naturally for groups without wins.

19.5.6 Deposits over the limit two months running

Problem. Accounts have a monthly income limit. A movements table holds deposits (`in`) and withdrawals (`out`). Flag accounts whose total deposits in a calendar month exceed the limit in at least two consecutive months.

`code/21_income_over_limit.sql`

```sql
CREATE TABLE accounts  (account_id int PRIMARY KEY, monthly_limit int);
CREATE TABLE movements (movement_id int PRIMARY KEY, account_id int, direction text, amount int, at timestamp);
INSERT INTO accounts VALUES (1,1000),(2,500),(3,800);
INSERT INTO movements VALUES
 (1,1,'in', 700,'2026-01-05'),(2,1,'in', 400,'2026-01-20'),   -- Jan 1100 > 1000
 (3,1,'in',1200,'2026-02-11'),                                -- Feb 1200 > 1000 -> flagged
 (4,2,'in', 600,'2026-01-03'),(5,2,'in', 700,'2026-03-03'),   -- Jan, Mar: not consecutive
 (6,2,'out',900,'2026-02-03'),                                -- withdrawals don't count
 (7,3,'in', 900,'2025-12-30'),(8,3,'in', 950,'2026-01-02');   -- Dec -> Jan crosses a year

WITH monthly AS (
  SELECT m.account_id, date_trunc('month', m.at)::date AS mon
  FROM   movements m JOIN accounts a USING (account_id)
  WHERE  m.direction = 'in'
  GROUP  BY m.account_id, date_trunc('month', m.at), a.monthly_limit
  HAVING sum(m.amount) > a.monthly_limit
)
SELECT DISTINCT account_id
FROM  (SELECT account_id, mon,
              lag(mon) OVER (PARTITION BY account_id ORDER BY mon) AS prev_mon
       FROM monthly) t
WHERE  prev_mon = mon - interval '1 month'
ORDER  BY account_id;
```

Output:

```
 account_id
------------
          1
          3
(2 rows)
```

Why it works. `date_trunc('month', at)` buckets movements by month and `HAVING` keeps over-limit months. `lag(mon)` brings the previous over-limit month alongside, so "consecutive" is `prev_mon = mon - interval '1 month'`. Account 3 is caught across December → January, which a naive "month number minus one" would miss.

Edge case. Account 2 is over in January and March but not February, so it is not flagged; its February withdrawal is ignored by `direction = 'in'`. For "N consecutive months" use the island trick on month numbers (`year * 12 + month`).

19.6 Pivot and unpivot (static and dynamic)

Pivot = turn distinct values of a column into separate columns; *unpivot* = the reverse. SQL needs the output columns fixed when a query is planned, so a pivot whose columns depend on the data requires **dynamic SQL**: a query that writes another query, which is then executed.

19.6.1 Names by region, side by side

Problem. A list of attendees (name, region; duplicates possible) from America, Asia and Europe. Produce three columns, `america`, `asia`, `europe`, each listing that region's names alphabetically from the top, with NULL where a column runs out.

`code/22_names_by_region.sql`

```
CREATE TABLE attendees (name text, region text);
INSERT INTO attendees VALUES
 ('Zoe','Europe'),('Ali','Asia'),('Ben','America'),('Kai','Asia'),
 ('Ada','Europe'),('Liv','Europe'),('Ben','America');     -- duplicate row kept

WITH numbered AS (
  SELECT name, region,
         row_number() OVER (PARTITION BY region ORDER BY name) AS rn
  FROM   attendees
)
SELECT max(name) FILTER (WHERE region = 'America') AS america,
       max(name) FILTER (WHERE region = 'Asia')    AS asia,
       max(name) FILTER (WHERE region = 'Europe')  AS europe
FROM   numbered
GROUP  BY rn
ORDER  BY rn;
```

Output:

```
 america |  asia  | europe
---------+--------+--------
 Ben     | Ali    | Ada
 Ben     | Kai    | Liv
 (null)  | (null) | Zoe
(3 rows)
```

Why it works. `row_number()` per region gives each name its line in its column. Grouping by that number puts every region's 1st name on row 1, 2nd names on row 2, and so on; `max(name) FILTER (...)` picks the single name per region on that line.

Alternative. `max(CASE WHEN region = 'Asia' THEN name END)` is the portable form of `FILTER` (MySQL and SQL Server lack `FILTER`). Duplicate rows stay duplicates (Ben twice); add `DISTINCT` in the inner query if the question wants unique names.

19.6.2 Dynamic pivot: one column per store

Problem. `shelf_prices(product_id, store, price)`. Produce one row per product with one column per store — columns sorted by store name, NULL where the product isn't sold — for whatever stores exist when the code runs.

`code/23_dynamic_pivot.sql`

```
SET client_min_messages = warning;   -- hide "view does not exist, skipping" notices
CREATE TABLE shelf_prices (product_id int, store text, price int, PRIMARY KEY (product_id, store));
INSERT INTO shelf_prices VALUES
 (1,'north',10),(1,'west',12),
 (2,'east', 7),(2,'north',8),(2,'west',9),
 (3,'west', 30);

-- Approach A: a procedure writes the SELECT with format() and (re)creates a view
CREATE PROCEDURE pivot_shelf_prices()
LANGUAGE plpgsql AS $$
DECLARE
  col_list text;
BEGIN
  SELECT string_agg(format('max(price) FILTER (WHERE store = %L) AS %I', store, store),
                    ', ' ORDER BY store)
  INTO   col_list
  FROM  (SELECT DISTINCT store FROM shelf_prices) s;

  DROP VIEW IF EXISTS shelf_prices_wide;       -- column set may change, so rebuild
  EXECUTE format('CREATE VIEW shelf_prices_wide AS
                  SELECT product_id, %s FROM shelf_prices GROUP BY product_id', col_list);
END $$;

CALL pivot_shelf_prices();
SELECT * FROM shelf_prices_wide ORDER BY product_id;

-- A new store appears: re-run the procedure and the view grows a column
INSERT INTO shelf_prices VALUES (3,'central',28);
CALL pivot_shelf_prices();
SELECT * FROM shelf_prices_wide ORDER BY product_id;

-- Approach B (psql only): build the query text, then \gexec runs it
SELECT format('SELECT product_id, %s FROM shelf_prices GROUP BY product_id ORDER BY product_id',
              string_agg(format('max(price) FILTER (WHERE store = %L) AS %I', store, store),
                         ', ' ORDER BY store))
FROM  (SELECT DISTINCT store FROM shelf_prices) s
\gexec
```

Output (first call, then after `central` appears; Approach B and the `crosstab` version in the file print the same four-store table):

```
 product_id |  east  | north  | west
------------+--------+--------+------
          1 | (null) |     10 |   12
          2 |      7 |      8 |    9
          3 | (null) | (null) |   30
(3 rows)

 product_id | central |  east  | north  | west
------------+---------+--------+--------+------
          1 |  (null) | (null) |     10 |   12
          2 |  (null) |      7 |      8 |    9
          3 |      28 | (null) | (null) |   30
(3 rows)
```

Why it works.

1. (A **procedure** is a named block of code stored in the database and run with `CALL`; a **view** is a saved query you can `SELECT` from like a table.) `string_agg(format(...), ', ' ORDER BY store)` builds the column list, e.g. `max(price) FILTER (WHERE store = 'east') AS east, ...`. `%L` quotes a **literal** (a data value such as `'east'`) and `%I` quotes an **identifier** (a name such as a column), so a store named `O'Neil's` or `Store 1` cannot break the SQL or inject code (**SQL injection**: data that sneaks extra SQL commands into a query).
2. `EXECUTE` runs the generated `CREATE VIEW`. A PL/pgSQL function must declare its result columns in advance — exactly what we don't know — so the procedure publishes a view that a plain `SELECT *` reads.

3. When `central` appears, calling the procedure again rebuilds the view with a new column. `DROP VIEW` first, because `CREATE OR REPLACE VIEW` can only *append* columns, not insert one in the middle. On an empty table `string_agg` returns NULL and the generated `SELECT product_id, FROM …` is invalid, so real code should check for that first.

Alternatives. In `psql`, `\gexec` runs each value the previous query returns as SQL — the quickest ad-hoc dynamic pivot. `crosstab()` (from the `tablefunc` **extension** — an optional add-on package for PostgreSQL — in the file) still needs the output columns spelled out in `AS ct(...)`. In MySQL the equivalent is `GROUP_CONCAT` to build the column list plus `PREPARE`/`EXECUTE` (watch `group_concat_max_len`); SQL Server has `PIVOT` plus `sp_executesql`.

19.6.3 Dynamic unpivot: store columns back to rows

Problem. `price_grid(product_id, <one int column per store>)`, where the store columns can change and NULL means "not sold". Produce rows `(product_id, store, price)` with no rows for NULL prices.

`code/24_dynamic_unpivot.sql`

```
SET client_min_messages = warning;   -- hide "view does not exist, skipping" notices
CREATE TABLE price_grid (product_id int PRIMARY KEY, east int, north int, west int);
INSERT INTO price_grid VALUES (1,NULL,10,12),(2,7,8,9),(3,NULL,NULL,30);

-- Approach A: no dynamic SQL at all -- turn each row into JSON and explode its keys
SELECT g.product_id, kv.key AS store, kv.value::int AS price
FROM   price_grid g
CROSS  JOIN LATERAL jsonb_each_text(to_jsonb(g) - 'product_id') AS kv
WHERE  kv.value IS NOT NULL
ORDER  BY g.product_id, store;

-- Approach B: read the column names from the catalog and generate UNION ALL branches
CREATE PROCEDURE unpivot_price_grid()
LANGUAGE plpgsql AS $$
DECLARE
  branches text;
BEGIN
  SELECT string_agg(format('SELECT product_id, %L AS store, %I AS price FROM price_grid WHERE %I IS NOT NULL',
                           column_name, column_name, column_name),
                    ' UNION ALL ' ORDER BY column_name)
  INTO   branches
  FROM   information_schema.columns
  WHERE  table_schema = current_schema()
    AND  table_name   = 'price_grid'
    AND  column_name <> 'product_id';

  DROP VIEW IF EXISTS price_grid_long;
  EXECUTE 'CREATE VIEW price_grid_long AS ' || branches;
END $$;

CALL unpivot_price_grid();
SELECT * FROM price_grid_long ORDER BY product_id, store;
```

Output (all queries return these rows):

```
 product_id | store | price
------------+-------+-------
          1 | north |    10
          1 | west  |    12
          2 | east  |     7
          2 | north |     8
          2 | west  |     9
          3 | west  |    30
(6 rows)
```

Why it works.

- **Approach A** needs no dynamic SQL: `to_jsonb(g)` turns the row into `{"product_id":1,"east":null,...}`, `- 'product_id'` drops that key, and `jsonb_each_text` explodes the rest into (key, value) text pairs (hence `::int`). New columns are picked up automatically.

- **Approach B** reads column names from `information_schema.columns` (the standard **catalog** views where a database describes its own tables and columns), generates one `SELECT ... WHERE col IS NOT NULL` per column, joins them with `UNION ALL`, and `EXECUTE`s the result.

Edge case. With a fixed, known set of columns, the simplest unpivot is `CROSS JOIN LATERAL (VALUES ('east', east), ('north', north), ('west', west)) AS v(store, price)` — as used for medals in 19.5.4. Use `UNION ALL` (stack results, keeping duplicates), not `UNION` (stack and remove duplicates): `UNION` would wrongly merge identical rows and costs a sort.

19.7 Self-joins, "both directions" and graph-like questions

Friendship, calls and matches store a *pair* in one row, but the question asks about *each person*. The standard move is to "unfold" the pair with `UNION ALL` (or `UNION`) so each participant gets their own row, then aggregate normally. Relational division ("took *all* courses"; defined in 19.7.6) also lives here.

19.7.1 Page suggestions from friends' likes

Problem. `friendships(a, b)` stores each friendship once, in either column order. `page_likes(user_id, page_id)`. Suggest to user 1 every page liked by at least one friend and not already liked by user 1, without duplicates.

`code/25_page_suggestions.sql`

```
CREATE TABLE friendships (a int, b int, PRIMARY KEY (a, b));      -- stored once, either direction
CREATE TABLE page_likes  (user_id int, page_id int, PRIMARY KEY (user_id, page_id));
INSERT INTO friendships VALUES (1,2),(3,1),(1,4),(5,6);
INSERT INTO page_likes VALUES
 (1,77),(2,77),(2,88),(3,88),(3,99),(4,55),(5,66),(6,11);

WITH my_friends AS (
  SELECT b AS friend FROM friendships WHERE a = 1
  UNION
  SELECT a FROM friendships WHERE b = 1
)
SELECT DISTINCT l.page_id
FROM   page_likes l
JOIN   my_friends f ON f.friend = l.user_id
WHERE  NOT EXISTS (SELECT 1 FROM page_likes me
                   WHERE me.user_id = 1 AND me.page_id = l.page_id)
ORDER  BY l.page_id;

-- Alternative: set difference
SELECT page_id FROM page_likes
WHERE  user_id IN (SELECT b FROM friendships WHERE a = 1 UNION SELECT a FROM friendships WHERE b = 1)
EXCEPT
SELECT page_id FROM page_likes WHERE user_id = 1
ORDER  BY page_id;
```

Output (all queries return these rows):

```
 page_id
---------
      55
      88
      99
(3 rows)
```

Why it works. A friendship may be stored as (1, x) or (x, 1), so `my_friends` looks both ways (2, 3, 4). Joining likes to friends gives candidates; `NOT EXISTS` drops page 77, which user 1 already likes, and `DISTINCT` collapses page 88 (liked by two friends).

Alternative. `EXCEPT` (set difference: rows in the first result that are not in the second) expresses "their pages minus my pages" directly and also removes duplicates. Avoid `page_id NOT IN (SELECT ...)` if the subquery column can be NULL — one NULL makes the whole condition unknown and returns nothing.

19.7.2 Popularity percentage

Problem. Friend pairs, stored once each. For every user, popularity = number of friends ÷ number of distinct users on the platform × 100, rounded to 2 decimals. Order by user.

`code/26_friend_share.sql`

```
CREATE TABLE friend_pairs (u1 int, u2 int, PRIMARY KEY (u1, u2));
INSERT INTO friend_pairs VALUES (1,2),(1,3),(2,3),(4,1),(5,6);

WITH both_ways AS (
  SELECT u1 AS person, u2 AS friend FROM friend_pairs
  UNION
  SELECT u2, u1 FROM friend_pairs
)
SELECT person,
       round(100.0 * count(*) / (SELECT count(DISTINCT person) FROM both_ways), 2) AS popularity_pct
FROM   both_ways
GROUP  BY person
ORDER  BY person;
```

Output:

```
 person | popularity_pct
--------+----------------
      1 |          50.00
      2 |          33.33
      3 |          33.33
      4 |          16.67
      5 |          16.67
      6 |          16.67
(6 rows)
```

Why it works. Unfolding each pair both ways makes "my friends" a simple `GROUP BY person` count. The denominator is the count of distinct people in the unfolded set (users 1–6). `100.0 *` at the front keeps everything numeric.

Common mistake. `3 / 6 * 100` is `0` in PostgreSQL (the file shows it), because `3 / 6` truncates first; put `100.0` first. If some users have no friends, take the denominator from a `users` table instead.

19.7.3 Same person first and last in the day

Problem. Calls store caller, callee and time. For each user and each day, look at the first and last call they took part in (as caller *or* callee). Return users for whom, on at least one day, both of those calls were with the same other person.

`code/27_same_first_last_contact.sql`

```sql
CREATE TABLE phone_calls (caller int, callee int, called_at timestamp, PRIMARY KEY (caller, callee, called_at));
INSERT INTO phone_calls VALUES
 (1,2,'2026-04-01 08:00'),(3,1,'2026-04-01 12:00'),(2,1,'2026-04-01 20:00'),  -- user 1: 2 ... 2
 (4,5,'2026-04-02 09:00'),(4,6,'2026-04-02 18:00'),                          -- user 4: 5 ... 6
 (7,8,'2026-04-03 10:00');                                                    -- one call: first = last

WITH legs AS (                          -- every call seen from both participants
  SELECT caller AS person, callee AS other, called_at FROM phone_calls
  UNION ALL
  SELECT callee, caller, called_at FROM phone_calls
), ends AS (
  SELECT person, called_at::date AS day, other,
         row_number() OVER (PARTITION BY person, called_at::date ORDER BY called_at)      AS from_start,
         row_number() OVER (PARTITION BY person, called_at::date ORDER BY called_at DESC) AS from_end
  FROM   legs
)
SELECT DISTINCT person
FROM   ends
GROUP  BY person, day
HAVING max(other) FILTER (WHERE from_start = 1) = max(other) FILTER (WHERE from_end = 1)
ORDER  BY person;
```

Output:

```
 person
--------
      1
      2
      3
      5
      6
      7
      8
(7 rows)
```

Why it works. `legs` shows every call from both sides, so `other` is always the other party. Ascending and descending `row_number()`s per (person, day) tag the first and last call; the two `max(other) FILTER (...)` extract those partners and `HAVING` compares them. User 1 talked to 2 first and last on 1 April (with 3 in between), and so did user 2 (both calls with 1); user 4 did not.

Edge case. A person with a single call that day has the same call as first and last, so they qualify — that is why 3, 5, 6, 7 and 8 appear. Confirm with the interviewer whether that counts; if not, add `HAVING count(*) > 1`.

19.7.4 League table

Problem. Clubs, plus fixtures with home/away clubs and goals. Award 3 points for a win, 1 for a draw, 0 for a loss. List every club with its points, highest first, ties by club id.

`code/28_league_table.sql`

```sql
CREATE TABLE clubs    (club_id int PRIMARY KEY, club text);
CREATE TABLE fixtures (fixture_id int PRIMARY KEY, home int, away int, home_goals int, away_goals int);
INSERT INTO clubs VALUES (1,'Rovers'),(2,'United'),(3,'Athletic'),(4,'Wanderers');
INSERT INTO fixtures VALUES (1,1,2,2,0),(2,3,1,1,1),(3,2,3,3,1),(4,1,3,0,1);

WITH per_club AS (                    -- one row per (club, match), from that club's side
  SELECT home AS club_id, home_goals AS gf, away_goals AS ga FROM fixtures
  UNION ALL
  SELECT away, away_goals, home_goals FROM fixtures
)
SELECT c.club_id, c.club,
       coalesce(sum(CASE WHEN p.gf > p.ga THEN 3 WHEN p.gf = p.ga THEN 1 ELSE 0 END), 0) AS points
FROM   clubs c
LEFT   JOIN per_club p USING (club_id)
GROUP  BY c.club_id, c.club
ORDER  BY points DESC, c.club_id;
```

Output:

```
 club_id |   club    | points
---------+-----------+--------
       1 | Rovers    |      4
       3 | Athletic  |      4
       2 | United    |      3
       4 | Wanderers |      0
(4 rows)
```

Why it works. `per_club` rewrites each match twice, once from each club's point of view (goals for, goals against), so one `CASE` scores every row. `LEFT JOIN` from `clubs` plus `coalesce(..., 0)` keeps Wanderers, who played nothing.

Alternative. Joining with `ON club_id = home OR club_id = away` and a bigger `CASE` (which side is the club?) also works, but `OR` joins use **indexes** (sorted lookup structures for finding rows fast) poorly. Unfolding first is cleaner and extends to goal difference.

19.7.5 Departments with two above-average earners

Problem. Payroll rows (id, name, department, salary). Return each department that has two or more people paid above that department's mean salary.

`code/29_depts_with_high_earners.sql`

```
CREATE TABLE payroll (emp_id int PRIMARY KEY, name text, dept text, salary int);
INSERT INTO payroll VALUES
 (1,'A','Ops',50),(2,'B','Ops',60),(3,'C','Ops',70),(4,'D','Ops',80),   -- avg 65: C, D above
 (5,'E','Law',90),(6,'F','Law',40),(7,'G','Law',41),                    -- avg 57: only E
 (8,'H','Art',30);

SELECT dept
FROM  (SELECT dept, salary, avg(salary) OVER (PARTITION BY dept) AS dept_avg FROM payroll) t
WHERE  salary > dept_avg
GROUP  BY dept
HAVING count(*) >= 2
ORDER  BY dept;
```

Output (the file's alternative returns the same rows):

```
 dept
------
 Ops
(1 row)
```

Why it works. `avg(salary) OVER (PARTITION BY dept)` attaches each department's average to every employee without grouping them away. Filter rows above that average, then group by department and keep those with two or more. Ops (avg 65) has C and D; Law (avg 57) only has E.

Alternative. A correlated subquery (in the file) computes the department average per row; a third route joins `payroll` to itself on department, groups by employee and keeps `HAVING p.salary > avg(q.salary)`. The window version reads the table once.

19.7.6 Students enrolled in every course

Problem. A course catalogue and an enrolment log (duplicates possible). Return students enrolled in every course in the catalogue. This is *relational division*: "which X are related to **all** Y?".

`code/30_took_every_course.sql`

```
CREATE TABLE catalogue   (course_id int PRIMARY KEY);
CREATE TABLE enrolments  (student_id int NOT NULL, course_id int REFERENCES catalogue);   -- duplicates possible
INSERT INTO catalogue VALUES (1),(2),(3);
INSERT INTO enrolments VALUES (10,1),(10,2),(10,3),(11,1),(11,1),(11,2),(12,3),(12,2),(12,1),(12,1);

SELECT student_id
FROM   enrolments
GROUP  BY student_id
HAVING count(DISTINCT course_id) = (SELECT count(*) FROM catalogue)
ORDER  BY student_id;

-- Alternative: "no course exists that this student did not take" (double NOT EXISTS)
SELECT DISTINCT e.student_id
FROM   enrolments e
WHERE  NOT EXISTS (SELECT 1 FROM catalogue c
                   WHERE NOT EXISTS (SELECT 1 FROM enrolments x
                                     WHERE x.student_id = e.student_id AND x.course_id = c.course_id))
ORDER  BY e.student_id;
```

Output (all queries return these rows):

```
 student_id
------------
         10
         12
(2 rows)
```

Why it works. Count each student's *distinct* courses and compare with the catalogue size. `DISTINCT` matters: student 11 has three enrolment rows but only two courses. The foreign key guarantees no course outside the catalogue can inflate the count.

Alternative. The double `NOT EXISTS` is the textbook formulation: "there is no course that this student has not taken". It does not depend on counting, so it stays correct even without the foreign key (enrolments in unknown courses would break the count version).

19.7.7 Shop revenue under two uniqueness rules

Problem. Shops with 2024 revenue, 2025 revenue and coordinates. Total the 2025 revenue of shops whose 2024 revenue equals at least one *other* shop's 2024 revenue **and** whose (lat, lon) is shared with *no* other shop. Round to 2 decimals.

`code/31_shop_revenue_filter.sql`

```
CREATE TABLE shops (shop_id int PRIMARY KEY, rev_2024 numeric(10,2), rev_2025 numeric(10,2),
                    lat numeric(8,5), lon numeric(8,5));
INSERT INTO shops VALUES
 (1,100.00,120.50,51.50000,-0.12000),
 (2,100.00,140.25,48.85000, 2.35000),
 (3,200.00,210.00,40.71000,-74.00000),
 (4,100.00,  90.00,40.71000,-74.00000),   -- same place as shop 3
 (5,300.00,330.00,35.68000,139.69000);

SELECT round(sum(rev_2025), 2) AS total_2025
FROM  (SELECT rev_2025,
              count(*) OVER (PARTITION BY rev_2024) AS same_rev,
              count(*) OVER (PARTITION BY lat, lon) AS same_spot
       FROM   shops) t
WHERE  same_rev > 1 AND same_spot = 1;
```

Output:

```
 total_2025
------------
     260.75
(1 row)
```

Why it works. Two window counts label each shop at once: how many shops share its 2024 revenue, and how many share its location. Keep `same_rev > 1 AND same_spot = 1`: shops 1 and 2 (shop 4 shares revenue but also shares a location with shop 3). 120.50 + 140.25 = 260.75.

Edge case. Equality on **floating-point** columns (approximate binary number types such as `float`) is fragile (0.1 + 0.2 ≠ 0.3 in binary floats), so values compared for equality belong in `numeric`. Grouping twice with `HAVING` and joining back gives the same answer in more steps.

19.8 Date bucketing and retention

Group by a *derived* date — the first day a user appeared, the calendar month, the booking day — and measure what happens inside each bucket. `generate_series` (produces a sequence of numbers or dates) fills missing buckets. A **cohort** is a group of users who started in the same period; **retention** is the share of a cohort that comes back later.

19.8.1 Day-1 retention

Problem. Game plays per player per day. A player's *install day* is their first play day. For each install day, report the number of installs and *day-1 retention* = share of those players who also played the next day, rounded to 2 decimals.

`code/32_day1_retention.sql`

```
CREATE TABLE plays (player_id int, device_id int, play_day date, games int, PRIMARY KEY (player_id, play_day));
INSERT INTO plays VALUES
 (1,9,'2026-03-01',3),(1,9,'2026-03-02',1),
 (2,8,'2026-03-01',0),(2,8,'2026-03-04',2),
 (3,7,'2026-03-02',5),(3,7,'2026-03-03',1),
 (4,6,'2026-03-02',1);

WITH firsts AS (
  SELECT player_id, min(play_day) AS install_day FROM plays GROUP BY player_id
)
SELECT f.install_day,
       count(*) AS installs,
       round(avg((EXISTS (SELECT 1 FROM plays p
                          WHERE p.player_id = f.player_id
                            AND p.play_day = f.install_day + 1))::int), 2) AS day1_retention
FROM   firsts f
GROUP  BY f.install_day
ORDER  BY f.install_day;

-- Alternative: LEFT JOIN to the next day's row
SELECT f.install_day, count(*) AS installs,
       round(count(p.player_id)::numeric / count(*), 2) AS day1_retention
FROM  (SELECT player_id, min(play_day) AS install_day FROM plays GROUP BY player_id) f
LEFT  JOIN plays p ON p.player_id = f.player_id AND p.play_day = f.install_day + 1
GROUP  BY f.install_day
ORDER  BY f.install_day;
```

Output (all queries return these rows):

```
 install_day | installs | day1_retention
-------------+----------+----------------
 2026-03-01  |        2 |           0.50
 2026-03-02  |        2 |           0.50
(2 rows)
```

Why it works. `firsts` gives each player one install day. `EXISTS` is true when a play exists on `install_day + 1`; averaging the boolean **cast** (type conversion, written `::int`) to int gives the fraction retained (true → 1, false → 0). Player 2 returned on 4 March, not the next day, so does not count.

Alternative. A `LEFT JOIN` to the next-day row and `count(p.player_id) / count(*)` does the same (the primary key on (player, day) guarantees at most one match). Note the `::numeric` cast — without it the integer division returns 0.

19.8.2 Transactions per visit histogram

Problem. Bank visits (customer, day) and transactions (customer, day, amount; duplicates allowed; every transaction happens on a visit day). Build a **histogram** (a count of how many items fall into each value or range): for each number k from 0 up to the largest per-visit count, how many visits had exactly k transactions. Include k values with zero visits.

`code/33_transactions_per_visit.sql`

```
CREATE TABLE branch_visits (customer_id int, visit_day date, PRIMARY KEY (customer_id, visit_day));
CREATE TABLE branch_txns   (customer_id int, txn_day date, amount int);   -- duplicates possible
INSERT INTO branch_visits VALUES (1,'2026-02-01'),(1,'2026-02-05'),(2,'2026-02-01'),(3,'2026-02-02'),(4,'2026-02-
03');
INSERT INTO branch_txns VALUES
 (1,'2026-02-01',50),(1,'2026-02-01',50),(1,'2026-02-01',20),   -- 3 in one visit
 (2,'2026-02-01',10),                                           -- 1
 (3,'2026-02-02',99);                                           -- 1

WITH per_visit AS (
  SELECT v.customer_id, v.visit_day, count(t.txn_day) AS txns
  FROM   branch_visits v
  LEFT   JOIN branch_txns t ON t.customer_id = v.customer_id AND t.txn_day = v.visit_day
  GROUP  BY v.customer_id, v.visit_day
)
SELECT n AS transactions_count, count(p.txns) AS visits_count
FROM   generate_series(0, (SELECT max(txns) FROM per_visit)) AS n
LEFT   JOIN per_visit p ON p.txns = n
GROUP  BY n
ORDER  BY n;
```

Output:

```
 transactions_count | visits_count
--------------------+--------------
                  0 |            2
                  1 |            2
                  2 |            0
                  3 |            1
(4 rows)
```

Why it works. `per_visit` counts each visit's transactions (`count(t.txn_day)` makes visits with none count 0 after the `LEFT JOIN`). `generate_series(0, max)` produces every k, including 2, which no visit has; a `LEFT JOIN` then counts matching visits, giving 0 for the empty bucket.

Alternative. Without `generate_series` (e.g. MySQL), use a **recursive CTE** (one that repeatedly feeds its own output back in) counting from 0 to the max, or `ROW_NUMBER()` over any big-enough table.

19.8.3 Titles borrowed per day

Problem. A borrowing log (date, title; duplicates possible). For each date, report how many distinct titles were borrowed and a comma-separated list of them in alphabetical order.

`code/34_titles_per_day.sql`

```
CREATE TABLE borrow_log (borrowed_on date, title text);          -- no key, duplicates allowed
INSERT INTO borrow_log VALUES
 ('2026-08-01','Dune'),('2026-08-01','Emma'),('2026-08-01','Dune'),
 ('2026-08-02','Ulysses'),('2026-08-02','Beloved'),('2026-08-03','Emma');

SELECT borrowed_on,
       count(DISTINCT title)                            AS titles,
       string_agg(DISTINCT title, ',' ORDER BY title)   AS title_list
FROM   borrow_log
GROUP  BY borrowed_on
ORDER  BY borrowed_on;
```

Output:

```
 borrowed_on | titles |   title_list
-------------+--------+-----------------
 2026-08-01  |      2 | Dune,Emma
 2026-08-02  |      2 | Beloved,Ulysses
 2026-08-03  |      1 | Emma
(3 rows)
```

Why it works. `count(DISTINCT title)` ignores the second "Dune". `string_agg(DISTINCT title, ',' ORDER BY title)` concatenates unique titles in sorted order; PostgreSQL allows `DISTINCT` together with `ORDER BY` only when the ordering expression is the aggregated one, which it is here.

Dialect note. MySQL: `GROUP_CONCAT(DISTINCT title ORDER BY title SEPARATOR ',')` (limited by `group_concat_max_len`, default 1024 bytes). SQL Server: `STRING_AGG(title, ',') WITHIN GROUP (ORDER BY title)`, with no `DISTINCT` — deduplicate in a subquery first.

19.8.4 Same-day first bookings

Problem. Clinic bookings record when a booking was made and the day the patient wanted. A booking is *same-day* when those dates match. Considering only each patient's first booking (earliest booking date), what percentage were same-day? Round to 2 decimals.

`code/35_first_booking_same_day.sql`

```sql
CREATE TABLE bookings_clinic (booking_id int PRIMARY KEY, patient_id int, booked_on date, wanted_on date);
INSERT INTO bookings_clinic VALUES
 (1,1,'2026-09-01','2026-09-01'),(2,1,'2026-09-05','2026-09-09'),
 (3,2,'2026-09-02','2026-09-04'),
 (4,3,'2026-09-03','2026-09-03'),
 (5,4,'2026-09-10','2026-09-12'),(6,4,'2026-09-01','2026-09-01');

SELECT round(100.0 * avg((booked_on = wanted_on)::int), 2) AS same_day_pct
FROM  (SELECT DISTINCT ON (patient_id) booked_on, wanted_on
       FROM   bookings_clinic
       ORDER  BY patient_id, booked_on) first_bookings;
```

Output:

```
 same_day_pct
--------------
        75.00
(1 row)
```

Why it works. `DISTINCT ON (patient_id) ... ORDER BY patient_id, booked_on` keeps each patient's earliest booking (patient 4's is 1 September, although inserted last). `(booked_on = wanted_on)::int` is 1 or 0, so 100 × its average is the same-day share: 75.00.

Alternative. The "min then join back" version (`min(booked_on)` per patient, join on patient and date) works everywhere; if a patient could have two bookings on their first day it would count both, whereas `DISTINCT ON` picks one — know which the question wants.

19.9 Ratios and cancellation rates

Rates are `count(condition) / count(all)`. The details that go wrong: filtering *which rows are in the denominator*, integer division, and division by zero. `avg(boolean::int)` is a compact way to compute a share.

19.9.1 Ride cancellation rate (the "trips and users" classic)

Problem. Rides (rider, driver, status, request date) and users (with a `banned` flag). For each day from 1 to 3 October 2026, compute the share of requests that were cancelled (by rider or driver), counting only rides where **both** the rider and the driver are not banned. Round to 2 decimals.

`code/36_ride_cancellation_rate.sql`

```sql
CREATE TABLE app_users (user_id int PRIMARY KEY, banned boolean, role text);
CREATE TABLE rides     (ride_id int PRIMARY KEY, rider_id int, driver_id int, city_id int,
                        status text CHECK (status IN ('completed','cancelled_by_driver','cancelled_by_rider')),
                        requested_on date);
INSERT INTO app_users VALUES
 (1,false,'rider'),(2,true,'rider'),(3,false,'rider'),
 (10,false,'driver'),(11,false,'driver'),(12,true,'driver');
INSERT INTO rides VALUES
 (1,1,10,1,'completed',          '2026-10-01'),
 (2,2,10,1,'cancelled_by_driver','2026-10-01'),   -- banned rider: ignored
 (3,3,11,1,'cancelled_by_rider', '2026-10-01'),
 (4,1,12,1,'completed',          '2026-10-02'),   -- banned driver: ignored
 (5,3,10,1,'cancelled_by_driver','2026-10-02'),
 (6,1,11,1,'completed',          '2026-10-03'),
 (7,3,11,1,'cancelled_by_rider', '2026-10-04');   -- outside the window

SELECT r.requested_on AS day,
       round(avg((r.status <> 'completed')::int), 2) AS cancellation_rate
FROM   rides r
JOIN   app_users ru ON ru.user_id = r.rider_id  AND NOT ru.banned
JOIN   app_users du ON du.user_id = r.driver_id AND NOT du.banned
WHERE  r.requested_on BETWEEN '2026-10-01' AND '2026-10-03'
GROUP  BY r.requested_on
ORDER  BY day;
```

Output (the file's alternative returns the same rows):

```
    day     | cancellation_rate
------------+-------------------
 2026-10-01 |              0.50
 2026-10-02 |              1.00
 2026-10-03 |              0.00
(3 rows)
```

Why it works. Joining `app_users` once as rider and once as driver, with `NOT banned` in each `ON`, removes rides with any banned party from numerator and denominator alike. `avg((status <> 'completed')::int)` is the cancelled share. Ride 2 (banned rider) and ride 4 (banned driver) disappear, so 1 October is 1 of 2 cancelled.

Alternative / edge cases. The file's `NOT IN` + `FILTER` version is safe because `user_id` is a non-NULL primary key. Days with no eligible rides produce no row; drive the query from `generate_series` if they must appear. On a `timestamp` column `BETWEEN` would miss most of the last day — use `>= start AND < end + 1 day`.

19.9.2 Delivery fulfilment rate

Problem. Orders (order, customer) and delivery attempts (order, time, `delivered` or `failed`). An order's rate is delivered attempts ÷ all attempts; an order with no attempts has rate 0. Report each customer's average order rate, rounded to 2 decimals.

`code/37_delivery_success_rate.sql`

```
CREATE TABLE shop_orders (order_id int PRIMARY KEY, customer_id int, placed_at timestamp);
CREATE TABLE attempts    (order_id int REFERENCES shop_orders, tried_at timestamp,
                          outcome text CHECK (outcome IN ('delivered','failed')), PRIMARY KEY (order_id,
tried_at));
INSERT INTO shop_orders VALUES (1,100,'2026-05-01'),(2,100,'2026-05-02'),(3,200,'2026-05-03'),(4,200,'2026-05-
04');
INSERT INTO attempts VALUES
 (1,'2026-05-02 10:00','failed'),(1,'2026-05-03 10:00','delivered'),   -- order 1: 0.5
 (2,'2026-05-03 11:00','delivered'),                                   -- order 2: 1.0
 (3,'2026-05-04 09:00','failed');                                      -- order 3: 0.0, order 4: no attempts

WITH per_order AS (
  SELECT o.order_id, o.customer_id,
         coalesce(count(*) FILTER (WHERE a.outcome = 'delivered')::numeric
                  / nullif(count(a.order_id), 0), 0) AS rate
  FROM   shop_orders o
  LEFT   JOIN attempts a USING (order_id)
  GROUP  BY o.order_id, o.customer_id
)
SELECT customer_id, round(avg(rate), 2) AS fulfilment_rate
FROM   per_order
GROUP  BY customer_id
ORDER  BY customer_id;
```

Output:

```
 customer_id | fulfilment_rate
-------------+-----------------
         100 |            0.75
         200 |            0.00
(2 rows)
```

Why it works. `LEFT JOIN` keeps order 4, which has no attempts. For it, `count(a.order_id)` is 0; `nullif(0, 0)` turns the divisor into NULL, the division yields NULL instead of an error, and `coalesce(..., 0)` applies the "no attempts = 0" rule. Customer 100 averages (0.5 + 1.0) / 2 = 0.75.

Dialect trap. In a MySQL `SELECT`, `x / 0` yields NULL (plus a warning under the default **SQL mode** — MySQL's setting for how strictly it treats questionable data; strict mode turns it into an error only inside `INSERT`/`UPDATE`), so `IFNULL(x / 0, 0)` "works" there. PostgreSQL raises `division by zero` and aborts the query — `NULLIF(divisor, 0)` is the portable guard.

19.9.3 Share of members who borrowed each book

Problem. Library members and a borrowings table ((book, member) unique). For each book, the percentage of all members who borrowed it, rounded to 2 decimals; order by percentage descending, then book id.

`code/38_borrow_share_per_book.sql`

```
CREATE TABLE members_lib (member_id int PRIMARY KEY, name text);
CREATE TABLE borrowings  (book_id int, member_id int, PRIMARY KEY (book_id, member_id));
INSERT INTO members_lib VALUES (1,'Al'),(2,'Bea'),(3,'Cas');
INSERT INTO borrowings VALUES (500,1),(500,2),(500,3),(501,2),(502,1),(502,3);

SELECT book_id,
       round(100.0 * count(DISTINCT member_id) / (SELECT count(*) FROM members_lib), 2) AS pct_of_members
FROM   borrowings
GROUP  BY book_id
ORDER  BY pct_of_members DESC, book_id;
```

Output:

```
 book_id | pct_of_members
---------+----------------
     500 |         100.00
     502 |          66.67
     501 |          33.33
(3 rows)
```

Why it works. The numerator counts distinct borrowers per book; the denominator is a scalar subquery over *all* members, including non-borrowers. `100.0 *` first avoids truncation, and `ORDER BY` can use the alias because it runs after `SELECT`.

Edge case. If there were no members, the denominator would be 0 and PostgreSQL would raise an error; `nullif(..., 0)` makes the result NULL instead.

19.10 String building and expression evaluation

Some questions treat SQL as a small **interpreter** (a program that reads and evaluates expressions): evaluate stored comparisons, render a formula, or validate shapes. The tools are `CASE`, `||` **concatenation** (joining strings end to end), `format()` and `string_agg` (an aggregate that joins a group's values into one string).

19.10.1 Evaluate stored comparisons

Problem. `vars(name, val)` holds integer variables; `exprs(lhs, op, rhs)` holds comparisons such as `x < y`, with `op` in `<`, `>`, `=`. Return every expression with its truth value.

`code/39_compare_variables.sql`

```
CREATE TABLE vars  (name text PRIMARY KEY, val int);
CREATE TABLE exprs (lhs text REFERENCES vars, op text CHECK (op IN ('<','>','=')), rhs text REFERENCES vars,
                    PRIMARY KEY (lhs, op, rhs));
INSERT INTO vars  VALUES ('x',4),('y',9),('z',4);
INSERT INTO exprs VALUES ('x','<','y'),('x','>','y'),('x','=','z'),('y','=','z');

SELECT e.lhs, e.op, e.rhs,
       CASE e.op WHEN '<' THEN l.val < r.val
                 WHEN '>' THEN l.val > r.val
                 WHEN '=' THEN l.val = r.val END AS result
FROM   exprs e
JOIN   vars l ON l.name = e.lhs
JOIN   vars r ON r.name = e.rhs
ORDER  BY e.lhs, e.op, e.rhs;
```

Output:

```
 lhs | op | rhs | result
-----+----+-----+--------
 x   | <  | y   | t
 x   | =  | z   | t
 x   | >  | y   | f
 y   | =  | z   | f
(4 rows)
```

Why it works. Joining `vars` twice puts both operand values on one row, and `CASE e.op WHEN ...` maps the symbol to the real comparison. The `boolean` result prints as `t`/`f`; add `::text` if the answer must read `true`/`false`.

Alternative. Dynamic SQL (`EXECUTE format(...)`) could evaluate any operator, but it lets data inject code and is overkill for three operators.

19.10.2 Build a polynomial equation string

Problem. Terms of a polynomial: `power` (0–100, unique) and non-zero integer `coef` (−100…100). Render the left-hand side as one string, highest power first, each term as sign + absolute coefficient + `X^power` (just `X` for power 1, nothing for power 0), followed by `=0`.

`code/40_polynomial_string.sql`

```
CREATE TABLE poly_terms (power int PRIMARY KEY CHECK (power BETWEEN 0 AND 100),
                         coef int CHECK (coef BETWEEN -100 AND 100 AND coef <> 0));
INSERT INTO poly_terms VALUES (0,-6),(1,5),(3,2),(2,-1);

SELECT string_agg(
         CASE WHEN coef > 0 THEN '+' ELSE '-' END
         || abs(coef)
         || CASE WHEN power = 0 THEN ''
                 WHEN power = 1 THEN 'X'
                 ELSE 'X^' || power END,
         '' ORDER BY power DESC) || '=0' AS equation
FROM   poly_terms;
```

Output:

```
     equation
------------------
 +2X^3-1X^2+5X-6=0
(1 row)
```

Why it works. Each term is sign `||` `abs(coef)` `||` power suffix. `string_agg(..., '' ORDER BY power DESC)` joins them with no separator; the `ORDER BY` inside the aggregate sets the order, not the table.

Edge case. `||` accepts an integer when the other side is text (it is cast); a NULL piece would make the whole term NULL, whereas `concat()` treats NULL as empty. MySQL: `GROUP_CONCAT(... ORDER BY power DESC SEPARATOR '')`.

19.10.3 Can four sides form a rectangle?

Problem. Rows of four side lengths `a, b, c, d`, taken in order around the shape. Report `Yes` if opposite sides are equal (a = c and b = d), otherwise `No`.

`code/41_rectangle_check.sql`

```
CREATE TABLE side_sets (a int, b int, c int, d int, PRIMARY KEY (a, b, c, d));
INSERT INTO side_sets VALUES (3,5,3,5),(4,4,4,4),(3,3,5,5),(2,7,2,8);

SELECT a, b, c, d,
       CASE WHEN a = c AND b = d THEN 'Yes' ELSE 'No' END AS rectangle
FROM   side_sets;
```

Output:

```
 a | b | c | d | rectangle
---+---+---+---+-----------
 3 | 5 | 3 | 5 | Yes
 4 | 4 | 4 | 4 | Yes
 3 | 3 | 5 | 5 | No
 2 | 7 | 2 | 8 | No
(4 rows)
```

Why it works. A row-level `CASE` needs no grouping. Order matters: 3, 3, 5, 5 has equal sides but *adjacent* ones, so it fails (if sides could be rearranged, sort the four values and compare the two smallest and the two largest).

Edge case. If any side could be NULL, `a = c` is unknown and the `CASE` falls to `ELSE 'No'`; mention that choice to the interviewer.

19.11 Pattern catalogue

Pattern	Recognise it by	Template query (PostgreSQL)
Nth highest value	"second highest", "Nth largest", "NULL if none"	`SELECT (SELECT DISTINCT v FROM t ORDER BY v DESC OFFSET n-1 LIMIT 1)`
Top-N per group	"top 3 in each", "highest per department"	`WHERE dense_rank() OVER (PARTITION BY g ORDER BY v DESC) <= n` (in a CTE)
First/last row per group	"first session", "latest status", "earliest booking"	`SELECT DISTINCT ON (g) * FROM t ORDER BY g, ts`
All tied for the max	"most ... include ties"	`WHERE cnt = (SELECT max(cnt) FROM c)` or `rank() = 1`
Capacity cut-off	"confirmed vs waitlist", "fits in the lift"	`row_number()` or `sum(w) OVER (ORDER BY pos)` compared with the limit
Running total	"cumulative", "so far", "to date"	`sum(v) OVER (PARTITION BY g ORDER BY ts)`
Compare with previous row	"improved over last", "change since"	`v - lag(v) OVER (PARTITION BY g ORDER BY ts)`
Consecutive ids/days (islands)	"N days in a row", "3+ consecutive"	`GROUP BY g, day - row_number() OVER (PARTITION BY g ORDER BY day)::int HAVING count(*) >= n`
Runs of equal values / streaks	"longest winning streak", "repeated 3 times in a row"	`row_number() OVER (PARTITION BY g ORDER BY ts) - row_number() OVER (PARTITION BY g, val ORDER BY ts)` as island id
Static pivot	fixed list of output columns	`max(v) FILTER (WHERE k = 'a') AS a, ... GROUP BY id`
Pivot of lists	"names under each continent"	`row_number() OVER (PARTITION BY k ORDER BY v)` then `GROUP BY rn` with `FILTER`
Dynamic pivot	"one column per store", columns unknown	`string_agg(format('... %L ... %I', k, k))` then `EXECUTE` (procedure + view) or `\gexec`
Unpivot	wide columns to rows	`CROSS JOIN LATERAL (VALUES ('a', a), ('b', b))` or `jsonb_each_text(to_jsonb(t) - 'id')`
Pair stored once	friendships, calls, matches	`SELECT a AS p, b AS o FROM t UNION ALL SELECT b, a FROM t`
Relational division	"took all", "bought every"	`HAVING count(DISTINCT y) = (SELECT count(*) FROM all_y)`
Compare with group average	"above department average"	`v > avg(v) OVER (PARTITION BY g)`
Uniqueness checks	"shared with another", "unique location"	`count(*) OVER (PARTITION BY k) > 1` / `= 1`
Cohort / retention	"install date", "came back next day"	`min(day)` per user, then `avg((EXISTS (... day = first + 1))::int)`
Histogram with empty buckets	"from 0 to max", "include zero"	`generate_series(0, max) n LEFT JOIN counts ON c = n`
Ratio / rate	"rate", "percentage", "share"	`round(100.0 * count(*) FILTER (WHERE cond) / nullif(count(*), 0), 2)`
String from rows	"list of titles", "build the equation"	`string_agg(expr, sep ORDER BY k)`

Chapter 19 TL;DR

- **Ranking**: `ROW_NUMBER` = exactly one winner, `RANK` = keep ties, `DENSE_RANK` = top N *distinct* values. `DISTINCT ON` is PostgreSQL's shortcut for "first row per group". Use a scalar subquery or aggregate when "no answer" must be a NULL row.
- **Running windows**: `SUM(...) OVER (PARTITION BY ... ORDER BY ...)` for totals so far, `LAG` for "vs previous"; mind the `RANGE` default when the order key ties.
- **Gaps and islands**: `value - ROW_NUMBER()` (or the difference of two row numbers) is constant inside a run; group by it. Deduplicate first.
- **Pivots**: `MAX(...) FILTER (WHERE ...)` per column; for data-driven columns generate SQL with `format()` (`%I`/`%L`) and `EXECUTE` it into a view, or `\gexec`. Unpivot with `LATERAL VALUES` or `jsonb_each_text`.
- **Pairs**: unfold a pair stored in one row with `UNION ALL` so each side gets a row; "all of them" is relational division.
- **Dates**: bucket with `date_trunc` or `min()` per user; fill empty buckets with `generate_series`.
- **Ratios**: `100.0` first, `NULLIF(x, 0)` on the divisor, and a deliberate choice of which rows form the denominator.
- **PostgreSQL traps**: integer division truncates, division by zero is an error, `NOT IN` with a NULL returns nothing, `COUNT(*)` after a `LEFT JOIN` counts the padding row, and `DESC` sorts NULLs first.

Next: Chapter 20 — Cheat sheets and glossary, which condenses the whole book into quick-reference tables, a pattern catalogue and an A–Z glossary.

Chapter 20 — Cheat Sheets, Interview Q&A and Glossary

This last chapter is the book on a few pages. It adds no new topics. It collects the syntax, rules, decision tables and vocabulary of Chapters 1–19 in one place so you can revise quickly before an interview, or look something up while you work. Every entry links back to the section that explains it properly, with examples and real output.

What you'll find here

- **Syntax cheat sheets** for each area of the book (20.2–20.14), most ending with the traps that matter most.
- **The logical query-processing order** as a diagram (20.1). Most "why does this error happen" questions are answered by it.
- **Decision tables**: which join (20.5), which subquery form (20.6), which window function (20.8), which index (20.11), which isolation level (20.12), which scaling tool (20.14) and which database (20.15).
- **A dialect table** for PostgreSQL, MySQL, SQL Server, SQLite and Oracle (20.16).
- **Verified behaviours** (20.17). About a hundred claims from these sheets, such as `7/2 = 3`, `NULL` logic, rank ties and frame defaults, are asserted by a runnable script on PostgreSQL 14. A second script checks the SQLite column of the dialect table.
- **The interview pattern catalogue** (20.18), linking every pattern to the solved problems in Chapters 18–19.
- **Fifty interview questions with short answers** (20.19), each linked to its section.
- **A glossary** of 595 terms, from *1NF* to *ZooKeeper* (20.20), covering every technical term, acronym and product name the book uses.

How to use it: read a sheet, cover the right-hand column of a table and say the answer out loud. Follow the link whenever an answer does not come easily. All SQL is PostgreSQL unless a row says otherwise. Other databases' syntax in the dialect table (20.16) was taken from their documentation and not run, except the SQLite column. Every technical term on these sheets, from *alias* to *ZooKeeper*, has a plain-English definition in the glossary (20.20): look a word up there the first time you meet it.

20.1 The logical query-processing order

You **write** `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`, but the database **evaluates** it (logically) starting from `FROM`. Each step receives the table produced by the step before it. The optimizer (the part of the database that decides how to run a query) may physically reorder the work, but the result is always as if these steps ran in this order.

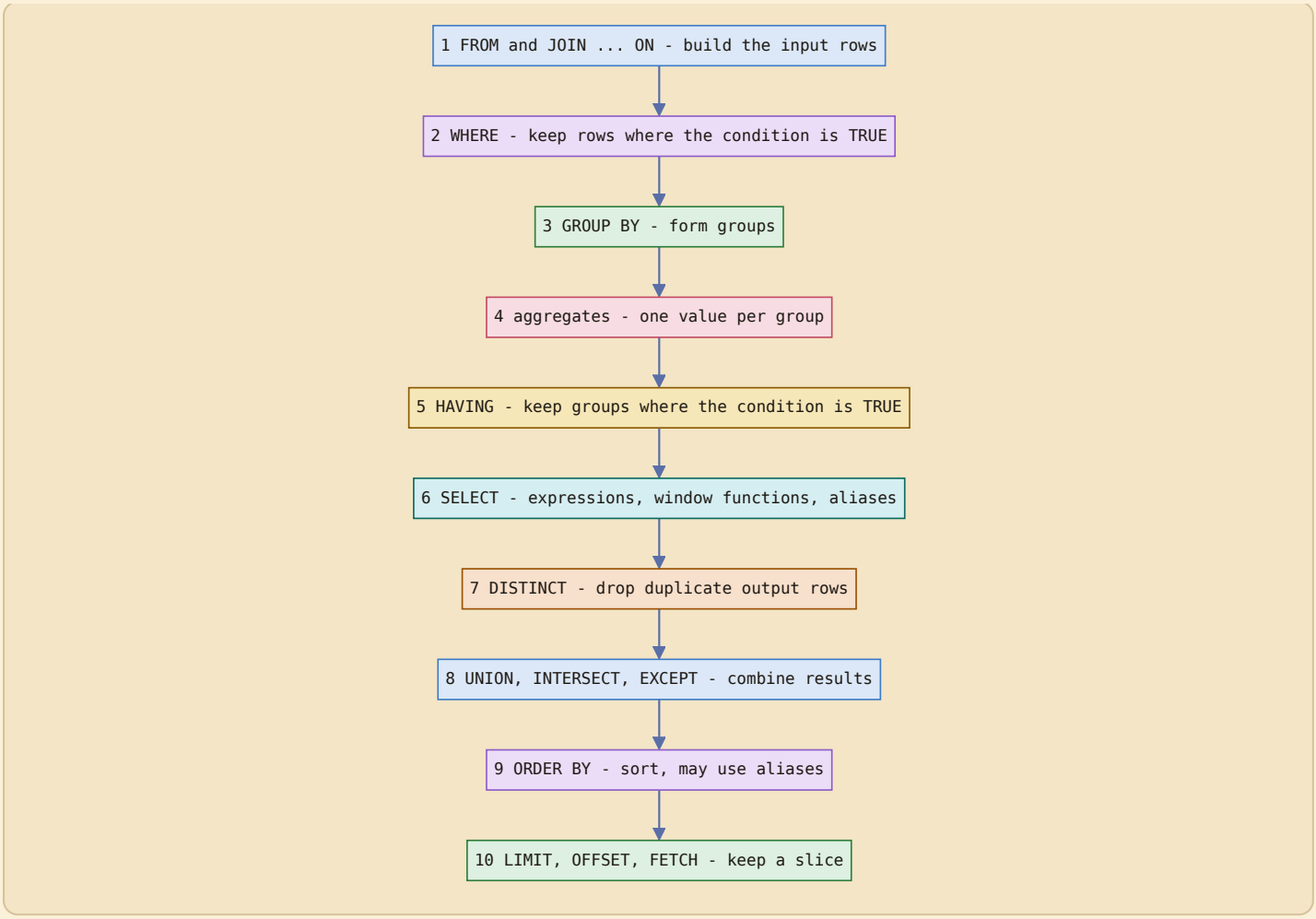


Figure 20.1 — Logical query-processing order (full explanation in §4.5 and §2.9).

Consequence	Because
`WHERE` cannot use a `SELECT` alias	aliases are created at step 6, `WHERE` is step 2
`WHERE` cannot contain `count(*)`, use `HAVING`	groups do not exist before step 3
A window function cannot appear in `WHERE` or `HAVING`	windows are computed at step 6. Filter them in an outer query or CTE (§8.10)
`ORDER BY` can use aliases and column positions	it runs after `SELECT`
`LIMIT` without `ORDER BY` returns arbitrary rows	only step 9 gives order
A filter in `WHERE` on the right table of a `LEFT JOIN` removes the padded rows	`WHERE` runs after the join has built them (§5.9)
With `SELECT DISTINCT`, `ORDER BY` may only use output columns	sorting happens after de-duplication

Two PostgreSQL relaxations: `GROUP BY` may refer to output aliases and positions (`GROUP BY 1`), and if you group by a table's primary key, the other columns of that table may be selected without being listed.

20.2 SELECT, filtering, NULL, sorting and paging

Covers Chapter 2.

```sql
SELECT DISTINCT e.dept_id, e.salary * 12 AS yearly     -- expressions + aliases
FROM   employees AS e                                  -- table alias
WHERE  e.salary BETWEEN 1000 AND 5000                  -- inclusive on both ends
  AND  e.dept_id IN (1, 2, 3)                          -- same as a chain of ORs
  AND  e.name ILIKE 'a%'                               -- LIKE is case-sensitive, ILIKE is not
  AND  e.manager_id IS NOT NULL                        -- never "= NULL"
ORDER  BY yearly DESC NULLS LAST, e.dept_id            -- add a tie-breaker
LIMIT  10 OFFSET 20;                                   -- or OFFSET 20 ROWS FETCH FIRST 10 ROWS ONLY
```

Need	Syntax	Remember
Rename a column	`expr AS name`	double-quote a name with spaces or capitals: `AS "Total"`
Pattern match	`LIKE 'a_%'`	`%` = any run of characters, `_` = exactly one. `ESCAPE` for literal `%`. `~` / `~*` for regular expressions (text patterns)
Range	`x BETWEEN a AND b`	inclusive. For timestamps use half-open `x >= a AND x < b`
NULL test	`IS NULL`, `IS NOT NULL`	`x = NULL` is never TRUE
NULL-safe compare	`a IS [NOT] DISTINCT FROM b`	treats two NULLs as equal
Default for NULL	`COALESCE(x, 0)`	returns the first non-NULL argument
Unique rows	`SELECT DISTINCT a, b`	applies to the whole output row, and NULLs count as equal
First row per group	`SELECT DISTINCT ON (g) … ORDER BY g, ts`	PostgreSQL only. The leading `ORDER BY` keys must match `g`
Sort	`ORDER BY a DESC NULLS LAST, b`	PostgreSQL puts NULLs **last** in ascending order and **first** in descending order
Page	`LIMIT n OFFSET k` / `OFFSET k ROWS FETCH FIRST n ROWS ONLY`	always with `ORDER BY`. Deep pages → keyset pagination, which continues after the last key seen (§12.14)
Keep ties at the cut-off	`FETCH FIRST n ROWS WITH TIES`	needs `ORDER BY` (PostgreSQL 13+)

Three-valued logic. A comparison with `NULL` gives `UNKNOWN` (shown as NULL). `WHERE` and `HAVING` keep only rows where the condition is TRUE. `CHECK` constraints reject only FALSE.

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

`NOT UNKNOWN` is UNKNOWN. Operator precedence is `NOT` > `AND` > `OR`, so add parentheses whenever you mix `AND` and `OR`.

Traps: `x <> 5` silently drops rows where `x` is NULL. `NOT IN (list containing NULL)` returns no rows (§2.5, §6.2). `BETWEEN '2024-01-01' AND '2024-01-31'` on a timestamp misses everything after midnight on the 31st. `OFFSET 100000` still reads and throws away 100,000 rows.

20.3 Functions and expressions

Covers Chapter 3.

Area	PostgreSQL	Notes
Cast	`CAST(x AS numeric(10,2))`, `x::int`	`::` is PostgreSQL-only. Bad input raises an error instead of guessing
Division	`7 / 2` → `3`, `7 / 2.0` → `3.5`, `7::numeric / 2` → `3.5`	integer ÷ integer truncates toward zero (`-7 / 2` → `-3`)
Remainder	`7 % 3`, `mod(7, 3)`	takes the sign of the dividend (`-7 % 2` → `-1`)
Rounding	`round(x, 2)`, `trunc(x)`, `ceil`, `floor`	`round(2.5)` → 3 on `numeric` (half away from zero), `round(2.5::float8)` → 2 (half to even)
Logs and powers	`ln(x)`, `log(x)` (base 10), `log(b, x)`, `power(a, b)`, `sqrt`	MySQL's `LOG(x)` is the natural log
Safe divide	`a / NULLIF(b, 0)`	`1 / 0` is an error (SQLSTATE 22012, a standard error code). The `NULLIF` version returns NULL. MySQL and SQLite return NULL for `x / 0` instead of failing (20.16)
Concatenate	`a || b`, `concat(a, b)`, `concat_ws(', ', a, b)`	`||` with a NULL gives NULL. `concat` skips NULLs
Length	`length(s)` / `char_length(s)` = characters, `octet_length(s)` = bytes	`'é'` is 1 character, 2 bytes in UTF-8 (the usual text encoding)
Pieces	`substring(s FROM 2 FOR 3)`, `left`, `right`, `split_part(s, ',', 2)`	positions are 1-based
Clean up	`trim`, `lower`, `upper`, `initcap`, `replace`, `translate`, `lpad`, `regexp_replace`	
Search	`position('x' IN s)`, `strpos(s, 'x')`	0 means "not found"
Now	`now()` = `current_timestamp` (fixed for the transaction), `clock_timestamp()` (real time)	`current_date` for today
Date arithmetic	`d + 7`, `ts + interval '2 hours'`, `date '2024-03-01' - date '2024-02-01'` → `29`	`date - date` is an integer number of days. Month addition clamps (Jan 31 + 1 month = Feb 29 in 2024)
Parts	`extract(year FROM ts)`, `date_part('dow', d)` (Sunday = 0), `date_trunc('month', ts)`, `age(a, b)`	
Format and parse	`to_char(ts, 'YYYY-MM')`, `to_date('03/05/2024', 'DD/MM/YYYY')`	
Time zones	`timestamptz` stores an instant. `ts AT TIME ZONE 'Europe/Paris'` converts	store `timestamptz`, not `timestamp`
Conditionals	`CASE WHEN … THEN … ELSE … END`, `COALESCE`, `NULLIF`, `GREATEST`, `LEAST`	no `ELSE` → NULL. PostgreSQL's `GREATEST`/`LEAST` ignore NULLs

Traps: integer averages and percentages (`100 * a / b` with integers truncates, so write `100.0 * a / b`), `float` for money (use `numeric`), `timestamp` without a time zone for events in several regions, and assuming `LOG` means the same thing in every database.

20.4 Aggregation

Covers Chapter 4.

```sql
SELECT   dept_id,
         count(*)                                 AS staff,        -- rows
         count(manager_id)                        AS with_manager, -- non-NULL values
         count(DISTINCT job)                      AS jobs,
         round(avg(salary), 2)                    AS avg_salary,   -- NULLs ignored
         count(*) FILTER (WHERE salary > 5000)    AS high_paid,    -- conditional aggregate
         sum(CASE WHEN remote THEN 1 ELSE 0 END)  AS remote_staff, -- portable version
         string_agg(name, ', ' ORDER BY name)     AS names
FROM     employees
WHERE    hired >= date '2020-01-01'               -- filters rows before grouping
GROUP BY dept_id
HAVING   count(*) >= 3                            -- filters groups after aggregation
ORDER BY staff DESC;
```

Function	Returns	NULL behaviour / empty input
`count(*)`	number of rows	0 for no rows
`count(x)`, `count(DISTINCT x)`	non-NULL values (distinct)	0 for no rows
`sum`, `avg`, `min`, `max`	as named	skip NULLs. **NULL** for no rows (use `COALESCE(sum(x), 0)`)
`avg(int_col)`	`numeric` in PostgreSQL	SQL Server returns an integer
`string_agg(x, sep ORDER BY …)`, `array_agg`, `json_agg`	concatenated value	`string_agg` skips NULLs
`bool_and`, `bool_or`	boolean	
`percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`	median (interpolated)	`percentile_disc` returns an actual value; `mode()` the most common
`GROUPING SETS ((a), (b), ())`, `ROLLUP (a, b)`, `CUBE (a, b)`	subtotals in one query	`grouping(a)` = 1 marks a rolled-up row. `ROLLUP (a, b)` = (a,b), (a), ()

Rules: every selected column must be grouped or aggregated (or be functionally dependent on a grouped primary key, meaning its value is fixed once the key is known). NULLs form **one** group. `GROUP BY` does not sort. An aggregate query without `GROUP BY` always returns exactly one row, even over zero rows. That is why "return NULL if there is no answer" questions use an aggregate or scalar subquery (P39).

Traps: `WHERE count(*) > 1` (use `HAVING`). `count(*)` after a `LEFT JOIN` counts the padding row, so use `count(right_table.id)` to get zero (§5.9). Averages of averages. Joining two child tables before aggregating multiplies the rows (fan-out).

20.5 Joins

Covers Chapter 5.

```sql
SELECT c.name, o.id, o.total
FROM   customers c
LEFT   JOIN orders o
       ON  o.customer_id = c.id
       AND o.placed_at >= date '2024-01-01';   -- right-table filter belongs in ON
```

Join	Returns	Typical question
`CROSS JOIN`	every pair, m × n rows	"every student × every subject" grid
`[INNER] JOIN … ON`	only pairs that match	"orders with their customer"
`LEFT [OUTER] JOIN`	every left row, with matches or NULL padding	"all customers, with their orders if any"
`RIGHT [OUTER] JOIN`	mirror of `LEFT`	rewrite as `LEFT` for readability
`FULL [OUTER] JOIN`	all rows of both sides	reconciling two lists
Self join	a table joined to itself under two aliases	employee → manager, pairs, previous row
Semi-join: `EXISTS`, `IN`	left rows that have at least one match, each once	"customers who ordered"
Anti-join: `NOT EXISTS`, `LEFT JOIN … WHERE r.id IS NULL`, `EXCEPT`	left rows with no match	"customers who never ordered"
`JOIN … USING (col)`	equality on same-named columns, one output column	fine. `NATURAL JOIN` is not (it silently joins on every shared name)
`JOIN LATERAL (subquery) ON true`	per-row subquery that may reference the left row	"top 3 orders per customer" (§6.7)

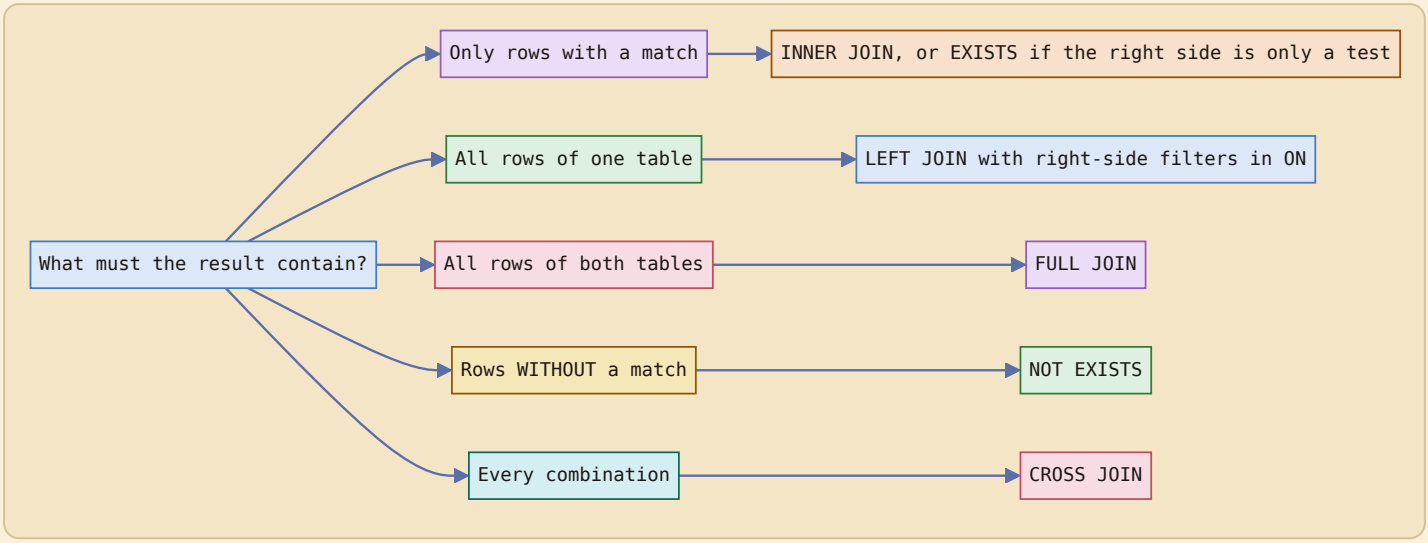


Figure 20.2 — Choosing a join.

Pitfalls (§5.9): fan-out when joining two one-to-many children, right-table conditions in `WHERE` after a `LEFT JOIN`, NULL join keys (never match), `count(*)` of padded rows, joining on names instead of keys, and `NOT IN` with NULLs. **How joins run:** nested loop (few outer rows, indexed inner key), hash join (large, equality only), merge join (both inputs sorted) (§12.15).

20.6 Subqueries and set operations

Covers Chapter 6.

Form	Example	Watch out for
Scalar subquery	`SELECT name, (SELECT max(total) FROM orders o WHERE o.cid = c.id) FROM customers c`	0 rows → NULL. 2+ rows → error
`IN` / `NOT IN`	`WHERE id IN (SELECT cid FROM orders)`	`NOT IN` returns nothing if the subquery yields a NULL
Correlated	inner query uses a column of the outer row	conceptually runs once per outer row. Qualify every column
`EXISTS` / `NOT EXISTS`	`WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.cid = c.id)`	the safe anti-join. The select list inside does not matter
`ANY` / `ALL`	`salary > ALL (SELECT …)`	`> ALL (empty)` is TRUE. `> ANY (empty)` is FALSE. `= ANY` = `IN`, `<> ALL` = `NOT IN`
Derived table	`FROM (SELECT …) AS t`	alias required before PostgreSQL 16
Row subquery	`WHERE (a, b) = (SELECT x, y …)`	also `(a, b) IN (…)`
`LATERAL`	`CROSS JOIN LATERAL (SELECT … WHERE o.cid = c.id LIMIT 3) t`	`LEFT JOIN LATERAL … ON true` keeps rows with no match

Set operator	Result	Notes
`UNION`	rows in either query, duplicates removed	costs a sort or hash
`UNION ALL`	everything, duplicates kept	the default choice when duplicates are impossible or wanted
`INTERSECT [ALL]`	rows in both	NULLs compare as equal here
`EXCEPT [ALL]`	rows in the first but not the second	not commutative. Oracle calls it `MINUS`

Rules: same number of columns, compatible types, column names come from the first query, `INTERSECT` binds tighter than `UNION`/`EXCEPT`, and `ORDER BY`/`LIMIT` apply to the whole combined result (§6.11).

20.7 CTEs, recursion and views

Covers Chapter 7.

```
WITH RECURSIVE chain AS (
    SELECT id, manager_id, name, 1 AS depth, ARRAY[id] AS path   -- anchor: the start rows
    FROM   employees WHERE manager_id IS NULL
  UNION ALL
    SELECT e.id, e.manager_id, e.name, c.depth + 1, c.path || e.id   -- recursive term
    FROM   employees e JOIN chain c ON e.manager_id = c.id
    WHERE  NOT e.id = ANY (c.path)                                    -- cycle guard
)
SELECT repeat('  ', depth - 1) || name AS org_chart FROM chain ORDER BY path;
```

Tool	What it is	Use it for	Remember
CTE `WITH x AS (…)`	a named subquery for one statement	readable step-by-step queries, reuse by name	PostgreSQL 12+ inlines (merges into the main query) single-use CTEs. `AS MATERIALIZED` / `NOT MATERIALIZED` override
Data-modifying CTE	`WITH moved AS (DELETE … RETURNING *) INSERT … SELECT * FROM moved`	several changes in one atomic statement	PostgreSQL only. Every part sees the same snapshot (frozen view of the data)
Recursive CTE	anchor `UNION [ALL]` recursive term	trees, graphs, number and date series	the recursive term sees only the previous round. Stops when a round adds no rows. Needs a stop condition
`CYCLE` / `SEARCH` clauses	built-in cycle detection and ordering	graphs	PostgreSQL 14+
`generate_series(a, b, step)`	a set-returning function (it returns rows)	calendars, filling gaps with zeros	inclusive of both ends. With `date` arguments and an `interval` step it returns `timestamptz`, so cast with `::date`
View	a stored query, no data	simplicity, security, stable interfaces	always current. Simple one-table views are updatable. `WITH CHECK OPTION`
Materialized view	a stored result	expensive reports	stale until `REFRESH MATERIALIZED VIEW [CONCURRENTLY]` (needs a unique index)

20.8 Window functions and frames

Covers Chapter 8.

```
function(args) OVER (
    PARTITION BY dept_id                -- independent groups; rows are NOT collapsed
    ORDER BY     hired, id              -- order inside each partition
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW   -- frame: which rows the function sees
)
```

Function	Result on scores 30, 20, 20, 10 (descending)	Uses the frame?
`row_number()`	1, 2, 3, 4 (ties broken arbitrarily unless you add a tie-breaker)	no
`rank()`	1, 2, 2, 4 (gaps after ties)	no
`dense_rank()`	1, 2, 2, 3 (no gaps)	no
`ntile(n)`	bucket numbers 1..n. Larger buckets first (7 rows into 3 → 3, 2, 2)	no
`percent_rank()` / `cume_dist()`	(rank − 1)/(n − 1) / (rows sorted before the current row + its peers)/n	no
`lag(x, k, default)` / `lead(…)`	value k rows before / after (NULL or `default` past the edge)	no
`first_value`, `last_value`, `nth_value`	value at a position **in the frame**	**yes**
`sum`, `avg`, `count`, `min`, `max` … `OVER`	aggregate over the frame	**yes**

Frame	Meaning
no `ORDER BY` in `OVER`	frame = whole partition (`sum(x) OVER ()` = grand total on every row)
`ORDER BY` without a frame clause	`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which includes all peers (ties) of the current row
`ROWS BETWEEN n PRECEDING AND CURRENT ROW`	physical rows. Use for moving averages and strict running totals
`RANGE BETWEEN interval '7 days' PRECEDING AND CURRENT ROW`	by value distance on the single `ORDER BY` key. Ties included
`GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW`	by peer groups (PostgreSQL 11+)
`EXCLUDE CURRENT ROW / GROUP / TIES`	remove rows from the frame (PostgreSQL 11+)

Patterns: running total `sum(x) OVER (PARTITION BY g ORDER BY t)`, share of total `x / sum(x) OVER ()`, change vs previous `x - lag(x) OVER (…)`, 7-row moving average with `ROWS 6 PRECEDING`, top-N per group (rank in a CTE, filter outside), de-duplication (`row_number() = 1`), gaps and islands (`value − row_number()` is constant inside a run) (§8.7–§8.9).

Traps: `last_value` with the default frame returns the current row (give it `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`). The default `RANGE` frame gives tied rows the same running total. Windows cannot be used in `WHERE`, and PostgreSQL has no `QUALIFY`. `IGNORE NULLS` arrives only in PostgreSQL 19 (in beta as of September 2026), so on 14–18 emulate it (§8.6). Each distinct window specification may need its own sort (§8.10).

20.9 DDL, DML, constraints and upsert

Covers Chapter 9.

```
CREATE TABLE orders (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id  bigint NOT NULL REFERENCES customers (id) ON DELETE RESTRICT,
    status       text   NOT NULL DEFAULT 'new'
                 CONSTRAINT orders_status_ck CHECK (status IN ('new', 'paid', 'shipped')),
    total        numeric(12, 2) NOT NULL CHECK (total >= 0),
    placed_at    timestamptz NOT NULL DEFAULT now(),
    UNIQUE (customer_id, placed_at)
);
CREATE INDEX ON orders (customer_id);                -- FKs are not indexed automatically

INSERT INTO orders (customer_id, total) VALUES (1, 20), (2, 35) RETURNING id;
UPDATE orders SET status = 'paid' WHERE id = 1 RETURNING *;
DELETE FROM orders o USING customers c WHERE o.customer_id = c.id AND c.banned;

INSERT INTO stock (sku, qty) VALUES ('A-1', 5)
ON CONFLICT (sku) DO UPDATE SET qty = stock.qty + EXCLUDED.qty;   -- upsert
```

Statement	Does	Remember
`CREATE TABLE [IF NOT EXISTS]`	defines a table	`CREATE TABLE t AS SELECT …` copies data but no constraints. `LIKE src INCLUDING ALL` copies structure
`ALTER TABLE … ADD / DROP / RENAME COLUMN`, `ALTER COLUMN … TYPE t USING expr`, `SET NOT NULL`, `ADD CONSTRAINT … NOT VALID` + `VALIDATE CONSTRAINT`	changes a table	many forms take an exclusive lock (blocking all other use of the table). Backfill (fill existing rows) before `SET NOT NULL`
`DROP TABLE [IF EXISTS] … [CASCADE]`	removes the table itself	`CASCADE` also drops dependent views and FKs
`TRUNCATE t [RESTART IDENTITY] [CASCADE]`	empties the table fast	no per-row triggers. Transactional in PostgreSQL
`DELETE FROM t WHERE …`	removes chosen rows	fires triggers, logs every row. Forget `WHERE` and every row goes
`INSERT … SELECT`	copies rows	list the target columns explicitly
`UPDATE … SET a = b, b = a`	swaps values	every `SET` expression sees the **old** row
`… RETURNING`	returns affected rows	PostgreSQL, SQLite 3.35+. SQL Server uses `OUTPUT`
`INSERT … ON CONFLICT (key) DO NOTHING / DO UPDATE SET c = EXCLUDED.c`	upsert	needs a unique index or constraint on the conflict target
`MERGE INTO t USING s ON … WHEN MATCHED … WHEN NOT MATCHED …`	standard upsert and sync	PostgreSQL 15+ (not on the book's server, 14)

Constraint	Rejects	Notes
`NOT NULL`	NULL	the most useful constraint
`PRIMARY KEY`	duplicates and NULLs	one per table. Creates a unique index
`UNIQUE`	duplicate non-NULL values	several NULLs allowed (SQL Server: one). `NULLS NOT DISTINCT` in PostgreSQL 15+
`CHECK (expr)`	rows where `expr` is FALSE	NULL passes. Cannot look at other rows
`FOREIGN KEY … REFERENCES`	child values with no parent	`ON DELETE CASCADE / SET NULL / SET DEFAULT / RESTRICT / NO ACTION` (default). NO ACTION can be deferred, RESTRICT cannot
`DEFAULT`	—	used when the column is omitted
`EXCLUDE USING gist (…)`	overlapping values	"no two bookings of one room overlap" (§17.2)

Key generation	Use
`GENERATED ALWAYS AS IDENTITY`	the standard choice. `serial` is the legacy form
Sequence (`nextval`)	shared or custom counters. Gaps are normal (rollbacks, caching)
`uuid` (`gen_random_uuid()`, PostgreSQL 13+)	ids made on many machines. Random UUIDs scatter B-tree inserts
`GENERATED ALWAYS AS (expr) STORED`	derived columns kept in sync automatically. PostgreSQL 18 adds `VIRTUAL` (computed on read), which becomes the default when neither keyword is given

Backups (§9.12): `pg_dump` makes a logical copy, the statements needed to rebuild the data (plain SQL, or `-Fc` custom / `-Fd` directory formats for `pg_restore`, which can restore selected objects and run in parallel). Physical base backups (file-level copies of the whole server) + archived WAL (the write-ahead log of every change) allow point-in-time recovery (PITR). Plan by RPO (how much data you may lose) and RTO (how long recovery may take), keep 3 copies on 2 media with 1 off-site, and test restores.

20.10 Data modeling and normalization quick rules

Covers Chapter 10, Chapter 11 and Chapter 17.

Why databases exist (§1.1). File-based systems suffered from redundancy and inconsistency (the same fact stored in several files, drifting apart), isolated data in different formats, no general way to ask a new question, integrity rules buried in program code, lost updates when two users wrote at once, no atomic recovery after a crash, and all-or-nothing security. A DBMS (database management system) answers each of these by mediating every access.

The three levels of a model (§10.2).

Level	Audience	Contains	Does not contain
Conceptual	business people	entities and relationships	data types, keys, DBMS details
Logical	designers	attributes, keys, cardinalities, normalized tables	a particular product's types or indexes
Physical	the DBMS	exact types, indexes, partitions, storage	—

ER notation (§10.5).

Concept	Chen notation	Crow's foot
Entity / weak entity	rectangle / double rectangle	box
Attribute	ellipse (underlined = key, double = multivalued, dashed = derived)	listed inside the box
Relationship / identifying relationship	diamond / double diamond	a line / a solid line to a weak entity
Cardinality	`1`, `N`, `M` on the lines	line ends: `||` exactly one, `o|` zero or one, `|{` one or many, `o{` zero or many
Participation	double line = total (mandatory)	inner mark: `|` mandatory, `o` optional

Mapping ER → tables (§10.15). Entity → table. Composite attribute → its parts. Multivalued attribute → its own table. Derived attribute → a view or generated column. 1:N → FK on the "many" side. M:N → junction table with a composite PK. 1:1 → FK + `UNIQUE`. Weak entity → owner's key + partial key as a composite PK, with `ON DELETE CASCADE`. Total participation → `NOT NULL` on the FK.

Keys (§10.13). Superkey (any column set that identifies a row) ⊇ candidate key (a minimal superkey) ⊇ the one chosen **primary key**. The other candidates are **alternate keys** (enforce with `UNIQUE`). A **natural key** comes from the data (email, ISBN), a **surrogate key** is made up (identity, UUID). A good default: surrogate PK + `UNIQUE` on the natural key. **Prime attribute** = belongs to some candidate key.

Functional dependencies (§11.1–§11.5). `X → Y` means rows that agree on X agree on Y. It is a business rule: data can disprove it but never prove it. **Closure X^+**: start with X, fire every FD whose left side is fully inside the set, repeat until nothing changes. X is a superkey when X^+ contains every attribute. **Candidate keys**: attributes that never appear on a right side must be in every key, so start from them. **Armstrong's axioms**: reflexivity, augmentation, transitivity (plus derived union, decomposition, pseudotransitivity). **Minimal cover**: single-attribute right sides, no extraneous left-side attributes, no redundant FDs.

Normal forms (§11.6–§11.13).

Form	Rule (informal)	Removes
1NF	every cell holds one atomic value, no repeating groups	lists in a column, `phone1, phone2, phone3`
2NF	1NF + no non-prime attribute depends on **part** of a composite key	partial dependencies
3NF	2NF + no non-prime attribute depends on another non-key attribute. Formally: for each FD X → A, X is a superkey or A is prime	transitive dependencies
BCNF	for every non-trivial FD X → Y, X is a superkey	the remaining FD redundancy
4NF	BCNF + no non-trivial multivalued dependency unless its left side is a superkey	independent lists stored together
5NF	every join dependency follows from the keys	facts that only make sense as three-way combinations

Two things people often get wrong about 1NF: atomicity depends on how the data is **used** (a full name is atomic if you never query first names separately), and a table is not in 1NF just because each cell holds one string. A comma-separated list in one string still breaks it.

Anomalies (why normalize): *insertion* (cannot record a fact without an unrelated one), *update* (one fact stored in many rows, so a change must touch all of them), *deletion* (removing one fact loses another). **Every decomposition must be lossless** (the shared columns are a key of at least one piece) and ideally **dependency-preserving**. 3NF can always have both. BCNF sometimes loses an FD, so most designs stop at 3NF (§11.10).

Design judgement calls (§17.14).

Decision	Rule of thumb
Type vs instance	separate the catalogue row from the physical item (title vs copy, room type vs room, seat vs seat-for-a-show)
Current state vs history	a status column for "now" plus an append-only events or ledger table for "what happened"
Snapshot vs reference	copy values that must not change later (price paid, shipping address) into the order row. That is a separate fact, not redundancy
Delete vs soft-delete	soft-delete (`deleted_at`) rows that history still references
Enforce where	rules in the schema (`CHECK`, `UNIQUE`, partial unique index, `EXCLUDE`) rather than only in application code
Denormalize when	only for a measured read problem, updated in the same transaction, with a reconciliation query (§11.16)
Scarce items	a guarded `UPDATE … WHERE status = 'free'` in a short transaction, or `FOR UPDATE SKIP LOCKED`

20.11 Indexes and reading EXPLAIN

Covers Chapter 12.

```
CREATE INDEX CONCURRENTLY orders_cust_date_ix ON orders (customer_id, placed_at DESC); -- composite
CREATE INDEX ON orders (customer_id) INCLUDE (total);              -- covering (index-only scans)
CREATE INDEX ON orders (placed_at) WHERE status = 'new';           -- partial
CREATE INDEX ON users (lower(email));                              -- expression
CREATE INDEX ON docs USING gin (body_tsv);                         -- full-text / jsonb / arrays
CREATE INDEX ON readings USING brin (taken_at);                    -- huge, append-only, time-ordered
EXPLAIN (ANALYZE, BUFFERS) SELECT … ;                              -- runs the query and shows the plan
```

Which index?

Query shape	Index	Why
`=`, `<`, `>`, `BETWEEN`, `ORDER BY`, `LIKE 'abc%'` (with `text_pattern_ops` or C collation, i.e. plain byte-order sorting)	**B-tree** (default)	sorted, supports ranges and order
`WHERE a = ? AND b > ?`	composite B-tree `(a, b)`	equality columns first, range column last
`WHERE a = ?` returning only a few columns	`(a) INCLUDE (x, y)`	index-only scan, no table visits
Only a small hot subset is queried (`status = 'new'`)	partial index `WHERE status = 'new'`	smaller, cheaper to maintain
`WHERE lower(email) = ?`	expression index on `lower(email)`	the index must match the expression
Equality only, very large keys	hash	cannot do ranges or ordering
JSONB `@>`, arrays `&&` / `@>`, full-text `@@`, trigram `LIKE '%x%'`	**GIN** (`pg_trgm` for trigrams)	inverted index: value → rows
Ranges overlapping, geometry, nearest neighbour, `EXCLUDE` constraints	**GiST**	tree of bounding boxes
Billions of rows physically ordered by time	**BRIN**	a few bytes per block range
Low-selectivity column (`is_active` true for 90 %)	usually **none**	a sequential scan is cheaper

Reading a plan (§12.10). Read from the innermost node outward. Each node shows `cost=startup..total` (planner units), estimated `rows`, and, with `ANALYZE`, `actual time`, actual `rows` and `loops` (multiply per-loop numbers by loops). `BUFFERS` shows pages hit in memory vs read.

Node	Means	Chosen when
Seq Scan	reads every page	a large fraction of the table is needed, or no usable index
Index Scan	walks the index, then fetches each row	few rows, or rows needed in index order
Index Only Scan	answers from the index alone	all needed columns are in the index and pages are all-visible (vacuumed)
Bitmap Index Scan + Bitmap Heap Scan	collects row locations, then reads pages in order	a medium number of scattered rows. Can combine indexes with AND/OR
Nested Loop	for each outer row, probe the inner	small outer side + index on the inner join key
Hash Join	builds a hash table on the smaller input	large inputs, equality joins
Merge Join	merges two sorted inputs	both sides already sorted (or cheap to sort)
Sort / Incremental Sort, HashAggregate / GroupAggregate	sorting and grouping	watch `Sort Method: external merge` (spilled to disk)

The big warning sign: estimated rows far from actual rows. Refresh statistics with `ANALYZE`, and consider extended statistics (statistics on combinations of columns) for correlated columns (§12.16).

Anti-patterns that disable an index (§12.13): a function or arithmetic on the indexed column (`WHERE date(ts) = …`, `WHERE price * 1.2 > …`), a leading wildcard (`LIKE '%x'`), implicit casts, `OR` across different columns (sometimes fixable with `UNION` or a bitmap), a composite index whose leading column is not filtered (PostgreSQL 18+, like Oracle and MySQL 8.0.13+, can sometimes skip-scan it when that column has few distinct values, but don't design around it), deep `OFFSET` paging (use keyset: `WHERE (ts, id) < (:last_ts, :last_id) ORDER BY ts DESC, id DESC LIMIT 20`), and N+1 query loops from application code (one extra query per item of a list). Every index slows writes and uses space, so index real query patterns, not columns.

20.12 Transactions, isolation and concurrency

Covers Chapter 13.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
  SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;   -- lock the row you will change
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  SAVEPOINT s1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
  -- ROLLBACK TO SAVEPOINT s1;   -- undo only the part after s1
COMMIT;                          -- or ROLLBACK
```

ACID. *Atomicity*: all or nothing (undo via log or, in PostgreSQL, by marking the transaction aborted). *Consistency*: constraints and application rules hold before and after. *Isolation*: concurrent transactions do not see each other's partial work (locks, MVCC = multiversion concurrency control, SSI = serializable snapshot isolation, both explained below). *Durability*: a committed change survives a crash (write-ahead log flushed at commit, checkpoints, replicas).

Anomalies (§13.4).

Anomaly	What happens	Example
Dirty write	overwrite another transaction's uncommitted change	two uncommitted updates on one row
Dirty read	read another transaction's uncommitted change	see a balance that is later rolled back
Non-repeatable read (read skew)	the same row read twice gives different values. Read skew: different rows seen at different moments	a report sees an account before a transfer and the other account after it
Phantom read	the same query returns a different **set** of rows	a count rises because someone inserted a matching row
Lost update	two read-modify-write cycles, the second overwrites the first	two sales both set stock 10 → 9
Write skew	two transactions read the same rows, then each updates a **different** row, breaking a rule both checked	two doctors both go off call, leaving nobody on call

Isolation levels — standard minimum vs PostgreSQL.

Level	Standard forbids	PostgreSQL actually prevents	Still possible in PostgreSQL
Read Uncommitted	—	dirty reads (it behaves as Read Committed)	as Read Committed
Read Committed (PostgreSQL default)	dirty read	dirty read and write	non-repeatable read, phantom, lost update, write skew
Repeatable Read (snapshot isolation)	+ non-repeatable read	+ phantoms. Lost updates abort with SQLSTATE 40001	write skew
Serializable (SSI)	+ phantom	everything. One transaction aborts with 40001	nothing, but you must retry

Which isolation level (and lock)?

Situation	Choose
Ordinary OLTP (many small transactions), single-statement changes (`SET qty = qty - 1`)	Read Committed
A consistent multi-query report or export	Repeatable Read, `READ ONLY`
Read a row, decide, write it back	`SELECT … FOR UPDATE` at Read Committed, or an atomic `UPDATE … WHERE`, or Repeatable Read + retry
Optimistic editing across user think-time	a `version` column: `UPDATE … SET version = version + 1 WHERE id = ? AND version = ?`
A rule spanning several rows (no double booking, one on call)	Serializable + retry, an `EXCLUDE`/unique constraint, or explicit locks
Many workers taking jobs from a queue	`SELECT … FOR UPDATE SKIP LOCKED LIMIT 1`
Must not wait at all	`FOR UPDATE NOWAIT` (errors immediately if locked)

Concurrency control families (§13.9). Pessimistic locking (shared/exclusive locks, two-phase locking: grow then shrink, strict 2PL holds write locks to commit). Optimistic (work freely, validate at commit, abort on conflict). Timestamp ordering (each transaction gets a timestamp, operations out of order are rejected). **MVCC** (writers create new row versions, readers see a snapshot, so readers never block writers. PostgreSQL marks versions with `xmin`/`xmax` and cleans dead ones with VACUUM).

Deadlocks (§13.11): two transactions each wait for a lock the other holds. PostgreSQL detects it after `deadlock_timeout` and aborts one (SQLSTATE 40P01). Prevent it by locking rows in a consistent order and

keeping transactions short. Application code should **retry on 40001 and 40P01**.

20.13 Advanced PostgreSQL: PL/pgSQL, JSONB, full-text search, security

Covers Chapter 14.

```
CREATE FUNCTION order_total(p_id bigint) RETURNS numeric
LANGUAGE sql STABLE AS $$ SELECT sum(qty * price) FROM order_lines WHERE order_id = p_id $$;

CREATE PROCEDURE archive_old() LANGUAGE plpgsql AS $$
BEGIN
  LOOP
    DELETE FROM events WHERE id IN (SELECT id FROM events WHERE ts < now() - interval '1 year' LIMIT 10000);
    EXIT WHEN NOT FOUND;
    COMMIT;                                  -- procedures may commit (when CALLed outside a transaction)
  END LOOP;
END $$;
CALL archive_old();

CREATE FUNCTION touch() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN NEW.updated_at := now(); RETURN NEW; END $$;
CREATE TRIGGER t_touch BEFORE UPDATE ON products FOR EACH ROW EXECUTE FUNCTION touch();
```

Feature	Syntax	Remember
Function	`CREATE FUNCTION f(args) RETURNS t LANGUAGE sql / plpgsql`	declare volatility honestly: `IMMUTABLE`, `STABLE`, `VOLATILE` (default)
Procedure	`CREATE PROCEDURE`, `CALL p()`	can `COMMIT` / `ROLLBACK` inside. `INOUT` parameters
Trigger	`BEFORE` / `AFTER` / `INSTEAD OF`, `FOR EACH ROW` / `STATEMENT`	BEFORE can modify `NEW` or reject (return NULL). AFTER reacts. Keep them fast
JSONB	`->` (jsonb), `->>` (text), `#>> '{a,b}'`, `@>` (contains), `?` (key exists), `jsonb_set`, `jsonb_each`, `jsonb_array_elements`	binary, keys de-duplicated (last wins) and reordered. GIN index for `@>` and `?`. In a quick demo, `VACUUM` after bulk updates before building the index, or older snapshots may ignore it
Arrays	`ARRAY[1,2]`, `a[1]` (1-based), `= ANY(a)`, `@>`, `&&`, `unnest`, `array_agg`	out-of-range subscript returns NULL
Full-text search	`to_tsvector('english', body) @@ websearch_to_tsquery('english', 'fast db')`, `ts_rank`	stemming and stop words. Store a generated `tsvector` column with a GIN index
Pivot	`max(v) FILTER (WHERE k = 'a') AS a … GROUP BY id`, or `crosstab` (tablefunc)	unpivot with `CROSS JOIN LATERAL (VALUES ('a', a), ('b', b))`
Roles and privileges	`CREATE ROLE app LOGIN`, `GRANT SELECT, INSERT ON t TO app`, `REVOKE`	group roles, least privilege
Row-level security	`ALTER TABLE t ENABLE ROW LEVEL SECURITY`, `CREATE POLICY p ON t USING (tenant_id = current_setting('app.tenant')::int)`	`USING` filters reads, `WITH CHECK` filters writes. Superusers and `BYPASSRLS` roles always bypass it; the table owner does too unless `FORCE ROW LEVEL SECURITY`
SQL injection defence	bind parameters: psycopg `%s`, sqlite3 `?`, JDBC `PreparedStatement` `?`	never concatenate user input. Identifiers: allow-list or `format('%I', …)`
From code	explicit transactions, `Decimal` / `BigDecimal` for money, batching or `COPY` for bulk loads, stream large results	a connection pool (small, reused) or PgBouncer (a pooling proxy) in front of many clients
ORMs	fast for CRUD (create, read, update, delete)	watch for N+1 queries. Hand-write complex and hot queries (§14.13)

20.14 Scaling: partitioning, sharding, replication

Covers Chapter 15.

```
CREATE TABLE events (id bigint, ts timestamptz NOT NULL, payload jsonb)
PARTITION BY RANGE (ts);
CREATE TABLE events_2024_06 PARTITION OF events
  FOR VALUES FROM ('2024-06-01') TO ('2024-07-01');       -- upper bound is exclusive
-- also PARTITION BY LIST (region) and PARTITION BY HASH (id) … FOR VALUES WITH (MODULUS 4, REMAINDER 0)
```

Tool	What it does	Choose it when	Cost
Scale up	a bigger server	first, almost always	a ceiling, one failure domain
Read replicas	copies that serve reads	read-heavy workloads, reporting	replication lag, stale reads
Partitioning (one server)	split a table by range, list or hash	huge tables, time-based retention (drop a partition), pruning	unique keys must include the partition key
Sharding (many servers)	partitions on different machines	writes or data exceed one server	cross-shard joins and transactions, rebalancing, operations
Caching	keep hot results in memory	repeated reads of the same data	invalidation
Denormalized / summary tables	precomputed answers	expensive repeated aggregates	keeping copies correct

Concept	One-line rule
Shard key	high cardinality (many distinct values), spreads load evenly, matches the most common query filter
Consistent hashing	only about 1/n of keys move when a node joins. Virtual nodes smooth the load
Synchronous vs asynchronous replication	sync = no committed data lost but slower writes. Async = fast, with lag and possible loss on failover
Single-leader / multi-leader / leaderless	one writer (simple) / several writers (conflicts to resolve) / quorums with R + W > N
Consistency models	strong (linearizable) > sequential > causal > session (read-your-writes) > eventual
CAP	during a network partition choose consistency or availability
PACELC	if Partitioned choose A or C, Else choose Latency or Consistency
Consensus (Raft, Paxos)	a majority of 2f + 1 nodes tolerates f crashes. Byzantine faults (nodes that lie, handled by PBFT) need 3f + 1
Two-phase commit	prepare, then commit. Blocks if the coordinator dies after prepare
Recovery	WAL first, checkpoints bound replay, redo committed work, undo uncommitted work
High availability	heartbeat → quorum decides → promote a replica → fence (block writes from) the old leader → redirect clients
Backups vs replicas	a replica copies your mistakes too. Only backups (with PITR) undo a bad `DELETE`

20.15 Which database?

Covers Chapter 16. Default to PostgreSQL for transactional data and add a specialised store only when a requirement clearly exceeds it (§16.10).

Family	Data shape	Strength	Weakness	Examples
Relational (OLTP)	tables, keys, constraints	joins, ACID, ad-hoc SQL	scaling writes beyond one node takes work	PostgreSQL, MySQL, SQL Server, Oracle
Key-value	key → opaque value	fastest get/put, TTL (automatic expiry), simple scale-out	no queries by value	Redis, DynamoDB (also document-like), Memcached
Document	nested JSON per record	flexible schema, whole-object reads	cross-document joins and transactions are limited or costly	MongoDB, Couchbase, Firestore
Wide-column	partition key + sorted clustering columns	huge write rates, time-ordered data per key	must model around queries, no joins	Cassandra, ScyllaDB, HBase, Bigtable
Graph	nodes and edges	multi-hop traversals, paths	bulk analytics over all data, sharding	Neo4j, Amazon Neptune
Time-series	timestamped measurements	compression, retention, downsampling	general-purpose queries	TimescaleDB, InfluxDB, Prometheus
Search engine	inverted index of documents	relevance ranking, facets, fuzzy text	not a system of record	Elasticsearch, OpenSearch
Columnar warehouse	column-oriented tables	scans and aggregates over billions of rows	slow single-row writes	BigQuery, Snowflake, Redshift, ClickHouse
NewSQL / distributed SQL	SQL tables on consensus-replicated shards	SQL + ACID across nodes and regions	latency of consensus, operational complexity	Spanner, CockroachDB, YugabyteDB, TiDB
In-memory	data kept in RAM, persisted by snapshot + log	microsecond latency	cost per GB, durability settings	Redis, SAP HANA, VoltDB

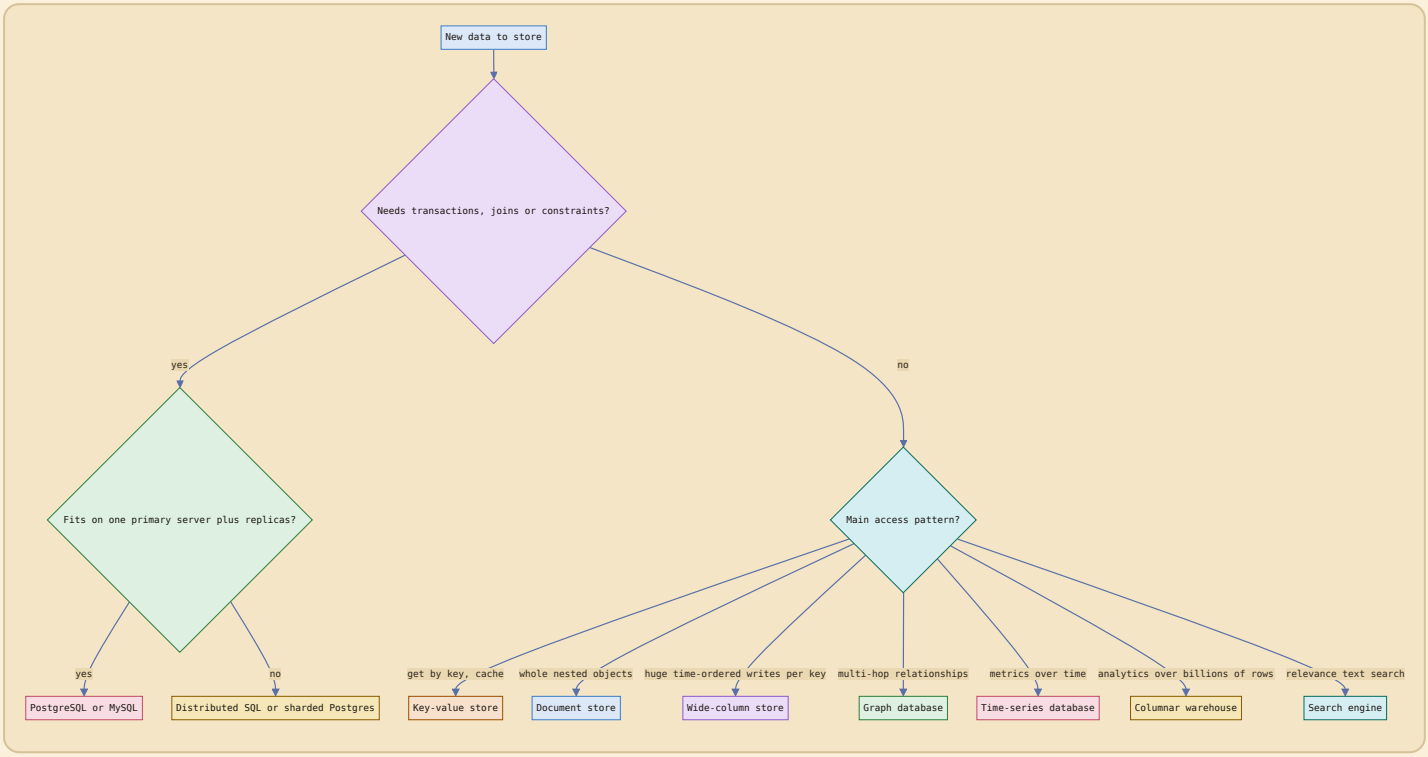


Figure 20.3 — A first-pass database choice. Real systems often combine several (polyglot persistence), fed from one source of truth by change data capture (streaming every committed change out of the main database).

ACID vs BASE. BASE = *Basically Available, Soft state, Eventually consistent*: stay up and accept writes, and let replicas converge later. It is a spectrum, not a switch: many NoSQL stores offer tunable or per-operation consistency.

20.16 Dialect differences

The book runs PostgreSQL 14. This table collects the differences you are most likely to hit when you move a query to another database. Versions in brackets are the first release with the feature. "—" means there is no built-in equivalent. Oracle means 19c, with features added in 21c or 23ai (renamed 26ai) marked. The SQLite column was checked on SQLite 3.43 by `code/02_sqlite_dialect.sqlite.sql` (20.17). The other columns come from each vendor's documentation and were not run here.

Syntax and basics

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
First n rows	`LIMIT n` / `FETCH FIRST n ROWS ONLY`	`LIMIT n`	`TOP (n)` / `OFFSET 0 ROWS FETCH NEXT n ROWS ONLY` (2012, needs `ORDER BY`)	`LIMIT n`	`FETCH FIRST n ROWS ONLY` (12c). Older: `ROWNUM`
Quote an identifier	`"Name"`	backticks around the name (`"` only in ANSI_QUOTES mode)	`[Name]` or `"Name"`	`"Name"` (backticks and `[Name]` also accepted)	`"Name"`
Unquoted names are folded to	lower case	case of table names depends on the OS. Column names are case-insensitive	kept. Case-sensitivity follows the collation (insensitive by default)	kept, compared case-insensitively	UPPER case
String concatenation	`a || b` (NULL if either is NULL), `concat()`	`CONCAT(a, b)` (NULL if any is NULL). `||` means OR by default	`a + b`, `CONCAT()` (NULL treated as '')	`a || b`	`a || b` (NULL treated as '')
Empty string `''`	a value, not NULL	a value	a value	a value	**is NULL**
`7 / 2`	`3`	`3.5000` (`7 DIV 2` = 3)	`3`	`3`	`3.5`
`1 / 0` in a `SELECT`	error (22012)	NULL + warning (strict mode errors only in `INSERT`/`UPDATE`)	error (8134)	NULL	error (ORA-01476)
Cast	`CAST(x AS t)`, `x::t`	`CAST`, `CONVERT(x, t)`	`CAST`, `CONVERT(t, x)`, `TRY_CAST`	`CAST`	`CAST`, `TO_NUMBER`, `TO_DATE`
Boolean type	`boolean`	`BOOLEAN` = `TINYINT(1)`	`BIT`	integers 0/1 (keywords `TRUE`/`FALSE` accepted)	`BOOLEAN` (23ai). Before: none in SQL
`SELECT` without `FROM`	yes	yes (or `FROM DUAL`)	yes	yes	`FROM DUAL` required before 23ai
Type checking	strict	lenient unless strict SQL mode (the default in 8.0)	strict	dynamic typing (type affinity). `STRICT` tables (3.37)	strict
Auto-increment	`GENERATED … AS IDENTITY`, `serial`	`AUTO_INCREMENT`	`IDENTITY(1,1)`	`INTEGER PRIMARY KEY` (optionally `AUTOINCREMENT`)	`GENERATED … AS IDENTITY` (12c), sequences
Sequences	yes	—	yes (2012)	—	yes
`DROP TABLE IF EXISTS`	yes	yes	yes (2016)	yes	23ai

NULLs, comparison and sorting

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
NULL-safe equality	`IS NOT DISTINCT FROM`	`<=>`	`IS NOT DISTINCT FROM` (2022)	`IS` (and `IS NOT DISTINCT FROM`, 3.39)	— (use `DECODE(a, b, 1, 0) = 1`)
NULLs in `ORDER BY … ASC`	last	first	first	first	last
`NULLS FIRST / LAST`	yes	— (sort by `x IS NULL` first)	— (sort by a `CASE` first)	yes (3.30)	yes
First non-NULL	`COALESCE`	`COALESCE`, `IFNULL`	`COALESCE`, `ISNULL`	`COALESCE`, `IFNULL`	`COALESCE`, `NVL`
`GREATEST(1, NULL)`	1 (NULLs ignored)	NULL	1 (2022)	use `max(a, b)` (NULL if any NULL)	NULL
`UNIQUE` column with several NULLs	allowed (`NULLS NOT DISTINCT` 15+)	allowed	**one** NULL only (use a filtered index)	allowed	allowed (all-NULL keys are not indexed)
`LIKE` case-sensitive?	yes (`ILIKE` for no)	no with the default collation	no with the default collation	no for ASCII	yes
Regular expressions	`~`, `~*`, `regexp_replace`	`REGEXP`, `REGEXP_LIKE` (8.0)	`REGEXP_LIKE`, `REGEXP_REPLACE` (2025). Before: `LIKE` patterns only	`REGEXP` only if a function is loaded (the shell has one)	`REGEXP_LIKE`, `REGEXP_REPLACE`

Joins, subqueries and set operations

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
`FULL OUTER JOIN`	yes	— (emulate with `UNION`)	yes	yes (3.39)	yes
`RIGHT JOIN`	yes	yes	yes	yes (3.39)	yes
`USING`, `NATURAL JOIN`	yes	yes	—	yes	yes
`LATERAL`	yes	yes (8.0.14)	`CROSS APPLY` / `OUTER APPLY`	—	`LATERAL`, `CROSS/OUTER APPLY` (12c)
`INTERSECT`, `EXCEPT`	yes	yes (8.0.31)	yes	yes	`INTERSECT`, `MINUS` (`EXCEPT` 21c)
Row values `(a, b) IN (SELECT …)`	yes	yes	— (use `EXISTS`)	yes (3.15)	yes
`UPDATE` with a join	`UPDATE t SET … FROM s WHERE …`	`UPDATE t JOIN s ON … SET …`	`UPDATE t SET … FROM t JOIN s ON …`	`UPDATE … FROM` (3.33)	`MERGE`, or a correlated subquery (direct joins 23ai)

Aggregation, CTEs and windows

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
String aggregation	`string_agg(x, ',' ORDER BY x)`	`GROUP_CONCAT(x ORDER BY x SEPARATOR ',')`	`STRING_AGG(x, ',') WITHIN GROUP (ORDER BY x)` (2017)	`group_concat(x, ',')` (`string_agg` 3.44)	`LISTAGG(x, ',') WITHIN GROUP (ORDER BY x)`
`FILTER (WHERE …)` on aggregates	yes	— (use `CASE`)	— (use `CASE`)	yes (3.30)	— (use `CASE`)
`avg()` of an integer column	exact `numeric`	decimal	**integer** (truncated)	floating point	exact `NUMBER`
Median	`percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`	—	`PERCENTILE_CONT` as a window function only	—	`MEDIAN(x)`, `PERCENTILE_CONT`
`ROLLUP` / `CUBE` / `GROUPING SETS`	all three	`WITH ROLLUP` only	all three	—	all three
Recursive CTE keyword	`WITH RECURSIVE`	`WITH RECURSIVE`	plain `WITH`	`WITH RECURSIVE`	plain `WITH` (column list required)
Window functions	yes (8.4)	yes (8.0)	ranking 2005, frames 2012	yes (3.25)	yes (8i)
`GROUPS` frames, `EXCLUDE`	yes (11)	—	—	yes (3.28)	yes (21c)
`WINDOW w AS (…)` clause	yes	yes	yes (2022)	yes	yes (21c)
`IGNORE NULLS` in `lag` / `first_value`	— (19, in beta as of September 2026)	—	yes (2022)	—	yes
`QUALIFY`	—	—	—	—	26ai
`DISTINCT ON`	yes	—	—	—	—

DDL, DML and transactions

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
Upsert	`INSERT … ON CONFLICT`, `MERGE` (15)	`INSERT … ON DUPLICATE KEY UPDATE`, `REPLACE`, `INSERT IGNORE`	`MERGE`	`INSERT … ON CONFLICT` (3.24), `INSERT OR REPLACE`	`MERGE`
Return changed rows	`RETURNING`	—	`OUTPUT inserted.*`	`RETURNING` (3.35)	`RETURNING … INTO` (PL/SQL)
DDL inside a transaction	yes, can be rolled back	no, causes an implicit commit	mostly yes	yes	no, DDL commits
`CHECK` enforced	yes	yes (8.0.16; parsed but ignored before)	yes	yes	yes
Default isolation level	Read Committed	Repeatable Read	Read Committed (locking)	Serializable	Read Committed
"Serializable" means	SSI (true serializability)	2PL: plain reads become locking reads	2PL (or `SNAPSHOT` level)	one writer at a time	snapshot isolation (write skew possible)
Stored-procedure language	PL/pgSQL (and others)	SQL/PSM-style	T-SQL	— (host language)	PL/SQL
Materialized views	yes, manual `REFRESH`	—	indexed views	—	yes, with incremental (fast) refresh

Indexes and plans

Feature	PostgreSQL	MySQL 8 (InnoDB)	SQL Server	SQLite	Oracle
Table storage	heap + separate indexes	clustered on the primary key	heap or clustered index	B-tree keyed by rowid	heap (index-organized tables optional)
Partial index	yes	—	filtered index	yes	— (function-based index returning NULL)
Covering `INCLUDE`	yes (11)	— (add to the key)	yes	—	—
Expression index	yes	functional key parts (8.0.13)	index a computed column	yes	function-based index
Show the plan	`EXPLAIN (ANALYZE, BUFFERS)`	`EXPLAIN ANALYZE` (8.0.18), `EXPLAIN FORMAT=TREE`	actual execution plan, `SET STATISTICS IO ON`	`EXPLAIN QUERY PLAN`	`EXPLAIN PLAN FOR` + `DBMS_XPLAN.DISPLAY`

Dates and functions

Task	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
Now	`now()`	`NOW()`	`SYSDATETIME()`, `GETDATE()`	`datetime('now')`	`SYSTIMESTAMP`, `SYSDATE`
Add 3 days	`d + 3`, `ts + interval '3 days'`	`DATE_ADD(d, INTERVAL 3 DAY)`	`DATEADD(day, 3, d)`	`date(d, '+3 days')`	`d + 3`, `d + INTERVAL '3' DAY`
Days between	`d2 - d1`	`DATEDIFF(d2, d1)`	`DATEDIFF(day, d1, d2)`	`julianday(d2) - julianday(d1)`	`d2 - d1`
Truncate to month	`date_trunc('month', d)`	`DATE_FORMAT(d, '%Y-%m-01')`	`DATETRUNC(month, d)` (2022)	`date(d, 'start of month')`	`TRUNC(d, 'MM')`
Year of a date	`extract(year FROM d)`	`YEAR(d)`	`YEAR(d)`	`strftime('%Y', d)`	`EXTRACT(YEAR FROM d)`
Characters / bytes	`length` / `octet_length`	`CHAR_LENGTH` / `LENGTH`	`LEN` (ignores trailing spaces) / `DATALENGTH`	`length` / `length(CAST(s AS BLOB))`	`LENGTH` / `LENGTHB`
If-then-else shortcut	`CASE`	`IF(c, a, b)`	`IIF(c, a, b)`	`iif(c, a, b)` (3.32)	`DECODE`, `CASE`
Random number 0–1	`random()`	`RAND()`	`RAND()`	`random() / 18446744073709551616.0 + 0.5`	`DBMS_RANDOM.VALUE`
JSON	`json`, `jsonb` (+ GIN)	`JSON` (binary)	`nvarchar` + `JSON_VALUE` etc. (native `json` type 2025)	JSON functions on text (built in since 3.38)	`JSON` type (21c)
Arrays	yes	–	–	–	collection types (`VARRAY`) in PL/SQL

20.17 Verified behaviours

Every concrete behaviour quoted on these sheets, such as `7 / 2 = 3`, `NULL = NULL` being unknown, rank ties, the default window frame, `NOT IN` with a NULL, upsert, and `->>` returning text, is asserted by `code/01_cheatsheet_checks.sql`. The fixtures and a small checker live in `code/00_schema.sql`:

```
CREATE FUNCTION chk(p_area text, p_claim text, p_got text, p_expected text)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  IF p_got IS DISTINCT FROM p_expected THEN
    RAISE EXCEPTION 'cheat-sheet claim FAILED [%] %: got %, expected %',
      p_area, p_claim, coalesce(p_got, 'NULL'), coalesce(p_expected, 'NULL');
  END IF;
  INSERT INTO check_log (area, claim, got, expected)
  VALUES (p_area, p_claim, coalesce(p_got, 'NULL'), coalesce(p_expected, 'NULL'));
END $$;
```

`chk` compares with `IS DISTINCT FROM`, so an expected NULL is checked properly too (a plain `<>` would let a NULL result slip through, which is exactly the trap of 20.2). The checks are grouped into `DO` blocks by area. A few examples:

```
DO $$ BEGIN
  PERFORM chk('NULL', '2 NOT IN (1, NULL) is NULL, not TRUE',     (2 NOT IN (1, NULL))::text, NULL);
  PERFORM chk('NULL', 'NOT IN over a list holding NULL keeps 0 rows',
              (SELECT count(*) FROM customers WHERE id NOT IN (SELECT customer_id FROM banned))::text, '0');
  -- ...
END $$;

DO $$ BEGIN
  PERFORM chk('Windows', 'row_number / rank / dense_rank on 30,20,20,10',
              (SELECT string_agg(format('%s:%s/%s/%s', name, rn, rk, drk), ' ' ORDER BY rn)
                 FROM (SELECT name,
                              row_number() OVER (ORDER BY points DESC, name) AS rn,
                              rank()       OVER (ORDER BY points DESC)       AS rk,
                              dense_rank() OVER (ORDER BY points DESC)       AS drk
                         FROM scores WHERE points IS NOT NULL) t),
              'Al:1/1/1 Bo:2/2/2 Cy:3/2/2 Di:4/4/3');
  PERFORM chk('Windows', 'default frame with ORDER BY is RANGE: peers share the running total',
              (SELECT string_agg(s::text, ',' ORDER BY day, amount)
                 FROM (SELECT day, amount, sum(amount) OVER (ORDER BY day) AS s FROM sales) t),
              '10,40,40,45');
  -- ...
END $$;

-- transactional DDL: a TRUNCATE inside a failed sub-block is rolled back
BEGIN
  TRUNCATE t_users;
  RAISE EXCEPTION 'undo';
EXCEPTION WHEN raise_exception THEN NULL;
END;
PERFORM chk('DDL', 'TRUNCATE is transactional in PostgreSQL (rows came back)',
            (SELECT count(*) FROM t_users)::text, '3');
```

If a claim were wrong, the file would stop with, for example, `ERROR: cheat-sheet claim FAILED [Functions] …: got 2/2, expected 1/2`. (That exact message appeared once while this chapter was being written, when the scratch database was accidentally created with the `SQL_ASCII` encoding. In that encoding `length('é')` counts bytes. The runner always creates UTF-8 databases.) The file ends by listing what was verified.

Output (`bash tools_run_sql.sh sql '20-*'` → `OK`, and the final queries):

```
     area     | claims
--------------+--------
 NULL         |     18
 Functions    |     18
 Aggregation  |     12
 Joins        |      5
 Subqueries   |      3
 Set ops      |      5
 DISTINCT     |      1
 ORDER BY     |      4
 WHERE        |      4
 Windows      |     12
 CTEs         |      3
 Constraints  |      3
 Upsert       |      2
 DML          |      1
 DDL          |      1
 Transactions |      2
 JSONB        |      5
 Arrays       |      3
 FTS          |      2
(19 rows)

 total_claims_verified
-----------------------
                   104
(1 row)
```

The SQLite column of the dialect table is checked the same way by `code/02_sqlite_dialect.sqlite.sql`. SQLite has no `DO` blocks, so each claim is inserted into a table whose `CHECK (ok = 1)` fails the script

when a claim is false:

```
CREATE TABLE claim (what TEXT, ok INTEGER CHECK (ok = 1));
INSERT INTO claim VALUES
 ('7/2 = 3 (integer division)',              7 / 2 = 3),
 ('a || NULL is NULL',                      ('a' || NULL) IS NULL),
 ('LIKE is case-insensitive for ASCII',     'Anna' LIKE 'a%'),
 ('NULLs sort first in ASC',                (SELECT name FROM s ORDER BY pts LIMIT 1) = 'c'),
 ('IS DISTINCT FROM exists (3.39+)',        (NULL IS NOT DISTINCT FROM NULL) = 1),
 -- ...
 ('UNIQUE allows several NULLs',            (SELECT count(*) FROM u) = 2);
BEGIN; CREATE TABLE gone (x); ROLLBACK;      -- DDL is transactional in SQLite too
INSERT INTO claim SELECT 'DDL rolls back', (SELECT count(*) FROM sqlite_master WHERE name = 'gone') = 0;
```

Output:

```
28 SQLite claims verified
```

20.18 Interview pattern catalogue

Most SQL interview questions are one of about twenty patterns. Recognise the pattern from the wording, write the template, then adapt it. Method and edge-case checklist: §18.1 and §19.1. Full catalogues: §18.10 and §19.11.

Pattern	Wording that gives it away	Core technique	Solved examples
Plain filter	"find rows where …", "not equal to"	`WHERE` with NULL-aware logic (`IS DISTINCT FROM`)	P1, P2, P4, 19.2.1
String and regex filter	"valid e-mail", "longer than N characters"	`~` with anchors, `char_length`	P6, P7
Row-level classification	"label each row", "is it a triangle"	`CASE`	P8, 19.10.3
Group and count	"per", "for each", "how many"	`GROUP BY` + aggregate, `HAVING`	P10, P12, P14, 19.2.2
Zero counts included	"including those with none"	`LEFT JOIN` + `count(right.col)` or `CROSS JOIN` grid	P20, 19.2.3
Distinct lists per group	"list the products sold each day"	`count(DISTINCT)`, `string_agg(DISTINCT … ORDER BY …)`	P15, P16, 19.10.2
Semi-join	"who has at least one"	`EXISTS`	P22
Anti-join	"never", "without", "no sales"	`NOT EXISTS` (not `NOT IN`)	P23, P24, P25, P26
Conditional aggregation / static pivot	"one column per status"	`count(*) FILTER (WHERE …)`, `SUM(CASE …)`	P27, P28, 19.6.1
Dynamic pivot / unpivot	"one column per store", columns unknown	`format('%I')` + `EXECUTE`, `LATERAL (VALUES …)`, `jsonb_each_text`	19.6.2, 19.6.3
Date ranges	"in February 2020", "last 30 days"	half-open `>= start AND < next_start`	P23, P29, P30, P31
As-of value	"price on that date"	latest row `<=` the date (`DISTINCT ON` or `LIMIT 1`) + default	P33
Ratio / rate	"percentage", "rate", "share"	`100.0 * …`, `NULLIF(denominator, 0)`, round last	P34, P35, P36, P37, 19.9.1, 19.9.2, 19.9.3
Nth highest	"second highest", "NULL if none"	`DISTINCT … OFFSET n-1 LIMIT 1` in a scalar subquery, or `dense_rank`	P39, 19.3.1
Top-N per group	"top 3 in each department"	`dense_rank() OVER (PARTITION BY …)` in a CTE, filter outside	19.3.2
First / last row per group	"first login device", "latest status"	`DISTINCT ON`, `row_number() = 1`	P40, 19.3.3
Ties must be kept	"all winners", "include ties"	`rank() = 1`, `WITH TIES`, `= (SELECT max …)`	P21, 19.3.8, 19.3.9
Running total / cut-off	"so far", "cumulative", "who fits"	`sum() OVER (ORDER BY …)`	P41, 19.4.1, 19.4.2
Compare with previous row	"improved", "change since last"	`lag()`	19.4.3, 19.5.6
Compare with group	"above the department average"	`avg() OVER (PARTITION BY …)`	P42, 19.7.5
Gaps and islands / streaks	"N in a row", "consecutive", "longest streak"	`value − row_number()`, or difference of two row numbers	P32, 19.5.1, 19.5.2, 19.5.3, 19.5.5

Pattern	Wording that gives it away	Core technique	Solved examples
Pairs stored once	friends, calls, matches	`UNION ALL` both directions	19.7.1, 19.7.2, 19.7.4
Relational division	"took every course"	`HAVING count(DISTINCT x) = (SELECT count(*) …)`	19.7.6
Retention / cohorts	"came back the next day"	first date per user, then join on `first + 1`	P38, 19.8.1
Histogram with empty buckets	"include zero"	`generate_series` + `LEFT JOIN`	19.8.2
Largest unique value	"largest number that appears once"	`HAVING count(*) = 1`, then `max` (NULL when none)	P9

20.19 Interview questions and answers

Concept questions (the coding problems are in Chapters 18–19). Each answer is the short version. Follow the link for the long one.

Fundamentals

1. **What is the difference between a database, a DBMS and SQL?** A database is an organised, persistent collection of related data. A DBMS is the software that stores it and controls access (queries, concurrency, recovery, security). SQL is the declarative language used to talk to a relational DBMS. → §1.2, §1.3, §1.9
2. **What are DDL, DML, DQL, DCL and TCL?** Define structure (`CREATE`, `ALTER`, `DROP`), change data (`INSERT`, `UPDATE`, `DELETE`), query (`SELECT`), grant rights (`GRANT`, `REVOKE`) and control transactions (`BEGIN`, `COMMIT`, `ROLLBACK`). → §1.9
3. **Relational vs NoSQL — how do you choose?** By access pattern and consistency need. Relational for structured data with joins, constraints and transactions. NoSQL families when a specific shape (key lookups, nested documents, huge write rates, graphs) dominates and you accept weaker guarantees. → §1.8, §16.10
4. **In what order is a `SELECT` evaluated?** FROM/JOIN → WHERE → GROUP BY → aggregates → HAVING → SELECT (windows) → DISTINCT → set operations → ORDER BY → LIMIT. That is why `WHERE` cannot see aliases and `ORDER BY` can. → §4.5
5. **What does NULL mean, and why does `x = NULL` not work?** NULL marks an unknown or missing value. Any comparison with it is UNKNOWN, and `WHERE` keeps only TRUE, so use `IS NULL` or `IS DISTINCT FROM`. → §2.5
6. **Why can `NOT IN` return no rows?** `x NOT IN (a, NULL)` means `x <> a AND x <> NULL`. The second part is UNKNOWN, so the whole condition is never TRUE. Use `NOT EXISTS`. → §6.2
7. **`WHERE` vs `HAVING`?** `WHERE` filters rows before grouping and cannot use aggregates. `HAVING` filters groups after aggregation. → §4.3
8. **`count(*)` vs `count(col)` vs `count(DISTINCT col)`?** All rows / non-NULL values / distinct non-NULL values. → §4.1
9. **`UNION` vs `UNION ALL`?** `UNION` removes duplicates (extra sort or hash). `UNION ALL` keeps everything and is cheaper. → §6.8
10. **`DELETE` vs `TRUNCATE` vs `DROP`?** Chosen rows (triggers fire, each row logged) / all rows quickly / the table itself. In PostgreSQL all three can be rolled back. → §9.6

Joins, subqueries, CTEs, windows

11. **Explain the join types.** Inner = matching pairs. Left/right = all of one side plus matches. Full = all of both. Cross = every pair. Self = a table joined to itself. → §5.2
12. **Why did my `LEFT JOIN` behave like an inner join?** A condition on the right table in `WHERE` removes the NULL-padded rows. Move it into `ON`. → §5.9

13. **What is a semi-join and an anti-join?** Semi: left rows that have a match, each returned once (`EXISTS`). Anti: left rows with no match (`NOT EXISTS`). → §5.8
14. **What is a correlated subquery?** A subquery that refers to the outer row, so conceptually it runs once per outer row. The planner can often turn it into a join. → §6.3
15. **`EXISTS` vs `IN`?** Both express a semi-join and PostgreSQL usually plans them the same way. The difference is `NOT IN` vs `NOT EXISTS` when NULLs are possible. → §6.4
16. **CTE vs subquery vs view vs materialized view?** A CTE names a step inside one statement. A subquery is inline. A view is a stored query, always current. A materialized view stores the result and must be refreshed. → §7.9
17. **How does a recursive CTE work?** The anchor produces the first rows. The recursive term runs again and again on the rows of the previous round until a round returns nothing. → §7.3
18. **Window function vs `GROUP BY`?** `GROUP BY` collapses each group into one row. A window function computes over related rows but keeps every row. → §8.1
19. **`ROW_NUMBER` vs `RANK` vs `DENSE_RANK`?** 1,2,3,4 vs 1,2,2,4 vs 1,2,2,3 on a tie for second place. → §8.5
20. **Why does my running total repeat on tied dates?** With `ORDER BY` and no frame clause the default frame is `RANGE … CURRENT ROW`, which includes all peers. Add a unique tie-breaker or use `ROWS`. → §8.3
21. **Why is `LAST_VALUE` returning the current row?** The default frame ends at the current row. Extend it to `UNBOUNDED FOLLOWING`. → §8.6
22. **How do you find the Nth highest salary?** `SELECT DISTINCT salary … ORDER BY salary DESC OFFSET n-1 LIMIT 1`, wrapped in a scalar subquery to get NULL when there is none, or `dense_rank() = n`. → P39
23. **What is the gaps-and-islands trick?** In a consecutive run, `value − row_number()` stays constant, so grouping by it groups each run. → §8.9

Design

24. **Primary key vs unique key vs foreign key?** PK: the chosen identifier, unique and not null, one per table. Unique: another identifier (alternate key), NULLs usually allowed. FK: a column that must match a key in another table. → §9.3
25. **Superkey vs candidate key?** A superkey identifies rows. A candidate key is a superkey with nothing extra: drop any of its columns and it no longer identifies rows. → §10.13
26. **Natural vs surrogate key?** Natural keys come from the business data and can change. Surrogate keys are generated and meaningless. A common default is a surrogate PK plus `UNIQUE` on the natural key. → §10.13
27. **Explain 1NF, 2NF, 3NF and BCNF.** Atomic values / no partial dependency on a composite key / no transitive dependency / every determinant is a superkey. → §11.7–§11.11
28. **What anomalies does normalization prevent?** Insertion, update and deletion anomalies, all caused by storing one fact in more than one place. → §11.6
29. **When would you denormalize?** For a measured read bottleneck, with a way to keep the copies correct (same transaction, trigger, or materialized view) and a reconciliation check. → §11.16
30. **How do you model a many-to-many relationship?** A junction table with two FKs and a composite PK (or a surrogate PK + `UNIQUE` on the pair). Relationship attributes go there too. → §10.15
31. **What is a weak entity?** An entity that cannot be identified without its owner, such as an order line within an order. Its key is the owner's key + a partial key. → §10.9

Performance

32. **How does a B-tree index speed up a query?** It keeps keys sorted in a shallow balanced tree (3–4 levels for millions of rows), so finding a key or the start of a range takes a few page reads instead of a full scan. → §12.6
33. **Clustered vs non-clustered index?** A clustered index stores the table rows in index order (InnoDB's primary key, SQL Server's clustered index). A non-clustered index is separate and points to the rows. PostgreSQL tables are heaps. `CLUSTER` reorders once but does not maintain the order. → §12.4
34. **How do you choose column order in a composite index?** Leftmost prefix rule: equality columns first, then the range or sort column. → §12.11
35. **What is a covering index?** One that contains every column the query needs, allowing an index-only scan. → §12.12

36. **Why is my index not used?** A function or cast on the column, a leading wildcard, low selectivity, stale statistics, a non-leading composite column, or a table so small that a seq scan is cheaper. → §12.13
37. **How do you read `EXPLAIN ANALYZE`?** Innermost node first. Compare estimated with actual rows, look at loops, the biggest actual times, sorts spilling to disk, and seq scans on big tables with selective filters. → §12.10, §12.17
38. **Why is `OFFSET` paging slow, and what is the fix?** The database still reads and discards all the skipped rows. Keyset pagination (`WHERE (ts, id) < (last_ts, last_id)`) jumps straight there through an index. → §12.14
39. **Nested loop vs hash join vs merge join?** Small outer input with an indexed inner / large equality joins / inputs already sorted. → §12.15
40. **What is the N+1 problem?** One query for a list, then one more query per item. Fix it with a join, `= ANY($1)` with a batch of ids, or ORM eager loading. → §12.13, §14.13

Transactions and concurrency

41. **Explain ACID.** Atomicity (all or nothing), consistency (rules hold), isolation (concurrent transactions do not interfere), durability (committed data survives crashes). → §13.1
42. **Name the isolation levels and anomalies.** Read Uncommitted, Read Committed, Repeatable Read, Serializable. Anomalies: dirty read, non-repeatable read, phantom, lost update, write skew. → §13.4
43. **What does PostgreSQL's Repeatable Read prevent that the standard does not require?** Phantoms (it is snapshot isolation), and it aborts lost updates with a serialization error. It still allows write skew. → §13.4
44. **How do you prevent lost updates?** An atomic `UPDATE … SET x = x - 1`, `SELECT … FOR UPDATE`, Repeatable Read with retry, or optimistic locking with a version column. → §13.7
45. **What is write skew?** Two transactions read the same data, then update different rows, together breaking a rule both checked. Serializable (SSI) or explicit locks prevent it. → §13.8
46. **What is MVCC?** Writers create new row versions, and each transaction reads the versions visible in its snapshot, so readers do not block writers. Old versions are removed by VACUUM. → §13.14
47. **Optimistic vs pessimistic locking?** Pessimistic locks before touching data (waits, deadlocks possible). Optimistic works freely and checks for conflicts at commit (aborts and retries). Low contention favours optimistic. → §13.9
48. **How are deadlocks handled?** The database detects the wait cycle and aborts one transaction (40P01). Prevent them with a consistent lock order and short transactions, and retry in code. → §13.11
49. **How do you build a job queue in SQL?** `SELECT … FOR UPDATE SKIP LOCKED LIMIT 1` inside a transaction, so each worker takes a different unlocked row. → §13.12

Security, scaling, beyond

50. **How do you prevent SQL injection?** Parameterized queries (bind variables). Never concatenate input. Allow-list identifiers. Least-privilege roles. → §14.9
51. **Partitioning vs sharding vs replication?** Partitioning splits a table inside one server. Sharding puts partitions on different servers. Replication keeps copies of the same data for availability and read scaling. → §15.2, §15.3, §15.5
52. **Explain the CAP theorem.** During a network partition a distributed store must choose between consistency (refuse some requests) and availability (answer, possibly stale). PACELC adds that without a partition you trade latency against consistency. → §15.7
53. **Synchronous vs asynchronous replication?** Sync waits for the replica before commit (no loss, higher latency). Async commits first (fast, may lose recent writes on failover). → §15.5
54. **What is the difference between a backup and a replica?** A replica copies every change, including mistakes, within seconds. A backup is a point-in-time copy you can restore from. PITR combines a base backup with archived WAL. → §15.11, §9.12

20.20 Glossary (A–Z)

Short definitions of the terms used across the book, in plain words. The last column links to the section that explains each term with examples. The terms from the course's key-term review cards (the problems that made databases necessary, ER notation, keys, functional dependencies, normal forms, modeling levels, and the case-study judgement calls) are included, with the judgement calls summarised as a table in 20.10.

595 terms. Jump to: 0–9 · A · B · C · D · E · F · G · H · I · J · K · L · M · N · O · P · Q · R · S · T · U · V · W · X · Y · Z

0–9

Term	Meaning	See
1NF (first normal form)	Every cell holds a single atomic value and there are no repeating groups of columns.	§11.7
2NF (second normal form)	1NF, and no non-prime attribute depends on only part of a composite candidate key.	§11.8
2PC (two-phase commit)	Distributed commit protocol: the coordinator asks every participant to prepare, then tells all to commit or all to abort. Blocks if the coordinator fails after prepare.	§15.9
2PL (two-phase locking)	A transaction acquires locks in a growing phase and releases them in a shrinking phase, never acquiring after releasing. Guarantees serializable schedules.	§13.10
3-2-1 backup rule	Keep three copies of the data on two different kinds of storage, with one copy off-site.	§9.12
3NF (third normal form)	2NF, and for every FD X → A either X is a superkey or A is a prime attribute. Removes transitive dependencies.	§11.9
4NF (fourth normal form)	BCNF, and every non-trivial multivalued dependency has a superkey on its left side.	§11.12
5NF (fifth normal form)	Every join dependency is implied by the candidate keys. Also called project-join normal form.	§11.13

A

Term	Meaning	See
ACID	Atomicity, consistency, isolation, durability: the four guarantees of a database transaction.	§13.1
Adjacency list	Storing a tree or graph as rows that each point to their parent (a `parent_id` column).	§7.4
Advisory lock	A PostgreSQL lock on a number the application chooses (`pg_advisory_lock`), not tied to any row or table.	§13.10
Aggregate function	A function that turns many input rows into one value, such as `count`, `sum`, `avg`, `min`, `max`.	§4.1
Aggregate push-down	Computing partial aggregates close to the data (on each shard, or before a join) so that fewer rows travel.	§15.4
Alias	A temporary name for a column (`AS total`) or a table (`FROM orders o`) within one query.	§2.1
Alternate key	Any candidate key other than the one picked as primary key. Enforced with `UNIQUE`.	§10.13
ALTER TABLE	DDL statement that changes an existing table: add, drop, rename or retype columns, add constraints.	§9.5
Analytic function	Another name for a window function (common in Oracle documentation).	§8.1
ANALYZE	Command that samples a table and refreshes the planner's statistics.	§12.16
Anchor (recursive CTE)	The non-recursive first part of a recursive CTE, which produces the starting rows.	§7.3
Anomaly (data)	An insertion, update or deletion problem caused by storing the same fact in more than one row.	§11.6
Anomaly (isolation)	A result that could not happen if transactions ran one at a time, such as a dirty read or write skew.	§13.4
ANSI / ISO SQL standard	The official SQL specification that every vendor implements, each with its own extensions and gaps.	§1.9
Anti-join	Returns rows of one table that have no match in another. Written `NOT EXISTS` or `LEFT JOIN … IS NULL`.	§5.8
ANY / SOME / ALL	Compare a value with a set: true if the comparison holds for at least one element (`ANY`, `SOME`) or for every element (`ALL`).	§6.5
AOF (append-only file)	Redis persistence mode that logs every write command so the data can be rebuilt after a restart.	§16.11
API	Application programming interface: the set of calls one program offers to others.	§1.8
Append-only table	A table that only ever gets new rows, never updates in place, such as an event log or a ledger.	§17.14
ARIES	A classic crash-recovery algorithm: analyse the log, redo all logged history, then undo transactions that never committed.	§15.9
Armstrong's axioms	Reflexivity, augmentation and transitivity: sound and complete rules for deriving functional dependencies.	§11.3
Array (PostgreSQL)	A column type holding an ordered list of values of one type, indexed from 1.	§14.5
Asynchronous replication	The primary commits without waiting for replicas. Fast, but a failover can lose the newest writes.	§15.5
Atomicity	A transaction's changes happen completely or not at all.	§13.2

Term	Meaning	See
Attribute	A named property of an entity or a column of a relation.	§10.6
Attribute closure (X^+)	Every attribute functionally determined by the set X under a given set of FDs.	§11.4
Audit columns	Columns such as `created_at`, `updated_at` and `created_by` that record when and by whom a row was changed.	§17.14
Augmentation (FD rule)	Armstrong's rule: if X → Y, then XZ → YZ for any attributes Z.	§11.3
Autocommit	Mode in which every statement is its own transaction and commits immediately.	§13.1
Autovacuum	PostgreSQL's background worker that runs VACUUM and ANALYZE automatically as tables change.	§12.16
Availability	The share of time a system answers requests. In CAP: every request to a working node gets an answer, possibly stale.	§15.1

B

Term	Meaning	See
Backfill	Filling in values for existing rows after adding a new column or rule.	§9.5
BASE	Basically available, soft state, eventually consistent: the looser guarantee model of many NoSQL systems.	§16.1
Base backup	A physical copy of a PostgreSQL server's whole data directory: the starting point for PITR or a new replica.	§9.12
BCNF (Boyce–Codd normal form)	For every non-trivial FD X → Y, X is a superkey.	§11.11
BEGIN / COMMIT / ROLLBACK	Start a transaction, make its changes permanent, or discard them.	§13.1
BETWEEN	Inclusive range test: `x BETWEEN a AND b` means `x >= a AND x <= b`.	§2.4
Big data	Data too large, too fast or too varied for one ordinary database server (volume, velocity, variety).	§16.9
BigQuery / Snowflake / Redshift	Managed cloud columnar data warehouses for analytical SQL over very large tables.	§16.8
Binary search	Finding a value in sorted data by repeatedly halving the search range: about $\log_2 n$ steps.	§12.5
Bind parameter (placeholder)	A marker such as `$1`, `%s` or `?` in the SQL text whose value the driver sends separately.	§14.9
BitmapAnd / BitmapOr	Plan nodes that combine the row bitmaps of several indexes with AND or OR before reading the table.	§12.10
Bitmap scan	Plan that collects matching row locations from one or more indexes into a bitmap, then reads the table pages in physical order.	§12.10
Bloat	Space wasted in a table or index by dead row versions and half-empty pages.	§13.14
Block	Another word for page: the fixed-size unit in which the disk and the DBMS read and write data.	§12.1
Blocking factor	How many records fit in one disk block (block size ÷ record size).	§12.5
Bloom filter	Compact probabilistic set that answers "definitely not present" or "probably present".	§12.8
BM25	The standard relevance score used by search engines such as Elasticsearch: rewards rare query words that appear often in a short document.	§16.11
BRIN index	Block range index: stores min/max per range of table pages. Tiny and effective when values follow physical order, such as time.	§12.9
BSON	MongoDB's binary form of JSON, with extra types such as dates and 64-bit integers.	§16.3
B-tree / B+ tree	Balanced, shallow, sorted tree index. In a B+ tree all keys live in linked leaf pages, which makes range scans cheap. PostgreSQL's default index.	§12.6
Bucket (hashing)	A slot, usually a page, that a hash function sends keys to.	§12.3
Buffer pool (shared buffers)	Memory area where the DBMS caches data pages read from disk.	§1.4
Bully algorithm	A leader-election method in which the live node with the highest id takes over.	§15.8
Byzantine fault	A failure in which a node keeps running but sends wrong or malicious messages, rather than simply stopping.	§15.8

C

Term	Meaning	See
Cache	A fast store, usually memory, that holds copies of often-read data so the slower source is asked less often.	§1.4
Cache invalidation	Removing or refreshing cached copies when the underlying data changes.	§7.8
Calendar table	A table (or `generate_series`) with one row per day, joined to data so that days with no activity still appear.	§7.6
Candidate key	A minimal set of attributes that uniquely identifies every row.	§10.13
Canonical pair	Storing an undirected pair (two friends) once in a fixed order, such as `user_a < user_b`.	§17.13
CAP theorem	If the network splits, a replicated store can stay fully consistent or stay fully available, but not both.	§15.7
Cardinality (ER)	The maximum number of instances on each side of a relationship: 1:1, 1:N or M:N.	§10.7
Cardinality estimate	The planner's guess of how many rows a plan node will produce.	§12.16
Cardinality (relation)	The number of rows in a relation.	§10.12
Cartesian product	Every row of one table paired with every row of another.	§5.1
CASCADE	Referential action: deleting or updating a parent row does the same to its child rows.	§9.3
CASE	Conditional expression returning the first `THEN` value whose `WHEN` is true, else the `ELSE` value (or NULL).	§3.5
Cassandra / ScyllaDB	Leaderless wide-column databases built for very high write rates spread over many nodes.	§16.4
Cast	Converting a value to another type: `CAST(x AS int)` or, in PostgreSQL, `x::int`.	§3.1
Catalog (data dictionary)	System tables describing all database objects: tables, columns, types, indexes, privileges.	§1.2
Causal consistency	Reads respect cause and effect: nobody sees a reply before the message it answers.	§15.6
CDC (change data capture)	Streaming committed row changes out of a database, usually by reading its log, into other systems.	§16.10
CHECK constraint	A row-level rule that rejects any row for which the expression is FALSE.	§9.3
CHECK OPTION (views)	`WITH CHECK OPTION` makes an updatable view reject inserts and updates whose rows the view would not show.	§7.7
Checkpoint	Point at which all dirty pages are flushed to disk, bounding how much log must be replayed after a crash.	§15.9
Chen notation	ER notation that draws entities as rectangles, attributes as ellipses and relationships as diamonds.	§10.5
Citus	A PostgreSQL extension that shards tables across several PostgreSQL servers and runs queries on them in parallel.	§15.3
ClickHouse	An open-source columnar database for fast analytical queries.	§12.1
Click-through rate (CTR)	Clicks divided by impressions (times the ad was shown), usually as a percentage.	§18.8

Term	Meaning	See
Closure table	A tree-storage pattern with one row for every ancestor–descendant pair, so subtree queries need no recursion.	§7.4
Clustered index	An index whose order is the physical order of the table rows (InnoDB primary key, SQL Server clustered index).	§12.4
Clustering column	In a wide-column store, the column that sorts the rows inside one partition.	§16.4
Clustering index	An index on a non-key field by which the data file is physically sorted.	§12.4
COALESCE	Returns the first non-NULL argument.	§3.5
CockroachDB	A distributed SQL database that speaks the PostgreSQL protocol and replicates data ranges with Raft.	§16.7
Cohort	A group of users who started in the same period, followed over time to measure retention.	§19.8
Collation	Rules that decide how text is compared and sorted, including case sensitivity.	§2.7
Co-location	Keeping related rows (one customer's orders and items) on the same shard so joins stay on one node.	§15.3
Column family	A group of columns stored together in wide-column stores such as HBase and Cassandra.	§16.4
Column store (columnar)	Storage that keeps each column's values together, making scans and compression of a few columns efficient.	§12.2
Compaction	Background merging of sorted files in an LSM-tree store, which throws away overwritten and deleted values.	§16.4
Compare-and-set (CAS)	An atomic "change it only if it still has the value I read" operation.	§13.1
Composite attribute	An attribute made of parts, such as an address made of street, city and postcode.	§10.6
Composite index	An index on several columns. Usable for filters on a leftmost prefix of its columns.	§12.11
Composite key	A key made of two or more columns.	§10.13
Compression	Storing data in fewer bytes. Column stores compress well because one column's values are alike.	§12.2
Conceptual model	Business-level model of entities and relationships, without types or DBMS details.	§10.2
Concurrency control	The mechanisms (locking, timestamps, MVCC, validation) that keep concurrent transactions correct.	§13.9
Conditional aggregation	Aggregating only the rows that meet a condition, with `FILTER (WHERE …)` or `sum(CASE …)`.	§4.4
Connection pool	A set of reusable open database connections shared by application threads.	§14.12
Consensus	Agreement of a group of nodes on one value or log order despite failures, as in Raft and Paxos.	§15.8
Conservative 2PL	A two-phase-locking variant that takes every lock a transaction needs before it starts, so it cannot deadlock.	§13.11
Consistency (ACID)	Every committed transaction leaves all constraints and business rules satisfied.	§13.3
Consistency (CAP)	In CAP: every read sees the most recent write, as if there were a single copy. Not the same as the C in ACID.	§15.7

Term	Meaning	See
Consistency model	The guarantee a distributed store gives about what reads can see: strong, causal, eventual and others.	§15.6
Consistent hashing	Placing nodes and keys on a hash ring so that adding a node moves only a small share of keys.	§15.3
Constraint	A rule enforced by the DBMS on the data: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, EXCLUDE.	§9.3
COPY	PostgreSQL's bulk command for loading or exporting many rows at once, far faster than row-by-row `INSERT`s.	§14.10
Correlated subquery	A subquery that refers to a column of the outer query and is logically evaluated per outer row.	§6.3
Cost (planner)	The planner's estimate of the work of a plan, in arbitrary units based on page reads and CPU.	§12.10
Covering index	An index containing every column a query needs, allowing an index-only scan.	§12.12
CPU	The processor that executes instructions. Sorting, hashing and expression evaluation cost CPU time.	§12.1
CRDT	Conflict-free replicated data type: a structure (counter, set) whose replicas can always be merged without conflicts.	§15.5
CROSS JOIN	Join that returns the Cartesian product.	§5.1
Crosstab	A function from PostgreSQL's `tablefunc` extension that pivots rows into columns.	§14.7
Crow's foot notation	ER notation that draws cardinality and optionality with symbols at the ends of relationship lines.	§10.5
CRUD	Create, read, update, delete: the four basic operations on stored data.	§14.13
CSV	Comma-separated values: a plain-text table format with one row per line.	§1.1
CTE (common table expression)	A named subquery defined with `WITH` and used by the statement that follows.	§7.1
CUBE	Grouping extension that produces subtotals for every combination of the listed columns.	§4.6
Cursor	A pointer for fetching a query result a batch of rows at a time.	§14.10
Cypher	Neo4j's graph query language, which writes patterns like `(a)-[:FOLLOWS]->(b)`.	§16.5

D

Term	Meaning	See
Database	An organised, persistent, self-describing collection of related data.	§1.2
Data independence	Changing how data is stored (physical) or parts of the schema (logical) without changing applications.	§1.3
Data integrity	The data is accurate and obeys its rules (types, keys, constraints).	§1.3
Data model	A description of the data's entities, attributes, relationships and rules.	§10.1
Data-modifying CTE	A `WITH` step containing `INSERT`, `UPDATE` or `DELETE … RETURNING` (PostgreSQL).	§7.2
Data type	The kind of value a column holds (integer, text, date …), which decides what operations are allowed.	§3.1
Data warehouse	A database built for analysis, loaded from operational systems and tuned for large scans and aggregates.	§11.16
DBMS	Software that defines, stores, queries, protects and shares a database.	§1.3
DCL	Data control language: `GRANT` and `REVOKE`.	§1.9
DDL	Data definition language: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`.	§1.9
Deadlock	Two or more transactions each waiting for a lock held by another, so none can proceed.	§13.11
Dead tuple	An old row version that no transaction can see any more, waiting to be removed by VACUUM.	§13.14
Declarative	Saying what result you want and letting the system decide how to compute it. SQL is declarative.	§1.4
Decomposition	Splitting a relation into smaller ones during normalization.	§11.10
Deduplication	Keeping one row per group of duplicates, often with `row_number() = 1` or `DISTINCT ON`.	§8.8
Deferrable constraint	A constraint whose check can wait until `COMMIT` (`DEFERRABLE INITIALLY DEFERRED`).	§9.3
Degree (ER)	The number of entity types taking part in a relationship: unary, binary, ternary.	§10.7
Degree (relation)	The number of attributes (columns) of a relation.	§10.12
DELETE	DML statement that removes the rows matching its `WHERE` clause.	§9.10
Delta encoding	Storing each value as the difference from the previous one, giving small numbers for sorted ids or timestamps.	§12.2
Denormalization	Deliberately storing redundant data to speed up reads, with a mechanism to keep it correct.	§11.16
Dense index	An index with one entry for every record or key value.	§12.4
DENSE_RANK	Ranking function that gives ties the same rank and leaves no gaps (1, 2, 2, 3).	§8.5
Dependency preservation	Every functional dependency can be checked within a single table after decomposition.	§11.10
Derived attribute	An attribute that can be computed from others, such as age from birth date.	§10.6
Derived table	A subquery in the `FROM` clause.	§6.6
Determinant	The left side of a functional dependency.	§11.1

Term	Meaning	See
Dialect	One vendor's version of SQL, with its own syntax and functions.	§1.9
Dictionary encoding	Replacing repeated values with small integer codes plus one lookup table of the distinct values.	§12.2
Differential / incremental backup	A differential backup copies everything changed since the last full backup. An incremental copies changes since the last backup of any kind.	§9.12
Dirty page	A page changed in memory but not yet written to disk.	§15.9
Dirty read	Reading another transaction's uncommitted change.	§13.4
Dirty write	Overwriting another transaction's uncommitted change.	§13.4
Disaster recovery	The plans and off-site copies that let you restore service after losing a whole server, site or data set.	§15.11
DISTINCT	Removes duplicate rows from a result. `DISTINCT ON (cols)` keeps the first row per group (PostgreSQL).	§2.6
Distributed SQL	Another name for NewSQL: SQL and ACID transactions over data sharded and replicated across many nodes.	§16.7
Distributed transaction	A transaction that changes data on more than one node, so it needs a protocol such as two-phase commit.	§15.9
DML	Data manipulation language: `INSERT`, `UPDATE`, `DELETE`, `MERGE`.	§1.9
Document database	NoSQL store that keeps each record as a nested document, usually JSON.	§16.3
Dollar quoting	PostgreSQL's `$$ … $$` string syntax, used for function bodies so inner quotes need no escaping.	§14.1
Domain	The set of allowed values for an attribute.	§10.12
Double-entry bookkeeping	Recording each money movement as a debit and an equal credit, so all entries always sum to zero.	§17.3
Downsampling	Replacing old detailed measurements with coarser summaries, such as hourly averages.	§16.8
DQL	Data query language: `SELECT`.	§1.9
Driver	The library a program uses to talk to a particular database, such as psycopg or a JDBC driver.	§14.10
DROP	DDL statement that removes a database object entirely.	§9.6
Dual writes	An application writing the same change separately to two systems, which drift apart if one write fails.	§16.11
Durability	Once committed, a transaction's changes survive crashes.	§13.5
Dynamic SQL	SQL text built at run time and executed with `EXECUTE`. Identifiers must be quoted safely with `format('%I', …)`.	§19.6
DynamoDB	Amazon's managed key-value and document database.	§16.2

E

Term	Meaning	See
Elasticsearch / OpenSearch	Distributed search engines built on Lucene that index JSON documents for full-text search and log analytics.	§16.10
ELK stack	Elasticsearch, Logstash and Kibana: a common set of tools to collect, search and chart logs.	§16.11
Embedded database	A database library that runs inside the application process, with no separate server. SQLite is the usual example.	§1.7
Entity	A distinguishable thing the database stores facts about, such as a customer or an order.	§10.6
Entity integrity	No primary-key value may be NULL.	§10.14
Entity set / entity type	The entity type is the kind of thing (Employee). The entity set is all its instances stored right now.	§10.6
Enum type	A type whose values must come from a fixed list, such as `'new'`, `'paid'`, `'shipped'`.	§3.1
Equi-join	A join whose condition is equality between columns.	§5.3
Erasure coding	Storing data plus computed parity pieces so lost pieces can be rebuilt, using less space than full copies.	§15.11
ER diagram	A diagram of entities, attributes and relationships.	§10.5
ETL	Extract, transform, load: copying data from source systems, reshaping it, and loading it into a warehouse.	§11.16
Eventual consistency	If writes stop, all replicas eventually return the same value.	§15.6
Eviction policy	The rule a cache uses to decide what to drop when it is full, such as LRU or LFU.	§16.2
Exactly-once	A processing guarantee that each message affects the result exactly one time, even when it is retried.	§16.9
EXCEPT	Set operation returning rows of the first query that do not appear in the second.	§6.10
EXCLUDE constraint	PostgreSQL constraint that forbids two rows whose values conflict under given operators, such as overlapping ranges.	§17.2
EXCLUDED (ON CONFLICT)	In `INSERT … ON CONFLICT DO UPDATE`, the pseudo-row holding the values that were proposed for insertion.	§9.11
Exclusive lock (X)	A lock that only one transaction can hold. It blocks every other lock on the same item.	§13.10
Executor	The DBMS component that runs the chosen plan and produces the result rows.	§1.4
Existence dependency	An entity that cannot exist without another one, such as an order line without its order.	§10.9
EXISTS	True if the subquery returns at least one row.	§6.4
EXPLAIN / EXPLAIN ANALYZE	Show the chosen query plan, and with `ANALYZE` run it and report actual rows and times.	§12.10
Expression index	An index on the result of an expression, such as `lower(email)`.	§12.12
Extended statistics	PostgreSQL statistics on combinations of columns (`CREATE STATISTICS`), which help estimates for correlated columns.	§12.16

Term	Meaning	See
Extendible hashing	A dynamic hashing scheme that grows by doubling a directory of pointers and splitting only the bucket that overflowed.	§12.7
Extension (PostgreSQL)	An add-on package installed with `CREATE EXTENSION`, such as PostGIS or pg_trgm.	§15.2

F

Term	Meaning	See
Failover	Promoting a replica to primary when the primary fails.	§15.10
False positive	A "yes" that is actually wrong. A Bloom filter can give false positives but never false negatives.	§12.8
Fan-out	Row multiplication when a row joins to several matching rows, often inflating sums.	§5.9
Fan-out on write / on read	Two feed designs: push each new post into every follower's feed when it is written, or build the feed when it is read.	§17.4
Fault tolerance	Continuing to work correctly when some parts fail.	§15.1
Fencing	Preventing a deposed leader from continuing to write after failover.	§15.8
FETCH FIRST	Standard syntax for limiting rows: `FETCH FIRST n ROWS ONLY` or `WITH TIES`.	§2.8
File organization	How records are arranged in a file: heap, sorted, hashed or clustered.	§12.3
Fillfactor	How full PostgreSQL packs table or index pages, leaving the rest free for later updates.	§12.3
FILTER clause	Restricts the rows fed to one aggregate: `count(*) FILTER (WHERE …)`.	§4.4
Floating-point number	An approximate binary number (`real`, `double precision`) that cannot hold most decimal fractions exactly. Avoid it for money.	§3.1
Foreign data wrapper	A PostgreSQL extension that lets you query another database or a file as if it were a local table.	§15.4
Foreign key	Columns whose values must match a key in a referenced table (or be NULL).	§9.3
Frame (window frame)	The subset of the partition that an aggregate or value window function sees for the current row.	§8.4
fsync	The operating-system call that forces written data out of caches onto durable storage.	§15.9
FULL OUTER JOIN	Returns all rows from both sides, padding with NULLs where there is no match.	§5.6
Full-text search	Searching documents by words with stemming and ranking. `tsvector @@ tsquery` in PostgreSQL.	§14.6
Functional dependency	`X → Y`: any two rows with equal X values have equal Y values.	§11.1
Fuzzy checkpoint	A checkpoint that writes dirty pages out gradually while transactions keep running, instead of pausing all work.	§15.9

G

Term	Meaning	See
Gaps and islands	Problems about runs of consecutive values or dates, solved by grouping on `value - row_number()`.	§8.9
Generated column	A column computed from other columns of the same row (`GENERATED ALWAYS AS (expr) STORED`).	§9.4
generate_series	PostgreSQL function that returns a series of numbers or timestamps as rows.	§7.6
Geohash	A short string that encodes a map cell from latitude and longitude. Nearby places share a prefix.	§16.8
GIN index	Generalized inverted index: maps each element (word, key, array item) to the rows that contain it.	§12.9
GiST index	Generalized search tree for overlap and nearest-neighbour queries on ranges, geometry and more.	§12.9
Gossip protocol	Nodes spread cluster state by repeatedly swapping what they know with random peers.	§16.11
GQL	The ISO-standard graph query language (2024), close to Cypher.	§16.5
Graph database	NoSQL store of nodes and edges optimised for traversing relationships.	§16.5
Gremlin	A step-by-step graph traversal language used by Apache TinkerPop databases such as Amazon Neptune.	§16.5
GROUP BY	Groups rows that share values so aggregates are computed per group.	§4.2
GROUPING()	A function that returns 1 on a subtotal row where the column was rolled up, and 0 otherwise.	§4.6
GROUPING SETS	Computes several groupings in one query, such as per region, per product and a grand total.	§4.6
GROUPS frame	A window frame measured in peer groups rather than rows or values.	§8.4

H

Term	Meaning	See
Half-open interval	A range that includes its start but not its end (`>= start AND < end`), so neighbouring ranges never overlap.	§2.4
HashAggregate / GroupAggregate	Plan nodes that group rows, either with an in-memory hash table or by reading input already sorted on the group key.	§12.15
Hash function	A function that turns a key into a fixed-size number. The same key always gives the same number.	§12.3
Hash index	Index that hashes keys into buckets. Supports equality lookups only.	§12.7
Hash join	Join that builds a hash table on one input and probes it with the other. Equality only.	§12.15
Hash partitioning	Assigning rows to partitions by a hash of the key, spreading them evenly.	§15.2
Hash table	A structure that stores items in buckets chosen by a hash of the key, giving lookups in about constant time.	§12.7
HAVING	Filters groups after aggregation.	§4.3
HBase	A wide-column store on top of Hadoop's HDFS, modelled on Google's Bigtable.	§16.1
HDFS	Hadoop Distributed File System: stores huge files as large blocks copied to several machines.	§16.4
Heap	Unordered table storage: new rows go wherever there is space.	§12.1
Heartbeat	A periodic "I am alive" message. Missing heartbeats suggest that a node has failed.	§15.8
High availability	Keeping a service running through failures, usually with replicas and automatic failover.	§15.10
Hinted handoff	A node temporarily keeps writes meant for an unreachable replica and hands them over when it comes back.	§15.5
Histogram (statistics)	Buckets of a column's value distribution that the planner uses to estimate how many rows a condition keeps.	§12.16
Horizontal scaling	Adding more machines (scale out) rather than a bigger one.	§15.1
Hot partition (hotspot)	A partition or shard that receives far more traffic than the others and becomes the bottleneck.	§15.2
HTAP	Hybrid transactional/analytical processing: one system that serves OLTP and analytics together.	§16.6
Hybrid logical clock	A timestamp made of physical time plus a logical counter, used to order events across machines.	§16.7

I

Term	Meaning	See
IaaS / PaaS / SaaS	Cloud service models: rent machines (infrastructure), a managed platform, or finished software.	§1.5
Idempotent	Having the same effect whether it runs once or several times, which makes retries safe.	§17.3
Identifier	The name of a table, column or other object. Double quotes make it case-sensitive (`"Name"`).	§2.1
Identifying relationship	The relationship that connects a weak entity to its owner and supplies part of its key.	§10.9
Identity column	A column whose values are generated automatically: `GENERATED ALWAYS AS IDENTITY`.	§9.4
Impedance mismatch	The friction between objects in code (nested, linked by references) and flat relational tables.	§14.13
IN	True if a value equals any element of a list or subquery result.	§6.2
Index	An auxiliary structure that finds rows by key without scanning the whole table.	§12.4
Index-only scan	Answering a query from the index alone, without visiting table pages.	§12.10
Index scan	Walking an index to find matching entries, then fetching each row from the table.	§12.10
InfluxDB	A time-series database for metrics and sensor data.	§16.8
information_schema	The SQL-standard set of views that describe the database's tables, columns and constraints.	§1.2
Inlining (CTE)	The planner folding a CTE into the main query so it can be optimised together with it (PostgreSQL 12+).	§7.1
In-memory database	A database that keeps its working data in RAM and persists it with snapshots and logs.	§16.6
INNER JOIN	Returns only pairs of rows that satisfy the join condition.	§5.3
InnoDB	MySQL's default storage engine: transactional, with each table stored clustered on its primary key.	§1.4
INSERT	DML statement that adds rows.	§9.8
Integer division	Dividing two integers drops the fraction: in PostgreSQL `7 / 2` is `3`.	§2.3
Intention lock (IS, IX)	A table-level lock announcing that the transaction will lock rows inside the table, so table and row locks can be checked quickly.	§13.10
INTERSECT	Set operation returning rows present in both queries.	§6.9
Interval	A PostgreSQL type for a length of time, such as `interval '2 hours'`.	§3.4
Inverted index	An index that maps each word or element to the list of rows or documents that contain it.	§12.9
I/O	Input/output: reading from or writing to disk or the network, usually the slowest part of running a query.	§12.1
IoT	Internet of Things: networked sensors and devices that send streams of measurements.	§16.4
ISAM	Indexed sequential access method: an older file organization of sorted records with a static multilevel index.	§12.3
Isolation	Concurrent transactions do not observe each other's intermediate states.	§13.4

Term	Meaning	See
Isolation level	How strongly a transaction is isolated: Read Uncommitted, Read Committed, Repeatable Read or Serializable.	§13.4

J

Term	Meaning	See
JDBC	Java Database Connectivity: Java's standard API for running SQL through a database driver.	§14.11
Join	Combining rows of two tables according to a condition.	§5.2
Join dependency	A relation equals the join of some of its projections.	§11.13
JPA / Hibernate	Java's standard ORM interface and its most common implementation.	§12.13
JSON	JavaScript Object Notation: a text format of nested objects, arrays, strings, numbers, booleans and null.	§14.4
JSONB	PostgreSQL's binary JSON type, which supports indexing and containment queries.	§14.4
Junction table	A table that implements a many-to-many relationship with foreign keys to both sides.	§10.15

K

Term	Meaning	See
Kafka	Apache Kafka: a distributed, replicated log used to stream events between systems.	§16.9
Keyset pagination	Paging by remembering the last key seen (`WHERE key > :last`) instead of using `OFFSET`.	§12.14
Key-value store	NoSQL store that maps keys to values with get and put operations.	§16.2

L

Term	Meaning	See
LAG / LEAD	Window functions returning a value from a row before or after the current one.	§8.6
Last-write-wins (LWW)	A conflict rule that keeps the value with the latest timestamp and silently drops the others.	§15.5
Latency	The time a single operation takes, from request to response.	§1.5
LATERAL	A subquery in `FROM` that may refer to columns of tables listed before it.	§6.7
Lazy / eager loading	ORM strategies: fetch related objects only when they are first used (lazy, which risks N+1) or up front with a join (eager).	§12.13
Leader election	Choosing one node to coordinate writes, typically via a consensus system.	§15.8
Leader / follower	In replication, the leader (primary) accepts writes and the followers (replicas, standbys) copy its changes.	§15.5
Leaderless replication	Any replica accepts writes. Clients write to and read from several replicas and rely on quorums.	§15.5
Lease	A lock or leadership right that expires automatically after a set time unless renewed.	§15.8
Ledger	An append-only table of money or stock movements. Balances are sums over it.	§17.3
Leftmost prefix rule	A composite index on (a, b, c) helps filters on a, (a, b) or (a, b, c), not on b alone.	§12.11
LEFT OUTER JOIN	Returns every left row, with matching right rows or NULLs.	§5.4
Lexeme	A normalised word (after stemming) stored in a `tsvector`.	§14.6
LIKE / ILIKE	Pattern matching with `%` and `_`. `ILIKE` is case-insensitive (PostgreSQL).	§2.4
LIMIT / OFFSET	Return at most n rows, after skipping k rows.	§2.8
Linear hashing	A dynamic hashing scheme that splits buckets one at a time in a fixed order as the file grows.	§12.7
Linearizability	The strongest consistency model: each operation appears to take effect at one instant, in real-time order. Also called strong consistency.	§15.6
List partitioning	Assigning rows to partitions by a list of discrete values, such as region codes.	§15.2
Lock	A marker that restricts other transactions' access to a row or table until released.	§13.10
Lock manager	The DBMS component that grants, queues and releases locks.	§1.4
Logical model	Model with attributes, keys and normalized relations, independent of a particular DBMS.	§10.2
Logical query processing order	The conceptual order in which the clauses of a SELECT are evaluated, starting with FROM.	§4.5
Logical replication	Replicating row changes (insert, update, delete) for chosen tables, rather than copying raw WAL bytes.	§15.5
Lossless join	A decomposition whose pieces join back to exactly the original rows.	§11.10
Lost update	Two read-modify-write transactions where the second silently overwrites the first.	§13.7
LRU / LFU	Cache eviction policies: drop the least recently used, or the least frequently used, item first.	§16.2

Term	Meaning	See
LSM tree	Log-structured merge tree: writes go to memory and sorted files that are merged in the background.	§16.4
LSN (log sequence number)	A position in the write-ahead log, used to order changes and to track recovery and replication progress.	§13.5
Lucene	The Java search library inside Elasticsearch and OpenSearch.	§16.11

M

Term	Meaning	See
MapReduce	A batch-processing model: map each record to key-value pairs, group them by key, then reduce each group to a result.	§16.9
MariaDB	An open-source fork of MySQL.	§1.7
Materialized path	Storing each tree node's full ancestor path, such as `/1/4/9/`, in a column.	§7.4
Materialized view	A stored query result that must be refreshed to reflect new data.	§7.8
Memcached	A simple distributed in-memory key-value cache.	§16.6
MERGE	Standard statement that inserts, updates or deletes target rows based on a source (PostgreSQL 15+).	§9.11
Merge join	Join that walks two inputs sorted on the join key.	§12.15
Merkle tree	A tree of hashes in which each parent hashes its children, so two replicas can find differences by comparing a few hashes.	§15.5
Metadata	Data about data, such as table and column definitions.	§1.2
Minimal cover	An equivalent set of FDs with single-attribute right sides, no extraneous attributes and no redundant FDs.	§11.5
Mirroring	Keeping an exact, continuously updated copy of a disk or database on another device.	§15.11
MongoDB	A widely used document database that stores BSON documents in collections.	§16.3
Monotonic reads	Once you have seen a value, later reads never show you an older one.	§15.6
Moving average	The average of the last n values, recomputed for every row.	§8.7
Multi-leader replication	Several nodes accept writes and replicate to each other, so conflicting writes must be resolved.	§15.5
Multivalued attribute	An attribute that can hold several values for one entity, such as phone numbers.	§10.6
Multivalued dependency	`X ↠ Y`: for each X, the set of Y values is independent of the other attributes.	§11.12
MVCC	Multiversion concurrency control: keep several row versions so readers see a snapshot and do not block writers.	§13.14
MySQL	A widely used open-source relational database.	§1.7

N

Term	Meaning	See
N+1 query problem	Running one query for a list, then one more per item, instead of a single set-based query.	§12.13
Natural join	A join on all columns with the same name. Risky because it changes silently when columns are added.	§5.10
Natural key	A key taken from real-world data, such as an ISBN or email.	§10.13
Neo4j	A widely used graph database, queried with Cypher.	§16.10
Nested loop join	For each row of the outer input, look up matches in the inner input.	§12.15
Network partition	A network failure that splits the nodes into groups that cannot reach each other.	§15.1
NewSQL	Distributed databases that keep SQL and ACID while scaling out, such as Spanner and CockroachDB.	§16.7
Non-equi join	A join whose condition is not plain equality, such as `BETWEEN` or `<`.	§5.1
Non-repeatable read	Reading the same row twice in a transaction and getting different values.	§13.4
Normal form	A level of design quality (1NF, 2NF, 3NF, BCNF …) defined by which dependencies a table may contain.	§11.6
Normalization	Organising tables so each fact is stored once, guided by functional dependencies and normal forms.	§11.6
NoSQL	Non-relational databases: key-value, document, wide-column and graph stores.	§16.1
NOT NULL	Constraint forbidding NULL in a column.	§9.3
NOWAIT	A locking option that raises an error at once instead of waiting for a locked row.	§13.12
NTILE	Window function that splits ordered rows into n roughly equal buckets.	§8.5
NULL	A marker for a missing or unknown value. Comparisons with it yield UNKNOWN.	§2.5
NULLIF	Returns NULL if its two arguments are equal, otherwise the first.	§3.5
NULL padding	The NULLs an outer join fills into the columns of the side that had no match.	§5.4
numeric (decimal)	An exact decimal type with a chosen precision and scale, such as `numeric(12, 2)`. The right choice for money.	§3.1

O

Term	Meaning	See
OLAP	Online analytical processing: large scans and aggregates over historical data.	§11.16
OLTP	Online transaction processing: many small reads and writes of current data.	§11.16
ON CONFLICT	PostgreSQL clause that turns an `INSERT` into an upsert or skips conflicting rows.	§9.11
ON DELETE action	What happens to child rows when a parent is deleted: CASCADE, SET NULL, SET DEFAULT, RESTRICT, NO ACTION.	§9.3
Operator precedence	The order in which operators bind: `NOT` before `AND` before `OR`, so add parentheses when you mix them.	§2.2
Optimistic concurrency control	Proceed without locks, validate at commit, and abort or retry on conflict.	§13.9
Optimizer (planner)	The DBMS component that chooses an efficient execution plan for a query.	§1.4
Oracle Database	A commercial relational database. Its procedural language is PL/SQL.	§1.7
ORDER BY	Sorts the result. The only way to guarantee row order.	§2.7
Ordered-set aggregate	An aggregate that needs sorted input, given with `WITHIN GROUP (ORDER BY …)`, such as `percentile_cont`.	§4.1
ORM	Object-relational mapper: a library that maps classes to tables and generates SQL.	§14.13
Orphan row	A child row whose foreign key points to a parent that does not exist.	§10.14
Outbox pattern	Writing an event row to an outbox table in the same transaction as the data change, then publishing it from there. Avoids dual writes.	§14.3
OUTER JOIN	LEFT, RIGHT or FULL join, which keeps unmatched rows padded with NULLs.	§5.2
Overflow page	An extra page chained to a full bucket or page.	§12.7

P

Term	Meaning	See
PACELC	Extension of CAP: under Partition choose Availability or Consistency, Else choose Latency or Consistency.	§15.7
Page	Fixed-size unit of storage and I/O (8 KB in PostgreSQL).	§12.1
Pagination	Returning a result one page at a time, with `LIMIT`/`OFFSET` or keyset paging.	§12.14
Parameterized query	A query whose values are sent separately from the SQL text as bind parameters, preventing injection.	§14.9
Parser	The DBMS component that checks a statement's syntax and turns it into an internal tree for the planner.	§1.4
Partial dependency	A non-prime attribute determined by a proper subset of a composite key.	§11.8
Partial index	An index that covers only rows matching a `WHERE` condition.	§12.12
Partial key (discriminator)	The attribute that identifies a weak entity among the other entities of the same owner.	§10.9
Participation	Whether every entity instance must take part in a relationship (total) or may not (partial).	§10.7
PARTITION BY (window)	Splits rows into independent groups for a window function without collapsing them.	§8.2
Partitioning	Splitting one table into smaller physical pieces by range, list or hash.	§15.2
Partition key	The column whose value decides which partition, shard or node stores a row.	§15.2
Partition pruning	Skipping partitions that cannot contain matching rows.	§15.2
Partition tolerance	In CAP: the system keeps working even when messages between nodes are lost.	§15.7
Patroni	A tool that runs PostgreSQL high availability: it watches the primary and performs automatic failover using a consensus store.	§15.10
Paxos	A classic consensus algorithm in which proposers get a majority of acceptors to agree on a single value.	§15.8
PBFT	Practical Byzantine Fault Tolerance: consensus that survives f lying nodes using 3f + 1 nodes.	§15.8
Peer rows	Rows with equal values of the window `ORDER BY` key.	§8.3
Percentile / median	The value below which a given share of the data falls. The median is the 50th percentile.	§4.1
Pessimistic concurrency control	Lock data before using it so conflicting transactions wait.	§13.9
PgBouncer	A lightweight connection pooler placed between applications and PostgreSQL.	§14.12
pg_dump / pg_restore	PostgreSQL's tools to export a logical backup and to load it back.	§9.12
pg_trgm (trigrams)	A PostgreSQL extension that splits text into three-character pieces so indexes can speed up `LIKE '%x%'` and similarity search.	§12.13
Phantom read	Re-running a query in a transaction returns a different set of rows because of inserts or deletes.	§13.4
Physical model	Model with exact types, indexes, partitions and storage for a specific DBMS.	§10.2
PITR (point-in-time recovery)	Restoring a base backup and replaying archived WAL up to a chosen moment.	§9.12

Term	Meaning	See
Pivot	Turning row values into columns, usually with conditional aggregation.	§14.7
PL/pgSQL	PostgreSQL's procedural language for functions, procedures and triggers.	§14.1
Polyglot persistence	Using several kinds of database in one system, each for what it does best.	§16.10
Polymorphic association	One column that may point to rows of several different tables (a type + id pair). No foreign key can enforce it.	§17.10
PostGIS	A PostgreSQL extension that adds geometry and geography types, spatial functions and spatial indexes.	§16.8
PostgreSQL	The open-source object-relational database used throughout this book (version 14).	§1.7
Posting list	In an inverted index, the list of rows or documents that contain one term.	§16.11
Predicate lock	A lock on a condition rather than on existing rows, used to prevent phantoms.	§13.10
Predicate push-down	Applying filters as early as possible (inside a scan, a subquery or a remote shard) so fewer rows flow on.	§15.4
Prepared statement	A statement parsed once and executed many times with different parameters.	§14.9
Primary index	An index on the field by which a sorted file is ordered, usually with one entry per block (sparse).	§12.4
Primary key	The chosen candidate key: unique and not NULL, one per table.	§9.3
Prime attribute	An attribute appearing in some candidate key.	§10.13
Principle of least privilege	Give each user or program only the rights it needs, and nothing more.	§14.8
Privilege	A permission such as SELECT or INSERT on an object, granted to a role.	§14.8
Procedure (stored)	Named server-side routine called with `CALL`. Can control transactions.	§14.2
Prometheus	A monitoring system with a built-in time-series database for metrics.	§16.8
Pseudotransitivity	A derived FD rule: if X → Y and WY → Z, then WX → Z.	§11.3
psql	PostgreSQL's command-line client.	§1.10

Q

Term	Meaning	See
Quadtree	A spatial index that recursively divides an area into four squares.	§16.8
Query builder	A library that builds SQL from code calls, without the full object mapping of an ORM.	§14.13
Query plan	The tree of operations (scans, joins, sorts) the executor runs for a query.	§12.10
Quorum	The number of replicas that must acknowledge an operation. `R + W > N` makes reads see the latest write.	§15.5

R

Term	Meaning	See
Raft	A consensus algorithm with an elected leader that replicates a log to a majority.	§15.8
RAID	Combining several disks for safety or speed: RAID 1 mirrors, RAID 5 and 6 add parity, RAID 10 mirrors and stripes.	§15.11
RAM	The computer's main memory: much faster than disk, but its contents are lost when the power goes.	§12.1
RANGE frame	A window frame defined by value distance from the current row's `ORDER BY` value. Includes peers.	§8.4
Range partitioning	Assigning rows to partitions by ranges of a key, usually dates.	§15.2
Range type	A PostgreSQL type holding an interval of values, such as `daterange` or `tstzrange`, with operators like `&&` (overlaps).	§17.1
RANK	Ranking function that gives ties the same rank and skips the next ranks (1, 2, 2, 4).	§8.5
RDBMS	Relational database management system: a DBMS based on the relational model, such as PostgreSQL.	§1.7
RDB snapshot	Redis's point-in-time dump of its whole data set to a file.	§16.11
Read Committed	Isolation level where each statement sees only data committed before it began. PostgreSQL's default.	§13.4
Read replica	A copy of the database that serves read queries.	§15.5
Read skew	Seeing different parts of the data at different moments within one transaction.	§13.4
Read-your-writes	A guarantee that a user always sees their own earlier writes.	§15.6
Rebalancing	Moving data between nodes when nodes are added or removed, to keep the load even.	§15.3
Recovery	Restoring a consistent state after a crash, using the log to redo committed and undo uncommitted work.	§15.9
Recursive CTE	A CTE that refers to itself, used for hierarchies, graphs and series.	§7.3
Redis	An in-memory data-structure store (strings, hashes, lists, sets, sorted sets, streams) used as a cache, queue or database.	§16.2
Redo / undo	Redo re-applies committed changes from the log. Undo reverses changes made by transactions that did not commit.	§15.9
Redundancy	The same fact stored more than once.	§11.6
Referential integrity	Every foreign-key value matches an existing referenced key (or is NULL).	§10.14
Reflexivity (FD rule)	Armstrong's rule: if Y is a subset of X, then X → Y.	§11.3
Regular expression	A pattern language for text, such as `^[a-z]+@`. PostgreSQL matches with `~` and `~*`.	§2.4
Relation	The formal name for a table: a set of tuples over a set of attributes.	§10.12
Relational algebra	The formal operations on relations (selection, projection, join, union, difference) that SQL is built on.	§10.11
Relational division	Finding rows related to every member of another set, such as students who took all courses.	§19.7

Term	Meaning	See
Relational model	Data as relations of atomic values, linked by values and queried declaratively (Codd, 1970).	§1.6
Relation schema / relation instance	The schema is a relation's name and attributes. The instance is the set of tuples it holds at a moment.	§10.12
Repeatable Read	Isolation level where rows read once keep their values. In PostgreSQL, snapshot isolation.	§13.4
Replication	Keeping copies of the same data on several servers.	§15.5
Replication lag	The delay before a change made on the primary appears on a replica.	§15.5
Result set	The rows a query returns.	§2.1
Retention policy	A rule for how long data is kept before it is deleted or summarised.	§16.8
RETURNING	Clause that returns the rows affected by `INSERT`, `UPDATE` or `DELETE`.	§9.8
RIGHT OUTER JOIN	Returns every right row, with matching left rows or NULLs.	§5.5
Rigorous 2PL	Two-phase locking that holds every lock, read and write, until the transaction ends.	§13.10
Role	A PostgreSQL user or group that owns objects and holds privileges.	§14.8
ROLLUP	Grouping extension that adds hierarchical subtotals and a grand total.	§4.6
Round-robin	Handing items to nodes in turn: 1, 2, 3, 1, 2, 3 …	§15.2
Row-level security	Policies that filter which rows a role may read or write.	§14.8
ROW_NUMBER	Window function giving each row a unique sequential number in its partition.	§8.5
Row-oriented storage	Storage that keeps all the columns of a row together. Good for OLTP reads and writes of whole rows.	§12.1
ROWS frame	A window frame counted in physical rows.	§8.4
Row value (row constructor)	Several values written as one, `(a, b)`, compared left to right: `(ts, id) < (:ts, :id)`.	§6.6
RPO / RTO	Recovery point objective (data you may lose) and recovery time objective (downtime you may have).	§9.12
R-tree	A spatial index that groups nearby objects into nested bounding rectangles.	§16.8
Run-length encoding (RLE)	Compressing runs of a repeated value as (value, count) pairs.	§12.2
Running total	A cumulative sum from the first row up to the current row.	§8.3

S

Term	Meaning	See
Saga	A long business process split into local transactions, each with a compensating step that undoes it if a later step fails.	§15.9
Sargable	A predicate the database can answer with an index seek, such as `col >= value` with no function on `col`.	§12.13
Savepoint	A marker inside a transaction you can roll back to without abandoning the whole transaction.	§13.2
Scalar function	A function that returns one value per input row, such as `upper` or `round`.	§4.1
Scalar subquery	A subquery that returns one value. Zero rows give NULL, more than one row is an error.	§6.1
Scatter-gather	Sending a query to every shard and merging their partial results.	§15.4
Schema	The structure of a database (tables, columns, constraints), or a PostgreSQL namespace that groups objects.	§9.2
Schema-on-read / on-write	Structure checked when data is read (typical of NoSQL) or when it is written (relational).	§1.8
Secondary index	An index on columns other than the file's ordering key, with one entry per record (dense).	§12.4
Selectivity	The fraction of rows a predicate keeps. Low fractions favour index scans.	§12.16
SELF JOIN	Joining a table to itself under two aliases.	§5.7
Semi-join	Returns rows of one table that have at least one match in another, each once.	§5.8
Semi-structured data	Data that carries its own flexible structure, such as JSON or XML.	§14.4
Semi-synchronous replication	The primary waits until at least one replica has received the change before confirming the commit.	§15.5
Sequence	A database object that hands out increasing numbers.	§9.4
Sequential scan	Reading every page of a table.	§12.10
serial / bigserial	PostgreSQL's older auto-increment shorthand: an integer column with a sequence as its default.	§9.4
Serializable	The strongest isolation level: the outcome equals some serial order of the transactions.	§13.8
Serialization failure	Error 40001: the database aborted a transaction to preserve isolation. Retry it.	§13.7
Session consistency	Guarantees such as read-your-writes and monotonic reads that hold within one client session.	§15.6
Sessionization	Grouping a user's events into sessions separated by gaps of inactivity.	§8.9
Set operation	UNION, INTERSECT or EXCEPT, which combine the results of two queries.	§6.8
Set-returning function	A function that returns many rows, such as `generate_series` or `unnest`, used in `FROM`.	§7.6
Sharding	Placing partitions of the data on different servers.	§15.3
Shard key	The column that decides which shard stores a row.	§15.3
Shared lock (S)	A lock several transactions can hold at once for reading. It blocks exclusive locks.	§13.10

Term	Meaning	See
Single-leader replication	One node takes all the writes and streams them to its followers.	§15.5
SKIP LOCKED	A locking option that skips rows other transactions have locked, so many workers can take different jobs from one queue.	§13.12
Slotted page	A page layout with an array of line pointers in the header pointing to variable-length rows stored from the end of the page.	§12.1
Snapshot	The set of committed data a transaction or statement is allowed to see.	§13.4
Snapshot isolation	Each transaction reads from a consistent snapshot taken at its start. Allows write skew.	§13.4
Soft delete	Marking a row as deleted (`deleted_at`) instead of removing it.	§17.14
Spanner	Google's globally distributed SQL database, which uses TrueTime clocks to order transactions.	§16.7
Spark	Apache Spark: distributed in-memory processing of large data sets, with SQL and DataFrame APIs.	§16.9
Sparse index	An index with entries for only some records, typically one per block.	§12.4
Split brain	Two nodes both believe they are the leader and accept writes, so the data diverges.	§15.8
SQL	Structured Query Language: the declarative standard language of relational databases.	§1.9
SQLAlchemy	Python's widely used SQL toolkit and ORM.	§14.12
SQL injection	An attack that smuggles SQL into a query through unescaped input.	§14.9
SQLite	An embedded relational database stored in a single file and run inside the application.	§1.7
SQL Server	Microsoft's relational database. Its dialect and procedural language is T-SQL.	§1.7
SQLSTATE	The standard five-character error code, such as `23505` (unique violation) or `40001` (serialization failure).	§13.6
SSI	Serializable snapshot isolation: snapshot isolation plus tracking of read-write conflicts, aborting when a cycle is possible.	§13.8
SSTable	Sorted string table: an immutable file of sorted key-value pairs, written by LSM-tree stores.	§12.3
Star schema	A warehouse design: one central fact table of events joined to surrounding dimension tables (date, product, store).	§11.16
Starvation	A transaction waits forever because others keep getting the resource first.	§13.13
Statistics	Per-column summaries (distinct counts, common values, histograms) used by the planner.	§12.16
Stemming	Reducing words to a common root (running → run) so a search matches every form.	§14.6
Stop word	A very common word, such as "the" or "and", that full-text search ignores.	§14.6
Storage engine	The DBMS layer that stores pages and rows on disk.	§1.4
Streaming replication	PostgreSQL's physical replication: replicas receive WAL records continuously and replay them.	§15.5
Strict 2PL	Two-phase locking that holds write locks until commit or abort, so no one reads uncommitted data.	§13.10

Term	Meaning	See
Subquery	A query nested inside another statement.	§6.1
Superkey	Any set of attributes that uniquely identifies rows.	§10.13
Superuser	A role that bypasses all permission checks.	§14.8
Surrogate key	An artificial key with no business meaning, such as an identity number or UUID.	§10.13
Synchronous replication	The primary waits for a replica to confirm before reporting commit.	§15.5

T

Term	Meaning	See
Table	The SQL form of a relation: rows and named, typed columns.	§1.6
Tablespace	A directory on disk where PostgreSQL stores chosen tables and indexes.	§9.12
TCL	Transaction control language: `BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`.	§1.9
Temporary table	A table that exists only for the session (or transaction) that created it.	§7.1
Thomas write rule	In timestamp ordering, a write that is already outdated is skipped instead of aborting the transaction.	§13.13
Three-valued logic	SQL logic with TRUE, FALSE and UNKNOWN.	§2.5
Throughput	How much work a system completes per unit of time, such as transactions per second.	§16.2
TiDB	A MySQL-compatible distributed SQL database built on the TiKV key-value store.	§16.7
TimescaleDB	A PostgreSQL extension for time-series data that partitions tables by time automatically.	§16.8
Time-series database	A store optimised for timestamped measurements, retention and downsampling.	§16.8
Timestamp ordering	Concurrency control that orders conflicting operations by transaction timestamps.	§13.13
timestamptz vs timestamp	`timestamptz` stores an exact instant, shown in the session's time zone. `timestamp` stores a wall-clock reading with no zone.	§3.1
TOAST	PostgreSQL's mechanism for compressing or moving large values out of the main table page.	§12.1
Tombstone	A marker saying a key was deleted, so the deletion reaches every replica and survives compaction.	§16.4
Top-N per group	Returning the N best rows inside each group, usually by ranking in a CTE and filtering outside.	§8.8
Transaction	A unit of work of one or more statements that succeeds or fails as a whole.	§13.1
Transaction manager	The DBMS component that starts, commits and aborts transactions and coordinates locks and logging.	§1.4
Transitive dependency	A → B and B → C where B is not a key, so C depends on the key only through B.	§11.9
Transitivity (FD rule)	Armstrong's rule: if X → Y and Y → Z, then X → Z.	§11.3
Trigger	A function run automatically before or after changes to a table.	§14.3
Trivial FD	An FD whose right side is contained in its left side.	§11.2
TrueTime	Google's clock service that returns the current time as an interval with bounded uncertainty, used by Spanner.	§16.7
TRUNCATE	Quickly removes all rows from a table.	§9.6
T-SQL	Transact-SQL: SQL Server's dialect and procedural language.	§1.7
tsvector / tsquery	PostgreSQL full-text types: a document as a list of lexemes, and a search condition matched against it with `@@`.	§14.6
TTL (time to live)	An expiry time after which a key or row is deleted automatically.	§16.2
Tuple	The formal name for a row.	§10.12

Term	Meaning	See
Type vs instance	Keeping the catalogue description (room type, book title) separate from the physical item (room, copy).	§17.14

U

Term	Meaning	See
Undo log	A log of old values used to roll back the changes of transactions that do not commit.	§13.14
UNION / UNION ALL	Combine two results, removing duplicates (`UNION`) or keeping them (`UNION ALL`).	§6.8
UNIQUE constraint	Forbids duplicate non-NULL values in a column or column set.	§9.3
Unique index	An index that also rejects duplicate keys. It is how UNIQUE and PRIMARY KEY are enforced.	§9.3
unnest	A function that expands an array into a set of rows.	§14.5
Unpivot	Turning columns into rows.	§14.7
Updatable view	A simple view that accepts `INSERT`, `UPDATE` and `DELETE`, passing them to the underlying table.	§7.7
UPDATE	DML statement that changes column values in matching rows.	§9.9
Update anomaly	A change to a fact must be repeated in many rows, and missing one leaves the data inconsistent.	§11.6
Upsert	Insert a row, or update it if it already exists.	§9.11
UTC	Coordinated Universal Time: the zone-free reference clock that servers usually store times in.	§3.4
UTF-8	The common text encoding that stores each character in 1 to 4 bytes.	§1.10
UUID	128-bit identifier that can be generated anywhere without coordination.	§9.4
UUIDv7	A UUID version that starts with a timestamp, so new ids are roughly in order and kinder to B-tree indexes.	§9.4

V

Term	Meaning	See
VACUUM	PostgreSQL maintenance that reclaims dead row versions and updates the visibility map.	§13.14
Vector clock	A list of per-node counters attached to a value, used to tell whether two versions are ordered or concurrent.	§15.5
Version column	A counter increased on every update and checked in the `WHERE` clause, which implements optimistic locking.	§13.7
Vertical scaling	Moving to a bigger machine (scale up).	§15.1
View	A named, stored query that behaves like a table and holds no data of its own.	§7.7
Virtual node	One of many small positions a physical node owns on a hash ring, so load spreads evenly.	§15.3
Visibility map	A PostgreSQL bitmap marking pages whose rows are visible to every transaction, which makes index-only scans possible.	§12.10
Volatility (function)	How stable a function's result is: `IMMUTABLE` (never changes), `STABLE` (fixed within a statement) or `VOLATILE` (may change on every call).	§14.1

W

Term	Meaning	See
Wait-die / wound-wait	Deadlock-prevention schemes that use transaction age to decide whether a transaction waits for a lock or is aborted.	§13.11
WAL (write-ahead log)	A log of changes written and flushed before the data pages, used for durability, recovery and replication.	§13.5
Weak entity	An entity identified only together with its owner entity.	§10.9
Weighted average	An average in which each value counts in proportion to a weight, such as units sold.	§18.8
WHERE	Filters rows before grouping. Keeps only rows where the condition is TRUE.	§2.2
Wide-column store	NoSQL store with partition keys and sorted clustering columns, such as Cassandra.	§16.4
Wildcard	A pattern character: in `LIKE`, `%` matches any run of characters and `_` exactly one.	§2.4
Window function	A function computed over a set of rows related to the current row, without collapsing them.	§8.1
WITH TIES	Extends `FETCH FIRST n` to include rows tied with the last one.	§2.8
work_mem	PostgreSQL setting for the memory one sort or hash may use before it spills to disk.	§12.15
Write amplification	One logical write causing several physical writes (log, page, index, compaction).	§12.4
Write skew	Two transactions read overlapping data and write different rows, together breaking a rule each checked.	§13.8

X

Term	Meaning	See
xmin / xmax	Hidden PostgreSQL row columns that hold the ids of the transactions that created and deleted that row version.	§13.14

Y

Term	Meaning	See
YugabyteDB	A distributed SQL database with a PostgreSQL-compatible interface.	§16.7

Z

Term	Meaning	See
ZooKeeper	Apache ZooKeeper: a replicated coordination service (using the ZAB protocol) for configuration, locks and leader election.	§15.8

Chapter 20 TL;DR

- **Evaluation order** explains most errors: FROM → WHERE → GROUP BY → HAVING → SELECT (windows) → DISTINCT → set operations → ORDER BY → LIMIT (20.1).
- **NULL** is unknown: `WHERE` keeps only TRUE, `NOT IN` with a NULL returns nothing, `count(col)` and other aggregates skip NULLs, and `sum` over no rows is NULL (20.2, 20.4).

- **PostgreSQL arithmetic**: `7 / 2 = 3`, so write `100.0 *` before dividing and guard with `NULLIF(x, 0)` (20.3).
- **Joins**: right-table filters of a `LEFT JOIN` go in `ON`. `NOT EXISTS` is the safe anti-join. Watch fan-out (20.5, 20.6).
- **Windows**: `row_number` / `rank` / `dense_rank` differ only on ties. The default frame with `ORDER BY` is `RANGE … CURRENT ROW`, so give `last_value` a full frame (20.8).
- **Design**: model entities, keys and relationships first, normalize to 3NF/BCNF, denormalize only with a reason and a way to keep copies right (20.10).
- **Indexes**: B-tree by default, equality columns before range columns, no functions on indexed columns, keyset instead of deep `OFFSET`, and compare estimated with actual rows in `EXPLAIN ANALYZE` (20.11).
- **Transactions**: PostgreSQL defaults to Read Committed. Repeatable Read is snapshot isolation (write skew still possible). Serializable (SSI) prevents everything but needs retries on 40001 (20.12).
- **Dialects** differ most on paging, concatenation, NULL sorting, upsert, `RETURNING`, integer division and transactional DDL (20.16).
- About 100 PostgreSQL claims and 28 SQLite claims on these sheets are checked by runnable scripts (20.17). The pattern catalogue (20.18) and the fifty Q&A (20.19) link back to the full explanations.

This is the end of the SQL Handbook. For more on the design side of data systems, see the DDIA notes, and for how databases fit into whole systems, the case studies in Chapter 17.